

DATA STRUCTURES · ALGORITHMS · SYSTEM DESIGN

क्रम

KRAMA

The Ordered Path

180 Days from First Principles to the Product-Company Interview

NISHANT GUPTA - GRANTH SKILLS

VOLUME VI OF EIGHT

Graphs

DAYS 125 – 142

DAY 001

THE ORDERED PATH

DAY 180



TWO TRACKS

18 OF 180 DAYS IN THIS VOLUME

180 STEPS

K R A M A

THE ORDERED PATH

क्रम

K R A M A

The Ordered Path

180 Days from First Principles to the Product-Company Interview

Nishant Gupta - Granth Skills

VOLUME VI · GRAPHS

DAYS 125 TO 142

KRAMA — THE ORDERED PATH

180 Days from First Principles to the Product-Company Interview

Volume VI of eight: **Graphs**. Days 125 to 142.

Copyright © 2026 Nishant Gupta - Granth Skills. All rights reserved.

ABOUT THE TEXT

This edition is generated from the Krama course sources. The lessons, stories, code, diagrams and interview material are original work written for this course.

Practice problems are named, with their source and one line on what they test. No problem statement is reproduced. Trade marks and problem titles belong to their respective owners and are used here only to identify where a problem can be found.

ABOUT THE MAKING OF IT

Typeset from Markdown by rendering to a browser engine and printing to PDF. Every tool in that chain is free software and every typeface is published under the SIL Open Font Licence. Appendix F lists all of them with their licences.

Text face: Source Serif 4 — SIL Open Font Licence 1.1

Display face: Fraunces — SIL Open Font Licence 1.1

Sans face: Inter — SIL Open Font Licence 1.1

Monospace face: JetBrains Mono — SIL Open Font Licence 1.1

Devanagari: Noto Serif Devanagari — SIL Open Font Licence 1.1

Diagrams: Mermaid — MIT Licence

Code colouring: Pygments — BSD 2-Clause

EDITION

First edition, 2026. Set in Source Serif 4 on a 6.75 by 9.5 inch page.

क्रम

krama, Sanskrit: order; sequence; the step that follows the step before it.

YOU CANNOT SKIP A STEP. THAT IS THE WHOLE METHOD.

Why this book teaches two things at once

This book teaches two subjects at the same time, on purpose.

Most preparation separates them. You spend three months on data structures and algorithms, then start again from nothing on system design, and by the time the second subject makes sense the first has gone quiet. Interviews do not work that way. A single loop at a product company will ask you to reverse a linked list in one room and design a rate limiter in the next, on the same day, from the same head.

So every chapter here teaches one topic from each track, side by side. Day 42 gives you binary search and the object-oriented design of a parking lot. Day 130 gives you graph colouring and message queues. You will not finish this book with one strong subject and one weak one.

Who this is written for

Someone who has never studied this before. Not a computer science graduate. Not someone brushing up after a few years in the job.

If you need to be told what a server is, you are the reader. The first chapter explains what happens when a program runs. The fourteenth explains what happens when you type an address into a browser. Nothing before that is assumed.

The other half of that promise matters just as much: you are someone who will sit in a real interview. So the writing never stops at understanding. Every chapter ends by putting the idea in an interviewer's mouth, giving you the first ninety seconds of an answer, and writing out a full model response you can compare yours against.

The one question every chapter answers

Every lesson in this book was written to satisfy a single test:

Does this leave the reader able to answer one real interview question out loud, from memory, to a stranger?

That test decided what went in and, more usefully, what stayed out. There are no formal proofs here. No potential functions, no decision-tree lower bounds, no interpreter internals for their own sake. If a thing will not be asked, and does not make an asked thing clearer, it is not in this book.

What is different about how it is written

Every lesson opens with a story about a person. Not an analogy dressed up as a story: an actual scene, with a name in it, told in ordinary words with no technical vocabulary at all. Someone sorting socks. Someone in a canteen queue. Someone laying tables for a wedding. The story comes before the definition because a beginner needs somewhere to put the idea before they have the word for it.

Every solution is shown in full. This book does not hide answers. Code is built up in fragments of a few lines, each one explained, and then given again complete and ready to run. You are never told to work it out yourself as a test of character.

The arithmetic is always done out loud. Not “this is slow” but “this line runs about twenty-five million times”. Not “that is a lot of data” but `50M x 20 x 2KB = 2TB`. Numbers are worked through where you can see them.

Errors are printed exactly as they appear. When something breaks, the real message is on the page - `IndexError: list index out of range` - not a description of it. You will recognise it when your own screen shows it.

How this edition was made

The text is generated from the course repository, so the book and the working material cannot drift apart. The typesetting was done by rendering the pages in a browser engine and printing them to PDF.

Every part of the toolchain is free software, and every typeface is published under the SIL Open Font Licence. Appendix F lists all of it, with licences. No proprietary tool, font, or copyrighted third-party text was used to produce this book.

That extends to the practice problems. Problems are named, with their source and one line on what they are really testing. Their statements are never reproduced. You are meant to read them where they live, and solve them there.

How to use this book

The book is one hundred and eighty chapters long. One chapter is one day's work. That is the only pacing instruction in it, and it is deliberate: a day here is a unit of subject, not a unit of hours. Some days will take you longer than others. That is information about the topic, not about you.

The shape of every chapter

Each chapter holds three pieces, in this order.

1. **The DSA lesson.** Nine sections, always the same nine, always in the same order. Appendix A sets out what each one is for.
2. **The system design lesson.** The same nine sections, on a different subject.
3. **The practice sheet.** Named problems to solve, and questions to answer out loud.

The repetition is the point. After a week you stop navigating and start reading. You know that section 5 has the complete code, that section 7 has the mistakes you are about to make, and that section 8 is the interview itself.

How to read a lesson

Read the story first, and do not skip it. It is section 2, it has no technical words in it, and it is the part that makes the idea stick. People who skip it can define the term and cannot use it.

Type the code. Do not read it. Section 5 builds the solution in fragments and then gives it complete. Type the fragments as they come. Run the finished program. Change a number and run it again.

Say section 9 out loud. Every lesson ends with a recall card - the two or three sentences that are the whole lesson. Say them from memory, aloud, in full sentences. Not in your head. Aloud. An answer you have only ever thought is an answer you have never tested.

Treat section 8 as a rehearsal, not a reading. It gives you real interviewer phrasings, a script for the opening ninety seconds, the three follow-ups that usually come next, and a written-out model answer. Deliver yours before you read theirs.

How to use the practice sheet

Solve the problems where they live. The sheet names each one, tells you where to find it, and says in one line what it is really testing - which is usually not what it appears to be about.

Solve it, then say what you did out loud, as if someone had asked. If you cannot narrate your own solution, you have not finished the problem.

When you get stuck

Go back one chapter, not forward. This course is built so that each day rests on the one before it. A chapter that will not open is almost always a chapter whose foundation is thin, and the foundation is behind you.

If you want the vocabulary, Appendix D is a glossary of every term the book defines, with the day it first appears. If you want to see the whole route, Appendix C prints all one hundred and eighty days on two facing pages.

Revision

Certain chapters are revision days, and they are marked as such in the contents. Do not read past them. They exist because recall fades on a schedule, and the schedule is not negotiable.

Appendix B collects the recall cards for every part in this volume. Cover the answer, say it, then check. That is the whole drill.

Where this volume sits

This volume is marked in colour below.

- **Volume I — Foundations**
Days 1 to 26. How Code Costs, and How the Internet Works
- **Volume II — Scanning and Searching**
Days 27 to 50. Pointers, Windows and Binary Search · Databases from Zero
- **Volume III — Order and Access**
Days 51 to 77. Sorting, Hashing, Stacks and Queues · Objects, SOLID and Patterns
- **Volume IV — Links and Recursion**
Days 78 to 97. Linked Lists, Recursion and Backtracking · Low-Level Design
- **Volume V — Trees and Priorities**
Days 98 to 124. Trees, Heaps and Tries · Scaling Fundamentals
- **Volume VI — Graphs**
Days 125 to 142. Graph Algorithms · Distributed Systems Core
- **Volume VII — Optimal Choices**
Days 143 to 170. Dynamic Programming and Greedy · High-Level Design Case Studies
- **Volume VIII — The Last Mile**
Days 171 to 180. Bits, Maths and Mock Interviews · Reliability and Security

The nineteen parts

PART I	Foundations: how code costs	Days 1–8
PART II	Arrays	Days 9–18
PART III	Strings	Days 19–26
PART IV	Two pointers and sliding window	Days 27–36
PART V	Prefix sums	Days 37–41
PART VI	Binary search	Days 42–50
PART VII	Sorting	Days 51–59
PART VIII	Hashing: maps and sets	Days 60–67
PART IX	Stacks and queues	Days 68–77
PART X	Linked lists	Days 78–86

PART XI	Recursion and backtracking	Days 87–97
PART XII	Trees and binary search trees	Days 98–112
PART XIII	Heaps and priority queues	Days 113–119
PART XIV	Tries	Days 120–124
Part XV	Graphs	Days 125–142
PART XVI	Dynamic programming	Days 143–163
PART XVII	Greedy and intervals	Days 164–170
PART XVIII	Bits and maths	Days 171–176
PART XIX	Final mocks and revision	Days 177–180

What is in this volume

PART XV · GRAPHS	1
125 What a graph is, and where graphs hide	2
DSA What a graph is, and where graphs hide	3
DESIGN Retries, backoff, and thundering herds	21
PRACTICPractice — What a graph is, and where graphs hide	38
126 Representing a graph: adjacency matrix versus adjacency list	44
DSA Representing a graph: adjacency matrix versus adjacency list	45
DESIGN Circuit breakers and bulkheads	64
PRACTICPractice — Representing a graph: adjacency matrix versus adjacency list	82
127 Breadth-first search on a graph	88
DSA Breadth-first search on a graph	89
DESIGN Distributed locks	107
PRACTICPractice — Breadth-first search on a graph	126
128 Depth-first search on a graph	132
DSA Depth-first search on a graph	133
DESIGN Distributed systems revision and interview questions	153
PRACTICPractice — Depth-first search on a graph	171
129 Connected components	177
DSA Connected components	178
DESIGN Message queues: why async changes everything	198
PRACTICPractice — Connected components	217
130 Grids are graphs: islands and flood fill	223
DSA Grids are graphs: islands and flood fill	224
DESIGN Kafka, explained	243
PRACTICPractice — Grids are graphs: islands and flood fill	261
131 Shortest path in an unweighted graph	267
DSA Shortest path in an unweighted graph	268
DESIGN Publish-subscribe versus point-to-point	287
PRACTICPractice — Shortest path in an unweighted graph	304
132 Cycle detection in an undirected graph	310
DSA Cycle detection in an undirected graph	311
DESIGN Stream processing basics	332
PRACTICPractice — Cycle detection in an undirected graph	350
133 Cycle detection in a directed graph	356
DSA Cycle detection in a directed graph	357
DESIGN Object storage, S3-style	378

	PRACTICPractice — Cycle detection in a directed graph	396
134	Topological sort	402
	DSA Topological sort	403
	DESIGN Blob storage versus storing files in the database	423
	PRACTICPractice — Topological sort	442
135	Course schedule and the dependency family	448
	DSA Course schedule and the dependency family	449
	DESIGN Search: how a search index actually works	471
	PRACTICPractice — Course schedule and the dependency family	489
136	Dijkstra's algorithm	496
	DSA Dijkstra's algorithm	497
	DESIGN Elasticsearch in a design	518
	PRACTICPractice — Dijkstra's algorithm	536
137	Bellman-Ford, and what negative edges break	542
	DSA Bellman-Ford, and what negative edges break	543
	DESIGN Time-series and metrics stores	565
	PRACTICPractice — Bellman-Ford, and what negative edges break	583
138	Union-Find: the disjoint set union	590
	DSA Union-Find: the disjoint set union	591
	DESIGN Data warehouses: OLAP versus OLTP	612
	PRACTICPractice — Union-Find: the disjoint set union	631
139	Minimum spanning trees: Kruskal and Prim	637
	DSA Minimum spanning trees: Kruskal and Prim	638
	DESIGN ETL, batch pipelines, and where data goes to be counted	659
	PRACTICPractice — Minimum spanning trees: Kruskal and Prim	677
140	Bipartite graphs and two-colouring	683
	DSA Bipartite graphs and two-colouring	684
	DESIGN Websockets, long polling, and server-sent events	705
	PRACTICPractice — Bipartite graphs and two-colouring	724
141	Multi-source BFS and 0-1 BFS	730
	DSA Multi-source BFS and 0-1 BFS	731
	DESIGN Push notifications, end to end	753
	PRACTICPractice — Multi-source BFS and 0-1 BFS	771
142	Graphs revision and mock round	777
	DSA Graphs revision and mock round	778
	DESIGN Geospatial indexing: geohash and quadrees	799
	PRACTICPractice — Graphs revision and mock round	818
BACK MATTER		
A	Appendix A - The nine sections, and what each is for	824
B	Appendix B - Recall cards	826

C	Appendix C - The complete 180-day map	844
D	Appendix D - Glossary	845
E	Appendix E - References and further reading, by section	864
F	Appendix F - Colophon: how this book was made	866
G	Appendix G - Problem index	867
H	Index	874

List of figures

125.1 Data structures and algorithms - What a graph is, and where graphs hide	2
125.2 System design - Retries, backoff, and thundering herds	2
125.3 System design - Retries, backoff, and thundering herds	2
126.1 System design - Circuit breakers and bulkheads	44
126.2 System design - Circuit breakers and bulkheads	44
127.1 Data structures and algorithms - Breadth-first search on a graph	88
127.2 System design - Distributed locks	88
128.1 Data structures and algorithms - Depth-first search on a graph	132
128.2 Data structures and algorithms - Depth-first search on a graph	132
128.3 System design - Distributed systems revision and interview questions	132
129.1 Data structures and algorithms - Connected components	177
129.2 System design - Message queues: why async changes everything	177
130.1 System design - Kafka, explained	223
131.1 System design - Publish-subscribe versus point-to-point	267
132.1 Data structures and algorithms - Cycle detection in an undirected graph	310
133.1 Data structures and algorithms - Cycle detection in a directed graph	356
133.2 Data structures and algorithms - Cycle detection in a directed graph	356
133.3 System design - Object storage, S3-style	356
133.4 System design - Object storage, S3-style	356
134.1 Data structures and algorithms - Topological sort	402
134.2 System design - Blob storage versus storing files in the database	402
134.3 System design - Blob storage versus storing files in the database	402
135.1 Data structures and algorithms - Course schedule and the dependency family	448
136.1 System design - Elasticsearch in a design	496
137.1 Data structures and algorithms - Bellman-Ford, and what negative edges break	542
138.1 System design - Data warehouses: OLAP versus OLTP	590
139.1 System design - ETL, batch pipelines, and where data goes to be counted	637
140.1 Data structures and algorithms - Bipartite graphs and two-colouring	683
140.2 System design - Websockets, long polling, and server-sent events	683
140.3 System design - Websockets, long polling, and server-sent events	683
141.1 System design - Push notifications, end to end	730
142.1 Data structures and algorithms - Graphs revision and mock round	777

PART XV

Graphs

Days 125 to 142 · 18 chapters

Alongside, on the system design track:

Distributed systems core, Building blocks of big systems

- | | |
|--|---|
| 125 What a graph is, and where graphs hide | 134 Topological sort |
| 126 Representing a graph: adjacency matrix versus adjacency list | 135 Course schedule and the dependency family |
| 127 Breadth-first search on a graph | 136 Dijkstra's algorithm |
| 128 Depth-first search on a graph | 137 Bellman-Ford, and what negative edges break |
| 129 Connected components | 138 Union-Find: the disjoint set union |
| 130 Grids are graphs: islands and flood fill | 139 Minimum spanning trees: Kruskal and Prim |
| 131 Shortest path in an unweighted graph | 140 Bipartite graphs and two-colouring |
| 132 Cycle detection in an undirected graph | 141 Multi-source BFS and 0-1 BFS |
| 133 Cycle detection in a directed graph | 142 Graphs revision and mock round |

What a graph is, and where graphs hide

DATA STRUCTURES AND ALGORITHMS

What a graph is, and where graphs hide

SYSTEM DESIGN

Retries, backoff, and thundering herds

What a graph is, and where graphs hide

1 What this is, and why they ask it

A graph is a set of things, and a set of connections between those things. That is the entire definition. Two words: **vertices** for the things, **edges** for the connections.

You have been using graphs for weeks without the word. A tree is a graph. A grid is a graph. A linked list is a graph. What changes today is that the connections stop being tidy — a vertex can have any number of neighbours, connections can point both ways or only one, and you can walk in a circle and end up where you started.

They ask about graphs constantly, and almost never by name. The interview question is not “traverse this graph”. It is “given a list of courses and their prerequisites”, or “find the shortest sequence of word changes”, or “how many islands are in this grid”. **The skill being tested is not traversal. It is noticing.** Roughly a quarter of medium and hard interview problems are graph problems wearing a costume, and the candidates who fail them mostly fail before writing any code, by not seeing what the vertices are.

Today is entirely about that. No shortest paths, no cycle detection — those are the next eighteen days. Today you learn the vocabulary precisely, and you learn to read a problem statement and say out loud: *the vertices are these, the edges are these, and the edge means this.*

By the end you can classify any graph in four words, spot a graph in a problem that never uses the word, and say what the two numbers **V** and **E** are for any problem you are handed.

2 The story

Suresh needs to get to the government hospital on Thursday morning, and there is no bus that goes there.

He has lived in the city for nineteen years and he has never once looked at a route map. What he has instead is a collection of facts, picked up one at a time, mostly from other people at bus stops.

He knows the 41 goes from the stop outside his lane to the big junction near the fly-over, because he takes it to work. He knows the 7 and the 12 both leave from that junction, because he has stood there and watched them. He knows the 12 goes towards the

railway station, because his brother-in-law takes it. And he knows — this one from a man at the tea stall about three years ago — that from the railway station there is a bus that stops right outside the hospital, though he cannot remember its number.

None of these facts is about the hospital and his house together. Each one is a small thing about two places and how to get between them.

On Wednesday evening he sits on the steps and puts them together. Lane to junction on the 41. Junction to station on the 12. Station to hospital on whatever that bus is. Three buses, and he has never made that journey in his life, but he is now fairly confident it exists.

Then he thinks about it a bit more, because three buses with two changes is a lot when you have to be somewhere by nine. He knows the 7 also leaves the junction, and he has a feeling it goes past the college. He does not know whether anything runs from the college towards the hospital. That is a gap in what he knows, not a gap in the city.

So on Thursday he does what he has always done. He gets on the 41, and at the junction he asks a conductor standing by the 7: “Does this go anywhere near the government hospital?”

The conductor says take the 7 to the college, then the 19 from the gate. Two buses.

Suresh has now added two facts to the collection. The next person who asks him will get a better answer than the one he gave himself last night.

3 The idea in plain English

Suresh’s collection of facts is a graph. Take it apart carefully, because every word in this section is one you will use for the next eighteen days.

The places are the vertices. A **vertex** is one of the things in your collection — a bus stop, a person, a city, a course, a web page, a cell in a grid. The word **node** means the same thing and both are used; this course will say vertex when talking about the structure and node when talking about code. If you have n things, you have n vertices, and the count is written V .

The bus connections are the edges. An **edge** joins two vertices and means “these two are directly connected”. “The 41 runs from the lane to the junction” is one edge. The count of edges is written E . Note that an edge is a fact about exactly two vertices — the hospital is not on any edge with Suresh’s lane, which is precisely why he needed to think.

A path is a sequence of edges you can follow. Lane → junction → station → hospital is a **path** of length three. The whole point of a graph is that a question about two vertices far apart gets answered by stitching together facts about vertices that are close together. **Almost every graph algorithm you will learn is a way of doing that stitching.**

Directed or undirected is the first question you ask. An **undirected** edge works both ways: if the 41 runs lane-to-junction, it also runs junction-to-lane. A **directed** edge works one way only, and you draw it with an arrow. One-way streets are directed. “A follows B” on a social network is directed — B does not necessarily follow A. “A is friends with B” is undirected. Getting this wrong is the single most common modelling mistake, and it usually produces an answer that is right on the sample input.

Weighted or unweighted is the second question. An **unweighted** edge just says “connected”. A **weighted** edge carries a number — a distance, a cost, a time, a fare. Suresh’s graph is unweighted while he is only asking “can I get there”, and becomes weighted the moment he starts asking “which way is cheapest”.

Cyclic or acyclic is the third. A **cycle** is a path that starts and ends at the same vertex without reusing an edge. Bus routes are full of cycles. Course prerequisites must not have any — if A requires B and B requires A, nobody can ever start. A directed graph with no cycles is called a **DAG**, a directed acyclic graph, and it turns up so often it gets its own name.

Connected or not is the fourth. A graph is **connected** if you can get from any vertex to any other. It is not always. There might be a whole neighbourhood on the other side of the river with its own buses and no route across, and a program that assumes everything is reachable from wherever it started will silently miss it. **Assume nothing is connected until the problem says so.**

The degree of a vertex is how many edges touch it. The junction has high degree — many buses. Suresh’s lane has degree one. In a directed graph you count separately: **in-degree** is arrows coming in, **out-degree** is arrows going out. In-degree is the whole idea behind topological sorting, which you meet on [day 134](#).

Dense or sparse decides how you store it. With V vertices, the most edges you can have is about V^2 (everything connected to everything). A graph near that is **dense**. A graph where E is closer to V is **sparse**. Real graphs are almost always sparse — a city has thousands of stops and each one connects to a handful of others, not to all of them. That single observation decides tomorrow’s lesson.

Now the part that actually matters: where graphs hide. The problem will not say “graph”. It will say one of these:

THE PROBLEM SAYS	THE VERTICES ARE	THE EDGES ARE
“courses and prerequisites”	courses	“must be taken before” (directed)
“cities and flights”	cities	flights (directed, weighted by price)
“a grid of land and water”	cells	“is adjacent to” (undirected)
“words differing by one letter”	words	“one letter apart” (undirected)
“people and friendships”	people	friendships (undirected)
“tasks and dependencies”	tasks	“depends on” (directed)
“states of a puzzle and legal moves”	states	moves (directed)
“packages and their requirements”	packages	“requires” (directed)

The recognition question, in one line: *are there things, and pairwise relationships between them, and is the question about following those relationships?* If yes, it is a graph, whatever the story says.

The two questions to answer out loud before writing a single line: **what is a vertex, and what does an edge mean?** Say them as full sentences. “A vertex is a course. An edge from A to B means A must be taken before B.” If you cannot finish those two sentences, you do not understand the problem yet, and code will not help.

4 The picture

Suresh’s collection of facts, drawn:

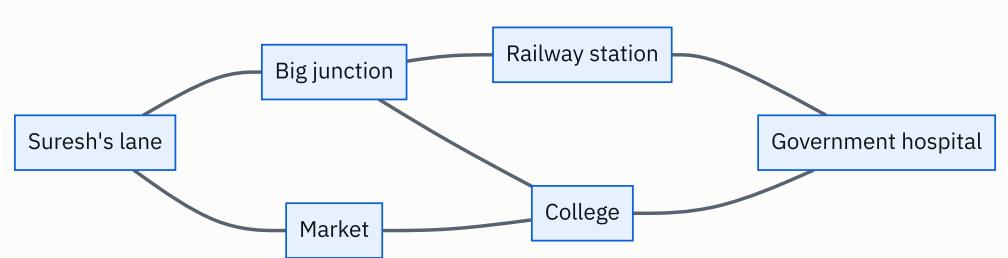


FIGURE 125.1

What to notice. Nothing in this picture is about “the lane and the hospital”. Every line is a fact about two places that touch. The answer to Suresh’s question is a *path* through the picture, and there are three of them: via the station, via the college, and the long way round through the market. Finding the shortest is [day 131](#); today it is enough to see that the question has become “is there a path”, and that the picture answers it.

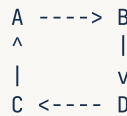
Undirected against directed, on the same four vertices:

UNDIRECTED — friendship



A can reach D two ways
and D can reach A

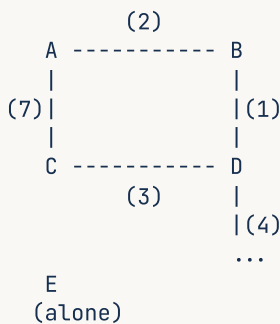
DIRECTED — "follows"



$A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$
but B cannot reach A
except by going all the way round

What to notice. The same four vertices and the same four connections, and a completely different set of questions have answers. In the undirected version every vertex reaches every other. In the directed version, B cannot get to A in one hop even though $A \rightarrow B$ exists. **Drawing the arrows is not decoration; it changes the answer.**

And the vocabulary, on one small graph:



V = 5 vertices: A B C D E
E = 5 edges

degree(A) = 2 degree(B) = 3
degree(E) = 0

path A-B-D-C-A is a CYCLE
E is UNREACHABLE from everything
so this graph is NOT CONNECTED

weights in brackets: this is WEIGHTED

What to notice. E sits there with no edges at all, and it is still a vertex. A program that starts at A and walks outwards will never see it. Every graph algorithm you write from now on needs an answer to “what about the parts I cannot reach from where I started”, and the answer is usually a loop over all vertices on the outside.

5 The code, built step by step

Today’s code is about **modelling** — turning a problem statement into vertices and edges. Tomorrow is about storage; the day after is about walking. Here you learn to build the thing.

The universal shape is a dictionary from a vertex to its list of neighbours. This is called an **adjacency list**, and it is what you will use in roughly every graph problem you ever write.

```

from collections import defaultdict

def build_undirected(edges: list[tuple[str, str]]) → dict[str, list[str]]:
    """Each edge goes in twice – once in each direction."""
    graph: dict[str, list[str]] = defaultdict(list)
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)          # ← this line is the whole difference
    return graph

```

Two appends for an undirected edge, one for a directed edge. That one line is the difference between “friendship” and “follows”, and forgetting it is the most common bug in this entire phase. `defaultdict(list)` means a vertex with no entry yet gives you an empty list instead of raising, which saves a check on every access.

Directed is the same function with the second append deleted:

```

def build_directed(edges: list[tuple[str, str]]) → dict[str, list[str]]:
    """A → B only. B does not gain a link back to A."""
    graph: dict[str, list[str]] = defaultdict(list)
    for a, b in edges:
        graph[a].append(b)
    return graph

```

Now the modelling. Here is a grid, which never mentions graphs at all:

```

def grid_neighbours(grid: list[list[str]], row: int, col: int) → list[tuple[int, int]]:
    """The vertices are cells; the edges are 'is next to and also land'."""
    rows, cols = len(grid), len(grid[0])
    out = []
    for delta_row, delta_col in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        r, c = row + delta_row, col + delta_col
        if 0 ≤ r < rows and 0 ≤ c < cols and grid[r][c] == "1":
            out.append((r, c))
    return out

```

There is no dictionary here and it is still a graph. The vertices are `(row, col)` pairs and the edges are computed on demand instead of stored. **This is the most important variation in the whole phase: an implicit graph**, where the neighbours are calculated rather than looked up. Grids, puzzle states and word ladders are all implicit, and if you wait to be handed an edge list you will not recognise them.

Course prerequisites, which is the interview classic:

```

def build_prerequisites(n: int, pairs: list[list[int]]) → dict[int, list[int]]:
    """pairs[i] = [course, needs_first]. Edge direction is the whole question."""
    graph: dict[int, list[int]] = {course: [] for course in range(n)}
    for course, needs_first in pairs:
        graph[needs_first].append(course)    # needs_first → course
    return graph

```

Read that append twice. The input says “to take `course`, first take `needs_first`”, and the edge points from `needs_first` to `course`, because the edge means “unlocks” — finish this one and the other becomes available. You could model it the other way and everything downstream reverses. **Neither is wrong; being unclear which one you chose is.** Say the direction out loud, in words, before you write the line.

Building the dictionary from `range(n)` instead of `defaultdict` matters here too: a course with no prerequisites and no dependents must still appear, or your later loop over `graph` will miss it entirely.

Now something to run. The simplest possible question — can I get from here to there — with a queue, which is tomorrow’s and the next day’s subject and is included today so you have a working program:

```
from collections import deque

def reachable(graph: dict[str, list[str]], start: str, goal: str) → bool:
    seen = {start}
    queue = deque([start])
    while queue:
        current = queue.popleft()
        if current == goal:
            return True
        for neighbour in graph[current]:
            if neighbour not in seen:
                seen.add(neighbour)           # mark on PUSH, not on pop
                queue.append(neighbour)
    return False
```

`seen` is what makes this terminate. Without it, a cycle — and Suresh’s bus network is full of them — means you go round forever. Marking a vertex as seen when you *push* it rather than when you pop it stops the same vertex being queued twice, which matters more than it looks and is the subject of [day 127](#).

The complete solution

A small program that builds Suresh’s graph, answers his question, and prints the facts about it that this lesson names.

```

"""Modelling a problem as a graph: vertices, edges, and the questions they answer."""

from __future__ import annotations

from collections import defaultdict, deque

def build_undirected(edges: list[tuple[str, str]]) → dict[str, list[str]]:
    graph: dict[str, list[str]] = defaultdict(list)
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)
    return graph

def reachable(graph: dict[str, list[str]], start: str, goal: str) → bool:
    """Is there any path from start to goal?"""
    if start not in graph or goal not in graph:
        return False
    seen = {start}
    queue = deque([start])
    while queue:
        current = queue.popleft()
        if current == goal:
            return True
        for neighbour in graph[current]:
            if neighbour not in seen:
                seen.add(neighbour)
                queue.append(neighbour)
    return False

def components(graph: dict[str, list[str]]) → list[list[str]]:
    """The separate pieces. A graph is not always one piece."""
    seen: set[str] = set()
    pieces: list[list[str]] = []
    for vertex in graph: # the OUTER loop over all vertices
        if vertex in seen:
            continue
        piece, queue = [], deque([vertex])
        seen.add(vertex)
        while queue:
            current = queue.popleft()
            piece.append(current)
            for neighbour in graph[current]:
                if neighbour not in seen:
                    seen.add(neighbour)
                    queue.append(neighbour)
        pieces.append(sorted(piece))
    return pieces

def describe(graph: dict[str, list[str]]) → None:
    vertices = len(graph)
    half_edges = sum(len(neighbours) for neighbours in graph.values())
    edges = half_edges // 2 # each undirected edge counted twice
    densest = vertices * (vertices - 1) // 2
    print(f"V = {vertices}, E = {edges}, max possible E = {densest}")
    print(f"density = {edges / densest:.2%} → {'dense' if edges > densest / 2 else 'sparse'}")
    for vertex in sorted(graph):
        print(f"    degree({vertex}) = {len(graph[vertex])}")

if __name__ == "__main__":
    bus_routes = [
        ("lane", "junction"),
        ("junction", "station"),
        ("junction", "college"),
        ("station", "hospital"),
        ("college", "hospital"),
        ("lane", "market"),
        ("market", "college"),
        ("depot", "workshop"), # a separate piece, across the river
    ]

```

```

]
city = build_undirected(bus_routes)

describe(city)
print("lane → hospital:", reachable(city, "lane", "hospital"))
print("lane → workshop:", reachable(city, "lane", "workshop"))
print("pieces:", components(city))

```

Running it:

```

V = 8, E = 8, max possible E = 28
density = 28.57% → sparse
degree(college) = 3
degree(depot) = 1
degree(hospital) = 2
degree(junction) = 3
degree(lane) = 2
degree(market) = 2
degree(station) = 2
degree(workshop) = 1
lane → hospital: True
lane → workshop: False
pieces: [['college', 'hospital', 'junction', 'lane', 'market', 'station'], ['depot', 'workshop']]

```

Look at the last two lines. `lane → workshop` is `False`, and the graph has **two** pieces. The depot exists, it has a bus, and no amount of walking from Suresh's lane will ever find it. Every function you write from here needs to have decided what it does about that.

6 What it costs

Graph costs are written in terms of two numbers, and you must state both.

V = the number of vertices
E = the number of edges

How big can E be? For an undirected graph with no repeated edges and no self-loops:

every pair of vertices, once $V \times (V - 1) / 2$
V = 1,000 $1,000 \times 999 / 2 = 499,500$

For a directed graph it is $V \times (V - 1)$, twice as many, because each pair can have an arrow each way.

How big is E in practice? Almost always far smaller. Three real examples:

city bus network	V = 5,000 stops	E ~ 15,000	E / V = 3
Facebook friendships	V = 3e9 people	E ~ 5e11	E / V ~ 170
web pages and links	V = 5e10 pages	E ~ 1.5e12	E / V ~ 30

In each case E is a small multiple of V, not V^2 . $E \approx 3 \times V$ versus $E \approx 500,000$ for a thousand-vertex graph is a factor of well over a hundred, and it is the reason the next lesson exists.

A graph is sparse when E is close to V and dense when E is close to V^2 . The rough dividing line people use is $E \approx V \log V$. Assume sparse unless told otherwise; you will be right nearly every time.

Building the adjacency list.

```
loop over E edges, two appends each    0(E)
plus initialising V empty lists        0(V)
→ 0(V + E) time and space
```

$0(V + E)$ is the phrase you will say more than any other in this phase. It means “look at every vertex once and every edge once”, and it is the cost of every basic traversal. Note that it is a *sum*, not a product — a graph with a million vertices and three million edges is four million units of work, not three trillion.

Count it out on the `reachable` function:

```
each vertex is pushed at most once      V pushes
each vertex is popped at most once      V pops
for each popped vertex, scan its neighbours  sum of all degrees = 2E
total                                   -----
                                         V + 2E steps → 0(V + E)
```

The $2E$ is because each undirected edge appears in two adjacency lists. That factor of two is real work and it disappears into the big-O, which is fine, but say “each edge is examined twice, once from each end” if asked to be precise.

Space.

```
adjacency list    V lists + 2E entries    → 0(V + E)
the seen set      at most V entries        → 0(V)
the queue         at most V entries        → 0(V)
```

Put numbers on it for a concrete case — a social graph of a million users averaging 200 friends:

```
V = 1,000,000
E = 1,000,000 x 200 / 2 = 100,000,000 edges
adjacency entries = 2E = 200,000,000
at 8 bytes per integer reference = 1.6 GB
plus 1,000,000 list objects at ~56 bytes = 56 MB
```

1.6 GB for the edges and 56 MB for the vertices. The edges are the memory. That ratio holds almost everywhere and it is why “can you fit the graph in memory” is a question about E , never about V .

7 The traps

The undirected edge added once

The near-miss, and by a wide margin the most common bug in the phase:

```
for a, b in edges:
    graph[a].append(b)
    # graph[b].append(a)    ← forgotten
```

The symptom is not a crash. It is an answer that is right on the sample and wrong on the real input:

```
>>> city = build_undirected([("lane","junction"), ("junction","hospital")])
>>> reachable(city, "lane", "hospital")
True
>>> reachable(city, "hospital", "lane")
False
```

Reachability that works one way and not the other, in a network of buses that obviously run both ways. **Before writing the loop, say out loud: “an edge here means A and B are connected in both directions” or “an edge here means A comes before B”.** Then write the one or two appends that sentence demands.

The vertex with no edges

```
graph = defaultdict(list)
for a, b in edges:
    ...
for vertex in graph:      # only vertices that appeared in some edge
    ...
```

A vertex mentioned in no edge never gets a key, so the loop skips it entirely. On “count the connected pieces among 6 people, with these 2 friendships”, the four people with no friends are four separate pieces and this code reports two:

```
>>> components(build_undirected([("a","b"), ("c","d")]))
[['a', 'b'], ['c', 'd']]      # where are e and f?
```

Fix it by initialising every vertex up front — `{v: [] for v in range(n)}` — whenever the problem gives you a vertex count. This is why the prerequisites builder above does exactly that.

Assuming the graph is connected

```
def visit_everything(graph, start):  
    # BFS from start  
    ...
```

Correct only if everything is reachable from `start`, which the problem must have promised. If it did not, you need the outer loop over all vertices that `components` has. The symptom is an undercount that nothing about the sample input reveals — sample graphs are almost always connected, and real ones are almost never.

The graph with a cycle and no `seen` set

```
def walk(graph, current):  
    for neighbour in graph[current]:  
        walk(graph, neighbour)           # no visited check
```

```
Traceback (most recent call last):  
  File "graph.py", line 12, in walk  
    walk(graph, neighbour)  
[Previous line repeated 996 more times]  
RecursionError: maximum recursion depth exceeded
```

A tree has no cycles, so tree traversals never needed this. A graph is not a tree. **Every graph traversal needs a `seen` set, without exception**, and forgetting it is the difference between the two structures made concrete.

Self-loops and repeated edges

An edge from a vertex to itself, or the same pair listed twice:

```
edges = [("a", "a"), ("a", "b"), ("a", "b")]
```

Neither is an error, and both change your counts. `degree("a")` becomes 5. Your `E` count is wrong. A cycle detector may report a cycle that is just a self-loop. The problem statement usually promises neither exists — **read that line, and if it is not there, ask**.

The off-by-one on vertex labels

Problems label vertices `0..n-1` or `1..n`, and they switch between the two without warning. Building a list of size `n` for `1..n` labels gives you:

```
Traceback (most recent call last):  
  File "graph.py", line 8, in build  
    graph[a].append(b)  
IndexError: list index out of range
```

The cheap defence is a dictionary rather than a list, which does not care what the labels are. The cheaper defence is to read the constraints line before writing the loop.

8 In the interview

How it gets asked

Nobody says “graph”. These do:

- “Given `n` courses and a list of prerequisite pairs, can all courses be finished?”
- “Given a grid of 1s and 0s, count the islands.”
- “Given a start word, an end word and a dictionary, find the shortest transformation sequence.”
- “Given flight routes and prices, find the cheapest way from A to B with at most one stop.”
- “You have a list of accounts with email addresses. Merge the ones belonging to the same person.”
- “Given a list of packages and what each requires, in what order do you install them?”

And directly, usually as an opener before the real question:

- “Is this a graph problem? What are the vertices and edges?”

The first ninety seconds

“Let me name the model before I write anything, because everything else follows from it.

A vertex is a course. An edge from A to B means A must be finished before B can be started — so it is **directed**, and I would say that explicitly, because if I get the direction wrong every later step inverts. It is **unweighted**, because the question is about order and not about cost. And the whole question is whether it contains a **cycle**, because a cycle means a set of courses that each wait for another and none can ever start.

Two things I would check before coding. First, is the graph guaranteed connected? Usually not — there can be courses with no prerequisites at all and no dependents, and any loop I write has to be over every vertex, not only the ones reachable from where I happened to start. Second, are the vertices labelled from zero or from one, and can a pair repeat?

For storage I would use an adjacency list — a dictionary from a course to the list of courses it unlocks. With n courses and m prerequisite pairs that is $O(n + m)$ memory, and every traversal over it is $O(n + m)$ time. I would not use a matrix here: the graph is sparse, most courses have one or two prerequisites, and a matrix would be n^2 for almost entirely empty cells.

Shall I write the cycle check, or would you like me to talk through the ordering version first?”

The follow-ups

“Why is this a graph and not just a list of pairs?”

“Because the question is about following the relationships, not about the relationships themselves. Any individual pair is easy — ‘does A come before B’ is a lookup. The question being asked is ‘is there a chain’, and a chain is a path. The moment the answer requires stitching together facts about different pairs, you are in a graph whether you call it one or not.

The practical test I use is: does the answer to the question involve anything I was not told directly? If the input says $A \rightarrow B$ and $B \rightarrow C$, and I need to know about A and C, that gap is exactly what a graph algorithm fills.”

“How do you decide the edge direction?”

“By writing the meaning as an English sentence first, and only then translating it. ‘To take course A, you must first take B’ becomes either ‘B unlocks A’ or ‘A depends on B’, and those give opposite adjacency lists.

I pick based on what I am going to traverse. If I want to walk forward from the courses I can start now to the ones they unlock, I store `unlocks`. If I want to walk backwards from a goal to what it needs, I store `depends on`. For topological sorting I want `unlocks` plus a count of incoming dependencies, so that is what I build.

The cost of getting it wrong is not a crash — it is an answer that is confidently backwards. So I say the sentence out loud, and on a sample of two courses I check by hand which list is populated.”

“The grid one — where is the graph in a grid?”

“The vertices are cells and the edges are ‘is orthogonally adjacent to’. It is undirected, unweighted, and the edges are not stored anywhere — they are computed from the coordinates. That is an **implicit** graph, and it is worth naming because it is the form people fail to recognise.

The consequence is that I never build an adjacency list. I write a neighbour function that takes `(row, col)` and yields the valid neighbours, and every algorithm from BFS onwards works unchanged against it. V is `rows × cols` and E is at most `2 × rows × cols`, since each cell has at most four neighbours and each edge is shared — so $O(V + E)$ is just $O(\text{rows} \times \text{cols})$.

The same shape covers puzzle states, where a vertex is a board position and an edge is a legal move, and word ladders, where a vertex is a word and an edge is ‘differs by one letter’. In all three, listing the edges up front would be enormous and computing them is trivial.”

“When is the graph too big to fit in memory?”

“That is a question about E , not V . On a million users with 200 friends each, V is a million and E is a hundred million, so the adjacency entries are two hundred million references — about 1.6 gigabytes — against 56 megabytes for the vertex objects. The edges are the memory, always.

Three moves when it does not fit. Do not materialise it: keep it implicit and compute neighbours from whatever store already holds the relationships. Partition it: shard by vertex and accept that a traversal now crosses machines, which changes the algorithm rather than just the storage. Or compress it: for a static graph, a compressed sparse row layout gets a hundred million edges into two flat arrays with almost no overhead, which is often a five-to-ten-times saving over Python lists.

I would also ask whether I need the whole graph. Most real questions touch a neighbourhood, not everything, and a bounded-depth traversal from a start vertex never has to see the rest.”

The model answer

“Here is a list of accounts. Each has a name and some email addresses. Two accounts belong to the same person if they share any email. Merge them.”

“Let me build the model out loud first, because this problem is entirely a modelling problem — the traversal at the end is four lines.

The instinct is to make accounts the vertices, with an edge between two accounts that share an email. That is correct and it is expensive: to find those edges I have to compare every account against every other, which is n^2 comparisons, and with a hundred thousand accounts that is five billion.

So I make the emails vertices too. A vertex is either an account or an email address. An edge joins an account to each of its emails — undirected, unweighted. Now two accounts that share an email are connected through that email vertex, in two hops, and I never compared any pair of accounts directly. Building it is one pass over the input: $O(\text{total emails})$.

That is the whole trick, and it is worth stating as a general one: when a relationship between two things is ‘they share a property’, make the property a vertex. It turns a quadratic edge-building step into a linear one. The same move works for ‘employees who worked on a shared project’ or ‘films with a shared actor’.

Then the question becomes connected components. Each piece of the graph is one person; the emails in that piece are their addresses, and the name is on any account vertex in it. A traversal from every unvisited vertex gives me the pieces in $O(V + E)$.

Two things I would be careful about. The graph is definitely not connected — it is supposed to have many pieces, one per person — so the loop is over every vertex, not from one start. And two different people can share a name, so I must never merge on the name; the name is data carried along, not part of the model. That is the trap the problem is actually setting.

Numbers. A hundred thousand accounts averaging three emails: V is 100,000 accounts plus up to 300,000 emails, so 400,000 vertices, and E is 300,000 account-email pairs, stored twice, so 600,000 entries. Around 5 megabytes and one linear pass, against five billion comparisons for the naive version.

Union-Find would also solve this and is arguably the more natural fit for ‘merge groups’ — I meet it on a later day. Either is a good answer; what makes the answer good is putting emails in the graph, and I would say that first whichever traversal I then used.”

9 Recall card

A graph is things plus pairwise connections. Vertices and edges. V and E , and you state both.

Four questions, always, before any code: directed or undirected? weighted or unweighted? can it have cycles? is it connected? Getting the first one wrong gives a confidently backwards answer, not a crash.

Say two sentences out loud before writing a line: “A vertex is . *An edge from A to B means* .” If you cannot finish them, you do not understand the problem yet.

Graphs hide in grids (cells and adjacency), prerequisites, word ladders, puzzle states and shared properties. An **implicit** graph computes its neighbours instead of storing them — that is the form people fail to spot.

Every traversal is $O(V + E)$ — a sum, not a product — and every one needs a **seen** set, because unlike a tree, a graph has cycles. E is the memory, not V .

Retries, backoff, and thundering herds

1 What this is, and why they ask it

A retry is the most obvious thing in distributed systems and one of the most dangerous. When a call fails, you try again. Everyone writes that. Almost nobody writes it correctly, and the incorrect version is how a small problem becomes an outage.

The mechanism is simple. A dependency slows down. Every caller times out and retries. Now the dependency is receiving three times the traffic while already struggling, so more calls time out, so more retries happen. The load that killed it was generated by the clients trying to recover from it. This is called **retry amplification**, and its close relative — everybody doing the same thing at the same instant — is a **thundering herd**.

They ask this because it is the most common cause of a self-inflicted outage in real systems, and because the answer has a specific, memorable shape: exponential backoff with **jitter**, plus a **retry budget**. The candidates who name jitter unprompted have almost always operated something. It also builds directly on [day 122](#) — retrying is only safe at all if the operation is idempotent — and on [day 124](#), because a retry is what you do about a suspected failure.

By the end of this lesson you can write a retry policy with numbers in it, compute the amplification factor for a layered system, explain why randomness is the essential ingredient rather than a nicety, and name the three situations where the correct number of retries is zero.

2 The story

The power goes at twenty past nine on a Saturday morning, which is the worst possible time, and Anwar's phone starts ringing about four minutes later.

He is the caretaker for a building of twenty-four flats. It is an old building, and the power comes and goes, and mostly what he does about it is wait. The line comes back on its own, usually within the hour.

At twenty to ten it comes back. He hears the corridor light click on and the lift start up, and he is halfway out of his chair to go and check the pump when everything goes off again.

He knows exactly what happened, because it has happened perhaps forty times in eleven years, and it took him about six of those years to work out.

When the power went, nobody switched anything off. Every fridge in twenty-four flats stayed on. Every water heater stayed on. Both the lift motors, the corridor lights, the water pump, four air conditioners on the upper floors and whatever else. They all just stopped, still switched on, waiting.

So when the line came back, all of it started at once. Twenty-four fridges, all with warm insides, all pulling hard. Two lift motors. The pump. Everything at the same instant, drawing far more than any of them draws when it is just running along. And the main fuse, which is quite happy carrying the building all day, saw that and went.

What Anwar does now takes twenty minutes and he does it in the dark with a torch.

He goes down to the meter room and pulls the main switches for the whole building. Then he waits for the line to come back — and when it does, nothing happens, because he has turned everything off. Then he brings it back in pieces. The corridor and the pump first. Two minutes. Then the first four flats. Two minutes. Then the next four. All the way up.

Nobody has ever thanked him for it, because from inside a flat it just looks like the power came back a bit slowly. What it looks like from the meter room is a fuse that stops blowing.

The one thing he has never managed to fix is the phone. When the power goes, twenty-four people call him within about five minutes, and there is nothing he can do about any of it. He answers the first three and puts it on silent.

3 The idea in plain English

Anwar's building is a thundering herd, and his fix is the whole lesson.

A retry is trying again after a failure. Straightforward, and correct in one specific circumstance: the failure was **transient**, meaning the same call would probably succeed now. A dropped packet, a brief network blip, a moment of overload. Retrying a transient failure is right.

Retrying a non-transient failure is pure waste. If the request was malformed, or the caller is not authorised, or the record does not exist, then trying again produces the same failure, three times as expensively. **Retry 503 Service Unavailable and 429 Too Many Requests**. **Never retry 400, 401, 403 or 404**. That single rule removes a surprising amount of pointless load.

And a timeout is the ambiguous case. From [day 122](#) you know a timeout does not mean the work did not happen. So retrying a timeout is only safe if the operation is idempotent. If it is not, a retry can charge a card twice. **Idempotency is a prerequisite**

for retrying, not an optional extra.

Retry amplification is why a small problem becomes a big one. Every caller retrying three times means the struggling service now receives three times the traffic. It was already failing at one time. This is Anwar's fridges: the load that blew the fuse is not the building's normal load, it is everything arriving together.

And it multiplies through layers. If the browser retries three times, and the gateway it calls retries three times, and the service behind that retries three times, one user action can become twenty-seven calls to the bottom of the stack. Nobody designed that. Each layer made a locally sensible decision.

Backoff is waiting longer between attempts. Instead of retrying immediately, wait one second, then two, then four, then eight. Doubling each time is **exponential backoff**, and it does two useful things: it gives the struggling service room to recover, and it thins out the retry traffic instead of concentrating it.

But backoff alone does not fix the herd, and this is the part people miss. If a thousand clients all fail at the same instant — because the service went down at that instant — then they all wait exactly one second and all retry at exactly the same moment. Then they all wait two seconds and retry together again. Backoff has made the spikes further apart, but each spike is just as sharp. Anwar's fridges are all waiting the same amount of time.

Jitter is randomness added to the wait, and it is the essential ingredient. Instead of waiting exactly four seconds, wait a random amount between zero and four. Now a thousand clients spread themselves over a four-second window instead of arriving in one instant. The peak load drops by a factor of the window width. This is Anwar bringing the flats back four at a time, two minutes apart — except that randomness does it without a caretaker.

A retry budget caps the whole thing. Rather than “each request may retry three times”, say “retries may not exceed ten percent of total requests”. When everything is healthy, ten percent is plenty and nobody notices. When everything is failing, retries are capped at ten percent extra load rather than three hundred percent, and the service gets a chance to recover. **This is the single most effective control**, because it bounds the amplification no matter how many layers there are.

Retry at one layer, not at every layer. Somebody has to decide where. The usual answer is: retry at the edge closest to the user, where you know the whole operation, and let the inner layers fail fast and report upward. Layers that all retry independently multiply.

And the herd has a second form, which is not about failures at all. When a cache entry expires and a thousand requests all miss at the same moment, they all go to the database together. When a deploy restarts two hundred instances at once, they all open connections together. When ten thousand devices are told to poll “every hour on the hour”, they all arrive on the hour. **Anything that synchronises independent clients creates a herd**, and the fix is always the same: add randomness.

4 The picture

The amplification, drawn:

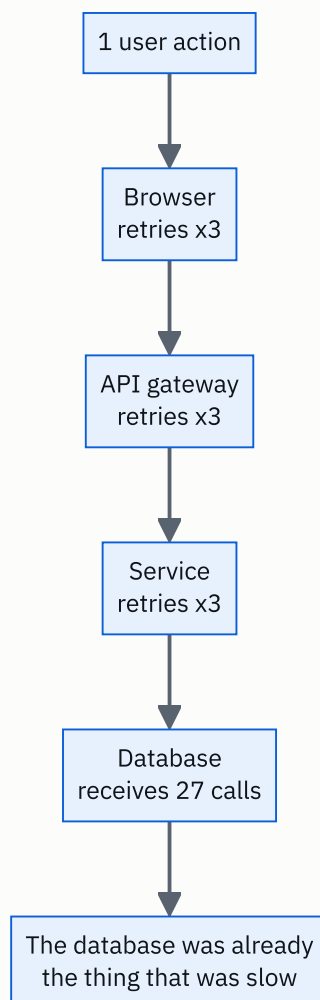
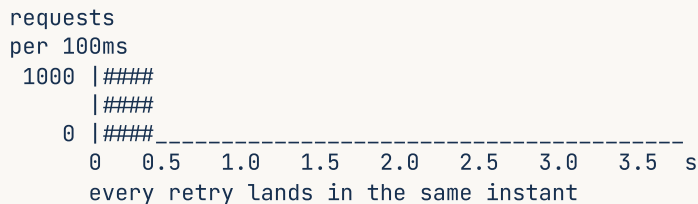


FIGURE 125.2

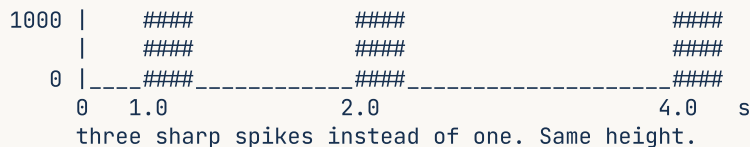
What to notice. No layer did anything unreasonable. Three retries is a normal setting. The 27 is emergent, and nobody owns it, which is why it survives code review.

Now the herd, and what jitter does to it:

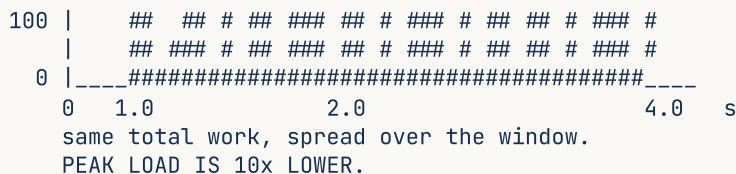
NO BACKOFF – 1,000 clients fail at t=0, retry immediately



EXPONENTIAL BACKOFF, NO JITTER – wait 1s, 2s, 4s



EXPONENTIAL BACKOFF WITH FULL JITTER – wait random(0, cap)



What to notice. The middle picture is the one that catches people. Exponential backoff, applied correctly, still produces a full-height spike — it just produces fewer of them. The height of the spike is what kills the service, and only the randomness reduces the height. **Backoff spaces the herd out in time; jitter breaks the herd up.**

And the retry budget, which is the control that does not depend on tuning:

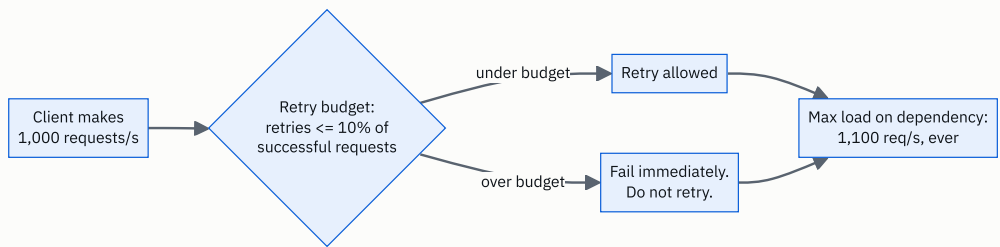


FIGURE 125.3

What to notice. The budget is a property of the client as a whole, not of a single request. That is what makes it work: no individual request can push total load past 110%, however many retries its own policy would have allowed.

5 How it actually works

The four backoff strategies, in order of how bad they are

```
import random

# 1. Immediate retry. Never do this against a shared dependency.
delay = 0

# 2. Fixed delay. Better, but a thousand clients still arrive together.
delay = 1.0

# 3. Exponential backoff. Spikes are further apart, still full height.
delay = min(base * 2 ** attempt, cap)

# 4. Exponential backoff with FULL JITTER. This is the answer.
delay = random.uniform(0, min(base * 2 ** attempt, cap))
```

AWS published the definitive comparison of these in 2015 (the “Exponential Backoff and Jitter” article) and the result surprised people: **full jitter — a uniform random value between zero and the computed cap — beat every cleverer scheme they tested**, on both total completion time and load on the server. Not half the cap plus jitter, not a small random nudge. The full range.

The “decorrelated jitter” variant, `delay = min(cap, random.uniform(base, previous_delay * 3))`, performed comparably and recovers slightly faster. Either is a good answer; full jitter is easier to explain and is what the AWS SDKs implement.

A retry policy with real numbers

```
import random
import time

def call_with_retry(operation, *, attempts: int = 3, base: float = 0.1, cap: float = 5.0):
    """Retry a transient failure with exponential backoff and full jitter."""
    for attempt in range(attempts):
        try:
            return operation()
        except Transient as error:
            # 429, 503, timeout - NOT 4xx
            if attempt == attempts - 1:
                raise
            delay = random.uniform(0, min(cap, base * 2 ** attempt))
            time.sleep(delay)
```

Four numbers, and each of them is a decision you should be able to defend:

- **attempts = 3**. Total attempts, not additional ones — be precise about which you mean, because “3 retries” and “3 attempts” differ by 33% of load. Three is the usual answer because the marginal success rate of a fourth attempt is tiny.
- **base = 100 ms**. Roughly the service’s p50 latency. Shorter and you retry before the first attempt could plausibly have succeeded.
- **cap = 5 s**. The point beyond which waiting is pointless because the caller has given up anyway.
- **The exception filter**. Retry timeouts, connection errors, 429, 502, 503, 504. Do not retry 400, 401, 403, 404, 409, 422. This filter removes more useless load than any tuning.

And the constraint that governs everything: **the total time across all attempts must fit inside the caller’s own timeout**. If the caller gives up after 2 seconds and your policy can sleep for 5, your last two attempts are running for nobody.

Retry budgets, and where they actually exist

Google’s SRE book describes a per-client retry budget: retries may not exceed a fixed percentage — they use 10% — of the request rate. Envoy implements this directly:

```
retry_policy:
  retry_on: "5xx,reset,connect-failure"
  num_retries: 2
  retry_budget:
    budget_percent: 20
    min_retry_concurrency: 3
```

gRPC’s built-in retry support has the same idea as `retryThrottling`, with a token bucket: each failed attempt costs a token, each success adds a fraction of one back, and when the bucket is empty retries stop entirely until things recover. Finagle at Twitter pioneered this and reported it as one of their highest-impact reliability changes.

Why the budget beats tuning `num_retries`: it degrades gracefully. Under normal conditions the budget is never hit and every request gets its retries. Under a broad failure the budget clamps total load at 110% or 120% instead of 300%, automatically, with no operator involved.

The other herds, and their fixes

Cache stampede. A popular key expires; a thousand concurrent requests all miss and all hit the database. Three fixes, in increasing order of effort:

- **Jittered TTL.** Set expiry to `600 + random(0, 60)` seconds instead of exactly 600, so a million keys written together do not expire together.
- **Request coalescing** (also called single-flight). The first miss takes a lock and fetches; the others wait for its result. Go's `singleflight` package and Python's per-key locks do exactly this. One database query instead of a thousand.
- **Probabilistic early refresh.** As a key approaches expiry, each request has a small and growing chance of refreshing it early, so the refresh happens once, before the stampede, without a lock.

Restart herd. Two hundred instances restart after a deploy and all open database connections and warm their caches simultaneously. Fix: staggered rollout — which is what a Kubernetes rolling update with `maxSurge` / `maxUnavailable` already gives you — plus randomised startup delay.

Cron herd. Ten thousand devices told to sync “at midnight” all sync at midnight. Fix: hash the device ID into the schedule so each device gets its own minute. `sync_minute = hash(device_id) % 60`. This is one line and it converts a spike into a flat line.

Reconnect herd. A load balancer restarts and a hundred thousand WebSocket clients all reconnect at once — this is the one that most often takes down the replacement. Fix: jittered reconnect on the client, and it must be built into the client from day one because you cannot deploy a fix to disconnected clients.

What retries look like when they are not yours

An important practical note: your retry policy is not the only one. The HTTP client library retries. The load balancer retries. The SDK retries. The service mesh retries. Envoy will happily retry on top of your application's retries and neither knows about the other.

The discipline is to **retry at exactly one layer and configure every other layer to zero**. Usually that layer is the one that owns the user-facing timeout, because it is the only one that knows how much time is left.

6 The numbers

Amplification through layers. Each layer's multiplier compounds:

```
browser      3 attempts
gateway      3 attempts
service      3 attempts
-----
3 x 3 x 3    = 27 calls to the database for one user action
```

At 1,000 user actions per second during an incident:

```
1,000 x 27    = 27,000 requests per second
               at a database that fell over at 1,200
```

Two layers instead of three, with 2 attempts each:

```
2 x 2          = 4x amplification instead of 27x
1,000 x 4       = 4,000 requests per second
```

Just removing a layer's retries and dropping to two attempts is a **7× reduction**, before any backoff.

What jitter does to the peak. A thousand clients fail simultaneously and retry at a computed delay of 4 seconds:

```
no jitter      all 1,000 arrive within ~10 ms
               peak = 1,000 / 0.01 s = 100,000 requests per second

full jitter    1,000 spread uniformly over 0-4 s
               peak = 1,000 / 4 s   = 250 requests per second
```

A four-hundred-fold reduction in peak load, from one call to `random.uniform`. This is the number to quote; it is what makes jitter sound essential rather than fussy.

Retry budget, sized. A client doing 1,000 requests per second with a 10% budget:

```
normal: 1% of requests fail, all retried
        10 retries/s, budget is 100/s      → never hit, full protection

incident: 100% of requests fail
        without budget: 3,000 retries/s    → 4,000 total, 4x load
        with budget:    100 retries/s      → 1,100 total, 1.1x load
```

The dependency sees 1.1× instead of 4× at the exact moment it can least afford it.

Cache stampede, sized. A key served 10,000 times a second with a 10-minute TTL:


```
without coalescing, on expiry:
  concurrent misses in the ~50 ms it takes to refill
  10,000/s x 0.05 s = 500 simultaneous database queries
  against a database sized for maybe 50 concurrent queries

with coalescing: 1 query. The other 499 wait ~50 ms.
```

When does the extra latency of backoff hurt? Full jitter's expected wait is half the cap:

```
attempt 1 fails, cap = 100 ms → mean wait 50 ms
attempt 2 fails, cap = 200 ms → mean wait 100 ms
attempt 3 succeeds
total added latency: ~150 ms on top of three call durations
```

If the p50 of the underlying call is 20 ms, a request that needed three attempts took roughly 210 ms instead of 20. That is the cost, and it is paid only by the small fraction of requests that fail — which is exactly why you look at p99 latency and not the mean when tuning this.

How much do retries actually help? With independent 1% failures:

```
1 attempt    99% success
2 attempts   1 - 0.01^2 = 99.99%
3 attempts   1 - 0.01^3 = 99.9999%
4 attempts   1 - 0.01^4 = 99.999999%
```

The third attempt takes you from four nines to six. The fourth adds two more nines that nobody will ever measure, and costs 33% more load in an incident. **That is the argument for three, and it is a real argument, not a convention.** It also collapses entirely when failures are *not* independent — when the dependency is down, all attempts fail together and no number of retries helps at all.

7 The trade-offs

Every retry is load you chose to add, at the worst possible moment. The service you are retrying is, by definition, the service that just failed. Retries make transient failures invisible and make sustained failures worse. That asymmetry is the whole design problem, and the budget is the tool that resolves it, because it lets you have the first behaviour without the second.

Backoff trades latency for load. Each additional attempt adds its own wait plus the call duration to the requests that need it. If you are behind a user-facing 2-second timeout, you cannot afford three attempts with a 5-second cap; the arithmetic has to

close. When it does not, the honest answer is fewer attempts, not a shorter cap that retries before recovery is plausible.

Retrying is unsafe unless the operation is idempotent. This is not a trade-off, it is a precondition. If the operation is not idempotent and you cannot make it so with an idempotency key, the correct number of retries is zero, and you say that rather than hoping.

Retry at one layer, and pick the outermost one that knows the whole operation. Retrying at every layer multiplies. Retrying too deep means the inner layer burns the caller's remaining time budget on an operation the caller may already have abandoned. Retrying at the edge costs a full round trip per attempt, which is more expensive per attempt but bounded and visible.

Jitter costs you determinism. Two identical failures now behave differently, which makes tests flaky and incident timelines harder to read. Accept it; the alternative is a synchronised herd. Log the actual delay chosen so the timeline is reconstructable afterwards.

Circuit breakers do what retries cannot. A retry policy still sends every request at a dead dependency, just more slowly. A circuit breaker stops sending entirely after a failure threshold and probes occasionally to see if it has recovered. Retries handle the transient case; breakers handle the sustained one, and you want both — which is [day 126](#).

When is the right number of retries zero? Three cases, and naming them is worth marks. When the operation is not idempotent and cannot be made so. When the failure is a client error — retrying a `400` is pure waste. And when the caller has no time left, because a retry that lands after the user has gone is load with no possible benefit.

8 In the interview

How it gets asked

- *“Everything retried at once and the service died. What went wrong?”* — the direct version.
- *“How would you implement retries?”* — and the answer is judged on whether you say jitter.
- *“A popular cache key expired and the database fell over. Explain.”* — cache stampede.
- *“You deployed and every instance restarted at once. What happens to the database?”*
- *“How many times would you retry, and how do you pick the number?”*
- *“Ten million devices need to check in hourly. Design that.”* — the cron herd, and it is a one-line answer if you see it.

The first ninety seconds

“The failure mode here is that the retries generated the load that killed it. A dependency slows down, every caller times out and retries, and now it is receiving three times the traffic while already struggling. That is retry amplification, and it compounds through layers — browser retries three times, gateway retries three times, service retries three times, and one user action becomes twenty-seven calls to the thing that was already the bottleneck. Nobody designed the twenty-seven; each layer made a locally reasonable choice.

Three fixes, and I would apply all three.

First, exponential backoff with **full jitter** — a delay chosen uniformly at random between zero and the computed cap. The jitter is the essential part and the part people leave out. Plain exponential backoff still has a thousand clients waiting exactly one second and arriving in the same instant; it makes the spikes further apart without making them shorter. Full jitter spreads a thousand clients over a four-second window, which drops peak load by a factor of several hundred for one line of code. AWS published the comparison and full jitter beat every cleverer scheme.

Second, a **retry budget**: retries may not exceed about ten percent of the request rate for that client. That is the control I would fight for, because it bounds amplification regardless of how many layers there are. Healthy, it never binds. In an incident it clamps the dependency’s extra load at ten percent instead of three hundred.

Third, **retry at exactly one layer** and set every other layer to zero attempts. Usually the layer that owns the user-facing timeout, because it is the only one that knows how much time is left.

And the precondition before any of it: retries are only safe if the operation is idempotent, because a timeout does not tell me the work did not happen.

Do you want me to go into the cache stampede version of this, or the circuit breaker?”

The follow-ups

“Why is jitter so important? Backoff already spreads them out.”

“Backoff spreads them out in *time*; it does not make each spike smaller. If a thousand clients all fail at the same instant — which is what happens when a service goes down, they do not fail at random times — then they all compute a one-second delay and all retry one second later, together. Then two seconds later, together. You have turned one spike into three spikes of the same height, and the height is what kills the service.

The arithmetic makes it concrete. A thousand clients arriving within ten milliseconds is a peak of a hundred thousand requests a second. The same thousand spread uniformly over four seconds is two hundred and fifty a second. Four hundred times lower peak, from one call to a random number generator.

The counter-intuitive part, which AWS measured, is that *full* jitter wins — random between zero and the cap, not a small nudge around the cap. It also finishes the whole retry sequence faster on average, because some clients get through early and stop competing.”

“How do you choose the number of attempts?”

“From how much the marginal attempt buys, against what it costs in an incident. With independent one percent failures, one attempt is 99%, two is 99.99%, three is six nines. The fourth adds two more nines nobody will ever measure and costs thirty-three percent more load exactly when the dependency is dying. That is why three, and it is an argument rather than a convention.

Then the constraint that usually decides it anyway: the total across all attempts, including the waits, has to fit inside the caller’s own timeout. If the user-facing budget is two seconds and each call takes 200 milliseconds, I have room for maybe three attempts with a short cap, and I would rather cut to two attempts than have a third that lands after everyone has given up.

And the assumption behind the nines is worth saying out loud: it only holds when failures are independent. When the dependency is actually down, all attempts fail together, the nines are fiction, and the retries are purely harmful. That is what the budget and the circuit breaker are for.”

“The database fell over when a cache key expired. Walk me through it.”

“Cache stampede. One popular key expires, and every concurrent request for it misses at the same moment and goes to the database together. If the key is served ten thousand times a second and a refill takes fifty milliseconds, that is five hundred simultaneous queries against a database sized for maybe fifty connections. It is not a slow query problem; it is a concurrency spike.

Three fixes and I would use the first two. **Jittered TTLs** — set expiry to ten minutes plus a random minute, so a million keys written during a warm-up do not all expire in the same second. That is one line and it removes the correlated case. **Request coalescing** — the first miss takes a per-key lock and fetches; everyone else waits for its result. One query instead of five hundred, and the others pay fifty milliseconds of latency they would have paid anyway.

The third, if the key is hot enough to matter, is **probabilistic early refresh**: as a key nears expiry, each request has a small and rising chance of refreshing it in the background. The refresh then happens before the stampede rather than during it, and no request ever sees a miss.

What I would not do is make the TTL longer. That reduces how often it happens without changing what happens when it does, and it makes the data staler.”

“Ten million devices need to check in every hour. Design it.”

“The trap is telling them all ‘check in at the top of the hour’, which gives me ten million requests in one minute — a hundred and sixty thousand a second — and nothing for the other fifty-nine.

The fix is one line: derive each device’s schedule from a hash of its own ID. `minute = hash(device_id) % 60`, and if I want finer, `second = hash(device_id) % 3600`. Now the ten million are spread evenly across the hour: about two thousand eight hundred a second, flat, forever. Same total work, one fiftieth of the peak capacity.

Two details that matter. The hash must be stable across restarts and app upgrades, so it is a hash of the device ID and not of anything ephemeral — otherwise devices shuffle and I get transient bunching. And the server should never send back an exact ‘retry at this time’ to many clients at once; if I need to shed load, I send `Retry-After` with a jittered value per client.

I would also add a jittered reconnect on the client for the same reason, and I would build it in from the first release, because when a hundred thousand devices reconnect together and take down the replacement instance, I cannot deploy a fix to devices that are not connected.”

“Retries or circuit breaker?”

“Both, and they solve different halves. Retries handle a transient failure — a dropped packet, one bad instance — where the same call would succeed if tried again. A circuit breaker handles a sustained one: after enough failures it stops sending requests entirely, fails fast, and periodically lets one probe through to test recovery.

The reason you need the breaker is that a retry policy still sends every request at a dead dependency, just more politely spaced. It also makes the caller slow — every request now waits out the full backoff sequence before failing — so a dead dependency turns into thread exhaustion in the caller. The breaker converts that into an immediate, cheap failure, which is what lets the caller shed load rather than pile up.

Order of operations: retry inside the breaker. Attempts that fail count against the breaker, and once it opens, the retry policy never runs at all.”

The model answer

“Your checkout service calls a payment gateway. The gateway degrades for ten minutes every few weeks. Design the client behaviour.”

“Let me separate what I do for a two-second blip from what I do for a ten-minute degradation, because they need opposite responses and conflating them is the usual mistake.

First, the precondition. The payment call must be idempotent before I retry anything. Client-generated idempotency key, sent on every attempt, checked at the gateway. Without that, a timeout retry can charge the card twice, and no amount of backoff design fixes that. If the gateway does not support idempotency keys, my answer to ‘how many retries’ is zero on the charge itself, and I move retries to a reconciliation job instead.

For the blip: three attempts, full jitter, tight caps. Base 100 milliseconds — about the gateway’s p50 — doubling, capped at one second. Expected added latency on a retried request is around 150 milliseconds. I retry timeouts, connection resets, 429 and 5xx. I do not retry 400, 402 — a declined card is not transient — or 403. The whole sequence has to fit inside the checkout’s own timeout, which I would set at around three seconds, so three attempts is the ceiling and I would verify that arithmetic rather than assume it.

For the ten-minute degradation: a circuit breaker, because retries are actively harmful here. After, say, fifty percent of calls failing over a rolling twenty-request window, the breaker opens and every subsequent call fails immediately without touching the gateway. One probe every five seconds tests recovery. That protects two things: the gateway, which stops receiving traffic it cannot serve, and my own checkout service, whose threads would otherwise all be sitting in backoff sleeps.

A retry budget on top, at ten percent. Belt and braces, and the thing I would insist on in review. It means that even if someone later raises `num_retries` to five in a config file, the gateway never sees more than $1.1 \times$ my normal rate.

What the user sees, which is the part that actually matters. With the breaker open I do not show an error and lose the sale. I accept the order in a `PENDING_PAYMENT` state, tell the customer their order is confirmed and payment is processing, and put the charge on a queue with a slow, heavily jittered retry — minutes apart, for up to an hour. That converts a ten-minute outage from lost revenue into slightly delayed confirmation emails. The saga and its compensation from [day 121](#) is what makes that safe: if the charge ultimately fails, I cancel the order and release the stock.

And one thing I would check in the existing system before changing anything: how many layers are already retrying. The HTTP client, the service mesh and the SDK all have retry defaults, and I have seen all three enabled at once without any-

one deciding it. I would set every layer except one to zero attempts, and the one that keeps them is the one that owns the user-facing timeout.

The numbers I would put on the design doc: normal load 500 calls per second, worst case with budget 550, peak during recovery capped by jitter at roughly 250 per second above baseline, and a bounded ten-minute window where checkout completes but payment lags. Those are the four numbers I would want an on-call engineer to know.”

9 Recall card

Retry only transient failures. Timeouts, `429`, `5xx`, connection resets. Never `400` / `401` / `403` / `404`. And never retry at all unless the operation is idempotent — a timeout does not mean it did not happen.

Amplification compounds through layers: $3 \times 3 \times 3 = 27$ calls to the thing that was already failing. Retry at exactly one layer and set the others to zero.

Backoff spaces the herd out; only jitter breaks it up. Plain exponential backoff still lands a thousand clients in the same instant. Full jitter — `random.uniform(0, min(cap, base × 2attempt))` — cuts peak load by hundreds of times, and AWS measured it beating every cleverer scheme.

A retry budget is the control that scales: retries capped at ~10% of the request rate. $1.1 \times$ load in an incident instead of $4 \times$, with no tuning and no operator.

The other herds are the same bug: cache stampede (jittered TTL + coalescing), restart herd (staggered rollout), cron herd (`hash(id) % 60`), reconnect herd (jitter in the client, shipped from day one).

DSA topic: What a graph is, and where graphs hide **System design topic:** Retries, back-off, and thundering herds

Code these, in this order

One rule for the whole set, and today it is not about code: **before writing a line, write two sentences at the top of the file.** “A vertex is .” “*An edge from A to B means .*” Then say whether it is directed, weighted, cyclic and connected. Four words. If you cannot fill them in, you have not understood the problem and the code will be a guess.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Find if Path Exists in Graph	LeetCode 1971 (Easy)	Building an undirected adjacency list and not forgetting the second append.
2	Find the Town Judge	LeetCode 997 (Easy)	In-degree and out-degree, with no traversal at all.
3	Number of Provinces	LeetCode 547 (Medium)	A matrix that is the graph, plus the outer loop over all vertices.
4	Keys and Rooms	LeetCode 841 (Medium)	Reachability, and noticing that “all rooms” means checking the unreachable ones.
5	Accounts Merge	LeetCode 721 (Medium)	The modelling move: make the shared property a vertex.

On every problem, do the modelling out loud first

For each of the five, write down the two sentences and the four words *before* looking at any solution. Then compare with what you actually implemented. Problem 5 is the one where a wrong model costs you an hour and a right model costs you four lines.

On problem 3, notice what you were given

The input is an adjacency matrix, not an edge list. Say out loud what that means for the cost of scanning neighbours, and what you would have preferred to be given. Then say why the problem gave you a matrix anyway.

On problem 5, count the comparisons both ways

Solve it once by comparing every account against every other account, and once by making emails into vertices. On 1,000 accounts with 3 emails each, count the comparisons for each version. Two numbers.

Then the recognition exercise

Take these five problem statements. For each, write the two sentences and the four words. Do not solve them.

1. “Given a list of dominoes, can they be arranged in a line so touching ends match?”
2. “Given exchange rates between currencies, is there a sequence of trades that ends with more than you started?”
3. “Given a list of people and who reports to whom, how long until a message reaches everyone?”
4. “You can transform a word by changing one letter, if the result is in the dictionary. Fewest steps from A to B?”
5. “Given a set of tasks, each taking some time, and which must finish before which, what is the earliest everything can be done?”

Two of the five are the same problem in disguise. Say which.

The vocabulary drill

1. Define vertex, edge, path, cycle and degree in one line each.
2. State the four questions you ask about every graph.
3. Give an example of a real relationship that is directed and one that is undirected.
4. Say what in-degree and out-degree are and give a problem where in-degree is the whole answer.
5. Say what dense and sparse mean and roughly where the line is.

The modelling drill

For each row of the hiding-places table, say the vertices and the edges out loud:

1. Courses and prerequisites.
2. A grid of land and water.
3. Words differing by one letter.
4. Puzzle states and legal moves.
5. Accounts and shared emails.
6. Packages and their requirements.

Then say which of the six are implicit graphs and what that changes.

The building drill

1. Write `build_undirected` from memory and say which line makes it undirected.
2. Write `build_directed` and say what changes.

3. Write the prerequisites builder and say the direction out loud as a sentence.
4. Say why the prerequisites builder uses `{v: [] for v in range(n)}` instead of `defaultdict`.
5. Write the grid neighbour function from memory.

The costs drill

1. Give the maximum `E` for an undirected graph on `V` vertices, and for a directed one.
2. Give `E/V` for three real graphs and say what that tells you.
3. Say what `O(V + E)` means in words and why it is a sum.
4. Count out `reachable`'s cost step by step and explain the factor of two.
5. Compute the memory for a million users with 200 friends each, splitting vertices from edges.

The break-it drill

Trigger each and record the exact output or error:

1. `build_undirected` with the second append removed, then reachability in both directions.
 2. `components` on a graph where two vertices appear in no edge.
 3. A traversal from one start on a disconnected graph, counting vertices.
 4. A recursive walk with no `seen` set, on a graph with a cycle.
 5. A list-based adjacency structure on 1-indexed vertex labels.
 6. A graph built with a self-loop, then a degree count.
-

The failure-mode drill

1. Say what retry amplification is in two sentences.
2. Compute the amplification for three layers at three attempts.
3. Say why nobody designed the 27 and why it survives code review.
4. Name the precondition that must hold before any retry is safe.

The status-code drill

1. List the responses you retry.
2. List the responses you never retry.
3. Say why retrying a `400` is worse than useless.
4. Say what makes a timeout the ambiguous case, and what it needs.

The jitter drill

1. Give the four backoff strategies in order and say what is wrong with each of the first three.
2. Explain in two sentences why backoff alone does not fix a herd.
3. Write the full-jitter delay expression from memory.
4. Compute the peak-load reduction from jitter on 1,000 clients over a four-second window.
5. Say what AWS found when they compared the schemes.

The budget drill

1. Define a retry budget in one sentence.
2. Compute total load with and without a 10% budget when everything fails.
3. Say why a budget beats tuning the attempt count.
4. Name two systems that implement it and what they call it.

The other-herds drill

For each, say the cause and the fix:

1. A popular cache key expires.
2. Two hundred instances restart after a deploy.
3. Ten million devices sync at midnight.
4. A load balancer restarts and every WebSocket reconnects.

Then say which one you cannot fix after the fact, and why.

The numbers drill

1. Compute requests per second at the database for 1,000 user actions with $27 \times$ amplification.
2. Compute the success rate for 1, 2, 3 and 4 attempts at a 1% independent failure rate.
3. Say which assumption in that calculation fails during a real outage.
4. Size a cache stampede: 10,000 requests a second, 50 ms refill.
5. Compute the mean added latency of three attempts with full jitter at base 100 ms.

The trade-offs drill

1. Say what every retry costs and when it costs it.
2. Say what backoff trades away, and the arithmetic that has to close.
3. Say which layer should retry and why that one.
4. Say what jitter costs you operationally, and how you mitigate it.

5. Say what a circuit breaker does that no retry policy can.
6. Name the three cases where the right number of retries is zero.

The failure drill

For each, say what happens and what you would build:

1. Every layer in a four-layer stack has retries enabled by default.
2. A thousand clients fail at the same instant with plain exponential backoff.
3. A payment gateway degrades for ten minutes and the client retries three times per request.
4. A non-idempotent charge endpoint is retried after a timeout.
5. A million cache keys are warmed at startup with an identical TTL.
6. A client-side reconnect with no jitter, after a load balancer restart.
7. Retries configured with a 5-second cap behind a 2-second user timeout.

Two of the seven cannot be fixed by tuning any number. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Is this a graph problem? What are the vertices and edges?* The two sentences, the four classification words, and the reason: the question is about following the relationships, not about any single one of them.
 2. *Given accounts with emails, merge the ones belonging to the same person.* The naive quadratic model, then the move that fixes it — make the shared property a vertex — then connected components, the not-connected warning, and the name trap.
 3. *Everything retried at once and the service died. What went wrong?* Amplification with the 27, then the three fixes: full jitter with the peak-load number, a retry budget with the $1.1 \times$ number, and retrying at exactly one layer. Idempotency named as the precondition.
-

Before you move on

- [] I can define vertex, edge, path, cycle and degree without hesitating.
- [] I ask the four questions about every graph before writing code.
- [] I write the two modelling sentences before any line of code.
- [] I know which line makes an adjacency list undirected.

- [] I know why a vertex with no edges can be missing from a `defaultdict`.
- [] I never assume a graph is connected.
- [] I know every graph traversal needs a `seen` set, and why trees did not.
- [] I can recognise an implicit graph and say what changes.
- [] I can name six places graphs hide, with vertices and edges for each.
- [] I can give the maximum `E` for directed and undirected graphs.
- [] I can explain `O(V + E)` in words and say why it is a sum.
- [] I can compute the memory split between vertices and edges.
- [] I solved Accounts Merge with emails as vertices.
- [] I can say what retry amplification is and compute it for three layers.
- [] I know exactly which responses to retry and which never to.
- [] I know idempotency is a precondition, not an extra.
- [] I can explain why backoff without jitter does not fix a herd.
- [] I can write the full-jitter delay expression from memory.
- [] I can quote the peak-load reduction jitter buys.
- [] I can define a retry budget and give the $1.1 \times$ versus $4 \times$ numbers.
- [] I know retries belong at exactly one layer, and which one.
- [] I can name four other herds and their fixes.
- [] I know which herd cannot be fixed after the fact.
- [] I can say what a circuit breaker does that retries cannot.
- [] I can name three cases where zero retries is correct.
- [] I answered all three questions above out loud.

Representing a graph: adjacency matrix versus adjacency list

DATA STRUCTURES AND ALGORITHMS

Representing a graph: adjacency matrix versus adjacency list

SYSTEM DESIGN

Circuit breakers and bulkheads

Representing a graph: adjacency matrix versus adjacency list

1 What this is, and why they ask it

There are two ways to store a graph, and the choice is not a matter of taste. One of them uses a thousand times more memory than the other on a typical real graph, and the other one is a thousand times slower at one specific question. You pick by counting edges.

An **adjacency matrix** is a $V \times V$ grid where the cell at row i , column j says whether there is an edge from i to j . An **adjacency list** stores, for each vertex, only the neighbours it actually has. That is the whole distinction, and everything else in this lesson follows from it.

They ask this because it is the fastest way to find out whether a candidate thinks about resources. “How will you store this graph?” is a question with a right answer and a reason, and the reason involves arithmetic you can do out loud. It also comes up implicitly in every graph problem you will ever be given: the input arrives in one form, and the first thing you do is decide what to convert it into. Getting that wrong costs you a `MemoryError` on the large test case, and it is the most common way a correct graph algorithm fails a judge.

By the end of this lesson you can build both from an edge list, state what each costs for each of the three operations that matter, choose between them from the density in one sentence, and recognise the three cases where the matrix is genuinely the better answer.

2 The story

The chess club meets on Sunday mornings in the back room of the community centre, and the board on the wall is older than most of the members.

It is a big square thing, painted white, with twenty-four names running down the left side and the same twenty-four names running across the top. Where any two rows and columns meet there is a small brass hook. When two people play a game, Anand — who has run the club for nineteen years — hangs a red tag on the hook where their names cross.

He is fond of it. When somebody comes up and says “have I played Sunil?”, he looks at the square where their two names meet, sees a tag or no tag, and answers. It takes him a second and a half. It does not matter whether there are twenty tags on the board or

two hundred.

What the board is not good at is the other question.

Last Sunday a boy asked him who Ravi had played, because he wanted to know if Ravi was any good. Anand ran his finger along Ravi's row — all twenty-four squares, one at a time, most of them empty — and found three tags. Sixteen seconds to find three names out of twenty-four squares.

And here is the thing that bothers him, now that he has noticed it. The board has five hundred and seventy-six hooks. By March, at the end of a season, there are usually about ninety tags on it. The rest of the board is empty. It has always been empty. He is looking after five hundred hooks that will never hold anything.

Farid, who is nineteen and joined in October, does not use the board at all. He keeps it in his phone. Under each person's name he has a short list of who they have played, and nothing else — no empty spaces, because he never writes down a game that did not happen.

Ask Farid who Ravi has played and he reads out three names in about two seconds.

Ask Farid whether Ravi has played Sunil, and he has to open Ravi's list and read down it until he finds Sunil or reaches the end. Usually quick, because the lists are short. But he is reading, and Anand is glancing.

They argue about it in a friendly way most weeks. Anand says the board tells you everything at a glance. Farid says the board is mostly empty and always will be.

They are both right, and neither of them has said the sentence that settles it, which is that a season has ninety games and the board has five hundred and seventy-six hooks.

3 The idea in plain English

Anand's board and Farid's phone are the two representations, and the sentence neither of them said is the rule for choosing.

The board is an adjacency matrix. An **adjacency matrix** for a graph with V vertices is a $V \times V$ grid. The cell at row i , column j holds 1 if there is an edge from i to j and 0 if there is not. Every possible connection has a place, whether or not it exists.

Farid's phone is an adjacency list. An **adjacency list** stores, for each vertex, a list of exactly the neighbours it has. Nothing is stored for a connection that does not exist. In Python this is a dictionary from a vertex to a list, or a list of lists when vertices are numbered 0 to $V-1$.

There is a third form you get handed but rarely keep: the edge list. An edge list is just a list of pairs — $[(0,1), (1,2), (0,3)]$. It is how graph problems almost always give you the input, and it is almost never what you want to work with, because answering “who are the neighbours of vertex 3” means scanning every edge. **Your first line of code in most graph problems is converting an edge list into an adjacency list.**

Three operations decide everything. Every difference between the two comes down to these:

1. **“Is there an edge from A to B?”** The matrix does this in one step — go to the cell, read it. The list has to scan A’s neighbours, which costs the degree of A.
2. **“Give me all the neighbours of A.”** The list does this in one step — hand back A’s list, which is exactly as long as A has neighbours. The matrix has to scan A’s entire row, all V cells, most of which are empty.
3. **“How much memory?”** The matrix always takes V^2 cells, whether the graph has a million edges or none. The list takes about $V + 2E$ entries.

That is the whole comparison, and it is worth memorising as three lines rather than as prose.

Now the sentence that settles it. Almost every algorithm you will write — breadth-first search, depth-first search, Dijkstra, topological sort — spends its whole life asking question 2. It walks to a vertex and asks “what is next to you”. It almost never asks question 1.

So the adjacency list wins by default, and the interesting question is when it does not.

The deciding number is density. With V vertices the most edges you can have is about V^2 . If E is close to that, the graph is **dense**, the matrix’s V^2 cells are mostly full, and it is not wasting anything. If E is closer to V , the graph is **sparse**, and the matrix is a board with five hundred empty hooks.

Real graphs are overwhelmingly sparse. A city’s roads, a social network, a package dependency tree, a course catalogue — in every one of them each vertex touches a handful of others, not all of them. **Assume sparse, use a list, and justify the matrix when you choose it.**

When the matrix is genuinely right, and there are three cases worth knowing:

1. **The graph really is dense** — E close to V^2 , which usually means a “everything is related to everything, with weights” problem such as distances between all cities.
2. **V is small and fixed** — if $V \leq 100$, V^2 is ten thousand cells, which is nothing, and the matrix is simpler to write and faster in practice because it is one contiguous block of memory.

3. You need question 1 constantly — some algorithms, notably Floyd-Warshall for all-pairs shortest paths, are *defined* on a matrix and ask “is there an edge from `i` to `j`” in their innermost loop.

And weights change nothing structurally. A weighted matrix stores the weight in the cell instead of `1`, with some agreed value — usually infinity — meaning “no edge”. A weighted list stores pairs: `graph[a]` is a list of `(neighbour, weight)`. Everything above still applies.

4 The picture

Five vertices and five edges — Anand’s board shrunk to something you can read.

edges: 0-1, 0-3, 1-2, 2-3, 3-4 (undirected)

ADJACENCY MATRIX

	0	1	2	3	4
0	0	1	0	1	0
1	1	0	1	0	0
2	0	1	0	1	0
3	1	0	1	0	1
4	0	0	0	1	0

25 cells, 10 of them 1

ADJACENCY LIST

0 → [1, 3]
1 → [0, 2]
2 → [1, 3]
3 → [0, 2, 4]
4 → [3]

10 entries total
(each edge stored twice)

What to notice. Twenty-five cells to hold five edges. The matrix is symmetric across the diagonal — `cell[0][1]` and `cell[1][0]` both say `1` — because the graph is undirected, and that symmetry is exactly the duplicate storage the list also has. The diagonal is all zeros because there are no self-loops.

Now scale it, which is where the argument stops being about taste:

$V = 1,000$ vertices, $E = 3,000$ edges (a realistic sparse graph)

MATRIX

$1,000 \times 1,000 = 1,000,000$ cells
of which 6,000 are 1

0.6% full

99.4% of the memory holds zeros

LIST

1,000 lists + 6,000 entries
= 7,000 slots

100% full

143x smaller

What to notice. The matrix is not slightly worse here. It is a hundred and forty times worse, and the gap widens as V grows because the matrix grows with V^2 while the list grows with E .

Directed changes only the symmetry:

directed edges: $0 \rightarrow 1$, $1 \rightarrow 2$, $2 \rightarrow 0$

MATRIX	LIST
0 1 2	$0 \rightarrow [1]$
+-----	$1 \rightarrow [2]$
0 0 1 0	$2 \rightarrow [0]$
1 0 0 1	
2 1 0 0	3 entries, not 6

NOT symmetric.
cell[0][1] = 1 but cell[1][0] = 0

What to notice. Each directed edge appears once, in both representations. The matrix's asymmetry is the direction. If you build a directed graph and your matrix comes out symmetric, you have written the assignment twice by accident, which is the mirror image of the bug from [day 125](#).

And the operation that decides it, drawn as the work each one does:

```
"who are the neighbours of vertex 3?"

MATRIX: scan the whole row
row 3:  [ 1 ][ 0 ][ 1 ][ 0 ][ 1 ][ 0 ][ 0 ] ... 1,000 cells
         ^       ^       ^
         read all 1,000 to find 3 neighbours

LIST: hand back the list
graph[3] = [0, 2, 4]
         3 entries, done
```

What to notice. A traversal asks this question once per vertex. Over a whole BFS the matrix does $V \times V = 1,000,000$ cell reads and the list does $2E = 6,000$ entry reads. That is the same $143\times$ again, now in time instead of memory.

5 The code, built step by step

Start with what you are given. Graph problems hand you an edge list and a vertex count.

```
n = 5
edges = [(0, 1), (0, 3), (1, 2), (2, 3), (3, 4)]
```

Build the matrix first, and immediately meet the trap.

```
def build_matrix(n: int, edges: list[tuple[int, int]]) → list[list[int]]:
    """A V x V grid. matrix[a][b] = 1 means there is an edge a → b."""
    matrix = [[0] * n for _ in range(n)]      # NOT [[0] * n] * n
    for a, b in edges:
        matrix[a][b] = 1
        matrix[b][a] = 1                      # drop this line for directed
    return matrix
```

The comment on line one is the most important line in this lesson. `[[0] * n] * n` creates **one** row and then makes `n` references to that same row, so setting `matrix[0][1] = 1` sets it in every row at once. Section 7 shows exactly what that looks like. The comprehension makes `n` distinct rows.

The adjacency list, which you already wrote yesterday:

```
from collections import defaultdict

def build_list(n: int, edges: list[tuple[int, int]]) → list[list[int]]:
    """One list per vertex, holding only the neighbours that exist."""
    graph: list[list[int]] = [[] for _ in range(n)]
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)                  # drop for directed
    return graph
```

A list of lists rather than a dictionary because the vertices are numbered `0..n-1`. Use a `defaultdict` when the vertices are strings or arbitrary labels. **Build from** `range(n)`, **not from the edges**, so that a vertex appearing in no edge still exists — the bug from yesterday.

Now the three operations, side by side, which is the whole comparison in twelve lines.

```
def has_edge_matrix(matrix: list[list[int]], a: int, b: int) → bool:
    return matrix[a][b] == 1                # 0(1)

def has_edge_list(graph: list[list[int]], a: int, b: int) → bool:
    return b in graph[a]                   # 0(degree(a))

def neighbours_matrix(matrix: list[list[int]], a: int) → list[int]:
    return [b for b in range(len(matrix)) if matrix[a][b]]    # 0(V)

def neighbours_list(graph: list[list[int]], a: int) → list[int]:
    return graph[a]                        # 0(1), no copy
```

Read `neighbours_matrix` carefully: it loops over every vertex in the graph to find the few that are neighbours. That loop is inside every traversal you will ever write, which is why the choice matters far more than it looks.

Weighted versions, because most real graphs are:

```

INF = float("inf")

def build_weighted_matrix(n: int, edges: list[tuple[int, int, int]]) → list[list[float]]:
    """No edge is INF, not 0 – because 0 is a legitimate weight."""
    matrix = [[INF] * n for _ in range(n)]
    for i in range(n):
        matrix[i][i] = 0  # distance from a vertex to itself
    for a, b, weight in edges:
        matrix[a][b] = weight
        matrix[b][a] = weight
    return matrix

```

Using `0` to mean “no edge” is a real bug, not a stylistic point: an edge of weight zero is perfectly legal — a free flight, a zero-cost transition — and it becomes invisible. `INF` is unambiguous, and it is also exactly what shortest-path algorithms want to start from.

```

def build_weighted_list(n: int, edges: list[tuple[int, int, int]]) → list[list[tuple[int, int]]]:
    graph: list[list[tuple[int, int]]] = [[] for _ in range(n)]
    for a, b, weight in edges:
        graph[a].append((b, weight))
        graph[b].append((a, weight))
    return graph

```

`(neighbour, weight)` pairs, and the order matters for a reason you will meet on [day 136](#): when these tuples go into a heap you usually want `(weight, neighbour)` instead, so that the heap orders by weight. Store them the way you will use them.

And the memory measurement, which is the thing to actually run:

```

import sys

def matrix_bytes(matrix: list[list[int]]) → int:
    return sys.getsizeof(matrix) + sum(sys.getsizeof(row) for row in matrix)

def list_bytes(graph: list[list[int]]) → int:
    return sys.getsizeof(graph) + sum(sys.getsizeof(row) for row in graph)

```

This does not count the integer objects themselves, which Python shares for small values, so it undercounts both. It is still enough to show the shape of the difference, and running it is more convincing than reading about it.

The complete solution

```
"""Both graph representations, the three operations, and the memory comparison."""

from __future__ import annotations

import random
import sys

INF = float("inf")

def build_matrix(n: int, edges: list[tuple[int, int]], directed: bool = False) → list[list[int]]:
    matrix = [[0] * n for _ in range(n)]          # n distinct rows
    for a, b in edges:
        matrix[a][b] = 1
        if not directed:
            matrix[b][a] = 1
    return matrix

def build_list(n: int, edges: list[tuple[int, int]], directed: bool = False) → list[list[int]]:
    graph: list[list[int]] = [[] for _ in range(n)]
    for a, b in edges:
        graph[a].append(b)
        if not directed:
            graph[b].append(a)
    return graph

def has_edge_matrix(matrix: list[list[int]], a: int, b: int) → bool:
    return matrix[a][b] == 1                      # 0(1)

def has_edge_list(graph: list[list[int]], a: int, b: int) → bool:
    return b in graph[a]                         # 0(degree(a))

def neighbours_matrix(matrix: list[list[int]], a: int) → list[int]:
    return [b for b in range(len(matrix)) if matrix[a][b]]    # 0(V)

def neighbours_list(graph: list[list[int]], a: int) → list[int]:
    return graph[a]                                     # 0(1)

def sizes(n: int, edges: list[tuple[int, int]]) → None:
    matrix = build_matrix(n, edges)
    graph = build_list(n, edges)
    matrix_bytes = sys.getsizeof(matrix) + sum(sys.getsizeof(r) for r in matrix)
    list_bytes = sys.getsizeof(graph) + sum(sys.getsizeof(r) for r in graph)
    cells = n * n
    entries = sum(len(r) for r in graph)
    print(f"V = {n}>5} E = {len(edges):>6}")
    print(f" matrix: {cells:>10,} cells {matrix_bytes:>10,} bytes")
    print(f" list: {entries:>10,} entries {list_bytes:>10,} bytes")
    print(f" ratio: {matrix_bytes / list_bytes:.1f}x")

if __name__ == "__main__":
    small_edges = [(0, 1), (0, 3), (1, 2), (2, 3), (3, 4)]
    m = build_matrix(5, small_edges)
    g = build_list(5, small_edges)

    for row in m:
        print(row)
    print(g)
    print("edge 0-3?", has_edge_matrix(m, 0, 3), has_edge_list(g, 0, 3))
    print("edge 0-2?", has_edge_matrix(m, 0, 2), has_edge_list(g, 0, 2))
    print("nbrs(3) :", neighbours_matrix(m, 3), neighbours_list(g, 3))

    print()
    random.seed(0)
    for n, e in ((100, 300), (1000, 3000), (1000, 400_000)):
```

```

pairs = {(min(a, b), max(a, b))
         for a, b in ((random.randrange(n), random.randrange(n)) for _ in range(e * 2))
         if a < b}
sizes(n, sorted(pairs)[:e])

```

Running it:

```

[0, 1, 0, 1, 0]
[1, 0, 1, 0, 0]
[0, 1, 0, 1, 0]
[1, 0, 1, 0, 1]
[0, 0, 0, 1, 0]
[[1, 3], [0, 2], [1, 3], [0, 2, 4], [3]]
edge 0-3? True True
edge 0-2? False False
nbrs(3) : [0, 2, 4] [0, 2, 4]

V = 100 E = 300
matrix: 10,000 cells      9,656 bytes
list: 600 entries        6,232 bytes
ratio: 1.5x

V = 1000 E = 3000
matrix: 1,000,000 cells   864,056 bytes
list: 6,000 entries       64,264 bytes
ratio: 13.4x

V = 1000 E = 400000
matrix: 1,000,000 cells   864,056 bytes
list: 800,000 entries     7,272,264 bytes
ratio: 0.1x

```

Read those three blocks in order, because they are the entire lesson. At `V = 100` the matrix is barely worse and is simpler to write. At `V = 1000` with a sparse graph it is thirteen times worse. And at `V = 1000` with four hundred thousand edges — a dense graph — **the matrix is ten times better than the list**, because Python's per-list and per-tuple overhead now dominates. The right answer flipped, and it flipped on density.

6 What it costs

The table you should be able to write from memory:

	ADJACENCY MATRIX	ADJACENCY LIST
Space	$O(V^2)$ always	$O(V + E)$
Is there an edge $A \rightarrow B$?	$O(1)$	$O(\text{degree}(A))$
All neighbours of A	$O(V)$	$O(\text{degree}(A))$
Add an edge	$O(1)$	$O(1)$
Remove an edge	$O(1)$	$O(\text{degree}(A))$
Add a vertex	$O(V^2)$ — rebuild	$O(1)$
Iterate every edge	$O(V^2)$	$O(V + E)$
Whole BFS or DFS	$O(V^2)$	$O(V + E)$

The bottom row is the one that decides it, because a traversal is what you will actually run.

Space, in bytes, worked out three ways. Take $V = 10,000$ vertices with $E = 50,000$ edges — a five-neighbour average, which is typical.

```
MATRIX, Python list of lists
10,000 x 10,000 = 100,000,000 cells
each cell a reference      8 bytes
                           = 800 MB
```

```
MATRIX, one bit per cell (bytearray or an integer bitmask per row)
100,000,000 bits / 8      = 12.5 MB
```

```
LIST, Python list of lists
10,000 list objects       10,000 x 56    = 0.56 MB
100,000 entries (2E)     100,000 x 8    = 0.8 MB
plus list over-allocation ~ 2x on entries
                           ~ 2.2 MB
```

Eight hundred megabytes against two. That is not a tuning difference; it is the difference between running and not running. And note the middle line: **bit-packing the matrix cuts it by 64×**, which is the answer when someone insists on a matrix for a large sparse graph. **12.5 MB** is perfectly workable, and it is worth knowing as the escape hatch.

Time, counted from the loops. A BFS over the whole graph:

LIST

each vertex popped once
for each, iterate its own list

V
sum of degrees = $2E$

 $V + 2E = O(V + E)$

$V = 10,000, E = 50,000 \rightarrow 110,000$ steps

MATRIX

each vertex popped once
for each, scan the whole row

V
 V

 $V \times V = O(V^2)$

$V = 10,000 \rightarrow 100,000,000$ steps

Nine hundred times more work for the same traversal on the same graph. In Python, at roughly ten million simple operations a second, that is 0.01 seconds against 10 seconds.

The crossover. The matrix stops being wasteful when E approaches $V^2/2$:

$V = 1,000$	
matrix cells	1,000,000
list entries at E edges	$2E$
equal when $2E = 1,000,000$	$E = 500,000$
maximum possible E	499,500

So for $V = 1,000$ the list only loses when the graph is essentially complete. Accounting for Python's per-object overhead — which the measured output above shows — the practical crossover is much earlier, around $E \approx V^2/20$. **The rule to say out loud: use a list unless the graph is dense or V is under a few hundred.**

One more cost nobody mentions. `has_edge_list` is $O(\text{degree})$ because it scans. If your algorithm asks that question a lot, store each vertex's neighbours as a **set** instead of a list:

list of neighbours	membership test $O(\text{degree})$,	iteration in insertion order
set of neighbours	membership test $O(1)$,	iteration in arbitrary order
	~4x the memory per entry	

That gives you the matrix's $O(1)$ edge test at the list's $O(V + E)$ memory, and it is the correct answer when somebody argues for a matrix purely on the edge-lookup point.

7 The traps

`[[0] * n] * n`

The one that catches everybody, exactly once, and costs an hour when it does.

```
matrix = [[0] * 3] * 3
matrix[0][1] = 1
print(matrix)
```

```
[[0, 1, 0], [0, 1, 0], [0, 1, 0]]
```

One assignment, three rows changed. `[row] * 3` does not copy the row — it makes three references to the same list object. Setting a cell in “row 0” sets it in all of them, because there is only one row.

```
>>> matrix[0] is matrix[1]
True
```

The fix is a comprehension, which evaluates `[0] * n` afresh each time:

```
matrix = [[0] * n for _ in range(n)]
```

```
>>> matrix[0] is matrix[1]
False
```

The reason this is so expensive is that the symptom looks like a graph bug. Your traversal visits vertices it should not, and you spend the hour reading the traversal.

The matrix that will not fit

```
n = 100_000
matrix = [[0] * n for _ in range(n)]
```

```
Traceback (most recent call last):
  File "graph.py", line 3, in <module>
    matrix = [[0] * n for _ in range(n)]
MemoryError
```

Ten billion cells. Eighty gigabytes at eight bytes each. This is the failure mode on a judge’s large test case, and it arrives with no warning after your solution passed every small test. **Before writing a matrix, compute V^2 in your head.** Ten thousand vertices is the point where you should stop.

The zero that means two things

```
matrix = [[0] * n for _ in range(n)]
matrix[a][b] = weight          # what if weight is 0?
```

An edge of weight zero is indistinguishable from no edge. Then:

```
>>> shortest_path(matrix, 0, 4)
inf                                     # there is a path, it just costs nothing
```

Use `INF` for “no edge” in a weighted matrix, always, and set the diagonal to `0` deliberately.

The neighbour scan hidden inside a loop

```
for vertex in range(n):
    for neighbour in range(n):          # ← this is  $O(V)$ , every time
        if matrix[vertex][neighbour]:
            ...
```

This looks like an adjacency scan and it is a full V^2 sweep. It is correct, and on a sparse graph it does a thousand times more work than necessary. The tell is a nested `range(n)` in a function whose complexity you claimed was $O(V + E)$. If you say $O(V + E)$ in an interview and then write this, you will be asked about it.

Building the adjacency list from the edges only

```
graph = defaultdict(list)
for a, b in edges:
    graph[a].append(b)
    graph[b].append(a)
for vertex in graph:                    # misses isolated vertices
    ...
```

The same trap as yesterday, and it is worth repeating because it survives into every problem in this phase. A vertex in no edge never becomes a key. Build from `range(n)` whenever you are given `n`.

The symmetric directed matrix

```
for a, b in edges:
    matrix[a][b] = 1
    matrix[b][a] = 1                    # left in by accident on a DIRECTED graph
```

No error. Your one-way streets are now two-way, your prerequisite graph has cycles it should not have, and your topological sort reports that the courses cannot be ordered. **Check the symmetry deliberately:** for a directed graph, assert that `matrix[a][b] != matrix[b][a]` for at least one pair in your sample.

8 In the interview

How it gets asked

- “How will you store this graph? Why not the other way?” — the direct version, usually asked immediately after you say “I will build an adjacency list”.
- “You have a million users and their friendships. How much memory?”
- “Your solution passed the small cases and got a memory limit error. What happened?”
- “When would you use a matrix?” — a real question with three real answers, and “never” is the wrong one.
- “How would you make the edge lookup fast without a matrix?” — the sets answer.

The first ninety seconds

“Adjacency list, and the reason is density. Let me put a number on it rather than assert it.

A matrix is $V \times V$ cells regardless of how many edges exist. A list is V lists plus $2E$ entries. So the comparison is V^2 against $V + 2E$, and for a typical graph — say a thousand vertices with three thousand edges — that is a million cells against about seven thousand slots. A hundred and forty times more memory to hold the same information, of which 99.4% is zeros.

And the time argument is the same argument. Every algorithm I am going to run here — BFS, DFS, topological sort, Dijkstra — spends its life asking one question: ‘what are the neighbours of this vertex’. A list answers that in $O(\text{degree})$; a matrix scans the whole row in $O(V)$. So a traversal is $O(V + E)$ on a list and $O(V^2)$ on a matrix, which on those numbers is a hundred and ten thousand steps against a hundred million.

The matrix is not always wrong, though, and I would name when I would use it. Three cases: the graph is genuinely dense, with E near V^2 ; V is small and fixed, say under a couple of hundred, where V^2 is nothing and the matrix is simpler and has better memory locality; or the algorithm itself is defined on a matrix, like Floyd-Warshall, which asks ‘is there an edge from i to j ’ in its innermost loop.

The one thing the matrix genuinely has is the $O(1)$ edge test. If I needed that and the graph were sparse, I would not switch to a matrix — I would store each vertex’s neighbours as a set instead of a list. That is $O(1)$ membership at $O(V + E)$ memory, which is the best of both.

Shall I write the build?”

The follow-ups

“A million users, two hundred friends each. Which one, and how much memory?”

“List, and it is not close. Let me do both.

Matrix: a million by a million is 10^{12} cells. At one byte each that is a terabyte, and in Python where a cell is an eight-byte reference it is eight terabytes. It does not exist.

Even bit-packed — one bit per cell — it is 10^{12} bits, which is 125 gigabytes. Still no.

List: E is a million times two hundred over two, so a hundred million edges, stored twice, so two hundred million entries. At eight bytes that is 1.6 gigabytes for the edges, plus a million list objects at about 56 bytes, which is 56 megabytes. Call it two gigabytes, and it fits on one machine.

The number worth noticing is that the edges are the memory and the vertices are a rounding error — 1.6 GB against 56 MB. So ‘can I fit this graph’ is always a question about E .

If two gigabytes were too much, the next step is compressed sparse row: two flat arrays, one holding all neighbours concatenated and one holding the offset where each vertex’s slice begins. That removes every per-list object and gets a hundred million edges into about 800 megabytes of contiguous integers, with much better cache behaviour. The cost is that it is static — you cannot add an edge without rebuilding.”

“When would you actually reach for a matrix?”

“Three situations, and I would recognise them rather than derive them each time.

One, a genuinely dense graph. All-pairs distances between forty cities is a complete weighted graph: E is essentially V^2 , so the list has no advantage and the matrix is simpler and faster. In Python the crossover comes earlier than the theory says, because a list of lists of integers carries enormous per-object overhead — I measured a case at a thousand vertices and four hundred thousand edges where the matrix was ten times *smaller*.

Two, small fixed V . Under about two hundred vertices, V^2 is forty thousand cells and none of this matters. I take the matrix because it is fewer lines and harder to get wrong.

Three, the algorithm demands it. Floyd-Warshall is three nested loops over V reading `dist[i][k] + dist[k][j]`; it is a matrix algorithm and rewriting it for a list makes it worse. Same for anything that uses matrix multiplication to count paths.

And the honest fourth case: when I am writing it on a whiteboard under time pressure and V is small, because a matrix is four lines and I will not make a mistake in it.”

“Your BFS is $O(V + E)$. Prove it, given your representation.”

“Each vertex is pushed to the queue at most once, because I mark it seen when I push it — so at most V pushes and V pops. When I pop a vertex I iterate exactly its own adjacency list, which has `degree(v)` entries. Summed over all vertices, the degrees add up to $2E$ in an undirected graph, because each edge contributes one entry at each of its two ends — or E in a directed one.

So the total is $V + 2E$ operations, which is $O(V + E)$. Note it is a sum, not a product: a million vertices and three million edges is seven million steps, not three trillion.

If I had used a matrix, the second half changes: instead of iterating `degree(v)` entries I scan V cells, so the total becomes $V + V \times V$, which is $O(V^2)$. Same algorithm, same graph, different representation, and the complexity class changed. That is why the representation question comes before the algorithm question.”

“The input is an edge list. Do you always convert?”

“Almost always, and I would say why rather than doing it reflexively. An edge list answers ‘iterate every edge’ perfectly and answers ‘who are the neighbours of vertex 7’ terribly — it costs a full scan, $O(E)$, and a traversal asks that once per vertex, so a traversal on a raw edge list is $O(V \times E)$.

The exceptions are real, though. Kruskal’s algorithm for minimum spanning trees sorts the edges and processes them in order; it never asks for neighbours, so an edge list is exactly right and converting would be wasted work. Bellman-Ford relaxes every edge V times, same story. So the rule is: convert when the algorithm walks the graph, keep the edge list when the algorithm processes edges as a collection.

Conversion is one pass, $O(V + E)$, so when in doubt it is cheap enough to just do.”

The model answer

“You are given the road network of a country — 200,000 junctions and 500,000 roads, each with a length. You need to answer ‘shortest route from A to B’ many times a second. How do you store it?”

“Let me settle the representation first, then say what else the access pattern forces.

Adjacency list, weighted, and the arithmetic makes it obvious. A matrix would be $200,000 \times 200,000 = 4 \times 10^{10}$ cells. At even one byte per cell that is 40 gigabytes, and I need weights, so it is realistically 160 gigabytes or more. It does not fit. The list is 200,000 vertices plus a million directed entries — each road stored at both ends — and each entry is a neighbour and a weight, so roughly 16 bytes: about 16 megabytes for the edges plus the per-vertex overhead. Four orders of magnitude apart, and one of them exists.

The density confirms it. E/V is 2.5 — each junction touches two or three roads, which is what road networks look like. That is about as sparse as a connected graph gets. A matrix would be 0.00125% full.

I would not use Python lists of tuples for this, though, and that is the second half of the answer. For a static graph queried at high rate, compressed sparse row is the right layout: one flat array of a million neighbour indices, one flat array of a million weights, and one array of 200,001 offsets marking where each junction’s slice begins. Total around 12 megabytes of contiguous integers, no per-object overhead, and the neighbours of a junction are adjacent in memory, so a traversal reads sequentially instead of chasing references. On a graph this size that is often a five-to-ten-times speed difference and it comes purely from layout.

The price of CSR is that it is immutable. Adding a road means rebuilding both arrays. For a road network that is fine — roads change monthly, not per second — and I would rebuild offline and swap the new arrays in, the same pattern as the autocomplete rebuild on [day 122](#). If roads changed constantly I would go back to lists of tuples and accept the overhead.

What I would keep an edge list for. Nothing in the query path, but if I ever need to recompute a minimum spanning tree or run an offline analysis over all roads, the edge list is the better input and I would keep the original around rather than reconstruct it.

And one thing I would flag as out of scope but real. At 200,000 junctions, plain Dijkstra per query is too slow for ‘many times a second’, and the fix is not the representation — it is precomputation: contraction hierarchies or a landmark-based heuristic. I mention it because I do not want to leave you thinking the storage choice alone makes this fast. The storage choice makes it *possible*; the precomputation makes it *fast*.”

9 Recall card

Matrix is $V \times V$ cells, always. **List** is $V + 2E$ entries. For $V = 1,000$ and $E = 3,000$ that is a million against seven thousand — $140\times$ — and the gap grows with V .

Three operations decide it: edge test is $O(1)$ on a matrix and $O(\text{degree})$ on a list; neighbours are $O(V)$ on a matrix and $O(\text{degree})$ on a list; memory is $O(V^2)$ against $O(V + E)$. Traversals only ever ask the second one, so **traversal is $O(V^2)$ on a matrix and $O(V + E)$ on a list.**

Use a list by default. Use a matrix when the graph is genuinely dense, when V is under a couple of hundred, or when the algorithm is defined on a matrix (Floyd-Warshall).

Need $O(1)$ edge tests on a sparse graph? Store neighbours as sets, not a matrix. $O(1)$ membership at $O(V + E)$ memory.

The bug: `[[0] * n] * n` makes one row referenced n times. One assignment changes every row, and the symptom looks like a traversal bug. Always `[[0] * n for _ in range(n)]`.

1 What this is, and why they ask it

A **circuit breaker** stops you calling a dependency that is clearly broken. After enough failures it “opens”, and every subsequent call fails instantly without touching the network, until a periodic probe finds that the dependency has recovered.

A **bulkhead** stops one slow dependency from consuming all of your capacity. You give each dependency its own limited pool of workers or connections, so when one of them stalls, the requests that need it queue up in their own compartment and everything else keeps running.

They are two answers to the same failure, and it is the failure that takes down more services than any other: **one slow dependency, and the caller dies before the dependency does.** Not the failing service — the healthy one in front of it. That is counter-intuitive the first time you meet it, and it is why interviewers like the question.

Yesterday’s answer, [retries with backoff](#), makes transient failures invisible. It does nothing for a dependency that is down for ten minutes — in fact it makes that case worse, because every request now sits in a backoff sleep before failing. Circuit breakers and bulkheads are what you add for the sustained case, and a complete answer to “how do you handle a failing dependency” contains all three.

By the end of this lesson you can draw the three-state machine, set every threshold with a reason, compute how many threads a slow dependency consumes and how quickly, size a bulkhead from a latency target, and say what your service returns when the breaker is open — which is the question that separates a design from a mechanism.

2 The story

Rafiq’s restaurant has six tables, four burners, and two people in the kitchen including him.

The fish is the problem. It always has been. The fish comes from a man who drives in from the coast, and about one Friday in six he is late, and on those Fridays the fish that arrives at seven in the evening needs another forty minutes before it is anything you can serve.

For years Rafiq handled it the way you would expect. Somebody orders fish, you take the order, you cook the fish. If it takes fifty minutes, it takes fifty minutes.

What he did not see for a long time is what that does to everything else.

Table four orders fish. That order goes on a burner, and the burner is now occupied for fifty minutes. Table two orders fish. Second burner gone. Two burners left, and one of them has the rice on it, which is always there. So the whole restaurant — six tables — is now running on one burner.

Table five, who ordered two dosas and a coffee, which takes six minutes, waits thirty-five. They are not waiting for fish. They did not order fish. They are waiting because there is nowhere to cook a dosa.

And then they complain, and Rafiq's assistant goes out to explain, and now there is nobody at the burner either.

The change he made was two things, and he made them about eight years apart.

The first one was the easy one. On a night when the fish is late, he tells the boy to go round the tables and say the fish is off the menu tonight. Not "it will be a while" — off. Then at half past eight he sticks his head in the back, looks at what has arrived, and if it is ready he tells the boy to put it back on. If it is not, he checks again at nine.

The second one took longer to see. He now keeps one burner — the small one on the left — for quick things only. Dosas, eggs, reheating. Nothing that takes more than ten minutes goes on it, no matter how busy it is and no matter how much easier it would be at the time. That burner is not available to the fish even when three burners are free.

His assistant thought that was mad for about a year. Four burners and you are only allowed to use three for the thing that takes longest.

But the fish stopped being able to take the whole kitchen down, and that is the entire point of it.

3 The idea in plain English

Rafiq's two changes are the two patterns, and the failure they prevent is the same one.

Start with the failure, because everything else is a response to it. A dependency slows down — not fails, *slows down*. Your service calls it. That call now takes 30 seconds instead of 50 milliseconds. The thread or task handling that request is stuck waiting.

Your capacity is finite, and slow requests eat it. If your service handles requests with a pool of 200 workers, and each request takes 50 milliseconds, you can serve 4,000 requests a second. If each one takes 30 seconds, you can serve about 6. The pool fills up in a few seconds, and then **every** request — including requests that never touch the slow dependency — waits for a free worker.

That is the whole disaster, and it is worth saying in one sentence: a slow dependency converts into total unavailability of the service in front of it, because the calling service runs out of capacity to wait with. Rafiq's dosa customers waited thirty-five minutes for something that takes six.

A circuit breaker is “the fish is off the menu tonight”. It watches the recent failure rate of calls to one dependency. When failures exceed a threshold, it **opens**: from then on, calls to that dependency fail instantly without being attempted. No network call, no waiting, no worker held.

It has three states, and you must be able to name all three.

- **Closed** — normal. Calls go through. Failures are counted.
- **Open** — tripped. Every call fails immediately. Nothing reaches the dependency.
- **Half-open** — testing. After a cooldown, a small number of calls are allowed through. If they succeed, the breaker closes. If any fails, it opens again and the cooldown restarts.

Half-open is the state people forget, and it is the important one. Without it, the breaker either stays open forever or slams the full traffic back at a dependency that may still be broken — Rafiq putting the fish back on the menu at half past eight without going to look at it first. Half-open is looking first.

Failing fast is the point, and it feels wrong. An open breaker is *deliberately returning errors* for calls that might have succeeded. That is the trade, and it is worth being explicit about: you accept a slightly higher error rate on one feature, in exchange for the rest of the service staying up. Waiting is not free, and a request that waits 30 seconds and then fails is strictly worse than one that fails in a millisecond.

A bulkhead is the burner reserved for dosas. The name comes from ships: a hull is divided into sealed compartments, so a hole in one does not sink the vessel. In a service, you give each dependency its own bounded pool — of threads, of connections, of concurrent permits — and when that pool is full, further calls to that dependency are rejected immediately rather than queueing on the shared pool.

The bulkhead limits the blast radius; the breaker shortens the duration. They are not alternatives. The bulkhead means that when the fish is slow, at most one burner's worth of the kitchen is affected, starting from the first order. The breaker means that after a few slow orders you stop taking fish orders at all. You want both, and if you can only have one, the bulkhead — because it works even for failures you did not anticipate and did not configure a breaker for.

And then the question that makes it a design rather than a mechanism: what do you return? An open breaker has to return *something*. Stale cached data. A default. A degraded page with the recommendations panel missing. An honest error. **Deciding that is product work**, and a design that says “the breaker opens” without saying what the user sees has not finished.

4 The picture

The three states, and every transition:

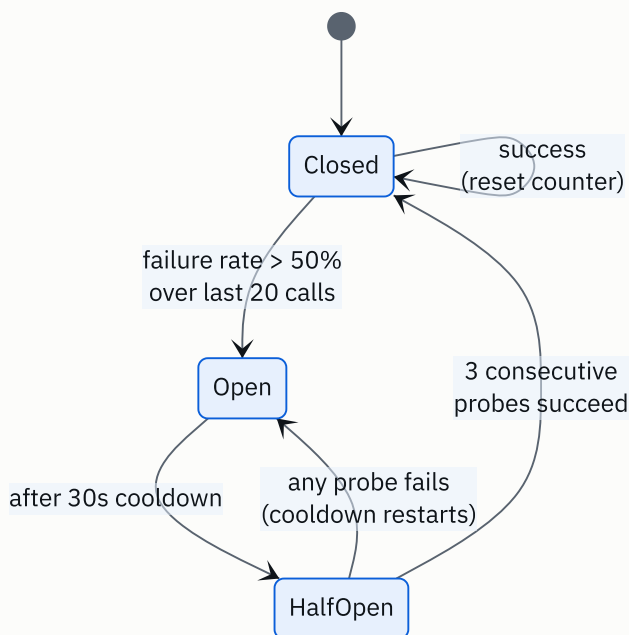


FIGURE 126.1

What to notice. Every arrow has a number on it, and each number is a decision you should be able to defend. “Failure rate over the last 20 calls” rather than “20 failures” matters: a service doing 10,000 requests a second reaches 20 failures in normal operation constantly, so an absolute count trips on a healthy system. A *rate* over a *recent window* does not.

Now the failure the bulkhead prevents, drawn as capacity:

NO BULKHEAD – one shared pool of 200 workers

t = 0s [AAAA BBBB CCCC] 200 free
A = payment (slow), B = catalogue, C = search

t = 2s [AAAAAAAAAAAAAAAAAAAA BBBB CC] 140 free
payment calls piling up at 30s each

t = 8s [AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA] 0 free
every worker is waiting on payment.
CATALOGUE AND SEARCH NOW FAIL TOO,
and neither of them is broken.

WITH BULKHEAD – 50 payment / 100 catalogue / 50 search

t = 8s payment [AAAAAAAAAAAAAAAAAAAA] 50/50 FULL
further payment calls rejected instantly
catalogue [BBBB] 4/100 fine
search [CC] 2/50 fine

Payment is broken. Payment is unavailable.
Nothing else is.

What to notice. In the second picture the payment feature is just as broken. The bulk-head does not fix it and does not try to. What it does is make the damage *proportional* — one feature down instead of the whole service — and that is the only property that matters at three in the morning.

And where the two sit relative to each other in a request's path:

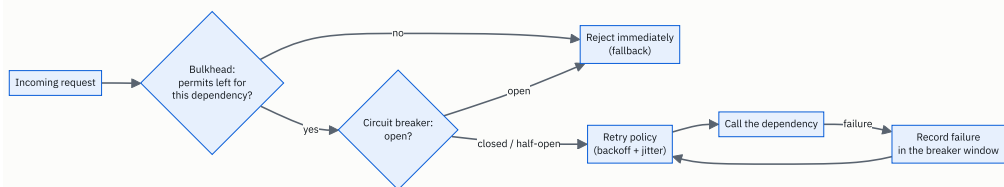


FIGURE 126.2

What to notice. The order is bulkhead, then breaker, then retries, then the call. Retries are *inside* the breaker, so failed attempts count towards opening it and so an open breaker prevents the retry sequence from running at all. Putting retries outside the breaker means you retry past an open breaker, which defeats both.

5 How it actually works

The breaker, in code

The whole mechanism is about forty lines, and writing it is a reasonable interview exercise.

```
import time
from collections import deque

class CircuitOpen(Exception):
    """Raised instead of calling the dependency."""

class CircuitBreaker:
    def __init__(self, *, window: int = 20, threshold: float = 0.5,
                  cooldown: float = 30.0, probes: int = 3) → None:
        self.window = window          # recent calls considered
        self.threshold = threshold     # failure fraction that trips it
        self.cooldown = cooldown       # seconds before probing
        self.probes = probes           # consecutive successes to close
        self.results: deque[bool] = deque(maxlen=window)
        self.state = "closed"
        self.opened_at = 0.0
        self.successes = 0
```

A `deque` with `maxlen` is the whole of “the last N calls” — it discards the oldest automatically, so the failure rate is always over a recent window and never over all history.

```
def call(self, operation):
    if self.state == "open":
        if time.monotonic() - self.opened_at < self.cooldown:
            raise CircuitOpen("breaker is open")
        self.state = "half-open"      # cooldown elapsed: let a probe through
        self.successes = 0
    try:
        result = operation()
    except Exception:
        self._record(False)
        raise
    self._record(True)
    return result
```

`time.monotonic()` and not `time.time()`, for exactly the reason from [day 123](#): a wall-clock correction can make the cooldown appear to have elapsed instantly, or never.


```

def _record(self, ok: bool) → None:
    if self.state == "half-open":
        if not ok:
            self._open()                # one bad probe: back to open
        else:
            self.successes += 1
            if self.successes ≥ self.probes:
                self.state = "closed"
                self.results.clear()
            return
    self.results.append(ok)
    if len(self.results) == self.window:
        failures = self.results.count(False) / self.window
        if failures ≥ self.threshold:
            self._open()

def _open(self) → None:
    self.state = "open"
    self.opened_at = time.monotonic()
    self.results.clear()

```

Two details worth pointing at. The rate is only evaluated once the window is **full** — `len(self.results) == self.window` — so a service that has made three calls, two of which failed, does not trip at 67%. That is a **minimum request volume** threshold, and without it breakers open constantly during low-traffic periods and at startup.

And in half-open, a **single** failure re-opens. There is no averaging in half-open; you are asking one question and one no is a no.

Real implementations and their defaults

Netflix Hystrix made this pattern famous and is now in maintenance mode; its defaults are still the ones everyone quotes: 20 requests minimum in a 10-second rolling window, 50% error threshold, 5-second sleep window before half-open, and thread-pool isolation of 10 threads per dependency by default.

Resilience4j is Hystrix's successor in the Java world, and its shape is worth knowing:

```

slidingWindowSize: 100           # count-based or time-based
minimumNumberOfCalls: 20         # do not evaluate below this
failureRateThreshold: 50         # percent
slowCallRateThreshold: 100       # percent
slowCallDurationThreshold: 2s    # a slow call counts as a failure
waitDurationInOpenState: 30s
permittedNumberOfCallsInHalfOpenState: 5

```

The line to notice is `slowCallDurationThreshold`. **A call that takes 30 seconds and then succeeds is as bad as a failure**, because it held a worker for 30 seconds. Counting slow calls as failures is the feature that catches the actual disaster case, and Hystrix did not have it.

Envoy does it at the proxy layer as *outlier detection*, which is a nice variation: instead of one breaker per dependency, it tracks each individual upstream host and ejects the ones that are failing.

```
outlier_detection:
  consecutive_5xx: 5
  interval: 10s
  base_ejection_time: 30s
  max_ejection_percent: 50
```

`max_ejection_percent: 50` is the important safety valve — it refuses to eject more than half the hosts, on the grounds that if more than half look broken, the problem is probably you and removing them all guarantees an outage.

Polly in .NET and **gobreaker** in Go implement the same state machine. The pattern is universal; only the configuration names change.

Bulkheads: two implementations

Thread-pool isolation gives each dependency its own executor.

```
payment      → pool of 20 threads
catalogue    → pool of 50 threads
search       → pool of 30 threads
```

When the payment pool is exhausted, further payment calls are rejected instantly. The cost is context switching and about 1 MB of stack per thread, plus the calling thread now hands work to another thread — which adds a fraction of a millisecond and, importantly, means the call can be *timed out and abandoned*, because the caller is not the one blocked in the socket read.

Semaphore isolation is cheaper: a counter of permitted concurrent calls, on the calling thread.

```
import threading

payment_permits = threading.Semaphore(20)

def call_payment(operation):
    if not payment_permits.acquire(blocking=False):
        raise BulkheadFull("payment")          # reject, do not queue
    try:
        return operation()
    finally:
        payment_permits.release()
```

No extra threads, almost no overhead. What you lose is the ability to walk away: if the call hangs, the calling thread is still stuck in it, so a semaphore bulkhead absolutely requires a hard timeout on the call itself. Hystrix defaulted to threads for exactly that reason; most modern async services use semaphores because an async task is cheap and the timeout is easy.

`blocking=False` is the entire point. A bulkhead that queues is not a bulkhead — it is a slower version of the shared pool. Rejecting immediately is what keeps the pressure from spreading.

In an async service, the bulkhead is a bounded concurrency limit — `asyncio.Semaphore(20)` or a bounded connection pool. And note that connection pools are already bulkheads if you give each dependency its own: one pool of 100 shared between three databases is not a bulkhead; three pools of 40 is.

Fallbacks, which are the part that makes it a design

An open breaker must return something. In descending order of how good it is:

1. **Cached or stale data.** “Recommendations from five minutes ago.” Usually invisible to the user.
2. **A degraded response.** The product page without the “customers also bought” panel. The feed without read receipts.
3. **A queued write.** Accept the order in `PENDING`, complete it later. This is the saga answer from [day 121](#).
4. **A sensible default.** Standard shipping estimate instead of a computed one.
5. **A clear, fast error.** Worse than the others, better than a 30-second hang.

What is never acceptable is a fallback that hides a correctness problem. Returning “your balance is ₹0” because the balance service is down is far worse than an error. The rule: fall back on things that are *nice to have*, never on things that are *the answer*.

6 The numbers

How fast does a slow dependency exhaust your capacity? This is the arithmetic that justifies the whole lesson.

service worker pool	200 workers
normal request duration	50 ms
normal capacity	$200 / 0.05 = 4,000$ requests per second

The payment dependency goes from 50 ms to 30 s, and 10% of requests use it:

incoming rate		1,000 requests per second
payment requests	$1,000 \times 0.1$	= 100 per second
each holds a worker for		30 s
workers consumed by payment	100×30	= 3,000 workers needed
workers available		200

time to exhaust the pool	$200 / 100$	= 2 seconds
--------------------------	-------------	-------------

Two seconds. After that, the other 900 requests per second — the ones that have nothing to do with payments — have no worker to run on. A dependency used by a tenth of your traffic takes down all of it, in the time it takes to read this sentence.

With a bulkhead of 20 permits for payment:

payment workers capped at		20
payment throughput	$20 / 30 \text{ s}$	= 0.67 per second
payment requests rejected	$100 - 0.67$	= ~99 per second
workers left for everything else	$200 - 20$	= 180
other capacity	$180 / 0.05$	= 3,600 per second

The payment feature is 99% broken and the rest of the service runs at 90% capacity. That is the trade, stated in numbers, and it is the right one.

Sizing a bulkhead. Little's Law gives you the number directly: concurrency = arrival rate $\times$ service time.

payment traffic		100 requests per second
normal payment latency		50 ms
concurrency needed	100×0.05	= 5
add headroom for p99 (200 ms):	100×0.2	= 20
		→ pool size 20

Size for the p99, not the mean, and not for the failure case. Sizing for 30-second calls would mean a pool of 3,000, which is exactly the unbounded behaviour you were trying to prevent.

Breaker thresholds, and why the minimum volume matters.

service at 10,000 requests/s		
normal error rate		0.1%
errors per second	$10,000 \times 0.001$	= 10 per second

With an absolute threshold of “20 failures”, the breaker trips **every two seconds** on a perfectly healthy service. With a rate threshold of 50% over a window of at least 20 calls, it does not trip at all, because 0.1% is nowhere near 50%.

What an open breaker saves. A ten-minute outage of a dependency used by 100 requests per second:

```
without a breaker
requests attempted      100/s x 600 s = 60,000
each waits for a timeout 10 s
worker-seconds consumed 60,000 x 10    = 600,000 worker-seconds
                                   = 167 worker-hours

with a breaker (opens after ~20 calls, ~4 s)
requests attempted      ~ 20
the rest fail in        ~ 0.1 ms
worker-seconds consumed ~ 200 + negligible
```

Three thousand times less capacity burned, and the capacity is what everything else needs.

The half-open cost. With a 30-second cooldown and 3 probes:

```
probes during a 10-minute outage  600 / 30 = 20 attempts
                                   x 3 probes = 60 calls
against 60,000 without a breaker
```

Recovery time. When the dependency comes back, the breaker notices within one cooldown:

```
worst case detection of recovery  30 s
```

That is the cost of the cooldown: up to 30 seconds of unnecessary failure after the dependency is healthy again. A shorter cooldown recovers faster and probes a broken dependency more often. 30 seconds is the common default because it is short relative to a human-noticeable outage and long relative to a restart.

7 The trade-offs

An open breaker returns errors for calls that might have succeeded. That is not a side effect, it is the mechanism. During the cooldown, if the dependency recovered in the first second, you fail requests for 29 more seconds. Shorter cooldowns reduce that and increase load on something that may still be broken. This is the same false-positive trade as [failure detection](#), and you price it the same way.

Breakers are per-instance, and that is usually fine but occasionally wrong. Each of your 50 service instances has its own view. With traffic spread evenly they all trip at roughly the same time, which is fine. With uneven traffic, an instance with low volume

never fills its window and never trips. Sharing breaker state across instances sounds appealing and introduces a coordination dependency into your failure path, which is exactly where you least want one. Almost everyone keeps them local, and the mitigation for the low-volume case is a time-based window rather than a count-based one.

Bulkheads waste capacity by design. Reserving 50 payment permits means that when payments are quiet those 50 slots sit idle while catalogue requests queue. Rafiq's assistant was right that it is inefficient; he was wrong that inefficiency is the deciding factor. **Isolation costs utilisation, and you buy it anyway,** the same way you pay for a spare tyre.

Thread-pool bulkheads cost memory and a context switch; semaphore bulkheads cost you the ability to walk away. Threads: about 1 MB of stack each, plus scheduling. Semaphores: nearly free, but the calling thread is the one that blocks, so a hanging call still holds it — which means a semaphore bulkhead without a hard timeout on the call is not actually a bulkhead.

Too many breakers is its own problem. A service with 30 dependencies and 30 breakers has 30 sets of thresholds that drift out of date, and an incident where four are open at once is genuinely hard to reason about. Put breakers on the dependencies whose failure you have thought about; put bulkheads on all of them, because bulkheads need no per-dependency tuning to be useful.

Fallbacks can be worse than failures. A stale recommendation is fine. A stale price is a financial problem. A defaulted permission check is a security hole. The rule is that you fall back on things that are nice to have and never on the answer itself, and if a dependency has no acceptable fallback, the honest design is a fast, clear error.

When would I not use a circuit breaker? When the dependency is not shared and not remote — an in-process call, a local cache — because there is no queue to protect and no network to fail. And when there is exactly one caller and one dependency at low volume, where a breaker adds a state machine and a set of thresholds to maintain in exchange for a failure mode that cannot happen. The bulkhead I would keep in almost every case, because it is a bounded pool and bounded pools are good practice regardless.

8 In the interview

How it gets asked

- “One downstream service is slow. How do you protect the rest?” — the direct version.
- “What is a circuit breaker? Draw the states.”

- *“Your service went down but its own code was fine. What happened?”* — thread pool exhaustion.
- *“What do you return when the breaker is open?”* — the fallback question, and the one that separates candidates.
- *“How do you pick the failure threshold?”*
- *“Retries or circuit breaker?”* — both, in a specific order.

The first ninety seconds

“The thing to name first is that the danger is not the slow dependency — it is what waiting for it does to me. Let me put a number on that.

Say I have 200 workers and requests normally take 50 milliseconds, so I serve 4,000 a second. Now one dependency goes from 50 milliseconds to 30 seconds, and only a tenth of my traffic touches it — a hundred requests a second. Each one holds a worker for 30 seconds. At a hundred a second I exhaust 200 workers in two seconds, and after that every request fails, including the 90% that never touch that dependency. **A dependency used by a tenth of my traffic takes down all of it in two seconds.**

Two mechanisms, and they do different jobs.

A bulkhead limits the blast radius. Each dependency gets its own bounded pool — say 20 permits for payment — and when it is full, further payment calls are rejected immediately rather than queueing on the shared pool. Payment breaks; nothing else does. Sizing comes from Little’s Law: arrival rate times p99 latency, so a hundred a second at 200 milliseconds p99 is 20. Explicitly **not** sized for the failure case, because that would be unbounded again.

A circuit breaker shortens the duration. It watches the recent failure rate; above a threshold it opens and every call fails instantly without touching the network. After a cooldown it goes half-open and lets a few probes through — if they succeed it closes, if one fails it opens again. Three states, and half-open is the one people leave out; without it you either stay open forever or slam full traffic at something still broken.

The two are not alternatives. The bulkhead bounds the damage from the first slow request; the breaker stops the calls a few seconds later. And I would say up front that the breaker only earns its place if I have decided what to return when it is open — that is a product decision, not a library setting.

Shall I go into the thresholds or the fallbacks?”

The follow-ups

“How do you pick the failure threshold?”

“Three numbers, and the one people get wrong is the third.

The threshold is a rate, not a count — 50% of recent calls, not 20 failures. A service doing ten thousand requests a second with a normal error rate of 0.1% produces ten errors a second, so an absolute count of 20 trips the breaker every two seconds on a perfectly healthy system.

The window is recent — the last hundred calls, or the last ten seconds. Over all history it never trips during an outage because the good calls outnumber the bad ones, and it never closes afterwards for the same reason.

And there is a minimum call volume before the rate is evaluated at all, typically 20. Without it, a low-traffic instance that has made three calls and failed two of them trips at 67% on a statistically meaningless sample. That is the third number and it is the one missing from most hand-rolled implementations.

One more thing I would configure that Hystrix did not have and Resilience4j does: **count slow calls as failures**. A call that takes 30 seconds and then succeeds has done all the damage of a failure — it held a worker for 30 seconds — so a `slowCallDurationThreshold` of, say, two seconds is what actually catches the disaster case.”

“What does the service return when the breaker is open?”

“That is a product decision, and I would push back on any design that answers it with ‘an error’ by default.

The hierarchy I would work through, best first. **Cached or stale data** — recommendations from five minutes ago, and the user cannot tell. **A degraded response** — the product page renders without the ‘customers also bought’ panel; nothing is broken from the user’s point of view, one panel is missing. **A queued write** — accept the order as pending, complete the payment later, which is the saga pattern and it turns an outage into a delayed confirmation email. **A sensible default** — a standard shipping estimate rather than a computed one. And last, **a clear fast error**, which is much better than a 30-second hang but is the weakest option.

The rule I would state is that you fall back on things that are nice to have, never on the answer itself. A stale recommendation is fine. A stale price is a financial incident. A defaulted authorisation check is a security hole. If a dependency has no acceptable fallback — an authorisation service, say — then the honest design is a fast error, and I would say so rather than invent a default that hides a correctness problem.”

“Bulkhead or breaker, if you can only have one?”

“Bulkhead, and I would justify it on coverage rather than on effectiveness.

A breaker protects the dependency I configured it for, at the thresholds I chose, for the failure modes I anticipated. A bulkhead is a bounded pool, and it protects me from *any* dependency behaving in *any* unexpected way — including the one I did not think about, and including a bug in my own code that holds workers. Bounded resources are a property, not a policy.

It also needs almost no tuning. A pool size from Little’s Law is a number I can derive; a failure threshold and a cooldown are numbers I have to guess and revisit.

The breaker’s advantage is duration: with only a bulkhead, a ten-minute outage means ten minutes of calls that each wait for a timeout, which burns capacity and makes every affected request slow rather than instantly failed. So I want both. But if I am triaging a service that has neither, I add bounded pools and hard timeouts first, because those two changes alone remove most of the failure mode.”

“How does this interact with retries?”

“Retries go *inside* the breaker, and the ordering matters in both directions.

Inside means failed attempts count towards opening the breaker, which is right — three failed attempts are stronger evidence of a problem than one. And it means that once the breaker is open, the retry policy does not run at all, so I am not backing off and retrying against a dependency I have already decided is down.

If I put retries outside the breaker, I would retry past an open breaker, which defeats it entirely, and my retry-driven load would not be visible to the breaker’s failure counting.

The full order in the request path is: bulkhead first — do I even have a permit for this dependency — then the breaker, then the retry policy with backoff and jitter, then the call. Four things, each doing a different job: bound the concurrency, stop calling a dead thing, survive a transient blip, and do the work.”

The model answer

“Your product page calls five services: catalogue, pricing, inventory, reviews and recommendations. One of them will eventually be slow. Design for it.”

“The first thing I would do is classify the five, because they are not equal and treating them equally is the mistake.

Catalogue and pricing are essential — without them there is no page. **Inventory is important** — I can render without it but the page is worse. **Reviews and recommendations are optional** — nobody has ever abandoned a purchase because the recommendations panel was missing.

That classification decides everything else, so I would say it out loud before any mechanism.

Every one of the five gets a bulkhead, because bounded pools cost nothing to reason about and protect me from failures I have not predicted. Sizing from Little’s Law: at 1,000 page views a second, a service called on every page at a p99 of 100 milliseconds needs 100 permits, and I would add a little headroom. Recommendations, at a p99 of 300 milliseconds, gets 50 permits and a hard 200-millisecond timeout — because it is optional, its timeout should be *tighter* than the essential services, not looser. That is counter-intuitive and it is right: I am willing to wait for the price, not for the recommendations.

Breakers on the three non-essential services, with fallbacks decided in advance. Recommendations open → render the page without the panel. Reviews open → show the cached aggregate rating and hide the review list. Inventory open → show ‘check availability at checkout’ rather than a possibly wrong in-stock badge, because a wrong stock claim is a customer-service problem and a missing one is not.

No breaker on catalogue or pricing, and this is the part worth defending. If pricing is down I cannot render a product page at any quality, so failing fast gains me nothing except a faster error. What those two get instead is aggressive caching — a short-TTL cache in front of pricing means a pricing outage degrades to serving prices up to sixty seconds old, which is a real product decision I would want signed off, and a far better outcome than an error page.

Thresholds: 50% failure rate over a rolling 100 calls with a minimum of 20, slow calls above two seconds counted as failures, 30-second cooldown, 3 probes to close. Same numbers on all three breakers, because having three different sets of thresholds to maintain buys nothing.

The numbers that justify it. With 200 workers and a 30-second stall on recommendations at 1,000 requests a second, without a bulkhead I lose the entire page in 2 seconds. With a 50-permit bulkhead I lose the recommendations panel and keep 150 workers, which is 75% of capacity for a service that is entirely functional. And with a breaker the recommendations calls stop happening at all after about 4 seconds, so those 50 permits come back too.

What I would monitor: breaker state transitions as events, not as a gauge — I want an alert on ‘the recommendations breaker opened’, with the failure rate and the p99 at that moment, because a breaker opening is the earliest clear signal that something downstream is wrong, and it is usually more actionable than the latency dashboard.”

9 Recall card

The failure is the caller, not the dependency. 200 workers, a 30-second dependency used by 10% of a 1,000/s stream, and the pool is exhausted in **2 seconds** — including the 90% of requests that never touch it.

Breaker = three states. Closed (counting), open (fail instantly), half-open (probe after cooldown; one failure re-opens). Threshold is a **rate over a recent window with a minimum call volume**, never an absolute count. Count slow calls as failures.

Bulkhead = a bounded pool per dependency, and it must reject rather than queue. Size with Little’s Law — arrival rate $\times$ p99 latency — never for the failure case. It costs utilisation and buys isolation.

Order in the request path: bulkhead $\rightarrow$ breaker $\rightarrow$ retries $\rightarrow$ call. Retries live inside the breaker, so attempts count towards opening it and an open breaker stops them running.

A breaker is not finished until you have decided what it returns. Stale data, degraded response, queued write, sensible default, fast error — in that order. Never fall back on the answer itself.

PRACTICE

Practice — Representing a graph: adjacency matrix versus adjacency list

DSA topic: Representing a graph: adjacency matrix versus adjacency list **System design topic:** Circuit breakers and bulkheads

Code these, in this order

One rule for the whole set: **write V and E at the top of the file before anything else, with the actual numbers from the constraints.** Then write one line saying which representation you chose and why. If the constraints say $n \leq 200$, that line is allowed to say “matrix, because 40,000 cells is nothing”.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Find Center of Star Graph	LeetCode 1791 (Easy)	Reading the edge list without building anything at all.
2	Number of Provinces	LeetCode 547 (Medium)	You are handed a matrix. Traverse it without converting.
3	Find if Path Exists in Graph	LeetCode 1971 (Medium)	Converting an edge list, at a size where a matrix would fail.
4	Course Schedule	LeetCode 207 (Medium)	Directed build, and the direction stated as a sentence first.
5	Network Delay Time	LeetCode 743 (Medium)	Weighted list of $(\text{neighbour}, \text{weight})$, stored the way the heap wants it.

On problem 2, resist the conversion

The input is an $n \times n$ matrix and $n \leq 200$. Solve it directly on the matrix. Then say out loud what your traversal costs — it is $O(n^2)$, and that is correct here, not a mistake. Then say at what n you would have converted.

On problem 3, compute the matrix size first

n goes up to 200,000. Before writing anything, compute n^2 and say the number out loud. Then write the adjacency list. This is the habit that stops the memory-limit failure.

Run the size comparison yourself

Take the complete program from the lesson and run it at three densities of your own choosing. Record the crossover point — the E at which the matrix becomes smaller than the list in Python — for $V = 500$ and for $V = 2,000$. Two numbers. Then say why the

theoretical crossover and the measured one differ.

Then deliberately write the aliasing bug

```
matrix = [[0] * 4] * 4
matrix[0][1] = 1
print(matrix)
print(matrix[0] is matrix[1])
```

Run it. Record the output. Then write it correctly and run it again. Do this once, properly, so that the hour you would otherwise lose to it later is spent now instead.

The three-operations drill

1. Give the cost of the edge test in both representations.
2. Give the cost of listing neighbours in both.
3. Give the space cost of both.
4. Say which of the three questions a traversal actually asks, and what follows from that.
5. Give the cost of a whole BFS in both, with the arithmetic.

The choosing drill

1. State the default choice in one sentence, with the reason.
2. Name the three cases where a matrix is right.
3. Say what density means and roughly where the crossover is.
4. Say what you do when you need $O(1)$ edge tests on a sparse graph.
5. Say when you would keep the edge list and not convert at all, with two algorithms.

The building drill

Write each from memory:

1. `build_matrix`, undirected.
2. `build_list`, undirected.
3. Both, directed.
4. The weighted matrix, including what “no edge” is and why.
5. The weighted list, and say which order you store the pair in and why.

The numbers drill

1. Compute matrix cells and list entries for $V = 1,000$, $E = 3,000$.
2. Compute both in bytes for $V = 10,000$, $E = 50,000$.
3. Compute the bit-packed matrix size for the same graph and say what it buys.

4. Compute BFS steps for both on that graph and give the ratio.
5. Compute where the theoretical crossover is for `V = 1,000`.
6. Say why the measured crossover in Python comes much earlier.

The break-it drill

Trigger each and record the exact output or error:

1. `[[0] * n] * n` followed by one cell assignment.
 2. A matrix at `V = 100,000`.
 3. A weighted matrix using `0` for “no edge”, with a genuine zero-weight edge.
 4. A directed graph built with both assignments.
 5. An adjacency list built from a `defaultdict` on a graph with an isolated vertex.
 6. A traversal claiming `O(V + E)` that contains a nested `range(n)`.
-

The exhaustion drill

1. Compute time-to-exhaust for 200 workers, 1,000 requests/s, 10% touching a 30-second dependency.
2. Say in one sentence why the healthy 90% of traffic fails too.
3. Recompute with a 20-permit bulkhead and give both remaining capacities.
4. Say what this failure looks like on a dashboard, and why the cause is hard to find.

The state-machine drill

1. Name the three states and every transition between them.
2. Say what half-open is for, and what happens without it.
3. Say what closes a half-open breaker and what re-opens it.
4. Write the `call` method from memory.
5. Say why `time.monotonic()` and not `time.time()`.

The thresholds drill

1. Say why the threshold is a rate and not a count, with the 10,000 QPS arithmetic.
2. Say why the window must be recent, and what breaks in both directions if it is not.
3. Say what the minimum call volume is for and what happens without it.
4. Say what `slowCallDurationThreshold` catches that a failure count does not.
5. Give the cost of a 30-second cooldown, stated as a user-visible number.

The bulkhead drill

1. Define a bulkhead in one sentence and say where the name comes from.

2. Size one with Little's Law for 100 requests/s at a 200 ms p99.
3. Say why you do not size it for the failure case.
4. Compare thread-pool and semaphore isolation: what each costs and what each cannot do.
5. Say why `blocking=False` is the whole point.

The fallback drill

1. List the five fallback options in order.
2. Give a real example of each.
3. State the rule about what you may and may not fall back on.
4. Give two fallbacks that are worse than an error, and say why.

The ordering drill

1. Give the four things in the request path, in order.
2. Say why retries go inside the breaker, giving both consequences.
3. Say what goes wrong if retries are outside it.
4. Say which of the four you would add first to a service that has none.

The failure drill

For each, say what happens and what you would build:

1. A dependency slows to 30 seconds and 10% of traffic uses it.
2. A breaker configured with an absolute threshold of 20 failures, on a 10,000 QPS service.
3. A breaker that has no half-open state.
4. A bulkhead that queues instead of rejecting.
5. A semaphore bulkhead with no timeout on the call.
6. A fallback that returns a default price.
7. Fifty instances, each with its own breaker, and traffic heavily skewed to five of them.

Two of the seven are configuration mistakes and five are design mistakes. Sort them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *How will you store this graph? Why not the other way?* V^2 against $V + 2E$ with a number, the three operations, the observation that traversals only ask one of them, and the three cases where a matrix is right.
2. *A million users, two hundred friends each. How much memory?* Both representations computed out loud, the edges-are-the-memory observation, and compressed sparse row as the next step with what it costs you.
3. *One downstream service is slow. How do you protect the rest?* The two-second exhaustion arithmetic first, then the bulkhead with Little's Law sizing, then the breaker with its three states, then the fallback decision — and the order in the request path.

Before you move on

- ☐ I can state the space cost of both representations.
- ☐ I can give the cost of all three operations in both.
- ☐ I know which question a traversal actually asks.
- ☐ I can give BFS cost in both and explain where V^2 comes from.
- ☐ I compute V^2 in my head before writing a matrix.
- ☐ I can name the three cases where a matrix is right.
- ☐ I know to use sets when I need fast edge tests on a sparse graph.
- ☐ I know when to keep the edge list and not convert.
- ☐ I have written the `[[0] * n] * n` bug and seen its output.
- ☐ I know why “no edge” is `INF` and not `0` in a weighted matrix.
- ☐ I can do the byte arithmetic for a ten-thousand-vertex graph.
- ☐ I know what bit-packing buys and when to reach for it.
- ☐ I can explain why the measured crossover differs from the theoretical one.
- ☐ I can compute time-to-exhaust for a slow dependency.
- ☐ I can say why the healthy traffic fails too.
- ☐ I can draw all three breaker states and every transition.
- ☐ I know what half-open is for.
- ☐ I know why the threshold is a rate over a recent window.
- ☐ I know what a minimum call volume prevents.
- ☐ I know why slow calls must count as failures.
- ☐ I can size a bulkhead with Little's Law.
- ☐ I know why a bulkhead must reject and not queue.
- ☐ I can compare thread-pool and semaphore isolation.

- [] I can list the five fallbacks and the rule about what not to fall back on.
- [] I know the order: bulkhead, breaker, retries, call.
- [] I know why retries live inside the breaker.
- [] I answered all three questions above out loud.

Breadth-first search on a graph

DATA STRUCTURES AND ALGORITHMS

Breadth-first search on a graph

SYSTEM DESIGN

Distributed locks

1 What this is, and why they ask it

Breadth-first search visits a graph in rings. Everything one step from the start, then everything two steps away, then three. It is nine lines of code, it never revisits a vertex, and it has one property that makes it the single most useful algorithm in this phase: **the first time BFS reaches a vertex, it has reached it by the fewest possible steps.**

You have written BFS before, on [day 101](#), where it walked a tree level by level. Everything changes and nothing changes. The code gains one line — a `seen` set — and that one line is the whole difference between a tree and a graph, because a graph has cycles and a tree does not.

They ask BFS constantly, and almost always in disguise: “fewest moves”, “shortest path”, “minimum number of steps”, “how many levels”. Interviewers at every product company use it, often as the second question, and the reason is that it is short enough to write in ten minutes and has exactly three places to get it wrong.

By the end of this lesson you can write BFS from memory in under two minutes, say why marking a vertex when you push rather than when you pop matters, produce distances and not just visitation, reconstruct the actual path, and handle a graph that is not connected — which is the case sample inputs never contain and real inputs always do.

2 The story

The doctor comes out at eleven at night and tells Devika that her father needs two units of blood, and that the hospital’s bank has none of his group.

She has one phone and about a hundred and forty numbers in it, and it is eleven at night.

She does not go through them alphabetically. She picks eleven people — the ones she would call about anything — and she sends each of them the same message: her father’s group, the hospital name, and please, whoever you know.

Four of them ring back within ten minutes to say they are not that group but they have forwarded it on. One does not reply at all. Two are asleep. By twenty past eleven her eleven messages have become, as far as she can tell, somewhere over a hundred.

At twenty-five past, a number she does not recognise calls. It is a man who says his sister-in-law’s colleague is that group, lives in Malad, and can he give her the number.

Devika works out afterwards how that call reached her. Her friend Roshni sent it to about thirty people in a group. One of them sent it to her brother. Her brother sent it to this man. Four steps: Devika, Roshni, somebody in the group, the brother, the man. And the donor is on the fifth.

What she notices, and it stays with her, is that the answer did not come from the person who tried hardest. It came from the shortest chain that happened to end at someone with B-negative blood. Her cousin in Pune was still forwarding it to people at one in the morning, five and six steps out, long after it was over.

The other thing she notices is the replies that said “I’ve already got this from two other people.”

By midnight the message had gone round and come back to her own building. Her neighbour on the ground floor sent it to her — the original, with her own words in it, having travelled through four other phones. Everyone who received it twice stopped, because there was no point sending it on again. The people it reached, it reached once, by whichever chain got there first.

The donor came at half past one. His name is Sameer and Devika has never met him.

3 The idea in plain English

Devika ran a breadth-first search, and every part of one is in that night.

The start is where you begin. Devika. In code, the vertex you are asked to search from.

The frontier is who you are asking right now. Her eleven friends are the first ring. Everyone *they* forwarded to is the second ring. **BFS processes one whole ring before it starts the next**, and that is what “breadth-first” means: wide before deep.

The queue holds the frontier in order. A **queue** is a line where things come out in the order they went in — first in, first out — which you met on [day 73](#). You push the start, then repeatedly pop the front vertex and push all of its unseen neighbours to the back. Because the ring-one vertices all go in before any ring-two vertex, they all come out first. **The queue enforces the rings by itself; you never write any level-counting logic to get that behaviour.**

“I’ve already got this from two other people” is the **seen** set. Every vertex is recorded the first time it is encountered and skipped forever after. Without it, a message going round in a circle never stops, and in code you loop forever or run out of memory. **A tree traversal did not need this because a tree has no cycles. Every graph traversal needs it, without exception.**

Now the property that makes BFS worth learning. When BFS first reaches a vertex, it has reached it by the fewest edges possible. Not “usually”. Always.

The reason is worth saying in one sentence, because interviewers ask: **BFS finishes ring k entirely before it begins ring $k+1$, so if a vertex were reachable in k steps, it would already have been found in ring k — it cannot first appear in ring $k+1$.** That is the whole argument, and it is enough.

The consequence: **BFS solves shortest path on an unweighted graph.** Every edge costs one step, and BFS finds the fewest steps. Devika’s donor came through the shortest chain not because anyone tried harder, but because the search reached him at whatever ring he sat in and nothing later could reach him sooner.

This is exactly why the property breaks when edges have weights. If some chains are “slower” than others — a friend who takes two hours to forward — then fewest *steps* is no longer cheapest *cost*, and BFS gives the wrong answer. That is [day 136](#).

Mark a vertex when you push it, not when you pop it. This is the detail that separates a working BFS from one that is quietly quadratic. If you only mark on pop, a vertex with five unseen neighbours pointing at it gets pushed five times before any of them is popped. The answer stays correct; the queue fills with duplicates and the cost blows up. Devika’s neighbour stopped forwarding the moment he *received* it the second time, not after he had finished acting on it.

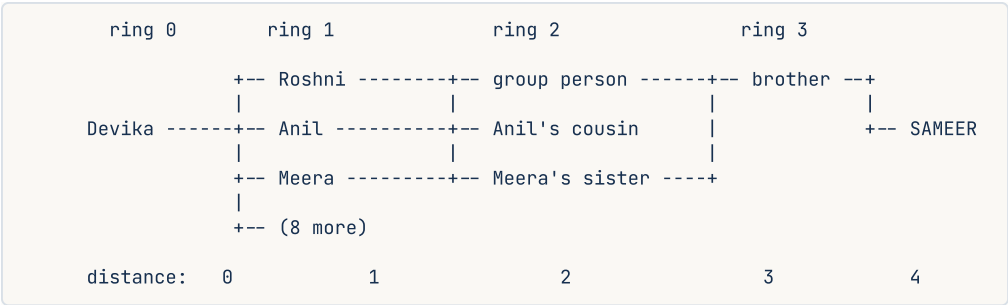
Distances come free. Store, alongside `seen`, how far each vertex is. When you push a neighbour of a vertex at distance `d`, its distance is `d + 1`. One extra dictionary and the search now answers “how far”, not just “reachable”.

Paths come nearly free. Store, for each vertex, which vertex you reached it from — its **parent**. When you arrive at the goal, walk the parents backwards to the start and reverse. Devika reconstructed the chain this way: the man came from the brother, who came from someone in the group, who came from Roshni.

And the graph may not be one piece. If some people have no phone, no chain reaches them. A single BFS from Devika finds exactly her own **connected component**, and nothing else. Whenever the question is about *all* vertices rather than the ones reachable from a start, you need an outer loop over every vertex. This is the mistake sample inputs never expose.

4 The picture

Devika’s network, drawn as rings:



What to notice. Sameer is at distance 4. Every one of the eleven friends is at distance 1, whatever order Devika sent the messages in. The search finishes all of ring 1 before any of ring 2 exists, and that ordering is produced entirely by the queue.

The mechanics, step by step, on a small graph:

graph: A -- B A -- C B -- D C -- D D -- E				
step	queue (front → back)	pop	push	seen
0	[A]	-	-	{A}
1	[B, C]	A	B, C	{A, B, C}
2	[C, D]	B	D	{A, B, C, D}
3	[D]	C	- (D seen)	{A, B, C, D}
4	[E]	D	E	{A, B, C, D, E}
5	[]	E	-	{A, B, C, D, E}
distances: A=0 B=1 C=1 D=2 E=3				

What to notice at step 3. C looks at D, sees it is already in `seen`, and does not push it. D was discovered from B at step 2, by a path of the same length. Whichever one got there first wins, and the other does nothing — the “I already got this” reply. Without the `seen` check D would be pushed twice and processed twice.

And the failure mode the `seen` set prevents:

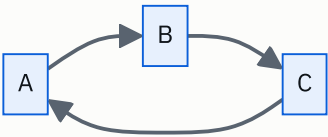


FIGURE 127.1

without <code>`seen`</code> :	A, B, C, A, B, C, A, B, C, ... forever
with <code>`seen`</code> :	A, B, C, done

What to notice. Three vertices and three edges is enough. You do not need a big graph to loop forever; you need one cycle, and real graphs are full of them.

5 The code, built step by step

Start with the shape you will write ninety percent of the time.

```
from collections import deque

def bfs(graph: dict[int, list[int]], start: int) → set[int]:
    """Every vertex reachable from start."""
    seen = {start}
    queue = deque([start])
    while queue:
        current = queue.popleft()
        for neighbour in graph[current]:
            if neighbour not in seen:
                seen.add(neighbour)          # mark on PUSH
                queue.append(neighbour)
    return seen
```

Nine lines. Read the third-from-last one again: `seen.add(neighbour)` sits immediately before the append, not after the pop. Section 7 shows what happens if you move it.

`deque` and not a list, because `list.pop(0)` is $O(n)$ — it shifts every remaining element left — and `deque.popleft()` is $O(1)$. On a graph with a million vertices that difference is the difference between seconds and hours.

Now distances, which is what the interview usually actually wants.

```
def bfs_distances(graph: dict[int, list[int]], start: int) → dict[int, int]:
    """Fewest edges from start to every reachable vertex."""
    distance = {start: 0}
    queue = deque([start])
    while queue:
        current = queue.popleft()
        for neighbour in graph[current]:
            if neighbour not in distance:          # distance IS the seen set
                distance[neighbour] = distance[current] + 1
                queue.append(neighbour)
    return distance
```

The `seen` set has disappeared, because `distance` does its job — a vertex is seen exactly when it has a distance. One dictionary instead of two, and the same guarantee. Vertices absent from the result are unreachable, which is more useful than a separate check.

Then the path itself.


```
def bfs_path(graph: dict[int, list[int]], start: int, goal: int) → list[int] | None:
    """One shortest path from start to goal, or None if there is none."""
    parent: dict[int, int | None] = {start: None}
    queue = deque([start])
    while queue:
        current = queue.popleft()
        if current == goal:
            break
        for neighbour in graph[current]:
            if neighbour not in parent:
                parent[neighbour] = current
                queue.append(neighbour)
    if goal not in parent:
        return None
    ...
```

`parent` now does three jobs at once: it is the seen set, it records where each vertex was reached from, and its keys are the reachable set. The `break` is a real optimisation — once you pop the goal, no later discovery can beat it, so there is nothing left to learn.

Walking the parents backwards:

```
path = []
node: int | None = goal
while node is not None:
    path.append(node)
    node = parent[node]
return path[::-1] # built backwards, so reverse
```

Four lines. The start's parent is `None`, which is what terminates the walk, and it is why `parent[start]` is set to `None` explicitly rather than left out.

Now the two things sample inputs hide. First, the graph is often not connected:

```
def all_components(graph: dict[int, list[int]]) → list[list[int]]:
    """Every piece, not just the one containing some start vertex."""
    seen: set[int] = set()
    pieces = []
    for vertex in graph:
        if vertex in seen:
            continue
        piece, queue = [], deque([vertex])
        seen.add(vertex)
        while queue:
            current = queue.popleft()
            piece.append(current)
            for neighbour in graph[current]:
                if neighbour not in seen:
                    seen.add(neighbour)
                    queue.append(neighbour)
        pieces.append(piece)
    return pieces
```

The inner block is unchanged BFS. The outer loop is the whole addition, and `seen` lives outside it so that a vertex found in one piece is never used to start another.

Second, level-by-level processing, when the question asks about rings rather than distances:

```
def bfs_levels(graph: dict[int, list[int]], start: int) → list[list[int]]:
    """The vertices at distance 0, 1, 2, ... as separate lists."""
    seen = {start}
    frontier = [start]
    levels = []
    while frontier:
        levels.append(frontier)
        nxt = []
        for current in frontier:
            for neighbour in graph[current]:
                if neighbour not in seen:
                    seen.add(neighbour)
                    nxt.append(neighbour)
        frontier = nxt
    return levels
```

No queue at all. Two lists — the current ring and the next one — swapped at the end of each round. This is often clearer than counting `len(queue)` at the top of a loop, and it is the shape to reach for whenever the problem says “in each round” or “per minute” or “per level”.

The complete solution

```
"""Breadth-first search on a graph: reachability, distances, paths, components."""

from __future__ import annotations
from collections import defaultdict, deque

def build(edges: list[tuple[str, str]], directed: bool = False) → dict[str, list[str]]:
    graph: dict[str, list[str]] = defaultdict(list)
    for a, b in edges:
        graph[a].append(b)
        if not directed:
            graph[b].append(a)
    return graph

def bfs_distances(graph: dict[str, list[str]], start: str) → dict[str, int]:
    """Fewest edges from start to every reachable vertex.  $O(V + E)$ ."""
    distance = {start: 0}
    queue = deque([start])
    while queue:
        current = queue.popleft()
        for neighbour in graph[current]:
            if neighbour not in distance:
                distance[neighbour] = distance[current] + 1
                queue.append(neighbour)
    return distance

def bfs_path(graph: dict[str, list[str]], start: str, goal: str) → list[str] | None:
    """One shortest path, or None. Ties are broken by adjacency order."""
    if start == goal:
        return [start]
    parent: dict[str, str | None] = {start: None}
    queue = deque([start])
    while queue:
        current = queue.popleft()
        for neighbour in graph[current]:
            if neighbour in parent:
                continue
            parent[neighbour] = current
            if neighbour == goal:
                # stop as soon as it is found
                path, node = [], neighbour
                while node is not None:
                    path.append(node)
                    node = parent[node]
                return path[::-1]
            queue.append(neighbour)
    return None

def components(graph: dict[str, list[str]]) → list[list[str]]:
    """Every piece of the graph. Needed whenever the question says 'all'."""
    seen: set[str] = set()
    pieces: list[list[str]] = []
    for vertex in graph:
        if vertex in seen:
            continue
        piece, queue = [], deque([vertex])
        seen.add(vertex)
        while queue:
            current = queue.popleft()
            piece.append(current)
            for neighbour in graph[current]:
```

```

        if neighbour not in seen:
            seen.add(neighbour)
            queue.append(neighbour)
        pieces.append(sorted(piece))
    return pieces

if __name__ == "__main__":
    contacts = [
        ("devika", "roshni"), ("devika", "anil"), ("devika", "meera"),
        ("roshni", "group"), ("anil", "cousin"), ("meera", "sister"),
        ("group", "brother"), ("sister", "brother"),
        ("brother", "sameer"),
        ("pune_uncle", "pune_friend"),          # a separate piece
    ]
    network = build(contacts)

    distances = bfs_distances(network, "devika")
    for person in sorted(distances, key=lambda p: (distances[p], p)):
        print(f" {distances[person]} {person}")

    print("path:", bfs_path(network, "devika", "sameer"))
    print("unreachable:", bfs_path(network, "devika", "pune_friend"))
    print("pieces:", components(network))

```

Running it:

```

0 devika
1 anil
1 meera
1 roshni
2 cousin
2 group
2 sister
3 brother
4 sameer
path: ['devika', 'roshni', 'group', 'brother', 'sameer']
unreachable: None
pieces: [['anil', 'brother', 'cousin', 'devika', 'group', 'meera', 'roshni', 'sameer', 'sister'], ['pune_friend', 'pune_uncle']]

```

Two things to look at. `brother` is at distance 3 and is reachable through both `group` and `sister` — the path returned goes via `group` because `roshni` was listed first, and a different adjacency order would give an equally short, different path. **BFS finds a shortest path, not *the* shortest path**, and if a problem demands a specific one you must add a tie-break.

And `pune_friend` is `None`. The uncle's part of the network exists and Devika's search will never reach it.

6 What it costs

Count it directly from the loops.

Time.

each vertex enters the queue at most once	V pushes
for each popped vertex, scan its neighbours	V pops
sum of all degrees	degree(v)
	2E (undirected)
	E (directed)

total	V + 2E → O(V + E)

Each vertex enters the queue at most once because it is marked on push. That single sentence is the proof, and it is what the interviewer is listening for. If you mark on pop instead, the bound becomes E pushes rather than V, and the cost becomes O(V + E) in time still but with a queue that can hold E entries instead of V.

O(V + E) is a **sum**, not a product. A graph with a million vertices and five million edges is eleven million operations — about a second in Python, instant in a compiled language.

Space.

seen / distance / parent	at most V entries	O(V)
the queue	at most V entries	O(V)
the graph itself	V + 2E	O(V + E)

		O(V + E)

How big can the queue actually get? This is the question people get wrong. It holds one ring at a time, so its maximum size is the size of the widest ring. On a graph where one vertex connects to a million others, the first ring is a million entries:

```

star graph, V = 1,000,001
ring 0      1 vertex
ring 1      1,000,000 vertices all queued at once
queue peak                      1,000,000 entries
at ~8 bytes per reference        8 MB, plus deque block overhead

```

On a long chain the queue never holds more than two:

```

path graph A-B-C-...-Z, V = 1,000,000
queue peak                      1 or 2 entries

```

Same O(V) bound, wildly different reality. Say “the queue holds the widest level” if asked; it is more precise than O(V) and shows you have thought about it.

Compared with the alternatives.

```
BFS on adjacency list       $O(V + E)$   
BFS on adjacency matrix    $O(V^2)$       - the row scan, from day 126
```

```
V = 10,000, E = 50,000  
list    10,000 + 100,000 = 110,000 steps  
matrix  10,000 x 10,000  = 100,000,000 steps    ~900x more
```

On an implicit graph — a grid, from [day 130](#) — the same formula applies with $V = \text{rows} \times \text{cols}$ and $E \leq 2 \times \text{rows} \times \text{cols}$, because each cell has at most four neighbours and each edge is shared:

```
1,000 x 1,000 grid  
V = 1,000,000  
E ≤ 2,000,000  
  
 $O(V + E) = 3,000,000$  steps
```

And the number that matters for interviews: BFS is linear, so if a problem's constraints say $n \leq 10^5$ vertices and 10^5 edges, BFS is comfortably fast and you do not need to justify it further. If the constraints say 10^9 , you are not building the graph at all and the question is about something else.

7 The traps

Marking on pop instead of on push

The near-miss, and it looks more natural than the correct version:

```
while queue:  
    current = queue.popleft()  
    if current in seen:          # check on pop  
        continue  
    seen.add(current)           # mark on pop  
    for neighbour in graph[current]:  
        queue.append(neighbour)  # push without checking
```

The output is correct. The cost is not. Every vertex is pushed once per edge pointing at it, so the queue holds E entries instead of V . On a dense graph:

```
V = 5,000 vertices, E = 12,000,000 edges  
mark on push:  queue peak ~ 5,000 entries  
mark on pop:   queue peak ~ 12,000,000 entries
```

```
MemoryError
```

Or, on a judge, `Time Limit Exceeded` with a solution you are certain is $O(V + E)$. **Mark on push. Say the sentence “each vertex enters the queue at most once” as you write it, and if that sentence is not true of your code, the code is wrong.**

`list.pop(0)` instead of `deque.popleft()`

```
queue = [start]
while queue:
    current = queue.pop(0)          # O(n), shifts everything left
```

No error, and correct answers on small inputs. On a large one:

```
V = 200,000
deque.popleft()  0.09 s
list.pop(0)      41 s
```

Removing from the front of a Python list moves every remaining element. Doing that V times is $O(V^2)$. `from collections import deque` is the fix and it should be automatic.

No `seen` set at all

```
def walk(graph, current):
    for neighbour in graph[current]:
        walk(graph, neighbour)
```

```
Traceback (most recent call last):
  File "bfs.py", line 6, in walk
    walk(graph, neighbour)
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

Or, in the iterative version, no error at all — just a queue that grows until the process is killed. One cycle is enough. A tree traversal never needed this, which is exactly why the habit is missing.

Assuming one component

```
distances = bfs_distances(graph, 0)
print(len(distances))          # "the number of vertices"
```

It is the number of vertices *reachable from 0*. On a disconnected graph that is an undercount, and the sample input is almost certainly connected so nothing tells you. Whenever the question says “all”, “every”, “how many groups” or “count the...”, write the outer loop.

The unreachable lookup

```
distances = bfs_distances(graph, start)
print(distances[goal])
```

```
Traceback (most recent call last):
  File "bfs.py", line 14, in <module>
    print(distances[goal])
KeyError: 'pune_friend'
```

A vertex that was never reached has no entry. Use `distances.get(goal, -1)` or check membership. Returning `-1` for “unreachable” is the convention most problems expect, and it is worth confirming with the interviewer rather than assuming.

BFS on a weighted graph

The subtle one, because it runs fine and gives a wrong answer:

```
A --1-- B --1-- C      BFS says A to C costs 2 steps
A -----10----- C   BFS says A to C costs 1 step

BFS returns the 1-step path. Its real cost is 10.
The 2-step path costs 2.
```

BFS minimises the number of edges, not the sum of the weights. They are the same thing only when every edge costs the same. The moment weights differ, you need Dijkstra. If all the weights happen to be 0 or 1, there is a neat middle option — 0-1 BFS with a deque — which is [day 141](#).

8 In the interview

How it gets asked

BFS is almost never named. These phrasings mean it:

- “Find the shortest path from A to B.” — in an unweighted graph.
- “What is the minimum number of moves to...”
- “How many steps until every orange rots / every computer is infected / everyone knows?”
- “Find the shortest transformation sequence from one word to another.”
- “How many people are within two degrees of separation?”
- “Count the islands.” — BFS as the component-finder, not as a path-finder.

The tell, in one line: “fewest”, “minimum number of steps”, or “level by level”, on something with equal step costs.

The first ninety seconds

“Fewest steps on an unweighted graph is breadth-first search, and I would say why rather than assert it: BFS finishes every vertex at distance k before it looks at anything at distance $k+1$, so the first time it reaches a vertex it has reached it by the fewest possible edges. That is the guarantee I am relying on, and it holds only because every edge costs the same — if there were weights I would need Dijkstra instead.

The structure is a queue and a seen set. Push the start, then repeatedly pop the front and push every unseen neighbour. Two details I want to get right as I write it.

First, **I mark a vertex as seen when I push it, not when I pop it.** If I mark on pop, a vertex with a hundred edges pointing at it gets queued a hundred times before any of them is processed — the answer is still correct but the queue holds E entries instead of V , and on a dense graph that is a memory limit error. Marking on push is what makes ‘each vertex enters the queue at most once’ true, and that sentence is the complexity proof.

Second, **deque, not a list**, because popping from the front of a list is $O(n)$ and doing it V times turns a linear algorithm quadratic.

For distances I use one dictionary rather than a set plus a dictionary — a vertex is seen exactly when it has a distance. For the actual path I keep a parent pointer and walk it back at the end.

Cost is $O(V + E)$ time and $O(V)$ extra space, and I would note the queue’s peak is the widest level, not the whole graph.

One question before I code: is the graph guaranteed connected, and what should I return for an unreachable target?”

The follow-ups

“Prove that BFS gives the shortest path.”

“By induction on the distance, and it is short enough to say out loud.

The start is at distance 0 and is correct trivially. Suppose every vertex at true distance k has been assigned distance k and is in the queue before anything at distance $k+1$. When I pop a vertex at distance k and look at an unseen neighbour, that neighbour is one edge from a vertex at distance k , so its true distance is at most $k+1$. It cannot be less than $k+1$, because if it were $j \leq k$ it would have been discovered while processing ring $j-1$, which has already finished. So it is exactly $k+1$.

The property the queue provides is that all of ring k is popped before any of ring $k+1$, which is exactly what first-in-first-out gives you when you push ring $k+1$ while popping ring k .

And the assumption I would name explicitly: every edge costs one. With weights, a two-edge path can be cheaper than a one-edge path, the induction breaks at the first step, and BFS confidently returns the wrong answer.”

“The graph is huge — a billion vertices. Now what?”

“BFS is linear, so the problem is not the algorithm, it is that $O(V)$ memory is a billion entries and the graph does not fit on one machine.

Three things I would do, and which one depends on the question. If I only need vertices within a small distance — ‘friends of friends’ — I bound the depth and stop, and the cost becomes the size of that neighbourhood rather than the graph. Two rings on a social graph with an average degree of 200 is about forty thousand vertices, not a billion.

If I genuinely need the whole traversal, **bidirectional BFS** halves the exponent when I have both endpoints: search forward from the start and backward from the goal, one ring each, alternating, and stop when the frontiers touch. If the branching factor is b and the distance is d , that is $2 \times b^{(d/2)}$ instead of b^d . At $b = 200$ and $d = 6$ that is two hundred thousand vertices explored instead of sixty four trillion. It is the single biggest win available on a large sparse graph and it costs about fifteen extra lines.

And if it truly does not fit, the traversal moves out of memory: keep the frontier as a set on disk or in a distributed store, and do each ring as a batch join against the edge list. That is what graph frameworks do, and the cost model changes from operations to passes over the data.”

“There are several shortest paths. Return all of them.”

“That changes the parent structure and it is worth being careful, because the naive change is wrong.

Instead of one parent per vertex, I keep a list of parents. When I reach a vertex that already has a distance, I do not ignore it — I check whether the distance I would assign equals the distance it already has. If it does, this is another equally short route, so I append to its parent list. If it is larger, I ignore it as before.

Then reconstructing is a backwards walk that branches at every vertex with more than one parent, which is a depth-first search over the parent structure.

The thing to flag is the output size. The number of shortest paths can be exponential in the number of vertices — a grid of alternating diamonds doubles the count at every level — so ‘return all of them’ can be an unbounded amount of output even though the search itself is still $O(V + E)$. I would ask whether they want all paths or just the count, because the count is a small extra field on each vertex and the paths are not.”

“Same question, but some edges cost 0 and some cost 1.”

“Then plain BFS is wrong, but full Dijkstra is more than I need. The right tool is 0-1 BFS.

Use a deque instead of a queue. When I relax an edge of weight 0, I push the neighbour to the **front**; when I relax an edge of weight 1, I push it to the **back**. That keeps the deque sorted by distance with at most two distinct values in it at any time, which is exactly the invariant a queue gives you for uniform weights.

The cost stays $O(V + E)$ — no heap, no log factor — and it is the whole answer for problems like ‘minimum number of walls to break to cross the grid’ or ‘minimum sign flips’, where the two costs are the only two. Above two distinct weights, the invariant breaks and it is Dijkstra.”

The model answer

“Given a start word, an end word, and a dictionary, find the length of the shortest transformation sequence, changing one letter at a time, where every intermediate word must be in the dictionary.”

“Let me name the model first, because the whole difficulty of this problem is in the model and not in the search.

A vertex is a word. An edge joins two words that differ in exactly one letter. Undirected, unweighted, and the question is fewest steps — so this is BFS, and the guarantee I want is that the first time I reach the end word I have reached it by the fewest transformations.

The graph is implicit, and building it explicitly is the trap. With n words of length L , comparing every pair is n^2 comparisons at L each. Ten thousand words is a hundred million comparisons before the search even starts.

So I compute neighbours instead of storing them. From a word of length L , I generate the $L \times 25$ candidates that differ in one position and keep the ones present in a set of the dictionary. For $L = 5$ that is 125 candidate strings and 125 set lookups per word — a constant, and completely independent of how many words the dictionary holds. Total work is $O(V \times L \times 25)$ instead of $O(V^2 \times L)$, which at ten thousand five-letter words is 1.25 million operations instead of 500 million.

The alternative model, if L were large, is to bucket words by wildcard patterns — `h*t`, `*ot`, `ho*` — so that all words sharing a pattern are neighbours. That is $O(V \times L)$ to build and $O(L)$ per lookup, and it wins when L exceeds about 25. I would mention it and stay with the direct generation for ordinary words.

Then it is plain BFS: distance dictionary as the seen set, mark on push, `deque`, return the distance of the end word or 0 if it never appears.

Two edge cases I would state before coding. The end word may not be in the dictionary at all, in which case the answer is 0 and I should check that in one line before starting rather than discovering it after a full traversal. And the start word is conventionally not required to be in the dictionary.

If they want it faster, bidirectional BFS. The branching factor here is genuinely large — a common five-letter word has dozens of neighbours — so searching from both ends and meeting in the middle roughly squares down the explored set. On the standard LeetCode version that is the difference between passing comfortably and passing marginally, and the extra code is about fifteen lines: two frontiers, always expand the smaller one, stop when a word appears in the other side’s distance map.

Cost: $O(V \times L \times 25)$ time and $O(V \times L)$ space for the dictionary set and the frontier. I would say those with L in them rather than hiding it in the constant, because for this problem the word length is a real parameter.”

9 Recall card

BFS visits in rings, and the first time it reaches a vertex, that is the fewest edges. Because ring `k` finishes entirely before ring `k+1` begins. **True only when every edge costs the same** — weights mean Dijkstra.

Nine lines: `deque`, a `seen` set, **pop the front, push unseen neighbours**. Never `list.pop(0)` — that is $O(n)$ and makes the whole thing quadratic.

Mark on push, not on pop. It is what makes “each vertex enters the queue at most once” true, and that sentence is the complexity proof. Marking on pop gives a queue of size `E` and a memory error on a dense graph.

One dictionary does three jobs: `distance` (or `parent`) is also the seen set, and its keys are the reachable set. Walk `parent` backwards and reverse for the actual path.

$O(V + E)$ time — a sum — and $O(V)$ space, where the queue’s peak is the widest level. And if the question says “all” or “how many groups”, wrap it in an outer loop over every vertex, because the graph is probably not connected.

1 What this is, and why they ask it

A distributed lock lets one machine, out of many, say “I am the one doing this right now” — and have the others believe it. A single cron job across ten servers. One leader among five replicas. One process rebuilding a cache while nine others wait.

The implementation is three lines of Redis. The problem is that those three lines are not correct, and neither is the next version, and neither is the version after that. **Every distributed lock has a failure mode where two holders exist at once**, and the interview is entirely about whether you know which one and what you do about it.

They ask it because it sits on top of everything in this phase. It needs [failure detection](#) to know when a holder has died, and it inherits that lesson’s impossibility: you cannot tell a dead holder from a slow one. It needs [fencing](#) to survive being wrong. It touches [clocks](#), because a lease expires on a clock and clocks lie. It is the single best question for finding out whether someone reasons about failure or recites patterns.

By the end of this lesson you can implement a lock correctly with the parts that matter, name the exact sequence in which two processes end up holding it, explain fencing tokens well enough that an interviewer stops asking, and — most importantly — say when the right answer is to not use a lock at all.

2 The story

The storeroom is on the ground floor behind the stairs, and there is one key. It hangs on a nail in the watchman’s cabin, on a loop of blue nylon rope with a wooden tag.

The rule has been the same for as long as anyone remembers. If you need something from the storeroom, you go to the cabin, take the key off the nail, do what you need to do, and put it back.

The nail does all the work. You do not have to ask anyone whether the storeroom is free. If the key is on the nail it is free, and if the nail is empty somebody is in there, and you come back later. Ganesh, who sits in the cabin, is not part of the system at all — he just happens to be sitting near the nail.

It worked perfectly for years, and then in March a man from the third floor took the key, got a stepladder out, and drove to his sister’s house in Vasai with the key still in his pocket.

The nail was empty until Tuesday.

Nobody could get in. The society's committee met about it, and what they decided was reasonable and is also where the trouble starts. They had a second key cut, and they gave it to Ganesh, and the rule now is that if the nail has been empty since the previous evening, Ganesh may open the storeroom with the second key.

Which mostly works. And about once a year it produces the afternoon that everyone remembers.

A painter took the key on a Thursday morning, went up to the terrace to do something on the way, and left the key in his bag on the terrace and forgot about it for the whole day and the whole night. Friday afternoon Ganesh, following the rule, opened the storeroom with the second key for the plumber, who started moving the paint tins to get at the pipe.

At which point the painter came down.

Two people in the storeroom, both certain they were allowed to be there, both correct according to the rule. Nothing terrible happened — there was some shouting and the plumber put the tins back — but the committee had built exactly what they had spent the year avoiding.

Ganesh's own view, which he offers to anyone who asks and which is more or less the correct one, is that you cannot look at an empty nail and know whether the person is inside working or halfway to Vasai. The nail cannot tell you that. Nothing hanging on a wall can.

3 The idea in plain English

Every part of a distributed lock is on that nail.

A lock is a shared flag that only one holder may have. The nail is the shared place. The key on it means free; the key gone means taken. Any process wanting the resource checks the same place, so they all agree.

In code the shared place is a single external store, because the processes have no memory in common. Redis, ZooKeeper, etcd, or a row in a database. The lock is a key in that store, and “taking the key” is a write that succeeds only if nobody has written it already.

Acquiring must be atomic — one indivisible step. Two people cannot both reach for the key at the same instant and both get it. In code that means the check and the write happen together, in the store, not in your process. `SET lock_name owner NX in`

Redis — `NX` meaning “only if it does not exist” — is one operation that either succeeds or fails. **The version where you `GET` first and then `SET` is broken**, because two processes can both `GET` “free” before either `SET`s.

Releasing is deleting the key — hanging it back on the nail.

The man who drove to Vasai is a crashed holder, and he is the whole problem. If a process takes the lock and dies, the key never comes back and the resource is locked forever. Everything else in this lesson is a consequence of that one failure.

The lease is the committee’s fix. A lease is a lock with an expiry: the key comes back on its own after 30 seconds whether or not the holder returns it. In Redis that is `SET Lock owner NX PX 30000`. Now a crashed holder costs you 30 seconds instead of forever.

And the lease creates the painter. If the holder is not dead but merely slow — a long garbage-collection pause, a stalled disk, a network hiccup — the lease expires while it is still working. Somebody else takes the lock. The first holder wakes up believing it still holds the lock, and now two processes are inside.

This is not a bug you can fix by choosing a better timeout. It is exactly the impossibility from [day 124](#): you cannot distinguish a crashed holder from a slow one. A longer lease means a crashed holder blocks for longer. A shorter lease means a slow holder gets overtaken more often. **There is no setting that avoids both, and saying that out loud is the answer the interviewer wants.**

So you stop trying to prevent two holders and make it harmless instead. That is fencing. Every time the lock is granted, a counter increases, and the holder is given that number — its **fencing token**. The holder sends the token with every operation on the protected resource, and **the resource itself remembers the highest token it has seen and rejects anything lower.**

The painter comes down holding token 33. The plumber has token 34. The storeroom has already seen 34, so the painter’s actions are refused. He finds out he no longer holds the lock from the refusal, which is the only channel that works — nothing else can reach him while he is paused.

Fencing is the difference between a lock that is a hint and a lock that is safe. Without it, correctness depends on the timeout being right, which it cannot always be. With it, an over-eager timeout costs you a wasted handover instead of corrupted data.

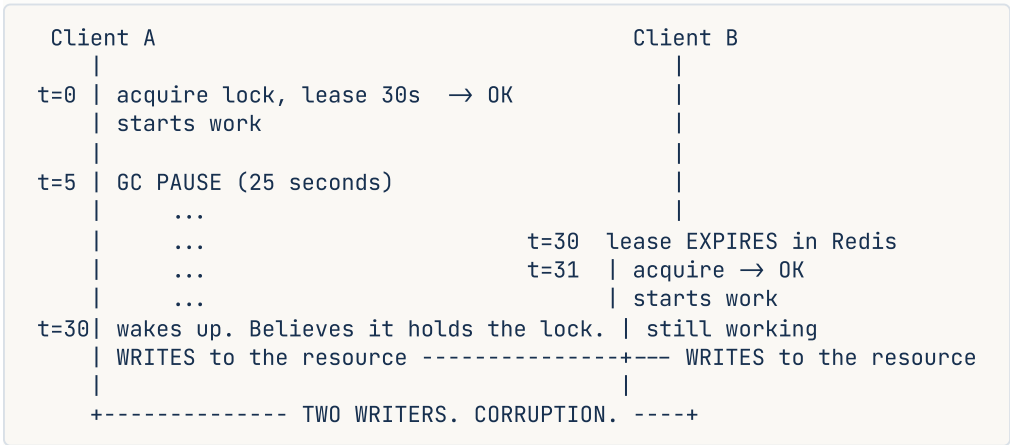
And there is one more thing to get right: only the owner may release. If the painter comes back and hangs his key on the nail while the plumber is inside, he has just released somebody else’s lock. So the lock value is not “taken” — it is a unique identifier

for the holder, and release must check it. In Redis that check and delete must themselves be atomic, which means a small script rather than a `GET` followed by a `DEL`.

Finally, and this matters more than the machinery: often you should not use a lock. The reason to lock is usually that two writers would conflict. If you can make the write itself safe — a conditional update, a unique constraint, an atomic increment, or routing all writes for a key to a single owner — you have removed the problem rather than guarded it. **A lock is what you reach for when you cannot do that**, and a good answer says so before reaching.

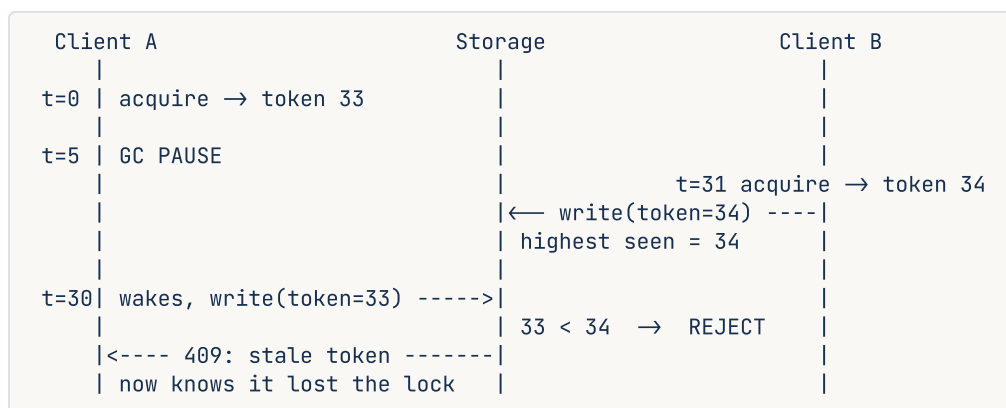
4 The picture

The failure everyone eventually meets, on a timeline:



What to notice. Client A did nothing wrong. It acquired the lock legitimately and was paused by its own runtime, which it cannot control and cannot detect. No lock implementation prevents this, because from Redis's side a paused client and a dead client are the same thing.

Now the same timeline with fencing, which does not prevent it and makes it harmless:



What to notice. The rejection happens at the **storage layer**, not in client A. That placement is the whole idea: client A cannot be trusted to check anything, because it was unconscious. The only component that can enforce the rule is the one receiving the writes.

And the state of a lock over its lifetime:

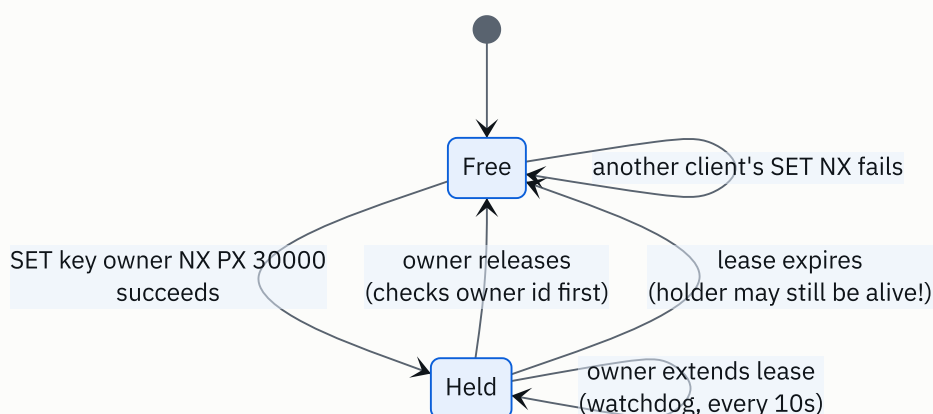


FIGURE 127.2

What to notice. Two different arrows lead out of **Held** to **Free**, and only one of them is safe. The expiry arrow is the one that produces two holders, and the watchdog loop — the self-transition — is what reduces how often it fires.

5 How it actually works

The Redis version, built correctly

Start with the version that is wrong, because you need to recognise it:

```
# BROKEN: check-then-act is two operations
if not redis.exists("lock:report"):
    redis.set("lock:report", "me")
```

Two clients can both pass the `exists` check before either `set`s. The whole point of a lock is gone.

The correct acquire is one command:

```
import uuid

token = str(uuid.uuid4())
acquired = redis.set("lock:report", token, nx=True, px=30_000)
#               ^^^^^^^^^^  ^^^^^^^^^^^^^
#               only if absent  expire in 30s
```

`nx=True` makes it atomic — Redis either creates the key or does not. `px=30000` is the lease, and it must be set **in the same command**. Setting the key and then calling `EXPIRE` is a second broken version: a crash between the two leaves a lock with no expiry, held forever.

The value is a random unique token, not `"locked"`, and that is what makes release safe.

Release must verify ownership, and the check and delete must be atomic too:

```
RELEASE = """
if redis.call("get", KEYS[1]) = ARGV[1] then
    return redis.call("del", KEYS[1])
else
    return 0
end
"""
redis.eval(RELEASE, 1, "lock:report", token)
```

A Lua script runs atomically in Redis. Without it:

```
# BROKEN
if redis.get("lock:report") == token:
    # ← the lease can expire and someone else can acquire, right here
    redis.delete("lock:report")    # deletes THEIR lock
```

This is the painter hanging his key back on the nail while the plumber is inside. It is a real production bug and it is subtle enough that it survives review.

The watchdog

A 30-second lease and a job that sometimes takes 45 seconds is a lock that expires mid-work every time. The answer is not a longer lease — that just makes a crash block for longer. It is to **extend the lease while still working**:

```
import threading

def watchdog(redis, key, token, ttl_ms=30_000, stop=None):
    """Re-extend the lease at a third of its length, while we still hold it."""
    EXTEND = ""
    if redis.call("get", KEYS[1]) == ARGV[1] then
        return redis.call("pexpire", KEYS[1], ARGV[2])
    else return 0 end
    """
    while not stop.wait(ttl_ms / 3000):          # every 10s for a 30s lease
        if not redis.eval(EXTEND, 1, key, token, ttl_ms):
            break                                # we lost it - stop working
```

Refreshing at a third of the lease gives two chances to survive a missed refresh before expiry. Redisson (the Java Redis client) does this by default with a 30-second lease and a 10-second refresh, and calls it exactly that — a watchdog.

And the important half: when the extension fails, the worker must stop. A watchdog that only extends and never reports failure gives you a worker that carries on after losing the lock, which is the failure mode you were trying to avoid.

Fencing tokens

The lock hands out a monotonically increasing number:

```
token = redis.incr("lock:report:fence")      # 33, then 34, then 35...
redis.set("lock:report", token, nx=True, px=30_000)
```

Every write to the protected resource carries it, and the resource enforces it:

```
UPDATE report_state
  SET data = %(data)s, fence = %(token)s
WHERE id = %(id)s
  AND fence < %(token)s;          -- rejects a stale holder
```

Zero rows updated means you lost the lock. **The check is in the `WHERE` clause of the write, at the storage layer** — not in the client, which may be paused, and not in the lock service, which does not see the writes.

This is Martin Kleppmann's argument against Redis-based locking as a safety mechanism, and it is the single most cited exchange in this area. It is worth knowing both sides: Kleppmann's point is that no lock service can guarantee mutual exclusion without the resource participating; Redis's author Antirez's reply is that for many workloads the lock

is an efficiency measure and occasional double execution is acceptable. **Both are right about different requirements, and naming that distinction — efficiency versus correctness — is the mark of a good answer.**

ZooKeeper and etcd, which do it properly

ZooKeeper's recipe uses **ephemeral sequential nodes**. A client creates a node under `/lock/`, and ZooKeeper appends a sequence number. The client with the lowest number holds the lock; everyone else watches the node immediately below theirs and waits.

Three properties come free and are the reason to prefer this:

- **Ephemeral** means the node is deleted automatically when the client's session ends — so a crashed holder releases the lock without any expiry guesswork. The session is maintained by heartbeats, so this is failure detection built into the store.
- **Sequential** gives you a fencing token for nothing: the sequence number is monotonically increasing.
- **Watching the node below you** rather than the lock itself avoids the herd from [day 125](#) — when the holder releases, exactly one waiter wakes, not all of them.

etcd is equivalent with leases and a `Txn` compare-and-swap on the key's `CreateRevision`, and Kubernetes uses exactly this for its own leader election — a `Lease` object with a holder identity and a renew deadline.

These are still not immune to the paused-client problem — a ZooKeeper session can expire while the client is paused, exactly like a Redis lease. What they give you is a much better-behaved failure detector and a fencing token you did not have to build.

Redlock, and why it is contentious

Redlock is the algorithm for acquiring a lock across `N` independent Redis instances: acquire on a majority, and only count it as held if you got most of them within a small fraction of the lease. It removes the single-Redis single point of failure.

What it does not remove is the paused client, and it adds a dependence on bounded clock drift across the instances. The consensus among practitioners is: **use Redlock if you want availability, use ZooKeeper or etcd if you want correctness, and use fencing tokens either way.** Saying that sentence is enough; relitigating the debate is not.

And the alternatives that are usually better

Before any of this, ask whether a lock is needed at all:

- **Conditional write.** `UPDATE ... WHERE version = 7` or DynamoDB's `attribute_not_exists`. Optimistic, no lock, no expiry, and the storage engine enforces it.

- **Unique constraint.** “Only one job per day” is a `PRIMARY KEY (job_name, date)` and an insert that fails the second time. This is the correct answer to most “run this once” questions and it is one line.
- **Single owner by partition.** Route every request for a key to one machine — by consistent hashing, or by Kafka partition. If only one machine ever handles a key, there is nothing to lock.
- **Idempotency.** From [day 122](#): if running twice is harmless, you do not need to prevent it.

“I would not use a lock here, and here is what I would use instead” is a stronger answer than any lock implementation.

6 The numbers

How long may the work take, relative to the lease? This is the sizing question.

lease	30 s
watchdog refresh interval	10 s (lease / 3)
missed refreshes tolerated	2

p99 job duration	8 s
p999 job duration	45 s

With a 30-second lease and no watchdog, every job in the slowest 0.1% loses its lock mid-flight:

jobs per day	100,000
p999 exceeds the lease	0.1%
	100 jobs a day running without a lock

A hundred double-executions a day from a lock that “works”. With the watchdog, the lease only expires if the process is genuinely unresponsive for 30 seconds, which is a much rarer event.

How often does a GC pause exceed the lease? For a JVM service with a 4 GB heap:

typical young-gen pause	10-50 ms
full GC pause	1-3 s
pathological (swap, huge heap)	10-60 s

```

full GCs per day per instance ~ 20
p(pause > 30 s)                ~ 1 in 10,000 GCs
instances                      50
                               50 x 20 = 1,000 GCs/day
                               1,000 / 10,000 = ~0.1 events per day

```

Roughly one every ten days across the fleet. That sounds negligible until you multiply it by the cost of a double write to a financial ledger, which is why the answer is fencing rather than a bigger number.

What the lock costs in throughput. A lock serialises everything it protects:

```

work under the lock           50 ms
lock acquire + release        2 ms (two Redis round trips)
-----
                             52 ms per holder, serially
maximum throughput            1 / 0.052 = 19 operations per second

```

Nineteen per second, no matter how many machines you have. That is the real cost of a lock and it is the number people forget. If the requirement is a thousand per second, the answer is not a faster lock — it is partitioning, so that a thousand independent locks each do nineteen.

Contention makes it worse. With `n` waiters polling every 100 ms:

```

50 waiters, polling every 100 ms
                             500 Redis calls per second of pure waiting

```

Which is why ZooKeeper's watch-the-node-below is better than polling: one wake-up per release instead of `n` per second.

Lock service capacity.

```

Redis SET NX                  ~ 100,000 ops/s on one instance
each acquire = 1 SET + 1 EVAL 2 ops
                             50,000 lock cycles per second

```

```

ZooKeeper write               ~ 10,000-20,000 ops/s (quorum write, fsync)
each acquire = 1 create + watches
                             ~ 10,000 lock cycles per second

```

Redis is five times faster and ZooKeeper is safer. That is the trade in one line, and it is why systems often use both: ZooKeeper for the handful of leadership decisions, Redis for the high-volume “do not do this twice right now” cases where a duplicate is annoying rather than fatal.

Lease length against blocked time.

```
lease 30 s, holder crashes at t=1
resource unavailable for      29 s

lease 5 s, holder crashes at t=1
resource unavailable for      4 s
but jobs longer than 5 s lose the lock constantly
```

Short lease plus a watchdog gets you both: a crashed holder blocks for one lease length, and a live holder keeps extending. That combination is the answer to “how long should the lease be”, and the number is “short — 10 to 30 seconds — with a refresh at a third of it”.

7 The trade-offs

A lock is a serialisation point, and that is its main cost. Everything it protects runs one at a time. Fifty milliseconds of work under a lock caps you at about nineteen operations a second across the entire fleet, and adding machines does not help. Before accepting that, check whether the lock can be per-key rather than global — a thousand independent locks scale, one global lock does not.

Short lease versus long lease has no right answer. Short means a crashed holder blocks briefly and a slow holder is overtaken often. Long means the reverse. The watchdog moves the trade — refreshing while alive means you can afford a short lease — but a watchdog cannot refresh during the pause that is the actual danger.

Redis is fast and is not a correctness mechanism on its own. One `SET NX` per acquire, a hundred thousand a second, and a failure mode where two clients hold the lock. If the consequence of that is a duplicate email, take it. If it is a duplicate ledger entry, do not — either move to ZooKeeper or etcd, or add fencing, and preferably both.

ZooKeeper and etcd are safer and slower and are another thing to operate. Ten to twenty thousand writes a second because every write is a quorum write with a disk sync. A three or five node cluster to run, monitor and upgrade. Worth it for leadership and for anything where two holders corrupt data; overkill for “only one worker should refresh this cache”.

Fencing tokens require the resource to cooperate, and often it will not. A `WHERE fence < ?` clause is easy in your own database. It is impossible against a third-party API, a legacy device, or an S3 write. When the resource cannot check tokens, your options are to make the operation idempotent instead, to accept the risk explicitly, or — in the hard-

ware case — to fall back on STONITH and forcibly stop the old holder. **Saying “fencing tokens, and here is what I do when the resource cannot check them” is a much better answer than “fencing tokens”.**

A lock is only as available as the lock service. If Redis is down, nothing can acquire, and every worker stops. That is a new single point of failure introduced by the safety mechanism. Decide in advance whether you fail closed (stop working — correct for correctness locks) or fail open (proceed unlocked — acceptable for efficiency locks), and write it down, because otherwise whoever writes the error handler decides for you.

When would I not use a distributed lock at all? Whenever the write can enforce the constraint itself. A unique constraint on `(job_name, date)`, a conditional update on a version column, an atomic increment, or partitioning so one owner handles each key. All four are cheaper, have no expiry to tune, no lock service to run, and no two-holders failure mode. **Reach for a lock when you must coordinate work that the storage layer cannot express as a constraint** — and say that you looked for the constraint first.

8 In the interview

How it gets asked

- “Implement a distributed lock. What if the holder crashes?” — the direct version, and the second half is the real question.
- “You have a cron job on ten servers and it must run once. Design it.”
- “How do you make sure only one worker processes this queue item?”
- “What is wrong with `SET NX` as a lock?”
- “What is a fencing token?” — asked when the interviewer wants to skip to the end.
- “Redis or ZooKeeper for locking?” — a trade-off question with a real answer.

The first ninety seconds

“Let me build it up, because each version fixes the previous one’s failure and the last failure is the interesting one.

The naive lock is a key in a shared store, taken atomically — `SET lock value NX` in Redis, which succeeds only if the key does not exist. Atomic matters: a `GET` followed by a `SET` lets two clients both see it free.

That version deadlocks the moment a holder crashes, because the key is never deleted. So: a **lease** — `SET lock token NX PX 30000`, expiry in the same command, because setting the key and then expiring it separately leaves a window where a crash gives you a permanent lock.

Two more things I would get right at this level. The value is a **unique token per holder**, not a constant, and release checks it — otherwise a client whose lease expired can come back and delete somebody else’s lock. And that check-and-delete has to be atomic, so it is a small Lua script, not a `GET` then a `DEL`.

Then the failure that has no fix. If the holder is **not dead but paused** — a long garbage collection, a stalled disk — the lease expires while it is still working, someone else acquires, and the first holder wakes up believing it still holds the lock. Two writers. And I cannot tune my way out of it, because a longer lease means a crashed holder blocks longer and a shorter one gets overtaken more often. **A paused client and a dead client are indistinguishable from the outside.**

So the answer is not to prevent it but to make it harmless: **fencing tokens**. Every grant increments a counter, the holder carries that number on every write, and the *resource* rejects any write with a token lower than the highest it has seen. The stale holder’s write fails, and it learns it lost the lock from the rejection — which is the only channel that reaches a process that was unconscious.

And before all of that, I would ask whether a lock is needed. If the constraint can be expressed as a unique index or a conditional update, that is strictly better. Which is the case here?”

The follow-ups

“What exactly goes wrong if the holder pauses? Walk me through it.”

“Concrete timeline. Client A acquires at $t=0$ with a 30-second lease and starts work. At $t=5$ its runtime begins a garbage-collection pause that lasts 25 seconds — A is completely stopped, executing nothing, unable to observe anything.

At $t=30$ the lease expires in Redis. Redis has no way to know A is alive; from its side an expired lease and a dead client look identical. At $t=31$ client B acquires successfully and starts working.

At $t=30$ -something A resumes. From A’s point of view nothing has happened — it has no idea time passed, it believes it holds the lock, and it does the next thing in its function, which is a write. Two writers.

The thing I want to stress is that A did nothing wrong. It did not fail to renew out of carelessness; it was not running. Any check A performs before writing has the same problem — it can be paused between the check and the write.

That is why the enforcement has to be at the resource, not in the client. With fencing, A writes with token 33, the storage has already seen 34 from B, and the write is rejected. The corruption becomes a failed request, which A can handle.”

“Redis or ZooKeeper?”

“It depends on whether the lock is for efficiency or for correctness, and I would ask which before answering.

Efficiency lock — ‘it would be wasteful for two workers to rebuild this cache’ — Redis. A hundred thousand operations a second on one instance, two round trips per lock cycle, trivially operated. If it occasionally lets two workers run, the cost is some duplicated work. Take the speed.

Correctness lock — ‘two workers must never both write this ledger entry’ — ZooKeeper or etcd, and fencing tokens on top. They are slower, ten to twenty thousand writes a second because every write is a quorum write with a disk sync, and they are another cluster to run. What you get is a much better failure detector — ephemeral nodes disappear when the session ends, so a crashed holder releases without any expiry guesswork — and a sequence number that is a fencing token, for free.

The other ZooKeeper advantage worth naming: waiters watch the node immediately below them in the sequence rather than the lock itself, so a release wakes exactly one waiter instead of all fifty. With Redis I would be polling, and fifty waiters polling every hundred milliseconds is five hundred wasted calls a second.

On Redlock specifically: it removes Redis’s single point of failure by requiring a majority across independent instances. It does not remove the paused-client problem and it adds an assumption about bounded clock drift. I would use it for availability, not for correctness, and I would still fence.”

“The job sometimes takes longer than the lease. What do you do?”

“Not a longer lease — that trades one failure for another, because a 5-minute lease means a crashed holder blocks the resource for 5 minutes.

A **watchdog**: a background thread that re-extends the lease while the work is still running, typically at a third of the lease length, so a 30-second lease refreshes every 10 seconds and can survive two missed refreshes. The extension is itself conditional on still owning the lock — the same Lua check-then- `PEXPIRE` — so it cannot extend a lock that has already been taken by someone else. Redisson does this by default and calls it a watchdog.

The half people leave out is what happens when the extension fails. If the refresh returns ‘you no longer own this’, the worker must stop immediately and treat its work as abandoned. A watchdog that silently keeps trying gives you a worker that carries on after losing the lock, which is the exact scenario you were defending against.

And I would still fence, because the watchdog cannot refresh during the pause that is the actual danger.”

“Do you actually need a lock for a once-daily cron job across ten servers?”

“No, and I would push back on it. A unique constraint is strictly better.

Create a table with a primary key of `(job_name, run_date)`. Every server tries to insert a row for today at the scheduled time. Exactly one insert succeeds; the other nine get a constraint violation and exit. That is the whole implementation, it is one statement, and the database enforces it with the same mechanism that makes primary keys work.

Compared with a lock: no expiry to tune, no lease to refresh, no two-holders failure mode, no lock service to run or monitor, and a permanent record of what ran when, which is useful for debugging. The only thing it does not give me is mutual exclusion *during* the run — if the winner crashes halfway, nobody takes over. So the row also has a status and a heartbeat column, and a job that has been `RUNNING` with a stale heartbeat for an hour can be reclaimed by another server, which increments an attempt counter that doubles as a fencing token.

More generally: before reaching for a lock I look for a constraint the storage layer can enforce — a unique index, a conditional update on a version, an atomic increment, or partitioning so one owner handles each key. A lock is what I use when the coordination cannot be expressed as a constraint on the data.”

The model answer

“Ten worker machines pull jobs from a queue. A job must be processed exactly once. Design it, and tell me what happens when a worker dies mid-job.”

“Let me start by trying not to use a lock, because for this shape there is usually something better, and then say where a lock is genuinely needed.

First choice: let the queue do it. SQS, Kafka with consumer groups, or Rabbit with acknowledgements all give me at-least-once delivery with an invisibility window — the message is hidden from other consumers for a visibility timeout, and if I do not acknowledge within it, it comes back. That is a lease, implemented by people who have already thought about this, and I get it without running a lock service.

So the real question becomes ‘exactly once’, and I would say plainly that exactly-once delivery does not exist. What I can build is at-least-once delivery plus idempotent processing, which gives exactly-once effect — the day-122 answer. Each job carries a stable ID; before processing I do a conditional insert on that ID, and if it conflicts, the job is already done and I acknowledge and move on. **That removes the need for mutual exclusion entirely**, because a second delivery is harmless rather than forbidden.

Now the case where I do need a lock. Suppose the job’s work is genuinely not idempotent — it calls a third-party API that has no idempotency key, say. Then I need mutual exclusion, and here is the design.

Lock key per job ID, in Redis, acquired with `SET lock:job:{id} {uuid} NX PX 30000`. Release via a Lua script that checks the token. A watchdog extending at 10 seconds, which stops the worker if the extension fails. Lease deliberately short, because the watchdog covers long jobs and a short lease means a crashed worker’s job is retried in 30 seconds rather than 5 minutes.

Fencing token from `INCR lock:job:{id}:fence`, carried into the job’s own state row, with the update conditional on `fence < :token`. If the third-party call cannot take a token, I record the attempt with its token *before* making the call, so a stale holder’s attempt row insert fails and it stops before calling. That is weaker — there is a window between the insert and the call — and I would name that weakness rather than pretend the design is airtight.

When a worker dies mid-job: the queue’s visibility timeout expires and the message is redelivered; the Redis lease expires and another worker acquires with a higher token; the job restarts from the beginning. That means partial work must be either idempotent or transactional — if the job wrote three of five records before dying, the retry must handle finding them. I would design the job’s writes as upserts keyed by job ID so that a partial run followed by a full run is indistinguishable from one full run.

The number I would put on it. Lock acquire and release is two Redis round trips, about 2 milliseconds, against a job that takes 500 milliseconds — so 0.4% overhead, fine. But the lock is per job ID, not global: ten thousand distinct jobs means ten thousand independent locks and the workers do not contend at all. **If I had used one global lock I would be capped at about two jobs a second regardless of how many workers I ran**, and that is the mistake I would be watching for in someone else's design.

And the operational decision I would write down: if Redis is unavailable, workers stop rather than process unlocked. For a non-idempotent third-party call, a pause is much cheaper than a duplicate. If the work were idempotent I would fail open instead — and the fact that the answer differs is the reason to make the operation idempotent in the first place.”

9 Recall card

Acquire atomically with a lease in one command: `SET key <unique-token> NX PX 30000`. Separate `SET` then `EXPIRE` leaves a permanent lock on a crash; `GET` then `SET` is not a lock at all.

Release must check ownership, atomically — a Lua script, not `GET` then `DEL` — or an expired holder deletes somebody else's lock.

The unfixable failure: a paused holder. The lease expires, another client acquires, the first wakes up still believing it holds the lock. You cannot distinguish paused from dead, and no timeout setting avoids both failure directions.

So fence instead of preventing. Monotonic token per grant, carried on every write, and **the resource** rejects anything below the highest token it has seen. A wasted handover instead of corrupted data.

Redis for efficiency locks (fast, ~100k ops/s, can double-grant); ZooKeeper or etcd for correctness locks (ephemeral nodes release on crash, sequence numbers are free fencing tokens). And before any of it, look for a unique constraint, a conditional update, or a single owner per key — a lock is what you use when the storage layer cannot express the rule.

DSA topic: Breadth-first search on a graph **System design topic:** Distributed locks

Code these, in this order

One rule for the whole set: **write** `seen.add(neighbour)` **on the line above** `queue.append(neighbour)` , **always, in that order**. Then say the sentence “each vertex enters the queue at most once” out loud. If your code does not make that sentence true, it is wrong, whatever answer it produces.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Find if Path Exists in Graph	LeetCode 1971 (Medium)	Plain reachability. Ten lines, and <code>deque</code> not a list.
2	Shortest Path in Binary Matrix	LeetCode 1091 (Medium)	BFS on an implicit grid, with eight directions instead of four.
3	Word Ladder	LeetCode 127 (Hard)	The neighbour function, and why building edges is the trap.
4	Rotting Oranges	LeetCode 994 (Medium)	Level-by-level, and the check for what never gets reached.
5	Open the Lock	LeetCode 752 (Medium)	An implicit graph of states, plus a set of forbidden vertices.

On problem 1, write both versions and time them

Solve it once with `deque.popleft()` and once with `list.pop(0)` . Run both on 100,000 vertices. Record the two times. This is the cheapest possible way to make `deque` automatic.

On problem 3, count the two models

Version A: compare every word against every other to build edges. Version B: generate the `L × 25` candidate strings and check a set. On the LeetCode input, count the operations each version performs before the search starts. Two numbers.

Then implement bidirectional BFS and record the number of words each version explores.

On problem 4, notice what “impossible” means

The problem asks for `-1` when some orange can never rot. Say out loud which BFS concept that is, and where in your code the check goes. It is not inside the loop.

Then the marking experiment

Take your problem 1 solution and move `seen.add()` from the push to the pop. Run it on a dense graph — 2,000 vertices, 500,000 edges. Record:

1. Whether the answer is still correct.
2. The maximum queue length in each version.
3. The runtime of each version.

Three pairs of numbers. This is the single most useful measurement in the whole graph phase.

The shape drill

1. Write plain BFS from memory in under two minutes.
2. Write the distances version, using one dictionary and not two.
3. Write the path version and the parent walk.
4. Write the level-by-level version with two lists and no queue.
5. Write the components version, and point at the line that makes it different.

The guarantee drill

1. State the shortest-path guarantee in one sentence.
2. Give the one-sentence reason it holds.
3. Give the full induction argument out loud.
4. State the assumption it depends on, and give a graph where it fails.
5. Say what you use instead when that assumption fails, and what you use when the weights are only 0 and 1.

The push-versus-pop drill

1. Say what changes if you mark on pop.
2. Say what stays correct and what breaks.
3. Give the queue size in each version, in terms of `V` and `E`.
4. Say which sentence stops being true.

The costs drill

1. Derive $O(V + E)$ from the loops, out loud.
2. Say why it is a sum and not a product.
3. Give the maximum queue size for a star graph and for a path graph, both with a million vertices.

4. Give BFS's cost on an adjacency matrix and say where the V^2 comes from.
5. Give V and E for a $1,000 \times 1,000$ grid.

The break-it drill

Trigger each and record the exact output or error:

1. `list.pop(0)` on 200,000 vertices.
2. No `seen` set, on a three-vertex cycle.
3. `seen.add()` moved to the pop, on a graph with 500,000 edges.
4. `distances[goal]` on an unreachable goal.
5. A single BFS used to count all vertices on a disconnected graph.
6. BFS on a weighted graph where a one-edge path costs 10 and a two-edge path costs 2.

The build-it-up drill

State each version and the failure that forces the next one:

1. `GET` then `SET`.
2. `SET NX` with no expiry.
3. `SET NX` then `EXPIRE`.
4. `SET NX PX` with a constant value.
5. `SET NX PX` with a unique token, released by `GET` then `DEL`.
6. The correct version.

Six steps. Say the failure at each one before giving the fix.

The paused-holder drill

1. Give the timeline with times on it.
2. Say why client A is not at fault.
3. Say why no timeout setting avoids both failure directions.
4. Say why a check inside client A does not help.
5. Say where the enforcement has to live, and why there.

The fencing drill

1. Explain a fencing token in three sentences.
2. Write the SQL `WHERE` clause that enforces it.
3. Say what zero rows updated means.
4. Say what you do when the resource cannot check tokens — three options.

5. Say what ZooKeeper gives you for free here.

The watchdog drill

1. Say why a longer lease is the wrong fix for a long job.
2. Give the refresh interval as a fraction of the lease, and why that fraction.
3. Say what the extension must be conditional on.
4. Say what the worker must do when the extension fails.
5. Say what the watchdog cannot protect against.

The choose-your-store drill

1. Define an efficiency lock and a correctness lock, with an example each.
2. Give Redis's throughput and its failure mode.
3. Give ZooKeeper's throughput and the three properties ephemeral sequential nodes buy.
4. Say why watching the node below you beats polling, with the arithmetic.
5. Say what Redlock fixes and what it does not.

The numbers drill

1. Compute maximum throughput under a lock with 50 ms of work.
2. Say why adding machines does not help, and what does.
3. Compute wasted Redis calls for 50 waiters polling every 100 ms.
4. Compute how many jobs a day exceed a 30-second lease at a p999 of 45 s and 100,000 jobs.
5. Estimate the frequency of a GC pause longer than the lease across 50 instances.

The do-not-lock drill

1. Name four alternatives to a distributed lock.
2. For the daily cron job, give the full non-lock design including takeover.
3. Say what the unique-constraint version gives you that the lock does not.
4. Say what it does not give you, and how you add it back.

The failure drill

For each, say what happens and what you would build:

1. A holder crashes with no lease set.
2. A holder's lease expires during a 25-second GC pause.
3. A client releases with `GET` then `DEL` after its lease expired.
4. A job takes 45 seconds under a 30-second lease with no watchdog.

5. A watchdog whose extension fails, and a worker that does not check.
6. Redis becomes unavailable while ten workers hold locks.
7. One global lock protecting ten thousand independent jobs.

Two of the seven are throughput problems and five are correctness problems. Sort them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Traverse this graph breadth-first, and find the shortest path.* Queue plus seen set, mark on push with the reason, one dictionary doing three jobs, the parent walk, the $O(V + E)$ derivation, and the unweighted assumption named before anyone asks.
2. *Find the shortest word transformation sequence.* Vertices and edges as a sentence, the implicit-graph move with the $L \times 25$ arithmetic, why building edges is n^2 , the two edge cases, and bidirectional BFS as the improvement.
3. *Implement a distributed lock. What if the holder crashes?* The six versions in order, each fixing the last, then the paused holder as the failure with no fix, then fencing at the resource — and “would a unique constraint do instead?” offered first.

Before you move on

- ☐ I can write BFS from memory in under two minutes.
- ☐ I mark on push and can say why in one sentence.
- ☐ I know what marking on pop does to the queue size.
- ☐ I use `deque` and know what `list.pop(0)` costs.
- ☐ I can state the shortest-path guarantee and give the reason.
- ☐ I can give the induction argument out loud.
- ☐ I know the assumption it depends on and what to use when it fails.
- ☐ I know what to use when weights are only 0 and 1.
- ☐ I can derive $O(V + E)$ from the loops.
- ☐ I know the queue's peak is the widest level, not V .
- ☐ I can write the distances version with one dictionary.
- ☐ I can reconstruct a path from parent pointers.
- ☐ I write the outer loop whenever the question says “all”.
- ☐ I know why an unreachable lookup raises `KeyError`.

- [] I recognise an implicit graph and compute neighbours instead of storing them.
- [] I can explain bidirectional BFS and what it buys.
- [] I can give the six lock versions and the failure at each.
- [] I know why the expiry must be in the same command as the set.
- [] I know why the lock value is a unique token.
- [] I know why release must be a script.
- [] I can give the paused-holder timeline with times.
- [] I know why no timeout setting fixes it.
- [] I can explain fencing and write the `WHERE` clause.
- [] I know what to do when the resource cannot check tokens.
- [] I can size a lease and a watchdog refresh.
- [] I know what the worker does when the extension fails.
- [] I can distinguish efficiency and correctness locks and pick a store for each.
- [] I can compute throughput under a lock and say what actually fixes it.
- [] I look for a unique constraint before reaching for a lock.
- [] I answered all three questions above out loud.

Depth-first search on a graph

DATA STRUCTURES AND ALGORITHMS

Depth-first search on a graph

SYSTEM DESIGN

Distributed systems revision and interview questions

1 What this is, and why they ask it

Depth-first search follows one path as far as it goes, and only when it can go no further does it back up and try the next option. Where **BFS** spreads out in rings, DFS commits to a direction and sees it through.

The code is shorter than BFS — five lines, recursive — and the shape is one you already know from [day 100](#), where it walked a tree. As with BFS, the whole difference on a graph is the `seen` set, because a graph has cycles.

They ask DFS because a specific family of problems has no good BFS answer. Cycle detection needs to know which vertices are on the *current path*, and only DFS has a current path. Topological sort falls out of the order DFS finishes vertices in. Connected components, bridges, articulation points, strongly connected components — all DFS. **BFS answers “how far”; DFS answers “what is the structure”**. Knowing which question you have is most of what is being tested.

There is also a practical trap they enjoy: the recursive version is elegant and blows the stack on a graph with a hundred thousand vertices in a line. Being able to write the iterative version, and knowing when you need it, separates people who have used this from people who have read about it.

By the end of this lesson you can write DFS recursively and iteratively, say precisely when each is right, handle a disconnected graph, explain the three colours that make cycle detection work, and name the two problems where DFS is the only reasonable answer.

2 The story

The wedding is on Sunday and the chain is not where it should be.

Sushila has been standing in front of the steel almirah in the back bedroom since half past nine. She knows the chain is in this room. She put it away herself after her niece’s engagement, and she remembers thinking at the time that she would not forget where.

She starts at the top shelf, on the left. There is a stack of folded sarees, and behind them a tin — the old Britannia one — and she pulls the tin out and opens it. Inside the tin there are four small cloth pouches. She opens the first pouch. Bangles. Second pouch: two rings and a broken earring. Third: buttons, for some reason. Fourth: empty.

She puts the four pouches back in the tin, in the order she took them out, and the tin back behind the sarees, and only then does she move her hand to the next thing on the top shelf.

This is how she does it, and it is why she is slow and why she will find it. She does not look in the tin, then get distracted and look in a box on the middle shelf, then come back. She finishes the tin. Completely. Every pouch inside it, and every fold of cloth inside every pouch, before the tin goes back.

Her daughter-in-law, who came in at ten to help, does not work this way at all and it is driving Sushila slightly mad. The girl opens the tin, looks in one pouch, then opens a plastic box on the middle shelf, then goes to the drawer, then comes back to the tin and cannot remember whether she checked the second pouch or not.

By eleven Sushila is on the middle shelf. Two boxes down, and inside the second box, under a folded shawl, there is a smaller box she does not recognise. She opens it. Inside is a paper packet, and inside the packet, in cotton wool, is a set of her mother's bangles that she has not seen for nine years and had assumed were gone.

She sits down on the bed with them for a while.

Then she puts the cotton wool back, and the packet in the small box, and the small box under the shawl, and the shawl in the big box, and the big box back on the middle shelf. And she carries on with the third box, because she has not finished the middle shelf.

She finds the chain at twenty to twelve, in the bottom drawer, in a bank envelope, exactly where a sensible person would have put it.

3 The idea in plain English

Sushila's search is a depth-first search, and her daughter-in-law's is what happens without the discipline.

Go deep before you go wide. The tin is on the top shelf, and inside the tin are four pouches. DFS opens the tin and finishes *everything inside it* before looking at the next item on the shelf. BFS would have looked at every item on the top shelf first, then everything one level inside each of them, and so on. Same almirah, different order.

The call stack is the “where I am” record. When Sushila opens the tin, then the pouch, she is three levels deep: shelf, tin, pouch. In a recursive program that is three stack frames. Each one remembers what it was in the middle of, so that when the pouch is finished, the tin knows which pouch is next. **You never write that bookkeeping; the recursion does it.** You met this on [day 88](#).

Backing out is the return. Putting the pouches back and the tin back is the function returning. This is the part people underestimate: in DFS, *finishing* a vertex is a real event, distinct from *starting* it, and several important algorithms depend on the difference.

The **seen** set is “I already looked there”. The daughter-in-law’s problem is that she has no record, so she re-opens the tin. In a graph with cycles, no record means an infinite loop. **Every graph DFS needs a seen set** — the same non-negotiable rule as BFS, and for the same reason.

Now the property that makes DFS different from BFS, and it is not about speed. Both are $O(V + E)$. Both visit every reachable vertex exactly once. What differs is *what you know while you are running*.

At any moment during a DFS, there is a current path — shelf → tin → pouch — and you know exactly which vertices are on it. BFS has no such thing; its queue holds a scattered ring of vertices with no relationship to each other.

That current path is the whole reason DFS exists as a separate tool:

- **Cycle detection.** A cycle is an edge that points back to a vertex on the *current path*. If you reach a vertex you have seen before but which is *finished*, that is not a cycle — it is just a different route to somewhere you already explored. Only DFS can tell those two cases apart, and the machinery for doing it is the three colours below.
- **Topological sort.** The order in which DFS *finishes* vertices, reversed, is a valid dependency order. That falls out for free, and it is [day 134](#).
- **Path-shaped questions.** “Is there a path with this property”, “find all paths”, “what is the longest path in a tree” — all need the current path, all are DFS.

The three colours. For directed graphs you need three states per vertex, not two:

- **White** — not visited yet.
- **Grey** — visiting: started, not finished, **on the current path right now**.
- **Black** — finished: it and everything below it are done.

An edge to a **grey** vertex is a cycle, because grey means “I am inside this call, above you on the path”. An edge to a **black** vertex is fine. **Two states cannot distinguish these**, and that is exactly the bug in most first attempts at directed cycle detection. It is [day 133](#), and today you meet the vocabulary.

Recursion is elegant and has a hard limit. Python allows about a thousand nested calls by default. A graph that is a straight line of a hundred thousand vertices needs a hundred thousand frames. The iterative version with an explicit stack has no such limit, and **on any graph where V could exceed a few thousand, you write the iterative one**.

And DFS gives no shortest-path guarantee. It finds a path, often a long and silly one. If the question says “fewest”, it is BFS. If the question says “is there” or “what is the structure”, it is DFS.

4 The picture

The same graph, walked both ways, so the difference is visible:

graph: A -- B A -- C B -- D C -- E D -- F

A
/ \
B C
| |
D E
|
F

DFS from A (taking B before C):

A
+- B
| +- D
| +- F
+- C
 +- E

visit order: A B D F C E
goes to the BOTTOM of the
B branch before touching C

BFS from A:

A
+- B C ← ring 1
+- D E ← ring 2
+- F ← ring 3

visit order: A B C D E F
finishes each ring
before starting the next

What to notice. DFS reaches **F**, which is three steps away, before it touches **C**, which is one step away. That is not a flaw — it is the definition. If the question is “how far is C”, DFS is the wrong tool.

The three colours, on a directed graph with a cycle:

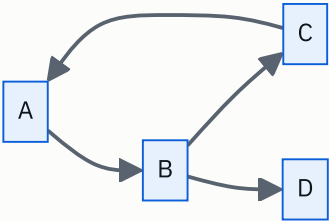


FIGURE 128.1

step	action	A	B	C	D	current path
1	enter A	grey	white	white	white	A
2	enter B	grey	grey	white	white	A B
3	enter C	grey	grey	grey	white	A B C
4	C looks at A	grey	grey	grey	white	A B C
	A is GREY → CYCLE FOUND: A is above us on the path					

What to notice at step 4. A has been seen, and that alone tells you nothing. What matters is that A is **grey** — the call to A has started and not returned, so A is on the path from the root to here, and an edge back to it closes a loop. If A had been black, C → A would just be a second route into a finished region.

And the case that a two-state check gets wrong:

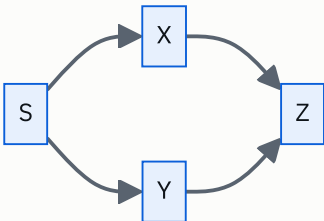


FIGURE 128.2

DFS from S: S, X, Z (finish Z, finish X), then Y, and Y looks at Z.

Z has been SEEN. A two-state check says "cycle!".
 Z is BLACK. Three states say "finished region, not a cycle".

There is no cycle in this graph. The two-state version is wrong.

What to notice. This shape — two paths converging on the same vertex — is completely ordinary. Any directed graph with a shared dependency has it. The two-state cycle detector reports a cycle on a perfectly valid dependency graph, which means “these courses cannot be completed” for a course list that is fine.

5 The code, built step by step

The recursive version first, because it is the one you should be able to write in thirty seconds.

```
def dfs(graph: dict[int, list[int]], start: int) → set[int]:
    seen: set[int] = set()

    def visit(vertex: int) → None:
        seen.add(vertex)                # mark on ENTRY
        for neighbour in graph[vertex]:
            if neighbour not in seen:
                visit(neighbour)

    visit(start)
    return seen
```

Five lines of actual work. Mark on entry — not after the loop — or a cycle sends you round again before the mark ever happens.

Note the difference from BFS: there the mark went next to the *push*, here it goes at the top of the *call*. Both mean the same thing: record it the instant you commit to it.

Now the iterative version, which is what you write when `V` might be large.

```
def dfs_iterative(graph: dict[int, list[int]], start: int) → list[int]:
    seen = {start}
    stack = [start]
    order = []
    while stack:
        vertex = stack.pop()            # pop() from the END: a stack
        order.append(vertex)
        for neighbour in reversed(graph[vertex]):
            if neighbour not in seen:
                seen.add(neighbour)
                stack.append(neighbour)
    return order
```

This is BFS with one character changed. `queue.popleft()` becomes `stack.pop()`, and first-in-first-out becomes last-in-first-out. That is the entire difference between the two algorithms, and it is worth saying out loud in an interview because it shows you see the shared shape.

`reversed()` is cosmetic: pushing neighbours in reverse means they come off the stack in the original order, so the visit order matches the recursive version. Drop it and the traversal is still correct, just mirrored.

But this version cannot tell you when a vertex finishes. It visits in something close to DFS order and never runs any “on the way back out” code. For cycle detection and topological sort you need the finish event, which means a slightly different iterative shape:

```
def dfs_with_finish(graph: dict[int, list[int]], start: int) → list[int]:
    """Iterative DFS that also reports finish order, using explicit markers."""
    seen: set[int] = set()
    finished: list[int] = []
    stack: list[tuple[int, bool]] = [(start, False)]
    while stack:
        vertex, done = stack.pop()
        if done:
            finished.append(vertex)          # second visit: everything below is finished
            continue
        if vertex in seen:
            continue
        seen.add(vertex)
        stack.append((vertex, True))        # push the "finish me later" marker FIRST
        for neighbour in reversed(graph[vertex]):
            if neighbour not in seen:
                stack.append((neighbour, False))
    return finished
```

The trick is the boolean flag. Each vertex is pushed twice: once to be entered, and once — underneath all its children — to be finished. When the marker comes back off the stack, everything below that vertex has already been processed. **This is the standard way to convert any recursive post-order into an iterative one**, and it is worth learning as a pattern rather than as a one-off.

Now the three colours, for directed cycle detection:

```
WHITE, GREY, BLACK = 0, 1, 2

def has_cycle(graph: dict[int, list[int]], n: int) → bool:
    colour = [WHITE] * n

    def visit(vertex: int) → bool:
        colour[vertex] = GREY                # on the current path
        for neighbour in graph[vertex]:
            if colour[neighbour] == GREY:
                return True                  # back edge: a cycle
            if colour[neighbour] == WHITE and visit(neighbour):
                return True
        colour[vertex] = BLACK              # done, and everything below is done
        return False

    return any(colour[v] == WHITE and visit(v) for v in range(n))
```

Read the two colour checks. **GREY** means “this vertex is an ancestor on the path I am currently on”, so an edge to it closes a loop. **BLACK** is skipped silently — it is a finished region, and re-entering it would only waste time.

The `any(...)` at the end is the outer loop over all vertices. **A directed graph can easily have a cycle that is unreachable from vertex 0**, so starting from one vertex is not enough. This is the disconnected-graph mistake in its most damaging form.

And the undirected version, which needs a different check entirely:

```
def has_cycle_undirected(graph: dict[int, list[int]], n: int) → bool:
    seen = [False] * n

    def visit(vertex: int, parent: int) → bool:
        seen[vertex] = True
        for neighbour in graph[vertex]:
            if not seen[neighbour]:
                if visit(neighbour, vertex):
                    return True
            elif neighbour ≠ parent:          # seen, and not where we came from
                return True
        return False

    return any(not seen[v] and visit(v, -1) for v in range(n))
```

In an undirected graph every edge looks like a back edge, because `A-B` puts `A` in `B`'s adjacency list and vice versa. So `B` looking at `A` and finding it seen is not a cycle — it is the edge you just walked. The `parent` parameter excludes exactly that one case. No colours needed, because there is no direction to be confused about. This is [day 132](#).

The complete solution

```
"""Depth-first search on a graph: recursive, iterative, components, and cycles."""

from __future__ import annotations

import sys
from collections import defaultdict

WHITE, GREY, BLACK = 0, 1, 2

def build(edges: List[tuple[int, int]], n: int, directed: bool = False) → dict[int, list[int]]:
    graph: dict[int, list[int]] = {v: [] for v in range(n)}
    for a, b in edges:
        graph[a].append(b)
        if not directed:
            graph[b].append(a)
    return graph

def dfs_recursive(graph: dict[int, list[int]], start: int) → list[int]:
    """Visit order. Elegant; limited by Python's recursion depth (~1000)."""
    seen: set[int] = set()
    order: list[int] = []

    def visit(vertex: int) → None:
        seen.add(vertex)
        order.append(vertex)
        for neighbour in graph[vertex]:
            if neighbour not in seen:
                visit(neighbour)

    visit(start)
    return order

def dfs_iterative(graph: dict[int, list[int]], start: int) → list[int]:
    """Same traversal, no recursion limit. BFS with a stack instead of a queue."""
    seen = {start}
    stack = [start]
    order: list[int] = []
    while stack:
        vertex = stack.pop()
        order.append(vertex)
        for neighbour in reversed(graph[vertex]):
            if neighbour not in seen:
                seen.add(neighbour)
                stack.append(neighbour)
    return order

def components(graph: dict[int, list[int]]) → list[list[int]]:
    """Every piece. The outer loop is the whole point."""
    seen: set[int] = set()
    pieces: list[list[int]] = []
    for start in graph:
        if start in seen:
            continue
        piece, stack = [], [start]
        seen.add(start)
        while stack:
            vertex = stack.pop()
            piece.append(vertex)
            for neighbour in graph[vertex]:
                if neighbour not in seen:
                    seen.add(neighbour)
                    stack.append(neighbour)
        pieces.append(sorted(piece))
    return pieces
```



```

def has_cycle_directed(graph: dict[int, list[int]], n: int) → bool:
    """Three colours. GREY means 'on the current path', which is the cycle test."""
    colour = [WHITE] * n

    def visit(vertex: int) → bool:
        colour[vertex] = GREY
        for neighbour in graph[vertex]:
            if colour[neighbour] == GREY:
                return True
            if colour[neighbour] == WHITE and visit(neighbour):
                return True
        colour[vertex] = BLACK
        return False

    return any(colour[v] == WHITE and visit(v) for v in range(n))

if __name__ == "__main__":
    undirected = build([(0, 1), (0, 2), (1, 3), (2, 4), (3, 5), (6, 7)], 8)
    print("recursive :", dfs_recursive(undirected, 0))
    print("iterative :", dfs_iterative(undirected, 0))
    print("components:", components(undirected))

    acyclic = build([(0, 1), (0, 2), (1, 3), (2, 3)], 4, directed=True)
    cyclic = build([(0, 1), (1, 2), (2, 0), (1, 3)], 4, directed=True)
    print("diamond has cycle?", has_cycle_directed(acyclic, 4))
    print("triangle has cycle?", has_cycle_directed(cyclic, 4))

    # The recursion limit, demonstrated.
    line = build([(i, i + 1) for i in range(5000)], 5001)
    print("iterative on a 5,000-long chain:", len(dfs_iterative(line, 0)))
    try:
        dfs_recursive(line, 0)
    except RecursionError as error:
        print("recursive on the same chain:", type(error).__name__, "-", error)

```

Running it:

```

recursive : [0, 1, 3, 5, 2, 4]
iterative : [0, 1, 3, 5, 2, 4]
components: [[0, 1, 2, 3, 4, 5], [6, 7]]
diamond has cycle? False
triangle has cycle? True
iterative on a 5,000-long chain: 5001
recursive on the same chain: RecursionError - maximum recursion depth exceeded

```

Three things to look at. The first two lines are identical, which is the point of `reversed()`. The diamond — `0→1→3` and `0→2→3` — correctly reports **no** cycle, which a two-state check would get wrong. And the last two lines are the reason the iterative version exists: the same graph, one function works and one does not.

6 What it costs

Time is identical to BFS, and for the same reason.

```
each vertex is entered once
for each, scan its neighbours
sum of degrees
```

```
V
degree(v)
2E undirected, E directed
-----
V + 2E → O(V + E)
```

DFS and BFS have exactly the same complexity. They differ in the *order* of the visits and in *what information is available* during the run, not in the amount of work. If you ever say one is faster than the other, you have said something false.

Space is where they differ in character, though not in the bound.

```
seen set          O(V)
call stack (or explicit stack)  O(depth of the deepest path)
```

Both are $O(V)$ in the worst case, but the shapes that hit the worst case are opposite:

	BFS peak	DFS peak
star graph (one hub, many leaves)	1,000,000 (ring 1)	2
path graph (a straight line)	2	1,000,000 (the whole chain)

BFS is expensive on wide graphs; DFS is expensive on deep ones. That is a genuinely useful thing to say when asked which to use, and it is the one memory difference between them.

The recursion limit, with numbers.

```
Python's default recursion limit    1,000
frames already used before your call ~ 20-40
usable depth                        ~ 960

a graph that is a chain of 5,000 vertices needs 5,000 frames
                                   → RecursionError
```

Raising the limit is possible and is usually the wrong answer:

```
sys.setrecursionlimit(200_000)
```

Each Python frame is roughly 500 bytes of interpreter state plus C stack, and the C stack is typically 8 MB by default:

```
200,000 frames x ~1 KB of C stack    = 200 MB
                                   → Segmentation fault (not a Python error)
```

A segfault is worse than a `RecursionError` — no traceback, no message, nothing to debug. So: use the iterative version when `V` might exceed a few thousand, and mention that you know `setrecursionlimit` exists and why you did not use it.

On a grid, which is the most common implicit graph:

```
1,000 x 1,000 grid of all-passable cells
V = 1,000,000
worst-case DFS depth = 1,000,000 (a snaking path through every cell)
    → recursion is impossible; iterative is mandatory
```

This is the exact case that fails on LeetCode's larger island problems, and it fails as a segfault or a stack overflow rather than as anything readable.

The extra cost of the finish-order version.

```
plain iterative DFS      V pushes
with finish markers      2V pushes (each vertex pushed twice)
    → same O(V + E), 2x the stack traffic
```

Twice the pushes, same complexity. Worth it only when you actually need the finish event — cycle detection, topological sort, strongly connected components.

7 The traps

`RecursionError` on a deep graph

The most common real failure:

```
Traceback (most recent call last):
  File "dfs.py", line 21, in visit
    visit(neighbour)
  File "dfs.py", line 21, in visit
    visit(neighbour)
[Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

It does not need an unusual graph. A linked-list-shaped graph, a long corridor in a grid, a chain of package dependencies — all reach a thousand easily. **The tell in the constraints is $n \leq 10^5$** . If you see that, write the iterative version.

And do not “fix” it with `sys.setrecursionlimit(10**6)`:

```
Segmentation fault (core dumped)
```

That is what happens when Python's limit is raised past what the C stack can hold. You have replaced a clear error with an unclear crash.

Marking after the loop instead of on entry

```
def visit(vertex):
    for neighbour in graph[vertex]:
        if neighbour not in seen:
            visit(neighbour)
    seen.add(vertex)          # ← too late
```

On any graph with a cycle, A calls B calls A — and A is not in `seen` yet, because it is still inside its loop:

```
RecursionError: maximum recursion depth exceeded
```

Mark on entry, first line of the function. The same rule as BFS's “mark on push”.

Two states instead of three, on a directed graph

```
def visit(vertex):
    seen.add(vertex)
    for neighbour in graph[vertex]:
        if neighbour in seen:
            return True          # "cycle"
        if visit(neighbour):
            return True
    return False
```

On the diamond `0→1→3`, `0→2→3`:

```
>>> has_cycle_two_state(diamond, 4)
True
```

There is no cycle. 3 was reached from 1, finished, and then 2 looked at it and found it seen. **Seen does not mean on the current path.** You need grey and black, or an explicit “currently on the stack” set alongside `seen`.

Forgetting the parent check on an undirected graph

```
def visit(vertex):
    seen[vertex] = True
    for neighbour in graph[vertex]:
        if seen[neighbour]:
            return True          # "cycle"
    ...
```

```
>>> has_cycle_undirected_broken({0: [1], 1: [0]}, 2)
True
```

Two vertices and one edge, and it reports a cycle. 0 walks to 1; 1 looks at 0 and finds it seen. But that is the same edge, walked backwards. **In an undirected graph every edge appears in both adjacency lists**, so you must exclude the vertex you came from.

The subtlety: excluding by *vertex* is wrong if the graph can have two parallel edges between the same pair, because those genuinely are a cycle. If parallel edges are possible, exclude by *edge* instead.

Only searching from one start

```
if has_cycle_from(graph, 0):
    ...
```

A directed graph can have a cycle in a region unreachable from vertex 0. On a course-schedule problem where courses 5, 6 and 7 form a loop that nothing else points into, this reports no cycle and the answer is wrong. **Wrap it in a loop over every vertex, with seen shared across the iterations.**

Assuming DFS finds the shortest path

```
path = dfs_path(graph, "A", "F")
print(len(path))                    # "the shortest distance"
```

It is a path. On the story's graph, DFS from A might reach F via B → D → F or, with a different adjacency order, wander through half the graph first. There is no guarantee and no partial one. **"Fewest" is BFS. Always.**

8 In the interview

How it gets asked

- "Traverse this graph depth-first. What if it is disconnected?" — the direct version, and the second half is what is actually being tested.
- "Does this directed graph contain a cycle?" — the three-colours question.
- "Count the connected components / count the islands." — either traversal works; DFS is shorter.
- "Find all paths from A to B." — DFS, because it needs the current path.

- “Your solution works on the small cases and crashes on the big one.” — recursion depth.
- “BFS or DFS here, and why?” — the question that wants the one-line rule.

The first ninety seconds

“DFS: go as deep as possible down one branch before backing up. Recursively it is five lines — mark on entry, then for each unseen neighbour, recurse. The mark has to be the first line of the function, not after the loop, or a cycle re-enters before the mark happens.

Two things I would say before writing it.

First, **it needs an outer loop over every vertex**, because the graph is probably not connected and a single start only finds its own component. For a directed graph this matters even more — a cycle can sit in a region nothing points into, so starting from vertex 0 can report ‘no cycle’ on a graph that has one. The `seen` set lives outside the loop so a vertex found in one component never starts a new one.

Second, **I would write it iteratively if `n` can be large**. Python’s recursion limit is about a thousand usable frames, and a graph that is a straight line of a hundred thousand vertices needs a hundred thousand. Raising the limit trades a `RecursionError` for a segmentation fault, which is worse because there is no traceback. The iterative version is BFS with `stack.pop()` instead of `queue.popleft()` — genuinely one character of difference — so it costs nothing to write.

Cost is $O(V + E)$ time and $O(V)$ space, exactly the same as BFS. **They differ in what you know while running, not in how much work they do**. DFS has a current path, which is what cycle detection and topological sort need. BFS has rings, which is what shortest path needs.

Which is it here — do you want reachability, or is there something about the structure you need?”

The follow-ups

“BFS or DFS? Give me the rule.”

“One line: ‘fewest’ or ‘shortest’ means BFS; ‘is there’ or anything about structure means DFS.

The reason is what each one has available. BFS finishes ring k before ring $k+1$, so the first time it reaches a vertex it is by the fewest edges — that guarantee is the only reason to prefer it, and it only holds when every edge costs the same.

DFS has a current path — the chain of calls from the root to where I am now — and BFS has nothing equivalent, because its queue holds a scattered ring. Cycle detection needs to ask ‘is this vertex on my current path’. Topological sort needs to know when a vertex *finishes*, which only DFS has a notion of. Finding all paths needs the path.

If neither property matters — ‘count the components’, ‘is B reachable from A’ — either works and I would pick DFS because it is shorter, unless the graph might be deep, in which case BFS avoids the stack problem without any extra code.

And on memory, they fail on opposite shapes: BFS’s queue holds the widest level, so a hub with a million neighbours is expensive; DFS’s stack holds the longest path, so a million-vertex chain is expensive.”

“Why three colours? Two seems enough.”

“Because ‘seen’ answers the wrong question. What I need to know is not ‘have I been here’ but ‘am I here right now, higher up on this same path’ — because a cycle is an edge that points back at an ancestor.

Concretely: take a diamond, `0→1→3` and `0→2→3`. DFS goes 0, 1, 3, finishes 3, finishes 1, comes back to 0, goes to 2, and 2 looks at 3. With two states, 3 is ‘seen’ and I report a cycle. There is no cycle — 3 is a shared dependency, which is completely normal in any real dependency graph.

Three states fix it. Grey means started and not finished, so it is on the current path and an edge to it is a genuine back edge. Black means finished, so it and everything below it are done and an edge to it is just a second route into explored territory — I skip it silently.

The equivalent formulation, if colours feel heavy, is a `seen` set plus an `on_stack` set, with vertices removed from `on_stack` as the call returns. Same thing, and the removal on the way out is the part people forget.

This is also exactly why undirected cycle detection is a *different* algorithm rather than the same one: there, every edge appears in both adjacency lists, so every edge looks like a back edge, and the fix is to exclude the vertex you came from rather than to track colours.”

“Convert your recursive DFS to iterative, including the finish order.”

“The plain conversion is easy: a stack, pop from the end, push unseen neighbours. That gives me the same visit order — reversed if I want to match exactly — and removes the depth limit.

What that version loses is the finish event. Recursion gives me ‘on the way back out’ for free; a plain stack loop has no equivalent moment.

The standard fix is to push each vertex twice with a flag. When I first pop a vertex I push it straight back with the flag set, *then* push its children on top. Because the children are above it on the stack, they all get fully processed before the flagged copy comes back off — and when it does, everything below that vertex is finished. So the flagged pop is exactly the recursive function’s return point.

That doubles the number of pushes, which is the same complexity and about twice the stack traffic, and it is the general recipe for turning any post-order recursion into a loop. I would use it for topological sort and for strongly connected components; for plain traversal I would not bother.”

“Find all paths from A to B.”

“DFS, and this is the one case where the `seen` set works differently, which is worth flagging because it looks like a bug.

For a normal traversal, once a vertex is seen it is never revisited. Here, a vertex that is not on the *current* path may well be on a different valid path, so I mark it on entry and **unmark it on the way out** — that is backtracking, from [day 94](#). What I am tracking is not ‘have I ever visited’ but ‘is this on the path I am currently building’, which is exactly the grey set again.

The cost is completely different from a traversal. There is no $O(V + E)$ bound, because the number of paths can be exponential — a graph of k diamonds in series has 2^k paths — so the honest complexity is $O(\text{number of paths} \times \text{path length})$, output-bound rather than graph-bound.

So the first thing I would ask is whether they want all the paths or just the count, because on a DAG the count is a simple DP over vertices in topological order and is linear, while enumerating them cannot be.”

The model answer

“Given n courses numbered 0 to $n - 1$ and a list of prerequisite pairs, determine whether all courses can be finished.”

“Let me state the model, then the algorithm, then the two things that make it wrong if I skip them.

A vertex is a course. An edge from A to B means A must be taken before B — so it is directed, and the question ‘can all courses be finished’ is exactly ‘is this graph acyclic’, because a cycle is a set of courses each waiting on another with no possible starting point.

So: directed cycle detection with three colours. White is untouched, grey is ‘started, on my current path’, black is ‘finished’. I DFS from every white vertex; if I ever reach a grey vertex, that vertex is an ancestor of where I am now and I have closed a loop, so I return true for ‘has cycle’. Reaching a black vertex is fine and I skip it.

The first thing that makes it wrong if I skip it is two states instead of three. Prerequisite graphs are full of shared dependencies — two courses both requiring linear algebra — and a two-state check reports a cycle on that perfectly valid shape. I would say this out loud because it is the actual bug in most submissions.

The second is the outer loop. A course catalogue is not a connected graph, and a cycle can easily sit among courses that nothing else points into. Starting from course 0 and stopping would report success on a catalogue that is broken. So the loop is over all n vertices, with the colour array shared.

The third thing, which is about the input rather than the algorithm: I would build the adjacency list from `range(n)`, not from the pairs, so that a course with no prerequisites and no dependents still exists in the structure. Otherwise my outer loop silently skips it — harmless here, but the same omission breaks the follow-up question about ordering.

Cost: $O(V + E)$ time, $O(V)$ space for the colours and the stack. With n up to ten thousand and edges similar, that is instant.

On recursion: the constraints here allow $n = 10^5$ in the harder variant, and a chain of a hundred thousand prerequisites is a legal input, so I would write it iteratively with an explicit stack and the two-push finish marker. If the constraints were small I would write it recursively for readability and say that I was choosing readability knowingly.

And the follow-up they will ask is ‘now give me a valid order’, which is the same traversal with one addition: append each vertex to a list when it turns black, then reverse the list. That reversed finish order is a topological sort, and it comes free from the algorithm I have already written — which is a good reason to build the three-colour version rather than a two-set version even when the question only asks for yes or no.”

9 Recall card

DFS goes deep, then backs up. Recursively: mark on entry — first line, not after the loop — then recurse into unseen neighbours. Iteratively: it is BFS with `stack.pop()` instead of `queue.popleft()`.

Same $O(V + E)$ as BFS. The difference is what you know while running: DFS has a **current path**, BFS has rings. Fewest steps → BFS. Cycles, topological order, all-paths, structure → DFS.

Three colours for directed cycles. White unvisited, **grey = on the current path**, black finished. An edge to grey is a cycle; an edge to black is a shared dependency and is fine. Two states report a cycle on a plain diamond.

Undirected is a different check: every edge appears in both adjacency lists, so exclude the vertex you came from rather than tracking colours.

Write it iteratively when `n` can exceed a few thousand. ~960 usable frames by default, and `setrecursionlimit` turns a `RecursionError` into a segfault. And always loop over every vertex — a directed cycle can hide in a region nothing points into.

Distributed systems revision and interview questions

1 What this is, and why they ask it

Sixteen days ago you did not know what a quorum was. Today the phase closes, and the way to close it is not to reread it — it is to answer questions about a system you have never seen, out loud, with someone pushing back.

This lesson has two halves. The first compresses the phase into the shape it actually has: one problem, and about eight responses to it. The second is a mock design round on an unseen system, written out — the questions in the order they come, and what a good answer sounds like, including the parts where you have to say “I do not know, here is how I would find out”.

The reason this phase matters more than its size suggests is that **it is the part of system design that cannot be memorised**. You can memorise what Kafka is. You cannot memorise what happens when the payment succeeded but the reply was lost, because the answer depends on the system in front of you. Interviewers know this, which is why the follow-ups in a senior design round almost always walk into this territory: *what if that node dies, what if that message arrives twice, what if the two of them disagree*.

By the end of today you can state the one problem underneath the whole phase, name the eight responses to it and when each applies, quote the dozen numbers that make an answer concrete, and take an unseen design from “what if it fails” to a defended answer.

2 The story

Sunil has catered four hundred and six weddings and he no longer thinks about food.

He thinks about the gas.

Eleven years ago, at a wedding in Parnell, two cylinders ran out within twenty minutes of each other at a quarter past eight, with four hundred people waiting, and it took ninety minutes to get replacements from a shop that had already closed. He has never forgotten it and it has shaped everything he has done since.

So now he carries a spare cylinder for every two in use. Not one spare — one per two. It costs him money and space in the van and he does it every single time, including for a lunch for sixty.

There are eleven of these rules and they are all like that.

He never sends both vans by the same road. If there is a way to go by the highway and a way to go through the town, the rice and the vegetables go one way and the sweets and the serving staff go the other. Two vans, two roads, because a lorry turned over on the Pune highway in 2019 and everything he owned was behind it for four hours.

He arrives with food for fifteen percent more people than he was told. Every family under-counts. Every one of them.

He puts one man at the entrance whose only job is to count people going in and to tell Sunil the number at half past seven. That man does nothing else all evening. His nephew thinks it is a waste of a person.

And the rule his nephew argues with most: if the sweet is going to be late, Sunil takes it off the counter and puts out fruit instead, and he decides that at eight o'clock, not at nine when people are already standing there. Something is served. It is not what was promised. Nobody complains about fruit; everybody complains about an empty counter with a sign on it.

His nephew asked him once, quite reasonably, why he does not just work with better suppliers so that none of this is necessary.

Sunil said something that the nephew repeated to other people for years afterwards, usually getting it slightly wrong.

What he said was that he has worked with the best suppliers in the district for twenty years, and the good ones fail about as often as the bad ones — they just fail less stupidly. The gas runs out. The van is late. The count is wrong. It is going to happen at some percentage of four hundred weddings no matter who he hires, and the only question that has ever mattered is whether the guests find out.

3 The idea in plain English

Sunil has arrived at the single idea underneath this entire phase.

One problem: partial failure. A distributed system is one where some of it can be broken while the rest carries on. That sounds mild and it is the source of every difficulty in sixteen days of material. In a single program, a component either works or the whole program crashes. In a distributed system, one machine is unreachable while nine are fine, and the nine have to decide what to do about it — **without being able to tell whether the tenth is dead, slow, or perfectly healthy behind a broken cable.**

Everything in the phase is a response to that. Here are the eight, each in one line, each with the day it came from.

1. You cannot tell dead from slow. [Day 124](#). Heartbeats and timeouts produce a *suspicion*, never a fact, and every timeout is a bet between declaring healthy nodes dead and sending work to corpses.

2. A timeout is not a failure — it is an unknown. [Day 122](#). The request may have succeeded with the reply lost. Which is why retrying is only safe when the operation is idempotent, and why every write worth money carries a client-generated key.

3. Time does not order events. [Day 123](#). Clock skew exceeds the gap between related events routinely, so you order by causality — happens-before, Lamport counters, vector clocks — and use the monotonic clock for every duration.

4. You cannot have consistency and availability during a partition. Days 113–115. CAP is a forced choice that only bites while the network is broken, and the real spectrum is linearizable at one end and eventually consistent at the other, with session guarantees in between.

5. Agreement costs a majority. Days 116–119. A quorum — $R + W > N$ — gives you overlap; Raft gives you an agreed order via a replicated log and a leader with a lease. Both need more than half the nodes, and both stop serving writes when they cannot get one.

6. Atomicity across services is a lie you replace with compensation. Days 120–121. Two-phase commit gives real atomicity and blocks on the coordinator; a saga gives eventual atomicity with compensating transactions, no isolation, and an outbox to make the write and the event atomic.

7. Retrying makes small failures invisible and large ones worse. [Day 125](#). Exponential backoff with **full jitter**, a retry budget, and retries at exactly one layer.

8. Bound everything, and fail fast when the bound is hit. [Day 126](#). A bulkhead per dependency and a circuit breaker on top, because the service that dies is usually the healthy one in front of the slow one.

And the ninth, which is not a mechanism but a stance, and is Sunil's actual point: you do not prevent failure, you make it survivable. Fencing tokens instead of a better failure detector. Idempotency instead of exactly-once delivery. A degraded response instead of an error. Fruit instead of an empty counter.

The recognition question for this whole phase, in one line: *what happens if this component is unreachable, and what does the user see?* If you cannot answer both halves for every box in your diagram, the design is not finished.

And the three answers that are always available, in the order to reach for them:

1. **Remove the coordination.** One owner per key, a unique constraint, a conditional write, an idempotent operation. Cheapest, safest, and always worth a sentence before anything else.
 2. **Bound the damage.** Bulkheads, budgets, quorums, leases. Nothing is unlimited.
 3. **Make being wrong survivable.** Fencing, compensation, reconciliation, expiry jobs.
-

4 The picture

The whole phase as one decision path:

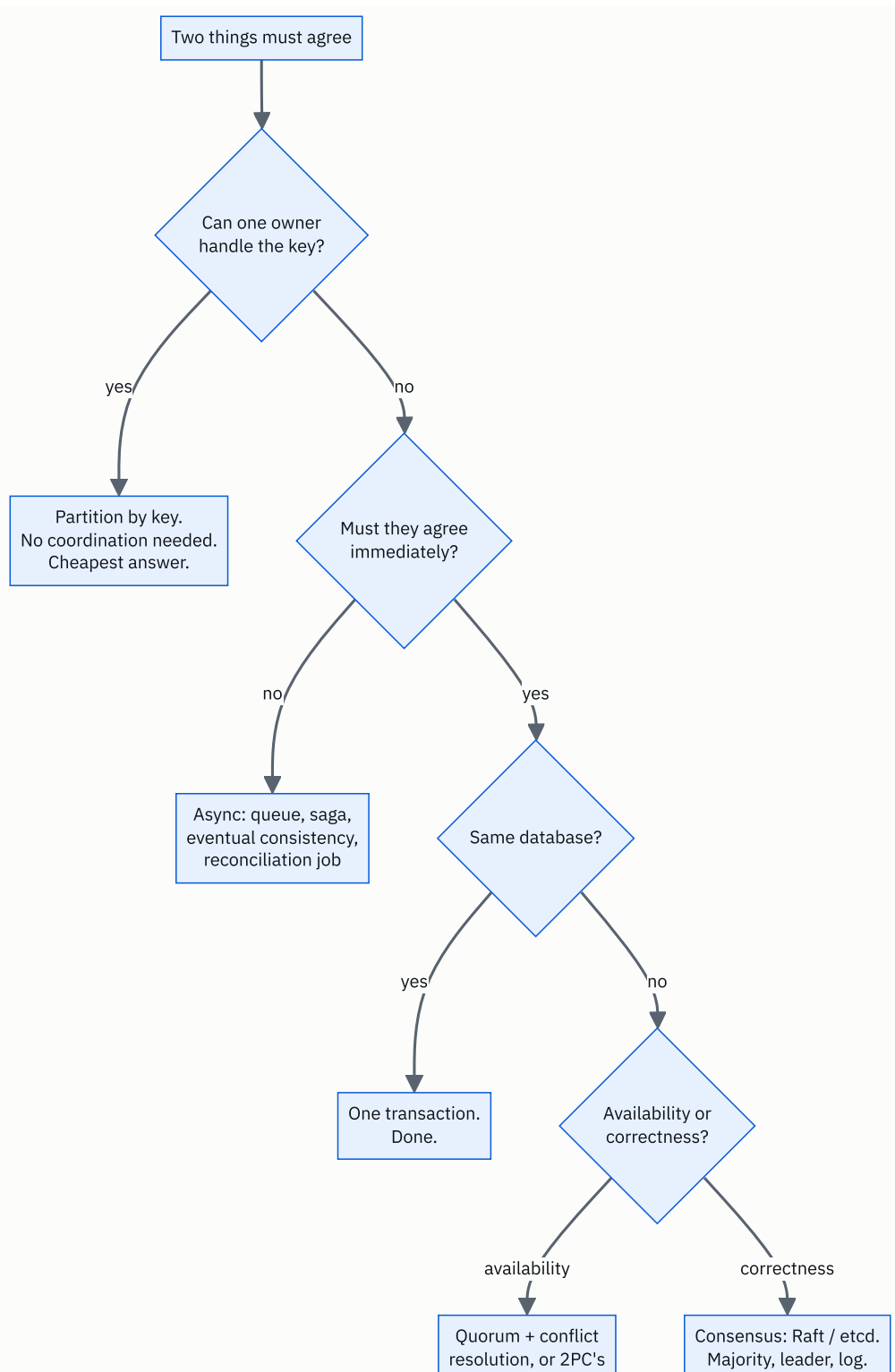


FIGURE 128.3

What to notice. The first question throws out the most work, and most real designs land in the top two boxes. Reaching for consensus is a strong move that should be justified, not a default.

And the failure ladder — what happens to one request as things go wrong, with the day each defence came from:

```

request arrives
  |
  v
[bulkhead] permits left for this dependency? ← day 126
  | no → reject instantly, fallback
  v yes
[breaker] is this dependency known-broken? ← day 126
  | open → fail fast, fallback
  v closed
[idempotency key] have I done this already? ← day 122
  | yes → return the stored response
  v no
[retry policy] backoff + full jitter, budget ← day 125
  v
[the call] ---- timeout ----> UNKNOWN, not failed ← day 122
  |                                     |
  v success                           v
[ fencing token checked at          retry if idempotent,
  the storage layer ] ← day 124    else reconcile later
  v
[write + outbox in one txn] ← day 121

```

What to notice. Six defences, and each one is cheap. None of them is clever. The design work is knowing which ones a given request needs, and what the user sees at each exit point.

5 How it actually works

The compression. Everything worth having automatic, in one place.

The consistency spectrum

LEVEL	GUARANTEE	COST	WHERE
Linearizable	Every read sees the latest write, globally	Consensus, a round trip to a majority	etcd, ZooKeeper, Spanner
Sequential	Everyone sees the same order, maybe stale	Leader with a log	Raft-backed stores
Causal	If A caused B, everyone sees A first	Vector clocks or HLCs	CockroachDB, MongoDB sessions
Read-your-writes	You see your own writes	Sticky routing or a session token	Most web apps
Eventual	Replicas converge, given quiet	Nothing	Cassandra, DynamoDB default, DNS

The move that wins interviews is choosing per-operation, not per-system. A social feed can be eventually consistent and its “did my post go through” read must be read-your-writes. Same system, two levels.

Quorums

```
N replicas, W acknowledge a write, R answer a read
R + W > N → read and write sets overlap → a read sees the latest write
W > N/2 → writes are serialised (no two conflicting writes both succeed)
```

```
N=3 W=2 R=2 the standard: survives one node down, both ways
N=3 W=3 R=1 fast reads, no write tolerance
N=3 W=1 R=1 fast everything, eventual consistency, conflicts possible
```

Raft, in five lines

Leader elected by majority vote with randomised 150–300 ms election timeouts. All writes go to the leader, which appends to its log and replicates. An entry is **committed** once a majority have it. Followers apply committed entries in log order. A leader that cannot reach a majority steps down.

Why it matters in a design: anything using Raft stops accepting writes when a majority is unreachable. That is a deliberate availability sacrifice and you should say so when you place it.

Distributed transactions

2PC: prepare, then commit. Real atomicity, and the coordinator’s failure between the two phases leaves every participant holding locks and waiting. Blocking, slow, and it couples availability multiplicatively.

Saga: local transactions plus compensating actions. Eventual atomicity, **no isolation**, compensation is not rollback — a refund is a new transaction with a fee, not an erasure. Order the steps so the irreversible one is last. Orchestrate beyond about three steps.

Outbox: write the business row and the event row in the same local transaction; a publisher polls and emits. Makes “the write happened” and “the world was told” atomic, at the cost of at-least-once publication.

Idempotency

Client generates the key, once per logical operation, reused on every retry. Receiver inserts the key with a unique constraint **before** doing the work, stores the response against it, and returns three ways: fresh, stored response, or 409 while in flight. Exactly-once delivery is impossible; at-least-once plus idempotent processing gives exactly-once effect.

Clocks

Skew 0.1–5 ms in a data centre, 10–100 ms across regions. Order by causality, not timestamps. Monotonic clock for every duration. Last-write-wins discards data inside the skew window and is Cassandra’s default. HLCs are the modern default; TrueTime buys global order for ~10 ms of commit-wait.

Failure handling

Detection time = interval × misses allowed. Pick the timeout from the check’s p999, not from a guess. Liveness only checks what a restart fixes. Gossip (SWIM) past a few hundred nodes, with indirect probes. Retries: exponential backoff, **full jitter**, three attempts, a 10% budget, one layer. Bulkhead per dependency sized by Little’s Law; circuit breaker with a failure *rate* over a recent window and a minimum call volume; and decide the fallback before you decide the thresholds.

Locks

`SET key <token> NX PX 30000`, release by Lua script checking the token, watchdog refreshing at a third of the lease. The paused holder is unfixable — fence at the storage layer. Redis for efficiency locks, etcd or ZooKeeper for correctness locks. And look for a unique constraint first.

6 The numbers

The dozen numbers that make an answer concrete. If you can quote these, your answers stop sounding like vocabulary.

Latency, the ladder everything is measured against:

memory read	100 ns	
SSD random read	100 us	(1,000x memory)
same-datacentre round trip	0.5 ms	(5,000x memory)
cross-region round trip (US-EU)	80 ms	
cross-planet (US-India)	200 ms	

Clocks:

NTP skew, one datacentre	0.1 - 5 ms
NTP skew, across regions	10 - 100 ms
unsynchronised drift	~2.6 s per day
TrueTime epsilon	1 - 7 ms; commit-wait ~10 ms per write

Failure detection:

Raft election timeout	150 - 300 ms
Kubernetes liveness default	10 s x 3 = 30 s
AWS ALB health check default	30 s x 2 = 60 s
Redis Sentinel down-after	30 s
ZooKeeper session	4 - 40 s

Retries and breakers:

retry attempts	3
backoff base	~ service p50, e.g. 100 ms
retry budget	10% of request rate
breaker threshold	50% failure rate over ≥ 20 recent calls
breaker cooldown	30 s
amplification, 3 layers x 3	27x

Throughput:

Redis SET NX	~100,000 ops/s per instance
ZooKeeper/etcd writes	~10,000 - 20,000 ops/s (quorum + fsync)
Postgres single-row writes	~10,000/s; hot row with locks ~200/s
Kafka per partition	~10 MB/s, ~50,000 msgs/s

And the derived ones you compute live, which is what actually impresses:

capacity exhaustion	
200 workers, 100 req/s at 30 s each	→ pool gone in 2 s
bulkhead size (Little's Law)	
100 req/s x 0.2 s p99	→ 20 permits
lock throughput	
50 ms of work under a lock	→ ~19 ops/s, whatever the fleet size
dedup storage	
10M requests/day x 670 B x 24 h	→ 6.7 GB
writes unorderable by timestamp	
10,000 writes/s x 50 ms skew	→ 500 writes
jitter's effect on peak	
1,000 clients over 4 s vs 10 ms	→ 400x lower peak

A worked example of the arithmetic an interviewer wants to see. “Ten million payments a day, and you must not double-charge”:

10,000,000 / 86,400	= 116 payments/s average
peak at 5x	= 580/s
timeouts at 0.1%	= 10,000 ambiguous requests/day
30% retried, 60% had succeeded	= 1,800 double charges/day
x Rs 800 average	= Rs 14,40,000/day at risk
dedup store: 670 B x 10M	= 6.7 GB for a 24-hour window
added latency	= 1 ms on a 250 ms call = 0.4%

Six lines, and the design decides itself. That is the skill: not knowing that idempotency keys exist, but turning “duplicates are bad” into fourteen lakh rupees a day against seven gigabytes.

7 The trade-offs

The five that come up in every round, each stated as the sentence you would actually say.

Consistency against availability, but only during a partition. “CAP forces the choice only while the network is broken. The rest of the time I have both, and what I am really trading day to day is latency against consistency — every consistency guarantee costs a round trip to somebody.” Choose per operation: the balance is linearizable, the profile picture is eventual.

Synchronous against asynchronous. “Synchronous is simple and couples availability — my uptime becomes the product of every dependency’s. Asynchronous decouples and costs me ordering, duplicate handling and a much harder debugging story.” Move work

off the request path when the user does not need the answer, and say what the user sees in the gap.

Strong coordination against no coordination. “Consensus gives me one agreed answer and stops serving when it cannot reach a majority. No coordination gives me availability and conflicts I have to resolve. In between is a single owner per key, which is what I would try first because it gives me strong ordering per key with no protocol at all.”

Detecting failure fast against detecting it correctly. “Any timeout is a bet. Short means I eject healthy nodes and lose capacity; long means I send work to corpses. There is no setting that avoids both, so I price the two mistakes and then make being wrong survivable — fence, so a false positive costs a wasted failover rather than corrupted data.”

Doing the work twice against not doing it at all. “At-least-once with idempotent handlers, essentially always. At-most-once loses data silently and exactly-once does not exist. The cost is a dedup store and a discipline that every handler must be idempotent, which is a real engineering constraint on every team that touches the system.”

And the meta-trade-off worth naming out loud: every one of these mechanisms adds a component that can itself fail. A dedup store, a lock service, a breaker’s shared state. **The best answer to a distributed systems problem is often to make it not a distributed systems problem** — one owner, one transaction, one constraint — and a candidate who reaches for that first is rated above one who reaches for Raft first.

8 In the interview

How it gets asked

Rarely as a topic. Almost always as a follow-up in the middle of a design:

- “Your write succeeded on two of three replicas. What do you tell the user?”
- “That service is down. What happens to a request that arrives now?”
- “The message was delivered twice. Is that a problem?”
- “Both nodes think they are the leader. How did that happen and what does it break?”
- “You put a queue there. What if the consumer crashes after processing but before acknowledging?”
- “How do you know that node is dead?”

The pattern to notice: every one of them is ‘what if this specific thing goes wrong’. Preparing for this phase means having an answer ready for every box in a diagram you have not drawn yet.

The mock round

“Design a system that lets users transfer money between accounts. Two banks, two databases. Forty-five minutes.” — and then the interruptions.

Minute 0 to 3 — set the shape, name the hard part.

“The core operation is: debit account A at bank 1, credit account B at bank 2. Two databases, so no single transaction. That is the whole difficulty and everything else follows from it.

Before I design anything I want to state the requirements I am assuming, because they change the answer. Money must never be created or destroyed — a debit without a credit is a bug I cannot ship. A transfer may be *slow*, and that is acceptable; people accept ‘processing’ on a bank transfer. And I must never double-debit, which is a stronger requirement than never double-crediting, because customers notice one and not the other.

Is that the right framing, or is there a latency requirement I should design to?”

Minute 3 to 12 — the design, with the failure named at each step.

“A saga, orchestrated, with an outbox. Two-phase commit is off the table: the other bank will not join my transaction, and even if it would, holding a lock on an account across a cross-bank call would be unacceptable on a busy account.

The steps are: create a transfer record in `PENDING`, debit A, credit B, mark `COMPLETE`. Compensation for the debit is a credit back to A — and I would say immediately that **that is not a rollback**. It is a second real transaction, visible on the customer’s statement, and for a cross-bank transfer it may carry a fee. The customer sees both entries.

The orchestrator writes its state before every call, so a restart resumes exactly where it stopped rather than reissuing a debit it already made.

Every step carries an idempotency key, generated once per transfer and reused on every retry, checked at the bank if it supports it and in my own dedup table if not. Insert-before-work with a unique constraint, so two concurrent retries cannot both debit.

The transfer record and the outbox row go in one local transaction, so ‘the debit happened’ and ‘the world was told’ cannot diverge.”

Minute 12 — first interruption. *“The debit succeeded and the credit call timed out. What now?”*

“A timeout is an unknown, not a failure, so I have three possibilities and I must not guess: the credit never happened, it happened and the reply was lost, or it is still in flight.

So I do not compensate immediately, and that is the important decision. I **query** — the credit carried an idempotency key, so I ask the other bank ‘what happened to key X’. If it says completed, I mark the transfer complete and stop. If it says nothing, I retry with the same key, which is safe precisely because of the key.

I retry with backoff and jitter for a bounded time — say five minutes. Beyond that I stop retrying forward and compensate: credit the money back to A, mark the transfer `FAILED`, notify the customer.

And I would name what makes that safe: **if the credit does eventually land after I have compensated, I have created money**. So the compensation carries its own key and the credit must be cancellable, or — better — the design uses a `RESERVED` state at the destination rather than a direct credit, so the compensation is releasing a reservation rather than reversing a completed credit. I would push for that in the requirements conversation.”

Minute 18 — second interruption. *“Your orchestrator is running on three machines. How do you stop two of them from processing the same transfer?”*

“First I would try to remove the need. **Partition by transfer ID** — consistent hashing, or a Kafka partition keyed by transfer ID — so exactly one machine ever handles a given transfer. That gives me mutual exclusion for free and with no lock service.

If that is not available, then a lock: `SET lock:transfer:{id} {uuid} NX PX 30000`, released by a Lua script that checks the token, with a watchdog refreshing at ten seconds.

And I would immediately name the failure that lock does not solve: if a holder pauses for longer than the lease — a garbage collection, a stalled disk — the lease expires, another machine acquires, and the first wakes up believing it still holds it. Two orchestrators, both issuing debits.

So the lock hands out a **fencing token**, and the transfer record’s update is conditional on it: `UPDATE transfers SET state = ... WHERE id = ? AND fence < ?`. Zero rows updated means I lost the lock and I stop. The enforcement is at the storage layer because the paused machine cannot be trusted to check anything.

The honest summary is that the lock is an efficiency measure and the fencing token is the correctness measure.”

Minute 26 — third interruption. *“The other bank is down for two hours. What does the user see?”*

“This is a product decision as much as an engineering one, so I would say what I would build and flag the decision.

A circuit breaker on the other bank, opening at a 50% failure rate over the last twenty-odd calls. Once open, I stop attempting — which protects them from a retry storm and protects me from having every worker sitting in a backoff sleep. That second half is the one people forget: without the breaker my own service dies of thread exhaustion two seconds after theirs does.

A bulkhead sized by Little’s Law, so even before the breaker opens, calls to that bank can consume at most, say, twenty permits and never the whole pool. Transfers to *other* banks keep working throughout, which is the property I actually care about.

What the user sees: the transfer is accepted and shows as pending, not failed. The money leaves their available balance immediately — I hold it — and the transfer sits in a queue with a slow, heavily jittered retry, minutes apart, for up to twenty-four hours. That turns a two-hour outage into a delayed confirmation rather than a rejected transfer.

And I would insist on the expiry job, because it is the safety net for everything above: any transfer in `PENDING` for more than twenty-four hours is compensated automatically and the customer is refunded and told. That single mechanism means a lost message costs me twenty-four hours of a held balance instead of money stuck forever. **That interval is my worst-case inconsistency window and it is a number I choose deliberately.**“

Minute 34 — fourth interruption. *“How do you know the money is right at the end of the day?”*

“Reconciliation, and I would put it in the design rather than treat it as an operations concern, because everything above is best-effort and this is the thing that catches what it misses.

Every night, compare my ledger against the other bank’s settlement file: every transfer I think completed, every one they think completed, and the symmetric difference. Three buckets — mine only, theirs only, amounts disagree — and each gets a defined action rather than an alert.

The reason I would defend this in front of a sceptical interviewer is that idempotency keys expire after twenty-four hours, breakers can be wrong, and a compensation can land in a dead-letter queue at three in the morning. Every mechanism in this design has a failure mode, and reconciliation is the only one that has the *ground truth* rather than a local opinion. In a payment system it is not a nice-to-have; it is where the real correctness lives, and the rest is there to keep its workload small.

I would also make it a metric, not a job: ‘unreconciled transfers older than one hour’ on a dashboard, with a page at some threshold. That number going up is the earliest honest signal that something in the design is wrong.”

Minute 40 — the last one. *“What would you build first?”*

“The ledger with the idempotency key and the reconciliation job, in that order, before any of the resilience machinery.

The reason is that a correct ledger with no breaker degrades badly under failure but never loses money. A beautiful resilience layer over a ledger that can double-debit loses money quietly, and quietly is the problem. So: get the write path provably safe, get the daily comparison running, and then add bulkheads, breakers and jittered retries as the load justifies them.

The one exception is the bulkhead, which I would put in from day one, because it is a bounded pool rather than a policy — it needs no tuning and it protects me from failures I have not thought of yet.”

The model answer

“In one minute: what actually makes distributed systems hard?”

“Partial failure, and specifically that you cannot observe it accurately.

In a single program, a thing works or it crashes. In a distributed system, one component is unreachable while the rest carries on — and from outside, a crashed machine, a paused machine, and a broken cable are indistinguishable. Every hard problem in the field comes from that one fact.

You cannot tell dead from slow, so failure detection produces a suspicion and every timeout is a bet. A timeout on a request is an unknown rather than a failure, so retrying needs idempotency. Clocks disagree by more than the gap between related events, so timestamps do not order anything and you fall back on causality. Agreement needs a majority, and a majority is not always available. Atomicity across services is unachievable, so you compensate instead of rolling back.

And the practical conclusion, which I think is the actual answer: **you stop trying to be correct about what happened and start making it survivable to be wrong.** Fencing tokens instead of a perfect failure detector. Idempotent handlers instead of exactly-once delivery. Reconciliation instead of trusting the write path. A degraded response instead of an error.

The best move available is usually to remove the coordination entirely — one owner per key, one transaction, one unique constraint — and I would look for that before reaching for consensus, because every mechanism I add is another component that can fail partially.”

9 Recall card

One problem: partial failure, unobservable. Dead, slow, and unreachable look identical from outside. Every other idea in the phase is a response to that.

Eight responses: suspicion not truth (detection); a timeout is an unknown (idempotency); causality not clocks; CAP only bites during a partition; agreement costs a majority (quorum, Raft); atomicity across services becomes compensation (saga, outbox); backoff with **full jitter** and a budget; bound everything (bulkhead) and fail fast (breaker).

The stance that matters more than the mechanisms: do not prevent failure, make it survivable. Fencing, idempotency, compensation, expiry jobs, reconciliation.

Reach in this order: remove the coordination (one owner, one constraint) → bound the damage → make being wrong survivable. Consensus is a strong move, not a default.

Have the numbers ready: 0.5 ms same-datacentre / 80 ms cross-region, skew 0.1–5 ms local and 10–100 ms remote, Raft 150–300 ms, 3 retries at 10% budget with full jitter, breaker at 50% over ≥ 20 calls, $R + W > N$, $27\times$ amplification, and the two-second pool exhaustion.

DSA topic: Depth-first search on a graph **System design topic:** Distributed systems revision and interview questions

Code these, in this order

One rule for the whole set: **read the constraints line before writing the first character, and if `n` can exceed a few thousand, write the iterative version.** Not “write it recursively and convert later”. The conversion is where the bugs are, and the constraint tells you which one you need before you start.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Number of Islands	LeetCode 200 (Medium)	DFS on a grid, plus the outer loop over every cell.
2	Course Schedule	LeetCode 207 (Medium)	Three colours, and the outer loop over every vertex.
3	Clone Graph	LeetCode 133 (Medium)	The <code>seen</code> map doing double duty as the old-to-new mapping.
4	All Paths From Source to Target	LeetCode 797 (Medium)	Marking on entry and unmarking on exit — the difference.
5	Pacific Atlantic Water Flow	LeetCode 417 (Medium)	Two DFS runs from the edges inward, not one from each cell.

On problem 1, write it three ways

Recursive, iterative with a stack, and BFS. Then run all three on a $1,000 \times 1,000$ grid of all land. Record which ones survive and what the failures look like. One of them produces a `RecursionError`; try raising the limit and record what you get instead.

On problem 2, deliberately write the two-state version

Implement cycle detection with a single `seen` set and run it on the diamond `0→1→3`, `0→2→3`. Record the answer. Then fix it with three colours and run it again. Write one sentence saying what “seen” failed to distinguish.

On problem 4, watch the unmark

Solve it, then delete the line that removes the vertex from the current path on the way out. Record how many paths each version finds on a small graph with two routes to the same vertex. Say in one sentence why a traversal must not unmark and a path enumera-

tion must.

Then the conversion drill

Take your recursive solution to problem 2 and convert it to iterative **with finish order**, using the two-push marker technique. Verify both give the same answer on ten random graphs. This is the hardest twenty lines in the phase and it is worth doing once, properly, now.

The shape drill

1. Write recursive DFS from memory in thirty seconds.
2. Write iterative DFS and say which single character differs from BFS.
3. Write the finish-order iterative version with the marker flag.
4. Write the components version and point at the line that makes it different.
5. Say where the mark goes in each version, and why there.

The BFS-or-DFS drill

1. State the one-line rule.
2. Say what DFS has that BFS does not, and name three problems that need it.
3. Say what BFS has that DFS does not, and name two problems that need it.
4. Give both memory profiles on a star graph and on a path graph.
5. Say which is faster, and why that is a trick question.

The colours drill

1. Name the three colours and what each means.
2. Say which one is the cycle test and why.
3. Walk the diamond example out loud and say where a two-state check goes wrong.
4. Give the equivalent formulation with two sets, and the line people forget.
5. Say why undirected cycle detection needs a completely different check.

The recursion-limit drill

1. Give Python's default limit and the usable depth.
2. Name three ordinary graph shapes that exceed it.
3. Say what `setrecursionlimit(200_000)` produces and why that is worse.
4. Give the depth of a DFS on a $1,000 \times 1,000$ open grid.
5. Say what in the constraints tells you to write the iterative version.

The costs drill

1. Derive $O(V + E)$ for DFS.
2. Say why it is identical to BFS.
3. Give the space cost of each and the graph shape that makes each one worst.
4. Give the extra cost of the finish-marker version.

The break-it drill

Trigger each and record the exact output or error:

1. The mark placed after the loop instead of on entry, on a graph with a cycle.
 2. Two-state cycle detection on a diamond.
 3. Undirected cycle detection with no parent check, on a single edge.
 4. Directed cycle detection starting only from vertex 0, on a graph whose cycle is elsewhere.
 5. Recursive DFS on a 5,000-vertex chain.
 6. `sys.setrecursionlimit(500_000)` followed by the same call.
 7. Path enumeration with no unmark on the way out.
-

The one-problem drill

1. State the single problem underneath the whole phase, in one sentence.
2. Say why it is different from failure in a single program.
3. Name the eight responses and the day each came from.
4. State the ninth — the stance, not a mechanism.
5. Give the recognition question for the phase, in one line.

The consistency drill

1. Name the five levels of the spectrum and what each guarantees.
2. Give the cost of each.
3. Give a real system for each.
4. Say why choosing per operation beats choosing per system, with an example.
5. State what CAP actually forces and when.

The quorum drill

1. State the two quorum inequalities and what each buys.
2. Give three (N, W, R) settings and what each is for.

3. Say what happens to writes when a majority is unreachable, and why that is deliberate.
4. Summarise Raft in five lines.

The transactions drill

1. Give 2PC's guarantee and its failure mode.
2. Give a saga's guarantee and its two losses.
3. Say why compensation is not rollback, with a concrete example.
4. State the ordering rule for saga steps.
5. Describe the outbox pattern and what it makes atomic.

The numbers drill

Quote each from memory, then check:

1. Memory read, SSD read, same-datacentre hop, cross-region hop, cross-planet hop.
2. Clock skew, local and remote.
3. Raft election timeout; Kubernetes liveness default; ALB default.
4. Retry attempts, budget percentage, breaker threshold and cooldown, amplification for three layers.
5. Redis, etcd, Postgres hot-row and Kafka per-partition throughput.

Then compute live:

6. Pool exhaustion for 200 workers and a 30-second dependency at 100 req/s.
7. Bulkhead size for 100 req/s at a 200 ms p99.
8. Throughput under a global lock with 50 ms of work.
9. The full ten-million-payments idempotency calculation, all six lines.

The trade-offs drill

State each as the sentence you would say out loud:

1. Consistency against availability.
2. Synchronous against asynchronous.
3. Strong coordination against none.
4. Fast detection against correct detection.
5. Doing it twice against not at all.
6. The meta-trade-off about adding components.

The interruption drill

For each, give a two-minute answer:

1. “Your write succeeded on two of three replicas. What do you tell the user?”
2. “The debit succeeded and the credit timed out. What now?”
3. “Two machines both think they hold the lock. How, and what breaks?”
4. “The downstream bank is down for two hours. What does the user see?”
5. “The consumer crashed after processing and before acknowledging.”
6. “How do you know the money is right at the end of the day?”
7. “What would you build first?”

Record yourself on three of them. Listen for whether you gave a number.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Traverse this graph depth-first. What if it is disconnected?* Mark on entry, the outer loop with `seen` shared, iterative when `n` is large with the segfault reason, and $O(V + E)$ identical to BFS with the difference stated as “what you know while running”.
 2. *Does this directed graph contain a cycle?* Three colours with grey as the current path, the diamond as the case two states get wrong, the outer loop over all vertices because a cycle can hide in an unreachable region, and topological sort as the free follow-up.
 3. *Your write succeeded on two of three replicas. What do you tell the user?* $R + W > N$ and what $W = 2$, $N = 3$ actually promises, then the honest part: the third replica will converge, a read at $R = 2$ will see it, and what you say to the user depends on whether this operation is the balance or the profile picture.
-

Before you move on

- [] I can write recursive DFS in thirty seconds.
- [] I can write iterative DFS and name the one character that differs from BFS.
- [] I can write the finish-order version with the marker flag.
- [] I mark on entry and know what happens if I mark after the loop.
- [] I can state the BFS-or-DFS rule in one line.
- [] I know what DFS has that BFS does not, and three problems that need it.
- [] I can name the three colours and say which one is the cycle test.

- [] I can walk the diamond example and say what two states get wrong.
- [] I know why undirected cycle detection is a different algorithm.
- [] I always loop over every vertex on a directed graph.
- [] I know Python's usable recursion depth and three shapes that exceed it.
- [] I know what raising the limit produces, and why it is worse.
- [] I know DFS and BFS have identical complexity.
- [] I know which graph shape makes each one's memory worst.
- [] I know path enumeration must unmark and traversal must not.
- [] I can state the one problem underneath this whole phase.
- [] I can name the eight responses and the day each came from.
- [] I can state the ninth — the stance.
- [] I can give the consistency spectrum with costs and real systems.
- [] I can state both quorum inequalities and what each buys.
- [] I can summarise Raft in five lines.
- [] I can give 2PC and saga guarantees and their failure modes.
- [] I can quote the latency ladder from memory.
- [] I can quote the clock, detection, retry and throughput numbers.
- [] I can compute pool exhaustion, bulkhead size and lock throughput live.
- [] I can do the ten-million-payments calculation in six lines.
- [] I can state all six trade-offs as sentences.
- [] I have answered all seven interruptions out loud.
- [] I answered all three questions above out loud.

Connected components

DATA STRUCTURES AND ALGORITHMS

Connected components

SYSTEM DESIGN

Message queues: why async changes everything

1 What this is, and why they ask it

A **connected component** is a piece of a graph where every vertex can reach every other, and no vertex can reach anything outside it. A graph is not necessarily one thing. It is a collection of separate pieces, and counting them is one of the most frequently asked graph questions there is.

The algorithm is four lines longer than the traversal you already know: loop over every vertex, and whenever you find one you have not seen, run a traversal from it and count that as one more piece. The `seen` set is shared across every run, which is what stops you counting a piece twice.

They ask this because it is the smallest possible problem that punishes the assumption everyone makes. Sample inputs are connected. Real inputs are not. A solution that starts from vertex 0 and stops passes every example in the problem statement and fails the hidden test. Interviewers know this and choose the test deliberately.

It is also the question underneath a surprising number of problems that never mention graphs: counting islands in a grid, merging duplicate accounts, grouping people into friend circles, finding how many components a network breaks into when a cable is cut. **The words “how many groups” almost always mean connected components**, and recognising that is the actual skill.

By the end of this lesson you can count components with either traversal, label every vertex with which piece it belongs to, handle a grid, name the case where Union-Find is the better answer, and say what changes when the graph is directed — because the answer there is genuinely different and it is the follow-up.

2 The story

Anil has been the plumber for the four buildings on Nehru Road for twenty-two years, and until last March he believed, like everybody else, that Sagar Apartments had one water system.

The complaint was ordinary. Flat 302 said their kitchen tap had gone weak. Anil went up, checked the tap, checked the line, found nothing obviously wrong, and did the thing he does when nothing is obviously wrong, which is to go up to the terrace and start closing valves.

There are three tanks on that terrace. Everybody knows about the two big ones. The third is smaller, sits behind the lift machine room, and has a coat of green paint on it from some year in the nineties.

He closed the valve on the first big tank and sent the watchman round the building to see which taps had stopped. Twenty minutes later the watchman came back with a list. Ground floor, first floor, and the left half of the second floor. Nothing above that.

He opened it again and closed the second tank. This time it was the right half of the second floor, all of the third, and the fourth.

That accounted for everything except three flats. 501, 502, and the office on the ground floor at the back.

Those three, it turned out, are on the green tank. They have been on the green tank since 1994, when somebody added the fifth floor and could not be bothered to run the line all the way down to the main tanks. Nobody currently alive in the building knew this. The three flats had never had a problem, so nobody had ever had a reason to find out.

Anil said the thing that struck him afterwards was how simple the method was, and how nobody had ever done it. You close one valve. You see who stops. Everything that stopped is on that tank, and everything that did not, is not. Then you do it again with the next one. When every tap in the building has been accounted for, you know how many separate systems there are, and you did not need to look at a single pipe inside a wall.

It took him one afternoon and a watchman.

The weak tap in 302 was a blocked aerator, which he fixed in four minutes.

3 The idea in plain English

Anil found the connected components of a plumbing network, and his method is exactly the algorithm.

A connected component is a piece where everything reaches everything. In an undirected graph, if you can get from **A** to **B** then you can get from **B** to **A**, so “reaches” carves the vertices into groups with no edges between them. The green-tank flats are a component: connected to each other, connected to nothing else.

Closing a valve and seeing what stops is a traversal. Anil picks a tank and finds everything reachable from it. In code you pick a vertex and run BFS or DFS from it; everything the traversal touches is in that vertex’s component. This is [day 127](#) or [day 128](#) unchanged — **the traversal is not the new part.**

The new part is the outer loop. After the first traversal you have one component and a set of vertices you have accounted for. Then you look for a vertex you have *not* accounted for and start again from there. Each fresh start is one more component. Anil did this three times and stopped when every tap was on his list.

The `seen` set is shared across every run, and that is the whole trick. If you reset it between traversals, the second run walks back over the first component and you count the same piece repeatedly. Anil kept one list for the whole afternoon, not one per tank.

So the algorithm is:

```
count = 0
for every vertex:
    if it has not been seen:
        count += 1
        traverse from it, marking everything reachable as seen
```

Four lines around a traversal you already have. That is genuinely all of it, and it is worth noticing how small the addition is compared with how often the omission is the bug.

Labelling is barely more work than counting. Instead of a counter, keep a dictionary from each vertex to its component number, filled in during the traversal. Then “are `A` and `B` in the same group?” is a comparison of two labels, and “how big is the largest group?” is a count. Most real problems want the labels, not the count.

BFS or DFS makes no difference here. Both visit exactly the vertices in the component, both cost $O(V + E)$, and the component you get is identical. Use DFS if you want fewer lines; use BFS if the graph might be deep enough to overflow the stack.

Grids are the most common disguise. A grid of land and water cells is a graph where the vertices are cells and the edges are “next to, and both land”. Counting islands is counting connected components, and the outer loop is the double loop over every cell. **You will meet this again properly on [day 130](#)**; the point today is that the algorithm is unchanged.

Union-Find is the other way to do this, and there is a rule for when. If the edges are given all at once and you traverse afterwards, a traversal is simpler and just as fast. If edges arrive *one at a time* and you must answer “how many groups now?” after each one, a traversal means re-running the whole thing per edge, and you want **Union-Find** — a structure that merges groups in nearly constant time. That is [day 138](#). **The rule: static graph → traversal; edges arriving over time → Union-Find.**

And directed graphs are a different question, which is the follow-up. In a directed graph, $A \rightarrow B$ does not mean $B \rightarrow A$, so “reaches” is no longer symmetric and there are two different things you might mean:

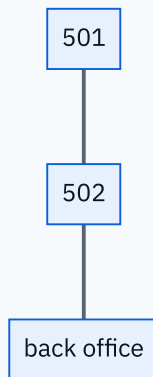
- **Weakly connected** — connected if you ignore the arrow directions. Just run the undirected algorithm on the graph with every edge doubled.
- **Strongly connected** — every vertex can reach every other *following the arrows*. This is a genuinely harder problem and needs a different algorithm.

If someone asks you to count components on a directed graph, ask which one they mean. It is a real ambiguity and asking is the right move.

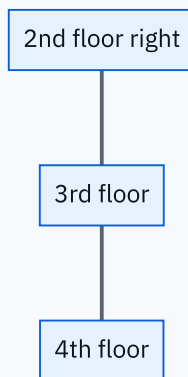
4 The picture

Anil's building, as a graph:

component 3 — the green tank



component 2 — big tank B



component 1 — big tank A



FIGURE 129.1

What to notice. There are no edges between the boxes. That is what makes them components rather than just clusters — not that they are loosely connected, but that they are *not connected at all*. A traversal starting anywhere in one box can never reach another box, however long you let it run.

The algorithm, traced:

vertices: 0 1 2 3 4 5 6 7 8
edges: 0-1 1-2 3-4 5-6 6-7 7-5

outer loop	vertex	seen?	action	seen after	count
1	0	no	traverse → {0,1,2}	{0,1,2}	1
2	1	YES	skip	{0,1,2}	1
3	2	YES	skip	{0,1,2}	1
4	3	no	traverse → {3,4}	{0,1,2,3,4}	2
5	4	YES	skip	...	2
6	5	no	traverse → {5,6,7}	{0..7}	3
7	6	YES	skip	...	3
8	7	YES	skip	...	3
9	8	no	traverse → {8}	{0..8}	4

					four components

What to notice at row 9. Vertex 8 has no edges at all, and it is still a component — a group of one. This is the case that a defaultdict built only from the edge list silently loses, because 8 never becomes a key. On “how many friend circles are there among six people with these two friendships”, the four people with no friends are four circles, and the answer is six, not two.

And the shared-seen mistake, drawn:

CORRECT: one seen set for the whole run

```
run 1 from 0:  seen = {0,1,2}      count 1
run 2 from 3:  seen = {0,1,2,3,4}   count 2      ← 0,1,2 still there
run 3 from 5:  seen = {0,...,7}     count 3
```

WRONG: seen reset inside the loop

```
run from 0:  seen = {0,1,2}   count 1
run from 1:  seen = {0,1,2}   count 2   ← same piece, counted again
run from 2:  seen = {0,1,2}   count 3   ← and again
...
answer: 9 components for a graph that has 4
```

What to notice. The wrong version returns `V` — one per vertex — which looks like a plausible answer and is not obviously wrong on a small example where every vertex is isolated.

5 The code, built step by step

Start with counting, which is the version most problems ask for.

```
from collections import deque

def count_components(graph: dict[int, list[int]]) → int:
    seen: set[int] = set()          # ONE set, outside the loop
    count = 0
    for start in graph:             # every vertex, not just one
        if start in seen:
            continue
        count += 1
        _flood(graph, start, seen)
    return count
```

Six lines, and two of them are the entire lesson: `seen` is declared before the loop, and the loop is over every vertex. Everything else you already had.

The flood itself is yesterday's traversal with nothing added:

```
def _flood(graph: dict[int, list[int]], start: int, seen: set[int]) → None:
    """Mark everything reachable from start. Iterative, so no depth limit."""
    stack = [start]
    seen.add(start)
    while stack:
        vertex = stack.pop()
        for neighbour in graph[vertex]:
            if neighbour not in seen:
                seen.add(neighbour)
                stack.append(neighbour)
```

`seen` is passed in rather than created here, which is what makes it shared. If this function created its own, every call would start from nothing and you would get the wrong answer shown in section 4.

Now labelling, which is what you usually actually want:

```
def label_components(graph: dict[int, list[int]]) → dict[int, int]:
    """Map every vertex to its component number, 0, 1, 2, ..."""
    label: dict[int, int] = {}
    current = 0
    for start in graph:
        if start in label:
            # `label` IS the seen set
            continue
        stack = [start]
        label[start] = current
        while stack:
            vertex = stack.pop()
            for neighbour in graph[vertex]:
                if neighbour not in label:
                    label[neighbour] = current
                    stack.append(neighbour)
        current += 1
    return label
```

The `seen` set has disappeared again, the same trick as BFS distances: a vertex is seen exactly when it has a label. One dictionary, three jobs — the seen set, the labels, and the count is `max(label.values()) + 1`.

With labels, two useful questions become one line each:

```
same_group = label[a] == label[b]
sizes = Counter(label.values())
largest = max(sizes.values())
```

Now the grid version, because it comes up more than the plain one:

```
def count_islands(grid: list[list[str]]) → int:
    if not grid or not grid[0]:
        return 0
    rows, cols = len(grid), len(grid[0])
    count = 0
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] ≠ "1":
                continue
            count += 1
            _sink(grid, row, col, rows, cols)
    return count
```

The double loop over cells is exactly the outer loop over vertices. Every unvisited land cell starts a new island.

```
def _sink(grid: list[list[str]], row: int, col: int, rows: int, cols: int) → None:
    """Flood this island, marking cells as visited by overwriting them."""
    stack = [(row, col)]
    grid[row][col] = "0"
    while stack:
        r, c = stack.pop()
        for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
            nr, nc = r + dr, c + dc
            if 0 ≤ nr < rows and 0 ≤ nc < cols and grid[nr][nc] = "1":
                grid[nr][nc] = "0"
                stack.append((nr, nc))
```

Overwriting the grid is the `seen` set — a visited land cell becomes water, so nothing revisits it. It costs no extra memory and it mutates the caller's input, which you should mention. The alternative is a separate `visited` grid of the same size.

`grid[nr][nc] = "0"` sits next to the `append`, not after the `pop`, for the reason from [day 127](#): otherwise the same cell is pushed once per neighbour that sees it, and the stack fills with duplicates.

The complete solution

```
"""Connected components: counting, labelling, sizes, and the grid version."""

from __future__ import annotations

from collections import Counter

def build(n: int, edges: list[tuple[int, int]]) → dict[int, list[int]]:
    """From range(n), so a vertex with no edges still exists."""
    graph: dict[int, list[int]] = {v: [] for v in range(n)}
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)
    return graph

def label_components(graph: dict[int, list[int]]) → dict[int, int]:
    """Vertex → component id. The dict doubles as the seen set."""
    label: dict[int, int] = {}
    current = 0
    for start in graph:
        if start in label:
            continue
        stack = [start]
        label[start] = current
        while stack:
            vertex = stack.pop()
            for neighbour in graph[vertex]:
                if neighbour not in label:
                    label[neighbour] = current
                    stack.append(neighbour)
        current += 1
    return label

def count_components(graph: dict[int, list[int]]) → int:
    labels = label_components(graph)
    return len(set(labels.values()))

def component_sizes(graph: dict[int, list[int]]) → list[int]:
    labels = label_components(graph)
    return sorted(Counter(labels.values()).values(), reverse=True)

def count_islands(grid: list[list[str]]) → int:
    """Grid version. Mutates the grid, marking visited land as water."""
    if not grid or not grid[0]:
        return 0
    rows, cols = len(grid), len(grid[0])
    count = 0
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] != "1":
                continue
            count += 1
            stack = [(row, col)]
            grid[row][col] = "0"
            while stack:
```

```

        r, c = stack.pop()
        for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
            nr, nc = r + dr, c + dc
            if 0 ≤ nr < rows and 0 ≤ nc < cols and grid[nr][nc] == "1":
                grid[nr][nc] = "0"
                stack.append((nr, nc))

    return count

if __name__ == "__main__":
    # Anil's building: three systems, plus a disconnected outdoor tap.
    building = build(9, [(0, 1), (1, 2), (3, 4), (5, 6), (6, 7), (7, 5)])
    print("count :", count_components(building))
    print("labels:", label_components(building))
    print("sizes :", component_sizes(building))

    islands = [
        list("11000"),
        list("11000"),
        list("00100"),
        list("00011"),
    ]
    print("islands:", count_islands(islands))
    print("after  :", ["".join(row) for row in islands])

```

Running it:

```

count : 4
labels: {0: 0, 1: 0, 2: 0, 3: 1, 4: 1, 5: 2, 6: 2, 7: 2, 8: 3}
sizes : [3, 3, 2, 1]
islands: 3
after  : ['00000', '00000', '00000', '00000']

```

Two things to look at. Vertex `8` gets its own label, `3`, and shows up in the sizes as a group of one — the isolated vertex handled correctly, which only works because `build` used `range(n)`.

And the last line: the grid is now entirely water. The function destroyed its input. That is fine when the caller does not need it and is a real problem when it does — say so before you write it, and offer the separate `visited` grid as the alternative.

6 What it costs

Time.

the outer loop	V iterations
each vertex is flooded at most once	(shared seen set)
total flooding work	$V + 2E$

total	$O(V + E)$

The outer loop does not multiply anything. This is the point people get wrong when they first see it. The loop runs V times, but the body only does real work the first time it lands in each component, and across all components every vertex and every edge is touched exactly once. The other $V - (\text{number of components})$ iterations are a single in check.

So counting components costs exactly the same as one traversal. Not V traversals — one traversal’s worth of work, spread across several starts.

Put numbers on it:

V = 1,000,000 vertices, E = 3,000,000 edges, 40,000 components	
outer loop checks	1,000,000
flood work	1,000,000 + 6,000,000 = 7,000,000

total	~8,000,000 operations

Eight million steps, about a second in Python. The forty thousand components change nothing about the cost.

Space.

seen / label	V entries	$O(V)$
stack or queue	at most V	$O(V)$
the graph	$V + 2E$	$O(V + E)$

On a grid:

rows x cols cells	
V = rows x cols	
$E \leq 2 \times \text{rows} \times \text{cols}$	(4 neighbours each, each edge shared) $O(\text{rows} \times \text{cols})$
1,000 x 1,000 grid → 1,000,000 cells, ~3,000,000 steps	

Marking by overwriting the grid means $O(1)$ extra space for visitation instead of $O(\text{rows} \times \text{cols})$ — a million booleans saved, which is a real number worth stating.

Compared with the alternative. For a static graph:

traversal	$O(V + E)$ once
Union-Find	$O(E \times \alpha(V))$ to build, then $O(\alpha(V))$ per query alpha is the inverse Ackermann function: ≤ 4 for any input that fits in the universe, so effectively constant

Both are effectively linear, and for a one-shot count the traversal is simpler. **The comparison changes completely when edges arrive over time:**


```
m edges arriving one at a time, "how many components now?" after each
traversal per query      m x O(V + E)    = quadratic. Unusable.
Union-Find                m x O(alpha)    = effectively m operations
```

```
m = 100,000 edges, V = 100,000
traversal:  100,000 x 200,000 = 20,000,000,000 steps
Union-Find: 100,000 x ~4      = 400,000 steps
```

Fifty thousand times faster, and that gap is the entire reason Union-Find exists. Say it as “static graph → traversal, incremental edges → Union-Find” and you have answered the follow-up before it arrives.

7 The traps

Starting from one vertex

The near-miss, and the most common wrong answer to any “how many” graph question:

```
seen = set()
_flood(graph, 0, seen)
return len(seen)          # "the number of vertices"
```

On a connected sample it is right. On the real input:

```
>>> graph = build(9, [(0,1), (1,2), (3,4), (5,6), (6,7), (7,5)])
>>> len(flood_from_zero(graph))
3                                # the answer is 9
```

No error, and the sample in the problem statement almost certainly does not catch it. **Whenever the question contains “all”, “every”, “how many groups”, or “count the”, the outer loop is required.**

Resetting `seen` inside the loop

```
for start in graph:
    seen = set()                # ← inside
    if start in seen:          # always False now
        continue
    count += 1
    _flood(graph, start, seen)
```

```
>>> count_components_broken(building)
9
```

Nine components for a graph with four. The `in` check is now meaningless, so every vertex starts a new traversal and every traversal is counted. The answer is always `V`, which is plausible enough to survive a glance.

Building from the edges instead of the vertex count

```
graph = defaultdict(list)
for a, b in edges:
    graph[a].append(b)
    graph[b].append(a)
return count_components(graph)
```

```
>>> count_components(from_edges_only(9, [(0,1),(1,2),(3,4),(5,6),(6,7),(7,5)]))
3                                     # vertex 8 does not exist, so it is not counted
```

Vertex `8` has no edges, never becomes a key, and is silently dropped. On “six people, two friendships, how many friend circles”, the answer is six and this returns two. **When the problem gives you `n`, build from `range(n)`.**

Marking on pop in the grid version

```
while stack:
    r, c = stack.pop()
    if grid[r][c] == "0":
        continue
    grid[r][c] = "0"           # mark on pop
    for dr, dc in ...:
        stack.append((nr, nc)) # push without checking
```

Correct answers, and on a large all-land grid:

```
1,000 x 1,000 grid, all land
mark on push:  stack peak ~ 2,000 entries
mark on pop:   stack peak ~ 2,000,000 entries
```

```
MemoryError
```

Each cell gets pushed once per neighbour that sees it — up to four times — and on a million-cell grid that is four million stack entries.

Recursion on a grid

```
def sink(grid, r, c):
    grid[r][c] = "0"
    for dr, dc in ((-1,0),(1,0),(0,-1),(0,1)):
        if in_bounds and grid[r+dr][c+dc] == "1":
            sink(grid, r+dr, c+dc)
```

```
Traceback (most recent call last):
  File "islands.py", line 8, in sink
    sink(grid, r + dr, c + dc)
[Previous line repeated 993 more times]
RecursionError: maximum recursion depth exceeded
```

A $1,000 \times 1,000$ grid of all land is a single island whose DFS depth is a million. This is the standard failure on the larger island test cases, and raising the recursion limit turns it into a segmentation fault. Write the iterative version.

Counting components on a directed graph

```
count_components(directed_graph)    # what does this even mean?
```

It runs. It returns a number. The number is neither the weakly connected count (because you did not double the edges) nor the strongly connected count (which needs a different algorithm entirely). It is the number of times a forward-only traversal happened to start, which depends on the iteration order of the dictionary.

Ask which one is meant. If it is weak, symmetrise the edges first. If it is strong, that is a different algorithm and you should say so.

8 In the interview

How it gets asked

The word “component” is rare. These mean it:

- “How many connected components does this graph have?” — the direct version.
- “Count the number of islands.” — the grid version, and the most-asked graph question anywhere.
- “How many friend circles / provinces / groups are there?”
- “Given accounts with shared emails, merge them.” — components after a modelling step.
- “A cable is cut. Does the network split?” — components before and after.
- “What is the size of the largest group?” — labels rather than a count.

The first ninety seconds

“This is connected components, and the algorithm is the traversal plus four lines — but the four lines are where it goes wrong, so let me be explicit about them.

I loop over **every** vertex. If a vertex has not been seen, I increment the count and run a traversal from it, marking everything reachable. The `seen` set is declared **outside** the loop and shared across every traversal, which is what stops a piece being counted more than once.

Two mistakes I would flag before writing. First, starting from a single vertex — that only finds the component containing it, and the sample input for this kind of problem is almost always connected, so nothing tells you. Second, if I build the adjacency structure from the edge list alone, any vertex with no edges never appears, and an isolated vertex is a component of size one. So I build from `range(n)`.

Cost is $O(V + E)$ — the same as a single traversal, not V traversals, because the shared `seen` set means every vertex and edge is touched exactly once across all the runs. Space is $O(V)$.

I would use DFS with an explicit stack rather than recursion, because if this is the grid version a thousand-by-thousand grid of all land is a single component a million deep, and recursion dies there.

One question: is the graph directed? If it is, ‘components’ is ambiguous — weakly connected means ignoring the arrows, strongly connected means every vertex reaches every other following them, and those are different algorithms.”

The follow-ups

“Why is this $O(V + E)$ and not V times $O(V + E)$?”

“Because the outer loop runs V times but only does work the first time it lands in each component.

Concretely: the loop body is an `in` check, which is $O(1)$, and then a traversal — but the traversal only happens if the vertex is unseen. Once a component has been flooded, every other vertex in it fails the check and costs one lookup.

So the total flooding work across all the starts is: every vertex is pushed exactly once, and every edge is examined exactly twice, once from each end. That is $V + 2E$ regardless of how the vertices are distributed among components. Add the V cheap checks from the outer loop and it is still $O(V + E)$.

The thing that makes it work is the shared `seen` set. If I reset it per traversal, then it genuinely would be V traversals and the cost would be $O(V \times (V + E))$ — and it would also give the wrong answer, which is the more obvious symptom.”

“Edges arrive one at a time and I need the count after each. Now what?”

“Then the traversal is the wrong tool and I would switch to Union-Find, and I would justify it with the arithmetic.

Re-running the traversal after each edge is m traversals, so $O(m \times (V + E))$. With a hundred thousand edges and a hundred thousand vertices that is about twenty billion steps. Not viable.

Union-Find keeps each vertex pointing at a representative for its group. Adding an edge means finding both endpoints’ representatives and, if they differ, pointing one at the other and decrementing a component counter. With path compression and union by rank, each operation is effectively constant — the true bound is inverse Ackermann, which is at most 4 for any input that will ever exist.

So the whole sequence is about $m \times 4$ operations, four hundred thousand instead of twenty billion.

The rule I would state is: **static graph, count once** → **traversal**. **Edges arriving over time, count as you go** → **Union-Find**. And the converse is worth saying too — Union-Find cannot easily handle *removing* an edge, so if edges are being deleted, neither works well and you are into much harder territory.”

“The grid is a thousand by a thousand and your solution crashes.”

“Recursion depth. A thousand-by-thousand grid that is all land is one island, and a depth-first search through it goes a million calls deep. Python’s usable limit is under a thousand frames.

Two things I would change. Rewrite the flood iteratively with an explicit stack — same algorithm, no limit. And mark cells as visited when I **push** them, not when I pop them, because on a grid each cell has up to four neighbours that could push it, so marking on pop lets the stack grow to four million entries on a million-cell grid.

I would not raise the recursion limit. Past roughly a hundred thousand frames Python exhausts the C stack and you get a segmentation fault instead of a `RecursionError` — no traceback, nothing to debug. That is a worse failure, not a fix.

On memory: I mark visited cells by overwriting the land with water, which costs nothing and destroys the caller’s grid. If the grid must be preserved I keep a separate visited array, which is a million booleans — about a megabyte as a `bytearray`. I would ask which the caller wants rather than silently mutating.”

“How many components are there if I remove this vertex?”

“Recompute is the honest first answer: delete the vertex and its edges, run the count again, $O(V + E)$. Fine if the question is asked once.

If they are asking it for *every* vertex, that is $V \times O(V + E)$ and there is a much better answer: **articulation points**, found by a single DFS. A vertex is an articulation point — a cut vertex — if removing it increases the component count, and there is a linear-time DFS that finds all of them at once by tracking, for each vertex, the earliest ancestor reachable from its subtree.

The same machinery finds **bridges** — edges whose removal disconnects the graph — which is the version of this question asked about cables rather than routers.

I would not write that from memory unprompted; it is a known algorithm with a couple of fiddly details around the root vertex. But naming it and saying it is one linear DFS rather than V of them is the answer to ‘can you do better than recomputing’.”

The model answer

“A company has a list of employees. Each employee has an ID and a list of the IDs of people they have worked with directly. Teams are defined as groups of people connected through shared projects. How many teams are there, and how large is the biggest one?”

“Let me do the modelling in one sentence and then be careful about the two things this problem is actually testing.

A vertex is an employee. An edge joins two employees who have worked together directly. Undirected — working together is symmetric — and unweighted. A team is a connected component, and the questions are the count and the largest size.

So: label rather than count. I keep a dictionary from employee to team number, filled in during the traversal, and increment the team number each time the outer loop starts a fresh flood. The count is the number of distinct labels and the largest team is the most common one — a `Counter` over the values. Both questions from one pass.

The first thing this problem is testing is the isolated employee. Someone who has never worked with anyone is a team of one. If I build the adjacency structure only from the collaboration lists, that person never appears and I undercount both answers. So I initialise from the full employee list. In an interview I would say this out loud because it is the difference between the sample passing and the hidden test passing.

The second is the symmetry of the input. Real data of this shape is often one-directional — A’s list mentions B but B’s list does not mention A, because somebody filled in a form. If I trust the input to be symmetric and it is not, I get too many teams. So I add every edge in both directions when building, regardless of what the input claims, which is one extra line and removes the whole class of problem. I would mention that I am doing it defensively rather than because the spec said so.

Cost: $O(V + E)$ time and $O(V)$ space. With ten thousand employees averaging thirty collaborators, that is $10,000 + 300,000$ — about a third of a million steps, instant.

Iterative traversal, not recursive, because a company where everyone is connected in a long chain of collaborations is a perfectly plausible input and would be ten thousand frames deep.

The follow-up I would expect is ‘now people join and collaborations get added over time, keep the counts live’. That is Union-Find rather than a traversal: each new collaboration is two finds and possibly a union, effectively constant, and I keep a running component count and a size per representative. Re-running the traversal per edge would be quadratic and would not survive.

And one thing I would raise as a product question rather than an engineering one: this definition makes everyone connected through any chain into one team, so a single person who has worked with two departments merges them. If the real question is ‘who works closely together’, components are too blunt and the answer is a clustering measure rather than connectivity. I would ask before building, because the two give very different answers on real collaboration data.”

9 Recall card

Components are the traversal plus four lines: loop over **every** vertex, and when one is unseen, count a new component and flood from it. The **seen** set lives outside the loop — resetting it inside returns V .

Cost is $O(V + E)$, the same as one traversal, not V of them, because the shared **seen** set means every vertex and edge is touched once across all the starts.

Label instead of counting when you need sizes or “same group?” — one dictionary does the seen set, the labels and the count.

Build from `range(n)`, not from the edge list. A vertex with no edges is a component of size one, and a `defaultdict` built from edges silently loses it.

Static graph → traversal. **Edges arriving over time** → Union-Find ($m \times 4$ instead of $m \times (V + E)$). And on a directed graph, “components” is ambiguous — ask whether they mean weakly or strongly connected.

1 What this is, and why they ask it

A **message queue** is a place to put work so that somebody else does it later. The caller writes a message, gets an acknowledgement immediately, and goes away. A separate process picks the message up and does the actual job.

That one change — the caller not waiting — is the most consequential thing you can do to an architecture. It turns a four-second signup into a two-hundred-millisecond signup. It lets a service survive a traffic spike that would otherwise have killed it. It means a dependency being down stops being an outage and becomes a backlog.

And it costs you a great deal. You lose the answer, so the user must find out some other way. You lose ordering unless you work for it. You gain duplicates, which every consumer must now handle. And you gain a debugging story where the interesting event happened somewhere else, four seconds ago, in a process nobody was watching.

They ask this because the phrase “just put a queue in front of it” is the most common half-answer in system design, and the follow-ups find out whether you know what you traded. This is also the first day of a new phase: everything up to now was about how distributed systems fail. From here you are assembling the actual components — queues, streams, object stores, search indexes, caches — into designs.

By the end of this lesson you can say exactly which work belongs off the request path, compute what asynchrony buys in latency and in capacity, name the four things you give up, size a queue and its consumers, and answer the question that decides whether the design is any good: **what does the user see while the work has not happened yet?**

2 The story

Iqbal's tailoring shop is nine feet wide and has been on that corner since his father's time.

For most of the year it is quiet. Somebody brings a shirt to be altered, Iqbal measures, and if it is small he does it there — the man waits twenty minutes on the bench by the door and takes it with him.

Then there are the six weeks before Diwali.

In those six weeks he takes something like sixty orders a day, and he can stitch about twenty-five. That arithmetic does not work and everybody who has ever run a small shop knows exactly what it feels like.

His father's way was to take the work and be vague. "Come next week." Then next week it was not ready, and the man came back on Thursday and it was still not ready, and there was an argument at the counter while four other people waited.

What Iqbal does is different and it is the whole thing.

He has a pad of numbered tags, the kind with a wire loop. When you bring cloth in, he measures — that takes four minutes — writes the number on a slip, ties the tag to the bundle, and says a date. Not "next week". A date, out loud, that he has worked out from how many bundles are already on the shelf behind him.

Then you leave. The shop is empty behind you in four minutes instead of forty, and the next person is at the counter.

The shelf behind him is the whole system. In the quiet season it has two or three bundles on it. In the third week of October it has a hundred and forty, and when he looks at it he knows immediately that the date he is telling people has to move from four days to nine.

Two things about that shelf that he has learnt the hard way.

The first is that the shelf can save him, but only for so long. A busy Saturday puts thirty bundles on it and he clears them over the following week. But if every day is a busy Saturday, the shelf does not help at all — it just grows, and the date he quotes goes from nine days to twenty, and at some point people stop coming back for the bundle at all.

The second is that when something goes wrong, it goes wrong quietly. In 2019 a bundle slipped down behind the shelf and stayed there for five months. Nobody noticed. The man who brought it in did not come back to ask, because he had assumed it was lost and bought something else, and Iqbal did not know it was missing because it was not in front of him. That is the difference between the man on the bench and the bundle on the shelf. The man on the bench, you cannot forget.

3 The idea in plain English

Iqbal's shelf is a message queue, and everything about queues is on it.

Synchronous means the caller waits. The man on the bench for twenty minutes. In code: your service calls another service, blocks, and returns to the user only when the whole thing is done. The user's response time is the sum of everything you did.

Asynchronous means the caller hands over the work and leaves. Write the message, get an acknowledgement, respond. The user's response time is the time to write the message — a millisecond or two — and nothing else.

The queue is the shelf. A **queue** is a durable, ordered store of messages that a producer writes to and a consumer reads from. Durable matters: if the shop burns down the bundles are gone, so a real queue writes messages to disk and replicates them before acknowledging.

The tag with a date on it is what the user gets instead of an answer. This is the part that separates a design from a diagram. If the user no longer waits for the work, they must find out about it some other way — a status they can poll, a notification, an email, a page that says “processing”. **A design that moves work into a queue and does not say what the user sees has not finished.**

Decoupling in time is the first thing you buy. The producer and the consumer no longer have to be running at the same moment. Iqbal takes orders when the shop is open and stitches when it is quiet. If the stitching stops entirely for a day, the counter keeps working. **A dependency being down becomes a backlog rather than an outage**, and that is worth more than the latency improvement.

Absorbing a burst is the second. The shelf holds thirty bundles from a busy Saturday and they are cleared over the week. In a system, a spike of ten thousand requests a second against a consumer that can do a thousand a second is fine for a while: the queue holds the difference. **This is called buffering, and it only works for bursts, not for sustained overload.** If the average arrival rate exceeds the average processing rate, the queue grows without limit and the delay grows with it. Iqbal’s nine days becoming twenty.

Smoothing the load is the third. The consumer processes at its own steady pace instead of at whatever rate requests happen to arrive, which means you size it for the *average* rather than for the peak. That is often a five-to-ten-times reduction in the machines you need.

Now the four things you give up, and you must be able to name all four.

One: you lose the answer. The caller gets an acknowledgement, not a result. If the work fails, the caller has already gone. Everything about error handling changes — there is nobody to return a `400` to.

Two: you gain duplicates. Queues deliver **at-least-once**: if the consumer does not acknowledge in time, the message comes back. That is not a bug, it is the design, and it means **every consumer must be idempotent** — [day 122](#)’s lesson, now a hard requirement rather than good practice.

Three: you lose ordering, unless you pay for it. With ten consumers reading from one queue, message 5 can finish before message 3. Ordering is only available *per partition* or *per key*, and getting it means routing related messages to the same place, which limits how much you can parallelise.

Four: debugging becomes archaeology. The thing that went wrong happened in another process, at another time, triggered by a message you cannot see any more. This is Iqbal's bundle behind the shelf: nothing failed loudly, it just never happened, and nobody found out for five months. **The answer is a correlation ID on every message and a dead-letter queue**, and both belong in the design from the start rather than after the first incident.

The rule for what goes in a queue, in one line: *does the user need the result to continue?* Sending the welcome email — no. Resizing the uploaded image — no. Charging the card — usually yes, though “accepted, processing” is a legitimate product decision. Validating the password — obviously yes.

4 The picture

The same signup, both ways:

SYNCHRONOUS – the user waits for everything

```
user → API ---> write user row      80 ms
                → send welcome email 1,400 ms ← an external service
                → create Stripe cust   600 ms ← another one
                → warm the cache       200 ms
                → index for search     120 ms
      <---
response                                     2,400 ms
```

and if the email provider is down, the signup FAILS.

ASYNCHRONOUS – the user waits for what matters

```
user → API ---> write user row      80 ms
                → publish 4 messages  6 ms
      <---
response                                     86 ms
```

```
queue → worker: email
      → worker: Stripe
      → worker: cache
      → worker: search
```

and if the email provider is down, the message waits.
The signup succeeded.

What to notice. The latency went from 2,400 ms to 86 ms — a factor of 28 — but the more important change is the last line of each block. In the synchronous version, the availability of the signup is the *product* of five services' availability. In the asynchronous version it is one service's.

Now the shape of a queue system, with the parts that are always there:

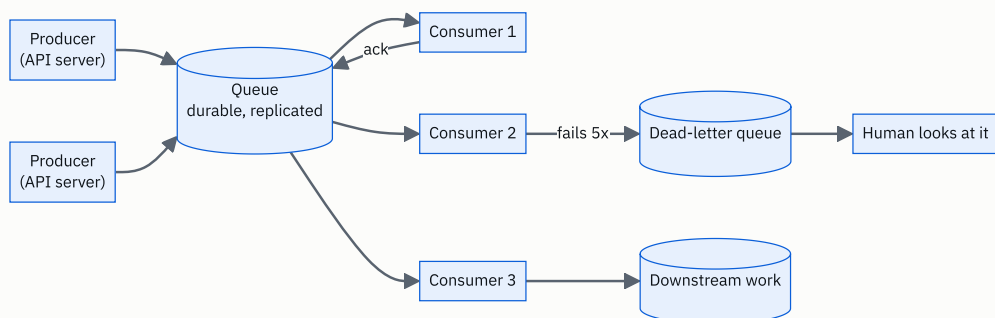


FIGURE 129.2

What to notice. The acknowledgement arrow going back from the consumer is the whole delivery guarantee: a message is only removed once a consumer says it is done. And the dead-letter queue is not optional decoration — without it, a message that always fails is retried forever and blocks or floods the system.

And the thing that decides whether the design works, drawn as a rate:

```
arrival rate L (messages per second)
service rate M (messages per second, all consumers combined)

L < M    queue stays near empty. Delay = processing time.    HEALTHY
L = M    queue length is unstable, delay grows unpredictably. FRAGILE
L > M    queue grows without limit. Delay grows without limit. BROKEN

a burst of B extra messages with L < M:
  drain time = B / (M - L)

30,000 message burst, M = 1,000/s, L = 800/s
  drain time = 30,000 / 200 = 150 seconds
```

What to notice. The drain time depends on the *headroom*, $M - L$, not on M . A consumer running at 95% utilisation has almost no headroom and takes an enormous time to recover from a burst. That is why queue systems are sized for $M \approx 2 \times L$, not $M \approx L$.

SQS defaults to 30 seconds and allows up to 12 hours. The bug is a job that takes longer than the timeout:

```
visibility timeout    30 s
job duration          45 s

t=0   consumer A receives, starts work
t=30  message becomes visible again
t=31  consumer B receives the SAME message, starts the same work
t=45  A finishes and acknowledges
t=76  B finishes and acknowledges (the message is already gone)

the work was done twice, and nothing reported an error
```

Two fixes, and you should name both. Extend the visibility while working — SQS calls this `ChangeMessageVisibility`, and it is the same watchdog pattern as the distributed lock on [day 127](#). Or make the work idempotent so the duplicate is harmless, which you need anyway.

Dead-letter queues

A message that fails is retried. A message that fails *every time* — malformed JSON, a reference to a deleted record, a bug — is retried forever, consuming capacity and, in an ordered queue, blocking everything behind it. That is a **poison message**.

```
maxReceiveCount: 5    → after 5 failed attempts, move to the DLQ
```

The dead-letter queue is where those go. Three things to say about it:

- **Alert on its depth, not just its existence.** A DLQ nobody looks at is a data-loss mechanism with extra steps.
- **Keep the failure reason** with the message, or triage is guesswork.
- **Make replay easy.** Most DLQ contents are fixable — a bug is deployed, and the messages should go back through. If replaying requires a hand-written script each time, it will not happen.

Ordering, and what it costs

Ordering is per partition, never global. Kafka hashes the message key to choose a partition, and messages with the same key land on the same partition and are consumed in order by one consumer.

```
key = user_id    → all events for one user are ordered
                  → parallelism is limited to the number of partitions
                  → a hot key (one very active user) becomes a hot partition
```

SQS FIFO queues use a `MessageGroupId` the same way, and cap throughput at 300 messages per second per group (3,000 with batching) — against effectively unlimited for a standard queue. **That number is the price of ordering** and it is worth quoting.

Backpressure, and what to do when the queue grows

A queue absorbing a burst is working. A queue growing steadily is a system that is broken and has not noticed yet. Four responses:

1. **Scale the consumers.** The correct answer when the work is parallelisable. Autoscale on queue depth or on message age, not on consumer CPU — CPU looks fine on a consumer that is keeping up with a queue it is not draining.
2. **Shed load at the producer.** Reject new work rather than accept work you cannot do. A bounded queue with a rejection at the edge is more honest than an unbounded one with an eight-hour delay.
3. **Prioritise.** Separate queues for different classes, so a flood of low-value work does not delay password-reset emails.
4. **Fix the consumer.** Often the real answer, and often the slowest.

The metric that matters is message age, not queue depth. A queue with a million messages that are all two seconds old is healthy. A queue with fifty messages that are four hours old is broken. Alert on the oldest message's age.

Real systems, briefly

- **SQS.** Managed, effectively unlimited standard queues, at-least-once, no ordering; FIFO variant with ordering and a throughput cap. Simplest correct choice on AWS.
- **RabbitMQ.** Rich routing — exchanges, bindings, topic patterns — plus priorities and delays. More operational work; the right answer when routing logic is complex.
- **Kafka.** A log, not a queue: retention, replay, consumer groups, ordering per partition, very high throughput. Chosen when events have more than one consumer or when replay matters. [Day 130](#) is entirely about it.
- **Redis lists / Streams.** Fast and simple; Redis Streams add consumer groups and acknowledgements. Durability depends on your Redis persistence settings, which is the caveat.
- **Celery, Sidekiq, BullMQ.** Application-level task queues on top of one of the above. What most teams actually use for “send this email”.

6 The numbers

What asynchrony buys in latency. The signup from section 4:


```
synchronous
  user row write      80 ms
  welcome email      1,400 ms
  payment customer    600 ms
  cache warm          200 ms
  search index        120 ms
  -----
                    2,400 ms
```

```
asynchronous
  user row write      80 ms
  4 queue publishes    6 ms
  -----
                    86 ms      → 28x faster
```

What it buys in availability. Synchronous couples them multiplicatively:

```
each service      99.9% available
five in series    0.999^5 = 99.5%
                  → 3.6 hours of downtime a month
```

```
async: only the user row and the queue are in the path
      0.999 x 0.9999 = 99.89%
      → 48 minutes a month
```

Four and a half times less downtime, from moving four calls off the request path.

What it buys in capacity. Traffic that peaks at five times its average:

```
average      1,000 requests/s
peak         5,000 requests/s
work per request 50 ms of CPU

synchronous: size for the peak
  5,000 x 0.05 = 250 cores

asynchronous: size for the average, buffer the peak
  1,000 x 0.05 = 50 cores
  -----
                5x fewer machines
```

Queue sizing. With L arrivals and M service capacity:

```
peak burst      5,000/s for 60 s    = 300,000 messages
steady capacity 1,000/s
arrival during drain 1,000/s
                  → net drain rate = 1,000 - 1,000 = 0 ...
```

That last line is the trap, and it is why you size for headroom:

consumers at 2x average	$M = 2,000/s$
drain rate	$2,000 - 1,000 = 1,000/s$
drain a 300,000 backlog	$300,000 / 1,000 = 300 \text{ seconds}$
peak queue depth	300,000 messages
storage at 2 KB each	600 MB

Five minutes to recover from a one-minute spike, and six hundred megabytes of queue. Both numbers are fine; both are worth stating rather than assuming.

How many consumers? Little's Law again:

target throughput	1,000 messages/s
processing time	200 ms each
concurrency needed	$1,000 \times 0.2 = 200$ concurrent workers
per machine (50 threads)	$200 / 50 = 4$ machines

Cost of a managed queue. SQS at roughly \$0.40 per million requests, and each message is at least two requests (send, receive) plus a delete:

100,000,000 messages/day	
x 3 API calls	$= 300,000,000 \text{ requests/day}$
	$= 9,000,000,000/\text{month}$
x \$0.40 per million	$= \$3,600/\text{month}$

Batching is the lever. SQS allows 10 messages per API call:

with batches of 10	$= \$360/\text{month}$
--------------------	------------------------

A factor of ten for one parameter, and it is the first thing to check on a queue bill.

Ordering's cost, in throughput:

SQS standard	effectively unlimited
SQS FIFO	300 messages/s per message group 3,000/s with batching
Kafka	ordering per partition parallelism capped at the partition count 12 partitions → at most 12 consumers doing useful work

Latency added by the hop:

publish to SQS	5 - 20 ms
publish to Kafka	1 - 5 ms (batched, async)
end-to-end delivery	50 ms - seconds, depending on consumer lag

So the user's request gets *faster* and the work itself gets *slower* — from 1.4 seconds to potentially several seconds before the email actually goes. **That is the trade in one sentence, and it is fine for an email and not fine for a login.**

7 The trade-offs

You trade the answer for the latency. The caller gets an acknowledgement, not a result. Every design that does this owes an answer to “how does the user find out?” — a status endpoint they poll, a websocket push, an email, or a page that just says “we are processing this”. Choosing badly here is what makes an async design feel broken even when it works.

You trade correctness-by-default for idempotency-by-discipline. At-least-once delivery is not a configuration you can turn off. Every consumer must handle the same message twice, forever, including the ones written next year by someone who has not read this. That is an ongoing engineering constraint on the whole team, not a one-time implementation cost, and it is the reason to keep the number of queues small.

You trade ordering for parallelism. Ten consumers on one queue means no order. Ordering means routing by key to a single consumer per key, which caps your parallelism at the number of keys or partitions and makes a hot key a hot partition. SQS FIFO's 300 messages per second per group is what that costs, concretely.

You trade a loud failure for a quiet one. A synchronous failure returns a `500` to a user who complains. An async failure is a message in a dead-letter queue at three in the morning that nobody has alerted on. **This is the single biggest practical cost** and the mitigations are unglamorous: correlation IDs, DLQ depth alerts, oldest-message-age alerts, and a replay path that someone has actually tested.

A queue does not create capacity. It absorbs bursts. If the average arrival rate exceeds the average service rate, the queue grows without bound and the delay grows with it, and the system is broken in a way that looks healthy on every dashboard except message age. **“Put a queue in front of it” is not a fix for an undersized consumer,** and saying that out loud is worth marks.

And a queue is a component that can fail. It is durable, replicated and highly available, and it is still another thing to operate, another dependency in the write path, and another bill. For a small system, a database table with a status column and a polling worker is a perfectly good queue and has none of the operational overhead. **The transactional-outbox version of that is often strictly better than a real queue,** because the write and the enqueue are then in the same transaction.

When would I not use a queue? When the user needs the result to continue — a login, a search, a price. When the work takes less time than publishing the message, which is common for small database writes. When strict global ordering is required and the volume is low, where a single synchronous path is simpler and correct. And when the team is small and the work is rare, because the operational cost of a queue is paid every day and the benefit is paid only under load.

8 In the interview

How it gets asked

- *“The signup email takes four seconds. Fix the latency.”* — the direct version.
- *“How would you handle a traffic spike ten times normal?”*
- *“What happens if the consumer crashes halfway through?”*
- *“How do you guarantee the messages are processed in order?”*
- *“The queue is growing. What do you do?”*
- *“Queue or Kafka?”* — a real question with a real answer.

The first ninety seconds

“Four seconds of that is work the user does not need the result of. The account row is written in eighty milliseconds; the email, the payment-provider customer, the cache warm and the search index are the other 2.3 seconds, and none of them changes what I return.

So: write the user row synchronously, publish messages for the other four, return. The response goes from about 2.4 seconds to under a hundred milliseconds.

The bigger win is availability rather than latency, and I would lead with it. Synchronously, my signup’s availability is the product of five services — five at three nines is 99.5%, which is three and a half hours a month. Asynchronously only the database and the queue are in the request path, so it is about 99.9%, and the email provider being down becomes a backlog instead of a failed signup.

Now what I am giving up, because ‘put a queue in front of it’ is only half an answer.

I lose the result, so I need to decide what the user sees. For a welcome email, nothing — they will never know. For anything they might ask about, a status they can check.

I get duplicates. Queues are at-least-once, so the consumer will occasionally see the same message twice — most often because it crashed after doing the work and before acknowledging. Every consumer has to be idempotent: the email sender keys on `(user_id, email_type)` and does nothing if a row already exists. That is a permanent constraint on everyone who writes a consumer, not a one-off.

I lose ordering across consumers, which for these four jobs does not matter, and I would say why rather than ignore it.

And failures become invisible, so a dead-letter queue with an alert on its depth and a correlation ID on every message go in from the start, not after the first incident.

Do you want me to size the consumers, or go into what happens when the queue starts growing?”

The follow-ups

“The consumer crashed halfway through. Walk me through it.”

“The message was never acknowledged, so the queue redelivers it. That is at-least-once working as designed and I would not try to prevent it.

Mechanically: on receipt the message became invisible for a visibility timeout — 30 seconds by default on SQS. The consumer crashed, no acknowledgement arrived, the timer expired, the message became visible again, and another consumer picked it up.

What matters is that the work must be safe to repeat. The consumer’s first action is a conditional insert on the message’s ID — `INSERT ... ON CONFLICT DO NOTHING` — and if that conflicts, the job is already done and I acknowledge and move on. Half-done work is the harder case, and the answer is to design each consumer’s writes as upserts keyed by the message ID, so a partial run followed by a full run is indistinguishable from one clean run.

The related bug worth naming: if the job legitimately takes longer than the visibility timeout, the message is redelivered while the first consumer is still working, and the work happens twice with no crash involved. The fixes are to extend the visibility while working — the same watchdog pattern as a distributed lock — or to rely on idempotency, which I need anyway.

And after a few failed attempts the message goes to a dead-letter queue rather than being retried forever, because a malformed message will fail identically every time and will otherwise consume capacity indefinitely.”

“How do you guarantee ordering?”

“Ordering is per key, never global, and I would ask what actually needs to be ordered before designing for it — usually it is ‘events for one user’ rather than ‘all events’.

Mechanically: partition by a key. In Kafka, the message key is hashed to a partition, and one consumer in a group owns a partition, so all messages with the same key are processed in order by one consumer. In SQS FIFO it is a `MessageGroupId` and the same idea.

Two costs. **Parallelism is capped by the number of partitions** — twelve partitions means at most twelve consumers doing useful work, and adding a thirteenth does nothing. And **a hot key becomes a hot partition**: if one user generates a third of the traffic, one consumer gets a third of the work and the rest idle.

SQS FIFO has a hard number worth quoting: 300 messages a second per group, 3,000 with batching, against effectively unlimited for a standard queue. That is what ordering costs.

The alternative I would offer is to not need it. If consumers are idempotent *and* commutative — or if the message carries a version and the consumer ignores anything older than what it has already applied — then out-of-order delivery is harmless and I keep unlimited parallelism. That is usually the better design and it is worth ten minutes of thought before accepting the partition cap.”

“The queue depth is growing. What do you do?”

“First, I would check the right metric. Depth alone does not tell me much — a million messages that are two seconds old is a healthy high-throughput queue. **The number that matters is the age of the oldest message.** Fifty messages that are four hours old is a broken system.

If the age is growing, the arrival rate exceeds the service rate and the queue will grow without limit. A queue absorbs bursts; it does not create capacity. So there are four responses.

Scale the consumers, and autoscale on queue depth or message age, not on consumer CPU — a consumer that is falling behind looks perfectly busy on CPU. This is the right answer when the work parallelises and the downstream can take it.

Shed load at the producer. If I genuinely cannot process it, accepting it and quoting a four-hour delay is worse than rejecting it now. A bounded queue with a clear rejection at the edge is more honest.

Prioritise. Split into separate queues by class, so a bulk import does not delay password-reset emails. One queue for everything means the least important work sets the latency of the most important.

Or fix the consumer, which is often the real answer and the slowest. I would look at what it waits on — usually a downstream service or an unbatched database write.

And I would size for headroom rather than for parity. Consumers at 95% utilisation take forever to drain a burst, because the drain rate is service minus arrival, not service. A one-minute spike of 300,000 messages at 2,000 a second capacity against 1,000 a second arrivals takes five minutes to clear; at 1,100 a second it takes fifty.”

“Queue or Kafka?”

“The question I would actually ask is whether the message is *work* or an *event*.

A task queue — SQS, RabbitMQ — is for work. ‘Send this email.’ One consumer does it, acknowledges, and the message is gone. Consumers compete, scaling is trivial, and there is no ordering to think about.

A log — Kafka — is for events. ‘User 4471 signed up.’ It stays for a retention period, several independent consumer groups can read it at their own pace, each tracking its own offset, and any of them can rewind and replay.

Three things push me to Kafka. **More than one consumer wants the same event** — email, analytics, and the recommendation model all care about a signup, and with a task queue I would be publishing three messages and keeping the list of interested parties in the producer. **Replay matters** — a bug in the analytics consumer is fixed by rewinding the offset and reprocessing a week, which a task queue simply cannot do. And **throughput is very high**, where Kafka’s sequential-write design is an order of magnitude cheaper.

Three things push me to SQS. Operational simplicity, because Kafka is a real cluster to run and SQS is an API call. Per-message retries and a dead-letter queue, which are built in for a task queue and are your own problem in Kafka. And genuinely independent tasks with no ordering requirement, where partitions buy nothing.

For a signup email in a system that is not already running Kafka, I would use a task queue and say so. For a stream of user activity that four teams want, Kafka.”

The model answer

“An image upload feature: users upload a photo, it needs to be resized into four sizes, scanned for inappropriate content, and appear in their gallery. Design it.”

“Let me split this by what the user needs before the response returns, because that decides the whole shape.

Synchronous: accept the bytes, validate them, store the original, create the record. Validation means size, type and dimensions — cheap checks that let me reject a bad upload with a real error message while the user is still there. The original goes to object storage, and the API returns a photo ID and a status of `PROCESSING`. That is maybe 200 milliseconds plus the transfer.

Asynchronous: everything else, as separate messages. Resize into four sizes, run the content scan, index for search, update the gallery cache. The resize is CPU-heavy and takes a few seconds for a large image; the content scan is an external API that might take two seconds or might be down. Neither belongs in the request.

What the user sees is the part I would spend most time on, because this is where async designs feel broken. The photo appears in their gallery immediately, showing the original with a blurred or low-quality placeholder, marked as processing. When the resize completes the client swaps in the real thumbnail — either by polling a status endpoint every couple of seconds, or over an existing websocket if there is one. The user never sees an empty screen and never waits for a spinner.

The content scan is the interesting case, and I would raise it rather than wait to be asked. It has to complete before the photo is visible to *anyone else*, even though it is asynchronous. So the photo is visible to its owner immediately and enters the public feed only after the scan passes. That is a product decision with a real risk window, and I would want it stated explicitly rather than implied by the architecture.

Idempotency: every message carries the photo ID, and every consumer's write is keyed on it — the resize writes to a deterministic object key, so doing it twice overwrites with identical bytes and is harmless. The content scan records its verdict with a conditional insert. Neither consumer cares how many times it runs, which is what makes at-least-once delivery a non-issue.

Ordering: the four resize jobs are independent, so no ordering needed and I get unlimited parallelism. The only ordering that matters is ‘scan before public visibility’, and I get that by making visibility depend on the scan's recorded verdict rather than on message order — which is much more robust than relying on the queue.

Sizing. Ten thousand uploads an hour is about three a second, peaking at maybe fifteen. Four resize jobs each is sixty messages a second at peak, at roughly two seconds of CPU each: Little's Law gives 120 concurrent workers, so about four

machines at 32 workers. I would autoscale on the age of the oldest message with a target of thirty seconds, and size the steady state at roughly twice the average so a burst drains in minutes rather than hours.

Failure handling. Five attempts, then the dead-letter queue, with the photo marked `FAILED` and the user told — ‘we could not process this image, please try again’ is far better than a photo that stays ‘processing’ forever, which is the default behaviour if nobody designs this. An alert on DLQ depth and on oldest-message age, and a correlation ID equal to the photo ID on every message so one log query reconstructs the whole lifecycle.

And the thing I would not do: put the original upload itself through the queue. The bytes are megabytes, queues are for small messages, and the object store already gives me durability. The message carries the object key, not the image.”

9 Recall card

Async means the caller hands over the work and leaves. Latency goes from the sum of everything to the time to write one message — 2,400 ms to 86 ms on a typical signup — and availability stops being the product of every dependency’s.

A queue absorbs bursts; it does not create capacity. If arrivals exceed service rate the queue grows without limit. Size consumers at about $2 \times \text{average}$, because drain rate is $M - L$, not M . Alert on **oldest-message age**, not depth.

Four things you give up: the answer (so design what the user sees); ordering (per key only — SQS FIFO caps at 300/s per group); at-most-once (delivery is at-least-once, so **every consumer must be idempotent**); and loud failures (so correlation IDs and a dead-letter queue from day one).

Visibility timeout is the classic bug: a job longer than the timeout gets redelivered while still running, and the work happens twice with no crash. Extend it while working, or rely on idempotency.

Work → task queue (SQS, RabbitMQ). Events → log (Kafka), when several consumers want the same thing or replay matters. And the rule for what goes in a queue: *does the user need the result to continue?*

DSA topic: Connected components **System design topic:** Message queues: why async changes everything

Code these, in this order

One rule for the whole set: **write the outer loop first, before the traversal**. Type `for v in range(n): if v not in seen:` and only then fill in the flood. Doing it the other way round is how the outer loop gets forgotten, and forgetting it is the single most common failure on this family of problems.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Number of Provinces	LeetCode 547 (Medium)	The outer loop, on a matrix input.
2	Number of Islands	LeetCode 200 (Medium)	The grid version, iteratively, marking on push.
3	Max Area of Island	LeetCode 695 (Medium)	Component <i>sizes</i> , not just a count.
4	Number of Connected Components in an Undirected Graph	LeetCode 323 (Medium)	Build from <code>range(n)</code> — the isolated vertex.
5	Accounts Merge	LeetCode 721 (Medium)	Components after a modelling step, with labels not counts.

On problem 4, break it on purpose

Solve it once building the adjacency structure from the edge list alone, and once from `range(n)`. Run both on `n = 6` with edges `[(0,1), (2,3)]`. Record both answers. Say in one sentence what the first one lost.

On problem 2, run all three failure modes

On a $1,000 \times 1,000$ grid of all land:

1. Recursive flood — record the error.
2. Recursive flood after `sys.setrecursionlimit(2_000_000)` — record what you get instead.
3. Iterative flood marking on **pop** — record the peak stack length and what happens.
4. Iterative flood marking on **push** — record the peak stack length.

Four results. Two of them are crashes and the difference between them is the point.

On problem 3, keep labels not counts

Return the sizes of every island, sorted, not just the maximum. Then answer “are cells (3,4) and (7,1) on the same island?” in one comparison. This is the version real problems want.

Then the Union-Find comparison

Take problem 4 and re-solve it with edges arriving one at a time, printing the component count after each edge. Do it twice: once by re-running your traversal after every edge, once with Union-Find. Time both at $n = 20,000$ and $m = 20,000$. Two numbers, and the ratio is the argument.

The algorithm drill

1. Write `count_components` from memory, and point at the two lines that are the whole lesson.
2. Say what happens if `seen` is created inside the loop, and what number you get.
3. Write the labelling version and say what the one dictionary is doing.
4. Say how you get “same group?” and “largest group” from labels.
5. Write the grid version and point at where the outer loop is.

The cost drill

1. Explain why the outer loop does not multiply the cost.
2. Derive $O(V + E)$ including the cheap checks.
3. Give the cost for $V = 10^6$, $E = 3 \times 10^6$, 40,000 components.
4. Give V and E for an $r \times c$ grid.
5. Say what marking by overwriting the grid saves, in bytes, on a $1,000 \times 1,000$ grid.

The Union-Find drill

1. State the rule for choosing between traversal and Union-Find.
2. Give the cost of each for m edges arriving one at a time.
3. Compute both at $m = 100,000$, $V = 100,000$.
4. Say what Union-Find cannot do that a traversal can.

The directed drill

1. Say why “components” is ambiguous on a directed graph.
2. Define weakly connected and how you would compute it.
3. Define strongly connected and say why it needs a different algorithm.
4. Say what you would ask the interviewer.

The break-it drill

Trigger each and record the exact output or error:

1. Flooding from vertex 0 only, on a four-component graph.
2. `seen` reset inside the outer loop.
3. Adjacency built from edges only, with an isolated vertex.
4. Grid flood marking on pop, on a million-cell all-land grid.
5. Recursive grid flood on the same grid.
6. `count_components` called on a directed adjacency list.

The what-goes-async drill

1. State the rule for what belongs in a queue, in one line.
2. Sort these: send welcome email, validate password, resize image, charge card, index for search, compute the search results.
3. For each one you moved async, say what the user sees instead.
4. Give one case where “charge the card” legitimately goes async, and what makes it legitimate.

The numbers drill

1. Compute the synchronous and asynchronous latency for the five-step signup.
2. Compute availability for five services at 99.9% in series, and for the async version.
3. Compute machines needed for a $5 \times$ peak, synchronous and asynchronous.
4. Compute drain time for a 300,000-message backlog at 2,000/s capacity and 1,000/s arrivals.
5. Recompute it at 1,100/s capacity and say what that shows.
6. Size consumers with Little’s Law for 1,000 msg/s at 200 ms each.
7. Compute the SQS bill for 100M messages a day, with and without batching.

The four-losses drill

Name each, then give its mitigation:

1. You lose the answer.
2. You gain duplicates.
3. You lose ordering.
4. Failures become quiet.

Then say which of the four is the biggest practical cost, and why.

The delivery drill

1. Define at-most-once, at-least-once and exactly-once in one line each.
2. Say which one every real queue actually does, and what that forces on you.
3. Explain the visibility timeout mechanism.
4. Give the timeline for a 45-second job under a 30-second timeout.
5. Give both fixes and say which you need anyway.

The ordering drill

1. Say why ordering is per key and never global.
2. Give the two costs of partitioning by key.
3. Quote SQS FIFO's throughput cap and what it is per.
4. Give the design that avoids needing ordering at all.

The backpressure drill

1. Say what metric you alert on, and why depth is the wrong one.
2. Name the four responses to a growing queue.
3. Say what you autoscale on and why not CPU.
4. Say why consumers at 95% utilisation are a problem, with the arithmetic.
5. Say what "put a queue in front of it" does not fix.

The choose-one drill

1. Give the three shapes of message system and one example each.
2. State the work-versus-event distinction in one sentence.
3. Give three reasons to pick Kafka and three to pick a task queue.
4. Say when a database table with a status column is the right queue.

The failure drill

For each, say what happens and what you would build:

1. A consumer crashes after doing the work and before acknowledging.
2. A job takes 45 seconds under a 30-second visibility timeout.
3. A malformed message that fails every time, in an ordered queue.
4. A dead-letter queue nobody has alerted on.
5. One very active user's key, in a partitioned topic.
6. Arrival rate 1,100/s against a consumer capacity of 1,000/s, for an hour.
7. A photo stuck in `PROCESSING` forever because its message went to the DLQ.

Two of the seven are design omissions rather than failures. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *How many connected components does this graph have?* The outer loop with `seen` shared outside it, why that is `O(V + E)` and not `V` traversals, building from `range(n)` for the isolated vertex, and the Union-Find rule offered before it is asked.
2. *Count the number of islands.* Cells as vertices and adjacency as edges, the double loop as the outer loop, iterative with marking on push, the two crash modes named, and the mutation of the caller's grid flagged.
3. *The signup email takes four seconds. Fix the latency.* What the user actually needs before the response, the latency and availability numbers, then the four losses named unprompted — with what the user sees, idempotency, ordering, and the DLQ.

Before you move on

- ☐ I write the outer loop before the traversal.
- ☐ I know `seen` lives outside the loop and what number I get if it does not.
- ☐ I can explain why the outer loop does not multiply the cost.
- ☐ I can derive `O(V + E)` for the whole component count.
- ☐ I build from `range(n)` and know which vertex I would otherwise lose.
- ☐ I can write the labelling version and get sizes and “same group?” from it.
- ☐ I can write the grid version iteratively.
- ☐ I mark grid cells on push and know the stack size if I do not.
- ☐ I know what recursion does on a million-cell island, and what raising the limit does.
- ☐ I know marking by overwriting mutates the caller's input, and I say so.
- ☐ I can state the traversal-versus-Union-Find rule and back it with numbers.
- ☐ I know “components” is ambiguous on a directed graph, and I ask.
- ☐ I can state the rule for what belongs in a queue.
- ☐ I can compute the latency and availability wins with numbers.
- ☐ I can compute the machine-count win from buffering a peak.
- ☐ I can name all four things async costs me.
- ☐ I always say what the user sees while the work is pending.
- ☐ I know at-least-once is not optional and what it forces on every consumer.

- [] I can explain the visibility timeout and its classic bug.
- [] I know both fixes and which one I need regardless.
- [] I know ordering is per key and can quote FIFO's throughput cap.
- [] I can give the design that removes the need for ordering.
- [] I alert on message age, not queue depth.
- [] I autoscale on queue depth or age, not consumer CPU.
- [] I can compute drain time and explain the headroom argument.
- [] I know a queue does not create capacity.
- [] I can distinguish work from events and pick the right system.
- [] I know when a database table is a good enough queue.
- [] I answered all three questions above out loud.

Grids are graphs: islands and flood fill

DATA STRUCTURES AND ALGORITHMS

Grids are graphs: islands and flood fill

SYSTEM DESIGN

Kafka, explained

1 What this is, and why they ask it

A grid is a graph. Every cell is a vertex, and every pair of touching cells is an edge. That sentence is the entire lesson, and once you have it, every grid problem in every interview becomes a traversal you already know how to write.

The reason it needs its own day is that grid problems do not look like graph problems. There is no edge list, no adjacency structure, nothing to build. The graph is **implicit** — the neighbours of a cell are computed from its coordinates, not looked up — and a large number of candidates never make the connection at all. They write nested loops and clever conditions and get lost, when the answer was BFS with a four-line neighbour function.

They ask grid problems constantly, more than any other kind of graph problem. Number of Islands is probably the single most-asked graph question in the industry. Flood fill, rotting oranges, surrounded regions, maximum island area, shortest path in a maze, walls and gates — all the same shape, all worth about twenty minutes, all solved by the same twenty lines.

There is also one specific detail that decides whether your code works, and it is not the traversal: **the bounds check**. Off the grid by one cell is an `IndexError` if you are lucky and a wrong answer if you are not, because Python's negative indices wrap around silently.

By the end of this lesson you can write the neighbour function from memory, choose between BFS and DFS for a grid on the right grounds, handle four-way and eight-way adjacency, recognise the multi-source variant, and name the two ways grid solutions crash on large inputs.

2 The story

It rained hard on Tuesday night and Ramesh goes up to the terrace on Wednesday morning to push the water off, the way he does after every serious rain.

The terrace is about forty feet by twenty-five, done in grey tiles, and it is not flat. It has never been flat. There is a slope towards the drain in the far corner that works reasonably well over most of the surface, and then there are the parts where it does not work at all.

He stands at the top of the stairs with the rubber squeegee and looks at what he has got.

There is a big one in the middle, roughly the shape of a comma, maybe fifteen feet across at its widest. There is a long thin one running along the parapet on the left where the tiles have sunk. There is a small round one near the water tank, and two more small ones over by the corner that he cannot see properly from here.

He starts with the big one. He puts the squeegee down at the near edge and pushes, and the whole thing moves — all of it, together, because it is all one body of water. Push, and the far end of the comma shifts too. That is the thing about a puddle: you cannot move part of it. It is one thing.

Eight minutes and it is gone down the drain.

Then he walks over to the long one by the parapet and starts again, and nothing he did to the first one had any effect on this one at all. They are three feet apart with a dry ridge between them, and they might as well be on different buildings.

By the time he has done all five he has been up there for half an hour, and the count is the count: five separate puddles, five separate jobs, and the size of the job for each one was however far the water spread before it stopped.

The one thing he does that his son does not, when his son is sent up instead, is that Ramesh works the terrace in strips. Left to right, top to bottom, and when he reaches water he clears it completely before he moves on. His son wanders about and pushes bits of different puddles and then cannot remember which ones he has done, and comes down after forty minutes having cleared perhaps three of them and made a mess of the fourth.

3 The idea in plain English

Ramesh's terrace is a grid, and his method is the algorithm.

Each tile is a vertex. A grid of r rows and c columns has $r \times c$ vertices, and each one is identified by a pair of numbers — its row and its column. Not a name, not an integer index. $(3, 7)$ is the vertex.

Two tiles that touch are joined by an edge. Usually that means the four tiles directly above, below, left and right — **four-way adjacency**. Sometimes the problem also counts the four diagonals, giving **eight-way adjacency**. Read the statement and decide which, because the answer changes: a diagonal connection joins two islands that four-way adjacency keeps separate.

Nothing is stored. This is the part to hold on to. There is no adjacency list. The neighbours of `(3, 7)` are `(2, 7)`, `(4, 7)`, `(3, 6)` and `(3, 8)`, and you *compute* them by adding and subtracting one. This is an **implicit graph** — the edges exist by rule rather than by record — and you met the idea on [day 125](#).

The water is a connected component. All the tiles a puddle covers are reachable from each other by touching, and none of them touches the next puddle. So counting puddles is counting connected components — [day 129](#) — and clearing one puddle is a single traversal.

Pushing the squeegee is a flood fill. A **flood fill** starts at one cell and spreads to every connected cell that satisfies some condition — same colour, also land, also water. It is exactly BFS or DFS, and the name is worth knowing because problems use it: “flood fill” and “paint bucket” mean this.

Working in strips is the outer loop. Ramesh scans the terrace left to right, top to bottom, and every time he meets water he has not cleared, that is a new puddle. In code that is the double `for` loop over every row and column, with a check for “have I already dealt with this cell”. **His son’s wandering is what happens without a `seen` record.**

Marking is usually done in the grid itself. When Ramesh clears a puddle it is gone, so he will not clear it twice. In code you can overwrite a visited land cell with water, which costs no extra memory and destroys the caller’s input. The alternative is a separate `visited` grid of booleans, which costs `r × c` and leaves the input alone. **Say which you are doing before you write it** — an interviewer may care.

Now the detail that decides whether your code works: the bounds check. Before you touch `grid[nr][nc]` you must confirm that `0 ≤ nr < rows` and `0 ≤ nc < cols`. Miss it and Python does something worse than crashing: `grid[-1]` is the *last* row, so falling off the top of the grid silently wraps to the bottom, and your islands connect across the edge of the world. **The check has four comparisons and all four are necessary.**

BFS or DFS? Both work and both are `O(rows × cols)`. The deciding argument is not elegance, it is depth: a `1,000 × 1,000` grid of all one colour is a single component whose depth-first path is a million cells long, and recursion dies at about a thousand. **On a grid, write it iteratively** — either BFS with a queue or DFS with an explicit stack — and use BFS specifically when the question asks for *distance*, because only BFS gives you shortest paths.

And the variant worth recognising immediately: many starting points. “How long until every orange rots”, “how far is each cell from the nearest gate”, “how far is each land cell from water” — these start from *every* source at once. You push all the sources

into the queue before the loop begins, and the BFS spreads from all of them simultaneously. That is **multi-source BFS**, it is a one-line change, and it turns a family of problems that look hard into problems that are not. It is [day 141](#), and today you should be able to spot it.

4 The picture

Ramesh’s terrace, as a grid:

	0	1	2	3	4	5	6
0	W W . . . W .						
1	W W 						
2	. . . W W W .						
3	 W . .						
4	W W						

W = water
.= dry

four-way adjacency → 5 puddles:

A: (0,0)(0,1)(1,0)(1,1)	size 4
B: (0,5)	size 1
C: (2,3)(2,4)(2,5)(3,4)	size 4
D: (4,0)	size 1
E: (4,6)	size 1

What to notice. (0,5) and (1,5) ... look again: (1,5) is dry, so (0,5) is alone. And (2,5) is water but does not touch (0,5) — they are two rows apart. Being close on the page is not being adjacent; only sharing an edge counts.

Now the same grid with eight-way adjacency:

four-way: 5 puddles

eight-way: 4 puddles

(0,5) and (2,5)? not adjacent either way – two rows apart.
But (1,4)? dry. (2,3) touches (1,2)? dry.

The change: (0,5) is diagonal to (1,4) and (1,6), both dry – still alone.
(4,0) is diagonal to (3,1), dry – still alone.

... on THIS grid four-way and eight-way agree except where a diagonal bridge exists. Change (1,4) to water and:

four-way: (1,4) joins nothing new – (0,4) dry, (2,4) water → joins C
eight-way: (1,4) also touches (0,5) diagonally → B and C MERGE

What to notice. One cell changing from dry to water merges two components under eight-way adjacency and not under four-way. The problem statement decides this and you must read it. “Connected horizontally or vertically” means four. “Connected in any of the eight directions” means eight.

The neighbour offsets, which you should be able to type without thinking:

FOUR-WAY

```

      (-1, 0)
        |
(0, -1) --+-- (0, +1)
        |
      (+1, 0)

```

((-1,0), (1,0), (0,-1), (0,1))

EIGHT-WAY

```

      (-1,-1) (-1, 0) (-1,+1)
           \   |   /
(0, -1) -- CELL -- (0, +1)
           /   |   \
      (+1,-1) (+1, 0) (+1,+1)

```

all 9 offsets except (0,0)

And the bounds trap, drawn:

```
cell (0, 3), moving up: nr = -1
```

WITHOUT the check:

```

grid[-1][3] → Python reads the LAST row
→ the top of the grid is connected to the bottom
→ no error, wrong answer

```

WITH the check:

```
0 ≤ -1 is False → skipped
```

What to notice. There is no `IndexError` here. Negative indices are legal Python and mean “from the end”, so the failure is silent and produces a grid that wraps like the surface of a doughnut. Falling off the *right* edge does raise `IndexError`, so half your bugs crash and half do not.

5 The code, built step by step

Start with the neighbour function, because it is the only genuinely new thing.

```
FOUR = ((-1, 0), (1, 0), (0, -1), (0, 1))
```

```

def neighbours(row: int, col: int, rows: int, cols: int):
    """Yield the in-bounds cells adjacent to (row, col)."""
    for delta_row, delta_col in FOUR:
        r, c = row + delta_row, col + delta_col
        if 0 ≤ r < rows and 0 ≤ c < cols:
            yield r, c

```

Four comparisons, and all four are necessary. Writing this as a generator means every algorithm below reads the same regardless of four-way or eight-way, and swapping `FOUR` for the eight-offset tuple changes the adjacency everywhere at once.

Now the flood fill, iteratively, because a grid can be deep.

```
def flood(grid: list[list[str]], row: int, col: int, target: str) → int:
    """Clear the whole connected region of `target` starting here. Returns its size."""
    rows, cols = len(grid), len(grid[0])
    stack = [(row, col)]
    grid[row][col] = "0"                    # mark on PUSH
    size = 0
    while stack:
        r, c = stack.pop()
        size += 1
        for nr, nc in neighbours(r, c, rows, cols):
            if grid[nr][nc] == target:
                grid[nr][nc] = "0"          # mark on PUSH, not on pop
                stack.append((nr, nc))
    return size
```

`grid[nr][nc] = "0"` sits immediately before the `append`. On a grid this matters more than anywhere else, because a cell has up to four neighbours that could each push it, so marking on pop lets the stack grow to four times the number of cells.

Overwriting the grid is the `seen` set. No extra memory, and it destroys the input.

Then the outer loop, which is Ramesh working in strips:

```
def count_islands(grid: list[list[str]]) → int:
    if not grid or not grid[0]:
        return 0
    rows, cols = len(grid), len(grid[0])
    count = 0
    for row in range(rows):                    # every cell, in order
        for col in range(cols):
            if grid[row][col] == "1":
                count += 1
                flood(grid, row, col, "1")
    return count
```

Nine lines and the problem is solved. The double loop is the outer loop over all vertices from [day 129](#); the `if` is the `seen` check; `flood` is the traversal.

If the input must survive, swap the marking for a separate grid:


```
def count_islands_pure(grid: list[list[str]]) → int:
    rows, cols = len(grid), len(grid[0])
    visited = [[False] * cols for _ in range(rows)]    # NOT [[False]*cols]*rows
    count = 0
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] == "1" and not visited[row][col]:
                count += 1
                _flood_visited(grid, visited, row, col, rows, cols)
    return count
```

`[[False] * cols for _ in range(rows)]` and not `[[False] * cols] * rows`, for the aliasing reason from [day 126](#). On a grid problem that bug marks every row visited at once and reports one island.

Now BFS, which you need whenever the question asks about distance:

```
from collections import deque

def shortest_path(grid: list[list[int]], start: tuple[int, int], goal: tuple[int, int]) → int:
    """Fewest steps from start to goal through cells equal to 0. -1 if unreachable."""
    rows, cols = len(grid), len(grid[0])
    if grid[start[0]][start[1]] ≠ 0:
        return -1
    distance = {start: 0}
    queue = deque([start])
    while queue:
        r, c = queue.popleft()
        if (r, c) == goal:
            return distance[(r, c)]
        for nr, nc in neighbours(r, c, rows, cols):
            if grid[nr][nc] == 0 and (nr, nc) not in distance:
                distance[(nr, nc)] = distance[(r, c)] + 1
                queue.append((nr, nc))
    return -1
```

Identical to [day 127](#)'s BFS with `neighbours` in place of `graph[vertex]`. **That substitution is the whole of “grids are graphs”** — every algorithm from the rest of this phase works on a grid by changing one line.

And multi-source, which is the variant worth recognising:

```

def rot_time(grid: list[list[int]]) → int:
    """Minutes until no fresh orange remains, or -1. 2 = rotten, 1 = fresh, 0 = empty."""
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                queue.append((r, c))          # EVERY source goes in first
            elif grid[r][c] == 1:
                fresh += 1
    minutes = 0
    while queue and fresh:
        for _ in range(len(queue)):          # one whole level = one minute
            r, c = queue.popleft()
            for nr, nc in neighbours(r, c, rows, cols):
                if grid[nr][nc] == 1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    queue.append((nr, nc))
        minutes += 1
    return -1 if fresh else minutes

```

Two things are new. Every rotten orange goes into the queue *before* the loop starts, so the search spreads from all of them at once — that is the one-line change that defines multi-source BFS. And `for _ in range(len(queue))` processes exactly one ring per iteration, which is how you count minutes rather than steps.

`return -1 if fresh else minutes` is the check people forget: a fresh orange walled off from every rotten one never rots, and the problem wants `-1`, not the time the rest took.

The complete solution

```
"""Grids as graphs: flood fill, island counting, BFS distance, multi-source."""

from __future__ import annotations
from collections import deque

FOUR = ((-1, 0), (1, 0), (0, -1), (0, 1))
EIGHT = ((-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0), (1, 1))

def neighbours(row: int, col: int, rows: int, cols: int, offsets=FOUR):
    """In-bounds neighbours. All four comparisons matter: -1 is a legal index."""
    for dr, dc in offsets:
        r, c = row + dr, col + dc
        if 0 ≤ r < rows and 0 ≤ c < cols:
            yield r, c

def island_sizes(grid: list[list[str]], offsets=FOUR) → list[int]:
    """Sizes of every island. Does NOT modify the caller's grid."""
    if not grid or not grid[0]:
        return []
    rows, cols = len(grid), len(grid[0])
    visited = [[False] * cols for _ in range(rows)]    # distinct rows
    sizes: list[int] = []
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] ≠ "1" or visited[row][col]:
                continue
            size = 0
            stack = [(row, col)]
            visited[row][col] = True
            while stack:
                r, c = stack.pop()
                size += 1
                for nr, nc in neighbours(r, c, rows, cols, offsets):
                    if grid[nr][nc] = "1" and not visited[nr][nc]:
                        visited[nr][nc] = True    # mark on push
                        stack.append((nr, nc))
            sizes.append(size)
    return sizes

def nearest_distance(grid: list[list[int]], source: int, passable: int) → list[list[int]]:
    """Multi-source BFS: distance from every cell to the nearest source. -1 if unreachable."""
    rows, cols = len(grid), len(grid[0])
    dist = [[-1] * cols for _ in range(rows)]
    queue = deque()
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] = source:
                dist[r][c] = 0
                queue.append((r, c))    # all sources, before the loop
    while queue:
        r, c = queue.popleft()
        for nr, nc in neighbours(r, c, rows, cols):
            if dist[nr][nc] = passable and dist[nr][nc] = -1:
                dist[nr][nc] = dist[r][c] + 1
                queue.append((nr, nc))
    return dist

if __name__ = "__main__":
    terrace = [
        list("1100010"),
        list("1100000"),
        list("0001110"),
        list("0000100"),
        list("1000001"),
    ]
```

```

]
print("four-way sizes :", island_sizes(terrace))
print("eight-way sizes:", island_sizes(terrace, EIGHT))
print("grid intact      :", "".join(terrace[0]))

maze = [
    [0, 0, 1, 0],
    [1, 0, 1, 0],
    [0, 0, 0, 0],
    [0, 1, 1, 0],
]
for row in nearest_distance(maze, source=1, passable=0):
    print(" ", row)

```

Running it:

```

four-way sizes : [4, 1, 4, 1, 1]
eight-way sizes: [4, 1, 4, 1, 1]
grid intact      : 1100010
    [1, 1, 0, 1]
    [0, 1, 0, 1]
    [1, 1, 1, 2]
    [1, 0, 0, 1]

```

Two things to look at. `grid intact` prints the original first row unchanged, because this version uses a separate `visited` grid — the version that overwrites the input would print `0000000`.

And the distance grid answers “how far is each cell from the nearest `1`”. The `1` cells themselves come out as `0`, because they *are* sources and start at distance zero. Cell `(2,3)` is the only `2` on the board: it is two steps from every source, and every other passable cell is directly beside one. **The important part is that this is a single BFS**, and it would still be a single BFS with five hundred sources instead of five.

6 What it costs

The two numbers.

$V = \text{rows} \times \text{cols}$	every cell is a vertex
$E \leq 2 \times \text{rows} \times \text{cols}$	4 neighbours each, each edge shared by 2 cells

So $O(V + E)$ collapses to something simple:

```
O(rows x cols + 2 x rows x cols) = O(rows x cols)
```

Every grid traversal is linear in the number of cells. Say it that way rather than $O(V + E)$ — it is more concrete and it is what the interviewer wants to hear.

Put numbers on it:

1,000 x 1,000 grid	
cells	1,000,000
neighbour checks	1,000,000 x 4 = 4,000,000

total	~5,000,000 operations → about half a second in Python

Space, and the choice you make.

overwriting the grid	O(1) extra	destroys the input
separate visited grid	rows x cols	input preserved
as a list of lists	1,000,000 x 8 bytes	= 8 MB
as a bytearray	1,000,000 x 1 byte	= 1 MB

the stack or queue	O(rows x cols) worst case
--------------------	---------------------------

The worst case for the frontier is worth knowing precisely, because it differs between the two traversals:

BFS on a 1,000 x 1,000 open grid	
the widest ring is the diagonal	→ ~1,000 cells in the queue
as (int, int) tuples	→ 1,000 x ~72 bytes = 72 KB
DFS on the same grid	
the deepest path snakes everywhere	→ up to 1,000,000 cells on the stack
	→ 72 MB

On an open grid, BFS's frontier is tiny and DFS's is enormous. That is a genuine reason to prefer BFS on grids, and it is the opposite of the general graph case where BFS's ring can be the expensive one. Say it if asked which to use.

And the recursion arithmetic that fails:

recursive DFS on a 1,000 x 1,000 all-land grid	
depth needed	up to 1,000,000 frames
Python's usable limit	~960
	→ RecursionError

Multi-source costs the same as single-source, which surprises people:

k sources, all pushed before the loop	
each cell is still dequeued exactly once	
	→ O(rows x cols), independent of k

Running `k` separate BFS runs and taking the minimum would be `O(k × rows × cols)`. With 500 gates on a million-cell grid that is 500 million steps against 1 million. **A five-hundred-fold difference from moving one loop before another**, and it is the single best reason to recognise the multi-source shape.

Eight-way costs the same order and $2\times$ the constant:

```
four-way    4 neighbour checks per cell
eight-way   8 neighbour checks per cell
            → same  $O(\text{rows} \times \text{cols})$ , twice the work
```

7 The traps

The missing bounds check

The one that produces a wrong answer instead of a crash:

```
for dr, dc in FOUR:
    if grid[r + dr][c + dc] == "1":      # no bounds check
        ...
```

From cell `(0, 3)`, going up gives `grid[-1][3]`, which Python cheerfully returns — the last row. So the top edge of your grid is glued to the bottom edge:

```
grid = ["1001",
        "0000",
        "0000",
        "1001"]

correct answer (four-way): 4 islands
without the bounds check: 2 islands ← the corners "connect" through the wrap
```

Going off the *right* edge does raise:

```
IndexError: list index out of range
```

So half the failures crash and half are silent, which is the worst possible combination. **All four comparisons, every time.**

Marking on pop

```
while stack:
    r, c = stack.pop()
    if grid[r][c] == "0":
        continue
    grid[r][c] = "0"                # marked on pop
    for nr, nc in neighbours(...):
        stack.append((nr, nc))      # pushed without checking
```

Correct answers. On a large grid:

```
1,000 x 1,000 all-land grid
mark on push: stack peak ~2,000 entries
mark on pop: stack peak ~4,000,000 entries (each cell pushed by up to 4 neighbours)
```

MemoryError

Recursion on a large grid

```
def sink(grid, r, c):
    grid[r][c] = "0"
    for nr, nc in neighbours(r, c, rows, cols):
        if grid[nr][nc] == "1":
            sink(grid, nr, nc)
```

```
Traceback (most recent call last):
  File "islands.py", line 9, in sink
    sink(grid, nr, nc)
[Previous line repeated 993 more times]
RecursionError: maximum recursion depth exceeded
```

And the “fix” that makes it worse:

```
sys.setrecursionlimit(2_000_000)
```

Segmentation fault (core dumped)

No traceback, no message. **A grid problem with `m, n ≤ 300` is safe for recursion; anything above that is not.** Read the constraints.

```
[[False] * cols] * rows
```

```
visited = [[False] * 3] * 3
visited[0][1] = True
print(visited)
```

```
[[False, True, False], [False, True, False], [False, True, False]]
```

One row, referenced three times. Marking one cell marks a whole column's worth across every row, and your island count comes out as 1. The symptom looks like a traversal bug and the cause is one line of initialisation. Always the comprehension.

Running BFS once per source

```
best = [[inf] * cols for _ in range(rows)]
for gate in gates:
    d = bfs_from(grid, gate)          # one full BFS per gate
    for r in range(rows):
        for c in range(cols):
            best[r][c] = min(best[r][c], d[r][c])
```

Correct, and:

```
500 gates on a 1,000 x 1,000 grid
500 x 1,000,000 = 500,000,000 cell visits
```

Time Limit Exceeded

Push every gate into the queue before the loop starts and it is 1,000,000 visits. **The tell is “nearest” or “any of these sources” in the problem statement.**

Forgetting what “unreachable” means

```
return minutes                                # after the BFS finishes
```

On rotting oranges, a fresh orange in a sealed corner never rots, and the answer is `-1`, not the time the reachable ones took. On walls-and-gates, a room no gate reaches keeps its initial value. **Every grid BFS needs an explicit answer for the cells it never touched**, and it is almost always a counter checked after the loop.

8 In the interview

How it gets asked

Grid problems are stated as stories, never as graphs:

- “Count the number of islands.” — LeetCode 200, the most-asked graph question there is.
- “Given a starting pixel and a new colour, fill the connected region.” — flood fill, LeetCode 733.
- “How many minutes until every orange rots?” — multi-source BFS, LeetCode 994.
- “Fill each empty room with the distance to the nearest gate.” — multi-source again, LeetCode 286.
- “Capture all regions surrounded by X.” — the inverted version: flood from the edges.

- “Shortest path through the maze.” — BFS, and only BFS.
- “What is the largest island you can make by flipping one water cell?” — components plus labelling.

The first ninety seconds

“This is a graph problem and the graph is the grid. A vertex is a cell, identified by its row and column, and an edge joins two cells that touch — I would confirm whether that means four directions or eight, because diagonals merge islands that four-way adjacency keeps separate.

Nothing is built. The graph is implicit: the neighbours of a cell are the four coordinate offsets, computed rather than stored. So I write a neighbour function with the bounds check in it and then every algorithm from the graph phase works unchanged — I substitute `neighbours(r, c)` for `graph[vertex]` and that is the whole translation.

Counting islands is counting connected components, so it is the double loop over every cell, and whenever I find land I have not visited, that is one more island and I flood from it.

Three things I would be deliberate about.

The bounds check is four comparisons and all four matter. In Python a negative index is legal and means from the end, so falling off the top of the grid silently connects it to the bottom. That is a wrong answer rather than an error, which is worse.

I would write it iteratively, not recursively. A thousand-by-thousand grid of all land is one island a million cells deep and recursion dies at about a thousand frames. Raising the limit turns that into a segmentation fault.

I mark cells when I push them, not when I pop them, because on a grid each cell has up to four neighbours that could push it, so marking on pop lets the stack reach four times the number of cells.

Cost is `O(rows × cols)` time — every cell visited once, four neighbour checks each. Space depends on whether I may modify the input: overwriting visited land with water is free, a separate visited grid is `rows × cols`. Which would you prefer?”

The follow-ups

“BFS or DFS for this?”

“Both are $O(\text{rows} \times \text{cols})$ and both give the same components, so it comes down to two things.

If the question asks about **distance** — fewest steps, minutes, nearest anything — it must be BFS, because only BFS gives shortest paths on an unweighted graph.

If it is just connectivity, I would still lean BFS on a grid, and the reason is the frontier size. On an open thousand-by-thousand grid, BFS’s queue holds one ring, which is roughly the diagonal — about a thousand cells, seventy kilobytes. DFS’s stack holds the deepest path, which on an open grid can snake through every cell — a million entries, seventy megabytes. That is the opposite of the general graph case, where BFS’s ring is the expensive one, and it is because grids have bounded degree and large diameter.

DFS is two lines shorter and I would use it when the grid is small and the question is just ‘how many’.”

“There are five hundred gates and you need each room’s distance to the nearest one.”

“Multi-source BFS, and the change from what I have written is one line: push **all** the gates into the queue before the loop starts, each at distance zero.

The search then spreads outward from every gate simultaneously, and because BFS marks a cell the first time it reaches it, each cell gets the distance from whichever gate reached it first — which is the nearest one. No comparison, no minimum, it falls out of the order.

The arithmetic is the argument. Running a separate BFS per gate and taking the minimum is 500 million cell visits on a million-cell grid. Multi-source is one million, because every cell is still dequeued exactly once. **The cost is independent of the number of sources**, which is the surprising part and the reason to recognise the shape.

The tell in a problem statement is the word ‘nearest’, or ‘any of these’, or a start condition that is a set rather than a point. Rotting oranges, walls and gates, distance to the nearest zero, and ‘how far is each land cell from water’ are all this.”

“The grid is 10,000 by 10,000. Does anything change?”

“A hundred million cells, so the algorithm is fine — still linear, about half a billion operations, which is minutes in Python and seconds in a compiled language. What changes is memory.

A separate visited grid as a list of lists is a hundred million Python objects and references — several gigabytes. So I would use a `bytearray` of a hundred million bytes, which is 100 MB, or overwrite the input if I am allowed. A single flat `bytearray` indexed as `r * cols + c` is also much better for cache locality than a list of lists.

The frontier is fine either way with BFS — the ring is about ten thousand cells.

If the grid were larger still, or sparse, I would stop materialising it: represent only the interesting cells in a set of coordinates and compute neighbours from that. A grid that is mostly empty does not need a hundred million entries.

And I would ask whether the grid actually needs to be traversed in one piece. Many real grid problems — image segmentation, map tiles — partition naturally, and processing tiles independently with a merge step at the boundaries parallelises trivially.”

“Capture all regions surrounded by X. How is that different?”

“It is the inverted version and it is worth recognising because the direct approach is much harder.

The direct approach is: for each region of `0`, check whether it touches the border, and if not, flip it. That works but requires flooding a region and *then* deciding, which means collecting the cells and going back over them.

The move is to flip the question. A region survives exactly when it touches the border. So: flood from every `0` on the **border**, mark everything reachable as safe, and then in one final pass flip every `0` that was not marked. Two passes, no collecting, no second thoughts.

That is a general trick for grid problems and I would name it as one: **when a condition is about reaching the edge, start from the edge**. Pacific-Atlantic water flow is the same idea run twice, once from each ocean’s border, taking the intersection — and the naive version there, flooding from every cell to see where the water goes, is `rows * cols` traversals instead of two.”

The model answer

“Given a grid where 1 is land and 0 is water, return the size of the largest island you could create by changing exactly one water cell to land.”

“Let me name the shape first: this is connected components plus labelling, and the labelling is what makes the efficient version possible.

The naive version is: for every water cell, flip it, count the island it now belongs to, flip it back. That is one traversal per water cell — $\text{rows} \times \text{cols}$ traversals, each $O(\text{rows} \times \text{cols})$, so quadratic in the number of cells. On a 500×500 grid that is 62 billion operations. Correct and unusable, and I would say so rather than write it.

The fix is two passes. First pass: label every island with an id and record each id's size — a dictionary from island id to cell count. That is one traversal over the whole grid, $O(\text{rows} \times \text{cols})$, using the component-labelling version rather than the counting version.

Second pass: for every water cell, look at its four neighbours, collect the **distinct** island ids among them, and the candidate answer is one plus the sum of those islands' sizes. Take the maximum over all water cells. That is four lookups per cell, so also $O(\text{rows} \times \text{cols})$.

The word 'distinct' is the whole bug in this problem. If two neighbours of a water cell belong to the same island — which happens constantly, for instance in a U shape — adding both sizes counts that island twice and the answer is too large. So the neighbours' ids go into a set before summing. I would write that as a set from the start and say why, because it is the thing the test cases are built to catch.

Two edge cases. A grid that is entirely land: there is no water cell to flip, and the answer is $\text{rows} \times \text{cols}$. A grid that is entirely water: the answer is 1, from flipping any single cell. Both fall out naturally if I initialise the best answer to the largest existing island and then let the second pass improve on it, rather than starting from zero.

Cost: two linear passes, so $O(\text{rows} \times \text{cols})$ time. Space is one integer label per cell — a flat array of $\text{rows} \times \text{cols}$ — plus the size dictionary, which has at most one entry per island. On a 500×500 grid that is 250,000 labels rather than 62 billion operations.

Implementation notes I would state while writing: labels start at 2, so they never collide with 0 for water and 1 for unlabelled land; the flood is iterative with marking on push; and the neighbour function carries the bounds check so neither pass repeats it.”

9 Recall card

A grid is a graph: a cell is a vertex, touching cells are edges, and nothing is stored — the neighbours are computed from the coordinates. Substitute `neighbours(r, c)` for `graph[v]` and every graph algorithm works unchanged.

The bounds check is four comparisons and all four matter. `grid[-1]` is the last row, so a missing check wraps the grid instead of crashing — a wrong answer, not an error.

Write it iteratively and mark on push. A million-cell island is a million frames deep (`RecursionError`, or a segfault if you raise the limit), and marking on pop lets the stack reach four times the cell count.

Cost is $O(\text{rows} \times \text{cols})$. On an open grid, BFS's queue holds one ring ($\sim \text{rows}$ cells) and DFS's stack can hold the whole snaking path — the opposite of the general graph case.

Multi-source BFS is one line — push every source before the loop — and costs the same as one source. The tells are “nearest”, “any of these”, or “how long until everything”. And when the condition is about reaching the edge, **start from the edge**.

1 What this is, and why they ask it

Kafka is a log. Not a queue — a log. Messages are appended to the end, they stay for a retention period, and each reader keeps track of its own position rather than consuming and destroying.

That one difference explains everything else about it. Because messages are not removed, several independent teams can read the same stream for different purposes. Because each reader has its own position, one of them can rewind a week and reprocess. Because the storage is an append-only file rather than a mutable structure, one machine can absorb hundreds of megabytes a second on ordinary disks.

They ask about Kafka because it is in almost every large architecture and because the phrase “we’ll put it on Kafka” is said far more often than it is understood. The questions that actually separate people are narrow and specific: *how does Kafka guarantee ordering?* — the answer is “it does not, except per partition, and here is why”. *What happens when a consumer is added? What is a consumer group? What have you lost by choosing it over a queue?*

You met the general shape yesterday on [day 129](#); today is the specific product, its vocabulary, and the numbers that let you place it in a design and defend it.

By the end of this lesson you can describe topics, partitions, offsets and consumer groups accurately, explain exactly what ordering guarantee exists and what it costs, size a cluster from a throughput requirement, and say the two or three sentences about replication and lag that show you have operated one.

2 The story

The set-top box in the Menon house records the nine o’clock serial every night, and keeps thirty days.

Nobody in the house watches it at nine o’clock.

Mrs Menon is the only one who is more or less up to date. She watches two or three a week, in the afternoon, and she is currently on the episode from last Thursday. She knows this because the box remembers where she stopped.

Her husband is eleven episodes behind. He knows exactly which one he is on — the one where the older brother comes back from Dubai — and he picks up from there whenever he has an evening free, which is not often. When he sits down and presses

play, it does not matter at all what his wife has watched. The box has one copy of each episode and two positions in it.

Their son, who is nineteen, watches four at a time on Sunday afternoons and complains about the whole thing while he does it. He is somewhere between his parents.

And Sarita, who comes to cook and stays for a couple of hours in the afternoon, watches on the small television in the kitchen and is further behind than anybody, because she only gets twenty minutes at a stretch.

Four people, one recording, four different places in it. Nobody's watching removes anything. If Mrs Menon finishes an episode, it is still there for her husband. That is the part that took a while for everyone to get used to, because it is not how a video shop worked.

There are two rules and the house has learnt both the hard way.

The first is that you cannot watch them out of order. Not really. You can, physically, but the son tried it once and had no idea what was going on, because episode 34 refers to things that happened in 31. The order they were recorded in is the order they mean anything in.

The second is the thirty days. Sarita fell forty-one days behind during the monsoon when her mother was ill, and when she came back the eleven oldest episodes were simply not there any more. The box had overwritten them. Nobody had done anything wrong; the box holds thirty days and she was outside it.

Mr Menon now checks how far behind he is every couple of weeks, and if it gets past about twenty he watches three in one evening to get back inside the window. He describes this as "catching up", and it is exactly that.

3 The idea in plain English

The Menons' set-top box is Kafka, and the mapping is close enough to be useful for the rest of your career.

The recording is a topic, and it is a log. A **topic** is a named stream of messages — `orders`, `page-views`, `payments`. Messages are appended to the end and never modified. **This is the single most important thing about Kafka and the thing people get wrong: reading does not remove.** A traditional queue hands out a message and deletes it. Kafka hands out a copy and the message stays.

Each viewer's position is an offset. An **offset** is just a number — the position in the log that a particular reader has got to. The log is numbered 0, 1, 2, 3 upwards, forever. Mrs Menon is at 112 and her husband at 101. Kafka stores these positions; the messages

themselves are untouched by anyone's reading.

Thirty days is the retention. Kafka keeps messages for a configured period — seven days is the common default — or up to a configured size, and then deletes the oldest. **Nothing in the system knows or cares whether anyone read them.** Sarita losing eleven episodes is a consumer whose lag exceeded the retention, and it is the most common way data is silently lost in a Kafka system.

How far behind you are is consumer lag. Lag is the difference between the newest offset in the log and where a consumer has got to. Mr Menon being eleven episodes behind is a lag of 11. **Lag is the single most important thing to monitor in a Kafka deployment,** because a lag that grows steadily means the consumer cannot keep up, and a lag that reaches the retention means data loss.

Watching in order matters, and here is where the analogy has to stretch. A topic is not one log. It is split into **partitions** — independent logs that together make up the topic. Think of it as the serial being recorded onto four separate discs, with each episode going to one disc.

Ordering is guaranteed within a partition and nowhere else. Episodes on disc 1 are in order relative to each other. An episode on disc 1 and an episode on disc 3 have no defined order at all. **This is the answer to “how does Kafka guarantee ordering”, and the honest version is: it does not, globally. It guarantees order per partition.**

Which partition a message goes to is decided by its key. Kafka hashes the message key and takes the remainder. So every message with the same key lands on the same partition and is therefore ordered relative to the others with that key. **Key by `user_id` and all events for one user are ordered; events for different users are not, and usually you do not care.**

Partitions exist for parallelism, and that is the trade. Four partitions can be read by four consumers at once, so four times the throughput. But ordering only holds within each one, so **the more you parallelise the less order you have.** One partition gives you total order and no parallelism at all.

A consumer group is a set of readers that share the work. Each partition is assigned to exactly one consumer in the group, so the group collectively reads every message exactly once. Add a consumer and the partitions are redistributed — that is a **rebalance**. **A consumer group is one logical reader,** whatever its member count.

Different groups are completely independent — that is the Menons. The analytics team's group and the email team's group each have their own offsets and each see every message. Adding a fifth consumer group costs the cluster almost nothing in storage, because there is still one copy of the data.

And the consequence of “one partition per consumer”: more consumers than partitions is wasted. Twelve partitions and twenty consumers means eight consumers sit idle. Partition count is the ceiling on your parallelism, and it is chosen up front and is awkward to increase later, because increasing it changes which partition a given key hashes to — and therefore breaks the per-key ordering you were relying on.

Replication is how it survives a machine dying. Each partition has a leader and some followers. Writes go to the leader, followers copy them, and the set that is caught up is the in-sync replica set (ISR). If the leader dies, one of the in-sync followers is promoted. `replication.factor = 3` and `min.insync.replicas = 2` is the standard setting: three copies, and a write is only acknowledged when at least two have it.

4 The picture

A topic with three partitions, and two consumer groups reading it independently:

TOPIC "orders"

partition 0: [0][1][2][3][4][5]

partition 1: [0][1][2][3]

partition 2: [0][1][2][3][4][5][6]

← append here

← append here

← append here

group "email"

p0 offset 3

p1 offset 2

p2 offset 2

group "analytics"

p0 offset 5

p1 offset 4

p2 offset 6

ONE copy of the data. TWO independent positions in it.
The email group is behind. That costs the analytics group nothing.

What to notice. The partitions have different lengths, because the key hash sends different amounts of traffic to each. That is normal, and a badly chosen key makes it extreme — which is the hot-partition problem below.

Now consumer group assignment, and what a rebalance does:

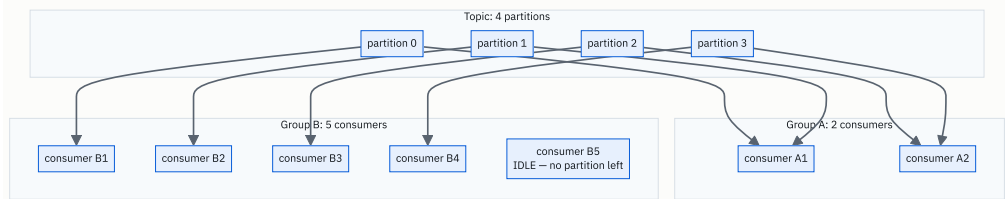


FIGURE 130.1

What to notice. Group A has two consumers for four partitions, so each takes two. Group B has five consumers for four partitions, so one does nothing at all — **the partition count is a hard ceiling on useful parallelism**, and scaling past it requires more partitions, not more machines.

And the key-to-partition mapping, which is where ordering comes from:

```
producer sends (key="user_4471", value={...})

partition = hash("user_4471") % 3 = 1

every message for user_4471 → partition 1 → in order, one consumer

key = null → round-robin across partitions → NO ordering at all

HOT PARTITION:
key = "country" on a topic where 60% of traffic is one country

partition 0: 60% of all messages ← one consumer doing 60% of the work
partition 1: 25%
partition 2: 15%

adding consumers does not help. The key is wrong.
```

What to notice. The key choice decides both the ordering guarantee you get and how evenly work is spread. Those two requirements pull against each other, and reconciling them is the actual design work.

5 How it actually works

Why it is fast

Kafka's throughput comes from three deliberately unclever decisions, and being able to name them is worth marks.

It appends to a file. No index to update, no tree to rebalance, no in-place modification. Writes are sequential, and sequential disk writes on spinning disks run at hundreds of megabytes a second — comparable to random *memory* access. Most storage systems are slow because they seek; Kafka never seeks on write.

It hands bytes straight from the page cache to the socket. Messages are written to the OS page cache and read from it, and Kafka uses `sendfile` to copy from the file to the network without passing through application memory. That is **zero-copy**, and it removes two copies and two context switches per message.

It batches everything. Producers accumulate messages for a few milliseconds and send them in one request; consumers fetch many at once; batches are compressed as a unit, which compresses far better than per-message. Batching is why per-message overhead effectively disappears at volume.

The consequence worth stating: **Kafka does not keep messages in memory and does not try to.** It relies on the operating system's page cache, which means a broker with 64 GB of RAM has 64 GB of hot log cached for free, and consumers reading near the tail never touch a disk at all.

The producer's durability choices

```
acks=0    # fire and forget. Fastest. Loses data on any failure.
acks=1    # the leader has it. Loses data if the leader dies before followers copy.
acks=all  # all in-sync replicas have it. Slowest. Safe.
```

`acks=all` with `min.insync.replicas=2` and `replication.factor=3` is the configuration for anything you care about: the write is acknowledged once two of three copies exist, and it survives one broker dying. If only one replica is in sync, the producer gets an error rather than a false acknowledgement — which is the point.

```
enable.idempotence=true # producer ID + sequence number; dedupes producer retries
```

This gives exactly-once semantics for producer retries within a session, per partition, by having the broker drop a sequence number it has already written. It is on by default in recent versions and it is worth turning on knowingly.

Consumer offsets and where the duplicate comes from

Consumers commit their offsets back to Kafka, into an internal topic called `__consumer_offsets`. The choice that matters:

```
auto-commit every 5 s      (enable.auto.commit=true, the default)
  process a message, crash before the next commit
  → up to 5 seconds of messages are reprocessed

manual commit after processing
  → at-least-once: crash between processing and commit means one redelivery

manual commit before processing
  → at-most-once: crash after commit means the message is lost
```

Almost everyone wants manual commit after processing, plus idempotent handlers. The default auto-commit is convenient and gives you a five-second replay window on every crash, which is fine for metrics and not fine for anything else.

Rebalancing, and why it hurts

When a consumer joins or leaves a group, partitions are reassigned. In the classic protocol this is a **stop-the-world** event: every consumer stops, the assignment is recomputed, and everyone resumes. On a large group that can take seconds, during which nothing is consumed and lag grows.

Three things trigger it, and two are avoidable:

- A consumer genuinely joining or leaving. Unavoidable and intended.
- A deploy restarting every consumer. Mitigate with **static membership** (`group.instance.id`), which lets a restarting consumer reclaim its own partitions without a full rebalance.
- A consumer taking too long between polls and being declared dead (`max.poll.interval.ms`, default 5 minutes). **This is the classic Kafka incident:** a slow message causes a rebalance, the rebalance causes lag, the lag causes more slow processing, and the group thrashes. The fix is to bound processing time or reduce `max.poll.records`.

Cooperative sticky assignment (the modern default) reassigns only the partitions that must move rather than stopping everything, and it is worth naming as the thing you would enable.

Log compaction

Alongside time-based retention, a topic can be **compacted**: instead of deleting old messages by age, Kafka keeps the *latest* message for each key and discards earlier ones.

That turns a topic into a durable, replayable snapshot of current state — “the current address of every user” rather than “every address change”. It is how Kafka Connect stores connector state, how Kafka stores consumer offsets, and how the Kafka-Streams state stores are backed. **Worth knowing because “how do I get the current value, not the history” has an answer inside Kafka rather than requiring a database.**

What replaced ZooKeeper

Kafka used ZooKeeper for cluster metadata — broker registration, topic configuration, controller election — until **KRaft**, which uses Kafka’s own Raft implementation and removes the external dependency. ZooKeeper mode is removed in Kafka 4.0. If asked, the useful sentence is: “metadata used to live in ZooKeeper and now lives in an internal Raft-replicated log, which removes a whole system from the deployment.”

When it is the wrong tool

- **Task queues with per-message retry and delay.** Kafka has no per-message acknowledgement and no built-in dead-letter behaviour; a poison message blocks its partition unless you build the handling yourself. SQS or RabbitMQ give you that for free.
- **Request-reply.** Kafka is one-directional. Correlating a reply topic is possible and unpleasant.
- **Small systems.** A three-broker cluster plus monitoring is real operational work for a workload a database table would handle.
- **Very large messages.** The default cap is 1 MB. Put the payload in object storage and send the reference.

6 The numbers

Throughput per broker.

sequential disk write	~ 100-500 MB/s per disk
Kafka per broker, real world	~ 100-300 MB/s sustained
messages/s at 1 KB each	~ 100,000 - 300,000 per broker

Sizing a cluster from a requirement. Take one million messages a second at 1 KB each:

1,000,000 x 1 KB	= 1 GB/s incoming
replication factor 3	= 3 GB/s written across the cluster
per broker at 250 MB/s	3,000 / 250 = 12 brokers
	+ headroom → 15-16 brokers

Storage.

1 GB/s x 86,400 s	= 86 TB/day of raw messages
x 3 replicas	= 259 TB/day
x 7 days retention	= 1.8 PB

with 5x compression (snappy on JSON is typically 4-6x)	
	= ~360 TB
per broker over 16 brokers	= ~23 TB each

That last step is the one to show. Compression on a Kafka topic is often the difference between a plausible cluster and an implausible one, and it costs a few percent of CPU.

Partition count.

target throughput	1,000,000 msg/s
per-partition throughput	~ 10 MB/s, so ~10,000 msg/s at 1 KB
partitions needed	1,000,000 / 10,000 = 100 minimum
	x2 for headroom and future growth = 200

And the constraint from the consumer side:

consumers you want to run	50
partitions must be $\geq$ 50	or some consumers idle
partition count is also the ORDERING granularity	

Rule of thumb: partitions = max(throughput requirement, desired consumer parallelism), with headroom, and not more than a few thousand per broker, because each partition costs open file handles, memory for its index, and rebalance time.

Consumer lag, and the number that matters.

production rate	50,000 msg/s
consumption rate	45,000 msg/s
lag growth	5,000 msg/s
	= 18,000,000 messages per hour behind
retention	7 days = 604,800 s
time until data loss	604,800 x 5,000 = 3 billion messages of headroom
	at 5,000/s growth: it takes 7 days to fall out of retention

Seven days of retention is seven days of warning, and that is the real argument for generous retention: it is not about replay, it is about how long you have to notice.

Latency.

producer batch wait (linger.ms)	0 - 100 ms, typically 5
broker write + replication	2 - 10 ms with acks=all
consumer fetch	1 - 50 ms

end to end, well-tuned	10 - 50 ms
end to end, batched for throughput	100 - 500 ms

Kafka is not a low-latency system by default and can be tuned to be one, at the cost of throughput. If someone needs single-digit milliseconds, `linger.ms=0` and small batches get you there and cut throughput several-fold.

Rebalance cost.

classic eager rebalance, 50 consumers	2 - 30 seconds of zero consumption
at 50,000 msg/s	100,000 - 1,500,000 messages of lag created
cooperative sticky	only moving partitions pause

What a hot partition costs.

```
100 partitions, evenly keyed
  each consumer handles 10,000 msg/s      → fine

100 partitions, one key is 30% of traffic
  that partition: 300,000 msg/s           → one consumer, one partition
  per-partition ceiling ~10,000 msg/s     → 30x over capacity
  lag on that partition grows forever
```

And adding brokers does not help at all. This is the failure mode that surprises people: the cluster is at 20% utilisation and one partition is melting.

7 The trade-offs

Ordering against parallelism, and you cannot have both. One partition gives you total order and one consumer's worth of throughput. A hundred partitions give you a hundred consumers and order only within each. The design work is choosing a key that gives you the ordering you actually need — usually per entity — while spreading traffic evenly. Those two goals conflict whenever your entities have wildly different volumes.

Retention against storage cost. Long retention is what makes replay possible and gives you time to notice a lagging consumer. It also multiplies your storage by the replication factor. Seven days at a gigabyte a second is a petabyte before compression, and that is a real bill.

Kafka's durability against its operational weight. Three brokers minimum, disks, monitoring, partition planning, rebalance tuning, and a team that knows what `min.insync.replicas` does. For a workload that is genuinely a queue, SQS is an API call and has none of that. **Choose Kafka when you need the log, not when you need a queue.**

No per-message acknowledgement. This is the concrete thing you lose against a task queue. A message that always fails blocks its partition, because the consumer cannot skip it without advancing the offset past it. Retry and dead-letter handling are your code, not the broker's. Every mature Kafka deployment ends up building retry topics and a DLQ topic, and that is a real cost to weigh.

Partition count is chosen early and is awkward to change. Increasing it changes `hash(key) % partitions`, so existing keys move and per-key ordering breaks across the change. You can only add partitions, never remove. **Over-provision modestly at the start** — twice what you need — because it is much easier than migrating later.

Exactly-once is exactly-once within Kafka only. Transactions cover a read-process-write where both ends are Kafka. The moment the consumer charges a card or writes to Postgres, the guarantee ends and you are back to idempotency keys from [day 122](#). Say-

ing this precisely is one of the better signals a candidate can give.

When would I not use Kafka? When there is one producer and one consumer and no replay requirement — that is a queue, and a queue is simpler. When the messages are large. When I need per-message delay or priority. And when the team is three people, because the operational cost is paid every week and the benefit only arrives at scale. Managed Kafka removes some but not all of that.

8 In the interview

How it gets asked

- “How does Kafka guarantee ordering?” — the classic, and the answer starts by narrowing the claim.
- “What is a consumer group?”
- “You added five consumers and throughput did not change. Why?”
- “How would you size a Kafka cluster for a million messages a second?”
- “Kafka or SQS?”
- “A consumer is falling behind. What do you do?”
- “What happens when a broker dies mid-write?”

The first ninety seconds

“The first thing I would say is that Kafka is a log rather than a queue, because everything else follows from that. Messages are appended and kept for a retention period; reading does not remove them; each consumer group tracks its own offset. So several independent teams can read the same stream, and any of them can rewind and reprocess — neither of which a task queue can do.

On ordering, the honest answer is that Kafka does not guarantee global ordering and cannot. A topic is split into partitions, each of which is an independent append-only log, and ordering holds *within a partition only*. Two messages in different partitions have no defined order at all.

What you control is which partition a message lands in: Kafka hashes the message key. So if I key by user ID, every event for one user is on one partition and therefore ordered, while events for different users are not — and usually that is exactly the guarantee the business actually needs.

The cost is that ordering and parallelism trade directly against each other. One partition means total order and one consumer’s throughput. A hundred partitions means a hundred consumers and order only per key. And a poorly chosen key gives me a hot partition — if one key is thirty percent of traffic, one consumer gets thirty percent of the work and adding brokers does not help at all.

A consumer group is a set of consumers sharing the partitions of a topic, each partition assigned to exactly one member, so the group as a whole reads every message once. Different groups are completely independent and cost the cluster nothing extra, because there is still one copy of the data.

Shall I size a cluster, or go into what happens when a consumer falls behind?”

The follow-ups

“You added five consumers and throughput did not change.”

“Almost certainly more consumers than partitions. Each partition is assigned to exactly one consumer in a group, so if there are twelve partitions and I run seventeen consumers, five of them are assigned nothing and sit idle. **The partition count is a hard ceiling on useful parallelism.**

The fix is more partitions, and it is not free. Adding partitions changes `hash(key) % partition_count`, so existing keys start landing somewhere new — which breaks per-key ordering across the boundary and means a consumer can see a user’s newer event before an older one that is still queued on the old partition. If ordering matters I would either drain the topic first, or create a new topic with the right count and migrate.

The second possibility, if partitions are not the issue, is a hot partition: the work is not evenly distributed because the key is skewed. That looks identical from a throughput dashboard and completely different from a per-partition lag dashboard, which is why per-partition lag is the metric I would ask to see.

And the third: the consumers are not the bottleneck at all — they are waiting on a downstream database, and adding more of them just adds contention.”

“A consumer group’s lag is growing. Walk me through it.”

“Lag is how far behind the newest offset a consumer is, and a lag that grows means the consumption rate is below the production rate. It will not recover on its own.

First I would check whether it is uniform or on specific partitions. Uniform means genuinely undersized consumers; concentrated on one or two partitions means a skewed key, and adding consumers will not help.

If it is uniform: scale consumers up to the partition count, and if I am already at it, add partitions. Check whether the consumer is doing something per-message that could be batched — the usual culprit is single-row database writes where a batch would be fifty times faster.

The number that makes this urgent is the retention. With seven days of retention and lag growing at five thousand messages a second, I can compute exactly how long until the consumer’s offset falls off the start of the log — and at that point Kafka has already deleted messages the consumer never read, silently. There is no error; the consumer just resets to the earliest available offset and a gap exists in whatever it was building. **That is the main way data is lost in a Kafka system** and it is why I would alert on lag as a *time* — ‘the oldest unread message is four hours old’ — rather than as a message count.

The emergency option is to increase retention temporarily to buy time, which is a config change and costs only disk.”

“Kafka or SQS?”

“The question is whether the message is an event or a task.

A task — ‘send this email’ — has one owner, is done once, and then has no further value. SQS is right: per-message retries, visibility timeouts, a dead-letter queue, no partitions to plan, and it is an API call rather than a cluster.

An event — ‘user 4471 signed up’ — may interest several teams, and might need to be reprocessed. Kafka is right: multiple consumer groups read the same data independently, retention lets any of them rewind, and throughput is an order of magnitude higher per pound.

The concrete thing I give up by choosing Kafka is per-message acknowledgement. A message that always fails blocks its partition, because the consumer cannot advance past it without committing an offset that skips it. Retry topics and a dead-letter topic are code I have to write, whereas SQS gives me `maxReceiveCount` and a DLQ for free. Every mature Kafka deployment builds that eventually.

If a system needs both — and most do — I would run both rather than force one to do the other’s job: Kafka for the event backbone, a task queue for work items with per-message retry semantics.”

“What happens if a broker dies mid-write?”

“It depends on `acks`, and this is where the configuration is the answer.

With `acks=all`, `replication.factor=3` and `min.insync.replicas=2`: the producer’s write is only acknowledged once at least two replicas have it. If the leader dies before acknowledging, the producer gets an error or a timeout and retries — and with `enable.idempotence=true` the retry carries a producer ID and sequence number, so if the write had actually landed the broker drops the duplicate. One of the remaining in-sync replicas is elected leader, and no acknowledged write is lost.

With `acks=1`, the leader acknowledges as soon as it has the message locally. If it dies before a follower copies it, that write is gone and the producer believes it succeeded. That is silent data loss, and it is the default in some client versions, which is why I would check it rather than assume.

The setting that catches people is `min.insync.replicas=1` with `replication.factor=3` — it looks safe and it means a single surviving replica can acknowledge writes, so you can lose acknowledged data with two failures. `min.insync.replicas=2` means the cluster refuses writes rather than accepting unsafe ones, which is a deliberate availability sacrifice and the right one for anything that matters.

On the consumer side, a broker failure triggers a metadata refresh and possibly a rebalance; consumers resume from their committed offsets, which is why processing must be idempotent — some messages after the last commit will be redelivered.”

The model answer

“Design the event pipeline for an e-commerce site. Order events need to reach the warehouse system, the analytics warehouse, the email service, and a fraud model. Volume is 5,000 orders a second at peak.”

“Four consumers of the same events, one of which needs replay for model retraining — that is the shape that makes this Kafka rather than a queue, and I would say that first.

One topic, `orders`, with four independent consumer groups. Warehouse, analytics, email, fraud. Each tracks its own offset, each sees every event, and adding a fifth team later costs the cluster nothing — there is still one copy of the data. If I used a task queue I would either be publishing four copies of every message or maintaining the subscriber list in the producer, and adding a consumer would mean changing the producer.

Key by `order_id`. That gives me ordering per order, which is the guarantee that actually matters — `order.created` before `order.paid` before `order.shipped` — while spreading load evenly, because order IDs are uniformly distributed. **I would explicitly not key by customer or by warehouse**, both of which are skewed enough to create hot partitions: a few large customers, and a few big warehouses.

Partitions: sixty. Five thousand orders a second at maybe 2 KB each is 10 MB a second, which one partition could nearly handle — but partition count is also my consumer parallelism ceiling and my headroom for growth, and it cannot be reduced later. Sixty gives every group room to run up to sixty consumers, at about 85 messages a second per partition, which is nothing. The cost of over-provisioning modestly is a few file handles; the cost of under-provisioning is a migration.

Replication 3, `min.insync.replicas 2`, `acks=all`, idempotent producer. Order events are money. The write is acknowledged when two of three copies exist, the cluster refuses writes rather than accepting unsafe ones when it cannot get two, and producer retries do not duplicate.

Retention: thirty days, not seven. The fraud model retrains on historical events, so replay is a first-class requirement rather than a safety net. Storage: $10 \text{ MB/s} \times 86,400 \times 30 \text{ days} \times 3 \text{ replicas}$ is about 78 TB, and with compression on JSON — typically four to six times — call it 15 TB, spread across six brokers. That is comfortable and I would rather pay for it than lose the ability to rewind.

The outbox pattern at the producer. The order row and the event row go into the same database transaction, and a publisher moves events to Kafka. Without that, a crash between the database commit and the Kafka publish means an order exists that nothing downstream knows about — and that is a warehouse that never ships. This is the detail I would insist on.

Every consumer is idempotent, because delivery is at-least-once and a consumer crash between processing and committing an offset guarantees redelivery. The warehouse consumer keys on order ID with a conditional insert; the email consumer

keys on `(order_id, email_type)`; analytics deduplicates on ingest.

Retry and dead-letter topics, because Kafka gives me neither. A consumer that fails a message publishes it to `orders.retry.5m`, and after a few attempts to `orders.dlq`, with an alert on DLQ depth. Otherwise a single malformed event blocks its partition indefinitely — which is the most common Kafka outage I would be designing against.

Monitoring: per-partition consumer lag, expressed in time, per group. ‘The email group’s oldest unread message is four hours old’ is actionable; ‘lag is 2.3 million’ is not. And an alert well before lag approaches retention, because that boundary is where data is lost silently.

What I would flag as out of scope but real: the warehouse consumer talks to a legacy system with no idempotency support, so ‘process each order exactly once’ there is not achievable by Kafka’s transactions — those only cover Kafka-to-Kafka. That consumer needs its own dedup table keyed on order ID, and I would name it as the weakest link in the design rather than let the architecture diagram imply it is solved.”

9 Recall card

Kafka is a log, not a queue. Messages are appended, kept for a retention period, and **reading does not remove them** — each consumer group tracks its own offset, so many teams read the same data and any of them can rewind.

Ordering is per partition only. The message key is hashed to choose a partition, so key by entity to get per-entity ordering. **Ordering and parallelism trade directly**, and a skewed key gives a hot partition that no amount of extra brokers fixes.

A consumer group is one logical reader. Each partition goes to exactly one member, so **more consumers than partitions means idle consumers** — the partition count is a hard ceiling, chosen early and awkward to raise.

`acks=all`, `replication.factor=3`, `min.insync.replicas=2` for anything that matters, plus an idempotent producer. `min.insync.replicas=1` looks safe and silently permits data loss.

Monitor consumer lag as a time, and alert long before it reaches retention — a consumer that falls outside the window loses messages with no error at all. And Kafka gives you no per-message retry or DLQ: you build retry topics yourself, or use a task queue for work.

DSA topic: Grids are graphs: islands and flood fill **System design topic:** Kafka, explained

Code these, in this order

One rule for the whole set: **write the neighbour function with its bounds check first, as a separate function, before you write anything else.** Every problem below uses it unchanged. Doing it once properly is the difference between five clean solutions and five chances to forget one of the four comparisons.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Flood Fill	LeetCode 733 (Easy)	The neighbour function, and the case where the new colour equals the old.
2	Number of Islands	LeetCode 200 (Medium)	The double loop as the outer loop, iteratively.
3	Max Area of Island	LeetCode 695 (Medium)	Component sizes, and returning 0 on an empty grid.
4	Rotting Oranges	LeetCode 994 (Medium)	Multi-source BFS, level counting, and the <code>-1</code> case.
5	Surrounded Regions	LeetCode 130 (Medium)	Flooding from the border — the inverted question.
6	Making A Large Island	LeetCode 827 (Hard)	Labelling plus the <i>distinct</i> neighbouring ids.

On problem 1, find the infinite loop

Run it with the new colour equal to the starting colour. Record what happens. Then say in one sentence why the `seen` mechanism failed, and write the one-line guard.

On problem 2, run the four failure modes

On a $1,000 \times 1,000$ grid of all land:

1. Recursive flood.
2. Recursive after `sys.setrecursionlimit(2_000_000)`.
3. Iterative, marking on pop — record peak stack length.
4. Iterative, marking on push — record peak stack length.

Then run BFS instead of DFS on the same grid and record the peak *queue* length. Compare it with (4). That number is the argument for BFS on grids.

On problem 4, build the impossible case

Construct a grid with a fresh orange that no rotten one can reach. Verify your solution returns `-1` and not a number. Then delete the `fresh` counter and record what it returns instead.

On problem 6, break the distinct check

Solve it, then replace the set of neighbouring island ids with a list. Find a grid where the two versions disagree — a U shape works — and record both answers.

Then the wrap-around experiment

Take your island counter, remove the bounds check entirely, and run it on:

```
1001
0000
0000
1001
```

Record the answer. Then say which of the four comparisons was responsible, and why the failure is silent in one direction and an `IndexError` in the other.

The model drill

1. Say what a vertex is and what an edge is, for a grid.
2. Say what “implicit graph” means and why grids are the classic case.
3. Write the four-way offsets and the eight-way offsets from memory.
4. Give a grid where four-way and eight-way give different island counts.
5. Say what one line of translation turns a graph algorithm into a grid algorithm.

The bounds drill

1. Write the check from memory. Count the comparisons.
2. Say what `grid[-1][3]` returns and why that is worse than an error.
3. Say which direction raises `IndexError` and which does not.
4. Say what shape the grid effectively becomes without the check.

The traversal-choice drill

1. Give both frontier sizes on a $1,000 \times 1,000$ open grid, BFS and DFS.
2. Say why that is the opposite of the general graph case.

3. Say when BFS is mandatory rather than preferable.
4. Say what recursion costs and what raising the limit costs.
5. Give the grid size below which recursion is safe.

The costs drill

1. Give `V` and `E` for an `r × c` grid and simplify `0(V + E)`.
2. Compute total operations for a `1,000 × 1,000` grid.
3. Compare the memory of a `visited` list-of-lists against a `bytearray`.
4. Give the cost of multi-source BFS with `k` sources and say why `k` does not appear.
5. Compute the naive alternative for 500 gates on a million cells.

The multi-source drill

1. Say what the one-line change is.
2. Say why each cell still gets the distance to its *nearest* source, with no comparison.
3. Give three problem statements that are this shape and the words that give it away.
4. Say how you count levels rather than steps, and where that loop goes.

The break-it drill

Trigger each and record the exact output or error:

1. No bounds check, on the four-corner grid.
 2. Marking on pop, on a million-cell all-land grid.
 3. Recursion on the same grid, before and after raising the limit.
 4. `[[False] * cols] * rows` as the visited grid.
 5. Flood fill where the new colour equals the old.
 6. Rotting oranges with the `fresh` counter removed.
 7. One BFS per gate, with 500 gates on a `1,000 × 1,000` grid.
-

The log-not-a-queue drill

1. State the one difference between Kafka and a queue, and three things that follow from it.
2. Define topic, partition, offset, consumer group — one line each.
3. Say what happens to a message when a consumer reads it.
4. Say what it costs the cluster to add a fifth consumer group.

The ordering drill

1. Answer “how does Kafka guarantee ordering?” in two sentences, narrowing the claim first.
2. Say how a message’s partition is chosen.
3. Say what ordering guarantee keying by `user_id` gives you, and what it does not.
4. Explain the ordering-versus-parallelism trade in one sentence.
5. Describe a hot partition, give an example key that causes one, and say why more brokers do not help.

The consumer-group drill

1. Say how partitions are assigned within a group.
2. Say what happens with 12 partitions and 17 consumers.
3. Say what a rebalance is and name three things that trigger it.
4. Name the classic rebalance incident and the two settings involved.
5. Say what cooperative sticky assignment changes.

The durability drill

1. Give the three `acks` settings and what each loses.
2. Say what `replication.factor=3, min.insync.replicas=2` guarantees.
3. Say why `min.insync.replicas=1` is the dangerous-looking-safe setting.
4. Say what the idempotent producer dedupes, and its exact scope.
5. Say where exactly-once stops, precisely.

The offsets drill

1. Say where consumer offsets are stored.
2. Give the three commit strategies and the guarantee each produces.
3. Say what the default gives you on a crash, and why that is usually wrong.
4. Say what redelivery forces on every consumer.

The numbers drill

1. Give per-broker throughput in MB/s and messages/s.
2. Size a cluster for 1M messages/s at 1 KB, showing the replication step.
3. Compute 7-day storage before and after compression.
4. Compute the partition count from both the throughput and the parallelism constraints.
5. Compute time-to-data-loss for a lag growing at 5,000/s with 7-day retention.

6. Give end-to-end latency, tuned for throughput and tuned for latency.
7. Compute the lag created by a 10-second rebalance at 50,000 msg/s.

The trade-offs drill

1. State the ordering-parallelism trade as a sentence.
2. State the retention-cost trade with a number.
3. Say what Kafka gives you that a queue does not, and vice versa.
4. Say why partition count is chosen early and what breaks when you change it.
5. Name four situations where Kafka is the wrong tool.

The failure drill

For each, say what happens and what you would build:

1. A broker dies after acknowledging with `acks=1`.
2. A consumer's lag exceeds the retention period.
3. A malformed message that fails on every attempt.
4. A key where one value is 30% of all traffic.
5. Seventeen consumers on a twelve-partition topic.
6. A consumer that takes six minutes to process a batch.
7. The database commit succeeds and the Kafka publish fails.

Two of the seven lose data silently. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Count the number of islands.* Cells as vertices and touching as edges, the implicit graph, the double loop as the outer loop, iterative with marking on push, the bounds check as four comparisons with the wrap-around consequence, and `O(rows × cols)`.
 2. *How long until every orange rots?* Multi-source BFS with all sources pushed before the loop, why the nearest-source distance falls out with no comparison, level counting for minutes, the `-1` case, and the arithmetic against one-BFS-per-source.
 3. *How does Kafka guarantee ordering?* Narrow the claim first — per partition only — then keys and hashing, per-entity ordering as the guarantee that matters, the parallelism trade, and hot partitions as the failure mode more brokers cannot fix.
-

Before you move on

- [] I can say what a vertex and an edge are for a grid.
- [] I write the neighbour function with bounds first, as its own function.
- [] I can write the four-way and eight-way offsets from memory.
- [] I know what `grid[-1]` does and why that failure is silent.
- [] I know which direction raises `IndexError` and which does not.
- [] I write grid traversals iteratively and know the two crash modes.
- [] I mark on push and know the stack size if I do not.
- [] I know BFS's and DFS's frontier sizes on an open grid.
- [] I know when BFS is mandatory.
- [] I can simplify $O(V + E)$ to $O(\text{rows} \times \text{cols})$ and say why.
- [] I know the memory cost of a visited grid and the `bytearray` alternative.
- [] I can spot multi-source BFS from the words in a statement.
- [] I know the one-line change and why `k` does not appear in the cost.
- [] I know to flood from the border when the condition is about reaching the edge.
- [] I know Kafka is a log and can name three consequences.
- [] I can define topic, partition, offset and consumer group.
- [] I can answer the ordering question by narrowing the claim first.
- [] I know how a partition is chosen and what a good key looks like.
- [] I can describe a hot partition and say why more brokers do not help.
- [] I know what happens with more consumers than partitions.
- [] I can name three rebalance triggers and the classic incident.
- [] I can give the three `acks` settings and what each loses.
- [] I know why `min.insync.replicas=1` is dangerous.
- [] I know exactly where Kafka's exactly-once stops.
- [] I know where offsets live and what the default commit gives me on a crash.
- [] I can size a cluster and its storage with the replication and compression steps.
- [] I can compute time-to-data-loss from a lag growth rate.
- [] I monitor lag as a time, not a count.
- [] I know Kafka has no per-message retry or DLQ, and what I build instead.
- [] I answered all three questions above out loud.

Shortest path in an unweighted graph

DATA STRUCTURES AND ALGORITHMS

Shortest path in an unweighted graph

SYSTEM DESIGN

Publish-subscribe versus point-to-point

1 What this is, and why they ask it

When every edge costs the same, breadth-first search finds the shortest path. Not usually — always, and by a guarantee you can prove in three sentences.

You wrote BFS on [day 127](#) and used the property in passing. Today it is the subject. There is a difference between *using* an algorithm and being able to say why it is correct, and this is the one place in the graph phase where interviewers actually ask for the argument. “Find the shortest path from A to B. Why does BFS work here?” is a real question with a real second half.

Today also covers what you do around the search rather than inside it: returning the path and not just its length, handling several shortest paths, searching from both ends when the graph is large, and — most importantly — recognising the exact moment the guarantee stops holding. **The instant edges have different costs, BFS is confidently wrong**, and knowing where that line sits is worth more than the algorithm.

By the end of this lesson you can state and prove the guarantee, reconstruct any shortest path, count how many there are, implement bidirectional search and say what it buys, and name the three variants that look unweighted and are not.

2 The story

Ammu’s father is under the scooter in the compound and he needs a twelve spanner, and the hardware shop shuts at eight.

It is twenty to eight. She is fourteen and she has been sent out to find one.

The obvious thing, and what she does first, is go next door to the Josephs. Uncle Joseph looks in his box and says no, only up to ten, but try the Nairs at the end because their son works on bikes.

Now, if she goes straight to the Nairs, she has done a sensible thing and possibly a slow one. The Nairs are at the end of the lane. If they do not have it either they will send her somewhere else, and by then it will be ten to eight and she will be three houses deep in a chain that started with a guess.

What she does instead — and she did not think about it, she just did it, because she wanted to be quick — is finish the lane first.

She goes to the Josephs. Then the Pillais on the other side. Then the flat above the Josephs. Then the shop at the corner that is not really a hardware shop but has a toolbox. Four places, all of them within thirty seconds of her own gate, before she walks anywhere at all.

The corner shop does not have a twelve. But the man there says his brother-in-law two lanes over definitely has a full set.

By then it is a quarter to eight and she has four suggestions, all of them one step away: the Nairs at the end, the brother-in-law two lanes over, somebody's cousin, and a mechanic near the main road.

She does the near ones first again.

She finds it at the Nairs, at eight minutes to eight. Their son had the whole set in a cloth roll under his bed, and it took her one step from Uncle Joseph's suggestion.

Walking back she works out that if she had followed the first suggestion straight through, she would have gone to the Nairs anyway — so it would have been fine. But if the Josephs had said “try the mechanic near the main road”, she would have walked ten minutes each way to be told to try somebody else, and the spanner would have been two doors down the whole time.

Doing the near ones first is not cleverness. It just means that when you find the thing, you know nothing closer was missed.

3 The idea in plain English

Ammu's method is breadth-first search, and the last line of the story is the guarantee.

Unweighted means every step costs the same. An **unweighted graph** is one where an edge is just a connection — no distance, no price, no time on it. Walking from one house to the next costs one step whether the house is next door or at the end of the lane. That single assumption is what everything today rests on.

The shortest path is the one with the fewest edges. With every edge costing one, “shortest” means “fewest steps”, and that is a much easier question than “cheapest”.

BFS explores in rings. All the houses one step from Ammu's gate, then all the houses one step from those, and so on. You met this on [day 127](#), and the queue produces the rings by itself.

The guarantee, stated precisely: the first time BFS reaches a vertex, it has reached it by the fewest possible edges. Not “usually”. Always, on an unweighted graph.

And the proof, in three sentences. BFS finishes every vertex at distance k before it looks at anything at distance $k+1$. So when a vertex is first discovered, it is discovered from a vertex at some distance d , and its own distance is $d+1$. If it could have been reached in fewer steps than that, it would have been discovered while processing an earlier ring — which has already finished. **Therefore the first discovery is the shortest.**

That is Ammu's last paragraph: because she does the near ones first, when she finds the spanner she knows nothing closer was missed.

The distance falls out of the search; the path takes one more field. Store, for every vertex, which vertex you reached it from — its **parent**. When you arrive at the goal, follow the parents back to the start and reverse. Ammu can tell you the route she took because she remembers who sent her where.

There can be several shortest paths, and BFS returns one of them. If two neighbours of the goal are both at distance 3, both give a path of length 4, and which one you get depends on the order the neighbours appear in your adjacency list. **Say “a shortest path”, not “the shortest path”** — and if a problem demands a specific one, you need an explicit tie-break.

Stopping early is legitimate. Once you pop the goal from the queue, no later discovery can beat it, so you can return immediately. On a large graph where the goal is close, that turns a full traversal into a small one.

Searching from both ends is the big optimisation. If you know both the start and the goal, run two searches — one forward from the start, one backward from the goal — one ring each, alternating, and stop when they touch. Because the number of vertices in a ring grows with the branching factor, two half-depth searches explore vastly fewer vertices than one full-depth search. This is **bidirectional BFS** and on a social graph it is the difference between minutes and milliseconds.

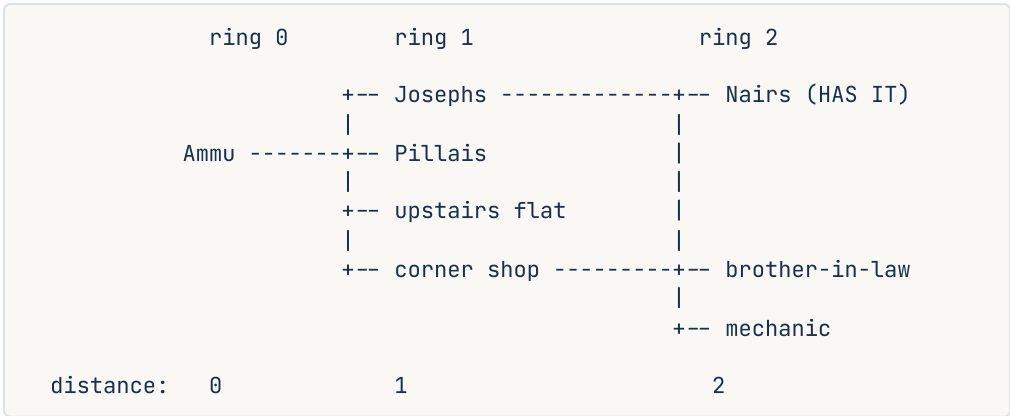
Now the boundary, and it is the important part of the lesson. The guarantee needs *every edge to cost the same*. Three situations look unweighted and are not:

- **Different edge costs.** A two-hop route that takes 10 minutes against a one-hop route that takes 90. BFS returns the one-hop route and is wrong. That is [day 136](#), Dijkstra.
- **Costs of only 0 and 1.** A maze where moving is free and breaking a wall costs one. BFS is wrong and Dijkstra is overkill; the answer is **0-1 BFS** with a deque — push a zero-cost neighbour to the front and a one-cost neighbour to the back. Still $O(V + E)$, no heap.
- **A cost per vertex rather than per edge.** Waiting at a node, a toll for entering. Usually convertible by pushing the vertex cost onto its incoming edges, and worth spotting rather than fighting.

The one-line test: *does every move cost exactly the same?* Yes → BFS. No → something else, and say which.

4 The picture

Ammu’s lane, with the rings marked:



What to notice. The Nairs are at distance 2, reached from the Josephs. The mechanic is also at distance 2. Both were discovered in the same ring, and Ammu would have been equally happy with either — there are two shortest paths of length 2 and no reason to prefer one.

The counter-example, which is what you must be able to draw:

UNWEIGHTED – BFS is right	WEIGHTED – BFS is WRONG
<pre>A ----- B ----- C +-----+ </pre>	<pre>A --1-- B --1-- C +-----10-----+ </pre>
A to C: 1 step (direct) 2 steps (via B)	A to C: BFS says 1 edge → cost 10 truth: 2 edges → cost 2
BFS returns the direct edge. Correct: 1 < 2.	BFS returns the SAME path, and it costs five times as much.

What to notice. The graph is identical. Only the numbers on the edges changed, and BFS’s answer did not — which is exactly the problem. It optimises the count of edges, and once edges are not interchangeable, the count is the wrong thing to optimise.

And bidirectional search, which is the picture that explains the saving:

```

branching factor b = 100, distance d = 6

ONE-DIRECTIONAL
ring 1: 100
ring 2: 10,000
ring 3: 1,000,000
ring 4: 100,000,000
ring 5: 10,000,000,000
ring 6: 1,000,000,000,000      ← explores 10^12 vertices

BIDIRECTIONAL — each side goes 3 deep, then they meet
forward: 100 + 10,000 + 1,000,000 = ~1,010,100
backward: same                    = ~1,010,100
-----
~2,000,000      ← 500,000x fewer

```

What to notice. The exponent halves. That is not a constant-factor improvement; it is the difference between possible and impossible on a large sparse graph, and it costs about fifteen extra lines.

5 The code, built step by step

Start with the distance, which is BFS with one dictionary.

```

from collections import deque

def shortest_distance(graph: dict[str, list[str]], start: str, goal: str) → int:
    """Fewest edges from start to goal, or -1 if unreachable."""
    if start == goal:
        return 0
    distance = {start: 0}
    queue = deque([start])
    while queue:
        current = queue.popleft()
        for neighbour in graph[current]:
            if neighbour in distance:
                continue
            distance[neighbour] = distance[current] + 1
            if neighbour == goal:
                return distance[neighbour]      # stop the moment it is found
            queue.append(neighbour)
    return -1

```

The `start == goal` line first, because otherwise a start with no edges returns `-1` for a distance that is obviously zero.

Returning as soon as the goal is *discovered* rather than when it is popped is a small extra saving and is correct for the same reason: a vertex's distance is fixed the moment it is first reached.

Now the path itself, which needs one more field.

```
def shortest_path(graph: dict[str, list[str]], start: str, goal: str) → list[str] | None:
    """One shortest path, or None. Ties broken by adjacency order."""
    if start == goal:
        return [start]
    parent: dict[str, str | None] = {start: None}
    queue = deque([start])
    while queue:
        current = queue.popleft()
        for neighbour in graph[current]:
            if neighbour in parent:
                continue
            parent[neighbour] = current
            if neighbour == goal:
                return _rebuild(parent, goal)
            queue.append(neighbour)
    return None
```

`parent` replaces `distance` and does three jobs: the seen set, the record of how each vertex was reached, and the set of reachable vertices. The start's parent is `None`, which is what stops the rebuild.

```
def _rebuild(parent: dict[str, str | None], goal: str) → list[str]:
    path: list[str] = []
    node: str | None = goal
    while node is not None:
        path.append(node)
        node = parent[node]
    return path[::-1] # built backwards
```

Four lines. Everything else about paths is a variation on this.

Counting all the shortest paths is a different change, and the one people get wrong:

```
def count_shortest_paths(graph: dict[str, list[str]], start: str, goal: str) → int:
    distance = {start: 0}
    ways = {start: 1} # how many shortest paths reach it
    queue = deque([start])
    while queue:
        current = queue.popleft()
        for neighbour in graph[current]:
            if neighbour not in distance:
                distance[neighbour] = distance[current] + 1
                ways[neighbour] = ways[current]
                queue.append(neighbour)
            elif distance[neighbour] == distance[current] + 1:
                ways[neighbour] += ways[current] # ANOTHER equally short route
    return ways.get(goal, 0)
```

The `elif` is the whole thing. Seeing a vertex again is normally ignored; here, if the distance you would assign *equals* the distance it already has, this is a second route of the same length and its count adds. If the distance would be larger, it is a longer route and you ignore it as usual.

All the shortest paths themselves need a list of parents rather than one:

```

from collections import defaultdict

def all_shortest_paths(graph, start, goal) → list[list[str]]:
    distance = {start: 0}
    parents: dict[str, list[str]] = defaultdict(list)
    queue = deque([start])
    while queue:
        current = queue.popleft()
        for neighbour in graph[current]:
            if neighbour not in distance:
                distance[neighbour] = distance[current] + 1
                parents[neighbour].append(current)
                queue.append(neighbour)
            elif distance[neighbour] == distance[current] + 1:
                parents[neighbour].append(current)
    ...

```

Then walk the parent structure backwards, branching at every vertex with more than one. **Flag the output size before writing it:** the number of shortest paths can be exponential, so this is bounded by the output rather than by the graph.

Now bidirectional search:

```

def bidirectional(graph: dict[str, list[str]], start: str, goal: str) → int:
    if start == goal:
        return 0
    front = {start: 0}
    back = {goal: 0}
    frontier_a, frontier_b = [start], [goal]
    while frontier_a and frontier_b:
        if len(frontier_a) > len(frontier_b): # always expand the smaller side
            frontier_a, frontier_b = frontier_b, frontier_a
            front, back = back, front
        nxt = []
        for vertex in frontier_a:
            for neighbour in graph[vertex]:
                if neighbour in back:
                    return front[vertex] + 1 + back[neighbour]
                if neighbour not in front:
                    front[neighbour] = front[vertex] + 1
                    nxt.append(neighbour)
        frontier_a = nxt
    return -1

```

Two details carry the whole benefit. **Always expand the smaller frontier** — that is the swap at the top, and without it a hub-heavy graph makes one side explode while the other sits still. And the meeting test is `neighbour in back`, checked *before* adding to `front`, so you notice the moment the two searches touch.

This works on an undirected graph as written. On a directed graph the backward search must follow **reversed** edges, which means either a reversed adjacency list or a graph structure that supports both directions — worth saying out loud, because it is the thing that makes people's bidirectional implementations quietly wrong.

The complete solution

```
"""Shortest paths on an unweighted graph: distance, path, counting, bidirectional."""

from __future__ import annotations

from collections import defaultdict, deque

def build(edges: list[tuple[str, str]]) → dict[str, list[str]]:
    graph: dict[str, list[str]] = defaultdict(list)
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)
    return graph

def shortest_path(graph: dict[str, list[str]], start: str, goal: str) → list[str] | None:
    """One shortest path, or None.  $O(V + E)$ ."""
    if start == goal:
        return [start]
    parent: dict[str, str | None] = {start: None}
    queue = deque([start])
    while queue:
        current = queue.popleft()
        for neighbour in graph[current]:
            if neighbour in parent:
                continue
            parent[neighbour] = current
            if neighbour == goal:
                path, node = [], neighbour
                while node is not None:
                    path.append(node)
                    node = parent[node]
                return path[::-1]
            queue.append(neighbour)
    return None

def count_shortest_paths(graph: dict[str, list[str]], start: str, goal: str) → int:
    """How many distinct shortest paths exist. The elif is the whole trick."""
    distance = {start: 0}
    ways = {start: 1}
    queue = deque([start])
    while queue:
        current = queue.popleft()
        for neighbour in graph[current]:
            if neighbour not in distance:
                distance[neighbour] = distance[current] + 1
                ways[neighbour] = ways[current]
                queue.append(neighbour)
            elif distance[neighbour] == distance[current] + 1:
                ways[neighbour] += ways[current]
    return ways.get(goal, 0)

def bidirectional(graph: dict[str, list[str]], start: str, goal: str) → int:
    """Search from both ends. Undirected only, as written.  $O(b^{(d/2)})$ ."""
    if start == goal:
        return 0
    if start not in graph or goal not in graph:
        return -1
    front, back = {start: 0}, {goal: 0}
    frontier_a, frontier_b = [start], [goal]
    while frontier_a and frontier_b:
        if len(frontier_a) > len(frontier_b):
            frontier_a, frontier_b = frontier_b, frontier_a
            front, back = back, front
```

```

    nxt: list[str] = []
    for vertex in frontier_a:
        for neighbour in graph[vertex]:
            if neighbour in back:
                return front[vertex] + 1 + back[neighbour]
            if neighbour not in front:
                front[neighbour] = front[vertex] + 1
                nxt.append(neighbour)
    frontier_a = nxt
    return -1

if __name__ == "__main__":
    lane = build([
        ("ammu", "josephs"), ("ammu", "pillais"),
        ("ammu", "upstairs"), ("ammu", "corner"),
        ("josephs", "nairs"), ("corner", "brother_in_law"),
        ("corner", "nairs"), ("corner", "mechanic"),
    ])
    print("path :", shortest_path(lane, "ammu", "nairs"))
    print("count:", count_shortest_paths(lane, "ammu", "nairs"))
    print("bidir:", bidirectional(lane, "ammu", "nairs"))
    print("gone :", shortest_path(lane, "ammu", "vasai"))

    # A diamond: four shortest paths of length 3 from s to t.
    diamond = build([
        ("s", "a"), ("s", "b"), ("a", "c"), ("a", "d"),
        ("b", "c"), ("b", "d"), ("c", "t"), ("d", "t"),
    ])
    print("diamond count:", count_shortest_paths(diamond, "s", "t"))

```

Running it:

```

path : ['ammu', 'josephs', 'nairs']
count: 2
bidir: 2
gone : None
diamond count: 4

```

Two things to look at. `count` is 2 — the Nairs are reachable in two steps via the Josephs *and* via the corner shop, and `shortest_path` returned only one of them, chosen by adjacency order. And the diamond gives 4, because there are two ways into the middle layer and two ways out, and $2 \times 2 = 4$. If your version returns 2 there, the `elif` is missing.

6 What it costs

Plain BFS.

each vertex enters the queue once	V
each vertex's neighbours scanned once	$2E$ undirected

	$O(V + E)$ time
distance / parent map	$O(V)$
queue	$O(\text{widest level})$

	$O(V)$ space

Early termination when the goal is found does not change the worst case — the goal might be the last vertex — but in practice it is enormous:

```

social graph,  $V = 1,000,000$ , average degree 200
goal at distance 2
  vertices explored:  $1 + 200 + 40,000 = \sim 40,000$ 
  instead of           $1,000,000$ 
                       $\rightarrow$  25x less work

```

Path reconstruction adds nothing to the order:

parent map	$O(V)$ space, already needed as the seen set
walking back	$O(\text{path length}) \leq O(V)$

Counting paths is the same BFS with one extra dictionary, so still $O(V + E)$. **Enumerating all paths is not**, and the difference is worth stating:

counting all shortest paths	$O(V + E)$	– a number
listing all shortest paths	$O(\text{paths} \times \text{length})$	– can be exponential

```

a chain of  $k$  diamonds:  $2^k$  shortest paths
 $k = 30 \rightarrow 1,073,741,824$  paths

```

Bidirectional search is where the interesting arithmetic is. With branching factor b and distance d :

one-directional	$O(b^d)$
bidirectional	$O(2 \times b^{(d/2)})$

$b = 200$ (an average social degree), $d = 6$

one-directional	$200^6 = 6.4 \times 10^{13}$	– the whole graph, many times over
bidirectional	$2 \times 200^3 = 16,000,000$	– four million times fewer

In practice both are capped by V , so on a million-vertex graph the honest comparison is:

one-directional	up to 1,000,000 vertices explored
bidirectional	~ 2 x sqrt-ish frontier, often tens of thousands → typically 10-100x on real graphs

Bidirectional costs about fifteen extra lines and doubles the memory — two distance maps instead of one — and needs both endpoints, so it is useless for “distance from A to everything”.

Compared with the alternatives:

BFS, unweighted	$O(V + E)$
0-1 BFS (deque)	$O(V + E)$
Dijkstra (binary heap)	$O((V + E) \log V)$
Bellman-Ford	$O(V \times E)$
Floyd-Warshall (all pairs)	$O(V^3)$

$V = 100,000$	$E = 500,000$
BFS	600,000 steps
Dijkstra	$600,000 \times 17 = \sim 10,000,000$ steps

Dijkstra is roughly seventeen times more work here for an answer BFS already has. That factor is the reason to say “BFS, because the graph is unweighted” rather than reaching for the general tool.

7 The traps

Using BFS on a weighted graph

The one that runs perfectly and returns the wrong answer:

```
graph = {"A": ["B", "C"], "B": ["C"], "C": []}
costs = {"A", "B"): 1, ("B", "C"): 1, ("A", "C"): 10}
print(shortest_path(graph, "A", "C"))
```

```
['A', 'C']
```

One edge, and it costs 10. The two-edge route costs 2. **BFS minimises the number of edges, not the sum of the weights**, and there is no warning of any kind. The tell in a problem statement is any number attached to a connection: a distance, a price, a duration, a toll. If there is one, the answer is not plain BFS.

Marking on pop

```
while queue:
    current = queue.popleft()
    if current in seen:
        continue
    seen.add(current)
    for neighbour in graph[current]:
        queue.append(neighbour)          # no check before pushing
```

The answer stays correct. The queue fills with duplicates:

```
V = 5,000, E = 12,000,000
mark on push:  queue peak ~5,000
mark on pop:   queue peak ~12,000,000
```

MemoryError

And when it does not run out of memory, it is `Time Limit Exceeded` on a solution you are sure is $O(V + E)$.

Computing the distance when you dequeue instead of when you discover

```
while queue:
    current = queue.popleft()
    for neighbour in graph[current]:
        if neighbour not in seen:
            seen.add(neighbour)
            queue.append(neighbour)
    distance[current] = ...          # too late, and from what?
```

Distances belong on discovery — `distance[neighbour] = distance[current] + 1` at the moment you push. Trying to compute them on the way out means you no longer know which vertex reached this one.

Forgetting `start == goal`

```
>>> shortest_distance(graph, "ammu", "ammu")
-1
```

The loop pops `ammu`, scans its neighbours, none of them is `ammu` because `ammu` is already in `distance`, and the queue empties. The answer should obviously be 0. One line at the top, and it is the first thing a hidden test checks.

The `KeyError` on an isolated vertex

```
for neighbour in graph[current]:
```

```
Traceback (most recent call last):
  File "sp.py", line 12, in shortest_path
    for neighbour in graph[current]:
KeyError: 'vasai'
```

A plain `dict` raises for a vertex that appears in no edge. A `defaultdict(list)` returns an empty list — but silently creates the key, which then makes the graph grow as you traverse it. `graph.get(current, [])` is the version that does neither.

Bidirectional search on a directed graph

```
bidirectional(directed_graph, "A", "Z")
```

The backward half follows forward edges, so it explores everything that `Z` points to rather than everything that points to `Z`. No error, and the answer is wrong whenever the graph is not symmetric. **The backward search needs a reversed adjacency list**, built once up front — one extra pass and $O(V + E)$ extra memory.

Assuming the returned path is canonical

```
assert shortest_path(graph, "ammu", "nairs") == ["ammu", "corner", "nairs"]
```

It might be `["ammu", "josephs", "nairs"]`. Both are length 3, both are correct, and which one comes back depends on the order neighbours were appended when the graph was built. **Tests should assert the length, or assert membership in a set of valid answers** — never a specific path, unless the problem defines a tie-break and you implemented it.

8 In the interview

How it gets asked

- “Find the shortest path from A to B. Why does BFS work here?” — the direct version, and the second half is the actual question.
- “What is the minimum number of moves to...” — puzzles, knight moves, lock combinations.
- “How many degrees of separation between these two people?”
- “Find the shortest transformation sequence between two words.”

- “There are several shortest paths. Return all of them.” — or, better, “how many are there?”
- “The graph has ten million vertices. Now what?” — bidirectional.

The first ninety seconds

“Every edge costs the same here, so shortest means fewest edges, and that is breadth-first search.

The guarantee I am relying on is that **the first time BFS reaches a vertex, it has reached it by the fewest possible edges** — and the reason is that BFS finishes every vertex at distance k before it looks at anything at distance $k+1$. So when a vertex is first discovered from something at distance d , its own distance is $d+1$; it cannot be less, because a shorter route would have been found in an earlier ring, and those rings are already finished.

Implementation: a queue and a `parent` dictionary that doubles as the seen set. Mark a vertex when I push it, not when I pop it — otherwise a vertex with a hundred edges pointing at it gets queued a hundred times, and on a dense graph that is a memory-limit failure rather than a slow solution.

For the path itself I walk the parents back from the goal and reverse. And I would say ‘a shortest path’ rather than ‘the shortest path’, because there can be several and which one I return depends on adjacency order — if you want a specific one I need a tie-break rule.

$O(V + E)$ time, $O(V)$ space, and I would stop as soon as the goal is discovered rather than finishing the traversal.

The assumption I want to state before you ask is that this only works because every edge costs one. If there were weights, a two-edge path could be cheaper than a one-edge path, the argument breaks at the first step, and BFS would return a confidently wrong answer with no warning. That would be Dijkstra — or, if the only weights were 0 and 1, a deque-based 0-1 BFS which is still linear.”

The follow-ups

“Prove it.”

“Induction on the ring number.

Base case: the start is at distance 0 and is correct.

Inductive step: suppose every vertex at true distance k has been assigned k and sits in the queue before anything at distance $k+1$. Now I pop a vertex u at distance k and look at an unvisited neighbour v . I assign v distance $k+1$. Is that right? It is at most $k+1$, because there is a path of that length through u . And it is at least $k+1$, because if v were reachable in $j \leq k$ steps it would have been discovered while I was processing ring $j-1$, which has already finished — and then it would not be unvisited now.

So it is exactly $k+1$.

The property the queue gives me is that all of ring k is popped before any of ring $k+1$, which follows from first-in-first-out: I only push ring $k+1$ vertices while popping ring k vertices, so they all go behind the ring- k entries that are still waiting.

The step that fails with weights is ‘at least $k+1$ ’. With different edge costs, a path through more edges can be cheaper, so a vertex’s cheapest route need not go through the ring that discovered it.”

“How many shortest paths are there?”

“One extra dictionary and one `elif`, and it stays $O(V + E)$.

Alongside the distance I keep `ways[v]` — the number of shortest paths that reach v . The start has one. When I discover a new vertex from u , it inherits `ways[u]`. And when I encounter a vertex I have already seen, I normally skip it — but here I check whether `distance[v] == distance[u] + 1`. If it is, this is a *second route of the same length*, so `ways[v] += ways[u]`. If the distance would be larger, it is a longer route and I ignore it as usual.

That `elif` is the whole algorithm and it is the part people miss, because every other BFS they have written ignores an already-seen vertex unconditionally.

If they want the paths themselves rather than the count, I keep a *list* of parents instead of one, and walk it backwards branching at every fork. But I would flag that the number of shortest paths can be exponential — a chain of thirty diamonds has a billion of them — so listing them is bounded by the output, not by the graph, and I would ask whether the count is enough.”

“Ten million vertices and you need the distance between two specific people.”

“Bidirectional BFS, and the exponent is the reason.

A one-directional search explores roughly b^d vertices, where b is the branching factor and d the distance. On a social graph b is a couple of hundred and d is about six, so that is 200^6 — far more than the graph contains, meaning I explore essentially all of it.

Searching from both ends, each side only goes half as deep, so it is $2 \times b^{(d/2)}$ — two times 200^3 , about sixteen million rather than sixty-four trillion. The exponent halves, which is a completely different class of improvement from a constant factor.

Two implementation details carry the benefit. **Always expand the smaller frontier**, alternating sides, because on a graph with hubs one side can explode while the other stays tiny. And check for the meeting point when I *generate* a neighbour, not when I dequeue it, so I stop the instant the two searches touch.

Two caveats I would state. It needs both endpoints, so it is useless for ‘distance from A to everyone’. And on a directed graph the backward search must follow **reversed** edges, which means building a reversed adjacency list up front — implementations that forget this are silently wrong on any asymmetric graph.”

“What if some moves cost more than others?”

“Then BFS is wrong and I would say so immediately rather than patch it.

If the costs are arbitrary positive numbers, it is Dijkstra: a priority queue instead of a plain queue, so I always expand the cheapest known vertex rather than the nearest by edge count. $O((V + E) \log V)$.

If the only costs are 0 and 1 — which is common, things like ‘moving is free, breaking a wall costs one’ — I would use **0-1 BFS**: a deque, pushing zero-cost neighbours to the **front** and one-cost neighbours to the **back**. That keeps the deque holding at most two distinct distance values, which is exactly the invariant a plain queue gives for uniform weights, so it stays $O(V + E)$ with no heap and no log factor.

And if there were negative weights, neither works and it is Bellman-Ford at $O(V \times E)$.

The general point I would make is that the cheapest correct algorithm depends entirely on the weight structure, and ‘is every edge the same cost’ is the first question I ask about any shortest-path problem — before I ask anything about the size of the graph.”

The model answer

“Two words of the same length, and a dictionary. Each step you may change one letter, and the result must be in the dictionary. Find the length of the shortest transformation sequence.”

“Shortest, with every step costing the same, so this is BFS. The interesting part is the modelling, and the naive model is the trap.

A vertex is a word. An edge joins two words that differ in exactly one letter. Undirected, unweighted.

I would not build the graph. With n words of length L , comparing every pair is n^2 comparisons at L each — ten thousand words is a hundred million comparisons before the search starts. The graph is implicit and I generate neighbours on demand.

Generating them: from a word of length L , produce the $L \times 25$ strings that differ in one position and keep those present in a set of the dictionary. For five-letter words that is 125 candidate strings and 125 set lookups per word — a constant, completely independent of dictionary size. So the whole search is $O(V \times L \times 25)$ rather than $O(V^2 \times L)$, which at ten thousand five-letter words is about 1.25 million operations against 500 million.

If L were large — say fifty — the direct generation gets expensive, and the alternative is a wildcard bucket: index every word under patterns like `h*t`, `*ot`, `ho*`, so all words sharing a pattern are neighbours. That is $O(V \times L)$ to build and $O(L)$ per lookup. The crossover is around $L = 25$, and I would mention it rather than pretend one model always wins.

Then it is plain BFS: distance dictionary as the seen set, mark on push, `deque`, return the distance when the end word is discovered, 0 if the queue empties.

Two edge cases before I code. The end word may not be in the dictionary at all, in which case the answer is 0 and I check that in one line rather than discovering it after a full traversal. And the start word is conventionally not required to be in the dictionary, which is worth confirming.

The optimisation I would offer next is bidirectional search, and here it is a genuine win rather than a flourish: the branching factor for a common five-letter word is dozens, so meeting in the middle roughly square-roots the explored set. Two frontiers as sets, always expand the smaller one, stop when a generated word appears in the other side's map. About fifteen extra lines, and on the standard version of this problem it is the difference between passing comfortably and timing out.

Cost: $O(V \times L \times 25)$ time, $O(V \times L)$ space for the dictionary set and the frontiers. I would state L explicitly rather than hide it in a constant, because for this problem the word length is a real parameter and the interviewer is likely to change it.”

9 Recall card

Every edge costs the same $\rightarrow$ shortest means fewest edges $\rightarrow$ BFS, and the first time BFS reaches a vertex is by the fewest possible edges.

The proof in one sentence: ring k finishes before ring $k+1$ begins, so a shorter route would already have been found. That step is exactly what breaks when edges have different costs.

Path: keep a `parent` map — it is also the seen set — and walk it backwards from the goal, then reverse. Say “a shortest path”, because ties are resolved by adjacency order.

Counting all shortest paths is one `elif`: when an already-seen vertex has `distance = distance[current] + 1`, add the counts. Still $O(V + E)$; *listing* them can be exponential.

Bidirectional halves the exponent — $2 \times b^{(d/2)}$ instead of b^d , four million times fewer on $b=200, d=6$. Always expand the smaller frontier, and on a directed graph the backward search needs **reversed** edges.

1 What this is, and why they ask it

There are two ways to deliver a message, and choosing wrongly is one of the most expensive mistakes in an architecture because it is very hard to undo later.

Point-to-point means one message, one recipient. The message goes into a queue, one consumer takes it, does the work, and it is gone. This is what a task queue does.

Publish-subscribe means one message, every interested recipient. The publisher does not know who is listening, and each subscriber gets its own copy. This is what a topic does.

They ask this because “one event, five consumers — queue or topic?” is a question with a correct answer and because the wrong answer has a specific, recognisable smell: the publisher ends up holding a list of everyone who cares, and adding a sixth consumer means changing and redeploying the publisher. **The real subject here is coupling**, not message delivery, and interviewers use this question to find out whether you think about who has to change when a requirement changes.

You met queues on [day 129](#) and Kafka on [day 130](#). Today is the decision between the two shapes, the hybrid that is what most real systems actually use, and the delivery guarantees that differ between them in a way that catches people.

By the end of this lesson you can pick the shape from the requirement in one sentence, describe the fan-out pattern that gives you both, name what each shape does about retries and failures, and explain why “tell everyone” and “do this once” need different machinery even though both look like sending a message.

2 The story

Manohar has been the watchman at Sai Krupa for nine years, and he does two completely different jobs with the same voice.

The first one happens when the tanker comes.

The building’s borewell has never been enough in April and May, so a tanker arrives some afternoons, and when it does everybody in all twenty-two flats needs to know within about two minutes, because the tanker fills the underground tank and then people run their motors and fill their own. If you miss it you wait for the next one.

So when the tanker turns into the gate, Manohar stands in the middle of the compound and shouts. Twice, loudly, facing up. Then he walks to the back and shouts again for the flats that face the other way.

He does not know who hears him. He does not have a list. He does not go door to door. Some flats are empty in the afternoon and those people simply miss it, and that is understood by everyone. Some flats have somebody who runs down to check the motor, some do not bother because they filled yesterday. Whether anyone acts on it is not his business. His job is that the announcement went out.

The second job is completely different, and it happens twenty times a day.

A courier turns up with a parcel for flat 402. Manohar signs for it, and then he takes it up to 402, or if nobody is in he keeps it in the cabin and tells them when they come home.

He does not shout about the parcel. Nobody in 301 needs to know that 402 got a parcel. And crucially, the parcel has to reach exactly one place — if it somehow got delivered to two flats there would be a problem, and if it got delivered to none there would be a bigger one, because the man in 402 is waiting for it and will ask.

So the parcel gets a record in the register, and it gets handed over, and if the person is not at home it stays in the cabin until they are. It is not finished until somebody has taken it.

Manohar has never once confused the two. Nobody has ever had to explain the difference to him.

The new man who covered for him in December got it wrong the first week. A parcel came for 402 and he was busy, so he announced it in the compound — “parcel for 402!” — and went back to the gate. Nobody came. The parcel sat there for two days, and the man in 402 was quite reasonably annoyed, because he had been told nothing and the announcement had gone to twenty-one people who did not care.

3 The idea in plain English

Manohar's two jobs are the two patterns, and the new man's mistake is the failure mode.

Point-to-point is the parcel. One message, exactly one recipient, and it is not finished until somebody has taken it. In code: a **queue**. A producer writes; several consumers may be reading the queue, but each message goes to exactly one of them. The consumers **compete** for messages.

Publish-subscribe is the shout. One message, every interested party gets a copy. In code: a **topic**. A publisher writes; every **subscriber** receives it. The subscribers do not compete — they each get everything.

The publisher does not know who is listening, and that is the point. Manohar does not have a list of flats that care about water. If a new family moves in, he does not change what he does. **Adding a subscriber does not touch the publisher**, and that is the property you are buying.

Point-to-point cares about completion; pub-sub cares about the announcement. The parcel has to reach somebody, so it is retried, held, and tracked. The shout goes out once — if you were not home, you missed it, and Manohar does not chase you. This difference runs through everything: **queues have acknowledgements and retries per message; a pure pub-sub broadcast often does not.**

The wrong shape shows up as a specific smell. If you use point-to-point where you needed pub-sub, the publisher ends up with a list of recipients — publishing five copies of the same event to five queues, with the list of interested parties hard-coded in the producer. Then adding a sixth consumer means changing and redeploying the producer, and the producer's team has to care about the analytics team's requirements. **That coupling is the cost, and it is invisible until the day somebody wants to add a consumer.**

And the other way round is the new man's mistake. Using pub-sub where you needed point-to-point means either nobody does the work, or everybody does it — five subscribers all charging the same card.

The vocabulary that tells you which one a system is offering:

- **Queue** — competing consumers, message consumed once. SQS, RabbitMQ queues, Celery.
- **Topic** — every subscriber gets a copy. SNS, Redis pub-sub, MQTT, RabbitMQ fanout exchanges.
- **Consumer group** — Kafka's answer, and it is genuinely both: **within** a group, consumers compete like a queue; **between** groups, each gets everything like a topic. That is why Kafka can serve both patterns from one topic, and it is worth naming precisely because people describe Kafka as “a queue” and it is not.

The hybrid is what real systems use, and it has a name: fan-out. Publish once to a topic; each interested team subscribes with **its own queue**. Now every team gets a copy — that is the pub-sub half — and within a team, several workers compete for messages from their own queue with proper retries and a dead-letter queue — that is the point-to-point half. **SNS to SQS is exactly this**, and it is the most common messaging shape in production systems.

The delivery guarantee differs, and this catches people. A queue holds a message until somebody acknowledges it. A pure broadcast — Redis pub-sub, for instance — delivers to whoever is connected *at that moment* and forgets. A subscriber that was restarting misses the message permanently, with no error anywhere. **“Fire and forget” is a real category and you must know which one you are using**, because it looks identical in a diagram.

And the filtering question. In pub-sub, every subscriber gets everything by default, and often a subscriber only wants a slice — “orders over ten thousand rupees”, “events for the Mumbai region”. Filtering at the broker saves the subscriber from receiving and discarding; filtering at the subscriber is simpler and wastes bandwidth. SNS supports message-attribute filters; Kafka does not, so you filter in the consumer or split into separate topics.

4 The picture

The two shapes side by side:

POINT-TO-POINT (queue)



message 1 → C1 only
message 2 → C2 only
message 3 → C1 only

consumers COMPETE
more consumers = more throughput

PUBLISH-SUBSCRIBE (topic)



message 1 → S1 AND S2 AND S3
message 2 → S1 AND S2 AND S3

subscribers each get EVERYTHING
more subscribers = same throughput,
more total work

What to notice. Adding a consumer to a queue makes the system faster. Adding a subscriber to a topic makes it do more work. Those are opposite effects and people conflate them constantly — “we’ll add another consumer to scale up” is right for a queue and meaningless for a topic.

The fan-out hybrid, which is what you should draw in an interview:

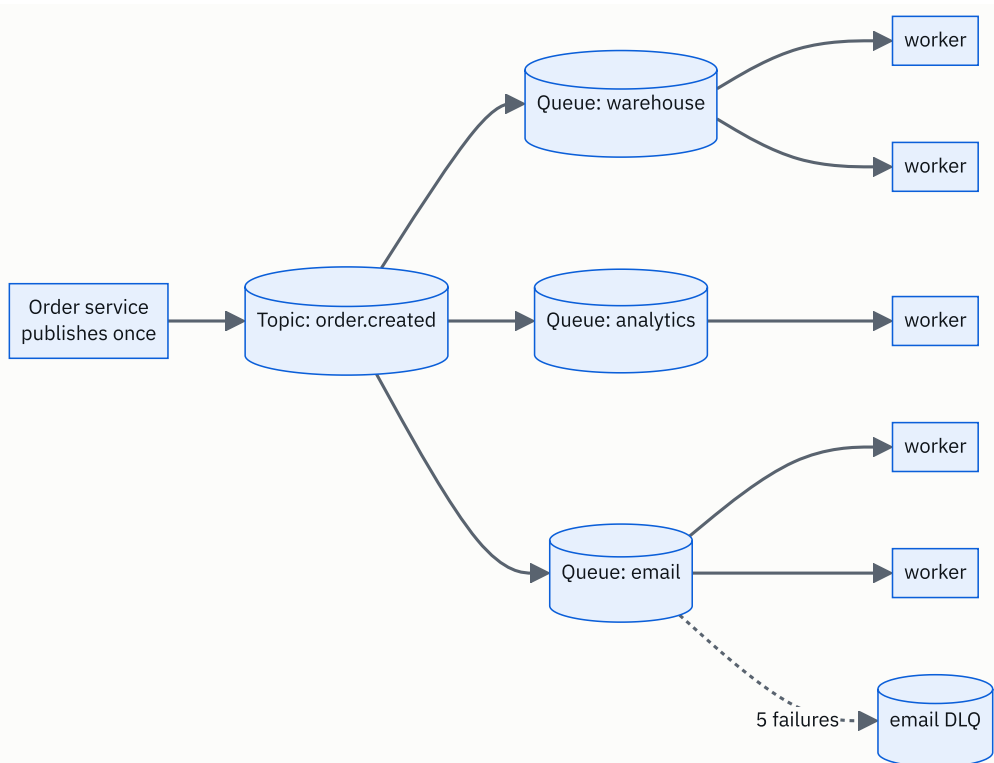


FIGURE 131.1

What to notice. The publisher has one arrow and knows about nothing downstream. Each team owns a queue, scales its own workers independently, and has its own dead-letter queue — so the email team’s poison message does not affect the warehouse team at all. **Adding a fourth team is one new queue and one new subscription, and the order service is not redeployed.**

And the anti-pattern, drawn so you recognise it:

```

Order service
|
+-----> warehouse queue
|
+-----> analytics queue      ← the producer now holds the LIST
|
+-----> email queue
|
+-----> fraud queue          ← adding this line = a producer deploy
  
```

Every consumer's existence is a fact the producer must know.
The analytics team's requirements are now the order team's problem.

What to notice. It works. It is not slow. The cost is entirely organisational, and it arrives months later in the form of “we need the order team to deploy before we can launch”.

5 How it actually works

The systems, and which shape each gives you

SYSTEM	SHAPE	RETRIES PER MESSAGE	NOTES
SQS	Queue	Yes, with DLQ	Competing consumers, at-least-once
SNS	Topic	Only to HTTP/Lambda targets	Usually fans out to SQS
SNS → SQS	Both	Yes, per queue	The standard production shape
RabbitMQ direct	Queue	Yes	Routing by exact key
RabbitMQ fanout exchange	Topic	Via bound queues	Each bound queue gets a copy
RabbitMQ topic exchange	Filtered pub-sub	Via bound queues	Wildcard routing keys
Kafka	Both	No, build it yourself	Groups compete; groups are independent
Redis pub-sub	Topic, fire-and-forget	No	Missed if not connected
Redis Streams	Both	Yes, via consumer groups	The durable version
MQTT	Topic	QoS-dependent	Device fan-out

The row worth memorising is Redis pub-sub. It is the classic “we used pub-sub and messages vanished” story: `PUBLISH` delivers to currently connected subscribers and to nobody else. A subscriber that is restarting, or briefly disconnected, misses the message with no error and no record. It is genuinely useful — cache invalidation, live dashboards, presence — and genuinely wrong for anything you must not lose. **Redis Streams is the durable alternative and is what you want most of the time.**

RabbitMQ’s exchanges, which is the clearest model

RabbitMQ separates publishing from routing, and the vocabulary is worth borrowing even when talking about other systems.

A publisher writes to an **exchange**, never to a queue. The exchange’s type decides what happens next:

direct exchange	routing key must match exactly	→ point-to-point
fanout exchange	ignore the key, copy to every bound queue	→ pub-sub
topic exchange	wildcard match: "order.*.created"	→ filtered pub-sub
headers exchange	match on message attributes	→ rarely used

publish to a topic exchange with key "order.mumbai.created"

queue A bound to "order.#"	→ receives it
queue B bound to "order.mumbai.*"	→ receives it
queue C bound to "order.delhi.*"	→ does not
queue D bound to "payment.#"	→ does not

The useful idea here is that the same publish can be point-to-point or pub-sub depending on what the receivers have bound. The publisher's code is identical in both cases, which is exactly the decoupling you want.

Kafka's version, precisely

Kafka does both with one mechanism and it is worth being able to state it exactly:

```
one topic, 6 partitions

consumer group "warehouse" with 3 consumers
  → each consumer gets 2 partitions
  → the group collectively sees every message ONCE (queue behaviour)

consumer group "analytics" with 1 consumer
  → that consumer gets all 6 partitions
  → it also sees every message (pub-sub behaviour)

the two groups have independent offsets and do not affect each other
```

Within a group: competing consumers. Between groups: broadcast. Say it in that form and it lands.

What Kafka does not give you is per-message retry, so a consumer that fails one message either blocks its partition or must publish it to a retry topic itself. **That is the thing SQS gives you free and Kafka does not**, and it is a real reason to put SQS behind SNS rather than use Kafka for work items.

Filtering, and where to do it

```
SNS message-attribute filter:
{"region": ["mumbai"], "amount": [{"numeric": [ ">", 10000 ]}]}
```

→ the subscriber never receives what it does not want

Three places you can filter, in increasing order of waste:

1. **At the broker** — SNS filter policies, RabbitMQ topic bindings. The subscriber receives only what it wants. Cheapest, and limited to whatever the broker can express.
2. **In the consumer** — receive everything, discard most. Simple, and you pay network and processing for messages you throw away. Fine when the filtered fraction is large.
3. **Separate topics per category** — `orders.mumbai`, `orders.delhi`. Perfect filtering, and it multiplies the number of topics and makes cross-category consumers awkward.

The rule: filter at the broker when it can express the condition, in the consumer when the discard rate is low, and split topics only when the categories are genuinely separate concerns.

Ordering, which differs between the shapes

A queue with competing consumers has **no ordering** — message 3 can finish before message 1 because different workers picked them up. Ordering requires either one consumer or partitioning by key, and both cap throughput.

A topic delivers to each subscriber independently, so each subscriber sees messages in whatever order that subscriber's own path preserves. **Pub-sub does not make ordering easier or harder; it multiplies whatever ordering problem you already had by the number of subscribers.**

Idempotency is required in both, for different reasons

- **Queue:** at-least-once redelivery after a consumer crash.
- **Pub-sub with durable subscriptions:** the same, per subscriber.
- **Fan-out:** both, plus the possibility that the fan-out itself duplicates.

There is no shape here that removes the need for idempotent consumers.

6 The numbers

What fan-out costs. One event, five subscribers:

publish	1 message written
delivered	5 copies
storage (durable queues)	5 x message size
processing	5 x the work

10,000 orders/s x 2 KB		
point-to-point	10,000 x 2 KB	= 20 MB/s
fan-out to 5	50,000 x 2 KB	= 100 MB/s

Five times the delivery volume for the same business events. That is the honest cost of pub-sub and it is worth stating before someone discovers it on the bill.

Kafka's version is much cheaper, and this is the strongest argument for it in a fan-out design:

```
Kafka: one copy stored, 5 consumer groups read it
storage          10,000 x 2 KB = 20 MB/s (x replication)
network out      5 x 20 MB/s   = 100 MB/s
```

```
SNS → 5 SQS queues: 5 stored copies AND 5 x network
Kafka, 5 groups:    1 stored copy AND 5 x network
```

The network cost is the same; the storage cost is five times lower. At 20 MB/s with seven days of retention that is the difference between 12 TB and 60 TB.

Cost in money, on managed services:

```
SNS publish      $0.50 per million
SQS operations    $0.40 per million (send + receive + delete = 3 calls)
```

```
10,000 orders/s = 864,000,000 orders/month
```

```
point-to-point (1 queue)
SQS: 864M x 3 = 2.6B ops      = $1,040/month
```

```
fan-out to 5 queues
SNS: 864M publishes           = $432
SQS: 864M x 5 x 3 = 13B ops    = $5,200
-----
                                $5,632/month
```

```
with SQS batching (10 messages per call)
SQS: 1.3B ops                  = $520
                                $952/month total
```

Batching is a factor of five to ten on the bill and it is the first thing to check.

Adding a subscriber: what changes.

point-to-point, producer holds the list	
code change in the producer	1 pull request
producer deploy	yes
coordination between teams	yes
risk to existing consumers	non-zero
fan-out	
new queue + subscription	1 config change
producer deploy	NO
coordination	none
risk to existing consumers	zero

The engineering cost of the wrong shape is not measured in milliseconds. It is measured in how many teams have to be in the room.

Throughput characteristics:

queue, N consumers	throughput scales with N (up to the broker's limit)
topic, N subscribers	throughput per subscriber unchanged;
	total broker work scales with N

SQS standard	effectively unlimited, ~3,000 msg/s per queue guidance
SQS FIFO	300 msg/s per message group (3,000 batched)
SNS	~9,000 publishes/s per topic (soft limit, raisable)
Redis pub-sub	~1,000,000 msg/s, zero durability
Kafka	~100,000+ msg/s per broker, durable

7 The trade-offs

Pub-sub buys you decoupling and costs you visibility. The publisher does not know who is listening, which is the entire benefit — and it also means nobody can answer “who consumes this event?” without going and looking. A topic with eleven subscribers, three of which are abandoned services nobody remembers, is a normal outcome after three years. **Mitigate with a registry: subscriptions documented and reviewed**, and it is worth saying that this is a process answer rather than a technical one.

Point-to-point buys you certainty and costs you flexibility. You know exactly who does the work, you get per-message retries and a dead-letter queue, and you can trace a message end to end. What you pay is that adding a second interested party means changing the producer or building a fan-out you should have had from the start.

Fan-out gives you both and multiplies your storage and your operational surface. Five queues means five sets of alarms, five DLQs, five sets of consumer scaling, and five copies of every message. It is the right default for events with several consumers, and it is not free.

Filtering at the broker saves bandwidth and limits what you can express. SNS filter policies handle equality, prefixes and numeric comparisons; anything more complex has to happen in the consumer. Pushing filtering into the broker also moves business logic into infrastructure configuration, where it is harder to test and easier to forget.

Fire-and-forget pub-sub is a different product wearing the same word. Redis pub-sub delivers to whoever is connected and forgets. That is perfect for cache invalidation, where a missed message costs one stale read, and catastrophic for order events. **The failure is silent** — no error, no metric, no DLQ — which makes it the worst kind of wrong choice.

And ordering is not helped by either shape. Competing consumers destroy order; multiple subscribers each have their own ordering problem. If order matters you are partitioning by key and capping parallelism, whichever shape you picked.

When would I not use pub-sub? When there is exactly one consumer and no realistic prospect of a second — a queue is simpler, cheaper and easier to reason about, and the fan-out can be introduced later with the consumer's queue kept in place. When the “event” is really a command — “charge this card” has one correct recipient, and broadcasting a command is how you get five charges. And when the volume is enormous and the consumers few, where Kafka's single-copy model beats a real fan-out on storage by the number of subscribers.

8 In the interview

How it gets asked

- “One event, five consumers. Queue or topic?” — the direct version.
- “How do you add a new consumer without touching the producer?”
- “What is the difference between a queue and a topic?”
- “You are using Redis pub-sub and messages are going missing. Why?”
- “How does Kafka do both?”
- “Should this be an event or a command?” — the framing question underneath all of it.

The first ninety seconds

“Topic, and specifically a fan-out: publish once to a topic, and each of the five consumers subscribes with its own queue.

The reason is not delivery, it is coupling. If I use five queues and have the producer write to all of them, it works — but the producer now holds the list of everyone who cares. Adding a sixth consumer means a code change and a deploy of the producer, and the new team has to coordinate with the producer’s team to launch. With a topic, the producer publishes one message and knows about nothing downstream; adding a consumer is one new subscription and one new queue, and the producer is not touched.

The reason I want a queue per consumer rather than plain pub-sub is that each team then gets proper per-message semantics: their own retries, their own dead-letter queue, their own scaling, and their own backlog. If the email consumer hits a poison message, the warehouse consumer is completely unaffected. Plain pub-sub to five endpoints gives me none of that.

On AWS that is SNS in front of five SQS queues, which is the standard shape. On Kafka it is one topic and five consumer groups, which gets me the same decoupling with one stored copy instead of five — within a group consumers compete like a queue, and between groups every group sees everything.

The costs I would name up front: five times the delivery volume and five times the storage on the SQS version; five sets of alarms and DLQs to operate; and reduced visibility, because nobody can now answer ‘who consumes this event’ without looking it up.

Are these five consumers doing genuinely different things, or is this one job being split for throughput? Because if it is the second, it is one queue with five workers and none of this applies.”

The follow-ups

“Why not just have the producer write to five queues?”

“It works and it is the shape I would push back on, because the cost is organisational rather than technical and it arrives later.

The producer now contains a list of its consumers. That means three things. Adding a consumer is a code change and a deploy of a service that has nothing to do with the new requirement — so the analytics team’s launch is blocked on the order team’s release cycle. The producer’s failure handling gets more complicated, because it now has to decide what to do if four writes succeed and the fifth fails. And the producer’s owners end up fielding questions about downstream systems they do not run.

There is also a correctness wrinkle worth naming: five separate writes are not atomic. If the process dies after three of them, three consumers have the event and two do not, and there is no record of the discrepancy. With a topic there is one write and the broker handles the fan-out, so it is one thing that either happened or did not.

The one case where I would accept it is two consumers that are genuinely never going to become three, where the extra topic is more machinery than the problem needs. But I would say that out loud as a deliberate choice rather than let it happen by default.”

“Messages are disappearing from our Redis pub-sub. What is going on?”

“Redis pub-sub is fire-and-forget. `PUBLISH` delivers to subscribers that are connected **at that instant** and to nobody else. There is no buffer, no storage, no acknowledgement and no replay. A subscriber that is restarting, or briefly disconnected by a network blip, or slow enough to have its output buffer filled and be dropped, misses those messages permanently — and nothing anywhere reports an error.

That is not a bug, it is what it is for. It is genuinely good at cache invalidation, live dashboards and presence, where a missed message costs you one stale read and the next message fixes it.

It is completely wrong for anything you must not lose, and the reason it keeps happening is that the API looks identical to a durable system. There is no signal in the code that tells you the difference.

The fix, in increasing order of change: **Redis Streams** with consumer groups gives me durability, acknowledgements and replay on the same server. Or move to a real broker — SNS to SQS, or Kafka — if the events matter enough to justify it. I would also add a monitoring answer regardless: if messages can be lost silently, I want a sequence number per publisher so a subscriber can at least *detect* a gap.”

“Should this be an event or a command?”

“This is the question underneath the whole choice and I would rather answer it than the queue-or-topic question, because it settles that one.

A command is an instruction to one recipient: ‘charge this card’, ‘send this email’, ‘ship this order’. It has one correct handler, it must happen exactly once in effect, and the sender expects it to be carried out. Commands are point-to-point. Broadcasting a command is how you get five charges on one card.

An event is a statement of fact: ‘order 4471 was placed’. It has already happened, the publisher has no opinion about who should react, and zero or five recipients are both fine. Events are pub-sub.

The naming is the tell. If the message is a verb in the imperative — `SendEmail` — it is a command. If it is a noun and a past-tense verb — `OrderPlaced` — it is an event.

And the design advice that follows: **publish events, not commands, when you have a choice**, because an event does not need to know its recipients and a command does. ‘Order placed’ lets the email service decide to send an email; ‘send order confirmation email’ means the order service has decided that, and now the order service knows about emails.”

“How does Kafka give you both, exactly?”

“Through consumer groups, and the precise sentence is: **within a group, consumers compete; between groups, every group sees everything.**

Concretely: a topic with six partitions. The warehouse group runs three consumers, so each gets two partitions and the group collectively processes every message once — that is queue behaviour. The analytics group runs one consumer, which gets all six partitions and also sees every message — and its offsets are completely independent of the warehouse group’s. Add a third group and it costs the cluster nothing extra in storage, because there is still one copy of the log.

That single-copy property is Kafka’s real advantage in a fan-out design. SNS to five SQS queues stores five copies; Kafka with five groups stores one. At 20 MB a second with seven days of retention that is 12 TB against 60 TB.

What Kafka does not give me is per-message acknowledgement, so there is no built-in retry or dead-letter behaviour — a message that always fails blocks its partition unless I build retry topics myself. That is the concrete thing SQS gives me free, and it is why I would not automatically choose Kafka for work items even in a system that already runs Kafka for events.”

The model answer

“A ride-hailing app. When a trip ends, several things must happen: charge the rider, pay the driver, send both receipts, update the driver’s rating eligibility, feed the analytics warehouse, and check for fraud. Design the messaging.”

“The first thing I would do is sort those six into events and commands, because they are not the same kind of message and putting them all on one bus is the mistake.

`TripCompleted` is one event, published once to a topic. It is a statement of fact with a trip ID, rider, driver, distance, duration and fare. The trip service publishes it and knows nothing about who reacts.

Analytics, fraud, and rating eligibility are pure subscribers. They react to the fact, nothing depends on their answer in the trip’s own flow, and if a fourth team wants trip data next quarter they add a subscription and nobody deploys anything. Each gets its own queue behind the topic, so each has independent retries, its own backlog, and its own dead-letter queue. Fraud being slow must not delay analytics.

Charging the rider is not an event, it is a command, and I would treat it differently. It has exactly one correct handler, it moves money, and it must not happen twice. So the trip service — or a payment orchestrator subscribed to the topic — sends a `ChargeRider` command to a dedicated queue with an idempotency key equal to the trip ID. Point-to-point, one consumer group, per-message retry, DLQ, and reconciliation. If I broadcast that as an event and let a payment service ‘react’, I have lost the ability to say who is responsible for it happening exactly once.

Paying the driver is the same shape and I would make it a separate command on a separate queue, not bundled with the charge, because the two fail independently and one failing must not block the other. The saga from [day 121](#) governs the pair: if the charge is ultimately impossible, the driver payment still happens and the loss is written off — a business decision I would confirm rather than assume.

Receipts are events, or rather they are triggered by them. The receipt service subscribes to `PaymentCaptured` rather than to `TripCompleted`, because a receipt for a payment that failed is worse than a late receipt. That ordering constraint comes from the data, not from the message bus, which is the robust way to express it.

Shape: SNS topic to SQS queues, or a Kafka topic with consumer groups, and I would choose on volume. At a million trips a day — about twelve a second — SNS to SQS is simpler and the storage multiplication does not matter. At a hundred times that, Kafka’s single stored copy across five groups saves real money, and I would accept building retry topics.

Every consumer is idempotent on the trip ID, because both shapes are at-least-once. The charge consumer uses a dedup table with a unique constraint; the analytics consumer deduplicates on ingest; the receipt consumer keys on `(trip_id, recipient)`.

What I would monitor: per-queue oldest-message age rather than depth, and DLQ depth per consumer with a named owner for each. And I would keep a subscription registry, because the failure mode of pub-sub two years in is eleven subscribers of which three belong to services nobody remembers deploying.”

9 Recall card

Point-to-point: one message, one consumer, consumers compete. Pub-sub: one message, every subscriber gets a copy, and **the publisher does not know who is listening** — which is the whole point.

The smell of the wrong choice: the producer holds a list of its consumers, so adding a consumer means deploying the producer. Adding a consumer to a queue makes the system *faster*; adding a subscriber to a topic makes it do *more work*.

Fan-out is the production shape: publish once to a topic, one queue per consumer. Pub-sub decoupling plus per-consumer retries, DLQs and scaling. SNS → SQS, or a Kafka topic with one consumer group per team.

Kafka does both with consumer groups: *within* a group consumers compete; *between* groups every group sees everything — and it stores one copy where SNS→SQS stores five.

Redis pub-sub is fire-and-forget — delivered only to whoever is connected right now, lost silently otherwise. Fine for cache invalidation, wrong for anything that matters; Redis Streams is the durable version. And: **publish events (facts, past tense, many listeners), send commands (imperatives, one handler).**

DSA topic: Shortest path in an unweighted graph **System design topic:** Publish-subscribe versus point-to-point

Code these, in this order

One rule for the whole set: **before writing anything, answer out loud “does every move cost exactly the same?”** If yes, BFS and say so. If no, name what you would use instead. Two of the problems below are deliberately chosen to make you check.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Shortest Path in Binary Matrix	LeetCode 1091 (Medium)	BFS on a grid with eight directions, and the blocked-start case.
2	Word Ladder	LeetCode 127 (Hard)	The implicit neighbour function, then bidirectional.
3	Open the Lock	LeetCode 752 (Medium)	An implicit state graph, plus a set of forbidden vertices.
4	Minimum Genetic Mutation	LeetCode 433 (Medium)	The same shape as 2, small enough to write in ten minutes.
5	Number of Ways to Arrive at Destination	LeetCode 1976 (Medium)	Counting shortest paths — and it is weighted . Check first.

On problem 5, notice the trap before you write

Read the constraints. The roads have times on them. Say out loud what that does to BFS’s guarantee, and what you need instead. Then write the counting logic — the `elif` idea is the same, only the search underneath changes.

On problem 2, write three versions and count

1. Build the graph explicitly by comparing every pair of words.
2. BFS with generated neighbours.
3. Bidirectional BFS with generated neighbours.

For each, count how many words are examined and how long it takes on the largest LeetCode case. Three pairs of numbers. The ratio between (1) and (2) is the modelling win; between (2) and (3) is the search win.

On problem 3, add path reconstruction

The problem asks only for the number of turns. Return the actual sequence of combinations as well, using a parent map. Then check whether your answer is the only shortest sequence, by counting them.

Then the weighted counter-example

Build this graph by hand:

```
A --1-- B --1-- C
A ----10----- C
```

Run your BFS shortest-path function on it, ignoring the weights. Record the path it returns and its true cost. Write one sentence explaining exactly which step of the correctness proof fails.

The guarantee drill

1. State the guarantee in one sentence.
2. Give the three-sentence proof.
3. Name the exact step that fails when edges are weighted.
4. Say what “a shortest path” versus “the shortest path” means and why it matters for tests.
5. Say when you may stop early, and why that is safe.

The variants drill

1. Write the distance version from memory.
2. Write the path version and the rebuild.
3. Write the counting version and point at the `elif`.
4. Say what changes to list *all* shortest paths, and what its complexity is bounded by.
5. Give a graph shape where the number of shortest paths is exponential.

The bidirectional drill

1. Give the cost of one-directional and bidirectional search in terms of `b` and `d`.
2. Compute both for `b = 200`, `d = 6`.
3. Say why you always expand the smaller frontier.
4. Say where the meeting check goes and why not later.
5. Say what changes on a directed graph.
6. Name two situations where bidirectional is useless.

The boundary drill

1. Give the one-line test for whether BFS applies.
2. Name the three situations that look unweighted and are not.
3. Describe 0-1 BFS and say why it stays linear.
4. Give the costs of BFS, 0-1 BFS, Dijkstra, Bellman-Ford and Floyd-Warshall.
5. Compute BFS versus Dijkstra on `V = 100,000`, `E = 500,000` and give the ratio.

The break-it drill

Trigger each and record the exact output or error:

1. BFS on the weighted counter-example above.
 2. Marking on pop, on a graph with 12 million edges.
 3. `shortest_distance(graph, x, x)` without the equality check.
 4. `graph[current]` on a plain dict with an isolated vertex.
 5. Bidirectional search on a directed graph without reversed edges.
 6. A test asserting one specific shortest path, with the adjacency order changed.
 7. Counting shortest paths with the `elif` removed, on a diamond.
-

The two-shapes drill

1. Define point-to-point and pub-sub in one line each.
2. Say what adding a consumer does in each case.
3. Give the smell of the wrong choice, in one sentence.
4. Say which shape cares about completion and which about the announcement.
5. Give the parcel and the shout for a real system you know.

The fan-out drill

1. Draw the fan-out shape from memory.
2. Say what the topic gives you and what the per-consumer queue gives you.
3. Say what happens to the warehouse consumer when the email consumer hits a poison message.
4. Name the AWS services and the Kafka equivalent.
5. Say what changes when a fourth team wants the data.

The systems drill

1. For each of SQS, SNS, RabbitMQ fanout, Kafka, Redis pub-sub, Redis Streams: which shape, and does it retry?

2. Explain RabbitMQ's four exchange types in one line each.
3. State Kafka's both-shapes sentence precisely.
4. Say what Kafka does not give you that SQS does.

The Redis drill

1. Say exactly what `PUBLISH` delivers to and what it does not.
2. Name three cases where that is the right behaviour.
3. Say why the failure is worse than an error.
4. Give the durable alternative and one fallback for detecting loss.

The events-versus-commands drill

1. Define each in one sentence.
2. Give the naming tell.
3. Say which shape each belongs to.
4. Say what goes wrong when you broadcast a command.
5. State the design advice that follows, and why.

The numbers drill

1. Compute delivery volume for 10,000 events/s at 2 KB, point-to-point and fanned out to five.
2. Compare stored copies for SNS→SQS and Kafka with five consumers.
3. Compute the monthly SQS bill at 864M messages, with and without batching.
4. List what changes when adding a subscriber, in each shape.
5. Give throughput limits for SQS standard, SQS FIFO, SNS and Redis pub-sub.

The trade-offs drill

1. Say what pub-sub costs you in visibility, and the mitigation.
2. Say what point-to-point costs you in flexibility.
3. Say what fan-out multiplies.
4. Give the three places to filter and the rule for choosing.
5. Say what neither shape helps with.
6. Name three cases where you would not use pub-sub.

The failure drill

For each, say what happens and what you would build:

1. The producer writes to five queues and dies after the third.

2. A subscriber restarts during a Redis `PUBLISH`.
3. A command is published to a topic with five subscribers.
4. A poison message in a shared queue serving three teams.
5. A topic with eleven subscribers, three of them abandoned.
6. Five SQS queues at 864M messages a month with no batching.
7. An event consumer that is not idempotent, after a crash before acknowledgement.

Two of the seven fail silently. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the shortest path from A to B. Why does BFS work here?* The guarantee, then the three-sentence proof, then the parent map and rebuild, then the unweighted assumption named before being asked — with the counter-example ready.
 2. *The graph has ten million vertices. Make it faster.* Bidirectional search, the `b^d` versus `2 · b^(d/2)` arithmetic with real numbers, expanding the smaller frontier, the meeting check placement, and the reversed-edges caveat for directed graphs.
 3. *One event, five consumers. Queue or topic?* Fan-out, and the reason stated as coupling rather than delivery — then the per-consumer queue for retries and DLQs, the AWS and Kafka shapes, and the storage multiplication named as the cost.
-

Before you move on

- ☐ I can state the shortest-path guarantee precisely.
- ☐ I can give the three-sentence proof.
- ☐ I know which step of the proof weights break.
- ☐ I can write the distance, path and counting versions from memory.
- ☐ I know why the counting version needs an `elif` and what it checks.
- ☐ I know listing all paths can be exponential and counting them is linear.
- ☐ I say “a shortest path” and know why tests must not assert one.
- ☐ I handle `start == goal` explicitly.
- ☐ I can implement bidirectional BFS.
- ☐ I can do the `b^d` versus `2 · b^(d/2)` arithmetic.
- ☐ I know to always expand the smaller frontier.

- [] I know the backward search needs reversed edges on a directed graph.
- [] I can name two cases where bidirectional is useless.
- [] I can give the one-line test for whether BFS applies.
- [] I can name the three unweighted-looking-but-not cases.
- [] I know what 0-1 BFS is and why it stays linear.
- [] I can compare BFS and Dijkstra with numbers.
- [] I can define point-to-point and pub-sub and say what adding a consumer does to each.
- [] I can name the smell of the wrong choice.
- [] I can draw the fan-out shape and say what each half buys.
- [] I know the AWS and Kafka versions of fan-out.
- [] I can state Kafka's both-shapes sentence precisely.
- [] I know Kafka has no per-message retry and what that forces.
- [] I know exactly what Redis pub-sub delivers to, and why loss is silent.
- [] I can distinguish an event from a command and give the naming tell.
- [] I know what goes wrong when a command is broadcast.
- [] I can compute the fan-out storage and bill, with and without batching.
- [] I can name three cases where pub-sub is the wrong choice.
- [] I answered all three questions above out loud.

Cycle detection in an undirected graph

DATA STRUCTURES AND ALGORITHMS

Cycle detection in an undirected graph

SYSTEM DESIGN

Stream processing basics

Cycle detection in an undirected graph

1 What this is, and why they ask it

A cycle is a path that leaves a vertex and comes back to it without reusing an edge. Detecting one in an undirected graph is a traversal with a single extra condition, and that condition is the entire lesson, because the obvious version of it is wrong.

The obvious version says: if I reach a vertex I have already seen, that is a cycle. In a directed graph that is nearly right. In an **undirected** graph it is completely wrong, and it reports a cycle on a graph with two vertices and one edge — because an undirected edge `A-B` appears in both adjacency lists, so the moment you walk from `A` to `B` and look around, you see `A`, which you have obviously already seen.

The fix is to ignore the vertex you just came from, and it is one parameter. Getting that right, and being able to say why the directed version needs a completely different mechanism, is what the question tests.

They ask it because it is short enough for a warm-up and because it underlies things they will ask next. “Is this graph a tree?” is “connected and acyclic”. “Can these `n - 1` edges connect everything?” is the same. Union-Find, which arrives on [day 138](#), detects cycles as a side effect of building groups, and comparing the two approaches is a standard follow-up.

By the end of this lesson you can write the DFS version with the parent check and the BFS version, say precisely why the parent check is needed and when excluding by vertex is not enough, detect a cycle with Union-Find, and state the edge-count shortcut that answers the question with no traversal at all.

2 The story

The intercom in Ashirwad Society has never worked properly, and in February Vasant finally set out to find out why.

He is an electrician, sixty-one, and he has done work in that building since it was put up. The intercom runs on a wire that goes from flat to flat through a series of small grey junction boxes screwed to the wall at the corner of every landing. There are about thirty of them.

Nobody has a drawing of it. The man who wired it originally died in 2011 and the work was extended twice after that by people who did not ask him anything.

So Vasant did the only thing available, which was to walk it.

He started at the box by the main gate, opened it, and picked one of the wires leaving it. He followed it to the next box and put a chalk mark on that one. Then a wire out of that box to the next, chalk. Then the next. Up the stairs, along the second-floor landing, through the box behind the lift shaft, and on.

The rule he gave himself was simple. If he ever opened a box and found chalk already on it, then this wire had come round in a loop, and that was worth knowing, because a loop in that wiring is exactly the kind of thing that makes an intercom hum.

Twenty minutes in, he hit the problem with the rule.

He had gone from box 14 to box 15, and box 15 had two wires leaving it — the one he came in on, and one more. He opened the second one, followed it, and it went to box 14. Which had chalk on it.

He stood there for a second and then laughed at himself, because of course it did. He had just come from box 14. The wire between 14 and 15 is one wire. Looking at it from the 15 end and finding that it goes to 14 is not a discovery. It is the same wire he walked in on.

So he added a second half to the rule, and it is the whole of what he learnt that morning: **a chalked box only means a loop if it is not the box you just came from.**

With that, the walk went properly. He found the real loop at about eleven — a wire from box 22 running all the way down the back of the building to box 9, put in by somebody in about 2016, presumably because it was shorter than going round. Two paths to the same place.

He cut it, and the humming stopped.

3 The idea in plain English

Vasant's rule, both halves, is the algorithm.

A cycle is a path that returns to where it started without reusing an edge. That last clause is what makes undirected cycle detection different from directed. Walking $A \rightarrow B \rightarrow A$ is not a cycle, because you used the same edge twice — you went out and came back on the one wire. A cycle needs at least three distinct vertices in an undirected simple graph.

The traversal is unchanged. DFS or BFS, a [seen](#) set, exactly as on [day 128](#). Nothing new.

The naive condition: “I reached a vertex already in `seen`, so there is a cycle.” In a directed graph this is nearly right. In an undirected graph it is wrong on the smallest possible input, because an undirected edge is stored twice — `graph[A]` contains `B` and `graph[B]` contains `A`. So walking `A → B` and then looking at `B`’s neighbours immediately shows `A`, which is in `seen`. That is not a loop, it is **the edge you just walked, seen from the other end**.

The fix: pass down where you came from, and skip it. Each recursive call carries the vertex that called it — its **parent** — and the check becomes “seen, **and not my parent**”. That is Vasant’s second half.

```
if neighbour not in seen:      recurse with parent = current
elif neighbour != parent:      CYCLE
```

Two lines, and the `elif` is the algorithm.

Why the directed version cannot use this. In a directed graph, `A → B` puts `B` in `A`’s list and puts nothing in `B`’s list. So there is no mirror edge to ignore, and the parent check has nothing to do. What a directed graph needs instead is the three-colour scheme from [day 128](#) — grey means “on the current path” — because there the danger is reaching an *ancestor*, and a vertex that is merely `seen` might be a finished region rather than an ancestor. **Two different problems that share a name.** [Day 133](#) is the other one.

And the case where excluding by vertex is not enough. If the graph can have two separate edges between the same pair — **parallel edges** — then `A-B` twice genuinely is a cycle, and skipping “the parent vertex” wrongly ignores it. The fix is to exclude by **edge** rather than by vertex: pass the id of the edge you came in on. Most problems promise no parallel edges; when they do not, ask.

A self-loop is also a cycle, trivially — an edge from a vertex to itself. The parent check does not catch it, because the neighbour equals the current vertex and not the parent. One extra condition, or a promise from the problem statement.

BFS works too, with the same idea. Store each vertex’s parent in a dictionary rather than passing it down, and the condition is identical. Use BFS when the graph might be deep enough to overflow the recursion stack.

And there is a shortcut that answers the question with no traversal at all. A connected undirected graph with `V` vertices and no cycle is a **tree**, and a tree has exactly `V - 1` edges. So:

```
E > V - 1 → there is definitely a cycle
E < V - 1 → the graph cannot be connected
E = V - 1 → cycle if and only if it is disconnected
```

On a graph promised to be connected, “is $E \geq V$ ” answers the whole question in constant time. That is worth knowing, worth saying, and worth not using when connectivity is not promised — because a graph with $V = 5$, $E = 4$ can be a tree, or it can be a triangle plus a separate edge.

Union-Find is the other real answer. Process the edges one at a time: for each, check whether its two endpoints are already in the same group. If they are, this edge closes a cycle. If not, merge them. That is effectively linear and it is the right tool when edges arrive over time — the same rule as [day 129](#).

4 The picture

Vasant’s mistake, drawn:

walk 14 → 15, then look at 15's neighbours

box 14 [chalked]

box 15 [chalked]

^

|

|

+

+

the SAME wire, seen
from the other end

naive rule: "14 is chalked!" → reports a cycle

real rule: "14 is my parent" → skip it

What to notice. There is one wire and two views of it. Every undirected edge produces exactly this false alarm, once, at every vertex — which is why the naive version reports a cycle on literally any graph with an edge in it.

The real loop:

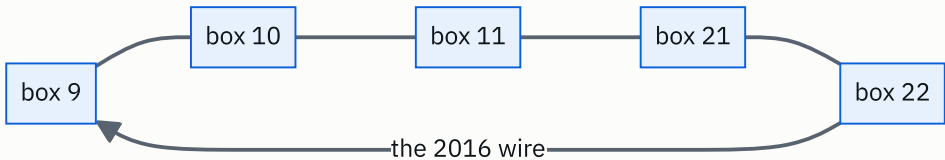


FIGURE 132.1

walk: 9 → 10 → 11 → 21 → 22, then 22 looks at 9

9 is seen. yes

9 is my parent? no – my parent is 21

→ CYCLE

What to notice. The vertex that closes the cycle is not the one you came from, and it is not adjacent in the walk. That is the difference between a genuine back edge and the mirror of the edge you just used.

And the edge-count shortcut, on three small graphs:

TREE	CYCLE	DISCONNECTED
<pre> A B --- C D </pre>	<pre> A --- B +-----+ </pre>	<pre> A --- B D --- E C </pre>
V=4, E=3 $E = V-1$ no cycle	V=3, E=3 $E > V-1$ CYCLE, certain with no traversal	V=5, E=3 $E = V-1$ cannot tell from the counts alone (here: no cycle)

What to notice. The middle case is decided by arithmetic. The right-hand case is not — $E = V - 1$ with a disconnected graph could be a triangle plus two isolated vertices, which has a cycle, or two separate paths, which does not. **The shortcut only decides things when the graph is promised connected, or when E is large enough that no arrangement avoids a cycle.**

5 The code, built step by step

The recursive version first, because it is four lines of logic.

```
def has_cycle(graph: dict[int, list[int]], n: int) → bool:
    seen: set[int] = set()

    def visit(vertex: int, parent: int) → bool:
        seen.add(vertex)
        for neighbour in graph[vertex]:
            if neighbour not in seen:
                if visit(neighbour, vertex):      # pass myself down as the parent
                    return True
            elif neighbour != parent:             # seen, and NOT where I came from
                return True
        return False

    return any(v not in seen and visit(v, -1) for v in range(n))
```

Three things to point at.

`visit(neighbour, vertex)` passes the current vertex down as the child's parent. That is how the child knows which neighbour to ignore.

The `elif` is the whole algorithm. `neighbour in seen` alone is not enough; `neighbour in seen and neighbour  $\neq$  parent` is the condition.

`-1` as the root's parent is a value no real vertex has, so the root ignores nothing. If your vertices are strings, use `None`.

And the `any(...)` at the end is the outer loop from [day 129](#). A cycle can live in a component that has no connection to vertex 0, so starting from one vertex is not enough.

Now the iterative BFS version, which you write when the graph might be deep:

```
from collections import deque

def has_cycle_bfs(graph: dict[int, list[int]], n: int) → bool:
    seen: set[int] = set()
    for start in range(n):
        if start in seen:
            continue
        parent = {start: -1}  # parent map instead of a parameter
        seen.add(start)
        queue = deque([start])
        while queue:
            current = queue.popleft()
            for neighbour in graph[current]:
                if neighbour not in seen:
                    seen.add(neighbour)
                    parent[neighbour] = current
                    queue.append(neighbour)
                elif neighbour  $\neq$  parent[current]:
                    return True
            # continue to the next component
    return False
```

Identical condition, different bookkeeping: the parent lives in a dictionary rather than on the call stack.

There is a subtlety here worth knowing. In BFS on an undirected graph, `neighbour  $\neq$  parent[current]` can fire for two vertices in the same ring that share an edge — and that is a genuine cycle, so the answer is right. But it also means BFS may detect the cycle from a different place than DFS would. Both are correct; the witnesses differ.

Handling parallel edges, when the problem allows them:

```
def has_cycle_edge_ids(adj: dict[int, list[tuple[int, int]]], n: int) → bool:
    """adj[v] holds (neighbour, edge_id). Excludes the EDGE you came in on."""
    seen: set[int] = set()

    def visit(vertex: int, in_edge: int) → bool:
        seen.add(vertex)
        for neighbour, edge_id in adj[vertex]:
            if edge_id == in_edge:
                # the same wire, not a loop
                continue
            if neighbour not in seen:
                if visit(neighbour, edge_id):
                    return True
            else:
                return True
        return False

    return any(v not in seen and visit(v, -1) for v in range(n))
```

The only change is excluding by `edge_id` rather than by vertex. Now two separate edges between `A` and `B` correctly report a cycle, because the second one has a different id.

The Union-Find version, which is the one to offer when edges arrive over time:

```
def has_cycle_union_find(n: int, edges: list[tuple[int, int]]) → bool:
    parent = list(range(n))

    def find(x: int) → int:
        while parent[x] ≠ x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for a, b in edges:
        ra, rb = find(a), find(b)
        if ra == rb:
            return True
        parent[ra] = rb
    return False
```

Read the `if`: an edge whose two endpoints are already in the same group must **close a cycle**, because they were already connected by some other path. That is the entire idea, and it is why Union-Find detects cycles without ever traversing.

And the shortcut, for the case where it applies:

```
def is_tree(n: int, edges: list[tuple[int, int]]) → bool:
    """A tree is connected AND acyclic. E = V-1 plus connected is enough."""
    if len(edges) ≠ n - 1:
        return False
    return count_components(build(n, edges)) == 1
```


With $E = V - 1$ established, connectivity alone decides it: a connected graph with exactly $V - 1$ edges cannot have a cycle, because any cycle would need an extra edge somewhere.

The complete solution

```
"""Cycle detection in an undirected graph: DFS, BFS, Union-Find, and the counting shortcut."""

from __future__ import annotations

from collections import defaultdict, deque

def build(n: int, edges: list[tuple[int, int]]) → dict[int, list[int]]:
    graph: dict[int, list[int]] = {v: [] for v in range(n)}
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)
    return graph

def has_cycle_dfs(graph: dict[int, list[int]], n: int) → bool:
    """Recursive. The parent parameter is the whole algorithm."""
    seen: set[int] = set()

    def visit(vertex: int, parent: int) → bool:
        seen.add(vertex)
        for neighbour in graph[vertex]:
            if neighbour not in seen:
                if visit(neighbour, vertex):
                    return True
            elif neighbour != parent:
                return True
        return False

    return any(v not in seen and visit(v, -1) for v in range(n))

def has_cycle_bfs(graph: dict[int, list[int]], n: int) → bool:
    """Iterative. Same condition; the parent lives in a dict. No recursion limit."""
    seen: set[int] = set()
    for start in range(n):
        if start in seen:
            continue
        parent = {start: -1}
        seen.add(start)
        queue = deque([start])
        while queue:
            current = queue.popleft()
            for neighbour in graph[current]:
                if neighbour not in seen:
                    seen.add(neighbour)
                    parent[neighbour] = current
                    queue.append(neighbour)
                elif neighbour != parent[current]:
                    return True
    return False

def has_cycle_union_find(n: int, edges: list[tuple[int, int]]) → bool:
    """An edge between two vertices already in one group closes a cycle."""
    parent = list(range(n))

    def find(x: int) → int:
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for a, b in edges:
        ra, rb = find(a), find(b)
        if ra == rb:
            return True
        parent[ra] = rb
    return False
```

```
def quick_verdict(n: int, edges: list[tuple[int, int]], connected: bool) → str:
    """Arithmetic only. Decides some cases in O(1)."""
    e = len(edges)
    if e > n - 1:
        return "cycle, certain"
    if connected and e == n - 1:
        return "no cycle, certain (it is a tree)"
    if e < n - 1:
        return "cannot be connected; cycle unknown"
    return "unknown without a traversal"

if __name__ == "__main__":
    tree = [(0, 1), (0, 2), (1, 3)]
    ring = [(0, 1), (1, 2), (2, 0)]
    two_pieces = [(0, 1), (1, 2), (2, 0), (3, 4)]

    for name, n, edges in (("tree", 4, tree), ("ring", 3, ring), ("two pieces", 5, two_pieces)):
        graph = build(n, edges)
        print(f"{name:12} dfs={has_cycle_dfs(graph, n)!s:5} "
              f"bfs={has_cycle_bfs(graph, n)!s:5} "
              f"uf={has_cycle_union_find(n, edges)!s:5} "
              f"| {quick_verdict(n, edges, connected=False)}")

    # The two-vertex case that breaks the naive rule.
    pair = build(2, [(0, 1)])
    print("single edge :", has_cycle_dfs(pair, 2), "(must be False)")
```

Running it:

```
tree          dfs=False bfs=False uf=False | unknown without a traversal
ring          dfs=True  bfs=True  uf=True  | cycle, certain
two pieces    dfs=True  bfs=True  uf=True  | unknown without a traversal
single edge : False (must be False)
```

Two things to look at. The `tree` row shows the shortcut being honest: with $E = 3$ and $V = 4$ it is $E = V - 1$, and because we did not promise connectivity, arithmetic alone cannot decide — the traversal has to. The `two pieces` row is the same arithmetic reaching the same non-answer on a graph that *does* have a cycle, which is exactly why the shortcut is a shortcut and not a solution.

And the last line is the test that matters most. Two vertices, one edge, and the answer must be `False`. If your implementation says `True` there, the parent check is missing and nothing else you test will tell you.

6 What it costs

DFS and BFS.

each vertex visited once	V
each vertex's neighbours scanned once	$2E$ (each edge twice, once per end)

seen set + parent map	$O(V + E)$ time
recursion stack or queue	$O(V)$

	$O(V)$ space

Both traversals are the same. There is no version of this problem where one is faster.

Early exit is worth a lot in practice. The moment a cycle is found you return, and a graph with a cycle near the start is decided in a handful of steps:

$V = 1,000,000$, $E = 3,000,000$, a triangle among the first few vertices
 worst case $4,000,000$ steps
 actual ~ 10 steps

The worst case is a graph with no cycle at all, where you must visit everything to be sure.

Union-Find.

per edge: two finds and possibly one union
 with path compression and union by rank/size: $O(\alpha(V))$ each
 alpha is inverse Ackermann: ≤ 4 for any n that fits in the universe

 $O(E \times \alpha(V)) \sim O(E)$
 parent array $O(V)$

Effectively linear too, so for a one-shot answer on a static graph the two are equivalent and the traversal is fewer lines. Where they diverge:

edges arriving one at a time, answer after each

traversal	$m \times O(V + E)$	quadratic
Union-Find	$m \times O(\alpha)$	linear

$m = 100,000$ edges, $V = 100,000$
 traversal: $100,000 \times 300,000 = 30,000,000,000$ steps
 Union-Find: $100,000 \times 4 = 400,000$ steps

Seventy-five thousand times faster, and that gap is the whole reason to know both.

The counting shortcut.

$E > V - 1$	$O(1)$	→ cycle, certain
connected and $E = V - 1$	$O(1)$	→ no cycle, certain
everything else		→ you still need the traversal

On a promised-connected graph this is the fastest possible answer and it is worth saying before you write anything:

```
V = 10,000,000, E = 10,000,001, connected
answer: cycle. Zero traversal, one comparison.
```

A cheap early check that helps in practice, even when it cannot decide:

```
if len(edges) > n - 1: return True    # one comparison, catches many inputs
```

Space, compared:

DFS recursive	$O(V)$ stack – and RecursionError past ~1,000 depth
BFS iterative	$O(V)$ queue – no limit
Union-Find	$O(V)$ array – the smallest constant of the three

For $V = 10^6$, Union-Find is one list of a million integers, about 8 MB. The traversal needs a `seen` set and a parent map — two hash structures, closer to 100 MB in Python. **When memory is tight, Union-Find wins on constants even though all three are $O(V)$.**

7 The traps

The missing parent check

The near-miss, and it fails on the smallest possible graph:

```
def visit(vertex):
    seen.add(vertex)
    for neighbour in graph[vertex]:
        if neighbour in seen:
            return True                # "cycle"
        if visit(neighbour):
            return True
    return False
```

```
>>> graph = build(2, [(0, 1)])
>>> has_cycle_broken(graph, 2)
True
```

Two vertices, one edge, and it reports a cycle. The walk goes `0 → 1`, then `1` looks at its neighbours and finds `0`, which is in `seen` — because it is the vertex we came from, along the one edge that exists.

Every undirected graph with at least one edge triggers this. Test with a single edge before anything else; it is the fastest possible way to find out whether the check is there.

Excluding the parent by vertex when parallel edges exist

```
elif neighbour != parent:
    return True
```

With edges `[(0, 1), (0, 1)]` — two separate wires between the same pair:

```
>>> has_cycle_dfs(build_multi(2, [(0,1), (0,1)]), 2)
False
```

That is wrong: two distinct edges between the same pair form a cycle of length two. The parent check suppresses both mirror views and one genuine edge along with them.

The fix is to exclude by edge id rather than by vertex. Most problems promise simple graphs, so **ask whether parallel edges are possible** rather than assuming either way.

Self-loops

```
edges = [(0, 0)]
```

```
>>> has_cycle_dfs(build(1, [(0, 0)]), 1)
False
```

`0` looks at its neighbour `0`, which is in `seen` — but `0` is also its own parent value... actually its parent is `-1`, so `neighbour != parent` is true and it *should* return `True`. Whether it does depends on whether your builder added the self-loop once or twice. **That ambiguity is the point:** self-loops are an edge case that different implementations handle differently by accident. Either handle them explicitly with `if neighbour == vertex: return True`, or confirm the problem excludes them.

Only searching from one vertex

```
return visit(0, -1)
```

A graph whose only cycle is among vertices 7, 8 and 9, with nothing connecting them to vertex 0:

```
>>> has_cycle_from_zero(graph, 10)
False # there is a triangle at 7-8-9
```

The outer loop over every vertex, with `seen` shared, is required. The sample input for these problems is almost always connected, so nothing warns you.

Using the directed algorithm

```
colour = [WHITE] * n
# ... grey/black three-colour scheme ...
```

On an undirected graph, the three-colour scheme reports a cycle immediately, for the same reason as the naive version: walking `A → B` makes `A` grey, and `B` sees `A` as grey. **The colours do not save you** — they solve a different problem, which is reaching an *ancestor* in a directed graph.

Conversely, using the parent check on a **directed** graph misses cycles, because there is no mirror edge and the check simply never fires usefully.

The two algorithms are not interchangeable and the word “cycle” is the same in both. Confirm which kind of graph you have before choosing.

Recursion depth

```
Traceback (most recent call last):
  File "cycles.py", line 9, in visit
    if visit(neighbour, vertex):
  [Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

A graph that is a long path — a chain of 100,000 vertices — needs 100,000 frames. `n ≤ 105` in the constraints means write the BFS version or the Union-Find version.

8 In the interview

How it gets asked

- “Does this undirected graph contain a cycle?” — the direct version.
- “Given `n` nodes and a list of edges, is this a valid tree?” — LeetCode 261, and it is this plus connectivity.
- “Can these edges be added without creating a loop?” — Union-Find framing.
- “Find the edge that, when removed, makes this a tree.” — LeetCode 684, redundant connection.

- “How many extra edges do you need to connect all the components?” — components plus the edge count.

The first ninety seconds

“A traversal with one extra condition, and the condition is the interesting part.

The naive version says ‘if I reach a vertex I have already seen, that is a cycle’. In an undirected graph that is wrong on the smallest possible input, and I would demonstrate why: an undirected edge `A-B` is stored in both adjacency lists, so as soon as I walk from `A` to `B` and look at `B`’s neighbours, I see `A`, which is in `seen`. That is not a loop — it is the same edge, seen from the other end. Two vertices, one edge, and the naive check reports a cycle.

So I pass down the vertex I came from, and the condition becomes: **seen, and not my parent**. That is the whole algorithm.

I would write it either as DFS with the parent as a parameter, or as BFS with a parent dictionary — same condition, and I would choose BFS if `n` can be large, because a graph that is a long chain overflows the recursion stack at about a thousand.

Two other things I would set up before coding. The loop is over **every** vertex with a shared `seen` set, because a cycle can sit in a component that nothing connects to where I started. And I would ask whether parallel edges or self-loops are possible, because excluding ‘the parent vertex’ also wrongly excludes a genuine second edge between the same pair — if those are allowed, I exclude by edge id instead.

`O(V + E)` time and `O(V)` space.

And before any of that: if the graph is promised connected, `E > V - 1` answers the question in one comparison, because a connected acyclic graph is a tree and has exactly `V - 1` edges. Is it connected?”

The follow-ups

“Why does the directed version need something different?”

“Because the thing that makes undirected tricky does not exist there, and a different thing does.

In an undirected graph, every edge appears in both adjacency lists, so every edge produces one false alarm — and that is what the parent check suppresses.

In a directed graph, $A \rightarrow B$ puts B in A ’s list and nothing in B ’s. There is no mirror edge, so there is nothing for a parent check to do. What matters instead is whether the vertex I have reached is an **ancestor** — still on the path from the root to where I am now — because an edge back to an ancestor closes a loop, while an edge into a region I have already finished exploring does not.

So the directed version needs three states: white for untouched, grey for ‘started and not finished, so on my current path’, black for finished. An edge to grey is a cycle; an edge to black is a shared dependency and is fine. Two states cannot tell those apart, and the classic failure is a diamond — $0 \rightarrow 1 \rightarrow 3$ and $0 \rightarrow 2 \rightarrow 3$ — where a two-state check reports a cycle on a graph that has none.

Same word, two genuinely different algorithms, and the first thing I would establish about any cycle question is which kind of graph it is.”

“Solve it with Union-Find instead.”

“Process the edges one at a time. For each edge, find the representative of both endpoints. If they are the same, the two vertices were already connected by some other path, so this edge closes a cycle. If they differ, merge the two groups and carry on.

That is about eight lines and it never traverses anything.

With path compression and union by size, each operation is effectively constant — inverse Ackermann, at most four for any realistic input — so the whole thing is $O(E)$ with a very small constant, and the memory is one array of V integers rather than a `seen` set and a parent map.

For a one-shot answer on a static graph, the traversal is fewer lines and I would probably write that. **The case where Union-Find is not just an alternative but the only option is edges arriving over time** — ‘after each new connection, is there a loop yet?’. Re-running a traversal per edge is $m \times O(V + E)$, which at a hundred thousand edges and vertices is thirty billion steps; Union-Find is four hundred thousand.

It is also the natural fit for the redundant-connection problem, where you want the *first* edge that creates a cycle — Union-Find gives you that directly, and a traversal would need extra work to identify which edge was responsible.”

“Is this a valid tree?”

“A tree is connected and acyclic, and the cheapest way to check both is to use the arithmetic first.

If $E \neq V - 1$, it is not a tree, immediately — too many edges guarantees a cycle, too few guarantees it is disconnected. That is one comparison and it rejects most invalid inputs.

Once $E = V - 1$ holds, I only need to check **one** of the two properties, because with exactly $V - 1$ edges, connected implies acyclic and acyclic implies connected. So a single traversal from vertex 0 that visits all V vertices is sufficient — if it does, it is a tree; if it does not, it is not.

That is a nice simplification and I would say why it works: a connected graph on V vertices needs at least $V - 1$ edges, and any cycle means some vertex is reachable two ways, which means the remaining edges cannot reach everything. So with exactly $V - 1$, the two failure modes cannot occur separately.

Union-Find gives the same answer in one pass: run through the edges, return false the moment an edge joins two vertices already in the same group, and at the end check that there is exactly one group left.”

“Which edge would you remove to break the cycle?”

“Depends on which edge they want, and I would ask.

With Union-Find and edges processed in the given order, the **first** edge that joins two already-connected vertices is the one that created the cycle in that ordering. That is what LeetCode’s redundant-connection problem wants — the last edge in the input that can be removed — and Union-Find gives it with no extra work.

If they want a specific edge — the heaviest, or one whose removal disconnects nothing else — that is different. Any edge *on* the cycle can be removed without disconnecting the graph, and finding the cycle’s edges means recording the DFS path: when the back edge is found, the cycle is the path from the ancestor down to the current vertex, plus the back edge itself. So I keep the current path as a list, and when `neighbour` in seen and `neighbour` $\neq$ `parent`, I slice the path from `neighbour` onwards.

And the related question that uses the same machinery is **bridges** — edges whose removal *does* disconnect the graph. An edge is a bridge exactly when it is on no cycle, and there is a linear DFS that finds all of them at once by tracking the earliest ancestor reachable from each subtree. I would name that rather than write it unprompted.”

The model answer

“You are given n computers numbered 0 to $n-1$ and a list of cables, each connecting two computers. Determine whether the network contains a redundant cable — one that could be removed without disconnecting anything.”

“A redundant cable is exactly an edge that lies on a cycle, so this is undirected cycle detection with a small extra step to identify the edge. Let me build it up.

First, the arithmetic, because it costs one comparison. A connected network with no redundancy is a tree and has exactly $n - 1$ cables. If there are more, redundancy is certain before I look at anything. If there are fewer, the network is definitely already disconnected, which is a different problem the operator probably cares about more. I would say both of those out loud because they are free.

Then the detection. Union-Find, and I would choose it over a traversal here specifically because the question asks *which* cable rather than *whether*. Processing the cables in order, the first one whose two endpoints are already in the same group is the cable that closed a loop — everything before it was tree-building. A traversal would tell me a cycle exists and then need more work to name the responsible edge.

The implementation is the standard one: a parent array, `find` with path compression, and union by size so the trees stay shallow. Each cable is two finds and at most one union, effectively constant, so the whole pass is $O(E)$ with an $O(V)$ array — for a million computers that is one list of a million integers, about 8 megabytes, against a `seen` set and parent map for a traversal, which in Python would be an order of magnitude more.

What I would be careful about. The network is almost certainly not connected — that is often the actual problem being investigated — so I must not stop at the first component. Union-Find handles that naturally, since it never assumes connectivity. And I would ask about two input possibilities: can the same pair be cabled twice, and can a cable loop back to the same computer? Both are physically plausible here, unlike in an abstract problem. A duplicate cable between the same pair is redundancy and Union-Find catches it correctly, because the second one finds both endpoints already merged — **which is a case the DFS parent check would silently miss**, and that is a real argument for Union-Find on this particular problem.

What I would return. Not just a boolean. The list of redundant cables, and the component count at the end — because ‘there is one redundant cable and the network is in three pieces’ is a completely different operational situation from ‘there is one redundant cable and everything is connected’, and the second number costs nothing once the structure exists.

The follow-up I would expect is ‘now cables are added one at a time and I want to know each time’ — and the answer is that Union-Find already does that. It is an online algorithm; the traversal is not. Re-running a traversal after each of a hundred thousand cables would be thirty billion steps against four hundred thousand.”

9 Recall card

A cycle returns to a vertex without reusing an edge. In an undirected graph every edge is stored twice, so “already seen” fires once per edge and is not a cycle — it is the edge you walked in on, viewed from the other end.

The condition is `seen AND not the parent`. Pass the current vertex down as the child’s parent (DFS) or keep a parent map (BFS). Test with two vertices and one edge: the answer must be `False`.

Directed is a different algorithm. No mirror edges there, so the parent check does nothing; you need three colours, where **grey means on the current path**. Establish which kind of graph you have first.

Union-Find detects a cycle without traversing: an edge whose endpoints are already in the same group closes one. Effectively $O(E)$, smaller constant memory, works **online** as edges arrive — and it catches parallel edges that the parent check misses.

The free check: a connected acyclic graph has exactly $V - 1$ edges. $E > V - 1$ means a cycle, with no traversal. And once $E = V - 1$, checking connectivity alone decides “is this a tree”.

1 What this is, and why they ask it

Stream processing is computing over data that never stops arriving. Not “run a job at midnight over yesterday’s data” — a program that is always running, updating an answer as events come in.

The idea is easy. The difficulty is entirely in one thing: **events do not arrive in the order they happened**. A phone that was in a lift sends its events four minutes late. A server in another region is behind. A retry re-delivers something from thirty seconds ago. So when you are asked “how many clicks happened between 10:00 and 10:01”, you have to decide when to stop waiting for stragglers and commit to an answer — and whatever you choose, some events will arrive after you have answered.

They ask this because it is where “just count the events” stops being easy, and because the vocabulary — **windows, event time, watermarks, late events** — is used loosely by people who have not thought about it. An interviewer asking “how do you count events per minute when events arrive late?” is checking whether you know that the question has no clean answer, only a set of explicit trade-offs.

It sits directly on [day 130](#)’s Kafka and feeds [day 168](#)’s ad-click aggregator, which is the full case study.

By the end of this lesson you can explain windowing without jargon, state the difference between event time and processing time and why it matters, describe what a watermark actually is, choose a lateness policy with a number attached, and say what you do about the event that arrives after you have already published the answer.

2 The story

Shalini closes the shop at ten, and closing the shop means counting the day.

The counting itself takes about six minutes. Cash from the drawer, the card machine total, and the amounts the two delivery boys bring back, and then a figure for the day that goes into her phone and that she has kept every day for eleven years.

The delivery boys are the problem, and they always have been.

Ravi is usually back by five past ten. Sohail is not. Sohail’s last drop is often the flats behind the petrol pump, and there is a gate there that shuts at ten, so some nights he has to go round, and he gets back at twenty past.

For a long time Shalini simply waited. She sat there until both of them were in, and some nights that was half past ten, and the whole family ate late.

What she does now is she waits until quarter past. That is the rule. At quarter past ten she counts what she has, writes the figure down, and that is the day. If Sohail walks in at twenty past with three hundred rupees from an order taken at half past nine, that money is a real part of Tuesday's business and it goes into Wednesday's count.

She picked quarter past because she watched for a couple of weeks. Almost every night both boys are back before then. About once a fortnight one of them is not.

Two things follow from that rule and she has thought about both.

The first is that her daily figures are slightly wrong, always, in a way that mostly cancels out. Tuesday is three hundred short and Wednesday is three hundred over. Over a month it does not matter. For the monthly total it does not matter at all, because the money is in there somewhere.

The second is that some nights it does matter, and then she breaks her own rule. On the last day of the month she waits for Sohail however late he is, because a month boundary is a real boundary and money on the wrong side of it makes the accounts harder to explain than a late dinner.

And there is a third thing, which happened once, in 2022. Sohail's cycle chain broke near the highway and he did not get back until quarter to twelve, with eleven hundred rupees. Shalini had already written the figure down and sent it to her husband. She went back into her phone, corrected Tuesday, and sent a message saying the earlier number was wrong.

She does not like doing that. But eleven hundred was too big to just push into Wednesday and forget.

3 The idea in plain English

Shalini has built a stream processor, including the parts that people skip.

A stream is data that keeps arriving. No end, no "all the data". You cannot wait for it to finish, because it does not finish. That single fact rules out every technique that assumes you can see everything before you answer.

A window is a bucket of time you compute over. Shalini's day. In code: "clicks per minute", "revenue per hour", "errors in the last five minutes". Windows come in three shapes:

- **Tumbling** — fixed, non-overlapping. 10:00–10:01, 10:01–10:02. Every event is in exactly one. This is Shalini’s day and it is what you want most of the time.
- **Sliding** — fixed length, overlapping, advancing by a smaller step. “The last five minutes, recomputed every minute.” An event belongs to five windows at once, so the work is five times as much.
- **Session** — no fixed length; a window ends after a gap of inactivity. “A user’s browsing session ends after 30 minutes of nothing.” Used for anything about a person’s activity, where the natural boundary is behaviour rather than the clock.

Now the distinction the whole subject rests on: event time against processing time.

Event time is when the thing actually happened — the order was taken at half past nine. **Processing time** is when your system found out about it — Sohail walked in at twenty past ten. **These are different, and the gap is unpredictable.**

Almost always you want event time. “How many orders were placed between nine and ten” is a question about when orders were placed, not about when the message reached your server. If you count by processing time, your numbers change depending on how fast your network was, which makes them useless for comparison and impossible to reproduce.

And event time is what makes it hard, because if you are bucketing by when things happened, you can never be certain a bucket is complete. There might always be one more event on its way from a phone that was in a lift.

A watermark is a promise about how far behind you might be. A watermark of “10:00” means: *I believe I have now seen essentially all events that happened before 10:00.* It is Shalini’s quarter past ten — the moment she declares the day’s takings complete.

A watermark is a guess, and it is chosen, not derived. The usual method is exactly hers: watch the actual delay for a while, and set the watermark to lag by more than almost all of it. If 99% of events arrive within 90 seconds, a watermark lagging two minutes is right. **The number comes from measurement, and if you cannot say where yours came from, you have not chosen it.**

A late event is one that arrives after its window’s watermark has passed. Sohail at twenty past. You have three choices and they are all legitimate:

1. **Drop it.** Simplest. The window is closed and the event is discarded, usually with a counter so you know how many you are losing. Right for approximate dashboards.
2. **Put it in the current window.** Shalini’s default: Tuesday’s money goes into Wednesday. The total over a long period is right; the per-window numbers are individually wrong. Right when only the aggregate matters.

3. Reopen the window and correct the answer. The 2022 cycle chain. The most correct and the most expensive: you must keep the window's state around for a defined **allowed lateness** period, and every downstream consumer must be able to handle a corrected number replacing one it already received.

Allowed lateness is a second number, and it is not the watermark. The watermark says when you emit an answer. Allowed lateness says how much longer you keep the state so you can revise it. Emit at two minutes, keep state for an hour, drop anything later than that.

And “keep the state” is the real cost. A stream processor computing per-minute counts for ten million users must hold one counter per user per open window. **Every window you keep open is memory**, and allowed lateness multiplies it directly.

The month-end rule is the last idea: some boundaries cannot be fudged. Shalini waits for Sohail on the last day of the month. In a system, a billing period or a regulatory report is a boundary where “it will come out in the wash” is not acceptable, and you either wait, or you publish a provisional figure and a final one. **Knowing which of your outputs are provisional is part of the design.**

4 The picture

The three window shapes on the same stream:

```
events:      a      b      c      d      e      f      g
time:      |-----+-----+-----+-----+-----+-----+-----|
           10:00   10:01   10:02   10:03   10:04   10:05

TUMBLING (1 minute)
  [ 10:00-10:01 ] [ 10:01-10:02 ] [ 10:02-10:03 ] [ ... ]
      a, b             c, d             e
every event in exactly ONE window

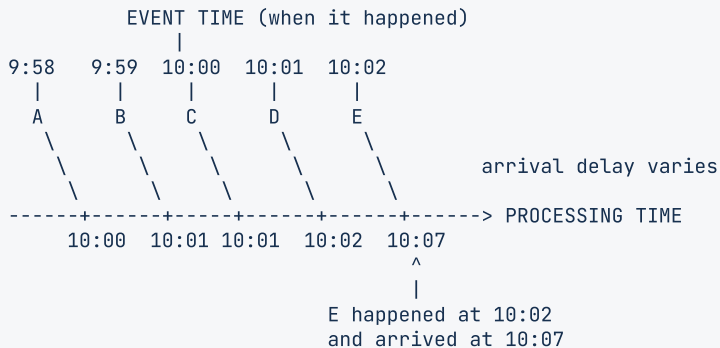
SLIDING (2 minutes, every 1 minute)
  [ 10:00-10:02 ]
      [ 10:01-10:03 ]
          [ 10:02-10:04 ]
every event in TWO windows → 2x the state and 2x the work

SESSION (gap of 90 seconds)
  [ a b ]      [ c d ]      [ f g ]
  closed by inactivity, not by the clock
```

What to notice. Sliding windows multiply everything by the overlap factor. A five-minute window sliding every ten seconds means each event is in thirty windows, which is thirty times the state and thirty times the update work — a completely different sizing

conversation from the tumbling version.

Event time against processing time, which is the picture worth memorising:



```
window 10:00-10:01 by EVENT time    → {C}
window 10:00-10:01 by PROCESSING time → {A, B, C} ← wrong bucket entirely

watermark at processing 10:03 says "I have everything before event time 10:01"
E arrives at 10:07 → LATE by 4 minutes
```

What to notice. The two groupings give different answers for the same minute, and only one of them is reproducible. Re-run the same stream tomorrow on a faster network and the processing-time answer changes; the event-time answer does not.

And the watermark advancing, which is the mechanism:

```
watermark = (largest event time seen so far) - allowed_delay

seen event times: 9:58, 9:59, 10:00, 10:01
max = 10:01, allowed_delay = 2 min
watermark = 9:59      → windows ending at or before 9:59 may be closed

a new event arrives with event time 10:05
max = 10:05
watermark = 10:03     → windows up to 10:03 close, and emit

NOTE: the watermark only moves when NEW events arrive.
A quiet stream never closes its windows. That is a real production incident.
```

What to notice at the last line. If a partition stops receiving events — a device goes off-line, a region goes quiet at night — its watermark stops advancing, and because the overall watermark is the *minimum* across partitions, **one idle partition freezes the whole job's output**. Systems handle this with an idle-source timeout, and knowing about it is a strong signal.

5 How it actually works

The systems

SYSTEM	MODEL	WINDOWS	EVENT TIME	STATE
Flink	true streaming, event at a time	all three	first-class, with watermarks	RocksDB-backed, checkpointed
Kafka Streams	library, runs in your app	tumbling, hopping, session	yes, with grace periods	local RocksDB + changelog topic
Spark Structured Streaming	micro-batch (and continuous)	tumbling, sliding	yes, with watermarks	checkpointed
Kinesis Data Analytics	managed Flink	as Flink	as Flink	managed
ksqlDB	SQL over Kafka Streams	tumbling, hopping, session	yes	as Kafka Streams

The distinction worth knowing: true streaming versus micro-batch. Flink processes each event as it arrives — latency in milliseconds. Spark Structured Streaming groups events into small batches, typically hundreds of milliseconds to seconds — higher latency, and simpler recovery because a batch is a unit. **For most business use cases the difference does not matter**, and Spark's advantage is that the same code runs over historical data.

What the code actually looks like

Kafka Streams, counting clicks per minute per ad:

```
builder.stream("clicks")
    .groupByKey()
    .windowedBy(TimeWindows.ofSizeAndGrace(
        Duration.ofMinutes(1),           // window size
        Duration.ofMinutes(10)))         // allowed lateness ("grace")
    .count()
    .toStream()
    .to("clicks-per-minute");
```

Two numbers, and they are the two decisions from section 3: the window size and how long you keep the state open for corrections.

Flink, the same thing, with the watermark declared explicitly:

```
stream.assignTimestampsAndWatermarks(
    WatermarkStrategy
        .<Click>forBoundedOutOfOrderness(Duration.ofSeconds(90))
        .withTimestampAssigner((e, t) → e.eventTime))
    .keyBy(click → click.adId)
    .window(TumblingEventTimeWindows.of(Time.minutes(1)))
    .allowedLateness(Time.minutes(10))
    .sideOutputLateData(lateTag)           // late events go somewhere, not nowhere
    .aggregate(new CountAggregate());
```

`forBoundedOutOfOrderness(90 seconds)` is the watermark, stated as a number in the code — “I assume events are at most 90 seconds out of order”. `allowedLateness(10 minutes)` is how long state is kept for revisions. And `sideOutputLateData` is the line most people leave out: **events later than the allowance go to a separate stream rather than being silently dropped**, so you can count them and know whether your 90 seconds was right.

State, and what makes it survivable

A stream processor holds state — the running count for every open window for every key. Two things make that production-grade:

Local state with a durable changelog. Kafka Streams keeps state in RocksDB on the local disk and writes every change to a compacted Kafka topic. If the instance dies, a new one replays the changelog and rebuilds. Fast reads, no remote lookup per event, and recovery is bounded by the changelog size.

Checkpointing. Flink periodically snapshots all operator state and the source offsets together, atomically. On failure it restores the snapshot and rewinds the sources to the matching offsets. **That pairing — state and offsets in one snapshot — is what gives exactly-once *effect* within the job**, in exactly the sense from [day 122](#). The moment the job writes to an external system, the guarantee needs an idempotent sink or a transactional one.

Triggers: when to emit

The default is “emit once, when the watermark passes the window’s end”. Two other patterns are common:

- **Early firing** — emit a partial result every ten seconds while the window is still open, so a dashboard updates continuously. The consumer must understand that the number will change.
- **Late firing** — emit an updated result when a late event arrives within the allowed lateness. The consumer must be able to *replace* the previous value rather than add to it.

Both mean downstream receives more than one answer for the same window, and the design has to say what downstream does with that. The usual answer is that the sink is keyed by window so a later write overwrites the earlier — an upsert, not an append. If the sink appends, early and late firing double-count.

The idle-partition problem

The overall watermark is the **minimum** across all input partitions, because the job cannot claim to have seen everything before time T until every partition has. So a single partition with no traffic holds the watermark back, and no windows close anywhere.

This happens constantly in real deployments: a device fleet where some devices are asleep, a region that goes quiet overnight, a topic partition that gets no traffic because of a skewed key. The fix is `withIdleness(Duration)` in Flink or its equivalent, which excludes a partition from the watermark calculation after a period of silence. **“The dashboard stopped updating at 2 a.m. and there was no error” is the symptom**, and it is a good thing to have a story about.

Joining two streams

Joining a stream to a table (enriching clicks with ad metadata) is straightforward: keep the table in local state and look up per event.

Joining a stream to a stream — impressions to clicks, orders to payments — needs a **window**, because you cannot hold every impression forever waiting for a click. “Join a click to an impression within thirty minutes” means keeping thirty minutes of impressions in state:

```
1,000,000 impressions/minute x 30 minutes = 30,000,000 rows in state
x 200 bytes                               = 6 GB per instance-set
```

The join window is a memory decision disguised as a business rule, and it is worth stating the number.

6 The numbers

Window count and state size. Per-minute counts for ten million users:

keys	10,000,000
open windows per key	1 (tumbling, no lateness)
state per entry	~100 bytes (key + count + timestamps)

	10,000,000 x 100 = 1 GB

Now add allowed lateness of ten minutes:

```
open windows per key      10 (one per minute still open)
                           10,000,000 x 10 x 100 = 10 GB
```

Allowed lateness multiplies state linearly, and it is the number people forget. Ten minutes of grace on a one-minute window is ten times the memory.

And a sliding window makes it worse:

```
5-minute window, sliding every 10 seconds
windows an event belongs to = 300 / 10 = 30
state                        10,000,000 x 30 x 100 = 30 GB
update work per event       30 counter updates instead of 1
```

Thirty times the memory and thirty times the CPU, from one configuration line. If someone asks for “the rolling five-minute count updated every ten seconds”, that is what they have asked for, and it is worth pricing before agreeing.

Choosing the watermark from measured delay. Measure `processing_time - event_time` over a day:

```
p50    1.2 s
p95     8 s
p99    45 s
p99.9  4 min
max     2 h      (a phone that was offline)
```

```
watermark lag = 60 s   → drops ~1% of events
watermark lag = 5 min  → drops ~0.1%, adds 5 min of latency to every window
watermark lag = 2 h    → drops ~0, and results are 2 hours stale
```

That table is the whole decision, and the answer depends entirely on what the output is for. A live dashboard takes 60 seconds and 1% loss. A billing figure takes the two hours, or is published provisionally and finalised.

Throughput.

```
Flink, simple aggregation    ~ 1,000,000 events/s per core-ish (heavily workload-dependent)
Kafka Streams                ~ 100,000 - 500,000 events/s per instance
Spark micro-batch             higher throughput, latency 100 ms - seconds
```

Latency, end to end:

event happens	t = 0	
reaches Kafka	+ 50 ms	- 2 s
consumed by the job	+ 10 ms	
window closes (watermark lag)	+ 60 s	← this dominates
result written	+ 20 ms	

	~ 61 s	

The watermark lag is almost always the dominant term, which is worth saying explicitly: tuning the processor's internals is pointless when 98% of the latency is a number you chose.

Late-event volume, and why the side output matters:

```

1,000,000 events/minute
0.1% arrive after a 10-minute allowance
    = 1,000 events/minute discarded
    = 1,440,000 per day

```

One and a half million discarded events a day. If they go to a side output you can count them, see which sources produce them, and decide. If they are dropped silently you find out when someone reconciles against a batch job and the numbers differ by 0.1%.

And the reconciliation number that makes the design defensible:

streaming count for yesterday	9,998,600
batch recount over the raw log	10,000,000
difference	0.014%

Running a nightly batch recount over the same raw data and comparing is the standard practice, and it is what turns “our streaming numbers are approximately right” into a number you can put in a document.

7 The trade-offs

Latency against completeness, and this is the whole subject. A short watermark lag gives fast answers and drops more events. A long one gives complete answers late. There is no setting that gives both, and the right answer depends entirely on what the number is used for — a dashboard and an invoice sit at opposite ends. **If someone asks for “real-time and exactly right”, they are asking for two things, and the design conversation is which one to relax.**

Event time against processing time. Event time gives correct, reproducible buckets and requires watermarks, state retention and a lateness policy. Processing time is trivial, has no late events by definition, and produces numbers that change if your network is slow. **Use processing time only when the question really is about your system** — “how many requests did we handle this minute” — and never for anything about the outside world.

Allowed lateness against memory. Every minute of grace multiplies open windows. Ten minutes on a one-minute window over ten million keys is ten gigabytes rather than one. And it is not just memory: it is checkpoint size, recovery time, and how much state has to be shipped when the job rescales.

Correcting results against a simple downstream contract. Emitting a revised number for a closed window is the most correct option and it forces every consumer to handle updates — the sink must upsert, the dashboard must replace rather than accumulate, and anyone who copied the earlier number has a stale figure. **Many teams choose to be slightly wrong rather than make every consumer handle revisions**, and that is a legitimate engineering decision as long as it is decided rather than defaulted.

Streaming against batch. A streaming job is always running, always costing money, and fails at three in the morning. A batch job runs once, is trivially re-runnable, and gives an answer hours later. **If the answer is only looked at once a day, a batch job is a better engineering decision than a streaming one**, and saying that is worth more than knowing Flink’s API.

And the honest position on exactly-once. Flink’s checkpoints give exactly-once *within the job* by snapshotting state and source offsets together. As soon as the job writes to an external system, you need an idempotent or transactional sink, and you are back to [day 122](#)’s idempotency keys. Claiming exactly-once without naming the sink is the tell that someone has read a marketing page.

When would I not use stream processing? When the answer is consumed daily — batch is simpler and cheaper. When the aggregation is small enough to do with a counter in Redis and a cron job, which covers a surprising number of real “real-time metrics” requirements. And when the team is small, because a Flink cluster is a serious operational commitment and a mis-tuned watermark produces wrong numbers silently, which is worse than a batch job that fails loudly.

8 In the interview

How it gets asked

- “How do you count events per minute when events arrive late?” — the direct version.

- *“What is the difference between event time and processing time?”*
- *“What is a watermark?”*
- *“An event arrives an hour late. What happens to it?”*
- *“Your dashboard stopped updating overnight and there were no errors. Why?”* — the idle-partition question.
- *“Design a real-time analytics pipeline.”* — where all of this shows up as follow-ups.

The first ninety seconds

“The counting is easy; the ordering is the problem. Events do not arrive in the order they happened — a phone that was in a lift sends four minutes late, a retry replays something from thirty seconds ago — so the real question is when I stop waiting for a minute’s events and commit to a number.

First decision: I bucket by event time, not processing time. ‘How many clicks happened between 10:00 and 10:01’ is a question about when the clicks happened. If I bucket by arrival, the same stream replayed on a faster network gives different numbers, which makes them useless for comparison.

Second: a watermark. A watermark of 10:01 is a claim that I have now seen essentially everything that happened before 10:01, so windows ending at or before then can be closed and emitted. It is a guess and I choose it from measurement — I would look at the actual distribution of `arrival minus event time` and set the lag above the p99. If p99 is forty-five seconds, a ninety-second watermark drops about one percent of events and adds ninety seconds of latency to every result.

Third: a policy for what is later than that. Three options and I would say which and why. Drop it, with a counter, if the output is a dashboard. Fold it into the current window, so the total over an hour is right and individual minutes are slightly wrong. Or keep the window’s state open for a defined allowed lateness — say ten minutes — and emit a corrected value. The third is the most correct and it forces every downstream consumer to handle a number being revised, so the sink has to upsert by window rather than append.

And whichever I choose, late events go to a side output rather than nowhere, so I can count them and find out whether my ninety seconds was right.

The cost I would flag immediately is state: ten million keys with a one-minute window is about a gigabyte, and ten minutes of allowed lateness makes it ten. Allowed lateness multiplies memory linearly.

What is this number used for — a dashboard, or billing? Because that changes the watermark and the lateness policy completely.”

The follow-ups

“What actually is a watermark? Be precise.”

“It is a marker flowing through the stream that says ‘no more events with an event time earlier than T are expected’. When it passes the end of a window, that window is complete and can be emitted.

Mechanically, the common strategy is: watermark equals the largest event time seen so far, minus a fixed allowance for out-of-orderness. So if I have seen events up to 10:05 and my allowance is two minutes, the watermark is 10:03, and every window ending at or before 10:03 fires.

Two consequences that matter and that people miss.

It is a heuristic, not a fact. Nothing prevents an event with time 10:01 from arriving after the watermark passed 10:03. That event is late by definition, and what happens to it is a separate policy.

And it only advances when new events arrive, because it is derived from the maximum event time seen. That is the source of a very specific production incident: if a partition goes quiet — a region overnight, a device fleet asleep — its watermark stops moving, and since the job’s watermark is the *minimum* across partitions, **one idle partition freezes output everywhere**. The symptom is a dashboard that stops updating with no error at all. The fix is an idleness timeout that excludes a silent partition from the calculation after a while.”

“An event arrives an hour late. Walk me through what happens.”

“It depends on two numbers I have already chosen, and I would restate them: the watermark lag, which decides when a window emits, and the allowed lateness, which decides how long I keep the state so I can revise.

Say watermark lag is ninety seconds and allowed lateness is ten minutes. An event an hour late is past both. Its window closed fifty-eight minutes ago and its state has already been discarded, so I cannot revise the answer even if I wanted to.

So it goes to the **side output** — a separate stream of late events. Not dropped silently. From there I do one of three things depending on what the pipeline is for. For a dashboard, I count them and expose ‘late events per minute’ as a metric, because a rise in that number means my watermark assumption has broken. For something that has to be right, they go into a table that a nightly batch job reconciles against, and the corrected figures replace the streaming ones. For billing, I would not rely on the streaming number at all — I would use the stream for the live view and a batch recount over the raw log as the number of record.

The thing I would push back on is a design that just discards them inside the operator. One and a half million discarded events a day at a one-million-per-minute stream with a 0.1% late rate is a real number, and the only way to know it is real is to have kept it.”

“How do you know your streaming numbers are right?”

“I do not, and the design should say so. What I do is measure the disagreement.

The standard practice is a nightly batch job that recomputes the same aggregation from the raw event log — the log is retained anyway — and compares it with what the streaming job published. The difference is a number I put on a dashboard: ‘streaming counted 9,998,600 and batch counted 10,000,000, a gap of 0.014%’.

That gives me three things. It tells me the streaming figures are trustworthy within a stated tolerance, which is much better than claiming they are exact. It turns a broken watermark or a stuck partition into a visible metric rather than a silent drift. And it gives me the authoritative number for anything that matters — billing, reporting — while the stream handles the live view.

This is essentially the lambda architecture, and I would name it and also name the criticism: two code paths computing the same thing will diverge, and someone has to maintain both. The modern answer is a kappa architecture — one streaming code path, replayed over historical data from Kafka when you need a recount — which avoids the duplication as long as your retention is long enough to replay.”

“Tumbling, sliding or session windows here?”

“Tumbling unless there is a reason, because it is the cheapest and every event is in exactly one window.

Sliding when the question is genuinely about a rolling period — ‘alert if errors in the last five minutes exceed a threshold’ — and I would price it before agreeing. A five-minute window advancing every ten seconds means every event belongs to thirty windows, so thirty times the state and thirty times the update work. That is a real number and it changes the cluster size.

Session when the boundary is behaviour rather than the clock — user sessions, a device’s period of activity, a support conversation. The gap parameter is the whole design: thirty minutes of inactivity ends a session is a product decision, not a technical one, and it should come from looking at the actual distribution of gaps.

A cheaper alternative to sliding that is worth offering: keep tumbling one-minute windows and have the *consumer* sum the last five. One thirtieth of the state, and the trade is that the granularity of the rolling figure is a minute rather than ten seconds. For most alerting that is completely acceptable and nobody asks for it because they did not know it was an option.”

The model answer

“Design a system that shows, in near real time, how many times each video on a platform was watched in the last minute. Ten million events a second at peak.”

“Let me settle the two numbers that decide everything, then the pipeline.

The two numbers are the watermark lag and the allowed lateness, and both come from measurement. I would instrument the ingest path first and look at the distribution of `arrival - event_time`. Mobile clients batch and retry, so I would expect a long tail: p50 around a second, p99 around a minute, and a small fraction of events hours old from devices that were offline. On that shape I would set a watermark lag of ninety seconds — covering p99 — and allowed lateness of ten minutes.

Bucketing by event time, one-minute tumbling windows, keyed by video ID. Not processing time: ‘watches per minute’ is a fact about viewers, and it has to be reproducible if we replay.

The pipeline. Clients publish to Kafka, partitioned by video ID so all events for one video land on one partition and are ordered — and so the aggregation is local, with no shuffle per event. Ten million a second at, say, 200 bytes is 2 GB/s, which at 250 MB/s per broker and replication factor 3 is about 24 brokers. Partitions: a few thousand, driven by consumer parallelism rather than by throughput.

A Flink job reading that topic, keyed by video, tumbling one-minute event-time windows, with the watermark and lateness above and `sideOutputLateData` on. Output goes to a store keyed by `(video_id, minute)` — so a revision **overwrites** rather than appends, which is what makes late firing safe. I would use something built for this shape rather than a relational table.

State sizing, which is the part I would show the arithmetic for. Ten million events a second, but the state is per *video*, not per event. Say two million distinct videos watched in any given minute, at about 100 bytes each: that is 200 MB per open window. With ten minutes of allowed lateness, ten windows stay open, so 2 GB — spread across the job’s parallel instances, so a few hundred megabytes each. Very comfortable, and the point is that it is driven by key cardinality, not by event rate.

Hot keys are the real risk, not volume. A single viral video can be a large fraction of all events, and since I keyed by video ID that is one partition and one task doing that work. Two mitigations: pre-aggregate at the edge — each ingest node counts locally for a second and emits partial counts, which cuts the event rate into the job by orders of magnitude for exactly the hot keys — and, if that is not enough, salt the key for the top videos into `video_id#0..9` and sum the ten at read time. **Pre-aggregation is the one I would do first**, because it helps everything and costs a second of extra latency.

What the user sees, and what I would be honest about. The dashboard number for the minute 10:00–10:01 appears at about 10:02:30 — ninety seconds of watermark plus processing — and it may be revised upward once or twice over the following ten minutes as late events arrive. So the UI shows the last complete minute, and anything more recent is marked as partial. **A design that pretends the current minute's number is final is lying**, and users work that out and stop trusting the whole thing.

And the correctness answer. The streaming figures are the live view, with a stated tolerance. A nightly batch job recomputes from the raw Kafka log and the difference is published as a metric. Anything that matters — creator payouts, reported figures — uses the batch number, not the stream. That separation is what lets me choose ninety seconds instead of two hours.”

9 Recall card

Stream processing is computing over data that never ends, and the whole difficulty is that events arrive out of order. Bucket by **event time** (when it happened, reproducible), never processing time (when you found out, depends on your network).

A watermark is a promise: “I have seen essentially everything before time T.” Usually `max event time seen - a fixed allowance`, and the allowance comes from measuring the actual delay distribution — set it above p99. It only advances when events arrive, so **one idle partition freezes the whole job's output**.

Late events need a stated policy: drop with a counter, fold into the current window, or keep state for an **allowed lateness** and emit a correction — which forces every sink to upsert by window rather than append. Send them to a side output either way, so you can count them.

Windows: tumbling (one window per event, cheapest), sliding (multiplies state and CPU by the overlap — a 5-minute window every 10 seconds is $30\times$), session (ends on a gap).

Latency is dominated by the watermark lag you chose, not by the engine. And the way you know the numbers are right is a nightly batch recount over the raw log, published as a percentage difference.

DSA topic: Cycle detection in an undirected graph **System design topic:** Stream processing basics

Code these, in this order

One rule for the whole set: **your first test is two vertices and one edge, and it must return “no cycle”**. Write that assertion before you write the function. It is the only test that catches the missing parent check, and every other test you would think of passes without it.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Graph Valid Tree	LeetCode 261 (Medium)	Connected <i>and</i> acyclic, and the <code>E = V - 1</code> shortcut.
2	Redundant Connection	LeetCode 684 (Medium)	Union-Find, and returning <i>which</i> edge closed the cycle.
3	Number of Connected Components	LeetCode 323 (Medium)	The same Union-Find, counting instead of detecting.
4	Find Critical and Pseudo-Critical Edges	LeetCode 1489 (Hard)	Cycles inside a minimum-spanning-tree build.
5	Detect Cycles in 2D Grid	LeetCode 1559 (Medium)	The parent check on an implicit grid graph.

On problem 1, use the shortcut first

Before writing any traversal, check `len(edges) == n - 1`. Say out loud what each of the three cases (`>`, `<`, `=`) tells you. Then write the single traversal that finishes the job, and explain in one sentence why checking connectivity alone is now sufficient.

On problem 5, notice what “parent” means on a grid

The grid version has the same trap in a different costume: the cell you came from is always a neighbour. Write down what you pass down instead of a vertex id, and why `(r, c)` coordinates work directly.

Then the two-vertex experiment

Write the naive version — `if neighbour in seen: return True`, no parent — and run it on:

1. Two vertices, one edge.

2. A path of five vertices.
3. A triangle.

Record all three answers. Two of them are wrong. Then add the parent check and rerun.

Then the parallel-edge experiment

Build a graph with two separate edges between vertices 0 and 1. Run:

1. The DFS parent-by-vertex version.
2. The DFS parent-by-edge-id version.
3. The Union-Find version.

Record all three answers. One of them is wrong, and it is the one you would have written by default.

Then the online comparison

Take 20,000 vertices and 20,000 edges arriving one at a time, and print whether a cycle exists after each edge. Do it once by re-running the traversal and once with Union-Find. Time both. The ratio is the argument.

The condition drill

1. State the cycle condition for an undirected graph in one line.
2. Say why “already seen” alone is wrong, and on what input it fails.
3. Say what you pass down and what the root’s value is.
4. Say why the directed version cannot use this, and what it uses instead.
5. Say when excluding by vertex is not enough.

The two-algorithms drill

1. Write the DFS version from memory.
2. Write the BFS version from memory and say where the parent lives.
3. Say when you would choose BFS.
4. Say what a self-loop does to each, and how you would handle it.

The Union-Find drill

1. Write `find` with path compression from memory.
2. Say in one sentence why an edge between two same-group vertices means a cycle.
3. Give the cost per operation and what alpha means.
4. Say when Union-Find is not just an alternative but the only option, with the arithmetic.

5. Say what Union-Find catches that the DFS parent check misses.

The counting-shortcut drill

1. State the three arithmetic cases and what each decides.
2. Say why $E = V - 1$ alone decides nothing without connectivity.
3. Give the one-comparison answer for a promised-connected graph.
4. Say why, once $E = V - 1$, checking only connectivity is enough for “is this a tree”.

The costs drill

1. Give time and space for DFS, BFS and Union-Find.
2. Say why early exit matters and what the worst case is.
3. Compare memory constants for the three at $V = 10^6$.
4. Compute traversal-per-edge versus Union-Find for $m = 100,000$.

The break-it drill

Trigger each and record the exact output or error:

1. The naive check on two vertices and one edge.
2. Parent-by-vertex on a graph with parallel edges.
3. Starting from vertex 0 only, with the cycle among vertices 7–9.
4. The three-colour directed algorithm on an undirected graph.
5. The parent check on a directed graph with a real cycle.
6. Recursive DFS on a 100,000-vertex path.

The out-of-order drill

1. Say in one sentence why stream processing is hard.
2. Define event time and processing time, and give an example where they differ by minutes.
3. Say why processing time makes numbers non-reproducible.
4. Say when processing time is the right choice.

The windows drill

1. Define tumbling, sliding and session windows in one line each.
2. Say how many windows an event belongs to in each.
3. Compute the state multiplier for a 5-minute window sliding every 10 seconds.
4. Give the cheaper alternative to a sliding window and what it costs.
5. Say what decides a session’s gap parameter.

The watermark drill

1. Define a watermark precisely.
2. Give the usual formula for computing one.
3. Say where the allowance number comes from.
4. Say why the watermark only advances when events arrive, and the incident that causes.
5. Say why the job's watermark is the minimum across partitions.

The lateness drill

1. Name the three policies for a late event.
2. Say which is right for a dashboard, for an hourly total, and for billing.
3. Distinguish the watermark from allowed lateness — two different numbers, two different jobs.
4. Say what a side output is for and why dropping silently is worse.
5. Say what late firing forces on every downstream consumer.

The numbers drill

1. Compute state for 10M keys, one-minute tumbling windows.
2. Recompute with ten minutes of allowed lateness.
3. Recompute for a 5-minute sliding window every 10 seconds.
4. Given a delay distribution (p50 1.2 s, p99 45 s, p99.9 4 min), choose a watermark and state what it drops.
5. Break down end-to-end latency and name the dominant term.
6. Compute discarded events per day at 1M/min and a 0.1% late rate.
7. Size the state for a 30-minute stream-to-stream join at 1M impressions a minute.

The correctness drill

1. Say how you find out whether the streaming numbers are right.
2. Describe the batch recount and what you publish from it.
3. Name the architecture that pairs them and the criticism of it.
4. Give the modern alternative and what it requires.
5. Say exactly where Flink's exactly-once guarantee stops.

The trade-offs drill

1. State the latency-versus-completeness trade in one sentence.
2. Say what allowed lateness costs, in more than just memory.

3. Say why many teams choose to be slightly wrong, and when that is right.
4. Give three cases where a batch job beats a streaming job.
5. Say what makes a mis-tuned watermark worse than a failed batch job.

The failure drill

For each, say what happens and what you would build:

1. A region goes quiet at 2 a.m. and the dashboard stops updating.
2. An event arrives an hour after its window closed.
3. A viral video makes one partition 40% of all traffic.
4. Allowed lateness is set to one hour on a one-minute window over 10M keys.
5. A sink that appends rather than upserts, with late firing enabled.
6. Someone asks for “real-time and exactly right”.
7. The streaming total and the batch total differ by 0.014%.

Two of the seven are working as designed. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Does this undirected graph contain a cycle?* The condition with the parent check, why “seen” alone fails on two vertices and one edge, the outer loop, the parallel-edge question asked rather than assumed, and the $E > V - 1$ shortcut offered first.
 2. *Given n nodes and these edges, is this a valid tree?* The arithmetic first, then why one traversal is now enough, then the Union-Find alternative and the case where it is the only option.
 3. *How do you count events per minute when events arrive late?* Event time not processing time and why; the watermark as a chosen number from a measured distribution; the three lateness policies with the one you would pick; the side output; and the state cost of allowed lateness.
-

Before you move on

- [] My first test is two vertices and one edge.
- [] I can state the undirected cycle condition in one line.
- [] I know why “already seen” alone is wrong and on what input.

- [] I can write both the DFS and BFS versions from memory.
- [] I know why the directed version needs a different mechanism.
- [] I ask about parallel edges and self-loops rather than assuming.
- [] I know excluding by edge id fixes the parallel-edge case.
- [] I always loop over every vertex with a shared `seen`.
- [] I can write Union-Find with path compression from memory.
- [] I can say in one sentence why it detects cycles.
- [] I know when Union-Find is the only option, with numbers.
- [] I know what Union-Find catches that the parent check misses.
- [] I can state the three arithmetic cases and what each decides.
- [] I know why `E = V-1` alone decides nothing.
- [] I know why connectivity alone finishes the tree question.
- [] I can compare the memory constants of all three approaches.
- [] I can say why stream processing is hard, in one sentence.
- [] I can define event time and processing time and say which to use.
- [] I can define a watermark precisely and give the formula.
- [] I know where the allowance number comes from.
- [] I know why an idle partition freezes the output.
- [] I can name the three lateness policies and pick one per use case.
- [] I know the watermark and allowed lateness are different numbers.
- [] I always send late events to a side output.
- [] I know what late firing forces on the sink.
- [] I can compute state for tumbling, sliding and lateness-extended windows.
- [] I know the watermark lag dominates end-to-end latency.
- [] I can describe the batch recount and what it buys.
- [] I know exactly where exactly-once stops.
- [] I answered all three questions above out loud.

Cycle detection in a directed graph

DATA STRUCTURES AND ALGORITHMS

Cycle detection in a directed graph

SYSTEM DESIGN

Object storage, S3-style

1 What this is, and why they ask it

In a directed graph a cycle is a set of vertices that each wait for the next, all the way round. Nothing in the set can ever go first. That is why the question matters: a directed cycle means a schedule that cannot be run, a build that cannot be ordered, a set of courses nobody can start.

The algorithm is a depth-first search with three states instead of two. **White** for untouched, **grey** for “started and not finished — so it is on the path I am standing on right now”, **black** for finished. An edge to a grey vertex is a cycle. An edge to a black vertex is not.

That distinction is the entire lesson, and the two-state version — “have I seen this before?” — is wrong in a way that is easy to miss, because it reports a cycle on the most ordinary shape in dependency graphs: two things depending on the same third thing.

They ask this because it is the gateway to topological sort, which is [day 134](#), and because “can all courses be finished?” (LeetCode 207) is one of the most-asked graph questions anywhere. It is also the cleanest example in the whole course of a bug that only appears on realistic input: every small test passes, and a graph with one shared dependency fails.

By the end of this lesson you can write the three-colour version recursively and iteratively, explain grey versus black in one sentence, use Kahn’s in-degree method as the second answer, return the actual cycle rather than just a boolean, and say precisely why the undirected algorithm from [yesterday](#) does not work here.

2 The story

Sunita needs a no-dues certificate before she can collect her son’s school leaving papers, and she has taken the morning off work to get it.

She starts at the school office at ten past nine. The clerk there looks at her form, turns it round, and points at a box near the bottom.

“Library clearance first. Second floor.”

The library is on the second floor and there is nobody at the desk until twenty to ten. When the woman comes, she reads the form and shakes her head.

“This needs the sports department stamp. They have to confirm he returned the kit.”

Sports is in the annexe behind the ground. The man there is helpful and quick and he tells her, without even sitting down, that he cannot stamp anything until the fee section confirms the term fees are cleared, because last year there was some trouble about it.

The fee section is back in the main building, first floor, and the queue is eleven people long. At half past eleven she gets to the counter.

The woman at the fee section listens to the whole thing, takes the form, looks at it for a moment, and says the sentence that ends Sunita's morning.

“We can't clear the fees until the office gives us the no-dues certificate.”

Sunita stands there.

She has walked to four places and every one of them is waiting for one of the others, and the last one is waiting for the first one. Nobody in that loop is being difficult. Every one of them has a reasonable rule and every rule points at somebody else in the same circle. There is no first step, and there is no amount of walking that will find one.

What is worth noticing, and what she works out on the bus, is that this is not the same as being sent to two places. Earlier in the year she had needed a transport form, and for that the office had wanted both the bus in-charge's signature and the class teacher's, and both of *them* had wanted the fee section's stamp. That was three offices and two of them wanting the same thing, and it took forty minutes, and it worked perfectly well.

Two people waiting on the same third person is fine. There is still somebody who can go first.

Four people waiting on each other in a ring is a different thing entirely, and no amount of patience fixes it.

3 The idea in plain English

Sunita's morning is a directed cycle, and her observation on the bus is the three-colour rule.

A directed edge means “this must happen first”. The fee section pointing at the office is an edge from the office to the fee section — office first, fee section after. The direction carries the whole meaning, and [day 125](#) made the point that getting it backwards produces a confidently wrong answer.

A cycle means nothing can go first. Office → library → sports → fees → office. Every vertex in that loop waits for another vertex in the loop. There is no starting point, and unlike an undirected loop, this is not a convenience issue — it makes the whole thing impossible.

Now the two states that are not enough. The naive check is “if I reach a vertex I have seen before, that is a cycle”. Sunita’s transport form is the counter-example: the bus in-charge and the class teacher both wait on the fee section. Walking office → bus in-charge → fees, then back up and office → class teacher → fees, you reach the fee section a second time. **You have seen it before, and there is no cycle.** It is a shared dependency, which is the most ordinary thing in any real dependency graph.

So “seen” is the wrong question. The right question is “am I currently inside it?”

Three states, and the middle one is the whole idea:

- **White** — not visited. Nobody has been here.
- **Grey** — visiting. The call to this vertex has started and has not returned, so **this vertex is on the path from the root down to where I am standing right now**. It is an ancestor of me.
- **Black** — finished. This vertex and everything below it have been fully explored and returned.

An edge to a grey vertex is a cycle, because grey means “you are above me on this path”, and an edge back up to an ancestor closes a loop. That is the fee section pointing at the office while Sunita is still standing in the chain that started at the office.

An edge to a black vertex is fine. Black means “I finished that region and came back out”. Reaching it again is a second route into explored territory — the transport form’s fee section, visited and done, and being pointed at again by a different department.

And an edge to a white vertex is just the search continuing.

The colours are exactly “recursion stack” versus “visited”. If you prefer sets to colours, keep a `seen` set and an `on_stack` set, add to both on entry and **remove from `on_stack` on the way out**. That removal is the line people forget, and forgetting it turns the algorithm back into the two-state version.

Why yesterday’s parent check does not work here. In an undirected graph, every edge appears in both adjacency lists, so every edge produces one false alarm — and the parent check suppresses it. In a directed graph, $A \rightarrow B$ puts `B` in `A`’s list and nothing in `B`’s. There is no mirror edge, so there is nothing for a parent check to suppress, and it would simply never fire usefully. **Same word, two different problems, two different algorithms**, and the first thing to establish about any cycle question is which kind of graph you have.

The second algorithm is Kahn’s, and it is worth knowing because it comes with a bonus. Count each vertex’s **in-degree** — how many arrows point at it. Repeatedly take any vertex with in-degree zero (nothing is waiting on it, so it can go now), output it, and

decrement the in-degree of everything it points at. If you run out of zero-in-degree vertices before outputting all of them, the ones left form a cycle.

That is Sunita's problem stated the other way: **a cycle is exactly the set of offices where nobody has in-degree zero**. And the bonus is that the output order is a valid schedule — a topological sort — so Kahn's answers "is there a cycle" and "give me a valid order" with one pass. That is [day 134](#), and it is why most people reach for Kahn's on the course- schedule family.

Returning the actual cycle takes one more thing: keep the current path as a list, and when you find a grey vertex, the cycle is the slice of the path from that vertex onwards. Interviewers ask for this as the follow-up, and it costs about four lines.

4 The picture

Sunita's two mornings, side by side:

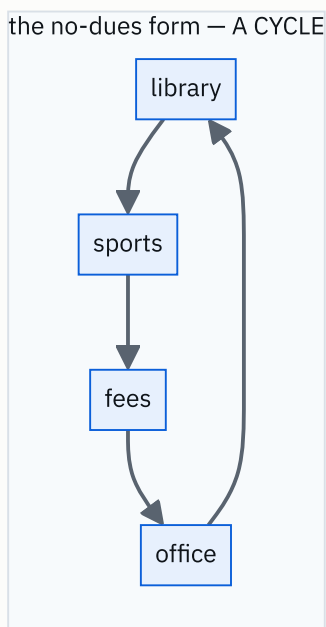


FIGURE 133.1

the transport form — a shared dependency, NO cycle

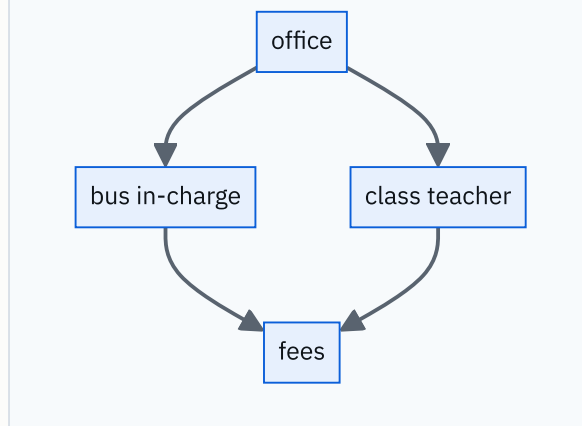


FIGURE 133.2

What to notice. In the second picture the fee section is reached twice, from two different places, and there is no cycle. **A two-state check reports a cycle on the second picture**, which is why it is wrong: the second picture is what almost every real dependency graph looks like.

The colours during a walk, traced:

graph: 0 → 1, 1 → 2, 2 → 0, 1 → 3

step	action	0	1	2	3	path
1	enter 0	GREY	white	white	white	[0]
2	enter 1	GREY	GREY	white	white	[0,1]
3	enter 2	GREY	GREY	GREY	white	[0,1,2]
4	2 looks at 0	GREY	GREY	GREY	white	[0,1,2]
	0 is GREY → CYCLE. The cycle is path[index_of(0):] = [0,1,2]					

And the diamond, where two states fail:

graph: 0 → 1, 0 → 2, 1 → 3, 2 → 3

step	action	0	1	2	3	
1	enter 0	GREY	white	white	white	
2	enter 1	GREY	GREY	white	white	
3	enter 3	GREY	GREY	white	GREY	
4	3 has no edges	GREY	GREY	white	BLACK	← 3 finishes
5	1 finishes	GREY	BLACK	white	BLACK	
6	enter 2	GREY	BLACK	GREY	BLACK	
7	2 looks at 3	GREY	BLACK	GREY	BLACK	
	3 is BLACK → fine, skip.					(two-state: "seen!" → WRONG)
8	2 finishes, 0 finishes. No cycle.					Correct.

What to notice at step 7. Vertex 3 has been visited, and it is not on the current path — it finished at step 4. Grey and black are what tell those two situations apart, and seen cannot.

Kahn's, on the same two graphs:

CYCLE GRAPH 0→1, 1→2, 2→0, 1→3

in-degrees: 0:1 1:1 2:1 3:1
vertices with in-degree 0: NONE
→ cannot start at all → cycle, immediately

DIAMOND 0→1, 0→2, 1→3, 2→3

in-degrees: 0:0 1:1 2:1 3:2
take 0 → decrement 1 and 2 → 1:0 2:0
take 1 → decrement 3 → 3:1
take 2 → decrement 3 → 3:0
take 3
output 4 of 4 vertices → no cycle, and [0,1,2,3] is a valid order

What to notice. Kahn's never recurses and never colours anything. It just repeatedly asks "who can go now?", and a cycle is the state where the answer is "nobody" while work remains.

5 The code, built step by step

The three-colour recursive version, which is the one to have automatic.

```

WHITE, GREY, BLACK = 0, 1, 2

def has_cycle(graph: dict[int, list[int]], n: int) → bool:
    colour = [WHITE] * n

    def visit(vertex: int) → bool:
        colour[vertex] = GREY                # on the current path
        for neighbour in graph[vertex]:
            if colour[neighbour] == GREY:
                return True                  # back edge: a cycle
            if colour[neighbour] == WHITE and visit(neighbour):
                return True
        colour[vertex] = BLACK              # finished, and so is everything below
        return False

    return any(colour[v] == WHITE and visit(v) for v in range(n))

```

Read the two colour tests. `GREY` returns true immediately. `BLACK` is skipped silently by falling through both conditions — no check needed, because neither branch matches.

`colour[vertex] = BLACK` is on the way *out*, after the loop. That placement is the whole difference between grey and black, and moving it before the loop turns this back into the two-state version.

The `any(...)` is the outer loop over every vertex. **A directed cycle can sit in a region that nothing points into**, so starting from vertex 0 and stopping is a real bug with no symptom on a connected sample.

Now the same thing with sets, if you find colours awkward:

```

def has_cycle_sets(graph: dict[int, list[int]], n: int) → bool:
    seen: set[int] = set()
    on_path: set[int] = set()

    def visit(vertex: int) → bool:
        seen.add(vertex)
        on_path.add(vertex)
        for neighbour in graph[vertex]:
            if neighbour in on_path:
                return True
            if neighbour not in seen and visit(neighbour):
                return True
        on_path.remove(vertex)                # THE line people forget
        return False

    return any(v not in seen and visit(v) for v in range(n))

```

`on_path.remove(vertex)` on the way out is exactly `colour = BLACK`. Leave it out and every visited vertex stays “on the path” forever, which is the two-state bug in a different costume.

Returning the cycle itself, which is the standard follow-up:

```
def find_cycle(graph: dict[int, list[int]], n: int) → list[int] | None:
    colour = [WHITE] * n
    path: list[int] = []

    def visit(vertex: int) → list[int] | None:
        colour[vertex] = GREY
        path.append(vertex)
        for neighbour in graph[vertex]:
            if colour[neighbour] == GREY:
                return path[path.index(neighbour):] + [neighbour]
            if colour[neighbour] == WHITE:
                found = visit(neighbour)
                if found:
                    return found
        colour[vertex] = BLACK
        path.pop() # leaving this vertex
        return None

    ...
```

`path` mirrors the grey set — push on entry, pop on exit — so when a grey vertex is found, the cycle is the slice from where it appears to the end. `path.index(neighbour)` is a linear scan; if you need it fast, keep a dictionary from vertex to its position in the path.

Now the iterative version, because a chain of a hundred thousand dependencies is a legal input:

```
def has_cycle_iterative(graph: dict[int, list[int]], n: int) → bool:
    colour = [WHITE] * n
    for start in range(n):
        if colour[start] != WHITE:
            continue
        stack = [(start, iter(graph[start]))]
        colour[start] = GREY
        while stack:
            vertex, neighbours = stack[-1]
            nxt = next(neighbours, None)
            if nxt is None: # exhausted: this vertex finishes
                colour[vertex] = BLACK
                stack.pop()
                continue
            if colour[nxt] == GREY:
                return True
            if colour[nxt] == WHITE:
                colour[nxt] = GREY
                stack.append((nxt, iter(graph[nxt])))
    return False
```

The trick is storing an **iterator** with each stack entry, so a vertex is only marked black when its neighbours are exhausted. That reproduces the recursive version’s “on the way out” moment exactly, and it is a cleaner conversion than the two-push marker technique when you need the finish event.

And Kahn’s, which is often the better answer:

```

from collections import deque

def has_cycle_kahn(graph: dict[int, list[int]], n: int) → bool:
    in_degree = [0] * n
    for vertex in range(n):
        for neighbour in graph[vertex]:
            in_degree[neighbour] += 1

    queue = deque(v for v in range(n) if in_degree[v] == 0)
    output = 0
    while queue:
        vertex = queue.popleft()
        output += 1
        for neighbour in graph[vertex]:
            in_degree[neighbour] -= 1
            if in_degree[neighbour] == 0:
                queue.append(neighbour)
    return output != n # left over → cycle

```

`output != n` is the cycle test, and it reads well: if some vertices were never emitted, nothing ever brought their in-degree to zero, which means each is waiting on something in a loop.

No recursion, no colours, and the emitted order is a valid schedule — so the same function answers the follow-up question for free.

The complete solution

```
"""Cycle detection in a directed graph: three colours, iterative, Kahn's, and the cycle itself."""

from __future__ import annotations

from collections import deque

WHITE, GREY, BLACK = 0, 1, 2


def build(n: int, edges: list[tuple[int, int]]) → dict[int, list[int]]:
    """edges are (from, to). Built from range(n) so isolated vertices exist."""
    graph: dict[int, list[int]] = {v: [] for v in range(n)}
    for a, b in edges:
        graph[a].append(b)
    return graph


def has_cycle(graph: dict[int, list[int]], n: int) → bool:
    """Three colours. GREY = on the current path. O(V + E)."""
    colour = [WHITE] * n

    def visit(vertex: int) → bool:
        colour[vertex] = GREY
        for neighbour in graph[vertex]:
            if colour[neighbour] == GREY:
                return True
            if colour[neighbour] == WHITE and visit(neighbour):
                return True
        colour[vertex] = BLACK
        return False

    return any(colour[v] == WHITE and visit(v) for v in range(n))


def find_cycle(graph: dict[int, list[int]], n: int) → list[int] | None:
    """The cycle itself, as a list of vertices, or None."""
    colour = [WHITE] * n
    path: list[int] = []

    def visit(vertex: int) → list[int] | None:
        colour[vertex] = GREY
        path.append(vertex)
        for neighbour in graph[vertex]:
            if colour[neighbour] == GREY:
                return path[path.index(neighbour):] + [neighbour]
            if colour[neighbour] == WHITE:
                found = visit(neighbour)
                if found is not None:
                    return found
        colour[vertex] = BLACK
        path.pop()
        return None

    for vertex in range(n):
        if colour[vertex] == WHITE:
            found = visit(vertex)
            if found is not None:
                return found
    return None


def has_cycle_iterative(graph: dict[int, list[int]], n: int) → bool:
    """No recursion limit. The stored iterator gives the 'finished' moment."""
    colour = [WHITE] * n
    for start in range(n):
        if colour[start] != WHITE:
            continue
        colour[start] = GREY
        stack = [(start, iter(graph[start]))]
        while stack:
            vertex, neighbours = stack[-1]
            nxt = next(neighbours, None)
```

```

        if nxt is None:
            colour[vertex] = BLACK
            stack.pop()
            continue
        if colour[nxt] == GREY:
            return True
        if colour[nxt] == WHITE:
            colour[nxt] = GREY
            stack.append((nxt, iter(graph[nxt])))
    return False

def kahn(graph: dict[int, list[int]], n: int) → list[int] | None:
    """A valid order, or None if there is a cycle. Answers both questions."""
    in_degree = [0] * n
    for vertex in range(n):
        for neighbour in graph[vertex]:
            in_degree[neighbour] += 1
    queue = deque(v for v in range(n) if in_degree[v] == 0)
    order: list[int] = []
    while queue:
        vertex = queue.popleft()
        order.append(vertex)
        for neighbour in graph[vertex]:
            in_degree[neighbour] -= 1
            if in_degree[neighbour] == 0:
                queue.append(neighbour)
    return order if len(order) == n else None

if __name__ == "__main__":
    ring = build(4, [(0, 1), (1, 2), (2, 0), (1, 3)])          # office → ... → office
    diamond = build(4, [(0, 1), (0, 2), (1, 3), (2, 3)])        # shared dependency
    far = build(6, [(0, 1), (3, 4), (4, 5), (5, 3)])           # cycle nothing points into

    for name, graph, n in (("ring", ring, 4), ("diamond", diamond, 4), ("far cycle", far, 6)):
        print(f"{name:10} recursive={has_cycle(graph, n)!s:5} "
              f"iterative={has_cycle_iterative(graph, n)!s:5} "
              f"kahn={(kahn(graph, n) is None)!s:5} "
              f"cycle={find_cycle(graph, n)!s:5}")

```

Running it:

```

ring      recursive=True  iterative=True  kahn=True  cycle=[0, 1, 2, 0]
diamond   recursive=False iterative=False  kahn=False cycle=None
far cycle  recursive=True  iterative=True  kahn=True  cycle=[3, 4, 5, 3]

```

Three things to look at. The **diamond** row must be `False` — that is the test that catches a two-state implementation, and it is the only one that does. The **far cycle** row has a loop among vertices 3, 4 and 5 that nothing reachable from 0 touches, so a search starting only at vertex 0 would report `False`. And `find_cycle` returns the loop with its first vertex repeated at the end, which is the conventional way to make the closure explicit.

6 What it costs

Three-colour DFS.

each vertex coloured GREY once, BLACK once	$2V$
each edge examined once	E (directed: once, not twice)

colour array	$O(V + E)$ time
recursion stack or explicit stack	$O(V)$
	$O(V)$

	$O(V)$ space

Note E and not $2E$: a directed edge lives in exactly one adjacency list, so the sum of out-degrees is E . That is a factor-of-two difference from the undirected case and worth stating precisely if asked.

Early exit matters and the worst case is the acyclic graph:

$V = 1,000,000$, $E = 3,000,000$, cycle among the first few vertices
worst case $4,000,000$ steps
actual ~ 10 steps

no cycle at all $\rightarrow$ every vertex and edge must be examined $\rightarrow$ the full $4,000,000$

Kahn's.

computing in-degrees	one pass over all edges	E
each vertex queued once		V
each edge decremented once		E

		$O(V + E)$ time
in-degree array + queue		$O(V)$ space

Identical complexity to the DFS version. The choice between them is not about speed.

Where they differ in practice:

recursion	three-colour DFS	Kahn's
gives a valid order	yes (unless converted)	never
gives the cycle itself	only via finish order	directly
memory constant	easily, via the path	only the set of leftovers
detects which vertices are in the cycle	colour array + stack	in-degree array + queue
	the cycle, exactly	everything in or reachable from a cycle

Kahn's tells you a cycle exists and which vertices are stuck; it does not hand you the loop. The three-colour version hands you the loop and needs care about recursion depth. Say that distinction and pick on what the question asks for.

Recursion depth, with the arithmetic:

Python usable frames	~960
a chain of dependencies 100,000 long	→ 100,000 frames
	→ RecursionError
sys.setrecursionlimit(10**6)	→ Segmentation fault (no traceback)

constraints say $n \leq 10^5$ → write the iterative version or use Kahn's
constraints say $n \leq 2000$ → recursion is fine

Memory constants at scale:

$V = 1,000,000$	
colour array (list of ints)	~8 MB
in-degree array	~8 MB
seen + on_path as Python sets	~100 MB (hash tables, boxed ints)

Arrays indexed by vertex number beat sets by an order of magnitude in Python, so when vertices are `0..n-1`, use lists. Sets are for string or tuple vertices.

And the cost of the follow-up:

has_cycle	$O(V + E)$
find_cycle	$O(V + E)$ plus path.index, which is $O(\text{path length})$ per hit → keep a position dict for $O(1)$ if it matters
kahn returning an order	$O(V + E)$, same pass

7 The traps

Two states instead of three

The near-miss, and the one that matters:

```
def visit(vertex):
    seen.add(vertex)
    for neighbour in graph[vertex]:
        if neighbour in seen:
            return True                # "cycle"
        if visit(neighbour):
            return True
    return False
```

```
>>> diamond = build(4, [(0,1), (0,2), (1,3), (2,3)])
>>> has_cycle_two_state(diamond, 4)
True
```

There is no cycle. Vertex 3 was reached from 1, finished, and then 2 reached it again. **A shared dependency is the most common shape in any real dependency graph** — two modules importing the same library, two courses requiring the same prerequisite — so this bug fires on essentially every realistic input while passing every trivial test.

Forgetting to remove from the path set

```
def visit(vertex):
    seen.add(vertex)
    on_path.add(vertex)
    for neighbour in graph[vertex]:
        if neighbour in on_path:
            return True
        if neighbour not in seen and visit(neighbour):
            return True
    return False                                # on_path.remove(vertex) missing
```

Identical symptom to the two-state bug, because `on_path` now grows monotonically and becomes `seen`. The removal is what makes the set mean “currently in progress”, and there is no error to tell you it is missing.

Marking BLACK before the loop

```
def visit(vertex):
    colour[vertex] = BLACK                    # ← wrong place
    for neighbour in graph[vertex]:
        if colour[neighbour] == GREY:
            return True
    ...
```

Nothing is ever grey, so no cycle is ever found:

```
>>> has_cycle_broken(ring, 4)
False                                # there is a 0→1→2→0 cycle
```

A false negative, which is worse than a false positive, because the system happily schedules something impossible.

Only starting from one vertex

```
return visit(0)
```

```
>>> far = build(6, [(0,1), (3,4), (4,5), (5,3)])
>>> visit_from_zero(far)
False                                # 3→4→5→3 is a cycle
```

In a directed graph this is much more likely than in an undirected one, because “reachable from vertex 0” is a far smaller set when edges only go one way. **The outer loop over all vertices, with a shared colour array, is required.**

Using the undirected parent check

```
def visit(vertex, parent):
    seen.add(vertex)
    for neighbour in graph[vertex]:
        if neighbour not in seen:
            if visit(neighbour, vertex): return True
        elif neighbour != parent:
            return True
```

On a directed graph there are no mirror edges for the parent check to suppress, so it mostly behaves like the two-state version — reporting cycles on diamonds — while also missing a genuine two-vertex cycle `A → B → A`, because `A` is `B`’s parent:

```
>>> has_cycle_with_parent_check(build(2, [(0,1), (1,0)]), 2)
False                                     # 0 → 1 → 0 is a cycle
```

Wrong in both directions at once. Yesterday’s algorithm and today’s are not interchangeable.

Recursion depth

```
Traceback (most recent call last):
  File "cycles.py", line 11, in visit
    if colour[neighbour] == WHITE and visit(neighbour):
[Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

A build system with a long dependency chain, a package graph, or the adversarial input on a judge. `n ≤ 105` in the constraints means Kahn’s or the iterative version.

Building the graph from the edges only

```
graph = defaultdict(list)
for a, b in edges:
    graph[a].append(b)
for vertex in graph:                                # misses vertices with no outgoing edges
    ...
```

A course with no prerequisites and no dependents never becomes a key. For the boolean answer that is harmless; for Kahn’s it is fatal, because `len(order) == n` will never hold if some vertices are not in the structure at all. **Build from** `range(n)`.

8 In the interview

How it gets asked

- *“Does this directed graph contain a cycle?”* — the direct version.
- *“Can all courses be finished?”* — LeetCode 207, and the follow-up is “now give me an order”.
- *“Detect a circular dependency in this build system / package manager.”*
- *“Find the cycle, not just whether one exists.”*
- *“Why does your undirected solution not work here?”*
- *“Which vertices are involved in the deadlock?”*

The first ninety seconds

“Depth-first search with three states, and the middle state is the whole algorithm.

White is untouched. **Grey means I have started this vertex and not finished it, so it is on the path from the root down to where I am standing right now** — it is an ancestor of me. Black means it and everything below it are finished.

An edge to a grey vertex is a cycle, because it points back up at an ancestor. An edge to a black vertex is not a cycle — it is a second route into a region I have already fully explored.

The reason two states fail is worth showing rather than asserting. Take a diamond: 0 points to 1 and 2, and both 1 and 2 point to 3. DFS goes 0, 1, 3, finishes 3, finishes 1, comes back to 0, goes to 2, and 2 looks at 3. With a plain `seen` set, 3 has been seen and I report a cycle. There is no cycle — that is a shared dependency, which is what every real dependency graph is full of. So the check has to be ‘is it on my current path’, not ‘have I been here’.

Two things about the structure. **The loop is over every vertex**, because in a directed graph a cycle can easily sit in a region nothing points into, and ‘reachable from vertex 0’ is a much smaller set than in an undirected graph. And **I would write it iteratively, or use Kahn’s, if n can be large** — a chain of a hundred thousand dependencies overflows the stack, and raising the limit gives a segfault instead of an error.

$O(V + E)$ time — note E , not $2E$, because a directed edge is in exactly one adjacency list — and $O(V)$ space.

The alternative I would probably actually write is Kahn’s, because it answers the follow-up for free: repeatedly take a vertex with in-degree zero and remove it; if you cannot output all n , the leftovers are in a cycle, and if you can, the order you emitted them in is a valid schedule. Do you want the boolean, or the order?”

The follow-ups

“Why exactly does the undirected version not work?”

“Because the two graphs have opposite problems.

In an undirected graph, every edge appears in **both** adjacency lists, so walking `A` to `B` and then looking at `B`’s neighbours always shows `A`. Every edge produces exactly one false alarm, and the parent check exists to suppress it.

In a directed graph, `A → B` puts `B` in `A`’s list and puts nothing in `B`’s. There is no mirror edge, so the parent check has nothing to suppress — and worse, it actively breaks things: on a genuine two-vertex cycle `A → B → A`, `B` looks at `A`, finds it seen, and skips it because `A` is its parent. So the parent check both fails to help and introduces a false negative.

What the directed case actually needs is the ancestor question, which the parent check cannot express because a cycle can close on a vertex several levels up, not just the immediate parent.

So: **undirected needs ‘is this the edge I came in on’, directed needs ‘is this vertex still on my path’**. Different questions, different mechanisms, and the first thing I establish about a cycle problem is which kind of graph I have.”

“Return the actual cycle.”

“One extra list, and it mirrors the grey set exactly.

I keep a `path` list: append on entry, pop on exit — so at any moment `path` holds precisely the grey vertices, in order from the root. When I find an edge to a grey vertex, that vertex appears somewhere in `path`, and the cycle is the slice from its position to the end, plus the vertex again to close it.

`path.index(v)` is a linear scan of the path, which is fine when you return immediately, but if I needed to find many cycles I would keep a dictionary from vertex to its index in the path so that lookup is $O(1)$.

The pop on the way out is the same line as `colour = BLACK`, and forgetting it is the same bug — `path` stops meaning ‘currently in progress’ and starts meaning ‘ever visited’.

One thing worth noting: this returns *a* cycle, the first one the search happens to close, not the shortest or the most important. If they wanted the smallest cycle, that is a different problem — a BFS from each vertex looking for a way back — and much more expensive at $O(V \times (V + E))$.”

“Kahn’s or DFS? Which would you write?”

“Kahn’s, for most versions of this question, and I would say why rather than treat it as a preference.

Three reasons. It answers ‘is there a cycle’ **and** ‘give me a valid order’ in the same pass, and the second is almost always the follow-up. It has no recursion, so a hundred-thousand-long dependency chain is not a problem. And the mechanism reads like the domain — ‘take anything nothing is waiting on, remove it, repeat’ is how a person would actually schedule the work.

What DFS gives me that Kahn’s does not is the cycle itself. Kahn’s tells me which vertices were never emitted, and that set is everything in a cycle **plus everything downstream of one**, so it over-reports for debugging purposes. If the question is ‘which packages are in the circular dependency’, I want the three-colour DFS with the path.

They are the same complexity, $O(V + E)$, so this is entirely about which output you need.”

“Which vertices are actually in the deadlock?”

“Kahn’s leftovers are not the answer, and I would say so, because it is a subtle over-report: a vertex that merely *depends* on something in a cycle also never reaches in-degree zero, so it appears in the leftovers without being in any cycle itself.

The precise answer is the **strongly connected components** of the graph with more than one vertex — a set where every vertex can reach every other one following the arrows is exactly a set of things mutually waiting. Tarjan’s or Kosaraju’s algorithm finds all of them in $O(V + E)$, and each non-trivial component is one independent deadlock. A single vertex with a self-loop counts too.

That is also the right answer operationally, because a build system with three separate circular dependencies wants three reports, not one big blob.

I would not write Tarjan’s from memory unprompted — it has a couple of fiddly details around the low-link values — but naming it, saying it is one linear DFS, and explaining why the Kahn’s leftovers are the wrong set is the substance of the answer.”

The model answer

“A build system has modules and dependencies. Detect circular dependencies, report them clearly to the developer, and if there are none, produce a build order.”

“Two outputs are wanted, so I would use two algorithms rather than force one to do both jobs badly.

The model first. A vertex is a module. An edge from A to B means **A must be built before B** — so the edge points from the dependency to the dependent, and I would state that direction as an English sentence before writing it, because the input file almost certainly expresses it the other way round (“module B requires A”) and reversing it silently is the classic bug.

The happy path is Kahn’s. Compute in-degrees, seed a queue with everything at zero — the modules with no dependencies — and repeatedly emit and decrement. If all `n` modules come out, there is no cycle and the emitted order is a valid build order. One pass, $O(V + E)$, no recursion, which matters because a real dependency chain can be thousands deep.

A nice property I would use: if I want to *parallelise* the build, Kahn’s gives me that for free. Everything in the queue at the same moment has no unbuilt dependencies, so it can all be built simultaneously. Emitting level by level rather than one at a time turns the topological sort into a build schedule with a critical path, and that is often what the user actually wants.

The unhappy path is where the design work is. If Kahn’s cannot emit everything, there is at least one cycle, and ‘circular dependency detected’ is a useless error message. So I run a second pass over the unemitted vertices only — three-colour DFS carrying the path — and report the actual loop: `auth → session → user → auth`. That is the difference between an error a developer can act on and one they file a ticket about.

And I would report *all* the cycles, not the first one. Kahn’s leftovers can contain several independent loops plus everything downstream of them, so after finding and reporting one cycle I would continue the DFS over the remaining unvisited vertices. The rigorous version is strongly connected components — each non-trivial component is exactly one circular dependency group — and I would name that as what I would use if the codebase is large enough that developers hit multiple cycles at once.

What I would be careful about in the input. Modules with no dependencies and nothing depending on them must still appear, so the structure is built from the full module list rather than from the dependency pairs — otherwise `len(order) == n` never holds and I report a phantom cycle. A module depending on itself is a legal input and a real cycle, and Kahn’s handles it naturally since its in-degree never reaches zero. And I would deduplicate repeated dependency declarations, because a

duplicate edge inflates the in-degree and leaves a module permanently unbuildable — **that is a cycle report on a graph with no cycle**, which is the most confusing possible failure.

Cost: $O(V + E)$ for the order, $O(V + E)$ for the cycle report, $O(V)$ memory. For a hundred thousand modules with half a million dependency edges that is well under a second, and the in-degree array as a list of integers is about a megabyte, against maybe ten times that if I used sets keyed by module name — so I would map names to integers once up front.”

9 Recall card

Three colours: white untouched, grey = started and not finished, so on the current path, black = finished. An edge to grey is a cycle; an edge to black is a shared dependency and is fine.

Two states report a cycle on a diamond — $0 \rightarrow 1 \rightarrow 3$, $0 \rightarrow 2 \rightarrow 3$ — which is the shape every real dependency graph is full of. If you use sets instead of colours, the missing line is `on_path.remove(vertex)` on the way out.

Yesterday's parent check is wrong here, in both directions: there are no mirror edges for it to suppress, and it silently misses a genuine $A \rightarrow B \rightarrow A$.

Kahn's is usually the better answer: repeatedly take a vertex with in-degree zero; if fewer than `n` come out, the leftovers are stuck in or behind a cycle — and the emitted order is a valid schedule, which is the follow-up. Same $O(V + E)$, no recursion.

Loop over every vertex (a directed cycle hides easily in an unreachable region), **build from** `range(n)`, and write it iteratively or use Kahn's when `n` can exceed a few thousand.

1 What this is, and why they ask it

Object storage is where the big things go. Images, videos, backups, logs, model weights, uploaded documents — anything that is a whole item you write once and read many times.

It is not a file system, and the difference is the lesson. There are no folders. There is no way to change the middle of a stored object. There is no way to append. You give it a name and a blob of bytes, and later you ask for that name and get those bytes back. That is the entire interface, and the restrictions are exactly what let it be enormously durable and enormously cheap.

They ask this because “where do the uploaded images actually go?” comes up in almost every design that touches media, and because the answer distinguishes people who have shipped something from people who have not. The naive answer — put them in the database — has a specific set of consequences you can quantify, and being able to do that arithmetic is the point.

It also carries the two most useful practical patterns in web architecture: **presigned URLs**, so uploads and downloads never pass through your servers, and **a CDN in front**, so reads never reach the store at all.

By the end of this lesson you can say what object storage is and is not, quote durability and cost numbers, design an upload path that does not melt your API servers, choose a storage class from an access pattern, and answer the “images in the database?” question with numbers rather than opinion.

2 The story

The cloakroom at Kalyan station is a green-painted room to the left of platform one, and Bhaskar has run it since 2014.

The system has not changed in that time and it does not need to.

You bring a bag. He looks at it, ties a tag to the handle with a number on it, tears off the matching half and hands it to you, and puts your bag somewhere. Then you go away and do whatever you came to do, and when you come back you give him the half-tag and he gives you the bag.

That is the whole thing. He does not ask what is in the bag. He does not open it. He would not know a laptop from a lunchbox and it makes no difference to him.

The rule that surprises people, usually students, is that you cannot get at part of your bag.

A boy came in one afternoon last month, about twenty minutes after leaving his rucksack, and said he needed the charger out of the front pocket. Bhaskar brought the whole bag out to the counter, the boy opened the pocket, took the charger, zipped it up, and Bhaskar tagged it again and took it back in. Two minutes. Perfectly possible, and note what actually happened: the entire bag came out and went back in. There is no arrangement under which the charger alone comes out and the bag stays where it is.

The other thing is the numbering, and this is the part that took Bhaskar a while to explain to the new boy who started in April.

The tags say things like `P1-14-233`. The new boy assumed that meant platform one, rack fourteen, slot two-thirty-three, and that he could find any bag by walking to that place. He was wrong. It is just a number. It is a long name that happens to have dashes in it, and where the bag actually sits is entirely up to Bhaskar and depends on how full the room was that morning. Some days he puts them on the high shelf, some days on the floor by the window. The tag identifies the bag. It does not describe a location.

The new boy found this very hard to accept for about a week, because the tags look so much like addresses.

What the room is genuinely excellent at is not losing things. In eleven years Bhaskar has lost one bag, in 2018, and that was because a man took the wrong one and brought it back the next day. The room is boring, the rules are few, and nothing has ever gone missing overnight.

3 The idea in plain English

Bhaskar's cloakroom is object storage, including the parts that annoy people.

An object is a whole thing with a name. You store bytes under a **key** and later fetch them by that key. The store does not look inside. A JPEG, a video, a database backup, a 2 GB log file — all the same to it.

A bucket is a namespace. All the keys in one bucket must be unique. That is the whole of the organisation.

There are no folders. The key `photos/2026/mumbai/img_4471.jpg` looks like a path, and it is not one — it is a single string that happens to contain slashes. **The namespace is flat.** The console draws folders by splitting on `/`, and the store itself has no concept

of them. That is the tag that looks like an address and is not, and it has real consequences: “rename a folder” is not an operation, it is a copy of every object with a new key followed by a delete of every old one.

You cannot modify part of an object. No seeking to byte 4,000 and writing. No appending a line. To change anything you upload the whole object again under the same key, which **replaces** it. That is the boy and his charger: the whole bag comes out and the whole bag goes back.

This is why it is not a file system, and why it is so durable. A file system has to support random writes, locks, permissions per directory, and metadata that changes constantly. Object storage supports “put a whole thing” and “get a whole thing”, and a system that only does those two operations can replicate aggressively, never worry about partial writes, and be spread across many machines without coordination.

Reads are cheap and enormous in parallel. Thousands of clients can fetch the same object at once; the store does not care. This is the property that makes it the right home for anything users download.

Durability is the headline number, and it is worth knowing. S3 advertises eleven nines — 99.99999999% — of durability for an object over a year. That means if you store ten million objects you would expect to lose one every ten thousand years. It is achieved by writing every object to multiple devices across multiple physically separated buildings before acknowledging the write.

Availability is a different and much lower number: about 99.99%, which is roughly an hour a year of not being able to reach it. **Durable means the data is not lost. Available means you can get it right now.** People conflate them constantly and they are different guarantees with different numbers.

And latency is the honest weakness. A first-byte read is tens of milliseconds, not the sub-millisecond of a database or a cache. It is built for throughput and durability, not for responsiveness. That is why anything user-facing sits behind a **CDN**, which you met on [day 70](#) — the object store is the origin and almost no read reaches it.

The two patterns that make it work in a web architecture:

Presigned URLs. Your server generates a URL that grants permission to write (or read) one specific key for a short time, and hands it to the browser. The browser uploads **directly to the store**. The bytes never touch your API servers. Without this, a 50 MB video upload occupies one of your web workers for the whole upload, and a hundred concurrent uploads is your entire fleet.

Store the reference, not the bytes. The database holds a row with the object's key, its size, its content type and who owns it. The bytes are in the store. The database is for things you query; the store is for things you fetch. **Mixing those two up is the mistake this lesson exists to prevent.**

4 The picture

The upload path, done badly and done properly:

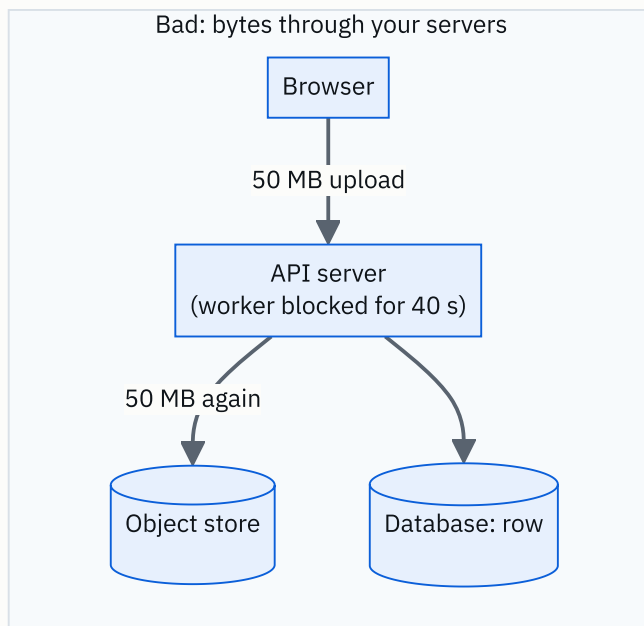


FIGURE 133.3

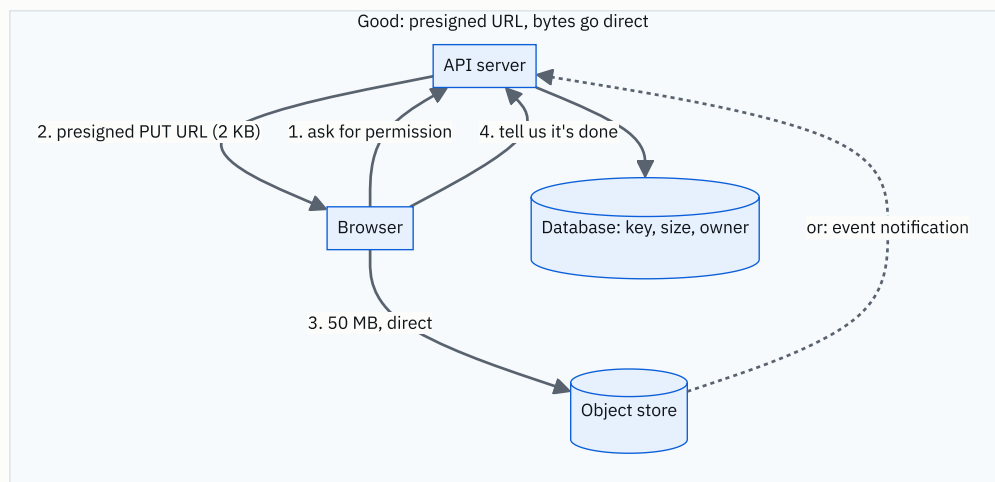


FIGURE 133.4

What to notice. In the good version your server handles two tiny requests and never sees a byte of the file. One web worker can serve thousands of uploads a minute instead of one at a time. The dotted arrow is the more robust variant: the store itself notifies you when the upload completes, so a browser that uploads successfully and then closes does not leave you with an object nobody knows about.

The read path, and where the traffic actually goes:

100,000 requests for the same image

WITHOUT a CDN

100,000 x GET from the object store

100,000 x \$0.0004/1000 = \$0.04 in requests

100,000 x 200 KB = 20 GB egress x \$0.09/GB = \$1.80

WITH a CDN (95% hit rate)

5,000 GETs reach the store

95,000 served from cache edges

egress from the store: 1 GB instead of 20

→ roughly 20x cheaper, and 10x faster for the user

What to notice. The saving is on egress, not on storage. Object storage is cheap to keep things in and expensive to move things out of, and that asymmetry drives most of the architecture around it.

And the thing that is not a folder:

keys in the bucket:

```
photos/2026/01/a.jpg
photos/2026/01/b.jpg
photos/2026/02/c.jpg
```

what the console shows

```
-----
photos/
 2026/
  01/
    a.jpg
    b.jpg
  02/
    c.jpg
```

what actually exists

```
-----
three strings.
That is all.
```

"list objects with prefix photos/2026/01/"
is a range scan over sorted keys,
not a directory read.

"rename photos/2026 to photos/old"
= copy every object to a new key, then delete every old one
= O(number of objects), not O(1)

5 How it actually works

The operations, and what is missing

PUT	bucket/key ← whole object	create or REPLACE
GET	bucket/key	whole object, or a byte range
DELETE	bucket/key	
HEAD	bucket/key	metadata only: size, type, etag
LIST	bucket?prefix=...	keys with a prefix, paginated, sorted
COPY	src → dst	server-side, no download

What is missing is the point: no append, no partial write, no move, no rename, no directory operations, no locking, no transactions across objects. Every one of those absences is what buys the durability and the scale.

GET with a **byte range** is worth knowing — you *can* read part of an object even though you cannot write part of one. That is what makes video seeking work and what lets a query engine read one column chunk out of a large Parquet file without downloading the whole thing.

Multipart upload

Large objects are uploaded in parts — typically 5 MB to 100 MB each — and assembled by the store.

1. initiate → upload id
2. upload part 1, part 2, ... part N (in parallel, each retryable on its own)
3. complete → the store assembles them into one object

Three benefits, and they are all practical: a failed part is retried alone instead of restarting a 5 GB upload; parts go in parallel so throughput is limited by your bandwidth rather than by one connection; and objects above 5 GB are only possible this way at all.

The trap: an abandoned multipart upload keeps its parts and you are billed for them, invisibly. They do not appear in a normal object listing. Every mature bucket has a lifecycle rule that aborts incomplete uploads after a few days, and forgetting it is a genuinely common source of a mysterious storage bill.

Consistency

S3 was famously eventually consistent for years: you could `PUT` an object and a subsequent `GET` might return the old version or a 404. Since December 2020 it provides **strong read-after-write consistency** for all operations, including listings.

Two things still deserve care. **Listing is still not a snapshot** — a `LIST` running while objects are being written may or may not include them, and paginating a large listing while it changes gives an inconsistent picture. And **other object stores are not S3**; if a design uses a different one, its consistency model is a question to ask rather than assume.

Storage classes and lifecycle

The same object can live at very different prices depending on how often you read it.

CLASS	COST/GB/MONTH	RETRIEVAL	USE FOR
Standard	~\$0.023	free, instant	active data
Infrequent Access	~\$0.0125	~\$0.01/GB, instant	monthly-ish access
Glacier Instant	~\$0.004	~\$0.03/GB, instant	quarterly access
Glacier Flexible	~\$0.0036	minutes to hours	archives
Deep Archive	~\$0.00099	up to 12 hours	compliance, 7-year retention

Deep Archive is roughly 23× cheaper than Standard, and the price of that is a retrieval measured in hours.

Lifecycle rules move objects automatically:

after 30 days	→ Infrequent Access
after 90 days	→ Glacier Flexible
after 365 days	→ Deep Archive
after 7 years	→ delete

Two traps. Infrequent Access has a 30-day minimum billing period and a 128 KB minimum object size, so moving a million tiny thumbnails there costs *more* than leaving them in Standard. And a transition itself is a per-object charge, so a lifecycle rule over

ten million small objects has a real one-off cost.

Versioning and deletes

With versioning on, a `PUT` to an existing key creates a new version and keeps the old one; a `DELETE` writes a **delete marker** rather than removing anything. That is genuine protection against an accidental overwrite or a bad deploy.

And it means “delete” does not reduce your bill. Old versions accumulate silently, and a bucket with versioning and no lifecycle rule on non-current versions grows forever. Pair versioning with “expire non-current versions after 30 days” every time.

Access control

- **Presigned URL** — a time-limited URL granting one operation on one key. Minutes for uploads, minutes to hours for downloads. This is how a private object is served to a browser without making the bucket public.
- **Bucket policy / IAM** — who may do what, at the identity level. The default should be that the bucket is private and only your application and the CDN can read it.
- **Public buckets** are how data breaches happen. Every major cloud now blocks public access by default, because the previous default caused years of incidents.

Event notifications

The store can emit an event when an object is created or deleted, to a queue, a topic, or a function. That is the robust way to trigger post-processing:

```
upload completes → event → queue → worker resizes the image
```

Better than trusting the browser to tell you, because the browser may close, lose its connection, or lie. **Design the pipeline off the store’s own event, and treat the client’s “I’m done” as an optimisation, not as the source of truth.**

The real products

S3 is the reference. **Google Cloud Storage** and **Azure Blob Storage** are equivalent with different names — Azure calls buckets “containers” and has a genuine append blob type. **MinIO** is S3-compatible and self-hosted. **Cloudflare R2** is S3-compatible with **zero egress fees**, which changes the arithmetic for anything download-heavy. **Ceph** is the open-source system underneath many private clouds.

“S3-compatible API” has become the standard, so designs usually say “object storage” and mean any of these.

6 The numbers

Durability and availability, stated separately:

```
durability    99.999999999%    (11 nines)
              10,000,000 objects → expect to lose 1 every 10,000 years

availability  99.99%           (4 nines)
              ~52 minutes of unreachability per year
```

Latency:

```
time to first byte      20 - 100 ms
throughput per connection ~ 100 MB/s
throughput with 10 parallel parts ~ 1 GB/s
vs a database row read   0.5 - 2 ms
vs a Redis read          0.2 ms
```

Object storage is 50-200× slower to first byte than a cache. That is why the CDN is not optional for user-facing media.

Cost, and the shape of it:

```
storage          $0.023 per GB per month
PUT/POST/LIST     $0.005 per 1,000 requests
GET              $0.0004 per 1,000 requests
egress to internet $0.09 per GB ← the expensive one
egress to a CDN   often $0.00 - 0.02
```

Worked example — a photo-sharing app. One million users, 20 photos each, 2 MB average:

```
objects      1,000,000 x 20      = 20,000,000
storage      20,000,000 x 2 MB    = 40 TB
                                     = 40,000 GB x $0.023 = $920/month
```

Now the reads. Ten million photo views a day:

```
requests      10,000,000 x 30 days = 300,000,000 GETs
               300,000 x $0.0004    = $120/month
egress        300,000,000 x 2 MB    = 600 TB
               600,000 GB x $0.09   = $54,000/month
```

Fifty-four thousand dollars of egress against nine hundred of storage. Storage is a rounding error and egress is the entire bill — which is the single most important cost fact about object storage.

With a CDN at a 95% hit rate:

origin egress	600 TB x 0.05 = 30 TB	x \$0.09 = \$2,700
CDN delivery	600 TB	x ~\$0.02-0.08 = \$12,000 - 48,000

Still large, and this is why an image-heavy product cares about thumbnail sizes: halving the average delivered size halves the biggest line on the bill.

Object storage against a database, for the same 40 TB:

object storage	40,000 GB x \$0.023	= \$920/month
managed Postgres	40,000 GB x \$0.115 (gp3 SSD)	= \$4,600/month storage alone
	+ the instance to serve it	
	+ backups of 40 TB	
	+ a restore that now takes hours instead of minutes	

Five times the storage cost before counting anything else, and that is the cheap part of the argument.

Multipart sizing:

5 GB file, 100 MB parts	= 50 parts
sequential at 100 MB/s	= 50 s
10 parallel at 100 MB/s ea.	= ~5 s

Small-object economics, which catches people:

1,000,000 objects of 10 KB each	
storage 10 GB x \$0.023	= \$0.23/month
requests 1,000,000 PUTs x \$0.005/1k	= \$5.00 one-off

The requests cost twenty times more than a month of storage. For very small objects, the per-request charge dominates, and the fix is to batch them into larger objects — which is exactly why data lakes store Parquet files of hundreds of megabytes rather than a file per row.

7 The trade-offs

You give up random writes, and that is the deal. No append, no in-place edit, no locking. Anything that needs to change part of a stored item — an actively edited document, a database file, a log being written — is the wrong shape for object storage. Rewriting a 1 GB object to change one byte is a 1 GB write.

You give up low latency. Tens of milliseconds to first byte, against sub-millisecond for a cache. Fine for a file a user downloads; wrong for something on the critical path of every request. The mitigation is caching at every level, and if that does not help, this is

the wrong store.

You give up querying. You can list by prefix and that is all. “Every photo tagged Mumbai taken last March” is not a question object storage can answer — the metadata lives in a database that holds the keys, and keeping those two in step is your problem. **An object with no database row is invisible; a row with no object is a broken link.** Both happen, and a reconciliation job that finds them is part of a serious design.

Egress is the bill. Storing is cheap, reading out is expensive, and a design that moves the same bytes repeatedly across a network boundary will cost more than anyone estimated. This is what makes CDNs non-negotiable for media, and it is what makes a zero-egress provider genuinely change the arithmetic.

Eleven nines of durability is not a backup. Durability protects against hardware failure. It does not protect against your own code deleting the wrong prefix, or a compromised credential. Versioning, a lifecycle policy, and replication to a separate account or region are what protect against that, and they are separate decisions with separate costs.

Small objects are inefficient. Per-request charges and per-object overhead mean that millions of tiny objects cost more in requests than in storage, and listing them is slow. Batch where you can.

When would I not use object storage? When the data is small, structured, and queried — that is a database. When it changes constantly in place — that is a file system or a database. When latency must be single-digit milliseconds and caching cannot cover it. And when you need a real file-system interface for legacy software, where a network file system is the honest answer even though it is more expensive and less durable.

8 In the interview

How it gets asked

- “Where do the uploaded images actually go?” — the direct version.
- “Should the images go in the database? Defend your answer.” — [tomorrow’s](#) question, and this lesson is the ammunition.
- “How does a user upload a 2 GB video without killing your servers?”
- “How do you serve a private file to a browser?”
- “Your storage bill is enormous and you only have 40 TB. Why?”
- “What happens if the upload succeeds and your database write fails?”

The first ninety seconds

“Object storage — S3 or an equivalent — and the database holds only a reference: the key, the size, the content type and the owner. The bytes never go in the database.

The mental model is a cloakroom rather than a file system. You put in a whole object under a name and later you get the whole object back. **There are no folders** — a key like `photos/2026/img.jpg` is one flat string that happens to contain slashes, so ‘rename a folder’ means copying every object and deleting the originals. And **you cannot modify part of an object**: changing anything means uploading the whole thing again.

Those restrictions are what buy the properties. Eleven nines of durability, effectively unlimited capacity, about two cents per gigabyte per month, and unlimited parallel reads.

The upload path is the part I would design carefully. The browser asks my API for permission; the API returns a presigned URL valid for a few minutes for one specific key; the browser uploads **directly to the store**. The bytes never touch my servers. Without that, a 50 MB upload occupies one web worker for the whole transfer, and a hundred concurrent uploads is my entire fleet doing nothing but copying bytes.

For large files, multipart upload — parts of 100 MB, uploaded in parallel, each retried independently, so a dropped connection at 4 GB does not restart the whole thing. And a lifecycle rule to abort incomplete uploads after a few days, because abandoned parts are billed and do not show up in a listing.

On the read side, a CDN in front, always. First-byte latency from the store is tens of milliseconds, and egress is by far the largest cost — for a photo app at ten million views a day, egress is about fifty-four thousand dollars a month against nine hundred for storage. A CDN at a 95% hit rate cuts the origin egress twentyfold and makes it faster.

Do you want me to go into the consistency between the store and the database, or the cost model?”

The follow-ups

“The upload succeeded and your database write failed. Now what?”

“I have an object nobody knows about — an orphan. It costs money, it is invisible to the application, and nothing will ever delete it.

Three defences, and I would use all three.

Order the operations so the failure is recoverable. Write the database row *first*, in a `PENDING` state, before handing out the presigned URL. Then an upload with no corresponding row cannot happen, because the row existed first. The failure mode inverts: I can have a row with no object, which is a broken link I can detect and clean up, rather than an object with no row, which is invisible.

Drive the confirmation from the store’s own event, not from the client. The store emits an object-created event to a queue, and a worker moves the row from `PENDING` to `READY`. A browser that uploads successfully and then closes, or lies, or has its network drop, does not leave the system inconsistent.

And a reconciliation job, because everything above is best-effort. Nightly, list the bucket and compare with the table: objects with no row get deleted after a grace period, rows in `PENDING` for more than a day get cleaned up. **That job is where the real correctness lives** — everything else is there to keep its workload small — and it is the same argument as reconciliation in a payment system.”

“How do you serve a private file to a browser?”

“Presigned URL, and the bucket stays private.

The browser asks my API for the file. The API checks authorisation — is this user allowed to see this object — and if so generates a URL that grants a `GET` on that one key, signed with my credentials and valid for a short window. The browser follows it and fetches directly from the store. My server never proxies the bytes.

Two things I would get right. The expiry should be short — minutes, not days — because a presigned URL is a bearer token: anyone who has it can use it until it expires, and they end up in browser history, in chat messages, and in logs. And the authorisation check happens when the URL is *generated*, so revoking a user’s access does not invalidate URLs already issued. That is a real gap and the mitigation is short expiry.

If the content also needs a CDN — private videos, for instance — the presigned URL alone is not enough, because the CDN would cache the response and serve it to the next person. The answer is the CDN’s own signed URLs or signed cookies, with the origin locked to the CDN only. That keeps the caching benefit and keeps the authorisation.

The alternative — proxying every download through my application — gives me finer control and per-request revocation, and costs me the bandwidth and the workers. I would only do it when there is a hard requirement to log or authorise every single byte.”

“Our storage bill is enormous and we only have 40 terabytes.”

“Then it is almost certainly not storage. Forty terabytes at about two cents a gigabyte is around nine hundred dollars a month. If the bill is far larger, the money is going to one of four places and I would check them in this order.

Egress. Nine cents a gigabyte to the internet. Ten million image views a day at two megabytes each is six hundred terabytes a month — fifty-four thousand dollars. This is almost always the answer, and the fixes are a CDN in front, smaller delivered images, and checking whether anything is repeatedly pulling the same objects across a region boundary.

Old versions. If versioning is on and there is no lifecycle rule on non-current versions, every overwrite keeps the previous copy forever. A bucket that looks like 40 TB in the console can be holding 300 TB of history, because the console shows current versions by default.

Abandoned multipart uploads. Parts from failed uploads are billed and do not appear in a listing at all. On a system with flaky mobile uploads this accumulates steadily and is invisible until you specifically look.

Requests, if the objects are small. A million ten-kilobyte objects is twenty-three cents of storage and five dollars of PUT requests. At a billion small objects the request charges dominate completely, and the fix is to batch them into larger objects.

The general shape of the answer is: **object storage is cheap to fill and expensive to read out of, and the bill almost never comes from the thing the question names.**“

“Would you use object storage for a database’s data files?”

“Not directly for the hot path, and the reason is the two restrictions.

A database needs random writes — update a page in place, append to a write-ahead log — and object storage gives me neither. Rewriting a whole object to change one page is absurd. And it needs sub-millisecond reads where the store gives me tens of milliseconds.

But the honest 2026 answer is that **separated storage and compute do exactly this**, with a layer in between. Snowflake, BigQuery, Databricks and increasingly the newer transactional systems keep their data as immutable objects in object storage and put a local SSD cache and a write-ahead log on fast storage in front. The trick is that the objects are **immutable** — written once, never edited — and a change writes a new object and updates a small piece of metadata. Table formats like Iceberg and Delta Lake are exactly that metadata layer.

So the accurate statement is: object storage is wrong for data that changes in place, and right for immutable data files with a mutable index elsewhere. That is why analytical systems live on it happily and why an OLTP database’s hot pages do not.

And for backups, snapshots and archives, it is obviously right — write once, read rarely, keep for years, and tier to Deep Archive at a twenty-third of the price.”

The model answer

“Design the media storage for a product where users upload photos and videos, view them constantly, and the company keeps everything forever.”

“Let me set the split first, because everything follows from it.

The database holds metadata; the object store holds bytes. A row per media item: id, owner, object key, size, content type, dimensions, upload time, state. Every question the product asks — ‘this user’s photos in date order’, ‘everything tagged Mumbai’ — is a database query, and the answer includes keys that the client then fetches. **The store is not queryable and I would not pretend otherwise.**

The upload path. The client asks the API for an upload slot. The API creates the row in `PENDING`, generates a key — a UUID, not the user’s filename, so nothing collides and nothing is guessable — and returns a presigned URL valid for fifteen minutes. Videos use multipart with 100 MB parts in parallel, so a phone on a bad connection retries one part rather than a 2 GB file. **No bytes touch my servers**, which means the API fleet is sized for requests per second and not for megabits.

Completion is driven by the store’s event, not by the client. Object-created fires to a queue; a worker validates the object — size, type, and that it really is an image and not something renamed — moves the row to `READY`, and enqueues derivative work. The client’s ‘done’ call is an optimisation for responsiveness, not the source of truth, because a browser that closes must not leave the system inconsistent.

Derivatives are separate objects. Thumbnails at a few sizes, video transcodes at several bitrates, each under a deterministic key derived from the original — so re-running the resize is idempotent, it overwrites with identical bytes, and at-least-once delivery is harmless.

The read path is a CDN in front, with the bucket private. Public content is served from the CDN with a long cache lifetime and immutable keys, so I never have to invalidate — a new version gets a new key. Private content uses CDN signed URLs with a short expiry, and the origin is locked to the CDN alone.

The numbers that justify the CDN. Twenty million objects at 2 MB is 40 TB, about nine hundred dollars a month to store. Ten million views a day at 2 MB is 600 TB of egress a month — fifty-four thousand dollars at origin prices. A 95% CDN hit rate takes origin egress to 30 TB. **Egress is sixty times the storage cost, so every decision about delivered image size is a decision about the bill**, and I would push hard on serving appropriately sized derivatives rather than originals.

Lifecycle, because ‘keep everything forever’ has a cost. Originals move to Infrequent Access after 30 days and Glacier Flexible after a year — a user’s photo from 2019 is fetched rarely, and if it is, a few hundred milliseconds extra is acceptable. Thumbnails stay in Standard forever because they are small and hot. **I would not tier the thumbnails**, because Infrequent Access has a 128 KB minimum billed size and moving millions of 20 KB thumbnails there costs more than leaving them.

Versioning on, with non-current versions expiring after 30 days, so a bad deploy that overwrites derivatives is recoverable without paying for history forever. And a lifecycle rule aborting incomplete multipart uploads after three days, because mobile video uploads fail constantly and the orphaned parts are billed silently.

The reconciliation job, nightly: objects with no row, rows with no object, and rows stuck in `PENDING`. That is the only thing in the design with a view of both sides, and without it the two drift apart quietly over years.

What I would flag as a real risk: a delete in this system is a delete from the database, a delete from the store, a delete of every derivative, and a CDN invalidation — four systems, no transaction across them. I would make it a soft delete in the database plus an asynchronous cleanup driven from a queue, so that ‘the user cannot see it’ happens instantly and ‘the bytes are gone’ happens eventually and idempotently. And if there is a regulatory deletion requirement, that asynchronous job needs its own monitoring, because ‘we deleted it eventually, probably’ is not an acceptable answer to a regulator.”

9 Recall card

Object storage is a cloakroom, not a file system. Whole objects under a flat key namespace — the slashes in `photos/2026/a.jpg` are just characters. **No append, no partial write, no rename;** changing anything means rewriting the whole object.

Durability 11 nines; availability 4 nines; first byte in 20–100 ms. Durable ≠ available, and it is not a backup — versioning and cross-account replication protect against *your own* mistakes.

Presigned URLs so bytes never touch your servers; multipart for large files; a CDN in front, always. Drive post-upload work from the store’s own event, not from the client.

Storage is cheap and egress is the bill: ~\$0.023/GB/month to store, \$0.09/GB to send. A photo app at 40 TB pays ~\$900 for storage and ~\$54,000 for egress before a CDN.

Database holds the reference, store holds the bytes — and a reconciliation job finds the orphaned objects and the broken links, because there is no transaction across the two.

DSA topic: Cycle detection in a directed graph **System design topic:** Object storage, S3-style

Code these, in this order

One rule for the whole set: **your first test is the diamond** — `0→1` , `0→2` , `1→3` , `2→3` — and it must return “no cycle”. Write that assertion before the function. It is the only test that catches a two-state implementation, and every trivial test passes without it.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Course Schedule	LeetCode 207 (Medium)	Three colours or Kahn’s, and the outer loop.
2	Course Schedule II	LeetCode 210 (Medium)	The same pass, returning the order — why Kahn’s wins.
3	Find Eventual Safe States	LeetCode 802 (Medium)	Colours again, but the answer is “not in or leading to a cycle”.
4	Longest Cycle in a Graph	LeetCode 2360 (Hard)	Each vertex has one outgoing edge; the path index is the trick.
5	Minimum Height Trees	LeetCode 310 (Medium)	Kahn’s on an undirected graph — peeling leaves, not sources.

On problem 1, write both algorithms and time them

Three-colour DFS and Kahn’s, on the largest LeetCode input. Record both times and both line counts. Then say which you would write in an interview and why — and the answer should mention problem 2.

On problem 1, deliberately write the two-state version

Run it on the diamond. Record the answer. Then run the correct version on the same input. One sentence on what “seen” failed to distinguish.

On problem 3, notice the difference

“Safe” is not “not in a cycle” — it is “cannot reach a cycle”. Write down what colour a safe vertex ends up being, and why the black/grey distinction gives you the answer with no extra work.

On problem 5, notice what changed

Kahn's here peels vertices of degree **one**, not in-degree zero, and the graph is undirected. Write one sentence on why the same idea transfers and what had to change.

Then the depth experiment

Build a directed chain of 100,000 vertices — `0→1→2→...` — and run:

1. Recursive three-colour DFS.
2. The same after `sys.setrecursionlimit(2_000_000)`.
3. The iterative version with stored iterators.
4. Kahn's.

Four results. Two are crashes and the difference between the crashes is the point.

Then the unreachable-cycle experiment

Build `0→1` plus a triangle among 3, 4, 5. Run a version that starts only at vertex 0, and the version with the outer loop. Two answers, and only one is right.

The colours drill

1. Name the three states and say what each means, in one line.
2. Say which one is the cycle test and why.
3. Say where `BLACK` is assigned and what moving it before the loop does.
4. Give the two-set formulation and name the line people forget.
5. Walk the diamond out loud, step by step, saying the colours.

The two-algorithms drill

1. Write the three-colour DFS from memory.
2. Write Kahn's from memory.
3. Say what `output  $\neq$  n` means, in words.
4. Give three reasons to prefer Kahn's and one reason to prefer DFS.
5. Say why Kahn's leftovers are not exactly "the vertices in a cycle".

The undirected-comparison drill

1. Say why the parent check does nothing on a directed graph.
2. Give a two-vertex directed cycle and say what the parent check reports.
3. Say what the three-colour scheme reports on an undirected graph, and why.
4. State the one question each algorithm is really asking.

The follow-ups drill

1. Return the cycle itself — say what structure you keep and how the slice works.
2. Say what it costs and how to make the lookup constant.
3. Say what you would use to find *all* the deadlocks precisely, and why Kahn's leftovers over-report.
4. Say how you would get a parallel build schedule out of Kahn's.

The costs drill

1. Derive $O(V + E)$ and say why it is E and not $2E$ here.
2. Say what the worst case is and why an acyclic graph is the expensive one.
3. Give Python's usable recursion depth and the two failure modes past it.
4. Compare memory for a colour array against two Python sets at $V = 10^6$.

The break-it drill

Trigger each and record the exact output or error:

1. Two states, on the diamond.
2. `on_path.remove` deleted.
3. `BLACK` assigned before the loop, on a triangle.
4. Starting only from vertex 0, with the cycle at 3–5.
5. The undirected parent check on `0→1`, `1→0`.
6. Recursion on a 100,000-vertex chain, before and after raising the limit.
7. Kahn's on a graph built from edges only, with an isolated vertex.

The model drill

1. Say what an object, a key and a bucket are.
2. Say what `photos/2026/a.jpg` actually is, and what “rename the folder” costs.
3. Name the operations object storage supports and the four it does not.
4. Say why `GET` with a byte range exists when partial writes do not.
5. Say why the restrictions are what buy the durability.

The numbers drill

1. Quote durability and availability separately and say what each means.
2. Give time-to-first-byte and compare with a database read and a cache read.
3. Give the four cost components and say which one dominates.
4. Compute storage and egress for 20M objects at 2 MB with 10M views a day.

5. Recompute origin egress at a 95% CDN hit rate.
6. Compare 40 TB in object storage against 40 TB on managed SSD.
7. Compute the request cost for a million 10 KB objects and compare it with a month of storage.

The upload-path drill

1. Draw the bad upload path and say what it costs per worker.
2. Draw the presigned-URL path, all four steps.
3. Say why the completion signal should come from the store, not the client.
4. Give the three benefits of multipart upload.
5. Name the multipart trap and the lifecycle rule that fixes it.

The consistency drill

1. Say what S3's consistency guarantee is today and what it was before 2020.
2. Say what is still not a snapshot.
3. Say what happens if the upload succeeds and the database write fails.
4. Give the three defences, in order.
5. Say which of the three is where the real correctness lives.

The lifecycle drill

1. Name the storage classes and the price ratio between the cheapest and the most expensive.
2. Write a lifecycle policy for photos kept forever.
3. Give the two traps that make tiering small objects cost *more*.
4. Say what versioning protects against and what it costs.
5. Say what pairs with versioning, always.

The access drill

1. Say how a presigned URL works and what it is, security-wise.
2. Give two things to get right about expiry and revocation.
3. Say why a presigned URL alone does not work behind a CDN, and what does.
4. Say what the alternative is and when you would accept its cost.

The trade-offs drill

1. Say what you give up by choosing object storage — four things.
2. Say why 11 nines of durability is not a backup.
3. Say why small objects are inefficient and what to do instead.

4. Give the accurate statement about databases on object storage.
5. Name four cases where object storage is the wrong choice.

The failure drill

For each, say what happens and what you would build:

1. The object uploads and the database write fails.
2. The browser closes immediately after a successful upload.
3. Versioning is on and there is no lifecycle rule on old versions.
4. A mobile app's video uploads fail 30% of the time, for a year.
5. A presigned download URL is pasted into a group chat.
6. A million 20 KB thumbnails are moved to Infrequent Access.
7. A user requests deletion and the derivative objects are missed.

Two of the seven produce a bill rather than an outage. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Does this directed graph contain a cycle?* Three colours with grey as the current path, the diamond as the case two states get wrong, the outer loop over all vertices, and Kahn's offered as the version that answers the follow-up too.
 2. *Can all courses be finished? Now give me an order.* Kahn's in one pass, `output  $\neq$  n` as the cycle test, the emitted order as the schedule, and the level-by-level variant as a parallel build plan.
 3. *Where do the uploaded images actually go?* Object store for bytes and database for the reference, presigned URLs so no bytes touch your servers, multipart for large files, a CDN in front — with the egress-versus-storage numbers as the reason.
-

Before you move on

- [] My first test is the diamond, and it returns “no cycle”.
- [] I can name the three colours and say what grey means.
- [] I know where `BLACK` is assigned and what moving it does.
- [] I know the missing line in the two-set formulation.
- [] I can walk the diamond aloud with the colours.

- [] I can write Kahn's from memory and explain `output ≠ n`.
- [] I can give three reasons to prefer Kahn's.
- [] I know why Kahn's leftovers over-report the cycle.
- [] I can return the cycle itself using the path list.
- [] I know why the undirected parent check fails here, in both directions.
- [] I always loop over every vertex.
- [] I build from `range(n)`.
- [] I know the recursion limit and both failure modes past it.
- [] I can compare array and set memory at a million vertices.
- [] I know object storage has no folders and no partial writes.
- [] I know what "rename a folder" actually costs.
- [] I can quote durability and availability separately.
- [] I know first-byte latency and how it compares with a cache.
- [] I can compute storage and egress and say which dominates.
- [] I can draw the presigned-URL upload path.
- [] I know why the completion signal comes from the store.
- [] I know the three benefits of multipart and its billing trap.
- [] I know what to do when the upload succeeds and the database write fails.
- [] I know reconciliation is where the real correctness lives.
- [] I can write a lifecycle policy and name the small-object traps.
- [] I know what versioning protects against and what it must be paired with.
- [] I know why a presigned URL alone fails behind a CDN.
- [] I know why 11 nines is not a backup.
- [] I can name four cases where object storage is wrong.
- [] I answered all three questions above out loud.

Topological sort

DATA STRUCTURES AND ALGORITHMS

Topological sort

SYSTEM DESIGN

Blob storage versus storing files in the database

1 What this is, and why they ask it

A topological sort is an ordering of a directed graph's vertices such that every edge points forwards. If A must happen before B, then A appears before B in the list. That is the whole definition.

It exists only for a **DAG** — a directed acyclic graph. If there is a cycle, no valid order exists, which is exactly [yesterday's](#) lesson from the other side: the same algorithm that produces the order tells you when there is none.

There are two ways to do it and you should know both. **Kahn's algorithm** repeatedly takes a vertex that nothing is waiting on, which reads like the domain and gives the cycle check for free. **DFS finish order, reversed** falls out of a depth-first search and is three lines shorter. Same complexity, different outputs when you need more than a list.

They ask it because dependency ordering is everywhere and because the questions built on it are among the most common medium-difficulty interview problems: course schedules, build systems, package installation, task runners, spreadsheet recalculation, compiler passes. **The tell is any phrasing of “in what order”, where some things must come before others** — and the candidate's job is to notice that before writing anything.

By the end of this lesson you can produce an order with both algorithms, say which to use and why, count or enumerate the valid orders, produce a *parallel* schedule rather than a serial one, and handle the follow-up about lexicographically smallest order that most people fumble.

2 The story

Nirmala is cooking for fourteen people on Sunday and it is Saturday evening.

The list in her head is not a list. It is a set of things that have to happen and a set of facts about which of them cannot happen until something else has.

The rice has to soak. Not for very long, but it has to happen before the rice can be cooked, and the rice has to be cooked before it can be sat in the vessel keeping warm. Three things, and their order is not negotiable.

The dough for the pooris needs to rest for at least an hour, so the dough has to be made an hour before anyone starts rolling. The paneer has to be pressed before it can be cut and the cutting has to happen before it goes in the gravy.

And then a whole set of things that have no order at all. The onions can be chopped whenever. The coriander can be washed whenever. The plates can come down from the top shelf on Saturday night or at half past seven on Sunday and it makes no difference to anything.

So on Saturday evening she does the things that are furthest back in the chains and that nothing is waiting on — the paneer gets pressed under a plate with two water bottles on it and left overnight, because nothing has to happen before that and everything else about the paneer has to happen after.

On Sunday morning at nine she puts the rice to soak, not because it needs four hours but because it is the first link in that chain and she does not want to be waiting for it at one o'clock. Then she makes the dough, because that is a one-hour clock that has to start early. Both of those are things nothing else is waiting for, and both start clocks that other things depend on.

Then, with those two running, she does the chopping and the washing and all the free-floating things, because the point of starting the slow chains first is that everything else can happen while they run.

Her sister asked her once whether she had a system, and Nirmala said no, but what she does is completely consistent, and it is this: **look at everything that is not waiting on something else, start whichever of those has the longest tail behind it, and repeat.**

The meal was on the table at ten past one. She had said one o'clock.

3 The idea in plain English

Nirmala's method is Kahn's algorithm, and her sister's question is the interview question.

A directed edge means “must come first”. An edge from *soak rice* to *cook rice* means the soaking has to be done before the cooking can start. The direction is the whole meaning, and getting it backwards produces a valid-looking order that is exactly wrong.

A topological order is any sequence where every edge points forwards. Every task appears after everything it depends on. **There is usually more than one valid order**, and that is important: Nirmala could have chopped the onions first or last, and the meal is the same. **A topological sort produces *an* order, not *the* order.**

It exists if and only if there are no cycles. A cycle means each of a set of tasks waits for another in the set, and no ordering can satisfy all of them. That is why the algorithm doubles as a cycle detector.

In-degree is “how many things am I still waiting for”. Count the arrows pointing at a vertex. Rice soaking has in-degree zero — nothing has to happen first. Rice cooking has in-degree one, waiting on the soaking.

Kahn’s algorithm is exactly what Nirmala does:

1. Compute every vertex’s in-degree.
2. Put everything with in-degree zero into a queue — these can start now.
3. Take one out, output it, and for everything it points at, decrement that in-degree. If any reaches zero, it can start now, so queue it.
4. Repeat until the queue is empty.

If you have output fewer than all the vertices, the rest are in a cycle. They never reached in-degree zero, which means each was permanently waiting on something. **The same pass answers “is there an order?” and “what is it?”**, which is why it is usually the right choice.

The second algorithm is DFS finish order, reversed. Run a depth-first search; whenever a vertex *finishes* — everything below it is done — push it onto a list. Reverse the list at the end.

The reason that works is worth having: when a vertex finishes, everything it depends on... no, everything that depends on *it* has already finished, because you reached them through it. So finishing order is dependency order backwards, and reversing it puts each vertex before everything it points to. **You met the finish event on [day 128](#)**; this is what it was for.

A DFS version needs the three-colour cycle check bolted on, because unlike Kahn’s it will happily produce a nonsense order on a cyclic graph rather than noticing.

Now the variant that is usually what people actually want: the parallel schedule. Kahn’s queue at any moment holds everything that can start *right now*, so if you process the queue **one whole level at a time**, each level is a set of tasks that can run simultaneously. The number of levels is the **critical path** — the minimum time to finish everything if you had unlimited workers. **A build system that gives you a serial order is much less useful than one that gives you the levels**, and it is a two-line change.

And the follow-up that catches people: the lexicographically smallest order. If several vertices have in-degree zero, Kahn’s picks arbitrarily. Replace the queue with a **min-heap** and it always picks the smallest available vertex, giving the smallest valid order. That is $O((V + E) \log V)$ instead of $O(V + E)$, and the reason a greedy choice is correct here is worth one sentence: taking the smallest available vertex never removes a smaller option later, because anything smaller that was available stays available.

4

Nirmala's Sunday, as a DAG:

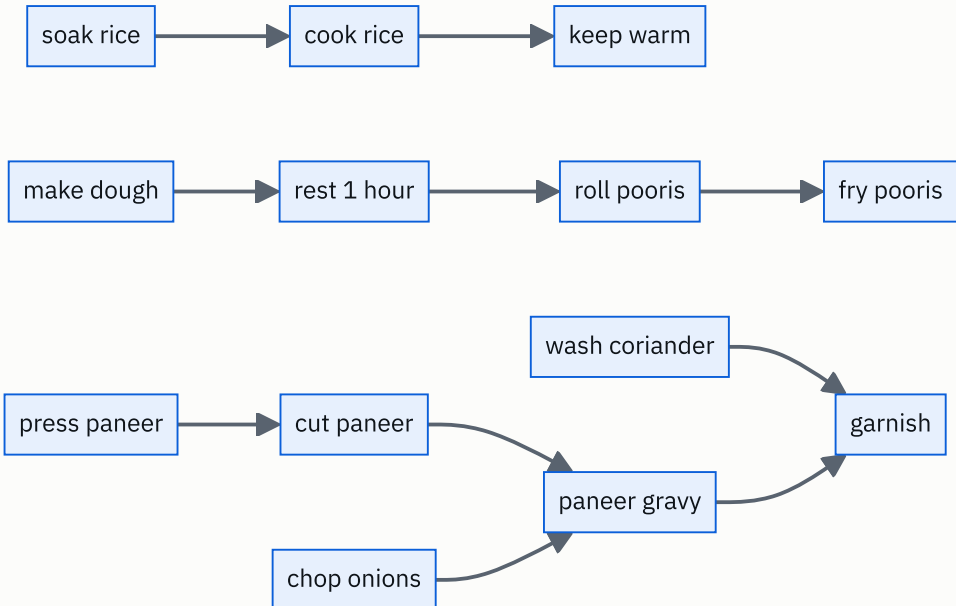


FIGURE 134.1

What to notice. `press paneer`, `soak rice`, `make dough`, `chop onions` and `wash coriander` all have in-degree zero — nothing is waiting on any of them, so any of them can be first. That is five valid starting points and therefore many valid orders.

Kahn's traced on a smaller graph:

graph: $0 \rightarrow 2, 1 \rightarrow 2, 2 \rightarrow 3, 2 \rightarrow 4, 3 \rightarrow 5, 4 \rightarrow 5$

```
in-degree:  0:0   1:0   2:2   3:1   4:1   5:2
```

step	queue	take	output	decrement	new zeros
1	[0, 1]	0	[0]	$2 \rightarrow 1$	—
2	[1]	1	[0, 1]	$2 \rightarrow 0$	2
3	[2]	2	[0, 1, 2]	$3 \rightarrow 0, 4 \rightarrow 0$	3, 4
4	[3, 4]	3	[0, 1, 2, 3]	$5 \rightarrow 1$	—
5	[4]	4	[0, 1, 2, 3, 4]	$5 \rightarrow 0$	5
6	[5]	5	[0, 1, 2, 3, 4, 5]	—	—

output 6 of 6 → no cycle, and this is a valid order

What to notice at step 3. After taking 2, both 3 and 4 reach zero and both enter the queue. They are independent and either could come next — which is where the “many valid orders” comes from.

And the same graph as a **parallel schedule**, which is the same algorithm read differently:

```
level 0:  [0, 1]      both can start immediately
level 1:  [2]         needs both of level 0
level 2:  [3, 4]      both need 2, independent of each other
level 3:  [5]         needs both of level 2

serial order:  6 steps
parallel levels: 4 steps

with unlimited workers, the job takes 4 units, not 6.
The number of LEVELS is the critical path.
```

What to notice. The serial order [0,1,2,3,4,5] hides the fact that 0 and 1 could have run together, and so could 3 and 4. **The level structure is strictly more information for the same work**, and it is what a build tool actually needs.

The cyclic case, for contrast:

```
graph:  0 → 1,  1 → 2,  2 → 0

in-degree:  0:1  1:1  2:1
vertices with in-degree 0: NONE
queue starts EMPTY → output 0 of 3 → cycle
```

5 The code, built step by step

Kahn’s first, because it is the one to reach for.

```
from collections import deque

def topological_sort(graph: dict[int, list[int]], n: int) → list[int] | None:
    """A valid order, or None if there is a cycle."""
    in_degree = [0] * n
    for vertex in range(n):
        for neighbour in graph[vertex]:
            in_degree[neighbour] += 1
```

One pass over every edge to build the counts. Note the loop is over `range(n)` and not over `graph`, so a vertex with no outgoing edges is still considered — the isolated-vertex bug from [day 129](#), which here is fatal rather than merely wrong, because `len(order) == n` will never hold if some vertices are missing from the structure.

```

queue = deque(v for v in range(n) if in_degree[v] == 0)
order: list[int] = []
while queue:
    vertex = queue.popleft()
    order.append(vertex)
    for neighbour in graph[vertex]:
        in_degree[neighbour] -= 1
        if in_degree[neighbour] == 0:
            queue.append(neighbour)
return order if len(order) == n else None

```

Read the decrement: taking `vertex` out means everything it points at is now waiting for one fewer thing. A neighbour that hits zero has nothing left to wait for, so it can start.

`len(order) == n` is the cycle test, and returning `None` rather than a partial list makes the caller handle it. A partial order is worse than no order, because it looks usable.

Now the parallel version, which is a two-line change:

```

def topological_levels(graph: dict[int, list[int]], n: int) → list[list[int]] | None:
    """Groups of vertices that can run simultaneously. len(levels) is the critical path."""
    in_degree = [0] * n
    for vertex in range(n):
        for neighbour in graph[vertex]:
            in_degree[neighbour] += 1

    frontier = [v for v in range(n) if in_degree[v] == 0]
    levels: list[list[int]] = []
    done = 0
    while frontier:
        levels.append(frontier)
        done += len(frontier)
        nxt: list[int] = []
        for vertex in frontier:
            for neighbour in graph[vertex]:
                in_degree[neighbour] -= 1
                if in_degree[neighbour] == 0:
                    nxt.append(neighbour)
        frontier = nxt
    return levels if done == n else None

```

No queue at all — two lists, swapped at the end of each round, exactly like the level-by-level BFS from [day 130](#). Everything in `frontier` is ready at the same moment, so it is one level.

The lexicographically smallest order, which is the standard follow-up:

```

import heapq

def smallest_topological_order(graph: dict[int, list[int]], n: int) → list[int] | None:
    in_degree = [0] * n
    for vertex in range(n):
        for neighbour in graph[vertex]:
            in_degree[neighbour] += 1
    heap = [v for v in range(n) if in_degree[v] == 0]
    heapq.heapify(heap)
    order: list[int] = []
    while heap:
        vertex = heapq.heappop(heap)
        order.append(vertex)
        for neighbour in graph[vertex]:
            in_degree[neighbour] -= 1
            if in_degree[neighbour] == 0:
                heapq.heappush(heap, neighbour)
    return order if len(order) == n else None

```

One data structure changed. The greedy choice is correct because taking the smallest currently-available vertex cannot make a smaller vertex unavailable later — anything already available stays available.

Now the DFS version, for completeness:

```

WHITE, GREY, BLACK = 0, 1, 2

def topological_sort_dfs(graph: dict[int, list[int]], n: int) → list[int] | None:
    colour = [WHITE] * n
    order: list[int] = []

    def visit(vertex: int) → bool:
        colour[vertex] = GREY
        for neighbour in graph[vertex]:
            if colour[neighbour] == GREY:
                return False
            if colour[neighbour] == WHITE and not visit(neighbour):
                return False
        colour[vertex] = BLACK
        order.append(vertex)
        return True

    for vertex in range(n):
        if colour[vertex] == WHITE and not visit(vertex):
            return None
    return order[::-1]

```

Two lines carry it: `order.append(vertex)` sits *after* the loop, on the finish event, and the result is reversed at the end. Appending before the loop gives you a pre-order, which is not a topological order and which looks plausible on small examples.

The three-colour check is not optional here. Without it the function returns a confident, wrong order on a cyclic graph.

The complete solution

```
"""Topological sort: Kahn's, DFS finish order, levels, and the smallest order."""

from __future__ import annotations

import heapq
from collections import deque

WHITE, GREY, BLACK = 0, 1, 2

def build(n: int, edges: list[tuple[int, int]]) → dict[int, list[int]]:
    """edges are (before, after). Built from range(n) so every vertex exists."""
    graph: dict[int, list[int]] = {v: [] for v in range(n)}
    for before, after in edges:
        graph[before].append(after)
    return graph

def kahn(graph: dict[int, list[int]], n: int) → list[int] | None:
    """A valid order, or None on a cycle.  $O(V + E)$ ."""
    in_degree = [0] * n
    for vertex in range(n):
        for neighbour in graph[vertex]:
            in_degree[neighbour] += 1
    queue = deque(v for v in range(n) if in_degree[v] == 0)
    order: list[int] = []
    while queue:
        vertex = queue.popleft()
        order.append(vertex)
        for neighbour in graph[vertex]:
            in_degree[neighbour] -= 1
            if in_degree[neighbour] == 0:
                queue.append(neighbour)
    return order if len(order) == n else None

def levels(graph: dict[int, list[int]], n: int) → list[list[int]] | None:
    """Parallel schedule. len(result) is the critical path length."""
    in_degree = [0] * n
    for vertex in range(n):
        for neighbour in graph[vertex]:
            in_degree[neighbour] += 1
    frontier = [v for v in range(n) if in_degree[v] == 0]
    out: list[list[int]] = []
    done = 0
    while frontier:
        out.append(frontier)
        done += len(frontier)
        nxt: list[int] = []
        for vertex in frontier:
            for neighbour in graph[vertex]:
                in_degree[neighbour] -= 1
                if in_degree[neighbour] == 0:
                    nxt.append(neighbour)
        frontier = nxt
    return out if done == n else None

def smallest_order(graph: dict[int, list[int]], n: int) → list[int] | None:
```

```

"""The lexicographically smallest valid order.  $O((V + E) \log V)$ ."""
in_degree = [0] * n
for vertex in range(n):
    for neighbour in graph[vertex]:
        in_degree[neighbour] += 1
heap = [v for v in range(n) if in_degree[v] == 0]
heapq.heapify(heap)
order: list[int] = []
while heap:
    vertex = heapq.heappop(heap)
    order.append(vertex)
    for neighbour in graph[vertex]:
        in_degree[neighbour] -= 1
        if in_degree[neighbour] == 0:
            heapq.heappush(heap, neighbour)
return order if len(order) == n else None

def dfs_order(graph: dict[int, list[int]], n: int) → list[int] | None:
    """Reversed finish order. Needs the three-colour cycle check."""
    colour = [WHITE] * n
    out: list[int] = []

    def visit(vertex: int) → bool:
        colour[vertex] = GREY
        for neighbour in graph[vertex]:
            if colour[neighbour] == GREY:
                return False
            if colour[neighbour] == WHITE and not visit(neighbour):
                return False
        colour[vertex] = BLACK
        out.append(vertex)
        return True

    for vertex in range(n):
        if colour[vertex] == WHITE and not visit(vertex):
            return None
    return out[::-1]

def count_orders(graph: dict[int, list[int]], n: int) → int:
    """How many valid orders exist. Exponential in general; fine for small n."""
    in_degree = [0] * n
    for vertex in range(n):
        for neighbour in graph[vertex]:
            in_degree[neighbour] += 1

    def recurse(remaining: int) → int:
        if remaining == 0:
            return 1
        total = 0
        for vertex in range(n):
            if in_degree[vertex] == 0:
                in_degree[vertex] = -1                # mark as used
                for neighbour in graph[vertex]:
                    in_degree[neighbour] -= 1
                total += recurse(remaining - 1)
                for neighbour in graph[vertex]:
                    in_degree[neighbour] += 1            # undo
                in_degree[vertex] = 0
        return total

```

```

    return recurse(n)

if __name__ == "__main__":
    tasks = build(6, [(0, 2), (1, 2), (2, 3), (2, 4), (3, 5), (4, 5)])
    print("kahn    :", kahn(tasks, 6))
    print("dfs     :", dfs_order(tasks, 6))
    print("smallest:", smallest_order(tasks, 6))
    print("levels  :", levels(tasks, 6))
    print("orders  :", count_orders(tasks, 6))

    ring = build(3, [(0, 1), (1, 2), (2, 0)])
    print("cycle   :", kahn(ring, 3), dfs_order(ring, 3), levels(ring, 3))

```

Running it:

```

kahn    : [0, 1, 2, 3, 4, 5]
dfs     : [1, 0, 2, 4, 3, 5]
smallest: [0, 1, 2, 3, 4, 5]
levels  : [[0, 1], [2], [3, 4], [5]]
orders  : 4
cycle   : None None None

```

Three things to look at. `kahn` and `dfs` give **different orders** and both are correct — every edge points forwards in each. If you were expecting them to match, that expectation is the bug.

`levels` shows the parallel structure: four rounds instead of six steps, so with two workers this finishes a third faster. And `orders` is 4 — `{0,1}` can be ordered two ways and `{3,4}` can be ordered two ways, so 2×2 .

6 What it costs

Kahn's.

building in-degrees	one pass over every edge	E
	plus initialising the array	V
each vertex queued once		V
each edge decremented once		E

		$O(V + E)$ time
in-degree array + queue + output		$O(V)$ space

DFS version.

each vertex coloured once, appended once	V
each edge examined once	E

	$O(V + E)$ time
colour array + output + recursion stack	$O(V)$ space

Identical. The choice between them is never about speed.

The lexicographic version:

heap push and pop	$\log V$ each
V pushes and V pops	$V \log V$

	$O((V + E) \log V)$

V = 100,000, E = 200,000	
Kahn's	300,000 steps
smallest	$300,000 \times 17 = \sim 5,000,000$ steps

About seventeen times more work for the ordering guarantee. Worth it when asked for; not worth it by default.

The levels version costs exactly the same as Kahn's — every vertex and edge is still touched once — and returns strictly more information. There is no reason to prefer the flat version except that the caller wants a flat list.

Counting all valid orders is a completely different problem:

counting orders exponential in general (it is #P-complete)
n independent vertices with no edges $\rightarrow n!$ orders
n = 12, no edges $\rightarrow 479,001,600$

the recursive version above:	fine for $n \leq 12$ or so
anything larger:	needs bitmask DP, $O(2^n \times n)$, fine for $n \leq 20$

Say this out loud if asked to count, because the naive expectation is that it is as cheap as producing one order, and it is not.

Space, at scale:

V = 1,000,000, E = 5,000,000	
in-degree as a list of ints	~ 8 MB
adjacency as a list of lists	~ 8 MB of list objects + 40 MB of entries
queue at peak	up to V, ~ 8 MB
output list	~ 8 MB

And the recursion caveat for the DFS version:

```
a dependency chain 100,000 long → 100,000 frames → RecursionError
```

Kahn's never recurses, which is a real and practical argument for it on large inputs and one of the reasons to default to it.

The critical path, which is the number that matters for a build:

```
1,000 tasks, average 5 seconds each
serial                    5,000 s = 83 minutes
levels = 12
critical path             12 x 5 = 60 s

with unlimited workers, one minute instead of 83.
the widest level tells you how many workers are worth having
```

7 The traps

Edge direction reversed

The near-miss that produces a confidently wrong answer:

```
for course, prerequisite in pairs:
    graph[course].append(prerequisite)    # backwards
```

The input says “to take `course`, first take `prerequisite`”, so the edge means “prerequisite before course” and must point *from* the prerequisite. Reversed, the algorithm runs, finds no cycle, and returns an order in which every course comes before its own prerequisites:

```
>>> topological_sort(backwards_graph, 4)
[3, 2, 1, 0]                # a valid topological order of the WRONG graph
```

No error, and the output is a plausible-looking list. Before writing the loop, say the meaning as an English sentence — “an edge from A to B means A must happen before B” — and then check by hand which list gets the append.

Building from the edge list

```
graph = defaultdict(list)
for a, b in edges:
    graph[a].append(b)
```

A vertex with no outgoing edges never becomes a key, and a vertex in no edge at all never appears anywhere:

```
>>> kahn(from_edges_only, n=5)
None                                # reports a cycle on an acyclic graph
```

`len(order) == n` cannot hold when the structure does not contain `n` vertices. **This produces a phantom cycle**, which is the most confusing possible failure, and it is fixed by building from `range(n)`.

Appending before the loop in the DFS version

```
def visit(vertex):
    colour[vertex] = GREY
    order.append(vertex)                # ← pre-order, not post-order
    for neighbour in graph[vertex]:
        ...
```

This produces the order vertices were *entered*, which is not a topological order:

```
graph: 0 → 1, 2 → 1
pre-order from 0 then 2: [0, 1, 2]
                        2 must come before 1, and it does not
```

It looks right on a simple chain, which is why it survives testing. **The append goes after the loop, and the result is reversed.**

DFS with no cycle check

```
def visit(vertex):
    seen.add(vertex)
    for neighbour in graph[vertex]:
        if neighbour not in seen:
            visit(neighbour)
    order.append(vertex)
```

On a cyclic graph this terminates and returns a list:

```
>>> dfs_no_check(ring, 3)
[2, 1, 0]                                # there is no valid order; this is nonsense
```

Kahn's cannot do this — it structurally cannot output all `n` vertices when a cycle exists — which is one more reason to prefer it.

Expecting a unique answer

```
assert kahn(graph, 6) == [0, 1, 2, 3, 4, 5]
```

Kahn's, the DFS version, and the same code with the adjacency lists built in a different order all give different valid answers. **Tests must verify the property, not the output:** for every edge `(a, b)`, check that `position[a] < position[b]`. That verification is five lines and it is what you should write.

Recursion depth on the DFS version

```
Traceback (most recent call last):
  File "topo.py", line 14, in visit
    if colour[neighbour] == WHITE and not visit(neighbour):
    [Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

A long dependency chain — real build graphs have them — and $n \leq 10^5$ in the constraints. Kahn's, or the iterative DFS with stored iterators.

Duplicate edges

```
edges = [(0, 1), (0, 1)]
```

The in-degree of `1` becomes 2 and only ever gets decremented once when `0` is processed — no, twice, because the adjacency list contains `1` twice. So it actually works. **But if you deduplicate the adjacency list without recomputing in-degrees, or compute in-degrees from a deduplicated edge set while the adjacency list keeps duplicates, the counts diverge and a vertex is either never released or released early.** The rule is that in-degrees and adjacency must be computed from the same data — which is why the code above derives the counts *from* the adjacency structure rather than from the raw edge list.

8 In the interview

How it gets asked

- “Order these tasks so that every dependency comes first.” — the direct version.
- “Can all courses be finished? Now give me a valid order.” — LeetCode 207 then 210.
- “In what order should these packages be installed?”
- “Given a build graph, produce the fastest schedule.” — the levels version.

- “There is an alien alphabet; here are some sorted words. Determine the letter order.” — LeetCode 269, and the modelling is the whole problem.
- “Return the lexicographically smallest valid order.”

The first ninety seconds

“Topological sort, and I would use Kahn’s algorithm.

The model first: a vertex is a task, and an edge from A to B means **A must happen before B**. I want to say that as a sentence before I write the append, because the input usually expresses it the other way round — ‘B requires A’ — and reversing it silently gives a valid order of the wrong graph, with no error.

Kahn’s: compute each vertex’s in-degree, which is how many things it is still waiting for. Seed a queue with everything at zero — those can start now. Repeatedly take one out, output it, and decrement the in-degree of everything it points at; anything that reaches zero is now unblocked, so queue it.

If fewer than n vertices come out, there is a cycle — the leftovers never reached zero, meaning each is permanently waiting on something. So the same pass answers both ‘is there an order’ and ‘what is it’, which is the main reason I prefer it to the DFS version.

The other reasons: it never recurses, so a hundred-thousand-long dependency chain is not a problem; and it reads like the domain, which matters when someone else has to maintain it.

$O(V + E)$ time and $O(V)$ space.

Two things I would set up carefully. The structure is built from the full vertex list, not from the edge pairs — a task with no dependencies and no dependents must still exist, or $\text{len}(\text{order}) = n$ never holds and I report a cycle that is not there. And I would not assert a specific output in a test, because many orders are valid; I would verify the property instead — for every edge, the source appears before the target.

And I would ask whether you want a serial order or a schedule, because if these tasks can run in parallel, processing the queue one whole level at a time gives me groups that can run simultaneously, and the number of groups is the critical path. That is a two-line change and it is usually what the caller actually needs.”

The follow-ups

“Kahn’s or DFS? Why?”

“Kahn’s, for four reasons, and they are all practical rather than about complexity — both are $O(V + E)$.

It detects the cycle structurally. It cannot output all n vertices when a cycle exists. The DFS version will happily return a confident, wrong order on a cyclic graph unless I bolt on the three-colour check, and that check is a thing to remember rather than a thing that cannot go wrong.

It never recurses. A long dependency chain overflows the stack in the DFS version, and raising the limit turns a `RecursionError` into a segfault.

It gives the parallel schedule for free by processing one level at a time, which is what a build system actually wants.

And it reads like the problem. ‘Take anything nothing is waiting on, remove it, repeat’ is what a person would do.

The DFS version’s advantages are that it is three lines shorter and that if I already need a DFS for something else — finding the cycle itself, or strongly connected components — I get the order as a by-product of the finish times I am already computing. That is a real argument in a larger piece of code.”

“Return the lexicographically smallest order.”

“Replace the queue with a min-heap. When several vertices are available, Kahn’s picks arbitrarily; a heap always picks the smallest.

The greedy choice is correct, and I would give the reason rather than assert it: taking the smallest currently-available vertex cannot make any smaller vertex unavailable later. Availability only ever increases as I remove things — nothing I output can block anything — so if a smaller vertex were available at some later point, it was available now, and I would have taken it.

The cost goes from $O(V + E)$ to $O((V + E) \log V)$, which on a hundred thousand vertices is roughly seventeen times more work. So I would only do it when asked.

The related question worth being ready for is ‘is the order unique?’ — and it is unique exactly when the queue contains at most one vertex at every step. That is a one-line check inside the same loop, and it is how you answer ‘can this sequence be reconstructed unambiguously’, which is LeetCode 444.”

“How would you parallelise the build?”

“The levels version, and it is the same algorithm read differently.

Instead of taking one vertex off the queue at a time, take the *whole* current frontier. Everything in it has in-degree zero at the same moment, so nothing in it depends on anything else in it, so it can all run simultaneously. Then process all of their outgoing edges and whatever reaches zero becomes the next level.

Two numbers fall out that a serial order does not give me. **The number of levels is the critical path** — the minimum wall-clock time with unlimited workers. And **the widest level is the maximum useful parallelism** — beyond that many workers, some sit idle.

Concrete: a thousand tasks averaging five seconds is eighty-three minutes serially. If there are twelve levels, the critical path is sixty seconds, and if the widest level is forty tasks, there is no point running more than forty workers.

The refinement I would mention is that levels assume every task takes the same time, which builds do not. With real durations, the right answer is to compute the earliest start time per task — the maximum over its dependencies of their finish times — which is a single pass in topological order and gives a true critical path. That is the same algorithm with a `max` instead of a counter, and it is one of the nicest things you get from having the order: **any DP over a DAG is one pass in topological order.**“

“Here are some words sorted in an unknown alphabet. Find the letter order.”

“The whole difficulty is the modelling; the sort is then routine.

A vertex is a letter. An edge from `a` to `b` means `a` comes before `b` in the alphabet. I get those edges by comparing *adjacent* words in the list: walk them together until the first position where they differ, and that one pair of characters gives me exactly one ordering fact. Everything after that position tells me nothing.

Only adjacent pairs, and only the *first* difference — those are the two things people get wrong. Comparing all pairs adds no information and adds edges that are implied. Taking more than the first difference adds edges that are simply false.

The edge case that is the actual test: if a word is a prefix of the previous one — `["abc", "ab"]` — that is invalid input, because a shorter prefix must sort first. Returning an empty result there rather than ignoring it is what the hidden tests check.

Then Kahn’s over the letters that actually appear. A cycle means the input is contradictory and there is no valid alphabet. Multiple valid orders are possible and any is acceptable unless they ask for the smallest.

Cost: building edges is `O(total characters)`; the sort is `O(V + E)` where `V` is at most 26 and `E` at most the number of word pairs. So it is linear in the input, and the graph is tiny.”

The model answer

“A CI system runs jobs. Each job declares which jobs must complete before it. Design the scheduler: detect invalid configurations, and run the jobs as fast as possible on a fixed pool of workers.”

“Three requirements — validate, order, schedule — and Kahn’s gives me all three from one structure.

The model. A vertex is a job. An edge from A to B means A must complete before B starts. The config file almost certainly says `needs: [A]` under job B, so the edge points from each entry in `needs` to the job declaring it — and I would write that direction down as a sentence and verify it on a two-job example before going further, because reversing it produces a schedule that runs everything backwards with no error.

Validation. Kahn’s, and if fewer than `n` jobs come out, the configuration has a circular dependency. But ‘circular dependency detected’ is not a usable error message, so I would run a second pass — three-colour DFS carrying the path — over just the unemitted jobs to report the actual loop: `test → build → lint → test`. And I would report *all* the cycles rather than the first, since a large config can have several. **The rigorous version is strongly connected components**, where each non-trivial component is one independent cycle, and I would name that as what I would use if configs get big.

Two more validations that are not cycles and matter as much. A job depending on a name that does not exist — a typo — should be caught before any graph work, with a clear message. And duplicate entries in a `needs` list, which I would deduplicate, because if in-degrees and the adjacency structure are computed from inconsistent data, a job is either released early or never released at all.

Scheduling. Not a flat order — the level structure. Everything at in-degree zero can start immediately; as each job finishes I decrement its dependents and anything reaching zero becomes runnable. **With a fixed worker pool, this is a ready-set rather than strict levels:** I keep a set of runnable jobs and hand them out as workers free up, rather than waiting for a whole level to finish. That is strictly better, because a level containing one 10-minute job and nine 5-second jobs would otherwise idle nine workers for ten minutes.

Which runnable job to hand out first is a real choice and I would make it deliberately: **the one with the longest remaining critical path behind it**, computed by a single reverse pass in topological order. That is classic list scheduling and it beats arbitrary choice noticeably, because starting a long chain late extends the whole build.

The numbers I would report to the user, because they are free once the graph exists: the critical path length, which is the fastest the build can possibly be; the widest level, which is how many workers are worth paying for; and the total work.

‘This build takes 83 minutes of work, its critical path is 6 minutes, and more than 12 workers will not help’ is genuinely useful output and almost no CI system shows it.

Failure handling, which is where the graph keeps earning. When a job fails, everything downstream of it is unreachable and should be marked skipped rather than left pending — that is a forward traversal from the failed vertex, and reporting ‘skipped 14 jobs because build failed’ is much better than fourteen timeouts.

Cost: $O(V + E)$ for everything, with V in the hundreds and E similar for a realistic config, so the scheduling is free relative to running anything. The one thing I would watch is that the config is re-parsed and re-validated on every run, so it should stay linear and not creep into anything quadratic as configs grow.”

9 Recall card

A topological order puts every vertex before everything it points to. It exists **iff** the graph is acyclic, and there are usually **many** valid orders — verify the property in tests, never a specific list.

Kahn’s: in-degree zero into a queue; pop, output, decrement; anything reaching zero is queued. If fewer than n come out, the leftovers are in a cycle. **Prefer it:** structural cycle detection, no recursion, parallel schedule for free, and it reads like the domain.

DFS version: append on *finish*, then reverse — and it needs the three-colour check bolted on, or it returns a confident wrong answer on a cyclic graph.

Process one whole level at a time and you get the parallel schedule. Number of levels = critical path; widest level = maximum useful workers. Same $O(V + E)$, strictly more information.

Two silent bugs: the edge direction reversed (valid order of the wrong graph), and building the structure from the edge list so isolated vertices vanish — which reports a **phantom cycle**. Build from $\text{range}(n)$ and say the edge’s meaning as a sentence first.

1 What this is, and why they ask it

You have a database and you have files, and somebody has to decide where the files live. Inside the database as a BLOB column, or outside in object storage with only a reference in the row.

Almost always the answer is outside. The interesting part is *why*, and the reason interviewers like this question is that it is one of the few design decisions where the argument can be settled with arithmetic rather than opinion. Backup time, cache efficiency, replication traffic, connection pool occupancy, cost per gigabyte — every one of those is a number, and a candidate who produces numbers is a different candidate from one who says “best practice is to use S3”.

It is also a question with a real minority answer. Below a certain size and volume, storing the bytes in the database is genuinely the better engineering choice, because it removes an entire class of consistency problem. **Knowing where that line sits, and being able to say what moves it, is the actual skill.**

Yesterday’s lesson was [what object storage is](#). Today is the decision, the arithmetic behind it, and the failure modes on both sides.

By the end of this lesson you can defend either answer with numbers, quantify what a BLOB column does to backups and cache, name the two consistency problems the external store creates and how to handle them, and say what changes the decision.

2 The story

There is one fridge in the Menon house and it has been the same fridge since 2009.

It is not large. Two shelves and a door, a small freezer at the top that mostly holds ice trays and one packet of peas that has been there since some time in 2023.

What lives in it is what you would expect. Milk, twice a day. Curd. Whatever was left from lunch. The bottle of water everybody wants. The vegetables in the drawer at the bottom. Every one of those is something a person opens the fridge for several times a day.

In May, Ravi’s uncle came from Nagpur and brought a watermelon.

It was an enormous thing, and it went in the fridge, because that is what you do with a watermelon in May, and for about four days it occupied roughly half of the lower shelf and part of the middle one, because it would not fit anywhere flat.

Nobody could get to anything.

The milk went behind it. The curd was somewhere on the left, under the coriander. To take out the water bottle you had to take out the watermelon, which is a two-hand job, and put it on the counter, and then find the water, and then put the watermelon back. Ravi's mother did this perhaps twelve times a day for four days.

There is a cool storeroom under the stairs where the rice and the oil tins are, and where the watermelon would have been perfectly happy for those four days. It is not cold, but it is not warm either, and a watermelon is not milk. Nobody thought of it, because the fridge is where cold things go and a watermelon is a cold thing.

The other consequence, which they noticed on the Saturday, was the cleaning. Every month or so the fridge gets emptied out and wiped down, and it takes about twenty minutes. That Saturday it took an hour, because the watermelon had to come out, and everything under it had to come out, and putting it all back in a way that worked took three attempts.

The fridge is small and expensive and everything in it is something you reach for constantly. The storeroom is large and cheap and nobody goes in there twice a week.

The watermelon was in the wrong room.

3 The idea in plain English

The fridge is the database and the storeroom is object storage.

A database is optimised for small things fetched constantly. Rows are small. Indexes assume small. The buffer pool — the memory the database keeps hot data in — is sized for a working set of rows. Every mechanism in it, from page layout to replication, assumes the unit of work is bytes-to-kilobytes.

A BLOB is a column holding raw bytes. Postgres calls it `bytea`, MySQL has `BLOB` and `LONGBLOB`, and they all work. You can absolutely put a 4 MB photo in a row. The question is what happens to everything else when you do.

The first cost is the buffer pool. The database keeps recently used pages in memory. A 4 MB photo evicts roughly a thousand 4 KB pages of actual rows and indexes — pages that many queries were using. **One photo read pushes a thousand useful pages out of memory**, so the cache hit rate for ordinary queries falls, and suddenly unrelated queries are slower. That is the milk going behind the watermelon.

The second cost is backups. A backup copies everything. A database of 20 GB of rows plus 4 TB of images is a 4 TB backup, and a restore is a 4 TB restore. **Recovery time is dominated entirely by data you did not need to protect that way**, because object storage already keeps eleven nines of durability without your help. That is the Saturday cleaning taking an hour.

The third cost is replication. Every byte written to the primary is shipped to every replica. Uploading a 100 MB video writes 100 MB to the write-ahead log and sends 100 MB to each replica. With two replicas that is 300 MB of network for one upload, and the replication lag that follows delays *every other query's* visibility on those replicas.

The fourth is connections and memory. A database connection is an expensive thing and there are typically a few hundred. Streaming a 50 MB BLOB out of one holds it for the duration, and many clients materialise the whole value in memory before sending it. **A hundred concurrent downloads is your connection pool gone**, and every ordinary query then waits.

The fifth is cost per gigabyte. Database storage is provisioned SSD with replication and backups — five to ten times the price of object storage for the same bytes, before counting the instance that has to serve it.

So: the database holds the reference, and object storage holds the bytes. A row with the key, the size, the content type and the owner. Everything the application queries stays small; everything large is fetched directly by the client from a store built for exactly that, through a CDN, without touching your servers.

But the external store creates two problems that the BLOB column does not have, and an honest answer names them.

No transaction spans the two systems. You can commit the row and fail to write the object, or write the object and fail to commit the row. There is no `BEGIN` that covers both. You get **orphans** — objects nobody references — and **broken links** — rows pointing at nothing. Both happen, and the answer is ordering plus a reconciliation job, exactly as on [day 133](#).

And deletion is no longer atomic. Deleting a row is one statement; deleting a row and its object and its derivatives and the CDN's copy is four systems and no transaction.

Those two problems are precisely what the minority case buys back. For small files, low volume, and a hard requirement that the file and its metadata are consistent — a signed contract, an audit attachment, a small avatar — putting the bytes in the database is defensible, because `INSERT` either happens or does not and there is nothing to reconcile. **Below roughly a few hundred kilobytes and a few gigabytes total, that is a real engineering choice, not a mistake.**

And there is a middle option worth knowing. Postgres stores large values out-of-line automatically — **TOAST** — in a side table, compressed, and only reads them when the column is selected. So `SELECT id, name FROM users` does not pull the avatar. That softens the buffer-pool argument considerably and is worth mentioning, because it is the strongest technical point on the “in the database” side. It does not soften the backup, replication or cost arguments at all.

4 The picture

Where the bytes go, and what each path costs:

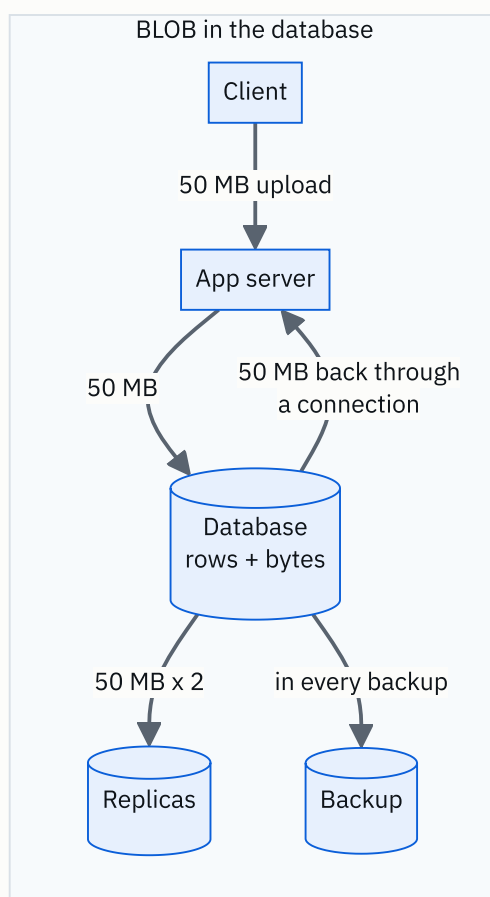


FIGURE 134.2

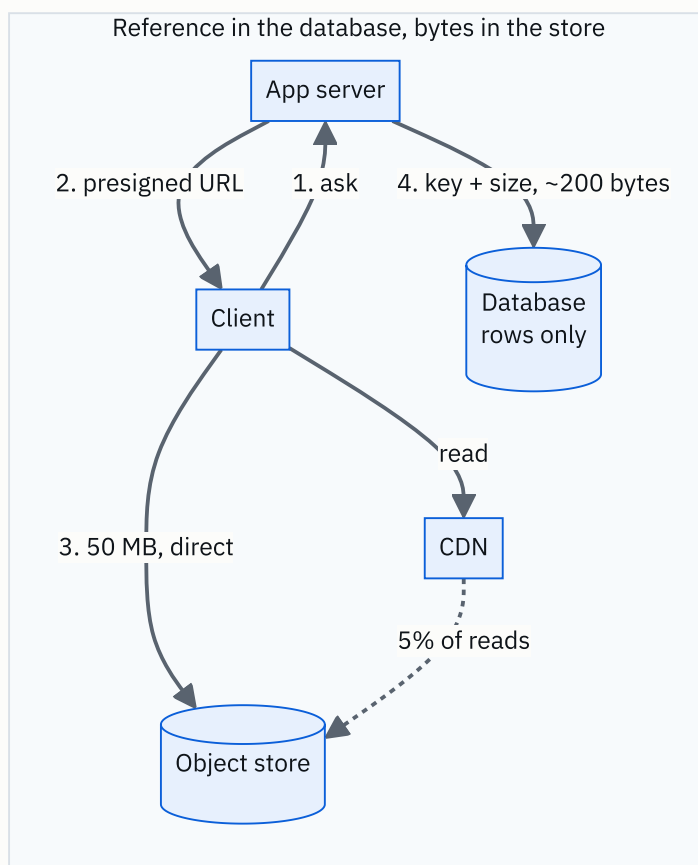


FIGURE 134.3

What to notice. In the second diagram the database sees two hundred bytes and the app server sees two small requests. In the first, the same 50 MB crosses the app server twice, the database once, the replication link twice, and lands in every backup forever.

The buffer pool, which is the argument people underestimate:

```
Database with 16 GB of buffer pool, 4 KB pages
= 4,000,000 pages of room
```

```
WITHOUT blobs
working set: 20 GB of rows and indexes
hot 16 GB stays resident
cache hit rate ~99%
```

```
WITH 4 MB photos in rows
one photo read = 1,000 pages loaded
100 photo reads per second = 100,000 pages/s evicted
the 4,000,000-page pool turns over every 40 seconds
→ row and index pages are constantly evicted
→ cache hit rate falls to ~70%
→ every unrelated query does disk reads it used to avoid
```

What to notice. Nothing about the photo queries got slow. **The damage is to every other query**, which is why this shows up as “the whole database got slower” and takes a long time to diagnose.

And the backup arithmetic, which is the argument that usually ends the discussion:

```
rows only          20 GB
+ images           4 TB
-----
backup size        4.02 TB

restore at 200 MB/s  4,020 GB / 0.2 GB/s = 20,100 s = 5.6 HOURS

rows only
restore at 200 MB/s  20 / 0.2 = 100 s = 1.7 MINUTES
```

What to notice. Your recovery time objective just went from under two minutes to nearly six hours, for data that object storage was already keeping at eleven nines of durability without being backed up at all.

5 How it actually works

The reference-in-the-database schema

```
CREATE TABLE media (  
  id          UUID PRIMARY KEY,  
  owner_id    BIGINT      NOT NULL REFERENCES users(id),  
  object_key  TEXT        NOT NULL UNIQUE,    -- 'media/9f/9f3a...-orig.jpg'  
  content_type TEXT       NOT NULL,  
  size_bytes  BIGINT      NOT NULL,  
  checksum    TEXT        NOT NULL,          -- verify what came back  
  state       TEXT        NOT NULL,          -- PENDING | READY | DELETED  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE INDEX ON media (owner_id, created_at DESC);
```

Two hundred bytes a row. A million media items is 200 MB of table — entirely cacheable — while the bytes themselves are terabytes elsewhere.

`state` is what makes the two-system problem manageable, and `checksum` is what lets you verify that the object you fetched is the one the row describes.

The write order that makes failures recoverable

Row first, then object, then confirm.

```
1. INSERT media (... , state='PENDING')    -- committed  
2. hand out a presigned URL for object_key  
3. client uploads directly to the store  
4. store emits an event → worker → UPDATE media SET state='READY'
```

Then the two failure modes are:

- **Row without object** — the upload never happened. Visible, queryable, cleaned up by a job that deletes `PENDING` rows older than a day. Harmless.
- **Object without row** — cannot happen, because the row was created first and the key came from it.

Getting this order backwards inverts the failure into the bad one: an object nobody references, invisible to the application, never deleted, costing money forever. **Always create the row first.**

The reconciliation job

Everything above is best-effort, so:


```

nightly:
  list the bucket by prefix
  full-outer-join against the media table on object_key

  object with no row, older than 24h    → delete (it is an orphan)
  row READY with no object              → alert (this is data loss)
  row PENDING older than 24h           → delete the row

```

The second line is the one that matters. A `READY` row whose object is missing is a broken link a user will hit, and it is the only way to find out that something deleted an object it should not have.

Deletion, which is genuinely harder

Deleting a media item touches the database row, the original object, every derivative, and the CDN's cached copies. Four systems, no transaction.

The workable pattern:

```

1. UPDATE media SET state='DELETED'      -- instant; the user stops seeing it
2. enqueue a cleanup job                 -- idempotent, retried
3. worker deletes derivatives, then the original, then invalidates the CDN
4. worker deletes the row (or leaves a tombstone for audit)

```

Soft-delete first so the user-visible effect is immediate and transactional, then clean up asynchronously. If there is a regulatory deletion requirement, step 3 needs its own monitoring and a report, because “eventually, probably” is not an acceptable answer to a regulator.

The case for the database, done properly

For small files it is genuinely simpler, and Postgres has real support:

- **bytea with TOAST.** Values above about 2 KB are stored out-of-line in a side table and compressed, and are only read when that column is selected. So `SELECT id, name FROM users` does not touch the avatar at all. **This is the strongest argument on this side** and it neutralises much of the buffer-pool objection.
- **Large objects (`lo`)** support streaming and seeking, at the cost of a separate API and manual cleanup.
- **SQLite** is a special case: its authors have published benchmarks showing that reading blobs under about 100 KB out of SQLite is *faster* than reading them as individual files, because of filesystem overhead per open. For a mobile or embedded application, in the database is often correct.

Where the line sits, in practice:

< 100 KB per file, < 10 GB total, and consistency matters	→ database is defensible
> 1 MB per file, or > 100 GB total, or served to browsers in between	→ object storage, clearly
	→ object storage by default, database if the consistency argument is strong

The third option: a filesystem

Occasionally the answer is neither — a network file system or local disk. It gives you real file semantics for legacy software that needs them, and it costs you: no eleven nines, a capacity ceiling per volume, backup is your problem, and scaling reads means your own replication. **It is the right answer mainly when something you cannot change requires a `path`**, and saying so is better than pretending object storage covers every case.

6 The numbers

Cost per gigabyte per month:

object storage (S3 Standard)	\$0.023
S3 Infrequent Access	\$0.0125
managed Postgres SSD (gp3)	\$0.115
managed Postgres provisioned IOPS	\$0.125+

4 TB of images	
object storage	4,000 x \$0.023 = \$92/month
in the database	4,000 x \$0.115 = \$460/month storage alone
	+ backup storage 4,000 x \$0.095 = \$380
	+ a larger instance to hold it
	→ ~\$1,000+/month vs \$92

Roughly ten times, before counting the instance size you needed to grow to.

Backup and restore:

rows only	20 GB	restore at 200 MB/s	= 100 s
rows + 4 TB blobs	4.02 TB	restore at 200 MB/s	= 5.6 hours

Backup window:

nightly backup of 20 GB	~2 minutes
nightly backup of 4 TB	~5.6 hours

A backup that takes almost six hours cannot run nightly without overlapping other work, and it holds locks or snapshot resources for that whole period.

Replication traffic:

```
1,000 uploads/day at 5 MB    = 5 GB/day written
x 2 replicas                  = 10 GB/day of extra replication
                             = 115 KB/s average, fine

10,000 uploads/day at 50 MB  = 500 GB/day
x 2 replicas                  = 1 TB/day
                             = 11.6 MB/s sustained, plus every burst
```

At the second scale, replication lag becomes visible on every read replica, delaying the visibility of completely unrelated writes — which is how a photo feature degrades a reporting dashboard.

Buffer pool damage:

```
16 GB pool / 4 KB pages      = 4,000,000 pages
one 4 MB blob read           = 1,000 pages
100 blob reads/second        = 100,000 pages/s
pool fully turned over every 4,000,000 / 100,000 = 40 seconds
```

```
cache hit rate before ~99%    → 1 disk read per 100 queries
cache hit rate after  ~70%    → 30 disk reads per 100 queries
                             → 30x more disk I/O on unrelated queries
```

Connection occupancy:

```
200 connections in the pool
50 MB download at 20 MB/s    = 2.5 s per download holding one connection
200 concurrent downloads     = the entire pool, for 2.5 s
                             → every ordinary query queues behind them
```

With presigned URLs the same 200 downloads use **zero** database connections.

Where the small case wins:

```
avatars: 50 KB each, 100,000 users
total                    5 GB
in the database          5 x $0.115 = $0.58/month
backup impact            5 GB added to a 20 GB backup → 25 GB, still ~2 min
buffer pool             TOAST keeps them out of the row; not read unless selected
consistency              atomic with the user row. No orphans. No reconciliation job.
```

Fifty-eight cents and one fewer distributed-systems problem. At this size the object store is more machinery than the problem needs, and saying that is a stronger answer than reflexively reaching for S3.

The crossover, roughly:

total blob bytes vs total row bytes

blobs < 25% of the database → the backup and cache damage is tolerable

blobs > 100% of the database → the database is now a file server with SQL bolted on

7 The trade-offs

Object storage costs you atomicity across two systems. No transaction spans the row and the object, so orphans and broken links are permanent possibilities and a reconciliation job is not optional. The database version has none of that: one `INSERT`, one `DELETE`, and consistency is free. **That is the real thing you are trading away**, and any answer that does not mention it is incomplete.

Object storage costs you a second failure domain. The store can be unavailable while the database is fine, and now a page renders with broken images and the app has to handle it. One system has one availability number; two systems have a product.

The database costs you everything at scale. Backup time, restore time, replication lag, buffer-pool pollution, connection occupancy, and ten times the storage price. Each of those is survivable alone and they arrive together, and they arrive as “the database got slow” rather than as anything that points at the cause.

TOAST genuinely weakens one of the arguments and not the others. Out-of-line, compressed, only read when selected — so the buffer-pool objection is much smaller in Postgres than folklore suggests. Backups, replication and cost are unaffected. **Knowing that TOAST exists and what it does and does not fix is what separates a considered answer from a recited one.**

Serving is the argument that is hard to counter. Files in a database must be read by your application and streamed to the client, so every download occupies a connection, a worker and your bandwidth. Files in object storage are fetched directly by the browser via a CDN. **For anything user-facing at any real volume, that alone settles it.**

When would I put files in the database? Small files, modest total volume, and a genuine requirement that the file and its metadata are consistent — legal attachments, signed documents, an avatar. Also whenever the whole system is one SQLite file, where the benchmark actually favours it under 100 KB. And in early-stage products where the reconciliation job is real work that has not been earned yet, with the migration path planned.

When would I not use either? When something I cannot change needs a real filesystem path, where a network file system is the honest answer despite being more expensive and less durable.

8 In the interview

How it gets asked

- *“Should the images go in the database? Defend your answer.”* — the direct version, and “defend” means numbers.
- *“Where do you store user uploads?”*
- *“Your database is 4 TB and restores take six hours. Why?”*
- *“The database got slower and none of the queries changed. What would you look at?”*
- *“What breaks if you keep the files outside the database?”* — the test of whether your answer is honest.

The first ninety seconds

“Outside the database, with only a reference in the row — but let me give the arithmetic rather than assert it, and then say what that choice costs me, because it does cost something.

Four numbers make the case.

Backups. A database of 20 GB of rows plus 4 TB of images is a 4 TB backup and a 4 TB restore. At 200 megabytes a second that is nearly six hours to recover, against under two minutes for the rows alone. My recovery time objective is now dominated by data that object storage already keeps at eleven nines without me backing it up at all.

The buffer pool. With a 16 GB pool and 4 KB pages, one 4 MB image read loads a thousand pages and evicts a thousand pages of actual rows and indexes. At a hundred image reads a second the whole pool turns over every forty seconds. **The image queries are fine; every other query gets slower**, which is why this shows up as ‘the database got slow’ and takes weeks to diagnose.

Connections. Streaming a 50 MB file out of the database holds a connection for the duration. Two hundred concurrent downloads is my entire pool, and every ordinary query queues behind them. With presigned URLs those downloads use zero database connections and zero of my bandwidth, because the browser talks to the store directly, through a CDN.

Cost. Database SSD is about eleven and a half cents a gigabyte-month; object storage is about two. For 4 TB that is roughly a thousand dollars a month against ninety, before the larger instance.

Now what I give up, because this is the part that makes the answer honest. There is no transaction across the two systems. I can commit the row and fail to write the object, or the reverse. So I get orphans and broken links, and I need an ordering rule and a reconciliation job. The database version has none of that — one `INSERT` and consistency is free.

And I would say where the answer flips. For files under about a hundred kilobytes with a few gigabytes total — avatars, small attachments — putting them in the database is defensible and I would probably do it, because the consistency is worth more than the arithmetic above, which barely applies at that scale.

How big are these files, and how many?”

The follow-ups

“What breaks if the files are outside?”

“Two things, and they are the whole cost.

No transaction spans the two systems. I write the object and commit the row as separate operations, and either can fail. That gives me orphans — objects nobody references, invisible to the app, costing money forever — and broken links, rows pointing at nothing, which a user hits as a missing image.

The mitigation is ordering plus reconciliation. **Create the row first, in a `PENDING` state, and derive the object key from it.** Then ‘object with no row’ is impossible by construction, and the remaining failure — row with no object — is visible, queryable, and cleaned up by a job that deletes stale `PENDING` rows. The upload’s completion is driven by the store’s own event rather than by the client saying so, because a browser that closes after uploading must not leave the system inconsistent.

Then a nightly reconciliation: list the bucket, join against the table, and act on both directions. **A `READY` row with no object is the alarming one** — that is real data loss and it is the only way to find out that something deleted an object it should not have.

The second thing is deletion. Deleting one item means the row, the original, every derivative, and the CDN copies — four systems, no transaction. So I soft-delete the row, which is instant and atomic and stops the user seeing it, then clean up asynchronously with an idempotent job. If there is a legal deletion requirement, that job needs monitoring and a report, because ‘eventually, probably’ does not satisfy a regulator.”

“The database has 4 TB of images in it already. What do you do?”

“Migrate, incrementally, and I would not touch the existing rows first.

Add the columns for the new path — `object_key`, `state` — and change all new writes to go to object storage. That stops the growth immediately, which is the urgent part, and it is a change to one code path.

Read path first, dual-source. The read code checks for an `object_key` and falls back to the BLOB column if there is not one. Now old and new coexist and nothing is broken.

Then backfill in the background, in batches, oldest or coldest first, at a rate that does not disturb production. Copy the bytes out, verify the checksum, write the key, then null the BLOB column.

The thing that catches people is that nulling the column does not reclaim the space. In Postgres the row versions are dead until vacuum, and the table file does not shrink without a `VACUUM FULL` or a rewrite, both of which take an exclusive lock on 4 TB. So the space comes back slowly, or during a planned maintenance window, or by moving the table entirely. **I would plan that explicitly rather than discover it**, and I would expect the backup to stay large for a while after the migration ‘finished’.

And I would measure the win, because it justifies the work: backup duration before and after, restore time, cache hit rate on the buffer pool, and p99 on unrelated queries. Those four numbers are the argument for having done it.”

“When is the database actually right?”

“Three situations, and I would rather name them than pretend the answer is universal.

Small files with a hard consistency requirement. A signed contract attached to a record, where the file and the record must exist together or not at all. An `INSERT` gives me that for nothing; object storage gives me a reconciliation job. At a hundred kilobytes and a few gigabytes total, the arithmetic against the database barely applies — five gigabytes of avatars is sixty cents a month and two minutes of extra backup.

SQLite, or anything embedded. SQLite’s own benchmarks show that reading blobs under about a hundred kilobytes out of the database is *faster* than reading them as files, because you avoid a filesystem open per item. For a mobile app, in the database is the correct answer, not a compromise.

Early-stage products. The reconciliation job, the presigned-URL flow and the cleanup worker are real engineering, and if the product might not exist in six months, a `bytea` column and a planned migration path is the right call. I would want the migration path written down, though, because the moment files get large the cost arrives all at once.

The thing I would push back on is the folklore version of the buffer-pool argument.

Postgres TOASTs large values out of the row automatically, compressed, and does not read them unless the column is selected — so `SELECT id, name FROM users` never touches the avatar. That neutralises much of the cache objection. It does nothing for backups, replication or cost, which are the arguments that actually decide it at scale.”

“The database got slower and nobody changed any queries.”

“If there are BLOBs in it, that is my first hypothesis and I can describe the mechanism.

The buffer pool is a fixed number of pages. Every large value read pulls in hundreds or thousands of pages and evicts an equal number of row and index pages that other queries were relying on. So the cache hit rate for ordinary queries falls, those queries start doing physical reads they used to avoid, and everything gets slower together — **while nothing about the image queries looks wrong.**

What I would look at, in order: buffer cache hit ratio over time, correlated against blob read volume; the ratio of table size to buffer pool size; and whether the slow queries are slow in *planning* or in *execution*, because this shows up purely as execution time with unchanged plans.

The other candidates for ‘slower with no query changes’ are worth naming so the answer is not one-note: a table that has grown past the point where an index fits in memory, autovacuum falling behind and bloating a hot table, a plan flip from stale statistics, or replication lag pushing read traffic onto a struggling replica. **I would check the cache hit ratio first because it distinguishes several of those at once.**“

The model answer

“A document management system for a law firm. Users upload contracts — PDFs, mostly 200 KB to 5 MB, some up to 100 MB. Around 500,000 documents, growing 200 a day. Full-text search is required, and documents must never be lost or become detached from their case record. Where do the files live?”

“The consistency requirement in that last sentence is doing a lot of work, so let me take the arithmetic first and then weigh it against that.

The arithmetic. 500,000 documents averaging, say, 2 MB is about 1 TB, growing 400 MB a day. In the database that is roughly \$115 a month of storage plus a similar amount of backup storage, and — more importantly — a backup that goes from minutes to about ninety minutes and a restore of the same. In object storage it is \$23 a month and the database stays around 500 MB of rows, which fits entirely in memory.

The 100 MB documents settle the serving question on their own. Streaming a 100 MB PDF out of the database holds a connection for several seconds and pushes 100 MB through my application. With twenty lawyers each opening a large document, that is a meaningful fraction of the connection pool doing nothing but copying bytes. Presigned URLs cost me zero connections and zero bandwidth.

So: object storage, with the reference in the database. But the consistency requirement means I would engineer the two-system problem properly rather than hand-wave it, and that is most of the design.

Row first, always. INSERT the document row in PENDING with a generated object key, then issue a presigned upload URL derived from that key. An object with no row becomes impossible by construction. The transition to READY is driven by the store’s object-created event, not by the client.

A checksum in the row, verified on write and on read. For legal documents, ‘the bytes are the ones we recorded’ is a stronger requirement than usual, and a SHA-256 in the row costs nothing and turns a silent corruption into a detectable one.

Versioning on in the bucket, with no expiry on old versions. A law firm’s requirement is ‘never lost’, and versioning means an overwrite or an accidental delete is recoverable. Combined with cross-region replication, that is genuinely stronger protection than a database backup — because eleven nines of durability plus versioning beats a nightly snapshot that can be up to 24 hours stale.

Reconciliation nightly, and I would make its output a report rather than a log line. Orphaned objects, PENDING rows older than a day, and — the one that matters — READY rows whose object is missing, which is a page-someone event in this domain rather than a metric.

Full-text search does not change the storage decision, and I want to be explicit about that, because it is the thing that tempts people back into the database. The text is extracted on upload by a worker, and the *extracted text* goes into a search index — Postgres full-text, or Elasticsearch if the requirements are richer. The PDF

bytes are never searched in place by anything. So the pipeline is: upload to the store, event fires, worker extracts text, text is indexed with the document id, and search returns ids that map to rows that map to presigned URLs.

Deletion is a legal question, not just a technical one. Soft-delete the row for instant effect, then an asynchronous job removes the object, its versions, the extracted text and the search entry. For a firm with retention obligations, that job needs an audit record — ‘document X purged at time T by request Y’ — and a report, because in this domain ‘we deleted it eventually, probably’ is the kind of sentence that ends up in front of a regulator.

The one part where I would consider the database, and I would say it out loud: if the requirement were 500,000 documents of 50 KB each — 25 GB total — with the same consistency demand and no large files, I would seriously consider `bytea` and TOAST, because 25 GB adds a couple of minutes to a backup and removes the entire orphan problem. It is the 100 MB documents and the serving path that make that indefensible here, not the count.”

9 Recall card

Database holds the reference; object storage holds the bytes. The four numbers: **backup/restore** (4 TB means ~6 hours instead of ~2 minutes), **buffer pool** (one 4 MB read evicts ~1,000 row pages, so *other* queries get slower), **connections** (a 50 MB stream holds one for seconds; presigned URLs use zero), and **cost** (~\$0.115/GB vs ~\$0.023/GB, before the bigger instance).

What you give up is atomicity across two systems — orphans and broken links, plus non-atomic deletes. The fix: **create the row first in `PENDING`**, drive completion from the store’s event, and run a reconciliation job where “READY row with no object” is an alert.

The minority answer is real: files under ~100 KB with a few GB total and a genuine consistency requirement — and SQLite, where blobs under 100 KB actually read faster from the database than from files.

Postgres TOAST stores large values out-of-line and compressed and does not read them unless selected — so it weakens the cache argument and **nothing else**. Know that, and you know which arguments survive.

Serving is the argument that settles it: bytes in a database go through your app; bytes in a store go browser → CDN → store, using none of your connections or bandwidth.

DSA topic: Topological sort **System design topic:** Blob storage versus storing files in the database

Code these, in this order

One rule for the whole set: **write the edge's meaning as an English sentence in a comment before writing the append**. “An edge from A to B means A must happen before B.” Then check by hand, on a two-vertex example, which adjacency list gets the entry. Reversing it is silent, and it is the most common failure on this family.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Course Schedule II	LeetCode 210 (Medium)	Kahn's, returning the order, and building from <code>range(n)</code> .
2	Alien Dictionary	LeetCode 269 (Hard)	The modelling: adjacent pairs, first difference, prefix case.
3	Minimum Height Trees	LeetCode 310 (Medium)	Kahn's on an undirected graph, peeling degree-1 leaves.
4	Sequence Reconstruction	LeetCode 444 (Medium)	Is the order unique ? One extra check inside the loop.
5	Parallel Courses III	LeetCode 2050 (Hard)	Longest path in a DAG — one pass in topological order.
6	Sort Items by Groups Respecting Dependencies	LeetCode 1203 (Hard)	Two topological sorts, nested.

On problem 1, write both algorithms

Kahn's and reversed DFS finish order. Verify that both outputs are valid using a property check — for every edge `(a, b)`, `position[a] < position[b]` — rather than comparing them to each other. They will differ.

On problem 1, break it three ways

1. Reverse the edge direction. Record the output and verify it against the property check.
2. Build the adjacency from the edge list instead of `range(n)`, with an isolated vertex present. Record the result and say why it is a phantom cycle.
3. In the DFS version, move the append before the loop. Record the output on `0→1`, `2→1`.

On problem 2, list the three input rules

Before coding: adjacent pairs only, first difference only, and the prefix case. Write each as one line. Then solve it. The prefix case is the hidden test.

On problem 5, notice what changed

The answer is not an order, it is a length. Write down the one-line recurrence and say why processing in topological order means every dependency's answer is already final when you need it.

Then build the parallel scheduler

Take your problem 1 solution and return levels instead of a flat list. Then print:

1. The number of levels — the critical path.
2. The widest level — the maximum useful worker count.
3. The total number of tasks.

Three numbers, and say what each tells a user.

The definition drill

1. Define a topological order in one sentence.
2. Say exactly when one exists.
3. Say why there are usually many, and what that means for tests.
4. Write the five-line property check that verifies any candidate order.

The Kahn's drill

1. Write it from memory.
2. Say what in-degree means, in domain words.
3. Say what the decrement represents.
4. Say what `len(order)  $\neq$  n` means and why the leftovers are stuck.
5. Give four reasons to prefer it over DFS.

The DFS drill

1. Write it from memory, including the colour check.
2. Say where the append goes and why.
3. Say why reversing the finish order gives a valid order.
4. Say what happens without the cycle check.
5. Give the one situation where you would prefer it.

The variants drill

1. Turn Kahn's into a parallel schedule and name the two numbers it produces.
2. Turn it into the lexicographically smallest order and give the cost.
3. Give the one-sentence reason the greedy choice is correct.
4. Add the uniqueness check and say where it goes.
5. Say how you compute the longest path in a DAG with the same pass.

The costs drill

1. Give time and space for Kahn's and for DFS.
2. Give the cost of the heap version and the ratio at $V = 100,000$.
3. Say what counting all valid orders costs, and give an example that explodes.
4. Give the recursion caveat and the input size at which it bites.
5. Compute the critical path win for 1,000 tasks, 5 s each, 12 levels.

The break-it drill

Trigger each and record the exact output or error:

1. Edge direction reversed, verified with the property check.
 2. Adjacency built from edges only, with an isolated vertex.
 3. DFS append before the loop, on $0 \rightarrow 1$, $2 \rightarrow 1$.
 4. DFS with no cycle check, on a triangle.
 5. A test asserting one specific order, with adjacency built in a different order.
 6. Recursive DFS on a 100,000-long chain.
-

The four-numbers drill

1. Compute backup and restore time for 20 GB of rows and 4 TB of blobs, both ways.
2. Compute buffer-pool turnover for a 16 GB pool at 100 blob reads a second.
3. Compute connection occupancy for 200 concurrent 50 MB downloads.
4. Compute monthly cost for 4 TB in each store, including backup storage.
5. Say which of the four is the one that shows up as "the database got slow".

The two-systems drill

1. Name the two problems the external store creates.
2. Give the write order that makes failures recoverable, and say why.
3. Say what happens if you reverse that order.
4. Describe the reconciliation job and its three outputs.

5. Say which of the three is the alarming one and why.

The deletion drill

1. Name the four systems a delete touches.
2. Give the soft-delete-then-clean-up pattern.
3. Say what makes the cleanup job safe to retry.
4. Say what changes when there is a regulatory deletion requirement.

The minority-case drill

1. Give the three situations where the database is the right answer.
2. Give the size and volume thresholds, roughly.
3. Say what TOAST does and which argument it neutralises.
4. Say which arguments TOAST does **not** affect.
5. Give SQLite's benchmark result and why it holds.

The migration drill

1. Give the four steps of migrating 4 TB of blobs out of a database.
2. Say what "null the column" does and does not reclaim.
3. Say what a `VACUUM FULL` on 4 TB costs.
4. Name the four metrics you would measure before and after.

The trade-offs drill

1. Say what object storage costs you, in two words, and expand each.
2. Say what the database costs you at scale — six things.
3. Give the argument that settles it for anything user-facing.
4. Give the rough crossover in terms of blob bytes versus row bytes.
5. Name the case where neither is right.

The failure drill

For each, say what happens and what you would build:

1. The object uploads and the row insert fails.
2. The row is created after the object, and the process dies in between.
3. A `READY` row whose object is missing.
4. A 4 TB database restore during an outage.
5. A hundred concurrent 50 MB downloads through the application.
6. A `SELECT *` on a table with 4 MB blobs, run by a reporting tool.

7. Nulling a blob column on 4 TB and expecting the disk to free up.

Two of the seven are data loss and five are performance. Sort them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Order these tasks so that every dependency comes first.* The edge meaning as a sentence, Kahn's with the in-degree explanation, `len(order) != n` as the cycle test, the four reasons to prefer it, and the parallel-schedule variant offered before being asked.
 2. *Give me a valid course order, and tell me if it is impossible.* One pass, both answers, and the two silent bugs — reversed direction and building from the edge list — named before you write them.
 3. *Should the images go in the database? Defend your answer.* The four numbers, then what the external store costs you — no transaction across two systems — then the ordering rule and the reconciliation job, then where the answer flips.
-

Before you move on

- ☐ I write the edge's meaning as a sentence before the append.
- ☐ I can define a topological order and say exactly when one exists.
- ☐ I verify orders with a property check, not by comparing to a fixed list.
- ☐ I can write Kahn's from memory.
- ☐ I can explain in-degree and the decrement in domain words.
- ☐ I know `len(order) != n` is the cycle test and why.
- ☐ I can give four reasons to prefer Kahn's.
- ☐ I can write the DFS version, including the colour check.
- ☐ I know the append goes after the loop and why.
- ☐ I know what happens to DFS without the cycle check.
- ☐ I can produce a parallel schedule and name its two numbers.
- ☐ I can produce the lexicographically smallest order and justify the greedy choice.
- ☐ I can add the uniqueness check.
- ☐ I know how to compute the longest path in a DAG with one pass.
- ☐ I know counting all orders is exponential.
- ☐ I build from `range(n)` and know it prevents a phantom cycle.

- [] I can compute backup and restore times both ways.
- [] I can explain buffer-pool damage and why it hits *other* queries.
- [] I can compute connection occupancy for large downloads.
- [] I can quote the cost per gigabyte both ways.
- [] I can name the two problems the external store creates.
- [] I know to create the row first, and what reversing that costs.
- [] I can describe the reconciliation job and its alarming output.
- [] I can give the soft-delete-then-clean-up deletion pattern.
- [] I can name the three cases where the database is right.
- [] I know what TOAST fixes and what it does not.
- [] I can give the migration steps and the vacuum caveat.
- [] I know the argument that settles it for user-facing files.
- [] I answered all three questions above out loud.

Course schedule and the dependency family

DATA STRUCTURES AND ALGORITHMS

Course schedule and the dependency family

SYSTEM DESIGN

Search: how a search index actually works

Course schedule and the dependency family

1 What this is, and why they ask it

Yesterday you learned topological sort. Today you learn to recognise it in a problem that never says “graph”, never says “dependency”, and often does not obviously involve ordering at all.

That recognition is the entire skill being tested. **Nobody in an interview asks you to topologically sort a graph.** They ask about courses, or recipes, or the order to rebuild a file system, or which employees can be promoted, or an alien alphabet, or a set of equations that must be evaluated. The algorithm is twenty lines you already have. The gap between hearing the problem and knowing which twenty lines to write is where candidates lose the round.

This lesson is a catalogue: the six shapes this family comes in, the phrasings that signal each, and the two modelling moves that turn an awkward statement into a graph you can run Kahn’s on. It also covers the variants that look like topological sort and are not, because being confidently wrong here is worse than being slow.

By the end of this lesson you can hear a problem statement and name the vertices, the edges and the edge’s meaning in one sentence; recognise all six shapes; handle the two hard modelling cases — deriving edges from comparisons, and dependencies grouped into layers — and know the three near-misses that need something else.

2 The story

Jaya has moved house eleven times, because her husband’s job moves every two or three years, and she no longer thinks about the city at all.

The eleventh was to Guwahati in June, and her sister rang the week before, worried, because Guwahati is far and neither of them knows anybody there. Was it not going to be very complicated?

Jaya said what she always says now, which is that it is exactly the same as the last ten.

Every move looks different from the outside. Different landlord, different school, different bank branch, different gas connection, different set of forms and different people being unhelpful in different ways. Her sister sees eleven different problems.

What Jaya sees, and has seen since about the fourth move, is one list that she rewrites in a different order.

The school will not give the admission without an address proof. The address proof needs the rent agreement. The rent agreement needs a flat chosen and a landlord who will sign. The gas connection needs the address proof too. The bank account transfer needs the address proof. The children's transfer certificates from the old school have to be collected before the new school will even look at the file, and that can be done any-time, from anywhere, weeks earlier.

So on the first day in a new city she does not think about schools, which is what everybody worries about. She thinks about which flat, because six other things are sitting behind it and nothing is sitting behind it.

And she does the transfer certificates before the move, in the old city, because that job needs nothing from the new place at all and there is no reason for it to be in the way later.

Her sister asked how she keeps track of it, and Jaya said there is nothing to keep track of. You look at what can be done today with nothing else needed first. You do those. Then you look again, because doing them has freed up other things. That is all it has ever been.

The eleventh move took nine days. The first one took most of two months, and she remembers standing in a queue at a school in Jabalpur in 2009 being told to come back with an address proof she had no way of getting yet, and only understanding on the bus home that she had been going about it in completely the wrong order.

3 The idea in plain English

Jaya's list is a topological sort, and her sister's mistake is the interview's.

The algorithm is not the hard part. Kahn's is twenty lines and you wrote it [yesterday](#): count in-degrees, queue everything at zero, pop and decrement, and if fewer than `n` come out there is a cycle. Nothing today changes that.

The hard part is hearing "topological sort" in a sentence that does not contain either word. That is what Jaya has and her sister has not: eleven different-looking situations, one shape underneath.

The recognition question, in one line: *are there things, and rules saying one must come before another?* If yes, it is this family, whatever the story is about.

And the two sentences you say before writing anything: “A vertex is ___. An edge from A to B means A must happen before B.” If you can finish both, the code is mechanical. If you cannot, no amount of coding will help.

The six shapes. Almost every problem in this family is one of these:

1. Can it be done at all? — “Can all courses be finished?” You need only the boolean. Kahn’s, and check whether all `n` came out. LeetCode 207.

2. Give me an order. — “In what order should these be installed?” The same pass, returning the list. LeetCode 210.

3. Give me a *specific* order. — smallest lexicographically, or one that satisfies a secondary preference. Same algorithm, a heap instead of a queue.

4. Give me a schedule, not an order. — “What is the minimum time to finish everything?” Process one whole level at a time, and the number of levels is the answer when every task takes the same time. When durations differ, one pass in topological order computing `finish[v] = duration[v] + max(finish of dependencies)` gives the true critical path. **This is the shape most people do not spot**, and it is worth more than the others because it is what real systems need.

5. Is the order unique? — “Can this sequence be reconstructed?” One extra check: the queue must never hold more than one vertex, because two available vertices means two valid orders. LeetCode 444.

6. Derive the edges from something else, then sort. — “Here are words in an unknown alphabet; find the letter order.” The graph is not given; you have to build it from comparisons. This is the hardest of the six and the modelling *is* the problem.

Now the two modelling moves that turn awkward statements into graphs.

Move one: edges come from adjacent pairs, not all pairs. When ordering information arrives as a sorted list, compare each item with the *next* one, and take the **first** position where they differ. That single character pair is one edge. Everything after the first difference tells you nothing, and comparing non-adjacent pairs adds only edges that are already implied. **Both mistakes are common and both produce wrong answers rather than errors.**

Move two: two levels of dependency need two sorts. Some problems have items that depend on items, *and* groups that depend on groups — tasks within projects, courses within departments. The move is to sort the groups among themselves, sort the items within each group, and then concatenate. **Two topological sorts, nested**, and the tell is any problem where things belong to categories that also have an order.

And the three near-misses, which look like this family and are not:

- **Undirected “dependencies”.** If the relation is symmetric — “these two cannot be scheduled together” — that is graph colouring or bipartite checking, not ordering.
- **Weighted shortest path.** “The cheapest order” with costs on the edges is Dijkstra’s territory, not Kahn’s. But note the exception: **on a DAG, longest and shortest path are both one linear pass in topological order**, which is faster than Dijkstra and handles negative weights.
- **Cyclic by nature.** Some real dependency graphs genuinely have cycles — mutually recursive modules — and the answer is not “impossible”, it is to find the strongly connected components, collapse each into one super-vertex, and topologically sort the result. **That collapsed graph is always acyclic**, and it is the right answer when the domain allows cycles.

4 The picture

Jaya’s move, as a DAG:

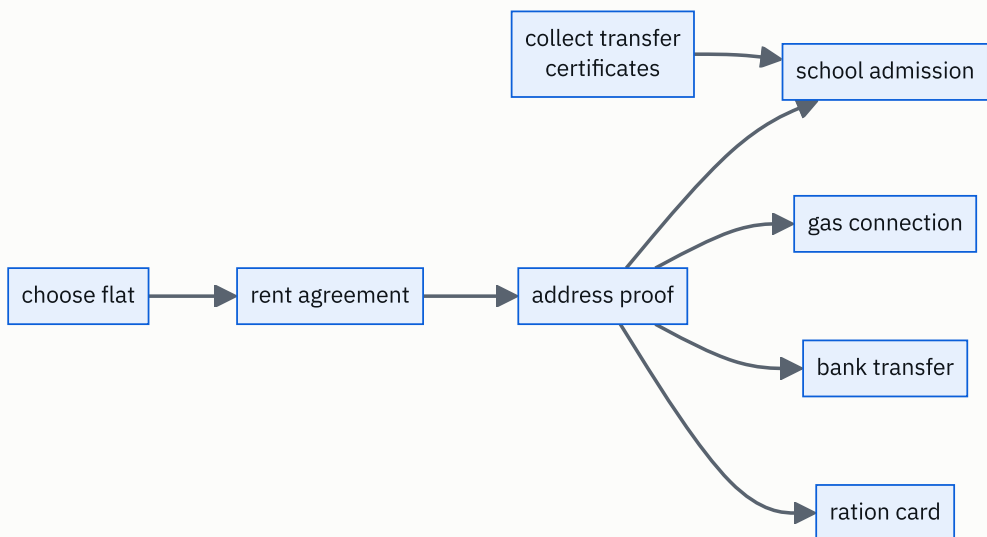


FIGURE 135.1

What to notice. `choose flat` and `collect transfer certificates` both have in-degree zero, so both can start immediately — and the transfer certificates can be done in the *old* city, weeks earlier, which is exactly what “in-degree zero” means in a domain. `address proof` has out-degree four: it blocks the most, so it is the thing to hurry. **In-degree tells you what can start; out-degree tells you what to prioritise.**

The six shapes on one graph:

graph: $A \rightarrow C$, $B \rightarrow C$, $C \rightarrow D$, $C \rightarrow E$, $D \rightarrow F$, $E \rightarrow F$

1. possible? yes (6 of 6 emitted)
2. an order [A, B, C, D, E, F]
3. smallest order [A, B, C, D, E, F] (heap instead of queue)
4. schedule [[A,B], [C], [D,E], [F]] $\rightarrow$ 4 levels, not 6 steps
widest level = 2 $\rightarrow$ two workers is enough
5. unique? NO – the queue held {A,B} and {D,E}
6. (edges given) n/a here

with durations A=2 B=5 C=1 D=4 E=2 F=3:

```
finish[A]=2 finish[B]=5
finish[C]=1+max(2,5)=6
finish[D]=4+6=10 finish[E]=2+6=8
finish[F]=3+max(10,8)=13
critical path = 13, and it runs B  $\rightarrow$  C  $\rightarrow$  D  $\rightarrow$  F
```

What to notice on the last block. With equal durations the answer was 4 levels. With real durations it is 13 units along a specific chain, and that chain — $B \rightarrow C \rightarrow D \rightarrow F$ — is the thing to shorten if you want the whole job faster. **Levels are an approximation; the max recurrence is the real answer.**

And the alien-alphabet modelling, which is where people go wrong:

words (sorted in the unknown alphabet):

"wrt", "wrf", "er", "ett", "rftt"

compare ADJACENT pairs, take the FIRST difference:

wrt / wrf	$\rightarrow$ position 2: t before f	edge t $\rightarrow$ f
wrf / er	$\rightarrow$ position 0: w before e	edge w $\rightarrow$ e
er / ett	$\rightarrow$ position 1: r before t	edge r $\rightarrow$ t
ett / rftt	$\rightarrow$ position 0: e before r	edge e $\rightarrow$ r

result: w $\rightarrow$ e $\rightarrow$ r $\rightarrow$ t $\rightarrow$ f

WRONG version 1: compare all pairs

"wrt" vs "ett" would give w $\rightarrow$ e, which is already implied. Harmless but wasteful.

WRONG version 2: take every differing position

"wrt" vs "wrf" would also give... nothing else, they match until position 2.

But "abc" vs "bad" would give a $\rightarrow$ b AND b $\rightarrow$ a AND c $\rightarrow$ d.

The last two are FALSE. Only the first difference is information.

THE EDGE CASE: ["abc", "ab"]

a prefix cannot come after the longer word. This input is INVALID.

Return "" rather than ignoring it.

5 The code, built step by step

Everything here reuses the same core. Start with the one function the whole family shares.


```

from collections import deque

def kahn(graph: dict, vertices: list) → list | None:
    """A valid order over `vertices`, or None if there is a cycle."""
    in_degree = {v: 0 for v in vertices}
    for vertex in vertices:
        for neighbour in graph.get(vertex, ()):
            in_degree[neighbour] += 1
    queue = deque(v for v in vertices if in_degree[v] == 0)
    order = []
    while queue:
        vertex = queue.popleft()
        order.append(vertex)
        for neighbour in graph.get(vertex, ()):
            in_degree[neighbour] -= 1
            if in_degree[neighbour] == 0:
                queue.append(neighbour)
    return order if len(order) == len(vertices) else None

```

Taking `vertices` as an explicit list rather than deriving it from the graph is what makes this reusable: the caller knows which vertices exist, and a vertex in no edge must still be counted. That is the phantom-cycle bug from yesterday, fixed once.

Shape 1 and 2 are now one-liners.

```

def can_finish(n: int, prerequisites: list[list[int]]) → bool:
    graph = {v: [] for v in range(n)}
    for course, needs_first in prerequisites:
        graph[needs_first].append(course) # needs_first BEFORE course
    return kahn(graph, list(range(n))) is not None

```

Read that append out loud: the input says “to take `course`, first take `needs_first`”, so the edge points **from** the prerequisite. Say it as a sentence, then write the line.

Shape 4, the schedule with real durations, which is the one worth knowing:

```

def minimum_time(n: int, relations: list[list[int]], time: list[int]) → int:
    graph = {v: [] for v in range(n)}
    in_degree = [0] * n
    for before, after in relations:
        graph[before - 1].append(after - 1)
        in_degree[after - 1] += 1

    finish = [0] * n
    queue = deque(v for v in range(n) if in_degree[v] == 0)
    for v in queue:
        finish[v] = time[v] # nothing to wait for
    while queue:
        vertex = queue.popleft()
        for neighbour in graph[vertex]:
            finish[neighbour] = max(finish[neighbour], finish[vertex] + time[neighbour])
            in_degree[neighbour] -= 1
            if in_degree[neighbour] == 0:
                queue.append(neighbour)
    return max(finish)

```

The `max` is the whole algorithm: a task can only start once its *slowest* dependency is done. And the reason this works in one pass is the topological order — **when a vertex is dequeued, every dependency's finish is already final**, because they were all dequeued before it. That sentence is the general principle: **any DP over a DAG is one pass in topological order.**

Shape 5, uniqueness:

```
def order_is_unique(graph: dict, vertices: list) → bool:
    in_degree = {v: 0 for v in vertices}
    for vertex in vertices:
        for neighbour in graph.get(vertex, ()):
            in_degree[neighbour] += 1
    queue = deque(v for v in vertices if in_degree[v] == 0)
    seen = 0
    while queue:
        if len(queue) > 1:
            return False # a choice exists → not unique
        vertex = queue.popleft()
        seen += 1
        for neighbour in graph.get(vertex, ()):
            in_degree[neighbour] -= 1
            if in_degree[neighbour] == 0:
                queue.append(neighbour)
    return seen == len(vertices)
```

Three lines added. Two available vertices at any moment means two valid orders, so the order is not unique.

Shape 6, deriving the edges, which is the hardest modelling:

```
def alien_order(words: list[str]) → str:
    letters = {c for word in words for c in word}
    graph: dict[str, list[str]] = {c: [] for c in letters}

    for first, second in zip(words, words[1:]): # ADJACENT pairs only
        for a, b in zip(first, second):
            if a != b:
                graph[a].append(b) # the FIRST difference only
                break
        else:
            if len(first) > len(second):
                return "" # "abc" then "ab" is invalid
    order = kahn(graph, sorted(letters))
    return "".join(order) if order else ""
```

The `for ... else` is doing real work: `else` runs when the loop finished without `break`, meaning one word is a prefix of the other. If the *longer* one came first, the input contradicts itself and there is no valid alphabet. **That branch is the hidden test.**

The two-level version, for when items belong to ordered groups:

```
def sort_in_groups(items, group_of, item_deps, group_deps):
    """Sort groups among themselves, sort items within each group, concatenate."""
    group_order = kahn(group_deps, sorted(set(group_of.values())))
    if group_order is None:
        return None
    result = []
    for group in group_order:
        members = [i for i in items if group_of[i] == group]
        inner = kahn(item_deps, members)
        if inner is None:
            return None
        result.extend(inner)
    return result
```

Two sorts, nested. A cycle at either level makes the whole thing impossible, and the tell for this shape is any problem where things belong to categories that also have an order between them.

The complete solution

```
"""The dependency family: one Kahn's, six shapes."""

from __future__ import annotations

import heapq
from collections import deque

def kahn(graph: dict, vertices: list) → list | None:
    """Shape 1 and 2: a valid order, or None on a cycle."""
    in_degree = {v: 0 for v in vertices}
    for vertex in vertices:
        for neighbour in graph.get(vertex, ()):
            if neighbour in in_degree:
                in_degree[neighbour] += 1
    queue = deque(v for v in vertices if in_degree[v] == 0)
    order = []
    while queue:
        vertex = queue.popleft()
        order.append(vertex)
        for neighbour in graph.get(vertex, ()):
            if neighbour not in in_degree:
                continue
            in_degree[neighbour] -= 1
            if in_degree[neighbour] == 0:
                queue.append(neighbour)
    return order if len(order) == len(vertices) else None

def smallest_order(graph: dict, vertices: list) → list | None:
    """Shape 3: lexicographically smallest. A heap instead of a queue."""
    in_degree = {v: 0 for v in vertices}
    for vertex in vertices:
        for neighbour in graph.get(vertex, ()):
            in_degree[neighbour] += 1
    heap = [v for v in vertices if in_degree[v] == 0]
    heapq.heapify(heap)
    order = []
    while heap:
        vertex = heapq.heappop(heap)
        order.append(vertex)
        for neighbour in graph.get(vertex, ()):
            in_degree[neighbour] -= 1
            if in_degree[neighbour] == 0:
                heapq.heappush(heap, neighbour)
    return order if len(order) == len(vertices) else None

def critical_path(n: int, edges: list[tuple[int, int]], duration: list[int]) → int:
    """Shape 4: earliest finish for everything. One pass, a max per edge."""
    graph: dict[int, list[int]] = {v: [] for v in range(n)}
    in_degree = [0] * n
    for before, after in edges:
        graph[before].append(after)
        in_degree[after] += 1
    finish = [0] * n
    queue = deque()
    for v in range(n):
        if in_degree[v] == 0:
            finish[v] = duration[v]
            queue.append(v)
    done = 0
    while queue:
        vertex = queue.popleft()
        done += 1
        for neighbour in graph[vertex]:
            finish[neighbour] = max(finish[neighbour], finish[vertex] + duration[neighbour])
```

```

        in_degree[neighbour] -= 1
        if in_degree[neighbour] == 0:
            queue.append(neighbour)
    return max(finish) if done == n else -1

def is_unique(graph: dict, vertices: list) → bool:
    """Shape 5: is there exactly one valid order?"""
    in_degree = {v: 0 for v in vertices}
    for vertex in vertices:
        for neighbour in graph.get(vertex, {}):
            in_degree[neighbour] += 1
    queue = deque(v for v in vertices if in_degree[v] == 0)
    seen = 0
    while queue:
        if len(queue) > 1:
            return False
        vertex = queue.popleft()
        seen += 1
        for neighbour in graph.get(vertex, {}):
            in_degree[neighbour] -= 1
            if in_degree[neighbour] == 0:
                queue.append(neighbour)
    return seen == len(vertices)

def alien_order(words: list[str]) → str:
    """Shape 6: derive the edges from adjacent pairs, then sort."""
    letters = sorted({c for word in words for c in word})
    graph: dict[str, list[str]] = {c: [] for c in letters}
    for first, second in zip(words, words[1:]):
        for a, b in zip(first, second):
            if a != b:
                graph[a].append(b)
                break
    else:
        if len(first) > len(second):
            return ""
    order = kahn(graph, letters)
    return "".join(order) if order else ""

if __name__ == "__main__":
    # Jaya's move.
    tasks = ["tc", "flat", "rent", "addr", "school", "gas", "bank"]
    deps = {
        "tc": ["school"], "flat": ["rent"], "rent": ["addr"],
        "addr": ["school", "gas", "bank"], "school": [], "gas": [], "bank": [],
    }
    print("order      :", kahn(deps, tasks))
    print("smallest:", smallest_order(deps, tasks))
    print("unique? :", is_unique(deps, tasks))

    # Shape 4, with durations.
    n = 6
    edges = [(0, 2), (1, 2), (2, 3), (2, 4), (3, 5), (4, 5)]
    print("critical:", critical_path(n, edges, [2, 5, 1, 4, 2, 3]))

    # Shape 6.
    print("alphabet:", alien_order(["wrt", "wrf", "er", "ett", "rftt"]))
    print("invalid :", repr(alien_order(["abc", "ab"])))
    print("cyclic  :", repr(alien_order(["z", "x", "z"])))

```

Running it:

```

order    : ['tc', 'flat', 'rent', 'addr', 'school', 'gas', 'bank']
smallest: ['flat', 'rent', 'addr', 'bank', 'gas', 'tc', 'school']
unique?  : False
critical: 13
alphabet: wertf
invalid  : ''
cyclic   : ''

```

Three things to look at. `order` and `smallest` differ and both are valid — the smallest one takes `flat` before `tc` because `f` sorts before `t`, and once `addr` is done it takes `bank` and `gas` before going back for `tc`. Check it against the property rule rather than against each other: in both lists, every task appears after everything it depends on. `unique?` is `False`, correctly: `tc` and `flat` are both available at the start, so there was a choice.

`critical` is 13, matching the hand calculation in section 4 — the chain `B → C → D → F` at $5 + 1 + 4 + 3$.

And the last two lines are the shape-6 edge cases: a prefix in the wrong order and a genuine contradiction both return the empty string, and if your version returns `"zx"` for the third one it is not detecting the cycle.

6 What it costs

Everything in this family is one Kahn's pass.

building the graph	one pass over the input	$O(E)$
in-degrees		$O(V + E)$
the main loop	each vertex once, each edge once	

	$O(V + E)$ time, $O(V)$ space	

Per shape:

1. possible?	$O(V + E)$	
2. an order	$O(V + E)$	
3. smallest order	$O((V + E) \log V)$	heap operations
4. schedule / levels	$O(V + E)$	same pass
4b. critical path	$O(V + E)$	one max per edge
5. unique?	$O(V + E)$	one length check per iteration
6. derive + sort	$O(\text{total input chars} + V + E)$	

Shape 3 is the only one that costs more, and it is a `log V` factor:

```
V = 100,000, E = 200,000
plain Kahn's    300,000 steps
heap version    300,000 x 17 = ~5,000,000 steps
```

Shape 6's cost is in the derivation, not the sort:

```
alien dictionary: n words, average length L
building edges    n comparisons, each up to L characters    O(n x L)
the alphabet      at most 26 vertices, at most n-1 edges
the sort itself    O(26 + n) → trivial
```

```
n = 10,000 words of length 8
edge building     80,000 character comparisons
the sort          ~10,000 steps
```

The graph is tiny and the input is not, which is the opposite of most graph problems and worth noticing.

Shape 4's two versions:

```
levels (equal durations)    number of levels = critical path
critical path (real times)    finish[v] = duration[v] + max(finish of deps)
```

```
1,000 tasks, average 5 s, 12 levels, longest chain 8 tasks averaging 9 s
serial total                  5,000 s = 83 min
levels approximation          12 x 5 = 60 s
true critical path            8 x 9 = 72 s
```

Levels under-estimate when durations vary, which is why the `max` version is the one to use when you have real numbers. Both are one pass.

The two-level version:

```
G groups, N items
sort the groups                O(G + group edges)
sort within each group        sum over groups = O(N + item edges)
-----
O(G + N + all edges) → still linear
```

Counting all valid orders, which is the shape people ask about and should not:

```
counting                    exponential (it is #P-complete)
n vertices, no edges        n! orders
n = 12                      479,001,600
bitmask DP                  O(2^n x n), fine up to n ~ 20
```

Space at scale:

```
V = 200,000, E = 500,000
in-degree as a list      ~1.6 MB
adjacency as lists      ~1.6 MB of objects + ~4 MB of entries
queue at peak           up to V
output                  up to V
                        → tens of megabytes, nowhere near a limit
```

And the reason to prefer Kahn's here specifically: none of these six shapes recurses, so a dependency chain 100,000 long is fine. The DFS version would need the iterative conversion for every one of them.

7 The traps

Reversing the edge direction

The near-miss, and the most expensive one, because there is no error:

```
for course, prerequisite in prerequisites:
    graph[course].append(prerequisite)          # backwards
```

```
>>> can_finish(2, [[1, 0]])
True                                     # correct by luck
>>> topological_order(4, [[1,0],[2,1],[3,2]])
[3, 2, 1, 0]                           # every course before its prerequisite
```

The result is a perfectly valid topological order **of the reversed graph**, which is exactly wrong. And for shape 1 the boolean answer is even the same, because a graph has a cycle if and only if its reverse does — so the bug is invisible until shape 2.

The defence: write the meaning as a sentence in a comment, then check on a two-item example by hand.

Building from the pairs instead of the vertex list

```
graph = defaultdict(list)
for a, b in prerequisites:
    graph[a].append(b)
order = kahn(graph, list(graph.keys()))
```

A course with no prerequisites and no dependents never becomes a key:

```
>>> can_finish(5, [[1, 0]])
False                                     # phantom cycle
```


`len(order) == len(vertices)` cannot hold when three vertices are not in the structure. **This reports a cycle on an acyclic graph**, which is the most confusing possible failure and takes ages to find.

Comparing all pairs in the derivation problems

```
for first in words:
    for second in words:
        ...                                # every pair
```

Adds only implied edges — harmless but $O(n^2)$ — and, worse, invites the second mistake:

```
for a, b in zip(first, second):
    if a != b:
        graph[a].append(b)                # no break: EVERY differing position
```

```
>>> alien_order(["abc", "bad"])
''                                         # reports a contradiction
```

`a→b` from position 0 is real. `b→a` from position 1 is **false**, and together they are a cycle. **Only the first difference is information**, because after that the words have already been distinguished.

Missing the prefix case

```
for a, b in zip(first, second):
    if a != b:
        graph[a].append(b)
        break
# no else clause
```

```
>>> alien_order(["abc", "ab"])
'abc'                                     # should be ''
```

`zip` stops at the shorter word, no difference is found, and the invalid input is silently accepted. A longer word cannot sort before its own prefix in any alphabet. **The `for ... else` branch is the hidden test on this problem.**

Using levels when durations vary

```
return len(levels(graph, n))                # "the minimum time"
```

Levels count *rounds*, and a round takes as long as its slowest member:

```
level 0: one task of 100 s and nine tasks of 1 s
levels answer: 1 round
real answer: 100 s
```

The level count is only the answer when every task takes the same time. **With real durations, use the `max` recurrence.**

Assuming a cycle means “impossible”

```
if kahn(graph, vertices) is None:
    return "circular dependency: cannot proceed"
```

For courses, correct. For a module graph where mutual recursion is legal, wrong — the answer is to collapse each strongly connected component into one super-vertex and sort the resulting DAG, which is always acyclic. **Ask whether cycles are an error in this domain or a fact about it**, because the two need different code.

`RecursionError` from the DFS version

```
Traceback (most recent call last):
  File "deps.py", line 22, in visit
    visit(neighbour)
  [Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

Any of these six shapes written with a recursive DFS, on a chain of 100,000 dependencies. Kahn’s does not recurse, which is the fourth reason to default to it.

8 In the interview

How it gets asked

None of these say “graph” or “topological”:

- “Can all courses be finished? Now give me a valid order.”
- “In what order should these packages be installed?”
- “Given a recipe’s steps and which must come before which, how long will it take with three cooks?”
- “Here are some words sorted in an unknown language. What is the alphabet?”
- “Can this sequence be uniquely reconstructed from these subsequences?”
- “Sort these items so that items in the same group are together and all dependencies are respected.”

- “Given a spreadsheet’s formulas, in what order do you recalculate the cells?”

The tell, in one line: things, plus rules that one must come before another.

The first ninety seconds

“This is dependency ordering — a topological sort — and I would name the model before writing anything, because that is where these go wrong.

A vertex is a course. An edge from A to B means A must be taken before B. The input says ‘to take `course`, first take `prerequisite`’, so the edge points **from** the prerequisite to the course. I want to say that as a sentence and check it on a two-course example, because reversing it produces a perfectly valid order of the reversed graph — no error, and for the yes/no version even the same answer, so it is invisible until someone reads the output.

Then Kahn’s: in-degree is how many prerequisites a course still has outstanding. Everything at zero goes into a queue. Pop, output, decrement the in-degree of everything it unlocks, queue anything that reaches zero. **If fewer than `n` come out, the leftovers are in a cycle** — a set of courses each waiting on another — so the same pass answers both questions.

Two setup details. I build the structure from the full list of `n` courses rather than from the prerequisite pairs, because a course with no prerequisites and no dependents must still exist — otherwise the count check fails and I report a cycle that is not there. And I would use Kahn’s rather than a recursive DFS, because a long prerequisite chain overflows the stack and Kahn’s never recurses.

`O(V + E)` time, `O(V)` space.

And before I code, one question: do you want an order, or a schedule? If these courses can be taken in parallel — several per semester — then processing the queue one whole level at a time gives me the number of semesters, which is a much more useful answer than a list. That is a two-line change.”

The follow-ups

“Now some courses take longer than others. Minimum time to finish everything?”

“Then the level count is wrong, and I would say why: levels count rounds, and a round takes as long as its slowest member, so a level containing one ten-week course and nine one-week courses is ten weeks, not one.

The correct version is a single pass in topological order computing, for each course, `finish[c] = time[c] + max(finish of everything it depends on)`. The answer is the maximum finish across all courses.

The reason one pass is enough is the topological order itself: when I dequeue a course, every one of its prerequisites has already been dequeued, so every `finish` I need is already final. Nothing is ever revised.

That is the general principle and I would state it, because it comes up constantly: **any DP over a DAG is one pass in topological order.** Longest path, shortest path, counting paths, earliest start, latest start — all the same shape, differing only in what you accumulate. And it handles negative weights, which Dijkstra does not, because the ordering removes the need to ever reconsider a vertex.

The follow-up to the follow-up is usually ‘which courses are on the critical path’, and that is a backward pass computing the latest each course could start without delaying the end — the difference between earliest and latest is the slack, and zero slack means critical.”

“Here are words in an unknown alphabet. Find the letter order.”

“The sort is routine; the modelling is the problem, and there are three rules I would state before writing anything.

One: compare adjacent pairs only. Non-adjacent pairs give me edges that are already implied by the chain of adjacent ones, so they add cost and no information.

Two: take the first difference only, then stop. Once two words differ at a position, everything after that position tells me nothing about the alphabet — the words are already ordered by that one character. Taking every differing position produces *false* edges: `abc` before `bad` would give `a→b` from position zero, which is true, and `b→a` from position one, which is not, and together they are a cycle that is not there.

Three: the prefix case. If one word is a prefix of the other and the *longer* one comes first — `abc` then `ab` — the input is contradictory in every possible alphabet, and I return empty. `zip` stops at the shorter word so no difference is found and this passes silently. In Python that is the `else` clause on the `for` loop, and it is the hidden test on this problem.

Then Kahn’s over the letters that actually appear. A cycle means the words contradict each other. Multiple valid alphabets are possible and any is acceptable unless they ask for the smallest, in which case it is a heap.

Cost is `O(total characters)` to build and trivial to sort, since the alphabet is at most 26 vertices — which is the unusual thing about this problem: **the input is large and the graph is tiny.**“

“How would you know if there is more than one valid order?”

“Three lines inside the same loop: if the queue ever holds more than one vertex, there is a choice, so the order is not unique.

The reason that is exactly right is that a choice at any point produces at least two distinct valid orders — take either of the two available vertices and both completions are legal. Conversely, if the queue holds exactly one vertex at every step, there was never a decision, so the order is forced.

This is what ‘can the sequence be uniquely reconstructed’ problems are asking, and the framing usually hides it: you are given a set of subsequences that a hidden sequence must satisfy, you build the ordering constraints from consecutive pairs in each subsequence, and then the question is whether the topological order is unique.

The related question is ‘how many valid orders are there’, and I would flag that as a genuinely different problem — it is #P-complete, exponential in general, and n vertices with no edges have $n!$ orders. For small n a bitmask DP over subsets does it in $O(2^n \times n)$, which is fine to about 20.”

“Some of these modules genuinely import each other. Now what?”

“Then a cycle is not an error and ‘impossible’ is the wrong answer, so I would ask that first — whether cycles are a bug in the input or a fact about the domain.

If they are a fact, the move is to **collapse each strongly connected component into a single super-vertex**. A strongly connected component is a maximal set where every member can reach every other one following the arrows, which is exactly a group of mutually dependent modules. Tarjan’s or Kosaraju’s finds all of them in one linear pass.

The graph of components is **always acyclic** — if two components had a cycle between them they would be one component — so I topologically sort *that*, and the result is a build order where each step is ‘compile this group of mutually recursive modules together’. Which is exactly what a compiler does with mutually recursive functions.

That is a genuinely useful answer for build systems and package managers, and it is the honest one for languages where mutual imports are legal. I would not write Tarjan’s from memory unprompted — the low-link bookkeeping is fiddly — but naming it, saying it is one linear DFS, and explaining why the condensed graph is acyclic is the substance.”

The model answer

“A spreadsheet. Cells contain values or formulas referring to other cells. When a cell changes, work out which cells need recalculating and in what order. Handle circular references.”

“This is the dependency family, and it has three parts: the ordering, the *incremental* part, and the cycle handling — and the second is what makes it a real problem rather than a textbook one.

The model. A vertex is a cell. **An edge from A to B means A must be computed before B** — so if $B3 = A1 + A2$, there are edges $A1 \rightarrow B3$ and $A2 \rightarrow B3$. The direction is ‘feeds into’, not ‘depends on’, because I want to traverse forwards from a changed cell to everything affected. I would write that sentence down, because the formula text naturally reads the other way and it is exactly the reversal trap.

The full recalculation is Kahn’s over the whole sheet, and it is $O(V + E)$ where V is the number of cells with formulas and E is the number of references. For a sheet with 50,000 formulas each referring to three cells, that is 200,000 steps — a few milliseconds, so a full recalculation is entirely affordable and is what I would do on file open.

But on every keystroke it is not, and that is the actual design. When one cell changes, only its *descendants* need recomputing. So: a forward traversal from the changed cell to collect the affected set — which is typically a handful of cells, not fifty thousand — and then a topological sort restricted to that subgraph. **The dirty set is usually tiny and the sheet is large**, so this is the difference between instant and laggy, and it is the reason spreadsheets feel fast.

The subtlety is that the restricted sort must still respect dependencies from *outside* the dirty set. A dirty cell may depend on a clean one, which is fine — that value is already final — so when computing in-degrees for the subgraph I only count edges whose source is also dirty. That is one condition and it is easy to get wrong.

Circular references are a fact about spreadsheets, not a bug, because users create them constantly and some are intentional — iterative calculations for goal-seeking. So the answer is not to refuse. It is: detect the strongly connected components; any component with more than one cell, or a cell referring to itself, is a cycle. Then, depending on the mode, either mark those cells with a circular-reference error and compute everything else normally, or, if iterative calculation is enabled, run that component repeatedly until it converges or hits an iteration cap.

The important product decision there is that a cycle in one corner must not break the rest of the sheet. Everything not downstream of the cycle is perfectly computable and should be computed. That falls out of the component approach naturally: collapse each component, sort the condensed DAG, and only the components that are actually cyclic get the error treatment.

What I would store. For each cell, its list of precedents (what it reads) and its list of dependents (what reads it). Both directions, because I need precedents to evaluate and dependents to propagate. Editing a formula means removing the old precedent edges and adding new ones, which is why I keep both lists rather than deriving one from the other on the fly.

The number that justifies it all: a full recalculation of 50,000 formulas is a few milliseconds, but at 60 frames a second while someone drags a fill handle across a thousand cells, that is a thousand full recalculations. The dirty-set version turns that into a thousand tiny ones. **Same algorithm, applied to the right subgraph** — and noticing that the subgraph is the design is what makes this a good answer rather than a recital.”

9 Recall card

The recognition question: things, plus rules that one must come before another. Then two sentences before any code: “A vertex is __” and “An edge from A to B means A must happen before B.”

Six shapes, one Kahn’s: possible? / an order / the *smallest* order (heap) / a **schedule** (levels, or `finish[v] = duration[v] + max(deps)`) / is it **unique**? (queue never holds more than one) / derive the edges then sort.

Two modelling moves: edges come from **adjacent pairs and the first difference only** — every other position is a false edge; and **two levels of dependency need two nested sorts**.

Any DP over a DAG is one pass in topological order — longest path, shortest path, earliest finish, path counts — because every dependency is final by the time you reach a vertex. Handles negative weights, unlike Dijkstra.

Three silent bugs: reversed edge direction (a valid order of the wrong graph, and identical for the boolean version); building from the pairs so isolated vertices vanish (a **phantom cycle**); and using level count as “minimum time” when durations vary.

1 What this is, and why they ask it

Every product has a search box, and almost every one of them starts as `WHERE description LIKE '%camera%'`. That works until it does not, and when it stops working it stops completely: it cannot use an index, so it reads every row of the table for every keystroke.

A **search index** turns the question inside out. Instead of storing documents and scanning them for words, it stores **words** and, for each one, the list of documents containing it. Then “find documents containing camera” is one lookup instead of a scan. That structure is an **inverted index**, and it is what Elasticsearch, Lucene, Solr, Postgres full-text search and Google all use.

They ask this because the naive answer is so tempting and so bad, and because the follow-ups have real substance: how do you handle “cameras” versus “camera”, how do you rank the results, how does the index stay in step with the database, and what do you do when the two disagree. **“Why not just use LIKE?” has a quantitative answer**, and giving it with numbers is the difference between a good answer and a recited one.

Tomorrow is [Elasticsearch in an architecture](#) — where the cluster sits and how it stays fresh. Today is what is inside it.

By the end of this lesson you can explain an inverted index and the analysis pipeline that feeds it, describe how results are ranked, quantify why `LIKE` does not scale, choose between Postgres full-text search and a dedicated engine, and name what search gives up in exchange for what it gives you.

2 The story

The hardware market off Abdul Rehman Street is about four hundred shops in a space you can walk across in ten minutes, and Ismail sits on a stool near the second entrance with a flask of tea.

He does not sell anything. He has not sold anything since 1998, when he closed his own shop. What he does is tell people where to go, and he is extremely good at it.

A man came in one morning looking for brass hinges. Not steel — brass, and a particular size, for a cupboard his father had made.

Ismail did not get up. He said: Farooq, third lane, left side, he will have it. If not, Prakash two doors after him, but he keeps mostly steel and you will have to ask. And there is a man at the far end near the water tap who had brass last year but I do not know about now.

Three names, in that order, in about four seconds.

The man went to Farooq and got them.

What is worth understanding is what Ismail did *not* do, which is what everybody else does. A person who does not know the market walks in and starts at the first shop and asks. Then the second. Then the third. Four hundred shops, and on a bad day you find the thing at shop two hundred and eleven after three hours, and on a worse day you give up.

Ismail has spent twenty-seven years building the other list. Not a list of shops and what each one sells — that is what the shops themselves have, and it is no use to anyone standing at the entrance. His list is the other way round. **For each thing, who has it.** Brass hinges: Farooq, Prakash, maybe the man near the tap. Hinges of any kind: about eleven shops. Screws: nearly everybody, so he asks what size before answering.

And the order matters as much as the names. Farooq first because he definitely has them and he is close. The man near the tap last because it is a long walk and Ismail is not sure. If he read out eleven names in a random order he would be technically correct and completely useless.

The other thing he does, and he does it without thinking about it, is that it makes no difference whether you say hinge, hinges, or brass hinge. Same three shops. He is not matching what you said; he is working out what you meant.

His nephew tried to learn the job in 2016 and lasted about five weeks. He could remember shops. What he could not do was hold the list the other way round, and he kept walking people to the third lane and back.

3 The idea in plain English

Ismail's list is an inverted index, and his nephew's failure is the `LIKE` query.

A forward index is documents to words. Each shop knows its own stock. Each row in your table has a description. That is the natural way to store things and it is useless for search, because answering “who has brass hinges” means asking all four hundred shops.

An inverted index is words to documents. For every word, the list of documents containing it. `hinges → [shop 14, shop 31, shop 208, ...]`. That list is called a **post-ings list**, and looking one up is a single dictionary lookup rather than a scan.

That inversion is the entire idea. Everything else is refinement.

A multi-word query is an intersection. “brass hinges” means fetch the postings list for `brass`, fetch the one for `hinges`, and intersect them. Postings lists are kept **sorted by document id** precisely so that the intersection is a linear merge rather than a hash join — walk both lists together, advance whichever is behind.

The words in the index are not the words the user typed, and that is the second idea. Ismail gives the same answer for “hinge”, “hinges” and “brass hinge”. The index does this with an **analysis pipeline**, which runs on the document when it is indexed **and on the query when it is searched** — and it must be the same pipeline both times, or nothing matches.

The pipeline, in order:

- **Tokenise** — split text into words. Harder than it looks: `wi-fi`, `C++`, `₹1,299`, and languages with no spaces all need decisions.
- **Lowercase** — so `Camera` and `camera` are the same term.
- **Remove stop words** — `the`, `and`, `of`. Optional and now usually skipped, because it breaks phrase queries like “to be or not to be”.
- **Stem or lemmatise** — reduce words to a root, so `running`, `runs` and `ran` all become one term. Stemming is a crude rule-based chop (`cameras` → `camera`); lemmatisation is dictionary-based and slower and more correct.
- **Synonyms**, optionally — map `laptop` and `notebook` to the same term.

Ranking is the third idea, and it is what makes search different from filtering. A database `WHERE` clause gives you a set. Search gives you an **ordered** list, and the order is most of the value — Ismail reading eleven names in a random order would be correct and useless.

The standard ranking function is **BM25**, and its three ideas are worth knowing in words:

- **Term frequency** — a document mentioning “camera” eight times is more about cameras than one mentioning it once. But with **diminishing returns**: the eighth mention adds much less than the second.
- **Inverse document frequency** — a word appearing in every document tells you nothing; a rare word is highly informative. Matching “hinges” is worth far more than matching “the”.
- **Field length normalisation** — a match in a five-word title means more than a match in a two-thousand-word description.

BM25 is TF-IDF with the diminishing returns and length normalisation made explicit, and it has been the default in Lucene since 2016. Saying “BM25, which is TF-IDF with saturation and length normalisation” is enough.

Now why LIKE cannot do this. `WHERE description LIKE '%camera%'` has a leading wildcard, so no B-tree index can be used — the index is sorted by the *start* of the string and you are asking about the middle. So it is a full table scan, comparing every row’s text against the pattern. **It also cannot stem, cannot rank, cannot match multiple words sensibly, and matches inside other words** — “camera” matches “camera-work” and “scameras”.

And the fourth idea: the index is a separate system, so it can be stale. The inverted index is not your database. Something has to keep it in step, and that something is asynchronous, which means there is a window where the two disagree. **Every search design has to answer “how stale can it be, and what happens when a user searches for something they just created?”** — and that is [tomorrow’s](#) lesson.

4 The picture

The inversion, drawn:

FORWARD (what the table has)	INVERTED (what search needs)
doc 1: "brass hinges for cupboard"	brass → [1, 4]
doc 2: "steel screws box"	hinges → [1, 3, 4]
doc 3: "hinges and handles"	cupboard → [1]
doc 4: "brass hinges heavy duty"	steel → [2]
	screws → [2]
	box → [2]
"which docs have brass?"	handles → [3]
forward: read all 4 docs	heavy → [4]
inverted: one lookup → [1, 4]	duty → [4]

What to notice. The forward direction is how the data naturally lives, and answering any query with it costs a pass over everything. The inverted direction costs a build, once, and then every query is a lookup.

A two-word query as a merge:

```
brass  → [1,      4      ]
hinges → [1,  3,  4      ]
           ^      ^
```

```
i=0: 1 = 1 → MATCH, advance both
i=1: 4 vs 3 → 3 is smaller, advance the hinges pointer
i=2: 4 = 4 → MATCH, advance both
```

What to notice. The lists are sorted by document id, and that is not incidental — it is what makes the intersection a linear merge with two pointers, the same two-pointer walk you learned on [day 28](#). An unsorted structure would need a hash set and much more memory.

```
DOCUMENT: "Brass Hinges for Cupboards"
tokenise   → ["Brass", "Hinges", "for", "Cupboards"]
lowercase  → ["brass", "hinges", "for", "cupboards"]
stop words → ["brass", "hinges", "cupboards"]
stem       → ["brass", "hing", "cupboard"]
```

```

QUERY: "cupboard hinge"
tokenise    → ["cupboard", "hinge"]
lowercase   → ["cupboard", "hinge"]
stem        → ["cupboard", "hing"]

```

If the query were NOT stemmed: "hinge" $\neq$ "hing" $\rightarrow$ no match at all.

What to notice. The stemmed forms are not real words — `hing` is nonsense — and that is fine, because nobody looks at them. **The only requirement is that the same pipeline runs on both sides**, and a mismatch there produces a search that returns nothing for obviously correct queries, with no error.

1,000,000 rows, average description 500 bytes

LIKE '%camera%'

read every row	1,000,000 rows
500 MB of text scanned	
at ~1 GB/s	~0.5 s per query
at 100 queries/second	impossible

inverted index

lookup "camera"	one dictionary hit
postings list	~2,000 doc ids
read + rank	~2,000 entries
	~1 ms

5 How it actually works

What a postings list actually holds

Not just document ids. A real postings entry carries enough to rank and to match phrases:

```
term "camera" → [  
  (doc 14, freq 3, positions [2, 47, 91]),  
  (doc 31, freq 1, positions [8]),  
  (doc 88, freq 7, positions [0, 5, 12, 30, 44, 60, 77]),  
  ...  
]
```

- **Frequency** feeds the ranking.
- **Positions** are what make **phrase queries** possible: "brass hinges" as an exact phrase means finding documents where `hinges` appears at position `p + 1` for some position `p` of `brass`. Positions typically double or triple the index size, which is why they can be turned off per field.

Compression, which is why the index is small

Postings lists are the bulk of the index, and two tricks shrink them enormously:

- **Delta encoding.** Store `[14, 31, 88, 91]` as `[14, 17, 57, 3]` — the gaps. Gaps are small numbers even when document ids are large.
- **Variable-byte or bit-packed integers.** A gap of 17 needs one byte, not four.

Together these routinely get postings down to **one or two bytes per entry**, which is what makes an index over hundreds of millions of documents fit on ordinary machines.

Segments, and why deletes are strange

Lucene — the library under Elasticsearch and Solr — writes **immutable segments**. A batch of documents is analysed, indexed and written as a self-contained little index, and then never modified.

Consequences worth knowing:

- **A search queries every segment and merges the results.** More segments means slower searches.
- **A background merge process** combines small segments into larger ones.
- **An update is a delete plus an insert.** The old document is marked deleted in a bit-map; the space is only reclaimed at the next merge.
- **A document is not searchable until its segment is written.** Elasticsearch calls this the **refresh interval**, and it defaults to **one second**. That is the source of “I created it and it does not appear in search yet”, and it is a setting rather than a bug.

Immutability is the same idea as the LSM tree you met on [day 31](#) — write-once files plus background compaction — and for the same reason: it makes writes sequential and reads predictable.

Ranking, concretely

BM25's shape, without the full formula:

```
score(doc, query) = sum over query terms of:
    IDF(term)           rare terms are worth more
    x TF saturation      more mentions help, with diminishing returns
    x length normalisation shorter fields score higher
```

Two parameters: `k1` controls how fast term frequency saturates (default ~1.2), `b` controls how strongly length is normalised (default 0.75). **You almost never tune these; you tune field boosts instead** — “a match in the title is worth three times a match in the body” is a much more effective lever and is what product teams actually change.

And relevance is a product problem, not only a technical one. Real systems layer business rules on top: boost in-stock items, boost recent content, demote low-rated sellers, personalise by past behaviour. **Saying that ranking is where the product lives is a strong thing to say**, because a technically perfect BM25 that ranks out-of-stock items first is a worse product than a crude ranking that does not.

Postgres full-text search

Postgres has this built in, and for many systems it is the right answer.


```

ALTER TABLE products ADD COLUMN search_vector tsvector
    GENERATED ALWAYS AS (
        setweight(to_tsvector('english', coalesce(title, '')), 'A') ||
        setweight(to_tsvector('english', coalesce(description, '')), 'B')
    ) STORED;

CREATE INDEX products_search_idx ON products USING GIN (search_vector);

SELECT id, title, ts_rank(search_vector, query) AS rank
FROM products, plainto_tsquery('english', 'brass hinges') query
WHERE search_vector @@ query
ORDER BY rank DESC
LIMIT 20;

```

`to_tsvector` is the analysis pipeline — tokenise, lowercase, stop words, stem. `GIN` is a genuine inverted index. `setweight` gives you title-versus-body boosting. `@@` is the match operator.

What you get: real search, in your existing database, **transactionally consistent with your data** — no sync, no staleness, no second system. That last property is worth a great deal and is the main reason to choose it.

What you do not get: distributed scaling beyond one machine, BM25 by default (`ts_rank` is cruder), built-in fuzzy matching and typo tolerance, faceted aggregations, and the operational tooling of a dedicated engine.

The rule of thumb: under a few million documents with modest query rates, **Postgres full-text search, and be glad you have one fewer system.** Beyond that, or when relevance features matter, a dedicated engine.

Trigram indexes, for the case that is not really search

`pg_trgm` indexes every three-character sequence, which makes `LIKE '%camera%'` and fuzzy matching genuinely indexable:

```

CREATE EXTENSION pg_trgm;
CREATE INDEX ON products USING GIN (title gin_trgm_ops);
-- now LIKE '%camera%' can use an index

```

This is the right answer for autocomplete on names, for typo tolerance, and for substring matching — cases where word-based search is the wrong model. It is not a replacement for full-text search; it is a different tool that happens to solve the `LIKE` problem specifically.

6 The numbers

Why **LIKE** does not scale:

```
1,000,000 rows, 500 bytes of text each = 500 MB
LIKE '%camera%' → sequential scan of all of it
at ~1 GB/s                                ~500 ms per query
at 10 queries/second                       5 CPU-seconds per second → saturated
```

```
inverted index
lookup                                ~1 dictionary hit
postings list for a common term        ~2% of docs = 20,000 entries
decode + score + top 20                ~20,000 operations
                                       ~2 ms
```

Two hundred and fifty times faster, and it gets better as the corpus grows, because the postings list grows linearly while the scan grows linearly too — but from a base that is already hundreds of times larger.

Index size:

```
1,000,000 documents, average 100 words, 20,000 distinct terms

postings entries      1,000,000 x 100 = 100,000,000
                      (each word occurrence is one entry)
raw, 4 bytes per doc id 400 MB
delta + varint encoded ~1.5 bytes each = 150 MB
with positions         ~2-3x           = 300-450 MB
```

```
original text      1,000,000 x 500 B = 500 MB
index without positions ~150 MB      = 30% of the source
index with positions  ~400 MB        = 80% of the source
```

A useful rule: a search index is roughly 20-50% of the source text without positions, and can approach 100% with them. If someone says “the index will be tiny”, that is the number to correct them with.

Query cost by term rarity:

```
rare term ("hinges")    postings list ~2,000 docs    ~0.5 ms
common term ("the")     postings list ~950,000 docs ~200 ms
```

Which is exactly why stop words used to be removed — and why modern engines instead process the *rarest* term first in a multi-term query, so the intersection shrinks immediately.

```
"the brass hinges"
start with hinges (2,000) → intersect with brass (30,000) → 400
start with the (950,000) → intersect with brass → 28,000 → then hinges → 400
same answer, 50x more work
```

Refresh and staleness:

```
Elasticsearch default refresh interval 1 s
→ a document is searchable ~1 s after indexing
raising it to 30 s fewer segments, faster indexing,
30 s of staleness
```

Indexing throughput:

```
Elasticsearch, one node, bulk indexing 10,000 - 50,000 docs/s
Postgres tsvector GIN, single inserts ~5,000 docs/s
(GIN updates are expensive)
```

Postgres versus a dedicated engine, sized:

```
Postgres full-text search
works well to ~1-5 million documents
query latency 5-50 ms
cost zero extra systems

Elasticsearch
works to billions, sharded
query latency 5-50 ms at any size
cost 3-node minimum cluster,
~1.5x the source data in RAM for good performance
```

The RAM figure is the one people miss: search performance depends heavily on the index being in the page cache, so a 400 GB index wants a cluster with a comparable amount of memory, and that is the real cost of a large search deployment.

7 The trade-offs

A search index is a second copy of your data, and second copies go stale. Between the write to the database and the update to the index there is a window — a second by default, minutes if the pipeline backs up, hours if it breaks. **Every search design owes an answer to “what does a user see immediately after creating something?”** and the usual answer is to read that one item from the database directly rather than from search.

Search is not a transaction. You cannot index a document and update a row atomically. So the index can contain deleted items, miss new ones, or hold a stale version. Mitigations are a reconciliation job and filtering search results against the database

before display — and the second one is why search should return *ids* that you then hydrate from the source of truth.

Ranking is subjective and never finished. BM25 is a starting point. Every real system layers business rules on top, and those rules are a product decision that changes constantly. **A system with no way to measure relevance cannot improve it**, so click-through rate on results and manually judged query sets are part of the design, not an afterthought.

More index features cost storage and indexing time. Positions for phrase queries double or triple the index. Trigrams for fuzzy matching multiply it again. Storing every field as both searchable and sortable doubles it. **Index only what you will query on**, and this is where most bloated search clusters come from.

A dedicated engine is a distributed system to run. Three nodes minimum, shard planning, memory sizing, version upgrades, and a whole class of failure modes. **For under a few million documents, Postgres full-text search removes all of that and stays transactionally consistent**, which is a genuine advantage rather than a compromise.

And `LIKE` is not always wrong. On a table of a few thousand rows, a sequential scan takes a millisecond and any index is over-engineering. The failure is not using `LIKE`; it is using it at a million rows and being surprised.

When would I not build search at all? When the user knows exactly what they want and filters would serve better — a dropdown of categories beats a search box for structured choices. When the corpus is small enough to scan. And when the real requirement is *lookup by identifier*, where an ordinary index is correct and a search engine is a strange way to spell `WHERE id = ?`.

8 In the interview

How it gets asked

- “How does full-text search work? Why not just use `LIKE`?” — the direct version.
- “Design search for a product catalogue.”
- “Why does searching for ‘cameras’ not find the item titled ‘camera’?” — the analysis question.
- “How would you rank the results?”
- “Elasticsearch or Postgres?”
- “A user creates a listing and cannot find it in search. What is happening?”

The first ninety seconds

“`LIKE '%camera%'` has a leading wildcard, so no B-tree index can help — the index is sorted by how strings *start* and the query is about the middle. So every query is a full table scan. On a million rows with 500-byte descriptions that is 500 MB of text read per query, roughly half a second, and at ten queries a second the database is saturated. It also cannot stem, cannot rank, and matches inside other words — ‘camera’ matches ‘camerawork’.

The fix is to invert the storage. Instead of documents containing words, store **words containing documents**: for each term, a sorted list of the document ids that contain it — a postings list. Then ‘find documents with camera’ is one lookup, and a two-word query is a linear merge of two sorted lists, not a scan.

The second half is the analysis pipeline, and it is what makes ‘cameras’ find ‘camera’. Tokenise, lowercase, optionally drop stop words, and stem to a root form — and crucially, **the same pipeline runs on the document at index time and on the query at search time**. If they differ, you get a search that returns nothing for obviously correct queries and reports no error.

The third is ranking, which is what makes it search rather than filtering. A `WHERE` clause gives a set; search gives an ordered list, and the order is most of the value. BM25 is the standard — essentially TF-IDF with two refinements: term frequency saturates, so the eighth mention adds much less than the second, and matches in shorter fields count for more.

Index size is worth knowing: roughly 20 to 50 percent of the source text without positions, and up to about 100 percent with them — positions are what phrase queries need.

And the thing I would raise before you ask: the index is a second copy, so it can be stale. Elasticsearch refreshes once a second by default, which is where ‘I just created it and cannot find it’ comes from.

How many documents, and is this Postgres already?”

The follow-ups

“Elasticsearch or Postgres full-text search?”

“Postgres, unless there is a reason — and I would give the reason rather than treat it as a default.

Postgres has a genuine inverted index: `tsvector` for the analysed form, a GIN index over it, `setweight` for field boosting, and `ts_rank` for ranking. A generated column keeps the vector in step with the row **inside the same transaction**, which is the big win: no sync pipeline, no staleness, no second system to operate, and no window where the index and the data disagree. For a catalogue of a few million products with modest query rates, that is the right engineering choice and it removes an entire class of problem.

I would move to Elasticsearch for four reasons, any one of which is sufficient. **Scale** — beyond roughly five million documents, or when the index no longer fits comfortably in one machine’s memory, because search performance is largely about the index being cached. **Relevance features** — proper BM25, typo tolerance, synonyms, phrase boosting, learning-to-rank. **Aggregations** — faceted navigation, ‘show me counts by brand and price band alongside the results’, which Postgres can do but not at speed. And **query volume**, where I want search traffic off the primary database entirely.

The cost of moving is a three-node cluster, shard planning, memory roughly comparable to the index size, and a sync pipeline with all the staleness and reconciliation that implies. **That last one is not a small cost** and it is the thing I would weigh hardest.”

“A user creates a listing and cannot find it in search.”

“Expected, and I would explain it as a design property rather than a bug — then say what I do about it.

Lucene writes immutable segments. A document is not searchable until its segment is written, and Elasticsearch’s default refresh interval is one second. So there is a floor of about a second, and if the indexing pipeline is a queue that has backed up, it can be minutes.

Three responses depending on what the product needs.

For ‘my own listing’, do not use search at all. After creating something, the user’s next view should read it from the database by id. That is the source of truth, it is instantly consistent, and it removes the problem rather than tuning it.

For ‘my listings’ lists, read from the database, not the index. Anything scoped to one user is a straightforward indexed query and does not need search.

For genuinely global search, accept the second and set expectations — and monitor the actual lag, not the configured refresh interval, because the real number is refresh plus queue depth plus indexing time. I would alert on the age of the oldest unindexed document, exactly like the consumer lag metric from [day 130](#).

What I would not do is call `refresh` after every write to force it. That creates a segment per document, the segment count explodes, merges thrash, and query performance falls off a cliff. It is the fix that makes everything worse.”

“How would you rank the results?”

“BM25 as the base, then business rules on top, and I would be clear that the second part is where the product actually lives.

BM25 in words: rare terms count for more than common ones; more occurrences help but with diminishing returns; and a match in a short field counts for more than one in a long field. Two parameters control the saturation and the length normalisation, and **I would not tune them** — the effective lever is field boosting, saying a title match is worth three or four times a description match, and that is one line of configuration with a large effect.

Then the layer that matters commercially. In-stock items above out-of-stock ones. Recent above stale, if recency matters in this domain. Highly rated sellers above poor ones. Possibly personalisation from past behaviour. **A technically perfect BM25 ranking that puts out-of-stock items first is a worse product than a crude ranking that does not**, and I would say that explicitly, because it is the thing engineers under-weight.

And I would insist on measurement, because relevance cannot be improved without it. Click-through rate on the top results, the position of the result the user actually clicked, and the rate of searches with no clicks at all — that last one is the best single signal that search is failing. Plus a judged set of maybe two hundred queries with hand-marked correct answers, run on every ranking change, so a ‘small tweak’ cannot silently break the common cases.”

“How do you keep the index in sync with the database?”

“Three options, and I would pick based on how much staleness is acceptable.

Dual write — the application writes to the database and to the index. Simplest, and wrong, because there is no transaction across the two: a crash in between leaves them permanently inconsistent, and it will happen. I would name it to reject it.

Outbox plus a consumer. The application writes the row and an outbox record in the same transaction; a consumer reads the outbox and updates the index. That is the pattern from [day 121](#) and it is what I would build: the write is atomic, delivery is at-least-once, and indexing is idempotent because it is keyed by document id.

Change data capture — read the database’s replication log directly with something like Debezium. No application changes at all, catches writes from anywhere including manual ones, and it is the more robust option at the cost of another moving part.

And whichever I use, a reconciliation job, because all three drift. Nightly, compare counts and checksums per shard or per id range, and re-index the differences.

This is not optional — the index will drift from the database over months, through failed consumers, replayed events and bugs, and the only way to know is to check.

One design detail that makes all of this safer: **search returns ids, and the application hydrates them from the database.** Then a stale index shows a slightly wrong set of results, but never wrong *content*, and deleted items are filtered out at hydration time rather than being served.”

The model answer

“Design search for a marketplace with 50 million listings. Users search by keywords, filter by category and price, and sort by relevance or price. New listings must be findable quickly.”

“Fifty million documents settles the engine question, so let me start there and then spend most of the time on the parts that are actually decisions.

Elasticsearch, not Postgres full-text search. At fifty million documents the index will be tens of gigabytes and wants to be resident in memory for good latency; that is beyond what I want a primary transactional database doing, and the faceted filtering — counts by category and price band shown alongside results — is something a dedicated engine does well and Postgres does not.

What goes in the index, and this is where clusters get bloated. Title and description analysed for search. Category, price, location and seller rating as filterable fields. Nothing else. **In particular I would not store the full listing body in the index** — search returns listing ids, and the application hydrates them from the database. That keeps the index small, and it means a stale index can show a slightly wrong result *set* but never wrong *content*, and a deleted listing is filtered out at hydration.

Analysis: standard tokenising, lowercase, English stemming, and a synonym list maintained by the product team — ‘mobile’ and ‘phone’, ‘cycle’ and ‘bicycle’. The same analyser on query and document, and I would put a test in CI that asserts a handful of query-document pairs still match, because an analyser change that breaks matching produces zero errors and a catastrophe.

Sharding: with fifty million documents and growth, maybe ten to twenty shards, sized so each is under about 30 GB, which is the range where recovery and merges stay manageable. **Shard count cannot be changed without reindexing**, so I would over-provision slightly rather than plan to migrate. Replicas of one for availability and read throughput.

Sync via the outbox pattern. The listing write and an outbox row in one transaction; a consumer bulk-indexes in batches of a few hundred, which is far more efficient than per-document indexing. Refresh interval left at one second so new listings are findable in about a second — and I would explicitly not lower it, because forcing refreshes per document destroys performance.

The ‘findable quickly’ requirement needs a product answer as well as a technical one. After posting, the seller is shown their listing from the database directly, and ‘my listings’ reads from the database. So the one-second window is invisible to the person who cares most about it. That is a better answer than trying to make search instantly consistent.

Ranking: BM25 with the title boosted about three times over the description, then business rules — recency matters a lot in a marketplace, so a decay on listing age; seller rating as a modest boost; and sold or expired listings excluded by a filter

rather than demoted, because showing them at all is a bad experience. Sorting by price is a filter-and-sort, not a relevance query, and I would make sure the UI is clear about which mode the user is in, because a price-sorted result set that also applies relevance ranking confuses everyone.

Monitoring: query latency at p99, the age of the oldest unindexed document from the outbox, the zero-result-search rate, and the click-through position. **The zero-result rate is the one I would put in front of the product team weekly**, because it is the clearest signal that search is failing users and it is usually fixable with synonyms rather than engineering.

And the reconciliation job, nightly: compare document counts and a checksum per id range between the database and the index, and re-index the differences. Over a year the index *will* drift — a consumer that died, a bad deploy, a bulk import that bypassed the outbox — and this is the only mechanism that finds it.”

9 Recall card

An inverted index maps terms to sorted lists of document ids (postings), so a query is a lookup and a multi-word query is a linear merge of sorted lists. `LIKE '%x%'` cannot use a B-tree at all — leading wildcard — so it scans everything: ~500 ms per query on a million rows against ~2 ms.

The analysis pipeline must be identical at index time and query time: tokenise, lowercase, stop words, stem. A mismatch returns nothing for correct queries, with no error.

Ranking is what makes it search rather than filtering. BM25 = rare terms count more (IDF), term frequency **saturates**, shorter fields count more. Tune field boosts, not the parameters — and layer business rules on top, because that is where the product is.

The index is a second copy and it goes stale. Elasticsearch refreshes ~1 s by default. Sync via the **outbox** or CDC, never dual write; **search returns ids and you hydrate from the database**; and run a reconciliation job, because it will drift.

Postgres full-text search (`tsvector` + GIN) up to a few million documents — transactionally consistent, one fewer system. Beyond that, or for facets, typo tolerance and real BM25, a dedicated engine — and budget memory comparable to the index size.

PRACTICE

Practice — Course schedule and the dependency family

DSA topic: Course schedule and the dependency family **System design topic:** Search: how a search index actually works

Code these, in this order

One rule for the whole set: **write the two sentences at the top of every file, as a comment.** “A vertex is __.” “An edge from A to B means A must happen before B.” Then, before writing the loop, work out by hand on a two-item example which adjacency list gets the entry. Every problem below can be silently reversed.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Course Schedule	LeetCode 207 (Medium)	Shape 1, and the reversal that is invisible here.
2	Course Schedule II	LeetCode 210 (Medium)	Shape 2, where the reversal finally becomes visible.
3	Parallel Courses III	LeetCode 2050 (Hard)	Shape 4 with real durations — the <code>max</code> recurrence.
4	Sequence Reconstruction	LeetCode 444 (Medium)	Shape 5, uniqueness, and building edges from subsequences.
5	Alien Dictionary	LeetCode 269 (Hard)	Shape 6 — adjacent pairs, first difference, prefix case.
6	Sort Items by Groups Respecting Dependencies	LeetCode 1203 (Hard)	Two nested sorts.

On problems 1 and 2, prove the reversal is invisible

Solve problem 1 with the edges deliberately reversed. Record the answer — it will be correct, because a graph has a cycle exactly when its reverse does. Then run the same reversed build on problem 2 and check the output with the property test. Write one sentence on why the bug hid in the first problem and not the second.

On problem 3, compare levels against the recurrence

Solve it once by counting levels (pretending all durations are equal) and once with `finish[v] = duration[v] + max(finish of deps)`. Construct an input where they differ by a factor of ten, and say what property of the input causes the gap.

On problem 5, write the three rules first

Adjacent pairs only. First difference only. The prefix case. Write them as three comment lines, then solve. Then break each one on purpose and record what each produces:

1. Compare all pairs — what changes?
2. Take every differing position — find an input where this invents a cycle.
3. Drop the `for ... else` — find the input that now passes and should not.

On problem 6, name the two graphs

Before coding, write down what the vertices and edges are for the outer sort and for the inner sorts. Then say what happens if either has a cycle.

Then the property-test habit

Write this once and reuse it for every problem above:

```
def is_valid_order(order, edges):  
    position = {v: i for i, v in enumerate(order)}  
    return all(position[a] < position[b] for a, b in edges)
```

Use it instead of comparing your output to a fixed list. That is the only correct way to test this family.

The recognition drill

1. State the recognition question in one line.
2. Give the two sentences you say before any code.
3. Name the six shapes and give a problem for each.
4. Give the phrasing that signals each shape.
5. Name the three near-misses and what each actually needs.

The modelling drill

1. State the adjacent-pairs rule and why non-adjacent pairs add nothing.
2. State the first-difference rule and give an input where ignoring it invents a cycle.
3. Give the prefix case and say what must be returned.
4. Describe the two-level pattern and its tell.

The shapes drill

Write each from memory:

1. Shape 1, the boolean.

2. Shape 2, the order.
3. Shape 3, the smallest order.
4. Shape 4, levels — and then shape 4 with durations.
5. Shape 5, uniqueness.
6. Shape 6, deriving the edges.

The DAG-DP drill

1. State the principle in one sentence.
2. Say why one pass is enough, referring to the ordering.
3. Give the recurrence for the longest path.
4. Give the recurrence for counting paths.
5. Say why this beats Dijkstra on a DAG, including the negative-weight point.

The costs drill

1. Give the cost of each of the six shapes.
2. Say which one costs more and by what factor at `V = 100,000`.
3. Give the cost of the alien-dictionary derivation and say what is large and what is small.
4. Say what counting all valid orders costs and give an exploding example.

The break-it drill

Trigger each and record the exact output or error:

1. Reversed edges on shape 1, then on shape 2.
 2. Adjacency built from the pairs, with an isolated vertex.
 3. Comparing all pairs, then taking every differing position, on `["abc", "bad"]`.
 4. `["abc", "ab"]` with no `for ... else`.
 5. Level count used as “minimum time” with one 100-second task among nine 1-second tasks.
 6. A recursive DFS version on a 100,000-long chain.
-

The inversion drill

1. Define a forward index and an inverted index.
2. Say what a postings list is and why it is sorted.
3. Show a two-term query as a merge and give its cost.
4. Say what else a postings entry holds besides the doc id, and what each is for.

The LIKE drill

1. Say why `LIKE '%x%'` cannot use a B-tree.
2. Compute the scan cost for a million rows of 500 bytes.
3. Compare with an inverted index lookup and give the ratio.
4. Name three things `LIKE` cannot do besides being slow.
5. Say when `LIKE` is actually fine.

The analysis drill

1. Name the five stages of the pipeline in order.
2. Say what stemming does and give an example of a nonsense stem.
3. State the rule about where the pipeline must run, and what breaks otherwise.
4. Say why stop-word removal is now usually skipped.
5. Say what happens if you stem documents and not queries.

The ranking drill

1. Give BM25's three ideas in words.
2. Say what "saturation" means and why it matters.
3. Say what you would tune instead of the parameters.
4. Say why ranking is a product problem, with an example.
5. Name four things you would measure to know whether search is working.

The staleness drill

1. Say why the index can be stale, mechanically.
2. Give Elasticsearch's default refresh interval and what it causes.
3. Give three responses to "I created it and cannot find it".
4. Say what forcing a refresh per document does.
5. Say what you alert on, and why it is not the configured interval.

The sync drill

1. Name the three sync options and reject one with a reason.
2. Describe the outbox version and what makes it safe.
3. Say what change data capture buys and costs.
4. Say why a reconciliation job is not optional.
5. Say why search should return ids rather than content.

The numbers drill

1. Compute index size for a million documents at 100 words each, with and without positions.
2. Give the index-size rule of thumb as a percentage of source text.
3. Compare query cost for a rare term and a common term.
4. Explain the rarest-term-first optimisation with the arithmetic.
5. Give indexing throughput for Elasticsearch and for Postgres GIN.
6. Say what memory a large index wants and why.

The choose-one drill

1. Write the Postgres full-text setup from memory: column, index, query.
2. Say what Postgres gives you that a dedicated engine does not.
3. Give four reasons to move to Elasticsearch, any one sufficient.
4. Say what `pg_trgm` is for and why it is a different tool.
5. Give the document-count rule of thumb.

The failure drill

For each, say what happens and what you would build:

1. Documents are stemmed and queries are not.
2. A user posts a listing and searches for it half a second later.
3. Someone calls refresh after every indexed document.
4. The indexing consumer dies for six hours.
5. A search returns a listing that was deleted an hour ago.
6. An analyser change is deployed without reindexing.
7. `LIKE '%camera%'` on a table that grew from 5,000 rows to 5,000,000.

Two of the seven produce wrong results with no error at all. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Can all courses be finished? Now give me a valid order.* The two modelling sentences, the direction checked by hand, Kahn's with in-degree in domain words, `len(order) != n` as the cycle test, and building from the full vertex list — with the phantom-cycle reason.

2. *Here are words in an unknown alphabet. Find the letter order.* The three derivation rules stated before any code, why the second one invents cycles when broken, the prefix case as the hidden test, and the observation that the input is large and the graph is tiny.
3. *How does full-text search work? Why not just use LIKE?* The leading-wildcard scan with numbers, the inversion, the analysis pipeline on both sides, BM25's three ideas, and staleness raised before it is asked.
-

Before you move on

- [] I write the two modelling sentences before any code.
- [] I check the edge direction by hand on a two-item example.
- [] I know why the reversal is invisible on the boolean version.
- [] I can name all six shapes and a problem for each.
- [] I can name the three near-misses and what each needs.
- [] I know the adjacent-pairs and first-difference rules and why each matters.
- [] I handle the prefix case with a `for ... else`.
- [] I can write the two-level nested sort.
- [] I can state the DAG-DP principle and say why one pass suffices.
- [] I know levels under-estimate when durations vary.
- [] I test with a property check, never a fixed list.
- [] I build from the full vertex list, not the pairs.
- [] I use Kahn's rather than recursion, and can say why.
- [] I can define forward and inverted indexes.
- [] I know why postings lists are sorted and what that buys.
- [] I can explain why `LIKE '%x%'` cannot use an index.
- [] I can compute the scan cost and the index cost and give the ratio.
- [] I can name the five analysis stages in order.
- [] I know the pipeline must be identical on both sides, and what breaks otherwise.
- [] I can give BM25's three ideas in words.
- [] I know to tune field boosts rather than parameters.
- [] I can name four search metrics worth measuring.
- [] I know why the index is stale and what the default interval is.
- [] I know the three answers to "I cannot find what I just made".
- [] I know what forcing a refresh per document does.

- ☐ I can name the three sync options and reject dual write with a reason.
- ☐ I know why search should return ids and not content.
- ☐ I can size an index as a percentage of source text.
- ☐ I know when Postgres full-text search is the right answer.
- ☐ I answered all three questions above out loud.

Dijkstra's algorithm

DATA STRUCTURES AND ALGORITHMS

Dijkstra's algorithm

SYSTEM DESIGN

Elasticsearch in a design

1 What this is, and why they ask it

Dijkstra's algorithm finds the cheapest path from one vertex to every other, when the edges have costs. It is BFS with one substitution: instead of a queue, which hands back whatever arrived first, you use a **priority queue**, which hands back whatever is cheapest so far.

That single change is the whole algorithm. Everything else — the distance map, the relaxation, the early exit — you already have from [day 127](#).

They ask it constantly, and the reason is that the answer has two halves and most candidates give only one. Writing Dijkstra is twenty lines. **Saying why the greedy choice is correct**, and **naming the exact assumption that makes it correct**, is the half that separates people. The assumption is that every edge cost is non-negative, and if you cannot say what breaks without it, you have memorised an algorithm rather than understood one.

There is also a specific implementation detail — what to do when you find a shorter route to a vertex already sitting in the heap — that has a right answer, a wrong answer that is slow, and a wrong answer that crashes.

By the end of this lesson you can write it from memory with the lazy-deletion pattern, prove the greedy choice in three sentences, give its cost with both heap types, reconstruct the path, and name the four situations where it is the wrong tool.

2 The story

Farhan has been delivering documents on a motorbike for eleven years, and he does not think about the city in kilometres.

He thinks in minutes, and the two are not the same thing at all. The route from the office to Lower Parel that looks direct on the map goes through the market, and between eleven and two that is forty minutes for three kilometres. The long way round by the fly-over is eleven kilometres and it is fifteen minutes.

New riders take the short one, every time, for about the first month.

What Farhan has in his head is a set of small facts. Not routes — pairs. From the office to the signal is four minutes. Signal to the bridge, seven. Bridge to Lower Parel, five. Office to the market entrance, two. Market entrance to Lower Parel, thirty-eight. Two hundred or so of these, built up over years.

When somebody hands him an address he has not been to, he does something quite specific and he does it fast.

He starts from where he is and looks at the places he can reach directly. The nearest is two minutes away, so he takes that one first — not because it is on the way, but because two minutes is the smallest number he has. He has now settled that place: whatever else he discovers, no route to it can beat two minutes, because everything else starts at four minutes or more.

Then from there, and from everywhere else he has settled, he looks at what is newly reachable and picks the smallest total again. Four minutes to the signal. Then two plus five is seven to the flyover entrance. Then four plus seven is eleven to the bridge.

He works outwards by *total time from the start*, never by distance and never by the order the roads happen to run in, and he stops the moment the address he wants comes up as the smallest remaining number.

His nephew, who joined last year, asked him why he does not just follow the shortest route on the phone and Farhan said something that took the boy a while to understand: the phone is doing the same thing, it just has more numbers than he does.

The one thing that would break his method — and this is the part that has never come up, because it cannot — is a road that gave you time back. If riding down one particular lane somehow made it earlier than when you started, then a place he had settled at two minutes might turn out to be reachable in less, and everything he had already decided would have to be reconsidered. Every road costs something. That is why the method works.

3 The idea in plain English

Farhan's method is Dijkstra's algorithm, and his last paragraph is the proof.

A weighted graph has a number on each edge. The **weight** is a cost — minutes, rupees, distance, risk. Shortest path now means **cheapest total**, the sum of the weights along the way, not the fewest edges.

BFS is wrong the moment weights appear, and this is worth being blunt about. BFS minimises the *number* of edges. A one-edge road costing 38 minutes and a two-edge route costing 7 are the same to BFS, and it picks the 38. It runs, it returns a path, and the path is wrong. That is [day 131](#)'s boundary, and today is what sits on the other side of it.

The fix is one substitution: a priority queue instead of a queue. A **priority queue** hands back the smallest item rather than the oldest. In Python that is `heapq`, from [day 115](#). Instead of expanding whatever arrived first, you expand whatever is **cheapest so**

far.

Every vertex has a tentative distance, and it only ever goes down. Start everything at infinity except the source, which is zero. As you discover routes, you improve the numbers.

Relaxation is the one operation. Standing at u with a known distance, look at an edge $u \rightarrow v$ with weight w . If $\text{distance}[u] + w$ is less than the best you knew for v , you have found a better route — record it, and put v in the priority queue with its new cost. **That is the entire body of the loop.**

Settling is the idea that makes it correct. When a vertex comes out of the priority queue, its distance is **final**. Not provisional — final. Farhan taking the two-minute place first and knowing nothing can beat it.

And here is the three-sentence proof, which is the half of the answer people miss. Suppose you pop vertex v with tentative distance d , and suppose some cheaper route to v exists. That route must leave the settled set at some point, through a vertex x still in the queue — and x 's tentative distance is at least d , because v was the smallest in the queue. **Since no edge has negative weight, the rest of the route from x to v cannot reduce the total below d .** So no cheaper route exists, and d is final.

That proof is where the non-negative requirement lives. Every step of it is fine except the last sentence, and the last sentence is exactly “the rest of the journey cannot give time back”. With a negative edge, a longer route through x could still end up cheaper, a settled vertex would need reopening, and the algorithm's central promise fails. **Dijkstra does not merely give a worse answer with negative weights — the reasoning it is built on stops holding,** and the fix is Bellman-Ford, which is [tomorrow](#).

Now the implementation detail that has a right answer. When you find a better route to a vertex already sitting in the heap, you would ideally *decrease* its key. Python's `heapq` has no such operation. Three options:

1. **Lazy deletion.** Push the vertex again with the new smaller cost, and when popping, skip anything whose popped distance is worse than the recorded best. The heap holds stale entries, and they are harmless. **This is the standard answer and the one to write.**
2. A `visited` set to skip vertices already settled — equivalent, and slightly less precise because it does not catch a stale entry for an unsettled vertex.
3. **An indexed heap** supporting `decrease-key`. Fewer heap entries, considerably more code, and almost never worth it in an interview.

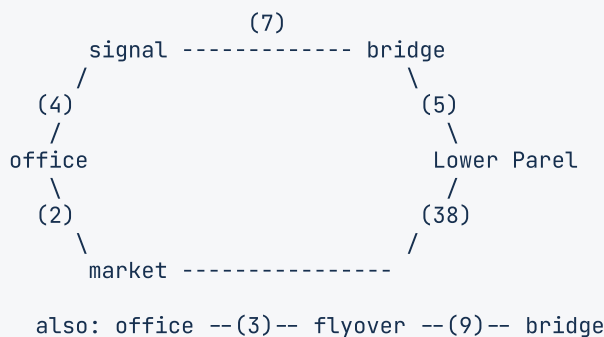
With lazy deletion the heap can hold up to E entries rather than V , and that is fine: the cost is $O(E \log E)$, and since $E \leq V^2$, $\log E \leq 2 \log V$, so it is the same as $O(E \log V)$. Say that if asked; it is the sort of detail that shows you have thought about it.

Early exit is legitimate. If you only want the distance to one target, return the moment you pop it — because popping means settled means final.

And the path, not just the distance, is a parent map, exactly as in BFS: record who improved you, and walk backwards at the end.

4 The picture

Farhan's city, with minutes on the edges:



DIJKSTRA from office:

settled	heap (cost, vertex)	distances
{} (initial)	[(0,office)]	office 0, rest inf
office	[(2,market), (3,flyover), (4,signal)]	market 2, flyover 3, signal 4
market	[(3,flyover), (4,signal), (40,LP)]	LP 40 (2 + 38)
flyover	[(4,signal), (12,bridge), (40,LP)]	bridge 12 (3 + 9)
signal	[(11,bridge), (12,bridge), (40,LP)]	bridge 11 (4 + 7) ← improved
bridge	[(12,bridge stale), (16,LP), (40,LP)]	LP 16 (11 + 5) ← improved pop (12,bridge) → STALE, skip
LP	answer: 16 minutes via signal and bridge	

What to notice. Two things. **bridge** is pushed twice — at 12 via the flyover and at 11 via the signal — and the stale 12 is popped later and skipped. That is lazy deletion in action, and the heap holding both is normal rather than a bug.

And **Lower Parel** was first discovered at 40 through the market. If this were BFS, the market route is **one edge** and would have won. Dijkstra finds 16.

BFS against Dijkstra on exactly that:

office --(38)-- Lower Parel	BFS:	1 edge	→ picks this, cost 38
office --(4)-- signal --(7)--			
bridge --(5)-- Lower Parel	Dijkstra:	3 edges	→ cost 16

What to notice. BFS is not approximately wrong here, it is wrong by a factor of two and a bit, and nothing about its output says so.

The settling frontier, which is the mental picture worth keeping:

settled (final)		frontier (in the heap)	unseen
+-----+		+-----+	
office 0		signal 4	
market 2	→	bridge 12	
flyover 3		Lower Parel 40	
+-----+		+-----+	
^		^	
distances are FINAL		distances are the best	
and never change		route found SO FAR	
the next pop is always the smallest number in the frontier,			
and it moves to the settled set permanently.			

5 The code, built step by step

Start with the graph shape. Weighted adjacency lists hold pairs.

```
graph: dict[int, list[tuple[int, int]]] = {
    0: [(1, 4), (2, 2)],          # (neighbour, weight)
    1: [(3, 7)],
    2: [(3, 38)],
    3: [],
}
```

Now the core loop, and it is BFS with the queue swapped.


```

import heapq

def dijkstra(graph: dict[int, list[tuple[int, int]]], n: int, source: int) → list[float]:
    distance = [float("inf")] * n
    distance[source] = 0
    heap = [(0, source)]
    while heap:
        cost, vertex = heapq.heappop(heap)
        if cost > distance[vertex]:
            continue
        for neighbour, weight in graph[vertex]:
            candidate = cost + weight
            if candidate < distance[neighbour]:
                distance[neighbour] = candidate
                heapq.heappush(heap, (candidate, neighbour))
    return distance

```

Eleven lines, and three of them carry the whole thing.

`heapq.heappop` gives the smallest, which is the substitution. The tuple is `(cost, vertex)` in that order so the heap sorts by cost.

`if cost > distance[vertex]: continue` is lazy deletion. When a vertex was improved after being pushed, the old, larger entry is still in the heap. Popping it and finding that its cost is worse than the recorded distance means it is stale, and skipping it is correct and cheap. **Without this line the algorithm is still correct** — the relaxation check `candidate < distance[neighbour]` prevents any damage — **but you re-scan the neighbours of stale vertices for nothing.**

`if candidate < distance[neighbour]` is relaxation, and it is the only place distances change.

Now the version that stops early, when you want one target:

```

def dijkstra_to(graph, n: int, source: int, target: int) → float:
    distance = [float("inf")] * n
    distance[source] = 0
    heap = [(0, source)]
    while heap:
        cost, vertex = heapq.heappop(heap)
        if vertex == target:
            return cost
        if cost > distance[vertex]:
            continue
        for neighbour, weight in graph[vertex]:
            if cost + weight < distance[neighbour]:
                distance[neighbour] = cost + weight
                heapq.heappush(heap, (cost + weight, neighbour))
    return float("inf")

```

Returning on the **pop**, not on the relaxation. Returning when the target is first *discovered* would be wrong — a cheaper route to it may still be found. Popping it means it is settled.

And the path:

```
def dijkstra_path(graph, n: int, source: int, target: int) → tuple[float, list[int]]:
    distance = [float("inf")] * n
    parent: list[int | None] = [None] * n
    distance[source] = 0
    heap = [(0, source)]
    while heap:
        cost, vertex = heapq.heappop(heap)
        if vertex == target:
            break
        if cost > distance[vertex]:
            continue
        for neighbour, weight in graph[vertex]:
            if cost + weight < distance[neighbour]:
                distance[neighbour] = cost + weight
                parent[neighbour] = vertex # who improved me
                heapq.heappush(heap, (cost + weight, neighbour))

    if distance[target] == float("inf"):
        return float("inf"), []
    path, node = [], target
    while node is not None:
        path.append(node)
        node = parent[node]
    return distance[target], path[::-1]
```

`parent[neighbour] = vertex` goes **inside** the relaxation, so it is only updated when the route actually improves. Putting it outside records the last vertex that looked at the neighbour, which is usually not the one on the best path.

The variant worth knowing: a state that is more than a vertex. Many interview problems are Dijkstra where the “vertex” is a pair — position plus something else.

```
# "cheapest flight with at most k stops": the state is (city, stops used)
heap = [(0, source, 0)]
best: dict[tuple[int, int], float] = {(source, 0): 0}
while heap:
    cost, city, stops = heapq.heappop(heap)
    if city == target:
        return cost
    if stops == limit + 1:
        continue
    for nxt, price in graph[city]:
        if cost + price < best.get((nxt, stops + 1), float("inf")):
            best[(nxt, stops + 1)] = cost + price
            heapq.heappush(heap, (cost + price, nxt, stops + 1))
```

The algorithm is unchanged; only the definition of a vertex expanded. Recognising that a problem is “Dijkstra over states” rather than “Dijkstra over places” is one of the most valuable things in this phase.

The complete solution

```
"""Dijkstra's algorithm: distances, path, and the state-space variant."""

from __future__ import annotations

import heapq

INF = float("inf")

def build(edges: list[tuple[int, int, int]], n: int, directed: bool = False):
    """edges are (a, b, weight). Adjacency of (neighbour, weight) pairs."""
    graph: dict[int, list[tuple[int, int]]] = {v: [] for v in range(n)}
    for a, b, weight in edges:
        graph[a].append((b, weight))
        if not directed:
            graph[b].append((a, weight))
    return graph

def dijkstra(graph, n: int, source: int) → list[float]:
    """Cheapest cost from source to every vertex.  $O((V + E) \log V)$ ."""
    distance = [INF] * n
    distance[source] = 0
    heap = [(0.0, source)]
    while heap:
        cost, vertex = heapq.heappop(heap)
        if cost > distance[vertex]:
            continue # stale heap entry
        for neighbour, weight in graph[vertex]:
            candidate = cost + weight
            if candidate < distance[neighbour]:
                distance[neighbour] = candidate
                heapq.heappush(heap, (candidate, neighbour))
    return distance

def dijkstra_path(graph, n: int, source: int, target: int) → tuple[float, list[int]]:
    """Cheapest cost and one cheapest path. Stops as soon as target is settled."""
    distance = [INF] * n
    parent: list[int | None] = [None] * n
    distance[source] = 0
    heap = [(0.0, source)]
    while heap:
        cost, vertex = heapq.heappop(heap)
        if vertex == target:
            break
        if cost > distance[vertex]:
            continue
        for neighbour, weight in graph[vertex]:
            candidate = cost + weight
            if candidate < distance[neighbour]:
                distance[neighbour] = candidate
                parent[neighbour] = vertex
                heapq.heappush(heap, (candidate, neighbour))
    if distance[target] == INF:
        return INF, []
    path: list[int] = []
    node: int | None = target
    while node is not None:
        path.append(node)
        node = parent[node]
    return distance[target], path[::-1]
```

```

def cheapest_with_stops(graph, source: int, target: int, limit: int) → float:
    """Dijkstra where the state is (city, stops used) – the shape to recognise."""
    heap: list[tuple[float, int, int]] = [(0.0, source, 0)]
    best: dict[tuple[int, int], float] = {(source, 0): 0.0}
    while heap:
        cost, city, stops = heapq.heappop(heap)
        if city == target:
            return cost
        if stops > limit:
            continue
        for nxt, price in graph[city]:
            state = (nxt, stops + 1)
            if cost + price < best.get(state, INF):
                best[state] = cost + price
                heapq.heappush(heap, (cost + price, nxt, stops + 1))
    return INF

if __name__ == "__main__":
    # Farhan's city. 0 office, 1 signal, 2 market, 3 flyover, 4 bridge, 5 Lower Parel
    city = build([
        (0, 1, 4), (0, 2, 2), (0, 3, 3),
        (1, 4, 7), (3, 4, 9), (4, 5, 5),
        (2, 5, 38),
    ], n=6)

    print("distances:", dijkstra(city, 6, 0))
    cost, path = dijkstra_path(city, 6, 0, 5)
    print("best to LP:", cost, path)

    flights = {0: [(1, 100), (2, 500)], 1: [(2, 100)], 2: []}
    print("≤0 stops :", cheapest_with_stops(flights, 0, 2, 0))
    print("≤1 stop  :", cheapest_with_stops(flights, 0, 2, 1))

```

Running it:

```

distances: [0, 4, 2, 3, 11, 16]
best to LP: 16 [0, 1, 4, 5]
≤0 stops : 500
≤1 stop  : 200

```

Two things to look at. `bridge` (vertex 4) settles at **11**, via the signal, not at 12 via the flyover — and both were in the heap at once, with the 12 popped later and skipped as stale.

And Lower Parel is 16 via `[office, signal, bridge, LP]`, a three-edge route, when a one-edge route existed at 38. **BFS would have returned that one-edge route**, which is the whole reason this algorithm exists.

The flights output shows the state variant working: with no stops allowed the only option is the direct 500; with one stop allowed, $100 + 100 = 200$ wins.

6 What it costs

Count it from the operations.

each vertex is pushed at least once	V pushes
each successful relaxation pushes once	up to E pushes

heap entries	$O(V + E) = O(E)$
each push and pop	$\log(\text{heap size})$

total	$O(E \log E)$

And since $E \leq V^2$, $\log E \leq 2 \log V$, so this is conventionally written:

$O((V + E) \log V)$ binary heap (Python's `heapq`)

With a **Fibonacci heap** it is $O(E + V \log V)$, which is asymptotically better and slower in practice for every realistic input because of its constant factors. **Mention it and say you would not use it** — that is the right answer, not reciting it as though it were the default.

On a dense graph, the array version wins:

no heap: scan all V vertices to find the minimum each time
 V iterations $\times$ V scan = $O(V^2)$

compare on a dense graph, $E \sim V^2$:

heap: $E \log V = V^2 \log V$

array: V^2

→ the ARRAY version is faster by a $\log$ factor

$V = 1,000$, $E = 500,000$ (dense)

heap $500,000 \times 10 = 5,000,000$

array $1,000,000$

Sparse graphs — which is nearly everything real — favour the heap:

$V = 100,000$, $E = 300,000$ (sparse, $E/V = 3$)

heap $300,000 \times 17 = 5,100,000$

array $100,000^2 = 10,000,000,000$

Two thousand times faster on a sparse graph, and two times slower on a dense one. That is the whole comparison and it is worth having ready.

Against BFS on the same graph:

```
V = 100,000, E = 300,000
BFS          400,000 steps
Dijkstra     5,100,000 steps    → ~13x more work
```

Which is the reason to say “BFS, because the graph is unweighted” rather than reaching for Dijkstra by default.

Space:

```
distance array    0(V)
parent array      0(V)
heap              up to 0(E) with lazy deletion
-----
0(V + E)
```

```
V = 1,000,000, E = 5,000,000
distance + parent ~16 MB
heap at worst     5,000,000 tuples x ~72 bytes = 360 MB
```

The heap is the memory, and that is a real consideration at scale. The `visited`-set variant bounds pushes better in practice, and an indexed heap bounds them to `V` at the cost of complexity.

Early exit, measured:

```
target 3 hops away in a graph with average degree 10
without early exit   the whole graph
with early exit      1 + 10 + 100 + 1,000 = ~1,111 vertices
                    → on a million-vertex graph, ~900x less work
```

And the comparison table to have ready:

BFS	$O(V + E)$	unweighted only
0-1 BFS (deque)	$O(V + E)$	weights only 0 and 1
Dijkstra	$O((V + E) \log V)$	non-negative weights
Bellman-Ford	$O(V \times E)$	negative weights, detects negative cycles
Floyd-Warshall	$O(V^3)$	all pairs, small V , negatives allowed
A*	$O((V + E) \log V)$	Dijkstra plus a heuristic; much faster in practice

7 The traps

Negative weights

The near-miss that runs and lies:

```
graph = {0: [(1, 5), (2, 2)], 2: [(1, -10)], 1: []}
```

```
>>> dijkstra(graph, 3, 0)
[0, 5, 2]
```

The true distance to vertex 1 is $2 + (-10) = -8$. Dijkstra says 5. What happened: vertex 1 was settled at 5 before vertex 2 was expanded, and **settled means never reconsidered**.

There is no error and no warning. **The only defence is to check the input**, and the answer when weights can be negative is Bellman-Ford.

The subtler version: some problems have negative weights hidden in a transformation — “maximise profit” turned into “minimise negative profit”. **If you negate weights to turn a maximisation into a minimisation, you have just created negative weights and Dijkstra is no longer valid.**

Returning on discovery instead of on pop

```
for neighbour, weight in graph[vertex]:
    if neighbour == target:
        return cost + weight                # WRONG
```

```
graph: 0 → 1 (cost 100), 0 → 2 (cost 1), 2 → 1 (cost 1)
>>> wrong_version(graph, 0, 1)
100                                     # the answer is 2
```

Discovering a vertex means you have a route to it. **Only popping it means you have the cheapest**. Return on the pop.

Forgetting the stale check

```
while heap:
    cost, vertex = heapq.heappop(heap)
    # no stale check
    for neighbour, weight in graph[vertex]:
        ...
```

Still **correct**, because relaxation guards against damage — but every stale entry causes a full re-scan of a vertex’s neighbours:

```
V = 100,000, E = 500,000, many improvements
with the check      ~500,000 neighbour scans
without             up to 2-3x more, and worse on graphs with many improvements
```

Not a crash, just slower, and on a judge it is the difference between passing and timing out.

Pushing an object with no tie-breaker

```
heapq.heappush(heap, (cost, node_object))
```

```
Traceback (most recent call last):
  File "di.py", line 18, in dijkstra
    heapq.heappush(heap, (candidate, neighbour))
TypeError: '<' not supported between instances of 'Node' and 'Node'
```

Two entries with equal cost make Python compare the second element. With integers that is fine; with objects it raises. Put an integer id between the cost and the object. **You met this on [day 116](#) and it will keep finding you.**

Mutable state in the heap

```
heapq.heappush(heap, (distance[v], v))
distance[v] = smaller                    # the heap entry does NOT update
```

The tuple captured the old value. This is not a bug in itself — it is exactly why lazy deletion exists — but people write code assuming the heap re-sorts when the distance array changes. **It does not.** The heap holds snapshots.

Using Dijkstra when BFS would do

```
dijkstra(unweighted_graph, n, source)    # all weights are 1
```

Correct, and about thirteen times slower on a typical sparse graph. **If every edge costs the same, say “BFS” and say why** — reaching for the general tool when the special one applies is a real signal in an interview.

The state-space explosion

```
# "cheapest path with at most k stops", k = 100, V = 100,000
```

The state is `(vertex, stops)`, so the state space is $V \times (k + 1)$ — ten million states here. That is still tractable, but it is worth computing before writing, because with two extra dimensions it stops being. **Always multiply out the state space and say the number.**

8 In the interview

How it gets asked

- *“Find the shortest path in a weighted graph.”* — the direct version.
- *“Cheapest flight from A to B.”* — often with a stop limit, which is the state variant.
- *“Network delay time: how long until every node receives the signal?”* — Dijkstra, take the maximum.
- *“Path with the minimum effort / maximum minimum weight.”* — Dijkstra with a different combining rule.
- *“Why can’t you use BFS here?”* — the boundary question.
- *“Why does your algorithm need non-negative weights?”* — the proof question, and it is the one that sorts candidates.

The first ninety seconds

“Weights, so BFS is out — it minimises the number of edges, and with weights a three-edge route can easily be cheaper than a one-edge route. This is Dijkstra.

It is BFS with one substitution: a priority queue instead of a queue, so instead of expanding whatever arrived first I expand whatever is **cheapest so far**. Everything else is the same shape — a distance map, a relaxation step, and an optional parent map for the path.

The invariant is that **when a vertex comes out of the heap, its distance is final**, and I would give the reason because that is the algorithm. Suppose I pop v at cost d and a cheaper route existed. That route has to leave the settled set through some vertex still in the heap, and that vertex costs at least d , because v was the smallest. Since **no edge is negative**, the rest of the route cannot bring the total below d . So no cheaper route exists.

That last sentence is where the non-negative requirement lives. With a negative edge, a longer route can still get cheaper, a settled vertex would need reopening, and the whole argument collapses — so Dijkstra is not just less accurate with negative weights, its reasoning stops holding. That case is Bellman-Ford.

Implementation detail worth stating: Python’s `heapq` has no decrease-key, so when I find a better route to a vertex already in the heap, I push it again and skip stale entries on pop by checking whether the popped cost is worse than the recorded distance. **Lazy deletion.** It means the heap can hold up to E entries rather than V , which is fine — $O(E \log E)$ is $O(E \log V)$ since $E \leq V^2$.

$O((V + E) \log V)$ time, $O(V + E)$ space, and if I only want one target I return the moment I **pop** it — not when I first discover it, because discovery only gives me a route.

Are the weights guaranteed non-negative?”

The follow-ups

“Why does it need non-negative weights? Give me a concrete failure.”

“Smallest possible example. Source S , with $S \rightarrow A$ costing 5 and $S \rightarrow B$ costing 2, and then $B \rightarrow A$ costing -10 .

Dijkstra pops S at 0, relaxes both, and the heap holds $(2, B)$ and $(5, A)$. It pops B at 2 — correct — and relaxes $B \rightarrow A$, finding $2 - 10 = -8$, which improves A . But if A had been popped first, or if the negative edge had been discovered after A was settled, A would be locked at 5.

Make it slightly bigger and it definitely breaks: $S \rightarrow A$ at 1, $S \rightarrow B$ at 2, $B \rightarrow A$ at -5 . A is popped first at 1 and settled. Then B is popped and offers A a route at -3 , and A is never reconsidered. The answer comes back as 1 and the truth is -3 .

The general statement is that Dijkstra never revisits a settled vertex, and a negative edge is exactly a reason to revisit one.

The trap I would watch for in a real problem is negative weights arriving through a transformation. If someone asks for the *maximum* profit path and I negate the weights to make it a minimisation, I have created negative weights and invalidated the algorithm. On a DAG that is fine — one pass in topological order handles any weights — but on a general graph it is Bellman-Ford, and if there is a negative cycle there is no answer at all.”

“There is a limit of at most k stops. How does that change things?”

“The algorithm does not change at all; **the definition of a vertex does**. The state becomes `(city, stops used)` rather than just `city`, and everything else is identical.

The reason it has to is that the cheapest way to reach a city is no longer a single fact about that city. Reaching Delhi in two stops for ₹300 and in four stops for ₹100 are both useful, and plain Dijkstra would settle Delhi once and discard one of them. With the state expanded, `(Delhi, 2)` and `(Delhi, 4)` are different vertices and both survive.

The cost I would compute before writing it: the state space is $V \times (k + 1)$, and the edges are $E \times (k + 1)$, so it is $O(E \cdot k \log(V \cdot k))$. With a hundred thousand cities and ten stops that is a million states and a few million edges — fine. With two extra dimensions it would not be, so **I would multiply it out and say the number rather than assume**.

The alternative for this specific problem is Bellman-Ford run exactly $k + 1$ times, which is $O(k \times E)$ and is often simpler because the stop limit maps directly onto the number of relaxation rounds. For LeetCode’s cheapest-flights problem that is the neater solution, and I would mention both.”

“The graph is a road network with ten million intersections and you need one route, fast.”

“Dijkstra explores in expanding circles from the source, so on a road network it explores a huge disc of roads in every direction, including directly away from the destination. For a city-scale query that is wasteful and for a country-scale one it is unusable.

A* is the direct improvement. Same algorithm, but the priority is `distance so far + an estimate of the distance remaining` — for a road network, straight-line distance to the target, which is easy to compute from coordinates. That biases the search towards the goal, and the exploration becomes an ellipse rather than a circle. On real road networks it typically cuts the explored set by an order of magnitude or more.

The requirement for A* to still be *correct* is that the estimate never overestimates the true remaining distance — it must be **admissible**. Straight-line distance qualifies, because no road is shorter than the straight line. An estimate that overestimates makes it fast and wrong.

Bidirectional search also helps, for the same reason as in BFS: two half-radius searches explore far less than one full-radius one. Combining bidirectional with A* needs care about the stopping condition, so I would do one or the other unless I had time to get it right.

And for a production routing system, none of the above is the real answer — it is precomputation. Contraction hierarchies preprocess the network once, adding shortcut edges, and then answer queries in microseconds rather than seconds. That is what a real map service does, and I would name it so you know I know the algorithm alone is not the product.”

“What if the cost is not a sum? Say I want the path whose worst segment is as good as possible.”

“Dijkstra generalises further than people expect, and this is a nice example of that.

The standard version accumulates with addition and compares with less-than. If instead I accumulate with `max` — the cost of a path is its worst edge — and still pick the smallest such value from the heap, the algorithm finds the path minimising the maximum edge. That is the ‘path with minimum effort’ or ‘swim in rising water’ family, and the code change is one line: `candidate = max(cost, weight)` instead of `cost + weight`.

The reason it still works is that the proof only needs one property: **extending a path must never make it better**. Addition with non-negative weights has that. So does `max`. So does multiplication by probabilities in `[0, 1]` if you are maximising reliability and take the largest each time.

What fails is anything where extending can improve the total — which is exactly negative weights again, stated more generally. So the honest version of ‘Dijkstra needs non-negative weights’ is ‘Dijkstra needs a combining rule under which paths never get better as they get longer’, and non-negative addition is the common case.”

The model answer

“Given `n` network nodes and travel times between them, find how long it takes for a signal sent from node `k` to reach every node. Return `-1` if some node is unreachable.”

“Single source, weighted edges, all distances — that is Dijkstra, and the answer is the **maximum** of the distances rather than any single one. Let me set it up.

The model. A vertex is a node. A directed edge from `u` to `v` with weight `w` means a signal takes `w` time to travel that way. Directed matters — the input gives `(u, v, w)` triples and the signal does not travel back — so one append, not two.

Then Dijkstra from `k`. Distance array initialised to infinity except the source at zero, a heap of `(cost, vertex)`, pop the cheapest, skip stale entries, relax each outgoing edge.

The answer is `max(distance)`, because the signal has reached everything when the last node receives it. And if any distance is still infinity, that node is unreachable and the answer is `-1` — so the check is one line: `return -1 if INF in distance else max(distance)`.

Two things I would state before coding. Weights are travel times and therefore non-negative, which is what makes Dijkstra valid — I would confirm that rather than assume it. And I would **not** exit early here, because I need every distance, not one; the early-exit version is for single-target queries.

Cost: $O((V + E) \log V)$. On the LeetCode constraints — 100 nodes, 6,000 edges — that is trivial, and I would note that at `V = 100` with `E = 6,000` the graph is dense enough that the simple $O(V^2)$ array version without a heap would actually be faster: ten thousand operations against sixty thousand heap operations. **I would still write the heap version,** because it is the one that generalises, but saying that I know the crossover exists is worth more than the micro-optimisation.

The bugs I would be watching for as I write it. Building the graph undirected when the input is directed — one wrong line, and the answer is too small with no error. Returning when the target is discovered rather than popped, which does not apply here since I want all distances but is the reflex to suppress. And pushing `(vertex, cost)` instead of `(cost, vertex)`, which makes the heap order by vertex id and produces a confidently wrong answer — I would write the tuple and immediately say ‘cost first’ out loud.

The follow-up I would expect is ‘what if some links can be down’ or ‘what if there are at most `k` hops’, and both are the state-space variant: the vertex becomes `(node, something)`, and the algorithm is unchanged. I would mention that so you know I have the general shape rather than the one problem.”

9 Recall card

Dijkstra is BFS with a priority queue instead of a queue — expand the cheapest-so-far rather than the oldest. Relaxation `if dist[u] + w < dist[v]` is the only place distances change.

Popped means settled means final, and the three-sentence proof is: a cheaper route would have to leave the settled set through a vertex costing at least as much, and **since no edge is negative**, the remainder cannot bring it below. That last clause is the whole non-negative requirement.

Lazy deletion: no decrease-key in `heapq`, so push again and `if cost > distance[vertex]: continue` on pop. The heap holds up to `E` entries; $O(E \log E) = O(E \log V)$.

Return on the pop, not on the discovery. Discovery gives you a route; popping gives you the cheapest. And `parent[v] = u` goes **inside** the relaxation.

$O((V + E) \log V)$ — about $13 \times$ BFS, so use BFS when weights are equal. Negative weights $\rightarrow$ Bellman-Ford. A stop limit or any extra condition $\rightarrow$ expand the *state*, not the algorithm. And the proof only needs “extending a path never improves it”, so `max` instead of `+` solves the minimise-the-worst-edge family unchanged.

1 What this is, and why they ask it

Yesterday you learned what an [inverted index](#) is. Today is the box on the architecture diagram: where the search cluster sits, what feeds it, what it is allowed to answer, and what happens when it disagrees with the database.

The single most important thing about that box is what it is **not**. Elasticsearch is not your database. It is a derived copy of your data, arranged for a different kind of question, and it can be rebuilt from scratch at any time. Treating it as a source of truth is the mistake this lesson exists to prevent, and it is a mistake that is very expensive to discover in production.

They ask this because “where does Elasticsearch sit, and how does it stay up to date?” is a design question with a right answer and several tempting wrong ones. The tempting wrong ones are: write to both from the application, use it as the primary store because it is fast, and let it return the data itself rather than just identifiers. **Each of those works in a demo and fails within a year.**

By the end of this lesson you can draw the box and its arrows, choose a sync mechanism and defend it, size a cluster from a document count, explain shards and replicas well enough to pick numbers, handle a reindex without downtime, and say precisely what the cluster is allowed to be authoritative about.

2 The story

Sudha’s shop sells sarees, and there are about four thousand of them in the storeroom at the back.

The storeroom is organised by supplier and by the date they came in, because that is what matters when you are paying bills and checking what arrived. Racks with numbers. Nobody who comes into the shop ever sees it, and nobody would find anything in it.

The front of the shop is a different thing entirely.

There are two windows and a long shelf behind the counter, and what goes there is chosen and arranged completely differently: by colour, by occasion, by what is selling this month. Reds and maroons together for wedding season. The lighter cottons at eye level in April. Whatever is in the film everybody is watching, at the front.

The boy who works there, Vinod, spends the first forty minutes of every morning moving things between the two. New stock comes out of the storeroom and into the window if it fits what the window is for. Things that stopped moving go back. If a saree is sold, it comes out of the window that day, or somebody points at it and asks for it and it is not there.

Two rules, and Sudha is very firm about both.

The window is never the record. If somebody asks whether they have the green Kanjivaram with the gold border, Vinod does not say yes because he can see one in the window. He goes and checks the rack, because the window is a display and the storeroom is the truth. Twice, years ago, somebody sold something that had already been sold, because the window still had one out.

And the window can be rebuilt. In 2019 a pipe burst overnight and the entire front of the shop had to be emptied and cleaned. Nothing was lost, because everything in the window also existed in the storeroom. It took Vinod a day and a half to build the display again from scratch, and he grumbled the whole time, and at the end of it the shop was exactly as it had been.

If the storeroom had burnt, the shop would have closed.

The thing Sudha will not do, and she has been asked, is keep the *only* copy of anything in the window. A supplier once suggested leaving a consignment out front because it was selling fast and there was no space at the back. She said no, and when he pushed she said it again without explaining, because she had explained it to enough people already.

3 The idea in plain English

The storeroom is the database, the window is Elasticsearch, and Sudha's two rules are the whole design.

Elasticsearch is a derived copy, arranged for a different question. The database is organised for correctness and for lookups by identifier. The search cluster is organised for “find things matching these words, ranked”. Same data, different arrangement, different purpose.

Rule one: it is never the source of truth. Anything authoritative — the price, the stock count, whether an order exists — is answered by the database. **Search returns identifiers, and the application fetches the real objects from the database.** That is the “check the rack” rule, and it buys you two things: a stale index can show a slightly wrong *set* of results but never wrong *content*, and a deleted item is filtered out when you try to fetch it.

Rule two: it must be rebuildable from scratch. If the cluster is lost entirely, you re-index from the database and you are back. That single property is what makes the whole arrangement safe, and it means you never need to treat search backups as critical.

Its consequence is worth stating: never store data only in Elasticsearch. Not user-generated content, not audit records, not anything you cannot recreate. The moment something exists only there, it stops being a derived copy and becomes a database you are operating badly.

Now the box and its arrows. Writes go to the database. Something moves them to the index. Reads for search go to the cluster, come back as ids, and are hydrated from the database. That is the shape, and everything else is a detail of the “something”.

The three ways to feed it, and one is wrong.

Dual write — the application writes to the database and to the index in the same request handler. Simple, and broken: there is no transaction across two systems, so a crash between the two leaves them permanently inconsistent, and it will happen. **Name it to reject it.**

Outbox — the application writes the row and an outbox record in one database transaction; a separate consumer reads the outbox and indexes. Atomic at the source, at-least-once delivery, idempotent because indexing is keyed by document id. This is the [day 121](#) pattern and it is what most teams should build.

Change data capture — a tool reads the database’s replication log and emits change events. No application changes at all, catches writes from anywhere including manual ones and other services, and it is the more robust option at the cost of another moving part.

Whichever you choose, the index will drift, so a reconciliation job is not optional. Consumers die, deploys skip records, someone runs an UPDATE by hand. Over months the two diverge, and the only way to know is to compare periodically and re-index the differences.

Shards and replicas are the two numbers you have to pick. A **shard** is a self-contained piece of the index; the documents are divided across shards by a hash of the document id. A **replica** is a copy of a shard on another node, serving reads and taking over if the primary fails.

- **More shards means more parallelism on write and more overhead on read,** because every search touches every shard and the results are merged.
- **Shard count is fixed when the index is created.** Changing it means reindexing into a new index — which is why it is a decision to get roughly right up front, and why aliases exist.

- **Replicas can be changed at any time** and are the lever for read throughput.

Aliases are how you change anything without downtime. An **alias** is a name that points at one or more indices. The application always queries the alias. To change the mapping or the shard count, you build a new index, re-index into it, and atomically switch the alias. **Never point an application at a concrete index name**, because the first time you need to change the mapping you will have to take an outage to do it.

And the thing that decides whether the whole design is honest: staleness. There is always a window between the write and the document being searchable — the refresh interval plus queue depth plus indexing time. **A design that does not say what a user sees inside that window is unfinished**, and the usual answer is that anything scoped to the user is read from the database, not from search.

4 The picture

The box and its arrows:

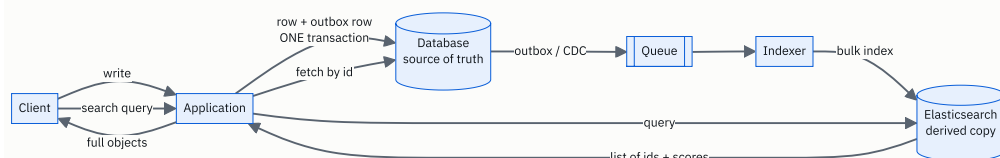


FIGURE 136.1

What to notice. The database has an arrow into Elasticsearch and Elasticsearch has none back. And the read path goes **through** the database on the way out — search says *which*, the database says *what*. That is Vinod checking the rack.

The three sync mechanisms, and where each can break:

DUAL WRITE	OUTBOX	CDC
app → DB → ES DB has it, ES does not. Nothing knows. Permanent.	app → DB + outbox (one txn) outbox → consumer → ES consumer retries. at-least-once, idempotent.	app → DB replication log → connector → ES catches ALL writes, including manual ones

What to notice. Dual write's failure is *permanent and silent* — nothing is retrying, nothing knows. The other two turn a crash into a delay.

Shards and replicas, made concrete:

```
index "products", 3 primary shards, 1 replica each
```

node A	node B	node C
+-----+	+-----+	+-----+
P0	P1	P2
R2	R0	R1
+-----+	+-----+	+-----+

```
a document's shard = hash(doc_id) % 3
a search hits ALL THREE shards (any copy) and merges the results
node B dies → R1 on node C is promoted to primary; nothing is lost
```

What to notice. A search touches every shard, so more shards means more work per query, not less. Shards buy you *write* parallelism and the ability to exceed one machine — not faster individual searches.

And the reindex with an alias, which is the operation you must know:

```
app queries the alias "products"           products → products_v1
```

1. create products_v2 with the new mapping
2. reindex products_v1 → products_v2 (minutes to hours)
3. catch up: replay anything written since step 2 started
4. atomically switch: products → products_v2
5. delete products_v1 after a safe interval

the application never changed, and there was no downtime.

5 How it actually works

The read path, in detail

1. user types "brass hinges"
2. app queries ES: match on title^3 and description, filter category, size 20
3. ES returns: [{id: 4471, score: 12.4}, {id: 991, score: 9.1}, ...]
4. app fetches: SELECT * FROM products WHERE id IN (4471, 991, ...)
5. app orders the rows by the score order and returns them

Two details that matter in step 4. **Preserve the ranking** — the `IN` clause returns rows in whatever order the database likes, so you re-sort by the search order in the application. And a **missing row is not an error**: a product deleted since indexing simply will not come back, and the result list is one item shorter. That is the deleted-item filter working, and it should be silent.

The alternative — storing the full document in Elasticsearch and returning it directly — is faster and worse. One round trip instead of two, and every stale field is now visible to the user: an old price, a sold item shown as available. **Store only what you need to search on and filter by, plus the id.**

Mappings, and the one that bites

A **mapping** is the schema: which fields exist and how each is treated.

```
{
  "properties": {
    "title": { "type": "text", "analyzer": "english" },
    "brand": { "type": "keyword" },
    "price": { "type": "scaled_float", "scaling_factor": 100 },
    "category_id": { "type": "keyword" },
    "created_at": { "type": "date" }
  }
}
```

text versus **keyword** is the distinction to know. **text** is analysed — tokenised, lowercased, stemmed — and is what you search. **keyword** is stored whole and is what you filter, sort and aggregate on. A brand field mapped as **text** cannot be aggregated sensibly; a title mapped as **keyword** cannot be searched by word. **Getting this wrong is the single most common Elasticsearch mistake**, and fixing it requires a reindex.

Mappings are mostly immutable. You can add a field. You cannot change an existing field's type. That is why aliases exist and why the reindex procedure is something to know rather than look up under pressure.

And dynamic mapping is a trap at scale. By default Elasticsearch guesses the type of a new field, so a document with an unexpected key silently creates a field. On a system indexing user-supplied JSON this produces **mapping explosion** — thousands of fields, enormous cluster state, and eventually a cluster that cannot recover. Set **dynamic: strict** on anything user-influenced.

Queries: filter versus query context

```
{
  "query": {
    "bool": {
      "must": [ { "match": { "title": "brass hinges" } } ],
      "filter": [ { "term": { "category_id": "hardware" } },
                  { "range": { "price": { "lte": 500 } } } ]
    }
  }
}
```

must **scores**; **filter** **does not**. A filter clause is a yes/no test, so its results are cacheable and it skips scoring entirely. **Put everything that is not a relevance signal into filter** — category, price range, in-stock, date bounds. This is usually the largest easy performance win in an Elasticsearch query, and it is also more correct: a category match should not make something *more relevant*, only *eligible*.

The sync pipeline, concretely

```
-- one transaction
BEGIN;
UPDATE products SET title = $1, price = $2 WHERE id = $3;
INSERT INTO outbox (aggregate_id, type, payload) VALUES ($3, 'product.updated', $4);
COMMIT;
```

Then a consumer batches:

```
read up to 500 outbox rows
  → build a bulk index request
  → POST /_bulk
  → mark the outbox rows processed
```

Bulk, always. Per-document indexing is roughly an order of magnitude slower than batching, and it creates far more segments. Batches of a few hundred to a few thousand documents, or a few megabytes, whichever comes first.

Idempotency is free because indexing is keyed by document id: indexing the same document twice overwrites with identical content. That is what makes at-least-once delivery harmless.

Refresh, and the thing not to do

A document is searchable only after the segment holding it is written, which happens on **refresh** — default every **one second**.

refresh_interval: 1s	default; ~1 s until searchable
refresh_interval: 30s	fewer segments, much faster bulk indexing, 30 s stale
refresh_interval: -1	disabled — for a bulk backfill, then re-enable

Never call `?refresh=true` on every write. It forces a segment per document, the segment count explodes, merging thrashes, and query latency collapses. It is the fix that makes everything worse, and it is extremely common in code written by someone debugging a staleness complaint.

Sizing

shard size	aim for 10-50 GB per shard
shards per node	keep well under a few hundred
heap	31 GB maximum (above that, JVM pointer compression is lost) → the other half is page cache, which is where search performance actually comes from

The page-cache point is the one people miss. Elasticsearch relies on the operating system caching index files, so a node with 64 GB gives 31 GB to the heap and leaves 33 GB of cache. If your index is far larger than the total cache across the cluster, queries hit disk and latency becomes unpredictable.

Failure modes worth naming

- **Split brain** — solved since version 7 by proper quorum-based master election; dedicated master-eligible nodes are still good practice.
- **Yellow cluster** — all primaries assigned, some replicas not. Reads and writes work; you have no redundancy. Usually a node down or nowhere to put a replica.
- **Red cluster** — some primary shard is unassigned. **Part of the index is unavailable**, and searches return partial results by default, which is worse than failing. Set `allow_partial_search_results: false` if a partial answer would be misleading.
- **Mapping explosion** — from dynamic mapping on uncontrolled input.
- **Deep pagination** — `from: 100000` makes every shard build and sort 100,000 hits before discarding them. Use `search_after` with a sort key for deep paging, and cap the page depth for users.

6 The numbers

Sizing a cluster from a document count. 50 million products, average 2 KB indexed:

raw indexed content	50,000,000 x 2 KB	= 100 GB
index overhead (~1.2x)		= 120 GB
x 1 replica		= 240 GB on disk
shards at 30 GB each	120 / 30	= 4 primary shards (8 shard copies with replicas)

nodes: 3 minimum for quorum		
disk per node	240 / 3	= 80 GB, plus headroom → 200 GB
RAM per node	want the working set cached	
	64 GB: 31 GB heap + 33 GB page cache	
	3 nodes → ~100 GB of cache vs 120 GB of index	→ good

Query throughput:

simple match query, warm cache	5-20 ms
with aggregations	50-200 ms
per node, simple queries	~500-2,000 queries/s

Indexing throughput:

bulk indexing, per node	10,000-50,000 docs/s
single-document indexing	~1,000/s → 10-50x worse

The full reindex, which is the operation you must be able to estimate:

50,000,000 documents at 20,000 docs/s per node	
3 nodes	= 60,000 docs/s
	50,000,000 / 60,000 = ~14 minutes

Fourteen minutes — which is why an alias-based reindex is a routine operation rather than a project, and why “we cannot change the mapping” is almost never true.

Staleness, decomposed:

outbox write to consumer poll	up to 1 s
bulk batch accumulation	up to 1 s
indexing	~50 ms
refresh interval	up to 1 s

typical end to end	1-3 s

And under load:

consumer lagging by 10,000 documents at 20,000/s	= 0.5 s
consumer lagging by 5,000,000	= 4 minutes

Which is why you alert on the age of the oldest unindexed record, not on the refresh interval — the configured number is the floor, and the real number is whatever the pipeline is doing.

Deep pagination cost:

from: 0, size: 20	each shard returns 20 → 4 shards x 20 = 80 hits merged
from: 100000, size: 20	each shard builds and sorts 100,020 hits
	4 x 100,020 = 400,080 hits, 400,000 discarded
	→ seconds, and heap pressure

Cost, roughly:

3 nodes x 64 GB RAM, 500 GB SSD	~\$500-900/month self-managed on cloud VMs
managed Elasticsearch service	~1.5-2x that

Versus Postgres full-text search on the same data: zero extra infrastructure, and a hard ceiling somewhere around a few million documents. **The crossover is not really about documents; it is about whether you need facets, typo tolerance and query volume off the primary database.**

7 The trade-offs

You are adding a second copy of your data and a pipeline to maintain it. That is one more system to run, one more thing to monitor, one more source of pager alerts, and a permanent consistency gap. **Everything below is a consequence of that one decision,** and it should be made deliberately rather than because search is a feature.

Staleness is not a bug you can fix, only a number you can choose. One to three seconds normally, minutes under load, hours if the pipeline breaks. The design must say what a user sees in that window, and the good answer is that anything scoped to the user reads from the database.

Shard count is chosen early and is awkward to change. Too few and you cannot spread writes or exceed one machine's disk; too many and every query pays the overhead of merging results from all of them, and cluster state grows. Aliases make the change possible without downtime, and it is still a reindex.

Elasticsearch is fast because it is memory-hungry. Search performance depends on index files being in the page cache, so the honest sizing rule is that you want cache comparable to your index size. **A cluster with far less RAM than index will have unpredictable tail latency,** and no amount of query tuning fixes it.

It is not transactional and never will be. No cross-document transactions, no read-your-writes guarantee without forcing a refresh, and partial search results on a red cluster by default. **Anything requiring correctness must be answered by the database.**

And the failure that costs most: treating it as a store. Data written only to Elasticsearch cannot be rebuilt, so the cluster becomes critical infrastructure with weaker durability guarantees than your database, worse backup tooling, and a version upgrade path that occasionally requires a reindex. **If it cannot be rebuilt from the database in an afternoon, the design has gone wrong.**

When would I not use it? Under a few million documents with modest query volume, where Postgres full-text search is transactionally consistent and free. When the requirement is really structured filtering rather than relevance — a dropdown of categories is better than a search box. When the team cannot operate a cluster, where a man-

aged service or Postgres is the honest choice. And when a vector or hybrid search product fits the requirement better, which is increasingly common for semantic search — though modern Elasticsearch and OpenSearch both do dense vectors too.

8 In the interview

How it gets asked

- “Where does Elasticsearch sit, and how does it stay up to date?” — the direct version.
- “Draw the write path and the read path for search.”
- “Can you use Elasticsearch as your primary database?” — the answer is no and the reasons matter.
- “How do you change the schema without downtime?” — aliases and reindex.
- “How many shards?”
- “Your search is showing deleted products. Why?”

The first ninety seconds

“It sits beside the database, not in front of it, and the single most important property is that **it is a derived copy that can be rebuilt from scratch**. Nothing lives only in Elasticsearch.

Write path: the application writes the row and an outbox record in one database transaction. A consumer reads the outbox and bulk-indexes into Elasticsearch. I would specifically not dual-write from the application — writing to the database and then to the index in the same handler has no transaction across the two, so a crash in between leaves them permanently inconsistent with nothing retrying and nothing knowing. Change data capture off the replication log is the other good option; it catches writes from anywhere, including manual ones, at the cost of another component.

Read path, and this is the part people get wrong: search returns identifiers, and the application fetches the objects from the database. Two round trips instead of one, and it buys two things. A stale index shows a slightly wrong result set but never wrong *content* — no old prices, no sold items shown as available. And a deleted item simply does not come back from the fetch, so it filters itself out.

Staleness is a number, not a bug. Outbox poll plus batching plus indexing plus the one-second refresh interval is typically one to three seconds. The design has to say what a user sees in that window, and my answer is that anything scoped to the user — ‘my listings’, the item they just created — reads from the database, so the window is invisible to the person most likely to notice it.

And a reconciliation job, because the index will drift regardless of the mechanism. Nightly comparison of counts and checksums by id range, re-indexing the differences.

How many documents, and do you need facets and typo tolerance? Because under a few million with plain keyword search, Postgres full-text search is transactionally consistent and removes this entire diagram.”

The follow-ups

“Can Elasticsearch be the primary database?”

“No, and I would give reasons rather than assert it.

No transactions. No multi-document atomicity, so you cannot express ‘decrement stock and create an order’ as one operation.

No read-your-writes by default. A document is not searchable for about a second after indexing, and forcing a refresh to fix that destroys performance by creating a segment per document.

Partial results on failure. If a shard is unavailable, a search returns results from the remaining shards and a flag saying so — by default, quietly. For a display that is a reasonable trade; for anything authoritative it is silently wrong data.

Weaker durability posture and a harder upgrade path. Major version upgrades have historically sometimes required a reindex, which is a routine operation for a derived copy and an outage for a store of record.

There is a narrow case where it is defensible: log and metrics data, where the data is append-only, loss of a small fraction is tolerable, and the query pattern is exactly what it is built for. That is what the whole observability stack is. **But for anything a business depends on being correct, it is a copy** — and the test I would apply is: if the cluster were deleted right now, could I rebuild it from the database this afternoon? If not, the design is wrong.”

“How do you change the mapping without downtime?”

“Aliases, and the reason to set this up on day one is that mappings are mostly immutable — you can add a field but not change an existing field’s type — so this will come up.

The application never queries a concrete index name. It queries an alias, say `products`, which points at `products_v1`.

To change the mapping: create `products_v2` with the new mapping; run a reindex from v1 to v2, which for 50 million documents on three nodes is about fifteen minutes; keep the sync pipeline writing to **both** indices during that window, or replay the outbox from the reindex start point afterwards; then atomically switch the alias to point at v2. The switch is a single API call and is instant. Delete v1 after a day or two, once you are confident.

The catch-up step is the one people forget. A reindex is a snapshot of a moving target, so anything written during those fifteen minutes is missing from v2 unless the pipeline is dual-writing or you replay. Dual-writing to both indices during the migration is the simplest correct answer.

And the sentence I would leave you with is: never point the application at a concrete index name. It costs nothing on day one and it is the difference between a fifteen-minute background operation and a planned outage.”

“How many shards, and why?”

“From the data size, targeting 10 to 50 gigabytes per shard, and I would work it out rather than guess.

Fifty million documents at 2 KB indexed is about 100 gigabytes, plus roughly 20% index overhead, so 120 gigabytes. At 30 gigabytes per shard that is four primary shards, and with one replica, eight shard copies across three nodes.

Why not more shards? Every search touches every shard and merges the results, so shard count is overhead on the read path, not a speed-up. Ten thousand tiny shards is a well-known way to make a cluster slow and its cluster state enormous.

Why not fewer? A shard is the unit of distribution, so one shard means one node’s disk and one node’s write throughput, with no ability to spread. And it cannot be split later without a reindex.

Replicas are the read-throughput lever, and unlike shards they can be changed at any time. So I would set the shard count from data volume with a little headroom for growth, and tune replicas afterwards based on query load.

The RAM point matters more than the shard count, honestly. Search performance comes from index files being in the page cache, so I want total cache across the cluster comparable to the index size — 31 gigabytes of heap per node maximum, and no more than half the machine’s memory, leaving the rest for cache. **A cluster with far less RAM than index has unpredictable tail latency and no query tuning fixes it.**“

“Search is showing products that were deleted.”

“Expected, and the design should already make it harmless — so I would check that first before treating it as an incident.

The mechanism: the deletion updated the database and the delete event has not yet reached the index. Normal lag is one to three seconds; if it is minutes, the sync pipeline is backed up; if it is permanent, the pipeline is broken or the delete path bypassed it.

Why it should not matter: search returns ids and the application fetches from the database, so a deleted product simply does not come back from the fetch and drops out of the result list silently. If deleted products are actually being *displayed*, then the read path is returning content straight from Elasticsearch — and that is the real bug, not the lag.

What I would check, in order. Is the read path hydrating from the database? Is the consumer lagging — and I would look at the age of the oldest unprocessed out-box record rather than at a queue depth. Does the delete path actually write an out-box record, because soft deletes done with a direct `UPDATE` that bypasses the application layer are a very common cause. And does the reconciliation job exist, because if it does, this class of drift should be self-healing overnight.

The permanent fix, beyond the immediate cause, is a filter on the query — `is_deleted: false` as a filter clause — plus the hydration step, so the same fact is enforced in two places.”

The model answer

“Add search to an e-commerce site: 20 million products, 5,000 searches a second at peak, faceted filtering by category, brand and price, and sellers update prices and stock constantly.”

“The constantly-changing prices and stock are the interesting part of this, so let me get the shape down and then deal with that specifically.

Shape. Postgres remains the source of truth. Elasticsearch holds a derived index. Writes go to Postgres with an outbox row in the same transaction; a consumer bulk-indexes. Search returns product ids; the application hydrates from Postgres — or, at 5,000 searches a second, from a Redis cache in front of Postgres keyed by product id, since that is a pure key lookup and eminently cacheable.

What goes in the index. Title and description analysed for search, with the title boosted. Category, brand and price as `keyword` and numeric fields for filtering and faceting. An `is_active` flag. **Not** the full product body, not the seller’s contact details, not anything I do not search or filter on — index bloat is the main cause of clusters that need more hardware than they should.

text versus keyword deliberately. Brand is `keyword` so it can be aggregated for the facet counts; title is `text` so it can be searched by word. Getting that pair wrong requires a reindex to fix, so I would write the mapping explicitly and set `dynamic: strict`, because product data has seller-supplied attributes and dynamic mapping on those is how you get a mapping explosion.

Query structure: the keyword match in `must`, and category, brand, price range and `is_active` in `filter`. Filters do not score and are cached, which at 5,000 queries a second is the largest easy win available. Facet counts come from aggregations in the same request.

Now the stock and price problem, which is the real question. Prices and stock change far more often than titles do, and every change would otherwise mean reindexing the document. Two things.

First, **I would not put stock in the index at all if I can avoid it.** Filtering out-of-stock items can happen at hydration time, in the database, where the number is authoritative. The cost is that the result count and the facet counts are slightly wrong, which for a product listing is acceptable and for a facet count is a product conversation. If out-of-stock filtering must be a facet, then stock goes in as a boolean `in_stock`, not a number — a boolean flips rarely, a count changes constantly.

Second, **price does have to be in the index**, because filtering and sorting by price are core. So price changes do trigger reindexing, and I would make the indexer coalesce: if the same product is updated five times in ten seconds, index it once. That is a per-id deduplication window in the consumer and it can cut indexing load by an order of magnitude on a marketplace where sellers run repricing bots.

Sizing. 20 million products at maybe 1 KB indexed is 20 GB, times 1.2 overhead, times two for a replica is about 50 GB on disk. That is one or two shards by the size rule — but at 5,000 queries a second I would use more nodes for query throughput, with replicas rather than shards, since replicas are the read lever and can be changed later.

Aliases from day one, because the mapping will change — a marketplace always adds a new filterable attribute — and I want that to be a fifteen-minute background reindex and an atomic alias switch rather than a planned outage.

Monitoring: query p99, the age of the oldest unprocessed outbox row, the zero-result search rate, and cluster health. **The zero-result rate is the one I would put in front of the product team,** because it is the clearest signal that search is failing users and the fix is usually synonyms rather than engineering.

And the nightly reconciliation, comparing counts and checksums per id range. Over a year the index will drift — a consumer that died over a weekend, a bulk price import that bypassed the outbox — and this is the only mechanism that finds it.”

9 Recall card

Elasticsearch is a derived copy, never the source of truth, and must be rebuildable from the database in an afternoon. If something lives only there, the design is wrong.

Write path: row + outbox in one transaction → consumer → bulk index. Never dual-write from the application — no transaction across two systems, and the inconsistency is permanent and silent. CDC is the alternative.

Read path: search returns ids; the application hydrates from the database. A stale index then shows a slightly wrong *set* but never wrong *content*, and deleted items filter themselves out.

Shards are fixed at creation and are read-path overhead (every query touches all of them); replicas are the read-throughput lever and can change any time. Size shards at 10–50 GB, keep heap ≤31 GB and ≤50% of RAM — **the other half is page cache, which is where search speed comes from.**

Query the alias, never a concrete index, so a mapping change is a reindex plus an atomic switch instead of an outage. `text` for searching, `keyword` for filtering and aggregating; put non-relevance conditions in `filter`; and run a reconciliation job, because the index *will* drift.

DSA topic: Dijkstra's algorithm **System design topic:** Elasticsearch in a design

Code these, in this order

One rule for the whole set: **write** `heapq.heappush(heap, (cost, vertex))` **and say “cost first” out loud.** The reversed tuple orders the heap by vertex id, produces a confidently wrong answer, and never raises. It is the fastest possible way to lose twenty minutes.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Network Delay Time	LeetCode 743 (Medium)	Plain Dijkstra, then <code>max</code> of the distances, then the unreachable check.
2	Path with Minimum Effort	LeetCode 1631 (Medium)	<code>max</code> instead of <code>+</code> — the generalised combining rule.
3	Cheapest Flights Within K Stops	LeetCode 787 (Medium)	The state variant, and why plain Dijkstra is wrong.
4	Swim in Rising Water	LeetCode 778 (Hard)	Same as 2, on an implicit grid.
5	Path with Maximum Probability	LeetCode 1514 (Medium)	Multiplication and a max-heap — the third combining rule.
6	Minimum Cost to Make at Least One Valid Path	LeetCode 1368 (Hard)	Weights are only 0 and 1 — use 0-1 BFS and compare.

On problem 3, prove plain Dijkstra fails first

Solve it with ordinary Dijkstra ignoring the stop limit, then apply the limit afterwards. Find an input where it gives the wrong answer, and write one sentence saying why — the phrase “settled once” should appear. Then expand the state and solve it properly. Then solve it a third time with Bellman-Ford limited to `k+1` rounds, and say which you would write in an interview.

On problems 2, 4 and 5, name the combining rule

For each, write down the accumulate operation and the comparison. Then state, in one sentence, the property all three share that makes Dijkstra's proof still hold.

On problem 6, compare against 0-1 BFS

Solve it with Dijkstra and with a deque-based 0-1 BFS. Time both on the largest input and record the ratio. Say why the deque version has no $\log$ factor.

Then the negative-weight experiment

Build this graph by hand and run your Dijkstra on it:

```
0 → 1 cost 1
0 → 2 cost 2
2 → 1 cost -5
```

Record what it returns for the distance to vertex 1, and what the true answer is. Then write the one sentence of the correctness proof that this input breaks.

Then the stale-entry measurement

Instrument your solution to count (a) heap pushes, (b) heap pops, and (c) pops skipped as stale. Run it on a graph with many improving paths — a dense random graph works. Three numbers. Then delete the stale check and count how many extra neighbour scans happen.

The substitution drill

1. State what Dijkstra is, in terms of BFS, in one sentence.
2. Say what the priority queue changes and what stays the same.
3. Write the eleven-line core from memory.
4. Point at the relaxation line and the lazy-deletion line and say what each does.
5. Say why the tuple is $(\text{cost}, \text{vertex})$ and not the other way round.

The proof drill

1. State the invariant in one sentence.
2. Give the three-sentence proof.
3. Point at the exact sentence that needs non-negative weights.
4. Give a three-vertex counter-example with a negative edge.
5. Say what “extending a path never improves it” generalises to.

The variants drill

1. Write the early-exit version and say why it returns on pop, not on discovery.
2. Write the parent-map version and say where the assignment goes.
3. Write the state-space version for a stop limit.

4. Give the state-space size for V vertices and k stops, and say when you would worry.
5. Give the `max`-instead-of-`+` version and the problem family it solves.

The costs drill

1. Derive $O(E \log E)$ and explain why it is written $O((V + E) \log V)$.
2. Give the array version's cost and the graph density where it wins.
3. Compute heap versus array at $V = 1,000, E = 500,000$ and at $V = 100,000, E = 300,000$.
4. Compare against BFS on the same sparse graph and give the ratio.
5. Compute the memory of the heap at $V = 10^6, E = 5 \times 10^6$.
6. Give the whole comparison table: BFS, 0-1 BFS, Dijkstra, Bellman-Ford, Floyd-Warshall, A^* .

The break-it drill

Trigger each and record the exact output or error:

1. A negative edge, on the three-vertex graph above.
2. Returning when the target is discovered rather than popped.
3. The stale check removed — measure, do not just observe.
4. `(vertex, cost)` instead of `(cost, vertex)`.
5. Pushing an object with no tie-breaker at equal cost.
6. Dijkstra on a graph where every weight is 1 — measure against BFS.
7. Negating weights to turn a maximisation into a minimisation.

The role drill

1. State what Elasticsearch is in the architecture, in one sentence.
2. State Sudha's two rules in system terms.
3. Say what "must be rebuildable" rules out.
4. Draw the write path and the read path from memory.
5. Say what the read path buys you by returning ids rather than content.

The sync drill

1. Name the three mechanisms and reject one with a specific reason.
2. Describe the outbox version, including what is in the transaction.
3. Say what CDC catches that the outbox does not.
4. Say why indexing is idempotent for free.

5. Say why a reconciliation job is not optional, with two causes of drift.

The mapping drill

1. State the `text` versus `keyword` distinction and what each is for.
2. Say what happens if you get it wrong and what fixing it costs.
3. Say what dynamic mapping does and why it is dangerous on user input.
4. Say which mapping changes are possible in place and which are not.

The query drill

1. State the difference between `must` and `filter`.
2. Say why filters are cheaper, in two ways.
3. Say what belongs in each, with three examples.
4. Say what deep pagination costs and what to use instead.

The alias drill

1. Say what an alias is and why the application must use one.
2. Give the five steps of a zero-downtime reindex.
3. Name the step people forget and what it costs.
4. Estimate the reindex time for 50M documents on 3 nodes.

The sizing drill

1. Compute index size on disk for 50M documents at 2 KB, with a replica.
2. Derive the shard count from that.
3. Give the heap rule and the reason for the 31 GB ceiling.
4. Say why the other half of RAM matters more than the heap.
5. Give query and indexing throughput per node, bulk and single-document.

The staleness drill

1. Decompose end-to-end staleness into its four components.
2. Give the typical total and what it becomes under load.
3. Say what you alert on and why not the configured interval.
4. Say what `?refresh=true` per write does and why it is the worst fix.
5. Give the three product answers to “I cannot find what I just created”.

The failure drill

For each, say what happens and what you would build:

1. The application writes to the database and then to Elasticsearch, and crashes in between.
2. A brand field mapped as `text`, and the product team asks for brand facets.
3. A user searches and sees a product deleted an hour ago.
4. Dynamic mapping on seller-supplied attributes, after a year.
5. A cluster goes red during a search.
6. `from: 100000` in a user-facing paginator.
7. The application queries `products_v1` directly and the mapping must change.

Two of the seven need a reindex to fix. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the shortest path in a weighted graph.* Dijkstra as BFS with a priority queue, the settled invariant, the three-sentence proof, the sentence that needs non-negative weights, lazy deletion, and returning on the pop.
 2. *There is a limit of at most k stops. What changes?* Nothing about the algorithm and everything about the vertex — the state becomes `(city, stops)`, why plain Dijkstra settles too early, the state-space size computed out loud, and Bellman-Ford as the neater alternative here.
 3. *Where does Elasticsearch sit, and how does it stay up to date?* Derived copy, never the source of truth, rebuildable. Outbox in one transaction, consumer, bulk index. Search returns ids and the app hydrates. Staleness as a number. Reconciliation as a requirement.
-

Before you move on

- ☐ I can state Dijkstra in terms of BFS in one sentence.
- ☐ I write `(cost, vertex)` and say “cost first”.
- ☐ I can write the eleven-line core from memory.
- ☐ I can state the settled invariant and give the three-sentence proof.
- ☐ I can point at the sentence that requires non-negative weights.
- ☐ I have a three-vertex negative-weight counter-example ready.
- ☐ I know why lazy deletion exists and what the check costs to omit.

- [] I return on the pop, not on the discovery, and can say why.
- [] I put the parent assignment inside the relaxation.
- [] I can write the state-space variant and compute the state-space size.
- [] I know the `max`-instead-of-`+` family and the property that makes it valid.
- [] I can derive the complexity and explain the two ways of writing it.
- [] I know where the array version beats the heap version.
- [] I know Dijkstra is $\sim 13 \times$ BFS and would say “BFS” when weights are equal.
- [] I can give the six-algorithm comparison table.
- [] I know Elasticsearch is a derived copy and must be rebuildable.
- [] I can draw both paths from memory.
- [] I know why search returns ids and not content.
- [] I can reject dual write with a specific reason.
- [] I know what CDC catches that the outbox does not.
- [] I know why a reconciliation job is required.
- [] I can state `text` versus `keyword` and what getting it wrong costs.
- [] I know why dynamic mapping is dangerous on user input.
- [] I know the `must` versus `filter` distinction and what belongs where.
- [] I query an alias, never a concrete index.
- [] I can give the five reindex steps and name the forgotten one.
- [] I can size shards from data volume and give the heap and cache rules.
- [] I can decompose staleness and say what I alert on.
- [] I know what forcing a refresh per write does.
- [] I answered all three questions above out loud.

Bellman-Ford, and what negative edges break

DATA STRUCTURES AND ALGORITHMS

Bellman-Ford, and what negative edges break

SYSTEM DESIGN

Time-series and metrics stores

Bellman-Ford, and what negative edges break

1 What this is, and why they ask it

Bellman-Ford finds shortest paths when edges can have negative weights. Dijkstra cannot, and yesterday's lesson explained why in one sentence: **Dijkstra settles a vertex permanently, and a negative edge is exactly a reason to unsettle one.**

Bellman-Ford's answer is to give up on settling. Instead of cleverly choosing what to expand next, it relaxes **every edge**, over and over, $V - 1$ times. That is slower — $O(V \times E)$ against $O((V + E) \log V)$ — and it is correct in cases where Dijkstra is confidently wrong.

It also does something Dijkstra cannot do at all: **detect a negative cycle**. If a loop of edges has a negative total, there is no shortest path — you can go round it again and get cheaper forever — and Bellman-Ford finds that in one extra pass.

They ask it mainly as a follow-up: “why can't you use Dijkstra here?” The interviewer wants to hear the proof step that fails, not just “negative weights”. The algorithm itself is eight lines and the interesting content is entirely in the reasoning, the negative-cycle detection, and knowing which of the four shortest-path algorithms fits a given problem.

By the end of this lesson you can write Bellman-Ford and its cycle check from memory, say exactly why $V - 1$ rounds are enough, explain what “no shortest path” means, use the SPFA optimisation and say why it is not always safe, and choose correctly between BFS, Dijkstra, Bellman-Ford and Floyd-Warshall.

2 The story

Gopal has eleven lorries and he has been sending goods between the same fourteen towns for nineteen years.

Most of it is straightforward. Nashik to Pune costs him a certain amount in diesel and driver time; he knows the number and it does not move much.

The complication is the return loads.

If a lorry is going empty from Solapur back to Nashik anyway, and somebody in Solapur has goods for Nashik, Gopal will take them for almost nothing — sometimes for genuinely nothing, and once or twice, when the alternative was an empty run and a

driver sitting idle, for less than nothing, because the customer also paid the driver's food for two days and Gopal came out ahead on a leg that on its own is a loss.

So on his list of legs there are about a hundred and forty entries, and nine of them have a number in front of them that is not a cost at all. It is a gain.

This is what makes his work different from the man in Nashik who does the same job and always takes the cheapest leg he can see.

That man's method works perfectly on ordinary legs. Take the cheapest thing available, commit to it, move on. It does not work on Gopal's list, and Gopal can tell you exactly why, because he lost money learning it in 2011.

He had goods for Aurangabad. The direct leg was 4,000 rupees and he took it, because it was the cheapest thing in front of him and there was nothing to think about. What he did not consider was that going Nashik to Solapur cost 6,000, and Solapur to Aurangabad was one of the return-load legs and would have *paid him* 3,000 — so the whole thing was 3,000 rupees, and he had paid 4,000 for the direct one.

He had committed to a number before he had looked at everything.

So what he does now takes longer and he does it on Sunday evenings. He goes down the whole list of a hundred and forty legs, one at a time, and updates his best-known cost to every town. Then he does it again from the top. Then again. He keeps going until a full pass through the list changes nothing at all, and then he stops, because if a whole pass changes nothing, another one will not either.

It usually settles after five or six passes.

The one thing that would worry him — and it has never happened, but he checks — is if a pass kept changing things forever. That would mean there was a circle of towns he could go round and come out ahead every time, and if that were true on his list, then either somebody had quoted him something wrong, or he should stop carrying goods and just drive the circle.

3 The idea in plain English

Gopal's Sunday evening is Bellman-Ford, and his 2011 mistake is Dijkstra on negative edges.

A negative edge is one where taking it reduces your total. In transport it is a return load. In finance it is a rebate or an arbitrage. In a puzzle it is a move that gives resources back. **Physical distances are never negative, which is why most shortest-path problems do not need this** — and why, when a problem does have negative weights, it is usually a deliberate choice by whoever set it.

Dijkstra fails because it settles. When a vertex comes out of the priority queue, Dijkstra treats its distance as final and never looks at it again. That is exactly Gopal committing to 4,000 rupees for the direct leg. The proof from [day 136](#) needed one clause — *since no edge is negative, the remainder of the route cannot bring the total below what we already have* — and a negative edge deletes that clause.

Bellman-Ford gives up the cleverness. No priority queue, no settling, no choosing what to expand. Just: **go through every edge and relax it, then do that again, $V - 1$ times.**

```
for round in 1 .. V-1:
    for every edge (u, v, w):
        if distance[u] + w < distance[v]:
            distance[v] = distance[u] + w
```

Six lines, and there is no data structure in it at all.

Why $V - 1$ rounds are exactly enough, and this is the argument to have ready. A shortest path can visit at most V vertices, so it has at most $V - 1$ edges — any more and it would repeat a vertex, meaning it contains a cycle, and a cycle with non-negative weight can be removed to make the path shorter or equal.

Now: **after round 1, every shortest path of one edge is correct.** After round 2, every shortest path of two edges is correct — because its first edge was already right and this round relaxed the second. By induction, after round k , every shortest path using at most k edges is correct. So $V - 1$ rounds cover every possible shortest path.

Notice what that argument does not depend on: the order you process the edges in. Any order works. A lucky order finishes sooner, which is why the early-exit optimisation below is valid.

Early exit: if a full pass changes nothing, stop. Gopal's rule. If no distance improved in a whole round, no distance can improve in the next one either, because the inputs to every comparison are unchanged. On real graphs this usually terminates in far fewer than $V - 1$ rounds.

Now the negative cycle, which is the part Dijkstra cannot do at all. A **negative cycle** is a loop whose edge weights sum to less than zero. Go round it once and you are cheaper; go round it again and you are cheaper still. **There is no shortest path** — not a large one, not an undefined one: the question has no answer, because for any path you name, a shorter one exists.

Detecting it is one extra round. After $V - 1$ rounds, everything reachable without a negative cycle is final. So run one more round: **if anything still improves, that improvement can only have come from a negative cycle.** Six extra lines, and it is the

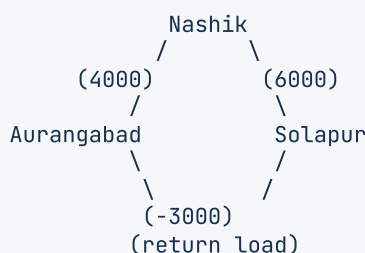
reason to reach for this algorithm even when Dijkstra would work.

And what “affected by a negative cycle” means matters. Not every vertex is. The vertices whose distance is genuinely $-\infty$ are those *reachable from* a negative cycle. A vertex the cycle cannot reach has a perfectly well-defined shortest distance. **If you need to report which vertices are affected, run a traversal from every vertex that improved on the extra round** and mark everything it reaches.

SPFA is the practical speed-up, with a caveat. Instead of blindly relaxing all E edges every round, keep a queue of vertices whose distance just changed — only those can improve anything. That is the **Shortest Path Faster Algorithm**, it is often dramatically faster on real graphs, and **its worst case is still $O(V \times E)$** and adversarial inputs exist that hit it. Competitive judges sometimes include such inputs deliberately. Know it, use it, and do not claim it is asymptotically better.

4 The picture

Gopal’s 2011 mistake, as a graph:



DIJKSTRA: pops Aurangabad at 4000, SETTLES it, never reconsiders.
answer: 4000

TRUTH: Nashik $\rightarrow$ Solapur $\rightarrow$ Aurangabad = 6000 - 3000 = 3000

Dijkstra is wrong by 1000, and reports no error.

What to notice. Dijkstra did not do anything careless. It popped the smallest available number, which is its whole design. The graph simply violates the assumption its correctness rests on.

Bellman-Ford on the same graph, round by round:

```

vertices: N(ashik)=source, A(urangabad), S(olapur)
edges:    N→A 4000, N→S 6000, S→A -3000

start      N=0      A=inf   S=inf

round 1     relax N→A:  0 + 4000 = 4000 < inf   → A = 4000
            relax N→S:  0 + 6000 = 6000 < inf   → S = 6000
            relax S→A:  6000 - 3000 = 3000 < 4000 → A = 3000 ← improved!

round 2     relax all three: nothing improves
            → early exit

answer      A = 3000      correct

```

What to notice at round 1. A was improved twice in a single round, because the edges happened to be processed in a helpful order. A different order would have taken two rounds. **Either is fine** — the $V - 1$ bound holds regardless of order, which is exactly what makes this algorithm so forgiving.

A negative cycle, and what it means:

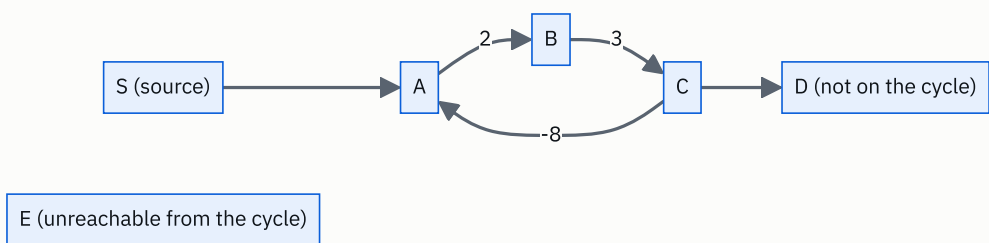


FIGURE 137.1

```

the loop A → B → C → A costs 2 + 3 - 8 = -3

go round once:    -3
go round twice:   -6
go round n times: -3n      → no lower bound → NO shortest path

which vertices are -inf?
  A, B, C         on the cycle
  D               reachable FROM the cycle
  E               NOT reachable from it → its distance is a normal, finite number

```

What to notice. “The graph has a negative cycle” does not mean every answer is meaningless. E’s distance is perfectly well defined. Reporting “no solution” for the whole graph over-states the problem, and a good answer distinguishes them.

And the four algorithms, on one picture:

every edge costs the same?	→ BFS	$O(V + E)$
weights are only 0 and 1?	→ 0-1 BFS	$O(V + E)$
weights non-negative?	→ Dijkstra	$O((V+E) \log V)$
weights can be negative?	→ Bellman-Ford	$O(V \times E)$
need ALL pairs, small V?	→ Floyd-Warshall	$O(V^3)$
graph is a DAG?	→ topological	$O(V + E)$ any weights!

What to notice on the last line. On a directed acyclic graph, any weights — including negative ones — are handled by a single pass in topological order, from [day 135](#). That is faster than all of them, and it is the answer people forget.

5 The code, built step by step

The edge list, because Bellman-Ford never asks for neighbours.

```
edges = [(0, 1, 4000), (0, 2, 6000), (2, 1, -3000)] # (from, to, weight)
```

This is the one graph algorithm where an edge list is the right representation — it iterates over all edges and never over one vertex's neighbours, so converting to an adjacency list would be wasted work. That is worth saying in an interview.

Now the core.

```
INF = float("inf")

def bellman_ford(n: int, edges: list[tuple[int, int, float]], source: int) → list[float]:
    distance = [INF] * n
    distance[source] = 0
    for _ in range(n - 1): # V - 1 rounds
        changed = False
        for u, v, w in edges:
            if distance[u] != INF and distance[u] + w < distance[v]:
                distance[v] = distance[u] + w
                changed = True
        if not changed:
            break # a whole pass did nothing
    return distance
```

Eight lines and no data structure. Three things to point at.

`distance[u] != INF` is not optional. Without it, `inf + (-5)` is `inf`, which compares as not-less-than `inf`, so it happens to work — until a weight is such that floating-point arithmetic misbehaves, or until you use a sentinel like `10**18` instead of `inf`, at which case `10**18 - 5 < 10**18` is true and you start propagating distances from unreachable vertices. **With an integer sentinel this check is mandatory**, and it is good practice regardless.

`changed` and the `break` are Gopal's rule: a pass that improves nothing means every subsequent pass improves nothing.

The order of `edges` is irrelevant to correctness. A helpful order finishes in fewer rounds; the `V - 1` bound holds either way.

Now the negative-cycle check, which is one more round:

```
def find_negative_cycle(n: int, edges, source: int) → bool:
    distance = bellman_ford(n, edges, source)
    for u, v, w in edges:
        # the V-th round
        if distance[u] ≠ INF and distance[u] + w < distance[v]:
            return True
    # still improving → negative cycle
    return False
```

After `V - 1` rounds everything is final unless a negative cycle exists, so any further improvement proves one does. Six lines, and it is the capability Dijkstra does not have.

To report *which* vertices are affected:

```
def negative_infinity_vertices(n: int, edges, source: int) → set[int]:
    distance = bellman_ford(n, edges, source)
    on_cycle = set()
    for u, v, w in edges:
        if distance[u] ≠ INF and distance[u] + w < distance[v]:
            on_cycle.add(v)
            # v is reachable from a neg cycle

    # everything reachable from those is also -inf
    graph: dict[int, list[int]] = {i: [] for i in range(n)}
    for u, v, _ in edges:
        graph[u].append(v)
    stack, affected = list(on_cycle), set(on_cycle)
    while stack:
        node = stack.pop()
        for nxt in graph[node]:
            if nxt not in affected:
                affected.add(nxt)
                stack.append(nxt)
    return affected
```

The extra traversal is the honest part: a vertex is `-∞` if it is **reachable from** a negative cycle, not merely if the graph contains one somewhere.

To recover the cycle itself:


```
def extract_cycle(n: int, edges, source: int) → list[int] | None:
    distance = [INF] * n
    parent: list[int | None] = [None] * n
    distance[source] = 0
    victim = None
    for round_number in range(n):
        victim = None
        for u, v, w in edges:
            if distance[u] ≠ INF and distance[u] + w < distance[v]:
                distance[v] = distance[u] + w
                parent[v] = u
                victim = v
    if victim is None:
        return None
    for _ in range(n):
        victim = parent[victim]
    cycle, node = [], victim
    while True:
        cycle.append(node)
        node = parent[node]
        if node == victim:
            break
    return cycle[::-1] + [victim]
```

Walking back n steps before collecting is the trick, and it is not obvious: the vertex that improved on round n might not itself be on the cycle, only reachable from it. After n parent steps you are guaranteed to be inside the cycle, because the parent chain must have entered it.

And SPFA, the queue-based version:

```
from collections import deque

def spfa(n: int, adjacency: dict[int, list[tuple[int, float]]], source: int) → list[float] | None:
    distance = [INF] * n
    distance[source] = 0
    in_queue = [False] * n
    times = [0] * n
    queue = deque([source])
    in_queue[source] = True
    while queue:
        u = queue.popleft()
        in_queue[u] = False
        for v, w in adjacency[u]:
            if distance[u] + w < distance[v]:
                distance[v] = distance[u] + w
                if not in_queue[v]:
                    queue.append(v)
                    in_queue[v] = True
                    times[v] += 1
                if times[v] ≥ n:
                    return None
    return distance
```

Only vertices whose distance just changed can improve anything, so there is no point relaxing edges out of unchanged vertices. That is the whole optimisation, and on typical graphs it is several times faster.

$\text{times}[v] \geq n$ is the cycle check: a vertex cannot legitimately be improved n times, so reaching that count means it is being fed by a negative cycle.

The complete solution

```
"""Bellman-Ford: shortest paths with negative weights, and negative-cycle detection."""

from __future__ import annotations

from collections import deque

INF = float("inf")

def bellman_ford(n: int, edges: list[tuple[int, int, float]], source: int) → list[float]:
    """Shortest distances, assuming no reachable negative cycle.  $O(V \times E)$ ."""
    distance = [INF] * n
    distance[source] = 0
    for _ in range(n - 1):
        changed = False
        for u, v, w in edges:
            if distance[u] ≠ INF and distance[u] + w < distance[v]:
                distance[v] = distance[u] + w
                changed = True
        if not changed:
            break
    return distance

def has_negative_cycle(n: int, edges, source: int) → bool:
    """One extra round: any further improvement proves a negative cycle."""
    distance = bellman_ford(n, edges, source)
    return any(
        distance[u] ≠ INF and distance[u] + w < distance[v]
        for u, v, w in edges
    )

def affected_by_negative_cycle(n: int, edges, source: int) → set[int]:
    """Vertices whose true distance is minus infinity: reachable FROM a negative cycle."""
    distance = bellman_ford(n, edges, source)
    seeds = {v for u, v, w in edges if distance[u] ≠ INF and distance[u] + w < distance[v]}
    graph: dict[int, list[int]] = {i: [] for i in range(n)}
    for u, v, _ in edges:
        graph[u].append(v)
    affected, stack = set(seeds), list(seeds)
    while stack:
        node = stack.pop()
        for nxt in graph[node]:
            if nxt not in affected:
                affected.add(nxt)
                stack.append(nxt)
    return affected

def spfa(n: int, edges, source: int) → list[float] | None:
    """Queue-based Bellman-Ford. Faster in practice, same worst case. None = negative cycle."""
    adjacency: dict[int, list[tuple[int, float]]] = {i: [] for i in range(n)}
    for u, v, w in edges:
        adjacency[u].append((v, w))
    distance = [INF] * n
    distance[source] = 0
    in_queue = [False] * n
    times = [0] * n
    queue = deque([source])
    in_queue[source] = True
    while queue:
        u = queue.popleft()
        in_queue[u] = False
        for v, w in adjacency[u]:
            if distance[u] + w < distance[v]:
                distance[v] = distance[u] + w
                if not in_queue[v]:
                    queue.append(v)
```

```

        in_queue[v] = True
        times[v] += 1
        if times[v] >= n:
            return None
    return distance

if __name__ == "__main__":
    # Gopal's 2011 mistake. 0 Nashik, 1 Aurangabad, 2 Solapur.
    transport = [(0, 1, 4000), (0, 2, 6000), (2, 1, -3000)]
    print("bellman-ford:", bellman_ford(3, transport, 0))
    print("spfa      :", spfa(3, transport, 0))
    print("neg cycle? :", has_negative_cycle(3, transport, 0))

    # A genuine negative cycle: 1 → 2 → 3 → 1 sums to -3.
    # 0 source, 4 reachable from the cycle, 5 unreachable from it.
    cyclic = [
        (0, 1, 1), (1, 2, 2), (2, 3, 3), (3, 1, -8), (3, 4, 1), (0, 5, 7),
    ]
    print("neg cycle? :", has_negative_cycle(6, cyclic, 0))
    print("affected  :", sorted(affected_by_negative_cycle(6, cyclic, 0)))
    print("spfa      :", spfa(6, cyclic, 0))

```

Running it:

```

bellman-ford: [0, 3000, 6000]
spfa      : [0, 3000, 6000]
neg cycle? : False
neg cycle? : True
affected   : [1, 2, 3, 4]
spfa      : None

```

Three things to look at. Aurangabad comes out at **3000**, not 4000 — the answer Dijkstra gets wrong.

The `affected` set is `{1, 2, 3, 4}`: the three vertices on the cycle plus vertex 4, which the cycle can reach. **Vertex 5 is not in it**, because the cycle cannot reach it — its distance of 7 is perfectly well-defined even though the graph contains a negative cycle. Reporting “no solution” for the whole graph would over-state the problem.

And `spfa` returns `None` on the cyclic graph, detecting the cycle through the requeue counter rather than through an extra round.

6 What it costs

Bellman-Ford.

rounds	$V - 1$
edges relaxed per round	E

	$O(V \times E)$ time
distance array	$O(V)$
edge list	$O(E)$

	$O(V + E)$ space

Put numbers on it against Dijkstra:

$V = 10,000, E = 50,000$	
Bellman-Ford	$10,000 \times 50,000 = 500,000,000$
Dijkstra	$50,000 \times 14 = 700,000$
	→ ~700x more work

$V = 1,000, E = 5,000$	
Bellman-Ford	$5,000,000$
Dijkstra	$50,000$
	→ 100x

Bellman-Ford is not slightly slower, it is a different order of magnitude, and that is why you only use it when you have to. In Python, five hundred million operations is minutes; seven hundred thousand is instant.

The early exit changes the practical picture a great deal:

worst case	$V - 1$ rounds
typical real graph	3-10 rounds until nothing changes
	→ $O(k \times E)$ with k small

$V = 10,000, E = 50,000$, settles in 6 rounds	
$6 \times 50,000 = 300,000$	→ comparable to Dijkstra

Say the worst case and then say the practical one, because “ $O(V \times E)$ ” but it usually terminates in a handful of rounds” is a more useful statement than either half alone.

SPFA:

worst case	$O(V \times E)$	-- same
typical	$O(E)$ to $O(k \times E)$ with very small k	
adversarial inputs	genuinely hit the worst case	

Do not claim SPFA is asymptotically better. It is a constant-factor optimisation with a bad worst case that judges sometimes construct on purpose.

Detecting a negative cycle:

```
one extra pass over all edges  O(E)
finding which vertices are -inf one traversal, O(V + E)
extracting the cycle itself    O(V) after the main run
```

All four algorithms, sized on the same graph:

```
V = 1,000, E = 10,000
```

BFS	11,000	unweighted only
Dijkstra	100,000	non-negative
Bellman-Ford	10,000,000	negatives allowed
Floyd-Warshall	1,000,000,000	all pairs

Floyd-Warshall's $O(V^3)$ is only viable to about $V = 500$:

V = 100	1,000,000	instant
V = 500	125,000,000	about a minute in Python
V = 1,000	1,000,000,000	too slow

But note what it buys: **all pairs**, negative edges allowed, and negative-cycle detection (a negative value on the diagonal). Running Bellman-Ford from every source is $O(V^2 \times E)$, which is far worse — so for all-pairs with negatives on a small graph, Floyd-Warshall is right.

And Johnson's algorithm, worth naming for all-pairs on a *sparse* graph with negatives: one Bellman-Ford run to compute a re-weighting that makes every edge non-negative, then Dijkstra from every source. $O(V \times E \log V)$, far better than $O(V^3)$ when $E \ll V^2$. Name it; you will not be asked to write it.

Space in practice:

```
V = 100,000, E = 500,000
distance array      800 KB
edge list as tuples 500,000 x ~72 bytes = 36 MB
```

The edge list is the memory, and holding edges as three parallel arrays rather than a list of tuples cuts that by roughly five times in Python — worth knowing if it ever matters.

7 The traps

Reaching for Bellman-Ford when Dijkstra would do

The most common real mistake, and it is not incorrect, just wasteful:

```
bellman_ford(n, edges, source)           # all weights are positive
```

```
V = 10,000, E = 50,000
Bellman-Ford    500,000,000 operations
Dijkstra        700,000
```

Time Limit Exceeded

Check the weights first. All non-negative means Dijkstra. All equal means BFS. It is a DAG means one pass in topological order, whatever the weights.

Forgetting the `INF` guard

```
for u, v, w in edges:
    if distance[u] + w < distance[v]:           # no check that u is reachable
        distance[v] = distance[u] + w
```

With `float("inf")` this happens to survive, because `inf + w` is `inf`. With an integer sentinel it does not:

```
INF = 10**18
distance = [INF] * n
# ... relax an edge out of an unreachable u with w = -5
# 10**18 - 5 < 10**18 is TRUE
```

```
>>> bellman_ford_no_guard(4, [(0,1,1), (2,3,-5)], source=0)
[0, 1, 99999999999999995, 99999999999999990]
```

Distances propagating from vertices that cannot be reached at all. **With any integer sentinel the guard is mandatory**, and competitive programming uses integer sentinels almost universally.

Running `v` rounds instead of `v - 1`

```
for _ in range(n):                             # one too many
```

Harmless for the distances — an extra round changes nothing on a graph with no negative cycle — but it **silently destroys the cycle detection**, because the check “did round `v` change anything” has already been absorbed into the main loop. Then you conclude there is no negative cycle when there is.

Assuming a negative cycle means the whole answer is undefined

```
if has_negative_cycle(n, edges, source):  
    return "no solution"
```

Only vertices **reachable from** the cycle are $-\infty$. On the example in section 5, vertex 5 has a perfectly good distance of 7. **Reporting “no solution” for the whole graph overstates the problem**, and on problems that ask for per-vertex answers it is simply wrong.

Undirected graphs with negative edges

```
edges = [(0, 1, -5), (1, 0, -5)]          # an "undirected" negative edge
```

An undirected edge of weight -5 is two directed edges of -5 , and going back and forth along it is a negative cycle of length two. **Every undirected graph with any negative edge has a negative cycle**, so the question “shortest path in an undirected graph with negative weights” has no answer at all. If a problem states that, it is either directed or you have misread it.

Claiming SPFA is faster

```
# "SPFA is  $O(E)$ "
```

It is not. Its worst case is $O(V \times E)$, identical to plain Bellman-Ford, and constructed inputs exist that achieve it — grid-like graphs with particular weight patterns. Competitive judges have included them deliberately since around 2018. **Use it, and describe it as a practical optimisation.**

The floating-point comparison

```
if distance[u] + w < distance[v]:          # with float weights
```

With floating-point weights, an “improvement” of $1e-17$ can keep firing round after round on a graph with no negative cycle, and the algorithm never converges — reporting a negative cycle that does not exist. **Use integers where possible**, and if you cannot, compare with a small epsilon: `distance[u] + w < distance[v] - 1e-9`.

8 In the interview

How it gets asked

- “*Why can’t you use Dijkstra here?*” — the most common form, and the answer is the proof step.
- “*Some edges have negative weights. Find the shortest path.*”
- “*Detect whether an arbitrage opportunity exists between these currencies.*” — the classic negative-cycle application.
- “*Cheapest flight with at most k stops.*” — Bellman-Ford limited to $k + 1$ rounds is the neat solution.
- “*How do you know there is no answer?*”
- “*Which shortest-path algorithm, and why?*” — the whole decision table.

The first ninety seconds

“Negative weights, so Dijkstra is out, and I want to say precisely why rather than just name the constraint.

Dijkstra’s correctness rests on one clause: when it pops a vertex, any cheaper route would have to leave the settled set through a vertex costing at least as much, and **since no edge is negative**, the remainder cannot bring the total lower. A negative edge deletes that clause — a longer route can still end up cheaper — so a settled vertex would need reopening, and Dijkstra never reopens. It does not give a slightly worse answer; its reasoning stops holding, and it returns a confidently wrong number with no error.

So: Bellman-Ford. It gives up on settling entirely. Relax **every** edge, then do it again, $V - 1$ times. No priority queue, no data structure at all, and an edge list rather than an adjacency list — this is the one graph algorithm that genuinely wants an edge list, because it never asks for a vertex’s neighbours.

Why $V - 1$ is exactly enough: a shortest path has at most $V - 1$ edges, because more would repeat a vertex. After round k , every shortest path using at most k edges is correct — by induction, since its first $k - 1$ edges were already right and this round relaxed the last one. So $V - 1$ rounds cover everything.

And one round more gives me something Dijkstra cannot do at all. After $V - 1$ rounds everything is final unless there is a negative cycle, so if a V -th round still improves something, a negative cycle exists — go round it again and you are cheaper forever, so there is no shortest path.

Cost is $O(V \times E)$, which on ten thousand vertices and fifty thousand edges is five hundred million operations against Dijkstra’s seven hundred thousand — about seven hundred times more. So I would only use it when I have to, and I would add the early exit: if a whole pass improves nothing, stop, which on real graphs usually terminates in five or six rounds.

Are negative cycles possible here, and do you want the distances or just whether a cycle exists?”

The follow-ups

“Give me a concrete example where Dijkstra fails.”

“Three vertices. Source S , with $S \rightarrow A$ costing 1 and $S \rightarrow B$ costing 2, and $B \rightarrow A$ costing -5 .

Dijkstra pops S at 0 and relaxes both, so the heap holds $(1, A)$ and $(2, B)$. It pops A at 1 — the smallest — and **settles it**. Then it pops B at 2 and relaxes $B \rightarrow A$, finding $2 - 5 = -3$, which is better. But A is settled and Dijkstra never revisits a settled vertex, so depending on the implementation either the improvement is discarded or A is re-pushed and the reported answer is already wrong.

The true distance to A is -3 . Dijkstra says 1.

The general statement is that Dijkstra never reopens a settled vertex, and a negative edge is precisely a reason to reopen one.

The trap I would watch for in a real problem is negative weights that arrive through a transformation. If I am asked for the *maximum*-value path and I negate the weights to turn it into a minimisation, I have just created negative weights and invalidated Dijkstra. On a DAG that is fine — one pass in topological order handles any weights — but on a general graph it is Bellman-Ford, and if a negative cycle exists there is no answer.”

“Detect arbitrage between currencies.”

“This is the classic application and the transformation is the interesting part.

Each currency is a vertex. An exchange rate r from A to B is an edge. Arbitrage means a cycle whose rates **multiply** to more than 1 — you go round and end up with more money than you started.

Bellman-Ford works with sums, not products, so I take logarithms: $\log(r_1 \times r_2 \times r_3) = \log r_1 + \log r_2 + \log r_3$. A product greater than 1 becomes a sum greater than 0. Then I negate, so the edge weight is $-\log(\text{rate})$ — and now a profitable cycle is a **negative** cycle, which is exactly what Bellman-Ford detects.

So: build the graph with weights $-\log(\text{rate})$, run Bellman-Ford, and run the extra round. If anything still improves, arbitrage exists. To report the actual sequence of trades, I walk the parent pointers back V steps to land on the cycle and then follow it round.

Two practical caveats I would name. The weights are floats, so an improvement of $1e-17$ can fire spuriously and report arbitrage that is not there — I would compare against an epsilon rather than exactly. And real exchange rates have spreads and fees, which are usually enough to remove any arbitrage the raw mid- rates suggest, so the model needs the fee subtracted from each edge before it means anything.”

“Cheapest flight with at most k stops.”

“This is where Bellman-Ford is genuinely neater than Dijkstra, and it is because of what a *round* means.

After k rounds of Bellman-Ford, every distance reflects the best path using **at most k edges** — that is exactly the induction from the correctness proof, read as a feature rather than a proof step. So ‘at most k stops’ means at most $k + 1$ flights, which means running exactly $k + 1$ rounds and stopping.

The one detail that catches people: **each round must relax against the distances as they were at the start of the round**, not as they are being updated within it. Otherwise a single round can chain two flights together and you allow more stops than intended. So I copy the distance array at the start of each round and read from the copy while writing to the live one.

Cost is $O(k \times E)$, which for $k = 10$ and five thousand flights is fifty thousand operations — much simpler and faster than the Dijkstra version, which needs the state expanded to $(\text{city}, \text{stops})$ and carries a heap.

I would offer both and say I would write the Bellman-Ford one here, because the stop limit maps directly onto the round count and there is no state expansion to get wrong.”

“Which algorithm, and why?” — walk me through the decision.

“Five questions, in this order, and each one rules out work.

Is it a DAG? If so, one pass in topological order handles *any* weights, including negative, in $O(V + E)$. That is faster than everything else and it is the answer people forget.

Does every edge cost the same? BFS, $O(V + E)$.

Are the only weights 0 and 1? 0-1 BFS with a deque — push zero-cost edges to the front, one-cost to the back. Still $O(V + E)$, no heap, no log factor.

Are all weights non-negative? Dijkstra, $O((V + E) \log V)$.

Can weights be negative? Bellman-Ford, $O(V \times E)$, and it also tells me whether there is a negative cycle — which is a question the others cannot answer at all.

And separately: **do I need all pairs?** If V is small, Floyd-Warshall at $O(V^3)$ — viable to about 500 in Python — and it handles negatives and detects negative cycles on the diagonal. If V is large but the graph is sparse, Johnson's: one Bellman-Ford run to re-weight every edge to non-negative, then Dijkstra from every source, $O(V \times E \log V)$.

The point of going through it in that order is that the cheapest applicable algorithm is usually several times faster than the general one, and **reaching for Bellman-Ford on a non-negative graph is a seven-hundred-fold slowdown for nothing.**“

The model answer

“A game where moving between rooms costs energy, and some rooms contain potions that restore energy. Find the minimum energy needed to get from the entrance to the exit. Also tell me if the player can farm energy forever.”

“The potions make this negative-weight, and the farming question is a negative-cycle question, so this is Bellman-Ford on both counts. Let me build it up.

The model. A vertex is a room. An edge from room A to room B is a passage, and its weight is `movement cost - potion value in B` — so a room with a large potion gives its incoming edges a negative weight. I would state that combination explicitly, because folding the room’s value into the incoming edges is the modeling step, and doing it wrong — for instance charging the potion on the way out — gives a subtly different and wrong answer.

Why not Dijkstra. With potions, a longer route through a well-stocked room can be cheaper than a direct one, so a room settled at some cost can turn out to be reachable for less. Dijkstra never reopens a settled vertex, so it would return the direct route’s cost and be confidently wrong.

So Bellman-Ford, `V - 1` rounds over the edge list, with the early exit when a pass changes nothing. On a game map of, say, a thousand rooms and four thousand passages, that is four million operations worst case and typically far less — well within budget for something computed once per level rather than per frame.

The farming question is the extra round, and it is the same code. If a `V`-th pass still improves something, there is a cycle of rooms the player can loop round gaining energy indefinitely. I would report that as a level-design bug rather than as an answer, because it means the level has no minimum energy — any figure I gave would be wrong.

And I would be careful to answer it precisely rather than globally. A negative cycle in one wing of the map does not make the whole map meaningless: only rooms **reachable from** the cycle have undefined cost. So I would report which rooms are affected — seed from the vertices that improved on the extra round, then traverse forwards — and give proper answers for the rest. For a level editor that is a far more useful output than ‘this level is broken’.

Two edge cases from the domain. If the exit is unreachable, the answer is infinity, and I need the `distance[u]  $\neq$  INF` guard so unreachable rooms do not propagate garbage — especially since I would use an integer sentinel here rather than a float, and with an integer sentinel that guard is mandatory rather than merely tidy. And if the *entrance* has a potion, that is a starting bonus rather than an edge weight, so it belongs in the initial distance, not in the graph.

The thing I would flag as a design question rather than an algorithm one: the model assumes a potion can be collected every time you enter a room, which is what makes farming possible. If potions are one-shot, the state is `(room, set of`

potions taken) and the state space is exponential — at which point this stops being a shortest-path problem and becomes a search problem, and I would want to know which the game means before writing anything.”

9 Recall card

Dijkstra settles a vertex permanently; a negative edge is exactly a reason to unsettle one. That is the proof clause that fails, and it is the answer to “why not Dijkstra”.

Bellman-Ford: relax every edge, $V - 1$ times. No data structure, and an **edge list** is the right representation. $V - 1$ because a shortest path has at most $V - 1$ edges, and after round k every k -edge shortest path is correct.

One extra round detects a negative cycle — anything still improving can only come from one. Only vertices **reachable from** the cycle are $-\infty$; the rest have valid answers.

$O(V \times E)$ — about $700 \times$ Dijkstra at $V=10^4, E=5 \times 10^4$ — so use it only when weights can be negative. Early exit when a pass changes nothing usually terminates in a handful of rounds. SPFA is a practical speed-up with the **same worst case**.

The decision order: DAG $\rightarrow$ one topological pass (any weights); equal weights $\rightarrow$ BFS; 0/1 $\rightarrow$ deque BFS; non-negative $\rightarrow$ Dijkstra; negative $\rightarrow$ Bellman-Ford; all pairs and small $V \rightarrow$ Floyd-Warshall. And with an integer sentinel, the `distance[u]  $\neq$  INF` guard is mandatory.

1 What this is, and why they ask it

Every system you build produces measurements. Requests per second, latency, queue depth, CPU, error counts, temperature from a sensor, price of a stock. They arrive constantly, they are never updated once written, and they are read in ranges — “the last six hours” — rather than one at a time.

That access pattern is unlike anything a relational database is built for, and at volume the difference is not a tuning problem. A million measurements a second into a Postgres table is not slow, it is impossible: the index maintenance alone exceeds what the machine can do.

A **time-series database** exploits the shape. Points arrive in timestamp order, consecutive values are similar, nothing is ever updated, and old data is wanted at lower resolution. Each of those facts buys an enormous saving, and together they turn 16 bytes per point into about 1.5.

They ask this because “where would you store the metrics for this system?” is a question every design touches, and because the failure mode has a name — **cardinality explosion** — that separates people who have run a monitoring system from people who have configured one. **The number of distinct series, not the number of points, is what kills these systems**, and knowing that is most of the answer.

By the end of this lesson you can say what makes time-series data special, size a metrics store from a fleet size, explain compression and downsampling with numbers, name the cardinality trap and how to avoid it, and choose between Prometheus, a wide-column store and a columnar analytical database.

2 The story

Kulkarni has read the society’s water meter at six every morning since 2011.

It is in a pit behind the pump room with a steel cover, and the routine is the same: lift the cover, wipe the glass, read the number, put it in his phone.

The numbers themselves are enormous and almost identical. 1,847,203. The next morning, 1,847,391. The morning after, 1,847,566.

He worked out early that what he actually cares about is never the number. It is the gap. One eighty-eight, then one seventy-five, then a Sunday when it is two forty because everybody is home and washing.

Once, in 2017, the gap was eleven hundred and he was down there before nine looking for the leak. Found it too, in the line to the back garden tap, and the committee reckoned it had been running for three days.

Two other things he has settled over the years, without anybody telling him to.

He does not keep every day forever. This month's daily readings, yes, all of them. Last month's, he keeps as a monthly figure and a rough daily average. For 2014, all he has is a number for the year and a note that it was high because of the tanker business in May. The old detail went, and he has never once wanted it back — when somebody at a meeting asks about water, the question is always “are we using more than last year” and never “what was the reading on the fourteenth of March 2014”.

And the second thing, which is the one his son found funny when he saw the phone. Kulkarni does not write down the whole number any more. He writes the gap. 188. 175. 240. Three characters instead of seven, and he can always add them up if he needs the total, which he does about once a year for the committee.

His son, who works with computers, looked at it for a moment and said something about how that was clever, and Kulkarni said it was not clever, it was that his thumbs are sixty-three years old and seven digits every morning for fourteen years is a lot of digits.

3 The idea in plain English

Kulkarni's meter book is a time-series database, and the three habits he arrived at are the three ideas.

A time series is a named sequence of timestamped values. The name is not just a string — it is a **metric name plus a set of labels**:

```
http_requests_total{service="checkout", method="POST", status="500", host="web-14"}
```

Every distinct combination of label values is a separate series. That sentence is where all the trouble comes from, and section 7 is mostly about it.

Four properties make this data unlike anything else, and each one buys a specific optimisation:

It arrives roughly in time order and is never updated. No `UPDATE`, no `DELETE` of individual rows. That means the storage can be append-only, sorted, and immutable — no random writes, no page splits, no index rebalancing.

Consecutive values are similar. CPU was 41.2% and is now 41.4%. Kulkarni writing 188 instead of 1,847,391. Storing the *difference* instead of the value makes the number tiny, and tiny numbers compress brutally well.

Timestamps are regular. Scrapes every 15 seconds means the gap between timestamps is 15,000 milliseconds, every time. So you store the difference of the differences — the **delta of delta** — which is almost always **zero**, and zero costs one bit.

Queries are ranges and aggregates, never point lookups. “Average latency per minute over the last six hours”, not “the value at 14:32:07.412”. So the storage is arranged in **chunks by time**, and a query reads a few contiguous blocks instead of seeking.

Now the arithmetic that follows. A naive row is a timestamp (8 bytes) plus a value (8 bytes) plus a series identifier — call it 16 bytes minimum, and in a relational table with an index far more. With delta-of-delta timestamps and XOR-compressed floats, the real-world figure is **1 to 2 bytes per point**. That is roughly a **tenfold saving**, and it is the difference between a metrics system that fits on one machine and one that does not.

Downsampling is Kulkarni’s second habit. Keep every point for a short window, then keep only summaries.

raw, 15-second resolution	kept for 15 days
1-minute averages	kept for 90 days
1-hour averages	kept for 2 years

And you keep more than the average. Storing only the mean loses the spikes, which is usually the thing you actually needed. Minimum, maximum, count and sum let you reconstruct averages over any longer window and still see the worst moment.

Retention is deletion by time, and it is cheap here. Because data is stored in immutable chunks by time range, expiring old data is deleting whole files. No `DELETE FROM ... WHERE timestamp < ...` scanning a table, no vacuum, no index bloat. **That is a real architectural advantage and it is worth naming.**

Counters versus gauges is the other thing to get right. A **gauge** goes up and down — temperature, queue depth, memory in use. A **counter** only increases — total requests served, total bytes sent — and what you want from it is the **rate**, not the value. Kulkarni’s meter is a counter, and the useful thing is the daily gap, not the reading.

Counters reset when the process restarts, so a rate calculation has to notice a value that went *down* and treat it as a reset rather than as a huge negative rate. Every real system handles this; it is worth knowing that it is a case that needs handling.

And the trap that dominates operating these systems: cardinality. Adding a label with many distinct values multiplies the number of series. A `user_id` label on a metric with a million users is a million series, each needing its own index entry and its own in-memory chunk. **Systems fall over from too many series, almost never from too many points**, and the rule is: **labels must have bounded, low cardinality.** Never a user id, never a request id, never a full URL path with ids in it, never an unbounded error message.

4 The picture

What a series looks like, and why the compression works:

```
series:  cpu_usage{host="web-14"}

timestamp      value      naive bytes  what is actually stored
-----
10:00:00.000   41.2       16           full timestamp + full value
10:00:15.000   41.4       16           delta-of-delta: 0   value XOR: tiny
10:00:30.000   41.3       16           delta-of-delta: 0   value XOR: tiny
10:00:45.000   41.5       16           delta-of-delta: 0   value XOR: tiny
10:01:00.000   44.9       16           delta-of-delta: 0   value XOR: small
-----
5 points      80 bytes      ~8 bytes
```

What to notice. The timestamps cost almost nothing after the first, because the gap is always 15,000 ms so the delta-of-delta is always zero — one bit each. And the values are floats that differ only in their low bits, so XOR-ing consecutive values leaves mostly zeros. **This is Facebook's Gorilla scheme and it is what every modern time-series store uses.**

Storage layout, and why range queries are fast:

```
BY TIME CHUNKS (what a TSDB does)

[ 00:00 - 02:00 ] [ 02:00 - 04:00 ] [ 04:00 - 06:00 ] ...
  all series      all series      all series
  immutable      immutable      being written

query "cpu of web-14, last 4 hours"
  → open 2 chunks, seek to this series, read contiguously
  → a few disk reads

RETENTION
  delete chunks older than 15 days → unlink whole files
  no row-by-row DELETE, no vacuum, no index bloat
```

Downsampling, drawn as what survives:

raw (15 s)		15 days
1-minute		90 days
1-hour		2 years

storage for one series over 2 years:

raw for 15 days	15 x 24 x 240	=	86,400 points
1-min for 90 days	90 x 24 x 60	=	129,600 points
1-hour for 2 years	730 x 24	=	17,520 points

			233,520 points
raw for 2 years would be			4,204,800 points
			→ 18x less

What to notice. Keeping full resolution forever costs eighteen times more for data nobody looks at. And the 1-minute rollup stores min, max, sum and count — not just the average — so a spike at 10:03:22 is still visible as a maximum a year later.

And the cardinality trap, which is the picture to remember:

```

http_requests_total{service, method, status}
  services 20
  methods  4
  statuses 8
                20 x 4 x 8 = 640 series           fine

... add host
  hosts    200
                640 x 200 = 128,000 series        large but workable

... add user_id
  users    1,000,000
                128,000 x 1,000,000
                = 128,000,000,000 series          the system is dead

the POINT count did not change at all. Only the SERIES count did.

```

5 How it actually works

The compression, concretely

Two techniques, both from Facebook's Gorilla paper, both now universal.

Delta-of-delta timestamps. Store the first timestamp fully. Then store the delta. Then, for each subsequent point, store `delta - previous delta`. On a regular scrape interval that value is 0, encoded as a single bit. A one-second jitter is a small number needing a handful of bits.

```

timestamps 10:00:00, 10:00:15, 10:00:30, 10:00:46, 10:01:01
deltas      15,          15,          16,          15
delta-of-deltas      0,          1,          -1
                  ^            ^            ^
                  1 bit      ~9 bits    ~9 bits

```

XOR-compressed floats. Consecutive float values usually share their sign, exponent and most of their mantissa, so `current XOR previous` is mostly zeros. Store the number of leading zeros, the number of meaningful bits, and those bits. A repeated value XORs to exactly zero — one bit.

```

41.2 XOR 41.4 → mostly zeros, ~14 meaningful bits
41.4 XOR 41.4 → all zeros, 1 bit

```

Together: Facebook reported an average of 1.37 bytes per point against 16 uncompressed, a twelvefold reduction, and that is the number to quote.

The systems, and what each is for

SYSTEM	MODEL	BEST FOR	WEAKNESS
Prometheus	pull, local storage	infrastructure monitoring, alerting	single node, ~weeks of retention
VictoriaMetrics / Thanos / Mimir	Prometheus-compatible, clustered	long retention, many clusters	more to operate
InfluxDB	push, purpose-built	IoT, application metrics	query language churn across versions
TimescaleDB	Postgres extension	time-series plus relational joins	inherits Postgres's write ceiling
ClickHouse	columnar OLAP	high-cardinality events, analytics	not a metrics system out of the box
Amazon Timestream / GCP Monitoring	managed	not wanting to run any of this	cost, lock-in

The most useful distinction is Prometheus's pull model. Prometheus **scrapes** targets over HTTP on a schedule rather than receiving pushes. That sounds like a detail and has real consequences:

- **Service discovery is built in** — it asks Kubernetes what exists and scrapes it, so a new pod is monitored automatically.
- **A target being down is itself a signal** — the `up` metric goes to 0, which is often the alert you actually want.

- **It does not work for short-lived jobs** that finish before a scrape, which is what the Pushgateway exists for and why that component always feels bolted on.

TimescaleDB is the interesting middle option and is worth knowing. It is a Postgres extension that partitions a table by time into **hypertable chunks**, adds columnar compression on older chunks, and gives you continuous aggregates for downsampling — while keeping full SQL and the ability to join metrics against ordinary relational tables. **If you need “average latency per customer tier”, where the tier lives in a customers table, that join is trivial in TimescaleDB and awkward everywhere else.**

Why a plain relational table fails

```
CREATE TABLE metrics (  
    series_id BIGINT,  
    ts        TIMESTAMPTZ,  
    value     DOUBLE PRECISION  
);  
CREATE INDEX ON metrics (series_id, ts);
```

This is correct and it dies at volume, for four reasons and all of them matter:

- **Index maintenance per row.** Every insert updates a B-tree. At a million rows a second, that is the bottleneck long before the disk is.
- **Row overhead.** Postgres row headers are ~24 bytes, so a 16-byte measurement costs about 40 bytes stored, plus the index. **Roughly 30 times a compressed TSDB.**
- **Deleting old data is expensive.** `DELETE FROM metrics WHERE ts < now() - '30 days'` scans, writes dead tuples, and then autovacuum has to reclaim them — on a hot table, at volume, this alone can be unsustainable.
- **No downsampling.** You either keep everything or write and maintain rollup jobs yourself.

Partitioning by time fixes the third problem completely — dropping a partition is instant — and it is why TimescaleDB’s hypertables exist. It does not fix the first two.

Query languages

PromQL is worth being able to read, because it appears in interviews:

```
# per-second rate of 5xx, averaged over 5 minutes, by service  
sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))  
  
# 99th percentile latency from a histogram  
histogram_quantile(0.99,  
    sum by (le, service) (rate(http_request_duration_seconds_bucket[5m])))
```

`rate()` is the function that matters, and it is what turns a counter into something meaningful. It also handles counter resets — a value going down is treated as a restart, not as a negative rate.

Histograms deserve a note, because percentiles are where naive metrics go wrong. You cannot average percentiles: the mean of each host's p99 is not the fleet's p99. So latency is recorded as a **histogram** — counts in buckets — and percentiles are computed from the summed buckets at query time. **Each bucket is a separate series**, so a histogram with 12 buckets multiplies cardinality by 12, which is the main reason not to put histograms on everything.

The other axis: logs, metrics, traces

Metrics are numbers over time, aggregated, cheap, and lose per-request detail. **Logs** are individual events with full detail and are far more expensive. **Traces** follow one request across services.

Metrics tell you something is wrong; traces tell you where; logs tell you why. That sentence is a good thing to say, and the practical consequence is that you do not solve a metrics cardinality problem by putting per-user labels on metrics — you solve it by putting the user id in a trace or a log, where high cardinality is normal and expected.

6 The numbers

Sizing from a fleet. 500 machines, 1,000 metrics each, scraped every 15 seconds:

series	$500 \times 1,000$	= 500,000
points per second	$500,000 / 15$	= 33,333 per second
points per day	$33,333 \times 86,400$	= 2,880,000,000

uncompressed at 16 bytes	= 46 GB per day
compressed at 1.5 bytes	= 4.3 GB per day
	→ ~10x

15 days of raw at 4.3 GB/day	= 65 GB
------------------------------	---------

Sixty-five gigabytes for a fortnight of full-resolution metrics on a 500-machine fleet, which fits on one node comfortably. That is why Prometheus's single-node design works for so many people.

With downsampling for long retention:

raw 15 s, 15 days	500,000 series x 86,400 pts x 1.5 B	= 65 GB
1-min, 90 days	500,000 x 129,600 x 1.5 B	= 97 GB
1-hour, 2 years	500,000 x 17,520 x 1.5 B	= 13 GB

		175 GB

raw kept for 2 years instead: 500,000 x 4,204,800 x 1.5 B = 3.15 TB
→ 18x more

Ingestion rates:

Prometheus, single node	~ 500,000 - 1,000,000 samples/s
VictoriaMetrics, per node	~ 1,000,000+ samples/s
InfluxDB, per node	~ 500,000 samples/s
TimescaleDB	~ 100,000 - 1,000,000 rows/s (with batching)
plain Postgres, indexed	~ 10,000 - 50,000 rows/s

Plain Postgres is one to two orders of magnitude short, and the gap is index maintenance rather than disk.

Memory, which is the real Prometheus constraint:

Prometheus holds ~2 hours of each active series in memory	
per active series, roughly	~3 KB of in-memory chunk + index
500,000 series	~1.5 GB
2,000,000 series	~6 GB
10,000,000 series	~30 GB and the process is fragile

The memory scales with the number of *series*, not with the number of points. That is the sentence that explains cardinality explosion in one line.

Cardinality, worked out:

base metric with 3 labels	
service 20 x method 4 x status 8	= 640 series
+ host (200)	= 128,000
+ a histogram with 12 buckets	= 1,536,000
+ user_id (1,000,000)	= 1.5 x 10 ¹² → dead

And the same story as an accident: somebody adds a `path` label to an HTTP metric, and the paths include ids — `/orders/8837412`. That is one series per order.

100,000 orders a day x 12 histogram buckets	= 1,200,000 new series per day accumulating
---	---

A monitoring system killed by a one-line change, and the point count barely moved.

Query cost:

```
"average CPU across 200 hosts, last 6 hours" at 15 s resolution
  200 series x 1,440 points          = 288,000 points read
  → tens of milliseconds

"p99 latency by endpoint, last 30 days" at raw resolution
  50 endpoints x 12 buckets x 172,800 pts = 103,680,000 points
  → seconds to minutes
  with 1-hour rollups: 50 x 12 x 720    = 432,000 → instant
```

Downsampling is not only about storage — it is what makes long-range queries possible at all.

Cost, roughly:

self-hosted Prometheus + Grafana, 500 hosts	1-2 VMs, ~\$200/month
managed metrics (Datadog-style)	often \$15-25 per host per month
	500 hosts = \$7,500-12,500/month

That gap is why large fleets self-host, and it is also why per-host pricing makes people delete metrics they should keep.

7 The trade-offs

You give up per-event detail. A metric is an aggregate: “142 requests returned 500 in this minute”. It cannot tell you *which* requests, or what the user id was, or what the payload looked like. That is what logs and traces are for, and the mistake is trying to recover the detail by adding labels — which is exactly the cardinality explosion.

You give up flexible querying. A time-series store answers “this series, over this range, aggregated this way” extremely well and “join these measurements against the customer table” not at all. TimescaleDB is the exception and it pays for that with a lower write ceiling.

Downsampling permanently destroys detail. Once the raw fifteen-second data is gone, no query can recover the shape of a spike inside a minute. **Storing min, max, sum and count rather than only the mean recovers most of what matters**, and choosing rollup windows is choosing what future incidents you will be able to investigate.

Cardinality is a hard limit, not a soft one. Systems degrade non-linearly: fine at a million series, sluggish at five, dead at twenty. And because a label is added in application code, the failure arrives from a deploy by someone who was not thinking about

monitoring at all. **Enforce it with review and with a limit on the ingest side**, because the alternative is discovering it during an incident when your monitoring is the thing that is down.

Push versus pull is a genuine trade. Pull gives you service discovery, up/down as a first-class signal, and no client-side buffering to get wrong. It cannot see short-lived jobs, and it needs network access from the monitor to every target — awkward across NAT or firewalls. Push handles ephemeral work and firewalled networks, and gives you no way to distinguish “healthy and silent” from “dead”.

And the operational one: your monitoring must not depend on the thing it monitors. A metrics system running inside the cluster it watches tells you nothing during the outage you most care about. The usual answer is a small independent monitoring stack watching the main one, and an external check watching that.

When would I not use a time-series database? When the volume is small — a few thousand points a minute in a Postgres table with a time-partitioned index is completely fine and is one fewer system. When the questions are really analytical, over high-cardinality event data, where a columnar store like ClickHouse is a better fit than a metrics system. And when the real requirement is per-request debugging, where the answer is tracing, not metrics with more labels.

8 In the interview

How it gets asked

- “Where would you store the metrics for this system?” — the direct version.
- “Why not just put them in a Postgres table?”
- “Your monitoring system fell over. What happened?” — almost always cardinality.
- “How do you keep two years of metrics without spending a fortune?”
- “How would you compute the p99 latency across the fleet?” — the histogram question.
- “Design a monitoring system.” — where all of this appears as follow-ups.

The first ninety seconds

“A time-series database, and the reason is that the data has four properties a relational table does not exploit.

It is append-only and arrives in time order, so storage can be immutable chunks sorted by time — no index rebalancing, no page splits, and expiring old data is deleting whole files rather than a `DELETE` that scans and then needs vacuuming.

Consecutive values are similar and timestamps are regular, so you store delta-of-delta timestamps — which on a fixed scrape interval is zero, one bit — and XOR-compressed floats. That takes 16 bytes per point down to about 1.5. Facebook’s Gorilla paper reports 1.37, and it is a tenfold saving.

Queries are ranges and aggregates, never point lookups, so the layout is optimised for reading contiguous blocks.

And old data is wanted at lower resolution, so you downsample: raw for a fortnight, one-minute rollups for three months, hourly for two years. That is about eighteen times less storage than keeping everything, and more importantly it is what makes a thirty-day query possible at all.

Concretely, for 500 machines with a thousand metrics each scraped every 15 seconds: that is 500,000 series, 33,000 samples a second, and about 4.3 gigabytes a day compressed. Fifteen days of raw data is 65 gigabytes, which fits on one machine — which is why single-node Prometheus works for a lot of people.

The thing I would raise before you ask is cardinality, because that is what actually kills these systems. Every distinct combination of label values is a separate series, and memory scales with the number of series, not the number of points. Adding a `user_id` label to a metric on a system with a million users creates a million series per existing combination, and the point count does not change at all. **Labels must have bounded, low cardinality — never a user id, a request id, or a URL path containing ids.**

How large is the fleet, and do you need to join metrics against business data? Because that last one is the question that pushes me towards TimescaleDB rather than Prometheus.”

The follow-ups

“Why not a Postgres table? Be specific.”

“Four reasons, and I would give the numbers rather than assert it.

Index maintenance. Every insert updates a B-tree, and at scale that is the bottleneck long before the disk is. Indexed Postgres does maybe ten to fifty thousand rows a second; a purpose-built store does a million. **One to two orders of magnitude**, and it is not a tuning gap.

Row overhead. A Postgres row header is about 24 bytes, so a 16-byte measurement costs around 40 bytes stored, plus the index entry. Against 1.5 bytes compressed, that is roughly thirty times the storage.

Deleting old data. `DELETE FROM metrics WHERE ts < now() - interval '30 days'` scans, marks dead tuples, and hands the problem to autovacuum, on a table that is being written to constantly. At volume this alone can be unsustainable. A time-series store deletes a whole chunk file.

No downsampling. You keep everything or you write and maintain rollup jobs yourself.

But I would not say never. At a few thousand points a minute, a time-partitioned Postgres table is completely fine and is one fewer system to run — partitioning by time fixes the deletion problem entirely. And TimescaleDB is genuinely the right answer when I need to join metrics against relational data, because ‘p99 latency by customer tier’, where the tier lives in a customers table, is trivial in SQL and awkward in PromQL.”

“Our monitoring fell over. What happened?”

“Cardinality, almost certainly, and I would say what to look for.

Memory in these systems scales with the number of active **series**, not with the number of points — roughly a few kilobytes of in-memory chunk and index per active series. Half a million series is a gigabyte and a half; ten million is thirty gigabytes and a fragile process.

The usual cause is a deploy. Somebody adds a label to an existing metric — a `path` label on an HTTP counter, and the paths contain order ids. That is one new series per order, multiplied by however many other labels already existed, and multiplied again by twelve if it is a histogram. A hundred thousand orders a day becomes 1.2 million new series a day, accumulating. **The person who did it was not thinking about monitoring at all**, which is why review does not catch it.

Diagnosis: look at series count over time and find the step change, then look at which metric contributed. Prometheus exposes this directly, and the query for the worst offenders by label is one line.

The fix has three parts. Immediately, drop the offending label at ingest with a relabel rule, which stops the bleeding without a deploy. Then remove it in code. Then prevent recurrence: a limit on series per metric at the ingest side, and a rule that anything unbounded — user ids, request ids, paths with ids, error strings — goes into a trace or a log, where high cardinality is normal, not into a metric label.

And the deeper point I would make: the monitoring system must not be able to be killed by an application deploy, so the ingest-side limit is the real fix and the code change is a follow-up.”

“How do you compute p99 latency across the whole fleet?”

“Not by averaging per-host percentiles, and that is the first thing to say, because it is the common mistake. The mean of each host’s p99 is not the fleet’s p99 — percentiles do not average, and the error is largest exactly when the load is uneven, which is when you care.

The correct approach is **histograms**. Each host records counts in latency buckets — under 10 ms, under 25, under 50, and so on — and every bucket is a counter. To get the fleet p99, you sum the buckets across hosts and then interpolate the quantile from the summed distribution. In PromQL that is `histogram_quantile(0.99, sum by (le) (rate(bucket[5m])))`, and the important part is that the summing happens **before** the quantile is computed.

The cost is cardinality: a histogram with twelve buckets is twelve series per label combination, so histograms multiply your series count by their bucket count. That is why you put them on the handful of metrics where latency distribution matters and not on everything.

The accuracy limit is the bucket boundaries. You cannot get a more precise answer than your bucket widths, so the boundaries have to be chosen for the range you care about — and if your p99 falls in the bucket ‘between 1 second and 10 seconds’, the interpolated answer is a guess. Native histograms in newer Prometheus versions fix this with exponential buckets, which is worth mentioning.

The alternative, summaries, computes the quantile on each host — which is accurate per host and cannot be aggregated at all, so it answers a different and usually less useful question.”

“Keep two years of metrics without spending a fortune.”

“Downsampling and tiered retention, and I would give the arithmetic.

Raw fifteen-second data for two years on 500,000 series is about 3.1 terabytes. Raw for fifteen days, one-minute rollups for ninety days, and one-hour rollups for two years is about 175 gigabytes. **Eighteen times less**, for data nobody looks at in full resolution after a fortnight.

The design detail that matters is what you store in a rollup. Not just the mean — minimum, maximum, sum and count. That way a spike at 10:03 is still visible as a maximum a year later, and you can recompute the average over any longer window correctly from the sum and count. Storing only the mean is the mistake that makes historical data useless for exactly the questions people ask of it.

And downsampling is not only about storage — it is what makes long queries possible. A thirty-day p99 query over raw data is a hundred million points and takes minutes; over hourly rollups it is four hundred thousand points and is instant.

Implementation: Prometheus alone does not do this well, so it is Thanos, Mimir or VictoriaMetrics for the long-term tier, with the older chunks in object storage — which at two cents a gigabyte makes even the raw retention affordable if you really wanted it. Or continuous aggregates in TimescaleDB, which does it in SQL.

The question I would ask back is what the two years are actually for. If it is capacity planning and year-on-year comparison, hourly is plenty. If it is a compliance requirement to reproduce an exact incident timeline, that is a different requirement and probably belongs in logs in object storage, not in a metrics system.”

The model answer

“Design the monitoring for a platform: 2,000 services across 10,000 containers, plus business metrics like orders per minute. Engineers need dashboards and alerts, and the data must be queryable for a year.”

“Let me split this into three tiers, because ‘metrics’ here covers three things with different requirements and treating them the same is the mistake.

Tier one: infrastructure and service metrics. Ten thousand containers, maybe 500 metrics each after exporters. That is five million series before any application labels — already large. Prometheus-compatible ingestion, but **not a single Prometheus**: I would shard by cluster or by service group, each instance scraping its own targets, with a global query layer — Thanos or Mimir — federating them. That keeps each instance’s memory manageable and means one cluster’s cardinality problem cannot take down monitoring for the others.

Pull for these, because service discovery is the killer feature at this scale: the monitor asks Kubernetes what exists and scrapes it, so a new deployment is monitored with no configuration. And the `up` metric — a target failing to respond — is often the alert I actually want.

Tier two: business metrics. Orders per minute, revenue, signup conversions. Lower volume, higher value, and they need to be joined against relational data — ‘orders per minute by customer tier’. **This is where I would use TimescaleDB rather than Prometheus**, because that join is trivial in SQL and impossible in PromQL, and the volume is low enough that Postgres’s write ceiling is irrelevant. Two different stores for two different jobs is the right answer here, not a compromise.

Tier three: what does not belong in metrics at all. Per-request detail — user ids, order ids, paths with ids — goes into traces and logs. I would state this as a rule with a mechanism behind it, because the biggest operational risk in this design is a single deploy adding a high-cardinality label. So: an ingest-side limit on series per metric, alerting when it is approached, and relabel rules that drop known-dangerous labels. **The monitoring system must not be killable by an application deploy.**

Retention: raw at 15 seconds for 15 days on local SSD; one-minute rollups for 90 days; one-hour rollups for two years, with the long tail in object storage through Thanos. Rollups store min, max, sum and count, not just the mean. Sizing: five million series at 1.5 bytes a point and a 15-second interval is about 43 gigabytes a day raw, so 650 gigabytes for the fortnight — spread across the shards, that is comfortable per node, and the yearly tier in object storage is a few terabytes at two cents a gigabyte.

Alerting is where I would spend the design time, because a monitoring system nobody trusts is worse than none. Alerts on symptoms rather than causes — error rate and latency that users experience, not CPU — with multi-window burn-rate

alerts against a stated error budget, so a brief blip does not page anyone and a slow burn does. And every alert has to have an owner and a runbook, or it gets deleted.

And the part people leave out: the monitoring must not depend on what it monitors. If Prometheus runs in the cluster it watches, it tells me nothing during the outage I care most about. So a small independent monitoring stack outside the main platform watches the platform, and an external service watches that one. **Two levels, and the outermost one is somebody else's problem** — which is the correct place for it to be.”

9 Recall card

Time-series data is append-only, arrives in order, has similar consecutive values and regular timestamps, and is queried in ranges. Each fact buys an optimisation: immutable time-chunks, delta-of-delta timestamps (one bit on a fixed interval), XOR-compressed floats. **16 bytes per point becomes ~1.5.**

Cardinality kills these systems, not point volume. Every distinct label combination is a series, and memory scales with **series count**. Never label with a user id, request id, or a path containing ids — that detail belongs in traces and logs.

Downsample and tier: raw for ~15 days, 1-minute for 90, 1-hour for 2 years — about **18× less storage**, and it is what makes long-range queries possible at all. **Store min/max/sum/count, not just the mean.**

Percentiles do not average. Use histograms, sum the buckets across hosts, then compute the quantile — and remember each bucket is a series, so histograms multiply cardinality by the bucket count.

Plain Postgres tops out around 10–50k rows/s against ~1M for a purpose-built store, from index maintenance and ~24-byte row headers. Fine at small volume with time partitioning; **TimescaleDB when you need to join metrics against relational data.**

PRACTICE

Practice — Bellman-Ford, and what negative edges break

DSA topic: Bellman-Ford, and what negative edges break **System design topic:** Time-series and metrics stores

Code these, in this order

One rule for the whole set: **before writing a line, check the weights and say which algorithm applies.** All equal $\rightarrow$ BFS. Only 0 and 1 $\rightarrow$ deque BFS. Non-negative $\rightarrow$ Dijkstra. It is a DAG $\rightarrow$ one topological pass, whatever the weights. Negative and cyclic $\rightarrow$ Bellman-Ford. Reaching for Bellman-Ford on a non-negative graph is a seven-hundred-fold slowdown for nothing.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Cheapest Flights Within K Stops	LeetCode 787 (Medium)	$k+1$ rounds — the round count is the edge limit.
2	Network Delay Time	LeetCode 743 (Medium)	Solve with both, and compare the operation counts.
3	Currency arbitrage	Not on LeetCode — build it	The $-\log(\text{rate})$ transformation and cycle detection.
4	Find the City With the Smallest Number of Neighbors	LeetCode 1334 (Medium)	Floyd-Warshall, and why $V \leq 100$ permits it.
5	Minimum Cost to Make at Least One Valid Path	LeetCode 1368 (Hard)	0-1 BFS — the algorithm the decision table picks.

On problem 1, write the copy explicitly

The version that reads and writes the same array in a round chains two flights together and allows more stops than intended. Write both versions, run them on the LeetCode example with $k = 1$, and record the two answers. Then write one sentence saying what a “round” means in the correctness proof.

On problem 2, count the operations

Instrument both solutions to count edge relaxations. Run on the largest input. Two numbers, and the ratio should be roughly what the complexity predicts. Then say which you would write in the interview and why.

On problem 3, build it yourself

Take five currencies and a rate matrix. Convert each rate to $-\log(\text{rate})$, run Bellman-Ford with the extra round, and report whether arbitrage exists. Then plant an arbitrage cycle deliberately and check you find it — and extract the actual sequence of trades with the parent-walk trick.

Then add a 0.3% fee to every trade and see whether the arbitrage survives. It usually does not, and that is the point.

Then the Dijkstra failure

Build the three-vertex graph — $S \rightarrow A$ cost 1, $S \rightarrow B$ cost 2, $B \rightarrow A$ cost -5 — and run your Dijkstra from [day 136](#) on it. Record the distance it gives for A and the true answer. Then write out the sentence of the correctness proof that this input breaks.

Then the sentinel experiment

Run Bellman-Ford with $INF = \text{float}("inf")$ and again with $INF = 10^{18}$, on a graph with an unreachable vertex that has a negative outgoing edge — and with the $\text{distance}[u] \neq INF$ guard removed in both cases. Record all four outputs. One of them is quietly catastrophic.

The why-not-Dijkstra drill

1. State Dijkstra's correctness clause and point at the part negative edges delete.
2. Give a three-vertex counter-example with the numbers.
3. Say what Dijkstra returns and what the truth is.
4. Name the transformation that accidentally creates negative weights.

The algorithm drill

1. Write Bellman-Ford from memory in eight lines.
2. Say why an edge list is the right representation here.
3. Explain why $V - 1$ rounds are enough, using the induction.
4. Say why edge order does not affect correctness.
5. Say what the early exit is and why it is valid.

The negative-cycle drill

1. Say what a negative cycle means for the answer.
2. Write the detection in six lines.
3. Say why one extra round is sufficient.
4. Distinguish “the graph has a negative cycle” from “this vertex is $-\infty$ ”.

5. Describe the parent-walk trick for extracting the cycle and say why the `n` steps are needed.

The SPFA drill

1. Write it from memory.
2. Say what the optimisation actually is, in one sentence.
3. Give its worst case and say what you must not claim.
4. Say how it detects a negative cycle without an extra round.

The decision-table drill

Say the algorithm and its cost for each, from memory:

1. Every edge costs the same.
2. Weights are only 0 and 1.
3. Weights are non-negative.
4. Weights can be negative.
5. The graph is a DAG with arbitrary weights.
6. All pairs, `V = 200`.
7. All pairs, `V = 10,000`, sparse, with negatives.

The costs drill

1. Compute Bellman-Ford against Dijkstra at `V = 10,000`, `E = 50,000`.
2. Give the practical round count with early exit and recompute.
3. Give Floyd-Warshall's cost at `V = 100`, `500` and `1,000`.
4. Say what Floyd-Warshall buys that justifies it at small `V`.
5. Name Johnson's algorithm, its cost, and when it wins.

The break-it drill

Trigger each and record the exact output or error:

1. Bellman-Ford on a positive-weight graph with `V = 10,000` — time it.
 2. The `INF` guard removed with an integer sentinel.
 3. `range(n)` instead of `range(n-1)`, then the cycle check.
 4. An “undirected” graph with one negative edge.
 5. Float weights with an exact `<` comparison, on a graph with no negative cycle.
 6. Reporting “no solution” for a graph with a negative cycle in one corner.
-

The four-properties drill

1. Name the four properties of time-series data.
2. Say what optimisation each one buys.
3. Give the compression figure, before and after.
4. Explain delta-of-delta in one sentence and say why it is usually zero.
5. Explain XOR float compression in one sentence.

The cardinality drill

1. Define a series precisely.
2. Say what memory scales with, and what it does *not* scale with.
3. Work the $20 \times 4 \times 8 \times 200 \times 12 \times 1,000,000$ example out loud.
4. Give three labels you must never use and say where that data belongs instead.
5. Describe how a monitoring system dies from a one-line application change.
6. Give the three-part fix, and say which part is the real one.

The retention drill

1. Give a three-tier retention policy with numbers.
2. Compute storage for each tier on 500,000 series.
3. Compare against keeping raw for two years.
4. Say what a rollup must store besides the mean, and why.
5. Say what downsampling buys beyond storage.

The percentiles drill

1. Say why you cannot average percentiles.
2. Describe how a histogram fixes it, and where the summing happens.
3. Say what histograms cost in cardinality.
4. Say what limits the accuracy and what fixes it.
5. Say what a summary computes instead and why it cannot be aggregated.

The why-not-Postgres drill

1. Give the four reasons with numbers.
2. Say which one partitioning fixes and which two it does not.
3. Give ingestion rates for Postgres and for a purpose-built store.
4. Say when a Postgres table is genuinely the right answer.
5. Say what TimescaleDB is for, with the query that motivates it.

The push-pull drill

1. Describe both models.
2. Give three advantages of pull.
3. Give two things pull cannot do.
4. Say what `up` is and why it matters.

The numbers drill

1. Size a metrics store for 500 machines $\times$ 1,000 metrics at 15 s.
2. Convert that to points per second and gigabytes per day, compressed and not.
3. Compute Prometheus memory for 500,000 and for 10,000,000 series.
4. Compute query cost for “p99 by endpoint over 30 days”, raw and rolled up.
5. Compare self-hosted cost against per-host managed pricing at 500 hosts.

The failure drill

For each, say what happens and what you would build:

1. A `path` label containing order ids is added to an HTTP histogram.
2. A counter’s process restarts mid-window.
3. Someone averages per-host p99s for a fleet dashboard.
4. A rollup that stores only the mean, and an incident from six months ago.
5. Prometheus running inside the cluster it monitors, during a cluster outage.
6. A short-lived batch job that finishes between scrapes.
7. A metrics table in Postgres at a million rows a second.

Two of the seven are working as designed and need a different tool, not a fix. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Why can’t you use Dijkstra here?* The correctness clause and the exact part negative edges delete, the three-vertex counter-example with numbers, then Bellman-Ford, the `V - 1` induction, and the extra round for cycles.
2. *Detect arbitrage between these currencies.* Currencies as vertices, `-log(rate)` as the weight and why, profitable cycle becomes negative cycle, detection in one extra round, extracting the trades — and the two practical caveats.

3. *Where would you store the metrics for this system?* The four properties and what each buys, the compression figure, the sizing arithmetic for a real fleet, downsampling tiers — and cardinality raised before being asked.

Before you move on

- ☐ I check the weights and pick the algorithm before writing anything.
- ☐ I can state Dijkstra's failing clause exactly.
- ☐ I have a three-vertex negative counter-example ready with numbers.
- ☐ I can write Bellman-Ford in eight lines from memory.
- ☐ I know why an edge list is the right representation here.
- ☐ I can give the $V - 1$ induction.
- ☐ I know edge order does not affect correctness.
- ☐ I use the early exit and can say why it is valid.
- ☐ I can write the negative-cycle check and say why one round suffices.
- ☐ I distinguish "has a negative cycle" from "this vertex is $-\infty$ ".
- ☐ I know the parent-walk trick and why it needs n steps.
- ☐ I can write SPFA and will not claim it is asymptotically better.
- ☐ I know the `INF` guard is mandatory with an integer sentinel.
- ☐ I know `range(n)` instead of `range(n-1)` breaks cycle detection.
- ☐ I know an undirected negative edge is always a negative cycle.
- ☐ I can recite the whole decision table with costs.
- ☐ I can name Johnson's algorithm and when it wins.
- ☐ I can name the four properties of time-series data and what each buys.
- ☐ I can explain delta-of-delta and XOR compression.
- ☐ I can quote the bytes-per-point figure.
- ☐ I can define a series and say what memory scales with.
- ☐ I can work the cardinality multiplication out loud.
- ☐ I know the three labels never to use and where that data belongs.
- ☐ I can give the three-part fix for cardinality and name the real one.
- ☐ I can give a retention policy with sizing arithmetic.
- ☐ I know a rollup stores min, max, sum and count.
- ☐ I know why percentiles cannot be averaged and how histograms fix it.
- ☐ I know histograms multiply cardinality by their bucket count.

- [] I can give the four reasons Postgres fails, with numbers.
- [] I know when Postgres and when TimescaleDB are the right answers.
- [] I answered all three questions above out loud.

Union-Find: the disjoint set union

DATA STRUCTURES AND ALGORITHMS

Union-Find: the disjoint set union

SYSTEM DESIGN

Data warehouses: OLAP versus OLTP

1 What this is, and why they ask it

Union-Find answers one question and performs one action. The question is “are these two things in the same group?” The action is “merge these two groups”. Both, with two small optimisations, take effectively constant time.

It is about twenty lines of code and it is the smallest structure in this course with a genuinely surprising property: **its running time is bounded by a function that is at most 4 for any input that will ever exist**. Not amortised-constant in a hand-wavy sense — the analysis is real, and the practical statement is that a `find` costs a handful of array lookups regardless of how many elements you have.

They ask it because a large family of problems is much easier with it and awkward without: connected components as edges arrive, cycle detection in an undirected graph, Kruskal’s minimum spanning tree ([tomorrow](#)), account merging, friend circles, redundant connections, and “how many islands after each addition”. **The tell is grouping that changes over time**, and the moment you hear that, a traversal is the wrong tool.

There is also a specific pair of optimisations, and knowing that you need **both** — and what each one does — is what separates a working answer from a fast one.

By the end of this lesson you can write it from memory, explain path compression and union by size in one sentence each, say why they combine to give near-constant time, name the four problem shapes it solves, and say precisely what it cannot do.

2 The story

The lists went missing three days before sports day, and Meena had two hundred and forty children to sort into four houses.

She is the PT teacher and she has run sports day for eleven years, and what she had instead of the lists was what the class teachers could give her: pairs. Sunita and Reshma are in the same house. Ajay is with Reshma. Vikram is with the boy who sits behind him, whose name somebody had to go and ask.

About four hundred of these, on scraps from eight different teachers, none of them saying which house — only that two children belong together.

So she stood in the yard on Thursday afternoon and did it by chains.

She picked a child. Farida. “Who are you with?” Reshma. She found Reshma. “Who are you with?” Sunita. Found Sunita, who did not have anyone above her, so Sunita was the top of that chain. Meena wrote a 1 next to Sunita’s name and Farida’s group was settled.

The first few took a long time. Some chains ran six or seven deep and she was walking across the yard for each link.

Then she started doing something that halved the work, and she did not plan it, it just seemed obvious after the fourth chain. **On the way back down a chain she had just traced, she told every child in it who the top was.** “You’re with Sunita. You’re with Sunita. You’re with Sunita.” So the next time anyone asked about Farida or Reshma, it was one question and not three.

By four o’clock most children answered immediately.

The other thing she worked out was about joining two groups together. When a teacher said “these two are in the same house” and the two children turned out to be in different chains, one of the chains had to be told to follow the other. **She always pointed the smaller group at the bigger one,** because telling nine children their top has changed is less work than telling forty. Her student teacher did it the other way once, out of politeness to a child who had been a top for a while, and it took twenty minutes longer.

Friday morning she read out four lists and nobody argued, which for sports day is unusual.

3 The idea in plain English

Meena’s afternoon is Union-Find, including both optimisations, and she arrived at both by wanting to stop walking.

The structure holds a collection of disjoint sets. Every element belongs to exactly one group, and no element is in two. That is what **disjoint** means, and the structure’s other name — **disjoint set union**, or DSU — says exactly what it does.

Each group has one representative. Sunita. It does not matter which member it is; what matters is that everyone in a group agrees on the same one. **Two elements are in the same group exactly when they have the same representative**, and that is the whole of the “are these connected?” question.

Every element points at its parent, and the representative points at itself. So a group is a tree, and the representative is the root. There is no list of members anywhere — the group exists only as a set of upward pointers.

`find(x)` walks up until it reaches the root. “Who are you with?” repeated until somebody says nobody.

`union(a, b)` finds both roots and makes one point at the other. One chain is told to follow the other, and the two groups become one.

Without optimisation this can degenerate badly. Merge in an unlucky order and you get one long chain, and then a `find` costs `n` steps — as slow as the naive approach it was meant to replace. That is Meena’s seven-deep chains before she started doing anything about it.

Optimisation one: path compression. After a `find` walks to the root, point every node it passed **directly at the root**. Meena telling the whole chain “you’re with Sunita” on the way back down. The next `find` on any of them is one step. **This is the optimisation with the strange property that reading also writes** — a lookup modifies the structure — and it is why the analysis is amortised rather than worst-case per operation.

Optimisation two: union by size, or by rank. When merging, attach the **smaller** tree under the larger one’s root. Meena pointing the group of nine at the group of forty. This keeps trees shallow, because a tree’s depth can only grow when two trees of similar size merge, and that cannot happen many times.

Union by **rank** is the same idea using an upper bound on height rather than the exact size; both work, size is easier to reason about, and size gives you group sizes for free, which problems often want.

You need both, and each does something the other does not. Union by size alone gives $O(\log n)$ per operation. Path compression alone gives $O(\log n)$ amortised. **Together they give $O(\alpha(n))$** — the inverse Ackermann function, which is at most 4 for any `n` you could ever store. That is not “roughly constant”, it is constant for every practical purpose, and it is one of the few genuinely surprising results in this course.

Now what it is for. Four shapes, and recognising them is the point:

1. **“Are these two connected?” as edges arrive.** The core use. A traversal answers this too, but has to be re-run after every edge; Union-Find answers it in constant time after each merge.

2. **Counting groups.** Start with `n` groups and decrement every time a `union` actually merges two different groups. **“How many islands after each addition”** is exactly this and is a well-known problem.

3. **Cycle detection in an undirected graph.** From [day 132](#): an edge whose two endpoints are already in the same group must close a cycle. No traversal, no parent tracking, and it catches parallel edges that the DFS parent check misses.

4. Kruskal's algorithm. Sort the edges by weight and add each one whose endpoints are in different groups. Union-Find is what makes that check cheap, and it is [tomorrow's](#) lesson.

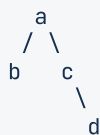
And what it cannot do, which is the follow-up. It cannot un-merge. There is no `split`. Once two groups are joined, the structure has no memory of the boundary. So a problem where edges are *removed* over time is not a Union-Find problem — and the standard trick when the whole sequence is known in advance is to **process it backwards**, turning removals into additions.

It also cannot tell you the path between two elements, only whether one exists. If you need the route, that is a traversal.

4 The picture

The structure as trees, and what each operation does:

```
after union(a,b), union(c,d), union(b,c):
```



```
find(d): d → c → a      2 steps
find(b): b → a           1 step
same root → same group
```

```
parent array:  index  a  b  c  d
                value  a  a  a  c
```

^

a points at itself: it is the root

Path compression, which is Meena's walk back down:

BEFORE find(e)

```
a
|
b
|
c
|
e
```

AFTER find(e)

```
      a
     / | \
    b  c  e
```

← everything on the path
now points straight at a

find(e) costs 3 steps once,
and pays for every future one

every later find on b, c or e costs 1

What to notice. The `find` did not just read the structure, it rewrote it. That is unusual and it is exactly why the cost is amortised: one expensive walk makes many subsequent walks free.

Union by size, and what goes wrong without it:

WITHOUT union by size, merging in a bad order:

union(1,2) union(2,3) union(3,4) union(4,5) ...


```
graph TD; 1 --> 2; 2 --> 3; 3 --> 4; 4 --> 5;
```

`find(5)` costs n steps.
The structure is a linked list.

WITH union by size:


```
graph TD; 1 --- 2; 1 --- 3; 1 --- 4; 4 --- 5;
```

merging a tree of size 1 into a tree of size 4
attaches the small one under the big root.
Depth stays at 1.

What to notice. Both structures represent the same grouping. Only the shape differs, and the shape is the entire cost.

And the counting use, traced:

```
n = 6 elements, start with 6 groups
```

edge	find(a)	find(b)	same?	action	groups
(0,1)	0	1	no	union	5
(1,2)	0	0(via 1)	...	wait: find(2) = 2	4
(0,2)	0	0	YES	nothing – a CYCLE	4
(3,4)	3	4	no	union	3
(4,5)	3	5	no	union	2

final: 2 groups {0,1,2} and {3,4,5}, and one edge was redundant

What to notice. The group count and the cycle detection fall out of the same loop, for free. That is why this structure appears in so many problems at once.

5 The code, built step by step

The whole structure is two arrays.

```
class DisjointSet:
    def __init__(self, n: int) → None:
        self.parent = list(range(n))    # everyone is their own root
        self.size = [1] * n             # every group has one member
        self.groups = n                 # and there are n groups
```

`parent[i] = i` means `i` is a root. Starting with `list(range(n))` makes every element its own group, which is the correct starting state for every problem in this family.

Now `find`, with path compression. The iterative version:

```
def find(self, x: int) → int:
    root = x
    while self.parent[root] ≠ root:
        root = self.parent[root]
    while self.parent[x] ≠ root:      # second pass: point everything at the root
        self.parent[x], x = root, self.parent[x]
    return root
```

Two passes: walk up to find the root, then walk up again pointing everything at it. **That second loop is path compression**, and it is Meena’s “you’re with Sunita” on the way back.

The one-line variant, **path halving**, is what most people write and is very slightly weaker and slightly faster:

```
def find(self, x: int) → int:
    while self.parent[x] ≠ x:
        self.parent[x] = self.parent[self.parent[x]]    # point at grandparent
        x = self.parent[x]
    return x
```

Every step makes a node point at its grandparent, halving the path length as it goes. **One pass, no recursion, and the same asymptotic behaviour.** This is the version to write in an interview.

The recursive version is the prettiest and has a real hazard:

```
def find(self, x: int) → int:
    if self.parent[x] ≠ x:
        self.parent[x] = self.find(self.parent[x])
    return self.parent[x]
```

Three lines, and full path compression. **But it recurses to the depth of the tree**, which before compression can be `O(log n)` with union by size and `O(n)` without it. On a hundred thousand elements built without union by size, this is a `RecursionError`.

Now `union`, with union by size:

```

def union(self, a: int, b: int) → bool:
    ra, rb = self.find(a), self.find(b)
    if ra == rb:
        return False # already together
    if self.size[ra] < self.size[rb]:
        ra, rb = rb, ra # ra is now the LARGER root
    self.parent[rb] = ra
    self.size[ra] += self.size[rb]
    self.groups -= 1
    return True

```

The swap is union by size: after it, `ra` is the bigger tree, and the smaller one is attached underneath.

Returning a boolean is what makes this structure so useful. `False` means the two were already connected, which in different problems means “this edge closes a cycle”, “this edge is redundant”, or “skip this edge in Kruskal’s”. One return value, four problems.

`self.groups -= 1` maintains the component count with no extra work, so “how many groups now?” is a field read.

And the query everybody needs:

```

def connected(self, a: int, b: int) → bool:
    return self.find(a) == self.find(b)

def group_size(self, x: int) → int:
    return self.size[self.find(x)]

```

`size` is only meaningful **at the root**, which is why `group_size` calls `find` first. Reading `size[x]` directly for a non-root gives you a stale number from when `x` was last a root, and that is a real bug that produces plausible wrong answers.

The complete solution

```
"""Union-Find (disjoint set union) with path compression and union by size."""
from __future__ import annotations

class DisjointSet:
    """Near-constant-time 'same group?' and 'merge groups'."""

    find and union are  $O(\alpha(n))$  amortised, where  $\alpha$  is the inverse
    Ackermann function: at most 4 for any  $n$  that fits in this universe.
    """

    __slots__ = ("parent", "size", "groups")

    def __init__(self, n: int) → None:
        self.parent = list(range(n))
        self.size = [1] * n
        self.groups = n

    def find(self, x: int) → int:
        """The representative of x's group. Compresses the path as it goes."""
        while self.parent[x] ≠ x:
            self.parent[x] = self.parent[self.parent[x]]    # path halving
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) → bool:
        """Merge. Returns False if they were already in the same group."""
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra                                # attach small under large
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        self.groups -= 1
        return True

    def connected(self, a: int, b: int) → bool:
        return self.find(a) == self.find(b)

    def group_size(self, x: int) → int:
        return self.size[self.find(x)]                    # size is valid at the ROOT

def count_components(n: int, edges: list[tuple[int, int]]) → int:
    dsu = DisjointSet(n)
    for a, b in edges:
        dsu.union(a, b)
    return dsu.groups

def first_redundant_edge(n: int, edges: list[tuple[int, int]]) → tuple[int, int] | None:
    """The first edge whose endpoints are already connected: it closes a cycle."""
    dsu = DisjointSet(n)
    for a, b in edges:
        if not dsu.union(a, b):
            return (a, b)
    return None

def islands_after_each(rows: int, cols: int, positions: list[tuple[int, int]]) → list[int]:
    """Classic shape: count groups as elements are added one at a time."""
    dsu = DisjointSet(rows * cols)
    land = [False] * (rows * cols)
    count = 0
    out: list[int] = []
```

```

for r, c in positions:
    index = r * cols + c
    if land[index]:
        out.append(count)
        continue
    land[index] = True
    count += 1
    for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        nr, nc = r + dr, c + dc
        if 0 ≤ nr < rows and 0 ≤ nc < cols and land[nr * cols + nc]:
            if dsu.union(index, nr * cols + nc):
                count -= 1
    out.append(count)
return out

if __name__ == "__main__":
    # Meena's yard, as pairs.
    names = ["farida", "reshma", "sunita", "ajay", "vikram", "imran", "leela"]
    index = {name: i for i, name in enumerate(names)}
    pairs = [("farida", "reshma"), ("reshma", "sunita"), ("ajay", "sunita"),
             ("vikram", "imran"), ("farida", "ajay")]

    houses = DisjointSet(len(names))
    for a, b in pairs:
        merged = houses.union(index[a], index[b])
        print(f"{a:8} + {b:8} → {'merged' if merged else 'already together':16} "
              f"groups now {houses.groups}")

    print("farida and ajay together?", houses.connected(index["farida"], index["ajay"]))
    print("farida and vikram?", houses.connected(index["farida"], index["vikram"]))
    print("farida's group size: ", houses.group_size(index["farida"]))
    print()
    print("components:", count_components(6, [(0, 1), (1, 2), (0, 2), (3, 4), (4, 5)]))
    print("redundant :", first_redundant_edge(6, [(0, 1), (1, 2), (0, 2), (3, 4)]))
    print("islands  :", islands_after_each(3, 3, [(0, 0), (0, 1), (1, 2), (2, 1), (1, 1)]))

```

Running it:

```

farida   + reshma   → merged           groups now 6
reshma   + sunita   → merged           groups now 5
ajay     + sunita   → merged           groups now 4
vikram   + imran    → merged           groups now 3
farida   + ajay     → already together groups now 3
farida and ajay together? True
farida and vikram?      False
farida's group size:    4
components: 2
redundant : (0, 2)
islands  : [1, 1, 2, 3, 1]

```

Three things to look at. The fifth pair reports **already together** and does not change the count — that is the cycle detection, for free, out of the same call.

`components` is 2 for a graph with a redundant edge among the first three, and `first_redundant_edge` correctly names `(0, 2)` — the edge that closed the triangle, not either of the two before it.

And the last line is the shape worth studying: `[1, 1, 2, 3, 1]`. Adding land at `(0,1)` next to `(0,0)` merges immediately so the count stays 1. Then two separate pieces appear. Then the final cell at `(1,1)` touches **three** existing islands at once and merges them all, dropping the count from 3 to 1. **A single addition can merge several groups**, which is why the inner loop over neighbours matters.

6 What it costs

The headline result:

```
with BOTH path compression and union by size:
    find    O(alpha(n))  amortised
    union   O(alpha(n))  amortised
```

α is the inverse Ackermann function, and the practical statement is the one to say out loud:

```
alpha(n) ≤ 4  for every n up to about 2^65536
               → larger than the number of atoms in the observable universe
               → effectively constant, and I would say "effectively constant"
```

What each optimisation buys separately, because interviewers ask:

neither	$O(n)$	per operation, worst case
union by size only	$O(\log n)$	worst case
path compression only	$O(\log n)$	amortised
both	$O(\alpha(n))$	amortised

Both, and neither alone is enough. Union by size bounds the *height*; path compression flattens what height remains. The proof of the combined bound is genuinely hard and is not interview material — knowing the result and that both are needed is.

Space:

```
parent array    n integers
size array      n integers
-----
                O(n), two flat arrays
```

```
n = 1,000,000
two Python lists of ints    ~16 MB
```

Two flat arrays is the smallest memory footprint of any structure in this phase, and against a traversal — which needs a `seen` set and an adjacency structure — it is often ten times less in Python. That is a real argument when memory is the constraint.

The comparison that matters: against re-running a traversal.

m edges arriving one at a time, "how many groups now?" after each

traversal per query	$m \times O(V + E)$
Union-Find	$m \times O(\alpha)$

m = 100,000 edges, V = 100,000		
traversal:	100,000 x 300,000	= 30,000,000,000 steps
Union-Find:	100,000 x 4	= 400,000 steps
		→ 75,000x

Seventy-five thousand times, and that gap is the entire reason to know this structure.

But for a single static count, they are equivalent:

count components once, static graph	
traversal	$O(V + E)$
Union-Find	$O(E \times \alpha) \sim O(E)$

Both linear. The traversal is arguably fewer lines. **So the rule is: static and asked once → traversal; edges arriving over time → Union-Find**, and saying that shows you know why the structure exists rather than just how to write it.

Kruskal's, which is tomorrow:

sort E edges	$O(E \log E)$
E union operations	$O(E \times \alpha)$

	$O(E \log E)$ -- dominated by the SORT

The Union-Find part is free relative to the sorting, which is a nice thing to notice.

Path compression's cost profile, measured:

1,000,000 elements, 1,000,000 random unions, then 1,000,000 finds	
no compression, no union by size	trees up to ~1,000,000 deep → unusable
union by size only	average find ~15 steps
both	average find ~1.5 steps

7 The traps

Only one optimisation

```
def find(self, x):
    while self.parent[x] != x:
        x = self.parent[x]          # no compression
    return x

def union(self, a, b):
    ra, rb = self.find(a), self.find(b)
    if ra != rb:
        self.parent[rb] = ra        # no union by size — always b under a
```

Both are individually reasonable-looking. Together they produce a linked list on the classic adversarial input:

```
for i in range(1, 100000):
    dsu.union(i, i - 1)           # always attaches the big tree under the new node
```

```
>>> dsu.find(99999)
# ~100,000 pointer hops
```

Time Limit Exceeded

Write both. They are two lines each.

Recursive `find` on a deep tree

```
def find(self, x):
    if self.parent[x] != x:
        self.parent[x] = self.find(self.parent[x])
    return self.parent[x]
```

```
Traceback (most recent call last):
  File "dsu.py", line 12, in find
    self.parent[x] = self.find(self.parent[x])
[Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

The recursion depth is the tree depth *before* compression. With union by size that is $O(\log n)$ and safe; without it, $O(n)$ and fatal. **The iterative path-halving version is three lines and has no such hazard** — write that one.

Reading `size` without finding the root

```
return self.size[x]                # x may not be a root
```

```
>>> dsu.union(0, 1); dsu.union(1, 2); dsu.union(2, 3)
>>> dsu.size[1]
1                                # 1 stopped being a root after the first union
>>> dsu.group_size(1)
4                                # correct
```

`size` is only valid at the root, and the stale value is a plausible number rather than an obvious error. Same for any per-group data you attach — a maximum, a sum, a label — it lives at the root and must be read through `find`.

Comparing elements instead of roots

```
if a == b:                        # comparing the elements
    ...
```

Should be `find(a) == find(b)`. This one is obvious when stated and surprisingly easy to write at speed, especially inside a loop where `a` and `b` are already the loop variables.

Expecting to un-merge

```
dsu.union(a, b)
# later...
dsu.split(a, b)                # does not exist
```

There is no split, and there cannot be a cheap one — path compression has destroyed the information about how the tree was built. **If edges are removed over time, this is the wrong structure.**

The standard trick, when the whole sequence of operations is known in advance: **process it in reverse**, so that removals become additions. That converts a dynamic-disconnection problem into a Union-Find problem, and it is a genuinely clever move worth remembering.

Forgetting that one addition can merge several groups

```
count += 1
for neighbour in neighbours:
    if land[neighbour]:
        dsu.union(index, neighbour)
        count -= 1                # decrements once PER NEIGHBOUR
```

```
>>> islands_after_each(3, 3, [(0,0), (0,2), (2,0), (0,1)])  
# the final cell touches two islands; this version subtracts 2 for one merge...  
# and would subtract more if two neighbours were already in the SAME group
```

Only decrement when `union` actually merged, which is what the boolean return is for. Two neighbours that are already in the same group cause one merge, not two.

Using it on a directed graph

```
dsu.union(a, b) # for a directed edge  $a \rightarrow b$ 
```

Union-Find has no notion of direction — merging is symmetric. So it computes **weakly connected components**, which is sometimes what you want and is never *strongly* connected components. If the question is about following the arrows, this is the wrong tool.

8 In the interview

How it gets asked

- “Are these two nodes in the same group? Now merge two groups.” — the direct version.
- “Count the number of provinces / friend circles / connected components.”
- “Find the redundant connection.” — LeetCode 684, and Union-Find names the edge directly.
- “How many islands are there after each land addition?” — LeetCode 305, the dynamic shape.
- “Merge accounts that share an email.”
- “Build a minimum spanning tree.” — Kruskal’s, tomorrow.
- “Can these equations be satisfied?” — LeetCode 990, equality as grouping.

The first ninety seconds

“Union-Find, and the reason is that the grouping changes over time.

The structure is two arrays. `parent`, where each element points at its parent and a root points at itself, and `size`, holding the size of each group at its root. Two elements are in the same group exactly when `find` returns the same root.

`find(x)` walks up to the root. `union(a, b)` finds both roots and points one at the other.

Two optimisations, and you need both.

Path compression: while walking up, make each node point at its grandparent, so the path halves as I go. The next lookup on anything I passed is much shorter. It is unusual in that a read modifies the structure, which is why the bound is amortised.

Union by size: attach the smaller tree under the larger root. That keeps trees shallow, because depth can only grow when two similarly-sized trees merge, and that cannot happen often.

Either one alone gives $O(\log n)$. Together they give $O(\alpha(n))$, inverse Ackermann, which is **at most 4 for any input that could physically exist** — so I would say effectively constant.

The detail that makes it so reusable is that `union` returns a boolean. `False` means they were already connected, which depending on the problem means ‘this edge closes a cycle’, ‘this edge is redundant’, or ‘skip this edge in Kruskal’s’. And I keep a `groups` counter decremented on every real merge, so ‘how many components now?’ is a field read.

Space is two flat arrays, $O(n)$ — considerably less than a traversal’s `seen` set plus adjacency structure.

The rule I would state: static graph asked once, use a traversal — it is fewer lines and the same linear cost. Edges arriving over time, use this — re-running a traversal per edge is $m \times (V + E)$, which at a hundred thousand edges is thirty billion steps against four hundred thousand.”

The follow-ups

“What does each optimisation actually do? Why both?”

“They attack different things.

Union by size bounds the height. A tree’s depth only increases when you attach one tree under another of at least the same size, so a node’s depth can increase at most $\log n$ times before its group would contain more elements than exist. That gives $O(\log n)$ worst case per operation, with no amortisation needed.

Path compression flattens whatever height is there, but it only helps *after* a path has been walked once. On its own — with arbitrary union order — the structure can still be built into a deep chain, and the first traversal of that chain is expensive. It gives $O(\log n)$ amortised.

Together the bound collapses to inverse Ackermann. The intuition is that union by size stops deep trees ever forming, and compression flattens the shallow trees that do, so the total work across m operations is essentially linear in m .

The proof is genuinely hard and is not interview material — I would say I know the result rather than pretend to derive it. What I would want to be judged on is knowing that both are needed and what each one is for, and that writing only one gives you a structure that fails on the simplest adversarial input: `union(i, i-1)` in a loop builds a linked list without union by size.”

“Can you undo a union?”

“No, and not for a shallow reason — path compression has already destroyed the information about how the tree was built, so there is nothing to restore. There is no cheap `split`.

Two things I would offer instead.

If the whole sequence of operations is known in advance, process it backwards. A problem of the form ‘edges are removed one at a time, report connectivity after each removal’ becomes ‘edges are added one at a time’ if you start from the final state and replay in reverse. That turns an impossible problem into a standard one, and it is the trick worth remembering.

If operations must be undone interactively, use a rollback-capable variant: union by size *without* path compression, keeping a stack of the changes each union made. Then undo is popping the stack and restoring two array entries. The cost is that without compression it is $O(\log n)$ per operation rather than effectively constant — which is the trade, and it is what competitive programmers use for offline dynamic connectivity.

And if edges are genuinely being removed at arbitrary times with no lookahead, that is dynamic connectivity, which is a much harder problem with $O(\log^2 n)$ amortised solutions that I would name rather than write.”

“How many islands after each addition?”

“The dynamic shape, and it is the cleanest demonstration of why this structure exists.

Keep a Union-Find over all `rows × cols` cells and a boolean grid of which are land. For each new position: if it is already land, the count is unchanged — and I would handle that explicitly, because duplicate positions are a real test case. Otherwise mark it land and **increment the count**, provisionally treating it as a new island. Then for each of its four neighbours that is land, `union` them, and **decrement only when `union` returns True**.

That boolean is the crucial part. A new cell can touch two neighbours that are already in the same island — then there is one merge, not two, and decrementing per neighbour gives a count that is too low. It is the most common bug on this problem and the return value prevents it exactly.

A single addition can also merge **three or four** separate islands at once, dropping the count by three. That is why the inner loop runs over all four neighbours rather than stopping at the first.

Cost is $O(k \times \alpha)$ for k additions, which is effectively $O(k)$. The alternative — re-running a flood fill after each addition — is $O(k \times \text{rows} \times \text{cols})$, so on a $1,000 \times 1,000$ grid with 10,000 additions that is ten billion operations against about forty thousand.”

“What can’t it do?”

“Four things, and knowing them is how you avoid using it wrongly.

It cannot un-merge, as discussed.

It cannot give you the path between two elements, only whether one exists. The parent pointers are not a route through the graph — path compression has rearranged them into whatever shape was convenient. If the question is ‘how are they connected’, that is a traversal.

It has no notion of direction. Merging is symmetric, so on a directed graph it computes weakly connected components — ignoring the arrows. If the question is about following the arrows, that is strongly connected components and a different algorithm.

It cannot enumerate a group’s members cheaply without extra bookkeeping. There is no member list; the group exists only as upward pointers. If you need the members, keep a separate dictionary from root to list and merge those lists on union — which makes union $O(\text{size of smaller})$ rather than constant, though with union by size that still totals $O(n \log n)$ across all merges, which is the small-to-large trick.”

The model answer

“Accounts have names and email addresses. Two accounts belong to the same person if they share any email. Merge them, returning each person’s name and their sorted list of emails.”

“This is grouping by a shared property, and Union-Find is the natural fit — but the modelling step comes first and is where the problem is actually won.

The naive model makes accounts the vertices and joins two accounts that share an email. Finding those pairs means comparing every account against every other: n^2 comparisons, which at a hundred thousand accounts is five billion. Correct and unusable.

The move is to key on the email instead. I keep a dictionary from email to the index of the *first* account that mentioned it. Then for each account, for each of its emails: if the email is new, record it as belonging to this account; if it has been seen, `union` this account with the one that first claimed it. **One pass over the input**, $O(\text{total emails} \times \alpha)$, and I never compare two accounts directly.

That is the same trick as making the shared property a vertex in a graph, and it turns a quadratic step into a linear one. I would say that out loud, because it is the insight rather than the data structure.

Then collecting the answer. Group the emails by `find(account)` — a dictionary from root to a set of emails, since duplicates within a person are possible. Then for each root, the name comes from any account in that group, because all accounts in a group belong to the same person.

The trap in this problem is the name, and I would name it. Two different people can have the same name. So the name is never used for grouping — only emails are — and the name is data carried along. Grouping by name silently merges two strangers who happen to be called Ravi, and no test with distinct names catches it.

Cost: one pass to union, $O(E \times \alpha)$ where E is the total number of emails; one pass to collect, $O(E)$; and a sort per group, which totals $O(E \log E)$ and is actually the dominant term. Space is the Union-Find arrays plus the email dictionary, $O(n + E)$.

Concretely: a hundred thousand accounts averaging three emails is three hundred thousand emails, so about a million operations plus the sorting, against five billion for the naive comparison version.

The follow-up I would expect is ‘now accounts are added continuously and the merges must stay live’, and the answer is that this already works — Union-Find is an online structure, so a new account is just more unions on the existing state. A traversal-based solution would have to recompute everything, which is the reason I chose this rather than connected components in the first place.”

9 Recall card

Two arrays: `parent` (a root points at itself) and `size`. Same group $\Leftrightarrow$ same `find` root. `union` finds both roots and points one at the other.

Two optimisations, and you need both. **Path compression** — point each node at its grandparent while walking up (path halving, iterative, three lines). **Union by size** — attach the smaller tree under the larger root. Either alone is $O(\log n)$; together it is $O(\alpha(n))$, at most 4 for any real input.

`union` returns a boolean, and that is why it is so reusable: `False` means already connected, which is “cycle”, “redundant edge”, or “skip in Kruskal’s” depending on the problem. Keep a `groups` counter for free component counts.

Static graph asked once $\rightarrow$ traversal. Edges arriving over time $\rightarrow$ Union-Find ($m \times \alpha$ against $m \times (V + E)$ — $75,000 \times$ at $m = V = 10^5$).

It cannot un-merge, cannot give you a path, has no direction, and does not list a group’s members. If edges are *removed* and the sequence is known, process it in reverse. And `size` is only valid at the root — read it through `find`.

1 What this is, and why they ask it

Your production database serves the application: small reads and writes, one order at a time, thousands per second, every one of them needing to be fast because a user is waiting.

Then somebody from the business asks for revenue by region by month for the last three years, and that query reads forty million rows. It takes four minutes. While it runs, the checkout gets slower, and nobody immediately understands why.

Those are two genuinely different workloads. **OLTP** — online transaction processing — is many small operations on a few rows each. **OLAP** — online analytical processing — is few enormous operations reading most of a table but only a handful of its columns. A system tuned for one is bad at the other, and the difference is not tuning, it is the physical layout of the bytes on disk.

They ask this because “the analytics query is locking the production table” is a real situation with a standard resolution, and because the reasoning — **row storage versus column storage** — has arithmetic behind it that you can produce on demand. It is also the question where “just add a read replica” is the tempting answer that is only half right, and knowing why it is only half right is the point.

By the end of this lesson you can state the two workloads’ properties, explain columnar storage and compute the saving, say what a read replica does and does not solve, describe the star schema and the ETL path, and choose between a warehouse, a lakehouse and just adding an index.

2 The story

Nandini’s stationery shop opens at nine and the first hour is the worst.

Children on the way to school, wanting one pen, two pens, a geometry box because they lost theirs. Every transaction is small and every one of them is over in twenty seconds — find it on the shelf, say the price, take the money, next.

It is the whole shape of the business. Two hundred customers a day, almost none of them buying more than four things, and the only thing that matters is that each one is quick, because there is a queue and the children have a bell to catch.

Her husband does the accounts.

For eleven years he did them at the counter, on the same stool, in the gaps between customers. And the gaps are fine for adding up yesterday, but once a month there was the monthly reckoning, and once a year the big one, and those are not gaps-between-customers work. Those are two hours of going through everything — every sale, all year, but only ever two things about each sale: what it was and what it cost. Never who bought it, never what time, never anything else.

What happened every year, without fail, was that a customer would come in at half past eleven in the middle of it and he would have to stop, find the pen, take the money, and then spend a minute finding his place again. Customers waited. He lost his place. Some years it took him a whole day.

The change was Nandini's and it took her about four years to insist on it properly.

He does not do the accounts at the counter any more. On the last Sunday of the month he takes the whole thing to the back room, where there is a table and nobody comes in, and he works through it in one go without being interrupted. It takes ninety minutes instead of a day.

And the second half of the change, which mattered more than the first: **he no longer works from the shop's own record.** He copies out what he needs the night before — just the two columns he actually uses — and works from the copy. So even if a customer wanted something at eleven, the shop's record is on the counter where it belongs and he is not holding it.

The copy is a day out of date. Nobody has ever cared. The question the accounts answer is “how did last month go”, and last month does not change.

3 The idea in plain English

The counter is OLTP and the back room is OLAP, and Nandini's two changes are the whole design.

OLTP is the counter. Many small operations, each touching a few rows, each needing to be fast because somebody is waiting. Insert an order. Read one customer. Update a stock count. **Thousands per second, each one milliseconds.**

OLAP is the accounts. Very few operations, each reading an enormous number of rows but only a few **columns** of them. Total revenue by month. Average basket size by region. Conversion rate by campaign. **A handful per hour, each one seconds to minutes.**

The properties are opposite in every dimension:

	OLTP	OLAP
Rows per query	one to a few	millions
Columns per query	most of them	two or three
Query rate	thousands/second	a few per hour
Latency target	milliseconds	seconds to minutes
Writes	constant, small	bulk loads
Data age	right now	yesterday is fine
Optimised for	finding one row fast	scanning one column fast

And here is the thing that makes it a storage question rather than a tuning question.

A row store keeps a row's fields together on disk. That is what a normal database does, and it is exactly right for OLTP: fetching one order means reading one page and getting every field of it.

It is exactly wrong for OLAP. “Sum the amount column over forty million orders” has to read all forty million rows — every field of every one — to get at one number per row. If a row is 400 bytes and the amount is 8, **you read fifty times more data than you need.**

A column store keeps each column together instead. All the amounts contiguously, all the dates contiguously, all the region ids contiguously. Now summing the amounts reads only the amount column: 8 bytes per row instead of 400.

That is Nandini's husband copying out only the two columns he uses.

And storing a column together makes it compress astonishingly well, because the values are all the same type and often very similar. A region column with twelve distinct values over forty million rows compresses to almost nothing. **Typical warehouse compression is five to ten times on top of the column pruning,** so the combined saving is often a hundredfold.

A read replica is the tempting half-answer. It gives you a second copy so the analytics query does not compete for CPU and locks with the checkout. That is real and it is worth doing. **But the replica is still a row store,** so the query still reads fifty times more bytes than it needs, still takes four minutes, and now also generates replication lag that affects anything else reading from it. **It moves the problem off the primary without making the query fast.**

A warehouse is a separate system with a different physical layout, loaded on a schedule, and that is the complete answer.

The data is reshaped as well as copied, and that is the star schema. Production tables are normalised — split up to avoid duplication, because updating one field in one place is the OLTP virtue. A warehouse does the opposite: a big central **fact table** of events, each row a sale with numbers on it, surrounded by **dimension tables** describing the things involved — customer, product, date, store. **Denormalised on purpose**, because a join across five tables on forty million rows is expensive, and duplicating a product name is free when storage is a column that compresses.

And it lags, deliberately. Loaded nightly, or hourly, and everybody accepts that. “How did last month go” does not need a figure from four seconds ago, and pretending otherwise is how you end up back at the counter.

4 The picture

Row storage against column storage, on the same table:

```
TABLE orders: id | customer_id | product | region | amount | created_at | ... (20 more)

ROW STORE — a row's fields are contiguous

disk: [1|44|pen|MH|120|2026-01-02|...][2|91|book|KA|340|2026-01-02|...][3|...
      \----- one row, ~400 bytes -----/

"SELECT sum(amount)" → must read every row to get at 8 bytes each
                     40,000,000 x 400 B = 16 GB read for 320 MB of data

COLUMN STORE — a column's values are contiguous

disk: id:          [1][2][3][4]...
      customer: [44][91][12][7]...
      product:  [pen][book][pen][pen]...
      region:   [MH][KA][MH][TN]...
      amount:   [120][340][95][210]... ← read ONLY this
      created:  [...]
```

```
"SELECT sum(amount)" → read one column
                     40,000,000 x 8 B = 320 MB
                     and it compresses to ~60 MB
```

What to notice. The row store reads 16 GB to compute a number that needs 320 MB of input. That is not a constant factor you tune away — it is what “the fields of a row are next to each other” costs when you only want one field.

The three options, and what each fixes:

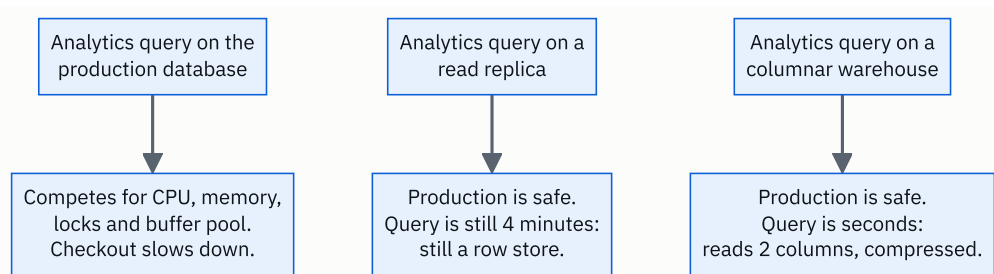
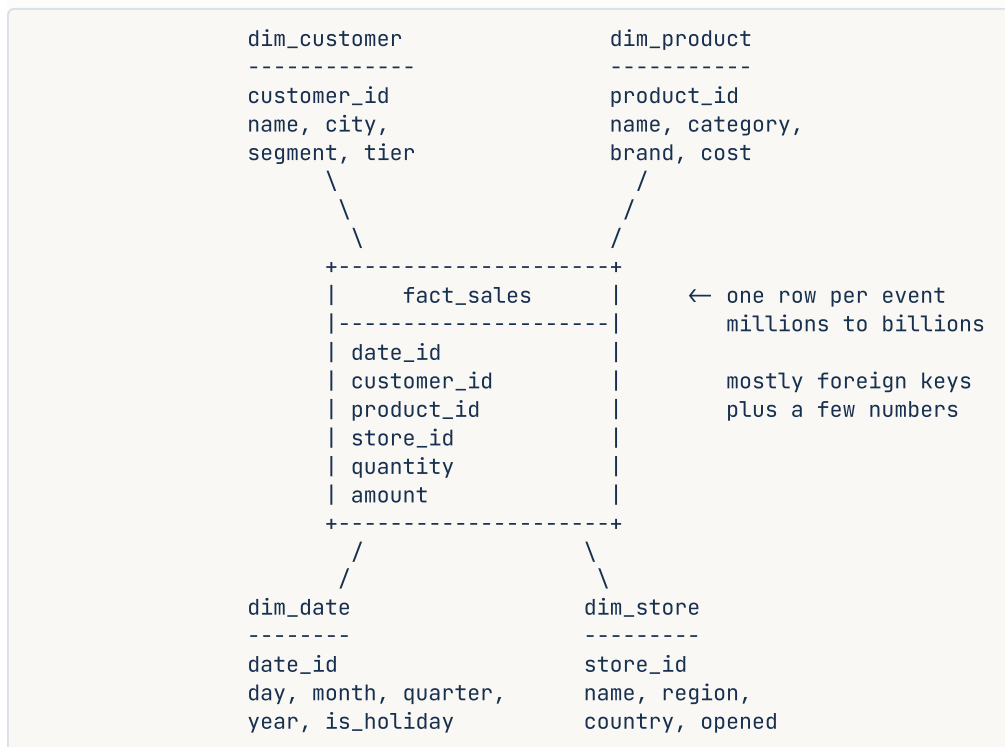


FIGURE 138.1

What to notice. The middle option is a genuine improvement and an incomplete one. It is the right first step and the wrong final answer.

The star schema:



What to notice. The fact table is enormous and thin — mostly integers. The dimensions are small and wide. A query joins the big table to a few small ones, which is a cheap shape, and the dimensions are duplicated rather than normalised because a `dim_product` with a thousand rows costs nothing and a five-way join over a billion rows costs plenty.

And the pipeline:

```
production DB  --(nightly / hourly / CDC)→  warehouse  →  dashboards
      OLTP                OLAP                BI tools
      row store          column store          SQL

the copy is HOURS old, and that is the design, not a defect
```

5 How it actually works

Why columnar is fast, in four mechanisms

1. Column pruning. Read only the columns the query names. A query touching 3 of 25 columns reads about an eighth of the data. This alone is usually the biggest factor.

2. Compression. Values in a column are the same type and often repetitive, so the standard encodings work extremely well:

```
run-length      region column: MH,MH,MH,MH,KA,KA → (MH,4),(KA,2)
dictionary      product names → small integers plus a lookup table
delta           sorted dates or ids → tiny differences
bit-packing     a column with 12 distinct values → 4 bits per value, not 32
```

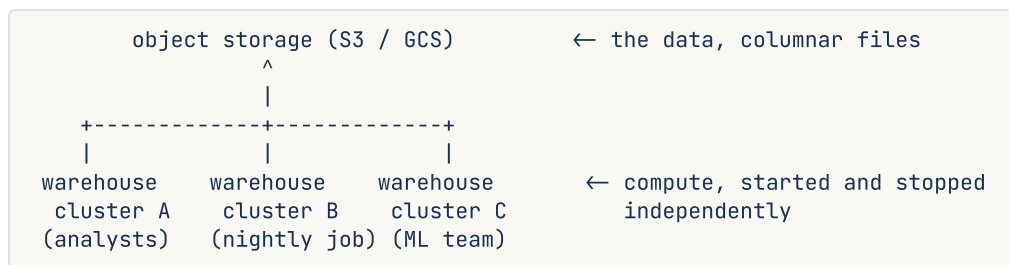
```
typical warehouse compression  5-10x on top of column pruning
```

3. Vectorised execution. Process a batch of values at a time rather than a row at a time, which keeps the CPU pipeline full and lets the compiler use SIMD instructions. A row-at-a-time engine spends most of its time on per-row overhead.

4. Zone maps and partition pruning. Each block of a column stores its minimum and maximum. A query filtering on `created_at ≥ '2026-01-01'` skips every block whose maximum is earlier — **without reading it**. Partitioning by date does the same at file granularity. On a well-partitioned table this often eliminates 95% of the data before any reading starts.

The separation of storage and compute

The change that defined the modern warehouse: **the data lives in object storage and the query engines are stateless machines that read it.**



Three consequences worth stating:

- **Scale compute independently of data.** A query needing more power gets a bigger cluster for ten minutes.
- **Isolate workloads.** The analysts' cluster cannot slow down the nightly job's cluster, because they share only immutable files.
- **Pay for what you use.** Compute stops when nobody is querying, which is most of the time.

This is what Snowflake commercialised and what BigQuery does implicitly. **It is also why “the warehouse is expensive” usually means “somebody left a cluster running”, not “storage costs a lot”** — storage is object storage prices.

ETL, ELT and the modern shape

ETL — extract, transform, load — was the older shape: pull from production, reshape it on a separate machine, load the clean result. Transformation happened before loading because warehouse compute was scarce and expensive.

ELT is the modern one: extract, load the raw data, and transform **inside the warehouse** with SQL. Because warehouse compute is now cheap and elastic, it is easier to load everything and reshape it in place — and it means the raw data is still there when somebody wants to reshape it differently.

```

sources → raw / bronze (as-is) → cleaned / silver (typed, deduped) → aggregated / gold (star schema, business metrics) → dashboards
  
```

dbt is the tool that made this standard: transformations are version-controlled SQL files with declared dependencies, which — and this is a nice connection — form a **DAG**, and running them in the right order is a [topological sort](#).

How data gets there

- **Batch dump.** Nightly `SELECT * FROM orders WHERE updated_at > yesterday.` Simple, and it puts a large read on production at the worst possible moment unless it runs against a replica.

- **CDC — change data capture.** Read the database’s replication log with something like Debezium and stream changes continuously. No load on production beyond what replication already costs, near-real-time, and it catches every write including manual ones. **This is the default choice now.**
- **Event streams.** Application events go to Kafka and are consumed into both the warehouse and elsewhere. Best for behavioural data — clicks, page views — that never existed in the production database at all.

The systems

	TYPE	NOTES
Snowflake	cloud warehouse	separated storage/compute, per-second billing, very easy to operate
BigQuery	cloud warehouse	serverless, priced per byte scanned — partition or pay
Redshift	cloud warehouse	older node-based model, RA3 separates storage
ClickHouse	columnar OLAP DB	extremely fast, self-hostable, more operational work
DuckDB	embedded OLAP	in-process, no server — genuinely useful for a few hundred GB
Databricks / Iceberg / Delta	lakehouse	columnar files (Parquet) in object storage plus a table format giving transactions and schema evolution

DuckDB is worth knowing as the small-scale answer. A few hundred gigabytes of Parquet queried in-process, no cluster, no bill. For a startup’s analytics that is often the whole solution, and offering it shows judgement rather than reflex.

The lakehouse deserves one sentence: store data as open columnar files in object storage, and add a table format — Iceberg, Delta Lake, Hudi — that provides atomic commits, schema evolution and time travel over those files. **You get warehouse behaviour without the data being locked inside one vendor’s system**, and any engine can read it.

6 The numbers

The core comparison, on one table. 40 million orders, 25 columns, ~400 bytes per row:

```
table size                40,000,000 x 400 B = 16 GB
query: SELECT region, sum(amount) ... GROUP BY region
columns needed            region (4 B) + amount (8 B) = 12 B
```

```
ROW STORE (production or a replica)
bytes read                16 GB
at 500 MB/s              32 seconds of pure I/O
plus row-at-a-time processing of 40M rows
realistically            2-5 minutes
```

```
COLUMN STORE
bytes read                40,000,000 x 12 B = 480 MB
compressed ~6x           ~80 MB
at 500 MB/s              0.16 seconds
plus vectorised processing
realistically            1-3 seconds
```

A hundredfold less I/O, and roughly a hundredfold faster. Those two numbers are the whole argument and they are worth being able to produce.

With date partitioning, on a query for one month:

```
partitioned by month, query touches 1 of 36 partitions
bytes read                80 MB / 36 = ~2 MB
→ milliseconds
```

Partition pruning is often a bigger factor than columnar storage itself, and it is the first thing to check when a warehouse query is slow.

What the analytics query costs on production, if you let it:

```
production DB: 16 GB buffer pool, serving 5,000 queries/s at ~2 ms
analytics query scans 16 GB
→ evicts the ENTIRE buffer pool
→ cache hit rate falls from ~99% to near zero
→ every OLTP query starts hitting disk
→ 2 ms becomes 20 ms, and the connection pool backs up
```

The damage is not the four minutes — it is that the four minutes destroy the cache for everything else. That is the same buffer-pool argument as [day 134's](#) blobs, and it is the reason “just run it at 3 a.m.” is only a partial answer.

Read replica, costed:

```
replica removes:          CPU contention, lock contention, buffer-pool damage on the primary
replica does NOT fix:    16 GB scanned, 2-5 minutes, and now the replica lags
                        during the scan, which affects anything else reading from it
```

Storage cost:

16 GB of orders in Postgres (gp3 SSD)	$16 \times \$0.115 = \$1.84/\text{month}$
same data in a warehouse, compressed 6x	$2.7 \text{ GB} \times \$0.023 = \$0.06/\text{month}$

Storage is never the reason a warehouse is expensive. Compute is.

Snowflake, one medium cluster	~\$4/hour
running 24/7	~\$2,900/month
running 2 hours/day	~\$240/month
BigQuery	~\$5 per TB scanned
a badly written unpartitioned query	
scanning 5 TB, run hourly	$5 \text{ TB} \times \$5 \times 24 \times 30 = \$18,000/\text{month}$

That last line is the one to say out loud. BigQuery's per-byte pricing makes partition pruning a financial control, not just a performance one, and `SELECT *` on a large table is a genuinely expensive mistake.

Freshness, and what each mechanism gives:

nightly batch	up to 24 hours stale
hourly batch	up to 1 hour
CDC streaming	seconds to minutes

Load volumes:

40,000,000 orders/year at ~400 B	16 GB/year raw
plus events (page views, clicks) at 100x	~1.6 TB/year
compressed in the warehouse	~250 GB/year

Behavioural events dwarf transactional data, usually by one to two orders of magnitude, and that is what actually sizes a warehouse.

7 The trade-offs

A warehouse is a second copy, so it is stale and it can drift. Nightly means up to a day behind. Everyone accepts that for “how did last month go” and nobody accepts it for “is this order paid”, so **the boundary between the two has to be explicit** — and the failure mode is a dashboard quietly becoming the number somebody makes an operational decision on.

Denormalising trades storage and correctness-on-update for query speed. A product name duplicated across forty million fact rows is cheap in a compressed column and means the name is wrong in forty million places if it changes. **Slowly changing**

dimensions are the standard treatment: keep versions of the dimension row with validity dates, so a historical sale still shows the price and category as they were. That is real complexity and it is the correct answer when history matters.

Columnar storage is bad at what row storage is good at. Fetching one complete row means reading from 25 separate places. Updating a single field means rewriting a block. **Warehouses do not do single-row updates well and are not meant to** — loads are bulk, and an `UPDATE ... WHERE id = ?` in a warehouse is a smell.

Separated compute is elastic and easy to leave running. The bill is per-second and the default is a cluster nobody turned off. Per-byte-scanned pricing is the same trap from the other side: an unpartitioned query in an hourly schedule can cost more than the entire rest of the platform.

More copies means more places for the truth to differ. Production, warehouse, and whatever the BI tool has cached. **Two teams computing “revenue” slightly differently is the most common data problem in any company**, and the technical answer — a single modelled definition in the transformation layer, with everything else reading from it — is a governance decision as much as an engineering one.

And the honest one: most companies do not need a warehouse. A read replica plus good indexes handles a very large amount of reporting. DuckDB over Parquet files handles a few hundred gigabytes with no infrastructure at all. **Reaching for Snowflake at ten gigabytes of data is a decision that costs money every month forever**, and saying that is a stronger answer than describing the architecture.

When would I not use one? Under about a hundred gigabytes, where a replica with the right indexes, or DuckDB, is enough. When the requirement is genuinely real-time — that is a streaming system, not a warehouse. When there is one analyst and one dashboard. And when the actual problem is one badly written query, which is a five-minute fix rather than a platform.

8 In the interview

How it gets asked

- “*The analytics query is locking the production table. What do you do?*” — the direct version.
- “*Why can’t you just run reports off the production database?*”
- “*What is the difference between OLTP and OLAP?*”
- “*Why is columnar storage faster for analytics?*” — the arithmetic question.
- “*Is a read replica enough?*” — the half-answer question.

- *“Design the analytics pipeline for this system.”*

The first ninety seconds

“These are two different workloads and the production database is built for exactly one of them.

The application does OLTP: thousands of small operations a second, each touching a few rows, each needing to be milliseconds because a user is waiting. The report does OLAP: one query reading forty million rows but only two or three of their twenty-five columns, and it can take seconds.

The reason they cannot share a system is the physical layout. A row store keeps a row’s fields together, which is exactly right for fetching one order. For summing one column over forty million rows it means reading every field of every row — sixteen gigabytes of I/O to compute something that needs about three hundred megabytes of input. **Fifty times more data than the query needs.**

And the damage is worse than the four minutes. Scanning sixteen gigabytes evicts the entire buffer pool, so the cache hit rate for ordinary queries collapses and every checkout starts hitting disk. **The report makes everything else slow, and nothing about the report looks wrong.**

Immediate fix: run it against a read replica. That removes the CPU contention, the lock contention and the buffer-pool damage from the primary, and it is the right first step. **But it is only half an answer** — the replica is still a row store, so the query still reads sixteen gigabytes and still takes minutes, and it now creates replication lag that affects anything else reading from that replica.

The complete answer is a columnar warehouse, loaded on a schedule. A column store keeps each column contiguous, so the same query reads two columns — about 480 megabytes, compressing to eighty — and the compression works well because a column holds one type with repetitive values. That is roughly a hundredfold less I/O, and with date partitioning on a single month it drops another thirtyfold.

The cost is staleness, and it is deliberate: nightly or hourly, because ‘how did last month go’ does not need a figure from four seconds ago.

How much data is there? Because under about a hundred gigabytes I would push back on building a warehouse at all — a replica with the right indexes, or DuckDB over Parquet files, does the job with no infrastructure.”

The follow-ups

“Why exactly is columnar faster? Give me the numbers.”

“Four mechanisms, and the first is the biggest.

Column pruning. A table with twenty-five columns at four hundred bytes a row; a query needing two of them needs twelve bytes a row. In a row store you read all four hundred because that is how the row is laid out. Forty million rows: sixteen gigabytes against four hundred and eighty megabytes. **Thirty-three times less, before anything clever.**

Compression. A column holds one type and usually repetitive values, so run-length, dictionary and bit-packing encodings all work extremely well — a region column with twelve distinct values needs four bits, not thirty-two. Five to ten times on top of the pruning, so the four hundred and eighty megabytes becomes about eighty.

Vectorised execution. Process batches of values rather than one row at a time, which keeps the CPU pipeline full and enables SIMD. A row-at-a-time engine spends most of its cycles on per-row overhead rather than on the arithmetic.

Zone maps and partition pruning. Each block stores the minimum and maximum of its column, so a filter on a date range skips blocks without reading them. **On a well-partitioned table this is often the single largest factor** — a query for one month out of three years touches one thirty-sixth of the data.

Multiply them: sixteen gigabytes read becomes single-digit megabytes, and four minutes becomes under a second. And in BigQuery, where you are billed per byte scanned, the same arithmetic is the difference between a five-dollar query and an eighteen-thousand-dollar monthly bill.”

“Is a read replica enough?”

“It is the right first step and it is not the whole answer, and I would be explicit about which part it fixes.

What it fixes: the analytics query no longer competes with the checkout for CPU, no longer takes locks that matter, and no longer evicts the primary’s buffer pool. That is genuinely most of the *harm*, and it is a configuration change rather than a project, so I would do it today.

What it does not fix: the query itself. The replica is byte-for-byte the same row store, so it still reads sixteen gigabytes and still takes minutes. The analyst’s experience is unchanged.

And it introduces something new: during that scan the replica’s own I/O is saturated, so replication lag grows, and anything else reading from that replica — which is usually the application’s own read traffic — now sees staler data. So I would want a replica dedicated to analytics, not the one serving the application.

The point at which a replica genuinely is enough: small data, a handful of reports, and indexes that cover them. If the reports are ‘yesterday’s orders for this store’, that is an indexed range scan on a replica and it is fine forever. It stops being enough when the queries are aggregations over most of the table, which is when you are doing OLAP on OLTP storage and no amount of indexing helps — an index does not reduce the bytes read when you are scanning everything anyway.”

“How does data get from production into the warehouse?”

“Three mechanisms, and I would choose based on freshness and on load.

Batch dump. Nightly `SELECT ... WHERE updated_at > yesterday`. Simple, and it puts a large read on production at whatever time you schedule it — so it should run against a replica, not the primary. Freshness is up to twenty-four hours. It also silently misses hard deletes unless the application soft-deletes, which is a real correctness gap.

Change data capture. Read the replication log with Debezium or equivalent and stream changes continuously. Near-real-time, no additional load on production beyond replication itself, and it catches every write — including manual ones and deletes, which is what fixes the batch version’s gap. **This is what I would default to now.**

Event streams. Application events published to Kafka and consumed into the warehouse. This is for behavioural data — page views, clicks, searches — which never existed in the production database at all, and which is usually one to two orders of magnitude more volume than the transactional data.

Then the shape inside: land the raw data, transform in place with SQL — ELT rather than ETL, because warehouse compute is now cheap and elastic and keeping the raw layer means you can re-derive things you did not anticipate. dbt is the standard tool, and the transformations form a dependency graph that is run in topological order, which is the same machinery from day 134.”

“When would you not build a warehouse?”

“More often than people expect, and I think saying so is the right answer rather than describing the architecture reflexively.

Under about a hundred gigabytes, a read replica with indexes designed for the reports handles it, and it is zero new infrastructure. If the reports are aggregations that need scans, **DuckDB over Parquet files exported nightly** is genuinely a complete solution — in-process, no server, no bill, and it will happily query a few hundred gigabytes on a laptop.

When the requirement is actually real-time, a warehouse is the wrong tool regardless of size. ‘Live orders in the last five minutes’ is a streaming problem or a materialised view, not a nightly load.

When the real problem is one query. Quite often ‘analytics is killing the database’ is a single missing index or a report written as a correlated subquery, and a five-minute fix beats a platform. I would look at that before proposing anything.

And the cost argument. A Snowflake medium cluster is about four dollars an hour, so nearly three thousand a month if nobody turns it off — and the default is that nobody turns it off. For a company with ten gigabytes of data that is a permanent monthly cost for something a replica does for free.

The threshold I would use is: more than one team asking analytical questions, more than a few hundred gigabytes, or reports that need to join behavioural events against transactional data. Any one of those, and a warehouse earns its place.”

The model answer

“An e-commerce platform: 50 million orders a year, plus clickstream data. The analytics team needs dashboards, the finance team needs monthly reports, and the ML team needs historical data for training. Design it.”

“Three consumers with three different requirements, so let me establish the shape and then be specific about where each one lands.

Production stays untouched. Postgres, row store, serving the application. No analytics query ever runs against it, and I would make that a rule rather than a convention, because the failure mode — one report evicting the buffer pool and slowing every checkout — is invisible in code review.

Ingestion, two paths, because the two data types are completely different. Transactional data — orders, customers, products — via CDC off the replication log. Near-real-time, no extra load on production, and it captures deletes, which a nightly `WHERE updated_at >` dump silently misses. Clickstream — page views, searches, add-to-cart — goes from the application into Kafka and is consumed into the warehouse directly. It never existed in Postgres and should not.

The volume ratio is the sizing fact. Fifty million orders a year at four hundred bytes is twenty gigabytes. Clickstream at, conservatively, a hundred events per order is five billion events a year — one to two terabytes raw, a few hundred gigabytes compressed. **The behavioural data is the warehouse, and the transactional data is a rounding error**, which is the opposite of most people’s intuition.

Storage: columnar files in object storage with a table format — Parquet plus Iceberg or Delta. That gives me atomic commits, schema evolution and time travel, and keeps the data readable by any engine rather than locked into one vendor. **Partitioned by date**, always, because a query for one month out of three years then touches a thirty-sixth of the data, and under per-byte pricing that is a financial control as much as a performance one.

Transformation in layers, with dbt: raw as ingested, then cleaned and typed and deduplicated, then a star schema — a `fact_orders` table and a `fact_events` table surrounded by `dim_customer`, `dim_product`, `dim_date`, `dim_store`. Denormalised deliberately, and **`dim_product` as a slowly changing dimension with validity dates**, so that a sale from 2024 still shows the price and category as they were at the time. Finance will ask for that in the first month and retrofitting it is painful.

Now the three consumers, each getting what it needs.

Analytics query the gold layer through a BI tool. Their cluster is separate, so a badly written exploratory query cannot slow anything else down — which is the concrete benefit of separated storage and compute.

Finance need the monthly close, and it needs to be *reproducible*: the same query run in March must give the same answer for January that it gave in February. So finance reads a snapshot table, written once when the month closes and never modi-

fied. **That is a governance requirement dressed as a technical one**, and it is the difference between a number people trust and one they argue about.

ML read the same Parquet files directly from object storage, bypassing the warehouse engine entirely, because training reads whole datasets and does not want SQL. That is the lakehouse's main practical benefit — one copy of the data, several engines.

Freshness: the transactional layer is minutes behind via CDC; the aggregated star schema rebuilds hourly. Nobody gets real-time from this, and I would say that explicitly, because the request will arrive. **If somebody needs 'orders in the last five minutes', that is a streaming aggregation, not a warehouse query**, and building it into the warehouse is how warehouses become slow and expensive.

And the one governance point I would insist on: revenue is defined once, in the transformation layer, and every dashboard reads that definition. **Two teams computing revenue slightly differently is the most common data problem in any company**, and it is not solved by better tooling — it is solved by there being one place the definition lives.”

9 Recall card

OLTP: many small operations, few rows, most columns, milliseconds. OLAP: few huge operations, millions of rows, two or three columns, seconds. Opposite in every dimension, and the difference is the **physical layout**, not tuning.

Row store reads every field to get one: $40\text{M rows} \times 400 \text{ B} = 16 \text{ GB}$ to sum a column that needs 320 MB. **Column store reads only that column, compressed** $5\text{--}10\times \rightarrow \sim 80 \text{ MB}$, plus vectorised execution and zone maps. $\sim 100\times$ less I/O, and partition pruning often beats all of it.

A read replica fixes the *harm (CPU, locks, buffer-pool eviction) and not the query* — still a row store, still minutes, and now it lags. It is the right first step and the wrong final answer.

Star schema: one enormous thin fact table plus small wide dimensions, denormalised on purpose. Slowly changing dimensions when history must be right. **ELT with CDC** is the modern pipeline; transformations form a DAG run in topological order.

Storage is never the cost — compute is (\$0.06/month for the data, $\sim \$2,900$ for a cluster left running; a 5 TB unpartitioned hourly query is \$18,000/month on per-byte pricing). **And under $\sim 100 \text{ GB}$, a replica with indexes or DuckDB over Parquet is the**

honest answer.

PRACTICE

Practice — Union-Find: the disjoint set union

DSA topic: Union-Find: the disjoint set union **System design topic:** Data warehouses: OLAP versus OLTP

Code these, in this order

One rule for the whole set: **write both optimisations before you write anything that uses the structure.** Path halving in `find`, the size swap in `union`. They are two lines each and either one alone gives you $O(\log n)$ — which on the adversarial input is the difference between passing and timing out.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Number of Provinces	LeetCode 547 (Medium)	The <code>groups</code> counter, decremented only on a real merge.
2	Redundant Connection	LeetCode 684 (Medium)	The boolean return naming the offending edge.
3	Accounts Merge	LeetCode 721 (Medium)	Keying on the shared property to avoid n^2 comparisons.
4	Number of Islands II	LeetCode 305 (Hard)	The dynamic shape — count as land is added.
5	Satisfiability of Equality Equations	LeetCode 990 (Medium)	Equality as grouping; process <code>=</code> before <code>≠</code> .
6	Most Stones Removed with Same Row or Column	LeetCode 947 (Medium)	Grouping by row and column indices, not by stone.

On problem 1, solve it both ways and time it

Once with a traversal and once with Union-Find. Same complexity, so record both times and both line counts. Then modify the problem: report the count after each edge is added, and time both again. The second ratio is the argument for the structure.

On problem 4, watch the boolean

Write the version that decrements once per land neighbour instead of once per successful `union`. Construct an input where a new cell touches two cells already in the same island, and record both answers. Then construct one where a new cell merges three separate islands at once and check your count drops by three.

On problem 5, notice the ordering

Process all `=` equations first, then check every `≠` against the finished groups. Try it in the other order and find an input that gives the wrong answer. One sentence on why order matters here and not in the other problems.

On problem 6, the modelling is the whole thing

Do not union stones. Union **row indices with column indices** — treating a row and a column as elements of the same structure. Write down the two sentences before coding, and say why the answer is `stones - number of groups`.

Then the adversarial input

Build a `DisjointSet` with **no union by size** and run `union(i, i-1)` for `i` in `range(1, 100000)`, then `find(99999)`. Time it. Add union by size and time it again. Two numbers.

Then remove path compression instead, keeping union by size, and time a million `find` calls. Three configurations, three numbers, and you will never write only one optimisation again.

Then the recursion experiment

Write `find` recursively, build a hundred-thousand-element chain without union by size, and call it. Record the error. Then add union by size and try again.

The structure drill

1. Say what the two arrays hold and what `parent[i] = i` means.
2. Write `find` with path halving from memory, iteratively.
3. Write `union` with union by size from memory.
4. Say what the boolean return means, in three different problem contexts.
5. Say why `size` must be read through `find`.

The optimisations drill

1. Say what path compression does, in one sentence.
2. Say what union by size does, in one sentence.
3. Give the cost with neither, with each alone, and with both.
4. Say what $\alpha(n)$ is and the practical statement about its value.
5. Give the adversarial input that breaks a structure with no union by size.

The recognition drill

1. Name the four problem shapes Union-Find solves.
2. State the rule for traversal versus Union-Find.
3. Give the arithmetic behind that rule at `m = V = 10^5`.
4. Say what it cannot do — four things.
5. Give the reverse-processing trick and when it applies.

The costs drill

1. Give time and space, and compare memory against a traversal's structures.
2. Compute traversal-per-edge against Union-Find for 100,000 edges.
3. Say what dominates Kruskal's cost and why the Union-Find part is free.
4. Say what a single addition can do to the group count, and why the loop matters.

The break-it drill

Trigger each and record the exact output or error:

1. No union by size, with `union(i, i-1)` in a loop.
2. Recursive `find` on a chain of 100,000 built without union by size.
3. Reading `size[x]` for a non-root.
4. Comparing `a == b` instead of `find(a) == find(b)`.
5. Decrementing the count per neighbour instead of per successful union.
6. Union-Find on a directed graph where you wanted strongly connected components.

The two-workloads drill

1. Give the seven-row comparison table from memory.
2. Say what makes them incompatible, and why it is not a tuning problem.
3. Give an example query of each kind on the same table.
4. Say what data freshness each one needs.

The columnar drill

1. Draw row layout and column layout for the same table.
2. Compute bytes read for `sum(amount)` over 40M rows in each.
3. Name the four mechanisms that make columnar fast.
4. Say which one is usually the largest factor, and why.
5. Give typical compression ratios and say why they are achievable.

The replica drill

1. Say what a read replica fixes — three things.
2. Say what it does not fix.
3. Say what new problem it introduces.
4. Give the case where a replica genuinely is enough, and where it stops being.
5. Explain the buffer-pool damage and why “run it at 3 a.m.” is only partial.

The schema drill

1. Draw a star schema and label both kinds of table.
2. Say why the warehouse denormalises when production normalises.
3. Say what a slowly changing dimension is and what it protects.
4. Say what the fact table looks like — wide or thin, big or small.

The pipeline drill

1. Name the three ways data gets into a warehouse.
2. Say what CDC catches that a nightly dump misses.
3. Say what ELT is and why it replaced ETL.
4. Name the layers and what each contains.
5. Say what dbt’s dependency graph is, in graph terms.

The numbers drill

1. Compute bytes read for the same query in a row store and a column store.
2. Recompute with monthly partitioning.
3. Compute storage cost for the same data in Postgres and in a warehouse.
4. Compute the monthly cost of a cluster left running.
5. Compute the cost of an unpartitioned 5 TB query run hourly under per-byte pricing.
6. Give the ratio of clickstream volume to transactional volume.

The when-not drill

1. Give the data-size threshold below which you would not build one.
2. Name the tool you would use instead at a few hundred gigabytes.
3. Say what to check before proposing any platform.
4. Give the three conditions, any one of which justifies a warehouse.

The failure drill

For each, say what happens and what you would build:

1. A monthly report runs against the production primary at 11 a.m.
2. The same report moved to the application's read replica.
3. A nightly `WHERE updated_at > dump`, and a row is hard-deleted in production.
4. A product's category changes, and last year's sales report is re-run.
5. `SELECT *` on an unpartitioned 5 TB table, scheduled hourly.
6. Two teams both computing "revenue" from the gold layer.
7. A Snowflake cluster nobody turned off.

Two of the seven are correctness problems and five are cost or performance. Sort them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Are these two nodes in the same group? Now merge two groups.* Two arrays, both optimisations with what each does, the `α(n)` result stated properly, the boolean return and the `groups` counter, and the traversal-versus-Union-Find rule with its arithmetic.
 2. *How many islands are there after each addition?* The dynamic shape, the provisional increment, decrementing only on a successful union, why one addition can merge three islands, and the cost against re-running a flood fill.
 3. *The analytics query is locking the production table. What do you do?* The two workloads, the row-versus-column arithmetic with the 16 GB figure, the buffer-pool damage, the replica as a half-answer, and the warehouse with its deliberate staleness — plus "how much data?" asked before recommending anything.
-

Before you move on

- ☐ I write both optimisations, always.
- ☐ I can write `find` with path halving iteratively from memory.
- ☐ I can write `union` with union by size from memory.
- ☐ I know what the boolean return means in three different problems.
- ☐ I keep a `groups` counter and decrement only on a real merge.
- ☐ I know `size` is valid only at the root.
- ☐ I can state what each optimisation does and the cost of each alone.
- ☐ I can state the `α(n)` result and its practical value.
- ☐ I know the adversarial input that breaks a missing union by size.

- [] I can name the four shapes Union-Find solves.
- [] I know the traversal-versus-Union-Find rule and its arithmetic.
- [] I know the four things it cannot do.
- [] I know the reverse-processing trick for removals.
- [] I know one addition can merge several groups.
- [] I can give the OLTP versus OLAP table from memory.
- [] I can draw row and column layouts and compute bytes read for both.
- [] I can name the four mechanisms behind columnar speed.
- [] I know partition pruning is often the biggest factor.
- [] I can explain the buffer-pool damage an analytics query does.
- [] I know what a read replica fixes and what it does not.
- [] I know what new problem a replica introduces.
- [] I can draw a star schema and say why it denormalises.
- [] I know what a slowly changing dimension protects.
- [] I can name the three ingestion mechanisms and what CDC catches.
- [] I know why ELT replaced ETL.
- [] I can compute warehouse storage cost against compute cost.
- [] I know why per-byte pricing makes partitioning a financial control.
- [] I know the threshold below which I would not build a warehouse.
- [] I can name what I would use instead at a few hundred gigabytes.
- [] I answered all three questions above out loud.

Minimum spanning trees: Kruskal and Prim

DATA STRUCTURES AND ALGORITHMS

Minimum spanning trees: Kruskal and Prim

SYSTEM DESIGN

ETL, batch pipelines, and where data goes to be counted

Minimum spanning trees: Kruskal and Prim

1 What this is, and why they ask it

A **spanning tree** connects every vertex of a graph using exactly $V - 1$ edges and no cycles. A **minimum** spanning tree is the cheapest one. The question it answers is: *connect all of these at the lowest total cost*.

There are two algorithms and they are both greedy, which is unusual — most greedy algorithms are wrong, and these two are provably right. **Kruskal's** sorts every edge and adds each one that does not create a cycle. **Prim's** grows one connected blob outward, always adding the cheapest edge leaving it.

They ask this because it is one of the cleanest greedy proofs in the syllabus, because both algorithms are short, and because the choice between them is a real decision driven by density. It is also where **Union-Find** earns its place — Kruskal's is essentially “sort the edges and run Union-Find over them”, and if you learned Union-Find yesterday you already have most of it.

The thing that separates candidates is the confusion that everybody has once: **a minimum spanning tree is not a shortest-path tree**. The cheapest way to connect everything is not the same as the cheapest way to get from A to everywhere, and being able to give a small graph where they differ is worth more than writing either algorithm.

By the end of this lesson you can write both, state the cut property that makes them correct, choose between them from the density, handle disconnected graphs, and say what changes when the graph is directed.

2 The story

Yusuf got the contract for the new colony in 2007, twenty-six houses on four lanes, and cable to every one of them.

The cable came on drums and he paid for it by the metre, so the whole job was one question: what is the least wire that reaches all twenty-six houses.

The signal comes in at the corner house on the main road. From there it can go to whichever house is nearest, and from any house it has reached, it can go on to any other. What it cannot do — and this is the part his assistant took a while to see — is form a

loop. If the cable runs from house 4 to house 9 and also from house 9 back round to house 4, that second run is doing nothing at all. Every house it touches is already connected. It is wire spent on nothing.

So: twenty-six houses, and twenty-five runs of cable. Never twenty-six.

He has done this two ways and he will tell you both.

The first way, which he did on that job, is to grow it. Start at the corner house. Look at every house not yet connected and ask which one is nearest to anything already wired. Run cable to that one. Now there are two connected houses; look again at everything unconnected and ask the same question. He worked outwards from the corner and the wired part grew like a stain, and at every step he ran the shortest piece that reached somebody new.

The second way he learned from a man in Bhiwandi and it feels wrong the first time. **You forget about connectedness entirely and you sort every possible run of cable by length, shortest first.** Then you go down the list. Nine metres between 12 and 13 — take it. Eleven metres between 4 and 5 — take it. Twelve metres between 5 and 6 — take it. And every so often you come to a run and both of its houses are already joined up, by some chain of other runs, and then you skip it, because it would be a loop.

What is strange about the second way is that for most of it you have several separate little islands of wired houses and nothing joining them, and it looks like a mess. Then near the end the islands snap together and it is finished. Same total length as the first method, every time. Yusuf measured it out both ways on two different sites before he believed it.

He uses the growing method now because on a real site you want the wired part to be one piece as you go, so you can test the signal. But the sorting method is faster to plan when he has all the distances in front of him and the houses are spread out.

3 The idea in plain English

Yusuf's two methods are Prim's and Kruskal's, and his observation about loops is the whole constraint.

A spanning tree connects every vertex with no cycles. For V vertices that is exactly $V - 1$ edges — fewer leaves something disconnected, more creates a cycle. Twenty-six houses, twenty-five runs of cable.

A minimum spanning tree is the one with the smallest total weight. Not the shortest path to anywhere — the cheapest total, for the whole structure.

A cycle means wasted wire. Every edge in a cycle can be removed without disconnecting anything, so a minimum spanning tree never contains one. That is Yusuf's assistant's realisation, and it is why both algorithms are fundamentally about avoiding cycles.

Kruskal's is the sorting method. Sort every edge by weight. Go down the list, and take each edge unless its two endpoints are already connected. Stop when you have $V - 1$ edges.

"Already connected" is exactly `find(a) == find(b)`, which is why this is Union-Find's home. Take the edge $\rightarrow$ `union`. `union` returning `False` $\rightarrow$ skip it, it would make a cycle. **Kruskal's is about eight lines once you have yesterday's structure.**

And the strange part is that it does not stay connected while it runs. For most of the process you have several separate fragments — Yusuf's islands — which merge near the end. That is fine, and it is what makes it work on a graph you have not thought about globally.

Prim's is the growing method. Start anywhere. Keep a set of vertices already in the tree. Repeatedly take the cheapest edge that leads from the tree to a vertex **not** in it, and add that vertex. Stop when everything is in.

The tree is always one connected piece, growing outward, which is Yusuf's stain. Finding "the cheapest edge leaving the tree" efficiently means a **priority queue** — which makes Prim's structurally almost identical to [Dijkstra's](#).

And that near-identity is the trap. The two differ in **one line**:

```
Dijkstra: key[v] = distance_from_source_to_u + weight(u, v)
Prim:     key[v] = weight(u, v)
```

Dijkstra accumulates the path cost; Prim's only looks at the single edge. That is the entire difference, and it is why the answers differ.

So a minimum spanning tree is not a shortest-path tree, and it is worth being able to show that on three vertices rather than asserting it. Section 4 does.

Now why greedy is correct here, which is the actual interview content. It rests on the **cut property**:

Take any way of splitting the vertices into two groups. The cheapest edge crossing between the groups is in some minimum spanning tree.

The argument is short. Suppose the cheapest crossing edge `e` is not in the tree. The tree still connects the two groups somehow, so it uses some other crossing edge `f`, which costs at least as much. Swap `f` for `e`: the result is still a spanning tree — still connec-

ted, still $V - 1$ edges — and its total is no larger. So a minimum spanning tree containing e exists.

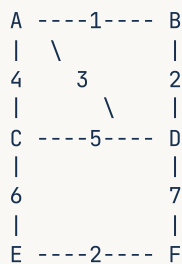
Both algorithms are just repeated applications of this. Prim’s cut is “in the tree” against “not in the tree”, and it takes the cheapest crossing edge. Kruskal’s cut is around whichever fragment the next edge would join. Say the cut property and the swap argument, and you have answered “why does greedy work here”, which is the question.

Ties are worth knowing about. If all edge weights are distinct, the minimum spanning tree is **unique**. With ties there may be several, all of the same total weight, and different algorithms — or the same algorithm with a different sort order — will return different ones.

And if the graph is not connected, there is no spanning tree at all. Both algorithms then produce a **minimum spanning forest** — one tree per component — and you detect it by counting: fewer than $V - 1$ edges taken means the graph was disconnected.

4 The picture

Yusuf’s colony, cut down to six houses:



edges sorted: AB 1, BD 2, EF 2, AD 3, AC 4, CD 5, CE 6, DF 7

Kruskal’s, step by step:

edge	weight	find(a)	find(b)	same?	action	fragments
A-B	1	A	B	no	TAKE	{AB} {C} {D} {E} {F}
B-D	2	A	D	no	TAKE	{ABD} {C} {E} {F}
E-F	2	E	F	no	TAKE	{ABD} {C} {EF}
A-D	3	A	A	YES	skip (cycle)	{ABD} {C} {EF}
A-C	4	A	C	no	TAKE	{ABCD} {EF}
C-D	5	A	A	YES	skip (cycle)	{ABCD} {EF}
C-E	6	A	E	no	TAKE	{ABCDE} {F} ← 5 edges, done

total = 1 + 2 + 2 + 4 + 6 = 15

What to notice. After three edges there were three separate fragments — {ABD}, {C}, {EF} — and nothing connecting them. Kruskal's does not care. It also skipped A-D at weight 3, which is cheaper than the A-C at 4 that it later took, because by then A and D were already connected. **Cheaper is not the same as useful.**

Prim's from A, on the same graph:

in tree	cheapest edge leaving it	take	total
-----	-----	-----	-----
{A}	A-B 1, A-C 4, A-D 3	A-B (1)	1
{A,B}	B-D 2, A-C 4, A-D 3	B-D (2)	3
{A,B,D}	A-C 4, C-D 5, D-F 7	A-C (4)	7
{A,B,C,D}	C-E 6, D-F 7	C-E (6)	13
{A,B,C,D,E}	E-F 2, D-F 7	E-F (2)	15
{A,B,C,D,E,F}	done		15

What to notice. Same total, 15, and the same set of edges — but discovered in a different order, and the tree was connected at every step. And at the fourth row Prim's took C-E at 6 while an edge of weight 2 (E-F) existed in the graph — because E-F did not touch the tree yet. **Prim's can only take edges leaving the current blob.**

And now the thing everyone gets wrong once:

MINIMUM SPANNING TREE
is
NOT
SHORTEST PATH TREE

```

      A
     / \
    1/   \1
   /       \
  B ----- C
    1

```

MST from any start:
any two of the three edges, total 2
say A-B and A-C

Shortest paths from B:
B→A = 1, B→C = 1 (direct)
In the MST above:
B→A = 1, B→C = 2 (via A)

The MST made B-to-C twice as expensive as it needs to be.

And more starkly:

```
A --1-- B --1-- C
|               |
+-----10-----+
```

MST: takes A-B and B-C, total 2
Shortest path A to C: 2, via B -- fine here

```
A --1-- B --1-- C
|               |
+-----3-----+
```

MST: A-B, B-C, total 2
Shortest A→C = 2 via B, or 3 direct. Still fine.

the divergence needs a star:

```
      hub
     /  |  \
    5   5   5
   /   |   \
  L1  L2  L3
   \___/___/
      6
```

MST: the three spokes, total 15
Shortest path leaf-to-leaf via hub: 10

but a 6-edge L1-L2 exists!
MST ignores it (cycle). Shortest path uses it.

What to notice. A minimum spanning tree minimises the **total** weight of the structure. A shortest-path tree minimises **each vertex's distance from one source**. Those are different objectives and they produce different trees.

5 The code, built step by step

Kruskal's first, because with yesterday's Union-Find it is almost nothing.

```
def kruskal(n: int, edges: list[tuple[int, int, int]]) → tuple[int, list]:
    """edges are (weight, a, b). Returns (total, chosen edges)."""
    edges.sort()                                # cheapest first
    dsu = DisjointSet(n)
    total, chosen = 0, []
    for weight, a, b in edges:
        if dsu.union(a, b):                     # False means already connected
            total += weight
            chosen.append((a, b, weight))
            if len(chosen) == n - 1:
                break                            # a spanning tree is complete
    return total, chosen
```

Eight lines. The sort puts the cheapest edges first; `dsu.union` returns `False` exactly when the edge would close a cycle; and the early `break` stops once `V - 1` edges are in, because nothing more can be added.

Storing edges as `(weight, a, b)` rather than `(a, b, weight)` means a plain `sort()` sorts by weight with no key function — a small thing that removes a place to make a mistake.

Detecting a disconnected graph is one line, and it is the check people forget:

```

if len(chosen) != n - 1:
    return -1, chosen
# not connected: a forest, not a tree

```

Prim's next, with a heap.

```

import heapq

def prim(n: int, graph: dict[int, list[tuple[int, int]]], start: int = 0):
    """graph[v] holds (neighbour, weight) pairs."""
    in_tree = [False] * n
    heap: list[tuple[int, int, int]] = [(0, start, -1)] # (weight, vertex, came_from)
    total, chosen = 0, []
    while heap and len(chosen) < n - 1:
        weight, vertex, came_from = heapq.heappop(heap)
        if in_tree[vertex]:
            continue # stale entry
        in_tree[vertex] = True
        if came_from != -1:
            total += weight
            chosen.append((came_from, vertex, weight))
        for neighbour, w in graph[vertex]:
            if not in_tree[neighbour]:
                heapq.heappush(heap, (w, neighbour, vertex))
    return total, chosen

```

Compare this with Dijkstra line by line and the only real difference is what goes into the heap:

# Dijkstra:	heapq.heappush(heap, (cost + w, neighbour))	accumulated
# Prim:	heapq.heappush(heap, (w, neighbour, vertex))	the edge alone

One term. Everything else — the stale-entry skip, the `in_tree` marking, the pop-the-cheapest — is identical. Saying that out loud shows you see the shared shape rather than having memorised two things.

The `if in_tree[vertex]: continue` is lazy deletion, exactly as in Dijkstra: the heap can hold several entries for the same vertex and the later ones are skipped.

The dense version of Prim's has no heap at all, and it is genuinely better on a complete graph:

```

def prim_dense(n: int, matrix: list[list[float]]) -> float:
    """O(V^2). Better than the heap version when E is close to V^2."""
    in_tree = [False] * n
    best = [float("inf")] * n # cheapest edge reaching each vertex
    best[0] = 0
    total = 0.0
    for _ in range(n):
        u = min((v for v in range(n) if not in_tree[v]), key=lambda v: best[v])
        in_tree[u] = True
        total += best[u]
        for v in range(n):
            # relax against the whole row
            if not in_tree[v] and matrix[u][v] < best[v]:
                best[v] = matrix[u][v]
    return total

```

$\text{best}[v]$ is “the cheapest single edge from the current tree to v ”, and each round takes the smallest of those. **No heap**, $O(V^2)$, and on a graph where $E \approx V^2$ that beats $O(E \log V) = O(V^2 \log V)$.

The complete solution

```
"""Minimum spanning trees: Kruskal's with Union-Find, and Prim's with a heap."""

from __future__ import annotations

import heapq

class DisjointSet:
    __slots__ = ("parent", "size", "groups")

    def __init__(self, n: int) → None:
        self.parent = list(range(n))
        self.size = [1] * n
        self.groups = n

    def find(self, x: int) → int:
        while self.parent[x] ≠ x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) → bool:
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        self.groups -= 1
        return True

def kruskal(n: int, edges: list[tuple[int, int, int]]) → tuple[int, list]:
    """Sort every edge, take each that does not close a cycle.  $O(E \log E)$ ."""
    dsu = DisjointSet(n)
    total, chosen = 0, []
    for weight, a, b in sorted(edges):
        if dsu.union(a, b):
            total += weight
            chosen.append((a, b, weight))
            if len(chosen) == n - 1:
                break
    if len(chosen) ≠ n - 1:
        return -1, chosen  # disconnected: a forest
    return total, chosen

def prim(n: int, graph: dict[int, list[tuple[int, int]]], start: int = 0) → tuple[int, list]:
    """Grow one connected tree, always taking the cheapest edge leaving it.

    Structurally identical to Dijkstra except for what is pushed:
        Dijkstra pushes (cost_so_far + w, ...) -- accumulated
        Prim pushes (w, ...) -- the single edge
    """
    in_tree = [False] * n
    heap: list[tuple[int, int, int]] = [(0, start, -1)]
    total, chosen = 0, []
    while heap and len(chosen) < n - 1:
        weight, vertex, came_from = heapq.heappop(heap)
        if in_tree[vertex]:
            continue
        in_tree[vertex] = True
        if came_from ≠ -1:
            total += weight
            chosen.append((came_from, vertex, weight))
        for neighbour, w in graph[vertex]:
            if not in_tree[neighbour]:
                heapq.heappush(heap, (w, neighbour, vertex))
```

```

if len(chosen) != n - 1:
    return -1, chosen
return total, chosen

if __name__ == "__main__":
    # Yusuf's six houses. 0=A 1=B 2=C 3=D 4=E 5=F
    edge_list = [
        (1, 0, 1), (3, 0, 3), (4, 0, 2),
        (2, 1, 3), (5, 2, 3), (6, 2, 4),
        (7, 3, 5), (2, 4, 5),
    ]
    adjacency: dict[int, list[tuple[int, int]]] = {v: [] for v in range(6)}
    for w, a, b in edge_list:
        adjacency[a].append((b, w))
        adjacency[b].append((a, w))

    k_total, k_edges = kruskal(6, edge_list)
    p_total, p_edges = prim(6, adjacency)
    print("kruskal:", k_total, sorted((min(a, b), max(a, b)) for a, b, _ in k_edges))
    print("prim    :", p_total, sorted((min(a, b), max(a, b)) for a, b, _ in p_edges))

    # Disconnected: two components, no spanning tree exists.
    print("forest :", kruskal(4, [(1, 0, 1), (1, 2, 3)])[0])

    # MST is not a shortest-path tree.
    triangle = [(1, 0, 1), (1, 0, 2), (1, 1, 2)]
    print("triangle MST total:", kruskal(3, triangle)[0], "(any two of three edges)")

```

Running it:

```

kruskal: 15 [(0, 1), (0, 2), (1, 3), (2, 4), (4, 5)]
prim    : 15 [(0, 1), (0, 2), (1, 3), (2, 4), (4, 5)]
forest  : -1
triangle MST total: 2 (any two of three edges)

```

Three things to look at. **Both algorithms found the same total and the same edge set** — because on this graph the weights are distinct, so the minimum spanning tree is unique. With ties they could legitimately differ.

`forest` returns `-1`: four vertices, two edges, two components, and no spanning tree exists. **The check is** `len(chosen) != n - 1` and it is the line most people leave out.

And the triangle: total 2, using two of the three unit edges. Every vertex is connected, and the pair not directly joined now costs 2 to travel between instead of 1 — which is the MST-is-not-shortest-path point, made concrete.

6 What it costs

Kruskal's.

sort the edges	$O(E \log E)$
E union operations at $O(\alpha)$	$O(E \times \alpha) \sim O(E)$

	$O(E \log E)$ -- the SORT dominates
space: the DSU arrays	$O(V)$

The Union-Find part is effectively free relative to the sorting. And since $E \leq V^2$, $\log E \leq 2 \log V$, so this is often written $O(E \log V)$ — the same thing.

If the edges are already sorted, or can be sorted in linear time — small integer weights, counting sort — Kruskal's becomes $O(E \times \alpha)$, effectively linear. That is a real optimisation and worth mentioning.

Prim's, with a binary heap.

each vertex popped once	V
each edge pushed at most once	E
each push and pop	$\log(\text{heap size}) = \log E$

	$O(E \log V)$
space: heap up to E entries	$O(E)$

Prim's, dense version with no heap.

V rounds, each scanning V vertices	$V \times V$
plus relaxing a row of V	$V \times V$

	$O(V^2)$
space: two arrays of V	$O(V)$

Now the comparison that decides which to use.

SPARSE: $V = 100,000$, $E = 300,000$

Kruskal	$300,000 \times \log(300,000) = 300,000 \times 18$	= 5,400,000
Prim (heap)	$300,000 \times 17$	= 5,100,000
Prim (dense)	$100,000^2$	= 10,000,000,000

DENSE: $V = 5,000$, $E = 12,500,000$ (complete graph)

Kruskal	$12,500,000 \times 24$	= 300,000,000
Prim (heap)	$12,500,000 \times 12$	= 150,000,000
Prim (dense)	$5,000^2$	= 25,000,000

On a sparse graph, Kruskal's and heap-Prim's are equivalent and the dense version is hopeless. On a dense graph, the dense version wins by an order of magnitude.

The rule to say:

sparse graph, edges given as a list	→	Kruskal (and the sort is the cost)
dense graph, or an adjacency matrix	→	Prim, $O(V^2)$, no heap
edges arriving over time	→	Kruskal, because Union-Find is online
need the tree connected as it grows	→	Prim

Memory, at scale:

V = 1,000,000, E = 5,000,000		
Kruskal	edge list 5,000,000 tuples	~360 MB
	DSU arrays	~16 MB
	plus the sort's working space	
Prim	adjacency 5,000,000 entries	~400 MB
	heap up to E	~360 MB

Kruskal's needs all the edges in memory to sort them, which is its main practical constraint. Prim's needs the adjacency structure plus the heap. Neither is obviously better; Kruskal's sort can be done externally if it truly does not fit.

Against Dijkstra, since the code is nearly identical:

same graph, V = 100,000, E = 300,000	
Dijkstra	5,100,000
Prim	5,100,000

Identical cost, different answer. That is worth saying, because it makes clear that the difference is semantic and not a performance trade.

7 The traps

Confusing a minimum spanning tree with shortest paths

The conceptual error, and it produces confidently wrong answers:

```
tree = prim(n, graph, start=source)
# "now I have the shortest path from source to everything"
```

```
triangle: A-B = 1, A-C = 1, B-C = 1
MST from A: {A-B, A-C}, total 2
distance B to C in the MST: 2
true shortest path B to C: 1
```

A minimum spanning tree minimises the total; a shortest-path tree minimises each vertex's distance from one source. Different objectives. The tell is the one-line difference in what goes into the heap, and if you copied Dijkstra to write Prim's, check that line.

Prim's written with Dijkstra's key

```
heapq.heappush(heap, (weight + w, neighbour, vertex))    # accumulated: WRONG for Prim
```

This runs, produces a spanning tree, and produces the wrong one:

```
A --1-- B --1-- C, A --3-- C
correct MST:  A-B, B-C, total 2
with the bug: A-B (1), then from B the key for C is 1+1=2, and A-C is 3
               → still picks B-C. Same answer here.

A --1-- B --5-- C, A --4-- C
correct MST:  A-B (1), A-C (4), total 5
with the bug: A-B (1), then key(C) via B = 1+5 = 6, via A = 4 → picks A-C. Same.
```

It agrees surprisingly often, which is what makes it dangerous — **on a graph where an expensive early edge leads to cheap later ones, the accumulated version starts preferring different edges**. Test with a long chain of small weights against a single medium edge.

Forgetting the connectivity check

```
return total
```

On a disconnected graph both algorithms terminate happily, having built a spanning forest:

```
>>> kruskal_no_check(4, [(1, 0, 1), (1, 2, 3)])
2                                     # a total, for a graph with no spanning tree
```

The number looks fine. `len(chosen)  $\neq$  n - 1` is the check, and the convention in most problems is to return `-1`.

The stale-entry skip missing from Prim's

```
while heap:
    weight, vertex, came_from = heapq.heappop(heap)
    in_tree[vertex] = True          # no check
    total += weight
```

A vertex can be pushed many times, once per edge reaching it, and without the skip you add several of those edges — producing a structure with cycles and a total that is too large. **The check is one line and it is the same lazy deletion as Dijkstra's.**

Sorting with a key function unnecessarily

```
edges.sort(key=lambda e: e[2])    # (a, b, weight)
```

Correct, and slower, and it invites the mistake of forgetting the key. **Store edges as (weight, a, b) and a plain sort() does the right thing** — one less thing to get wrong, and measurably faster on large inputs because it avoids a Python-level call per comparison.

Using it on a directed graph

```
kruskal(n, directed_edges)
```

Union-Find has no notion of direction, so this computes a minimum spanning tree of the **undirected** version of the graph and reports it as though the directions were respected. The directed analogue is a **minimum spanning arborescence** — every vertex reachable from a chosen root, following the arrows — and it needs a completely different algorithm (Chu–Liu/Edmonds’). **Name it and do not attempt it**; the point is knowing that Kruskal’s and Prim’s do not apply.

Assuming the tree is unique

```
assert kruskal(n, edges)[1] == expected_edges
```

With tied weights there can be many minimum spanning trees, all of the same total. **Assert the total, not the edge set** — the total is unique, the tree is not.

8 In the interview

How it gets asked

- “Connect all the cities at minimum cost.” — the direct version.
- “Min Cost to Connect All Points.” — LeetCode 1584, and the graph is implicit.
- “Given a network of cables with costs, which do you lay?”
- “Optimize water distribution in a village.” — LeetCode 1168, with the virtual-vertex trick.
- “Which edges are critical?” — LeetCode 1489, and the answer runs Kruskal’s many times.
- “Why does the greedy choice work?” — the cut property question.

The first ninety seconds

“Minimum spanning tree — connect every vertex with $V - 1$ edges at the lowest total cost, and no cycles, because every edge in a cycle can be removed without disconnecting anything.

Two algorithms, both greedy and both provably correct.

Kruskal's sorts every edge by weight and takes each one whose endpoints are not already connected. ‘Already connected’ is a Union-Find query, so this is about eight lines: sort, then loop, and `union` returning `False` means this edge would close a cycle and I skip it. $O(E \log E)$, dominated entirely by the sort — the Union-Find part is effectively free.

Prim's grows one connected tree, repeatedly taking the cheapest edge that leaves it. That needs a priority queue, and the code is almost exactly Dijkstra's — **the only difference is what goes into the heap**. Dijkstra pushes the accumulated cost from the source; Prim's pushes the single edge weight. One term, and it is the difference between two different answers.

Which one depends on density. On a sparse graph both are about $E \log V$ and I would use Kruskal's, because it is shorter and because the edges usually arrive as a list anyway. On a dense graph — or when I am handed an adjacency matrix — the $O(V^2)$ version of Prim's with no heap wins: at five thousand vertices complete, that is twenty-five million operations against a hundred and fifty million for the heap version.

Two things I would state before coding. If fewer than $V - 1$ edges get taken, the graph was disconnected and there is no spanning tree — I would return `-1` rather than a total for a forest. And with tied weights there can be several minimum spanning trees of the same total, so I would assert the total in tests, not the edge set.

And the thing I would say before you ask: this is not a shortest-path tree. On a triangle with all edges of weight 1, the MST takes two of them, and the two vertices not directly joined are now distance 2 apart instead of 1. Different objective.”

The follow-ups

“Why does the greedy choice work? Prove it.”

“The cut property, and the argument is a swap.

The cut property: split the vertices into any two groups. The cheapest edge crossing between them is in some minimum spanning tree.

The proof: suppose the cheapest crossing edge e is not in a given minimum spanning tree T . T still connects the two groups, so it contains some other crossing edge f , and f costs at least as much as e because e was the cheapest crossing edge. Now remove f from T and add e . The result is still connected — e reconnects the two halves that removing f separated — and it still has $V - 1$ edges, so it is still a spanning tree, and its total weight is no larger. So a minimum spanning tree containing e exists.

Both algorithms are repeated applications of that. Prim’s cut is ‘in the tree’ against ‘not in the tree’, and it takes the cheapest edge crossing it — which the cut property says is safe. Kruskal’s cut is around whichever fragment the next edge would attach to; every edge it takes is the cheapest one leaving that fragment, because anything cheaper has already been considered and either taken or rejected as a cycle.

The related property, for the other direction, is the cycle property: the heaviest edge in any cycle is not in any minimum spanning tree. That is what justifies skipping — an edge whose endpoints are already connected closes a cycle, and being last in the sorted order makes it the heaviest in that cycle.”

“Kruskal’s or Prim’s? What decides it?”

“Density, and what form the input arrives in.

Sparse — E close to V — both are about $E \log V$ and I would take Kruskal’s, because it is shorter, because the input is usually already an edge list, and because it reuses Union-Find which I probably have anyway.

Dense — E close to V^2 , or an adjacency matrix — the $O(V^2)$ Prim’s without a heap. On a complete graph of five thousand vertices that is twenty-five million operations, against a hundred and fifty million for heap-Prim’s and three hundred million for Kruskal’s. **The heap becomes the overhead when almost every pair is an edge.**

Two more considerations. If edges arrive over time, Kruskal’s fits naturally because Union-Find is an online structure — although a true incremental MST needs more care than just re-running it. And if I need the tree to be connected at every stage — laying cable, and wanting to test the signal as I go — Prim’s gives me that and Kruskal’s does not, because Kruskal’s has disconnected fragments for most of its run.

One more, on memory: Kruskal’s needs every edge in memory to sort them. On a graph with fifty million edges that is the binding constraint, and Prim’s streaming over an adjacency structure may be the only option — or an external sort.”

“The points are on a plane and any two can be connected, with cost equal to the distance.”

“Then the graph is **implicit and complete**, and the first thing I would say is the edge count: n points means $n(n-1)/2$ edges. At a thousand points that is half a million — fine. At a hundred thousand it is five billion, and building the edge list is impossible.

For n up to a few thousand: the dense Prim’s, $O(n^2)$, and never build the edge list at all. Keep an array of ‘cheapest distance from the tree to each point’, pick the minimum each round, add it, and update every remaining point against the newly added one. Two arrays, no heap, n^2 distance computations, and the memory is $O(n)$ rather than $O(n^2)$. **That is the answer to LeetCode 1584 and it is worth recognising immediately** — Kruskal’s here would build and sort half a million edges to solve something that needs a million distance calculations and no storage.

Beyond a few thousand points, the geometry helps: the minimum spanning tree of points on a plane is a subgraph of the **Delaunay triangulation**, which has only $O(n)$ edges. So compute that first — $O(n \log n)$ — and run Kruskal’s over its edges. That takes a hundred thousand points from impossible to routine. I would name it rather than implement it, but knowing that the complete graph is not necessary is the insight.”

“Which edges must be in every minimum spanning tree, and which are in none?”

“Three categories, and the standard approach runs the algorithm repeatedly.

Critical — removing it increases the MST’s total, so it is in every minimum spanning tree. Test: compute the MST weight normally; then compute it again with that edge **excluded**. If the total goes up, or the graph becomes disconnected, the edge is critical.

Pseudo-critical — it is in *some* minimum spanning tree but not all. Test: compute the MST weight with that edge **forced in** — take it first, then run Kruskal’s on the rest. If the total equals the normal minimum, it belongs to some MST.

Neither — in no minimum spanning tree at all.

That is $O(E)$ runs of Kruskal’s, so $O(E^2 \log E)$, which is fine for the few hundred edges these problems use and would not scale. **The linear-time answer uses the cycle property**: an edge is in no MST exactly when it is the unique heaviest edge on some cycle, and Tarjan’s approach finds all of them together — I would name that as what I would reach for if E were large.

The one implementation detail worth stating: when forcing an edge in, you must add it before sorting the rest, and when excluding one, you must skip it entirely rather than just deprioritise it. Both are easy to get subtly wrong and both produce plausible totals.”

The model answer

“A village has n houses. You can dig a well in any house for a cost, or lay a pipe between two houses for a cost. Every house must have water. Minimise the total cost.”

“This looks like it has two different kinds of decision — dig or connect — and the whole problem is noticing that it does not. Let me build it up.

The naive reading is that this is not a spanning tree problem at all, because a spanning tree connects things and digging a well connects nothing. And if I try to model it as ‘find the MST and then decide where to dig’, the two decisions interact and there is no clean way to separate them.

The move is a virtual vertex. Add one extra node — call it node 0, the water source — and turn every well into an **edge** from node 0 to that house, with weight equal to the well’s cost. Now there is only one kind of decision: which edges to take. Digging a well in house i is ‘connect house i to the source’, and laying a pipe is ‘connect house i to house j ’.

And now the problem is exactly a minimum spanning tree over $n + 1$ vertices. Every house must end up connected to the source, directly or through other houses; the cheapest way to connect everything is the MST; and it will contain n edges, some of which are wells and some pipes — which is precisely the answer, and I never had to decide the two things separately.

That reframing is the entire problem, and I would spend the first ninety seconds on it rather than on the algorithm, because the algorithm is then eight lines.

Implementation: Kruskal’s. Build the edge list — n well edges from node 0, plus the given pipe edges — sort by cost, and take each edge whose endpoints are not already connected. Union-Find over $n + 1$ elements. Stop at n edges.

Cost: with n houses and m possible pipes, the edge list is $n + m$, so $O((n + m) \log(n + m))$, dominated by the sort. For a village that is trivially fast. Space is $O(n + m)$ for the edges plus $O(n)$ for the Union-Find.

Two things I would check. The graph is now guaranteed connected — every house has a well edge to node 0, so a spanning tree always exists and I will never return -1 . That is a nice property of the reframing and worth noticing. And node 0 must be included in the vertex count, so the Union-Find is sized $n + 1$ and the target is n edges, not $n - 1$ — the classic off-by-one here.

The general pattern I would name, because it recurs: **when a problem has a ‘do it standalone’ option alongside a ‘connect to something’ option, model standalone as an edge to a virtual source.** The same trick turns ‘build a warehouse here or ship from an existing one’ and ‘run a generator or connect to the grid’ into plain spanning-tree problems.”

9 Recall card

A spanning tree is $V - 1$ edges, connected, no cycles. Minimum = cheapest total. A cycle is wasted weight, which is why both algorithms are about avoiding one.

Kruskal's: sort all edges, take each one where union returns True. $O(E \log E)$ — the sort dominates; Union-Find is free. Fragments stay disconnected for most of the run, and that is fine.

Prim's: grow one blob, always taking the cheapest edge leaving it. Structurally identical to Dijkstra except for one line — Dijkstra pushes $\text{cost_so_far} + w$, Prim pushes w . Sparse $\rightarrow$ either; dense or a matrix $\rightarrow$ the $O(V^2)$ Prim with no heap.

Why greedy is correct: the cut property. Split the vertices any way; the cheapest crossing edge is in some MST — proved by swapping it for whatever crossing edge the tree used. Both algorithms are repeated applications of it.

Three things to state: an MST is **not** a shortest-path tree (triangle of 1s: MST makes two vertices distance 2 apart); fewer than $V - 1$ edges means **disconnected**, return -1 ; and with tied weights the tree is not unique, so assert the **total**.

ETL, batch pipelines, and where data goes to be counted

1 What this is, and why they ask it

Somebody looks at a dashboard and reads a number. “Yesterday’s revenue: ₹48,21,600.” That number did not exist until about four in the morning, when a chain of about thirty jobs ran in a specific order, each one waiting for the ones before it, and produced it.

That chain is a **batch pipeline**, and this lesson is about what it is made of: how jobs are ordered, what happens when one fails at three in the morning, how you re-run yesterday without producing yesterday’s revenue twice, and how you notice when the number is wrong rather than merely late.

They ask this because pipelines are where data engineering actually lives, and because the interesting parts are all failure handling. Writing a job that computes revenue is a `GROUP BY`. Writing one that can be safely re-run, that handles a source arriving four hours late, that backfills six months of history without melting the warehouse, and that fails loudly when the input is wrong — that is the job.

It is also the day where the graph track and the system design track meet: **a pipeline is a DAG, and running it is a topological sort**, which is [day 134](#). That is not a coincidence or a cute analogy — Airflow’s core object is literally called a DAG.

By the end of this lesson you can trace a number from a production write to a dashboard, design a job that is safe to re-run, explain idempotency and backfills concretely, place data-quality checks, and say when a pipeline should not be batch at all.

2 The story

The milk comes in between half past five and seven, and by eight o’clock Bhaskar needs one number.

He runs the collection centre for about two hundred and forty farmers. Each of them brings a can. It gets weighed, a sample is drawn, the sample is tested for fat, and the two numbers together — litres and fat percentage — decide what that farmer is paid for the day. At the end of the morning all of it goes into one total that goes to the union, and that total is what the centre itself gets paid on.

The order is fixed and it cannot be shuffled. You cannot test the sample before you have drawn it. You cannot draw the sample before the can is weighed, because the weighing is when the can is opened. And you certainly cannot work out anybody’s pay-

ment before the fat test is back from the machine.

Most mornings it runs like a clock and Bhaskar is done by twenty past eight.

The mornings he remembers are the other ones.

The machine. In 2021 the fat testing machine failed at ten past six, with about sixty cans already weighed and sampled. The weighing was done and correct. Everything after it was stuck. When the replacement machine came at half past nine, he did not re-weigh the sixty cans — that work was finished and still good. He picked up from the testing step and carried on.

The late farmer. Nagappa's tempo breaks down perhaps once a month, and he arrives at half past nine with eleven cans from the far village. By then the total has already gone to the union. So the eleven cans are recorded against that day — they *are* that day's milk — and a corrected total goes up at eleven. The union accepts corrections until the end of the day and after that they do not.

The day it ran twice. Two years ago a boy who was new fed the morning's sheet into the total a second time, and the centre's figure came out at double. Nobody noticed for four days, because the number was plausible and nobody had a reason to check it against anything. What Bhaskar does now is that the total is not built by adding things up as they arrive. It is built by taking the day's records and computing the total from scratch. Run it twice and you get the same number, because it is not adding, it is recalculating.

And the one that taught him the most. In 2019 a scale was out of calibration for nine days and every can was weighed about 4% light. Nothing failed. Every job ran. Every number was produced on time and every one of them was wrong, and it took a farmer complaining about his monthly payment to find it. Now the first thing Bhaskar does every morning, before anything else, is put a known five-litre measure on the scale, and if it does not read five, nothing else happens that day until it does.

3 The idea in plain English

Bhaskar's morning is a batch pipeline, and each of his four bad days is one of the four things this lesson is about.

A batch pipeline is a set of jobs with dependencies, run on a schedule. Extract from the sources, reshape, aggregate, load into tables people query. Each job waits for the ones it depends on.

The dependencies form a DAG, and running the pipeline means running the jobs in topological order. That is [day 134](#), applied directly: an orchestrator computes the order, runs what has no unmet dependencies, and unblocks the next things as each finishes.

Jobs at the same level run in parallel, and the length of the longest chain is the pipeline's critical path — the fastest it can possibly be.

ETL versus ELT is about where the reshaping happens. ETL — extract, transform, load — transforms on a separate machine before loading, which is what you did when warehouse compute was scarce. **ELT** loads the raw data first and transforms **inside** the warehouse with SQL. That is the modern default, for two reasons: the warehouse's compute is elastic and cheap, and keeping the raw layer means you can re-derive things you did not anticipate. **You cannot re-transform data you threw away.**

Now the four properties that make a pipeline survivable.

One: jobs are partitioned by time, and a job owns exactly one partition. The job that runs at 4 a.m. on the 12th computes the figures **for the 11th**, and writes only the 11th's partition. It does not append to a running total. That single design choice is what makes everything else possible.

Two: re-running must be safe. That is idempotency, and here it means *overwrite*, not *append*. Bhaskar's boy fed the sheet in twice and doubled the total, because the total was built by adding. The fix was to compute the total from the day's records instead — run it again, get the same answer. In SQL that is `DELETE FROM revenue WHERE date = '2026-01-11'` followed by the insert, or an atomic partition replace. **A job that appends is a job you can never safely re-run**, and you will need to re-run it.

Three: failure resumes rather than restarts. The fat machine broke and the weighing did not have to be done again. In a pipeline, each job's success is recorded, so a retry starts from the failed job rather than from the beginning. That means jobs must be small enough to be worth resuming, and it means the orchestrator has to track state — which is most of what an orchestrator is for.

Four: late data has a stated policy. Nagappa arrives at half past nine and his milk belongs to that morning. Sources are late constantly — a partner's file, a mobile app syncing after a day offline, a payment provider's settlement report. **The two options are to reprocess the affected partition, or to fold the late data into the current one**, and the pipeline must say which. Reprocessing is correct and means yesterday's published number changes; folding forward is simpler and means individual days are slightly wrong while the total over a month is right.

Because partitions are overwritable, reprocessing is easy — which is the payoff for property one.

A backfill is the same mechanism run over history. New metric, or a bug fixed: re-run the pipeline for the last six months, one partition per day. This works **only** because each run overwrites its own partition and touches nothing else, and it is why “can you

backfill it?” is a question with a one-word answer in a well-built pipeline and a project in a badly built one.

And the last idea, which is Bhaskar’s calibration check: a pipeline that runs successfully can still produce wrong numbers. Every job succeeded for nine days and every figure was 4% light. **Nothing in “did the job run?” catches that.** So the pipeline needs assertions on the *data*, not only on the jobs — row counts within expected bounds, no nulls in key columns, totals reconciling against the source — and a failed assertion has to stop the pipeline rather than warn, because a wrong number that reaches a dashboard is worse than a missing one.

4 The picture

The pipeline as a DAG, which is what an orchestrator actually holds:

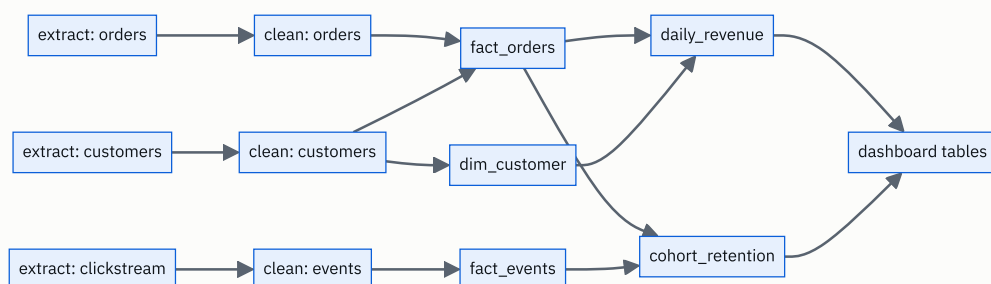


FIGURE 139.1

What to notice. The three extracts have no dependencies, so they run in parallel — that is level zero of the topological sort. `daily_revenue` cannot start until both `fact_orders` and `dim_customer` are finished. **The number of levels is the critical path**, and the widest level tells you how many workers are worth paying for. Both fall straight out of [day 134](#).

Partitioned, idempotent processing:

```
run for 2026-01-11, at 04:00 on 2026-01-12

reads:  source rows WHERE created_at ≥ '2026-01-11'
        AND created_at < '2026-01-12'
writes: DELETE FROM daily_revenue WHERE date = '2026-01-11'
        INSERT ...                                     ← one transaction

run it again → same input, same output. Nothing doubles.
run it for 2026-01-10 → touches only that partition. Nothing else moves.
```

THE VERSION THAT CANNOT BE RE-RUN:

```
INSERT INTO daily_revenue SELECT ... FROM orders WHERE ...
```

^

appends. Run twice, and the day is counted twice.
And you WILL run it twice, because retries exist.

Failure and resume:

```
extract  clean  fact    daily_revenue  dashboard
OK → OK → OK → FAILED → (not started)
```

|

v

03:12 retry: starts HERE, not at the beginning
the three completed jobs are not re-run

BUT: the retry must be safe, because "failed" might mean
"wrote half the rows and then crashed"
→ which is exactly why the write is DELETE + INSERT
in one transaction, not an append

What to notice. Resuming is only safe because each job's write is idempotent. A job that failed halfway through an append has already corrupted its output, and a retry makes it worse. **The two properties are not independent** — resumability requires idempotency.

And the backfill:

```
new metric added. Recompute the last 180 days.

for each date in the last 180 days:
  run the pipeline for that date      ← each overwrites its own partition

180 independent runs, parallelisable, each safe alone
and none of them touches any other day
```

5 How it actually works

The orchestrator

An **orchestrator** — Airflow, Dagster, Prefect, or a cloud equivalent — does five things:

1. Holds the dependency graph.
2. Triggers runs on a schedule.
3. Runs jobs in topological order, in parallel where possible.
4. Records each job's state, so a retry resumes.
5. Retries, alerts, and exposes the history.

Airflow's core object is literally a `DAG` :

```
with DAG("daily_revenue", schedule="0 4 * * *", start_date=..., catchup=True) as dag:
    extract = PythonOperator(task_id="extract_orders", ...)
    clean   = PythonOperator(task_id="clean_orders", ...)
    fact    = PythonOperator(task_id="build_fact_orders", ...)
    revenue = PythonOperator(task_id="daily_revenue", ...)

    extract >> clean >> fact >> revenue           # the edges
```

`catchup=True` is the **backfill mechanism**: if the DAG is deployed with a start date six months ago, Airflow schedules one run per day for the whole period. That only works if the jobs are partitioned and idempotent — which is the whole point of both properties.

The execution date is the parameter every job takes. A run “for 2026-01-11” reads that day's slice and writes that day's partition, regardless of when it actually executes. **A job that uses `now()` instead of the execution date cannot be backfilled and cannot be re-run**, because re-running it tomorrow would compute tomorrow's data. This is the single most common bug in badly written pipelines.

Idempotent writes, three ways

```
-- 1. Delete and insert, in one transaction. Simple, works everywhere.
BEGIN;
DELETE FROM daily_revenue WHERE date = :execution_date;
INSERT INTO daily_revenue SELECT ... WHERE date = :execution_date;
COMMIT;
```

```
-- 2. Partition swap. Atomic, and no delete cost.
CREATE TABLE daily_revenue_20260111_new AS SELECT ...;
ALTER TABLE daily_revenue EXCHANGE PARTITION ... ;
```

```
-- 3. MERGE / upsert, keyed on the natural key.
MERGE INTO daily_revenue t USING staged s ON t.date = s.date AND t.region = s.region
WHEN MATCHED THEN UPDATE SET ...
WHEN NOT MATCHED THEN INSERT ...;
```

Number one is the default and is right most of the time. Number two is what warehouses and lakehouse table formats do internally and is the cheapest at scale. Number three is needed when a partition's rows arrive from several sources and no single job owns the whole partition.

What none of them is: `INSERT` alone.

Incremental versus full refresh

FULL REFRESH recompute the whole table from all history, every run
simple, always correct, and gets slower forever
right for small dimension tables

INCREMENTAL compute only the new partition and add it
fast, and drifts if a bug ever produced a wrong partition
right for large fact tables

the practical compromise:
incremental daily, plus a FULL refresh weekly or monthly
→ the drift is bounded by the refresh interval

That last line is the answer to “how do you know the incremental table is right?”
and it is the same reconciliation idea as everywhere else in this phase.

Late-arriving data

Two designs, and the pipeline must state which:

Reprocess. A source that can be up to three days late means the pipeline re-runs the last three days’ partitions every night, not just yesterday’s. Costs three times the compute for those tables; gives correct per-day figures. **The consequence to state out loud: a published number can change.** Anything downstream that snapshotted it is now inconsistent.

Fold forward. Late rows are counted in the day they arrived. Individual days are wrong, the monthly total is right, and no published number ever changes. **Right when the aggregate is what matters and stability is worth more than per-day precision** — which is Bhaskar’s normal policy, and the union’s end-of-day cutoff is the reprocessing window.

The related design detail: **partition by event time, not by arrival time.** A row that happened on the 11th belongs in the 11th’s partition even if it arrived on the 14th. Partitioning by arrival makes every query about “what happened on the 11th” wrong.

Data quality, and where the checks go

Bhaskar’s five-litre measure. Three kinds of assertion, and they belong at different points:

```
SOURCE checks      before transforming
    row count within 50-200% of the trailing average
    no nulls in the primary key
    the max timestamp is recent (the source is not stale)

TRANSFORM checks    after each significant step
    row counts before and after a join (a fan-out means a duplicate key)
    no unexpected nulls introduced

OUTPUT checks       before anything is published
    the total reconciles against an independent source
    no day-over-day change beyond a threshold without a flag
    referential integrity: every fact row's dimension key exists
```

A failed check must stop the pipeline, not warn, because a wrong number on a dashboard is worse than a missing one — a missing number gets investigated and a wrong one gets acted on. dbt calls these `tests`; Great Expectations is the dedicated tool; a `CASE WHEN ... THEN RAISE` is a perfectly good version at small scale.

The row-count fan-out check catches the most common transformation bug: a join whose right side has duplicate keys silently multiplies rows, and the total revenue quietly doubles. Comparing counts before and after every join catches it every time, and almost nobody does it.

Scheduling and SLAs

```
schedule    04:00 daily, computing the previous day
SLA          published by 08:00
runtime      typically 40 minutes
buffer       3+ hours for retries
```

Alert on the SLA, not on the job. “The pipeline failed at 04:12 and the retry succeeded at 04:30” is not an incident. “It is 07:45 and yesterday’s revenue does not exist” is. **A pipeline that pings someone for every transient failure gets its alerts muted, and then the real one is missed.**

And the dependency on an upstream source is the usual cause of a missed SLA. A sensor — a job that waits for a file to appear or a table to be updated — is how the pipeline expresses “do not start until the source is ready”, and it should time out and alert rather than wait forever.

6 The numbers

A concrete pipeline. 50 million orders a year, 5 billion clickstream events, 30 jobs:

daily volume			
orders	50,000,000 / 365	= 137,000 rows/day	x 400 B = 55 MB
events	5,000,000,000 / 365	= 13,700,000/day	x 200 B = 2.7 GB

The behavioural data is $50\times$ the transactional data, which is what actually sizes the pipeline.

Runtime:

extract (CDC, continuous)	0 min of batch time
clean + type events (2.7 GB)	~8 min on a medium warehouse cluster
build fact tables	~12 min
aggregations	~15 min
quality checks	~5 min

critical path	~40 min
total compute across all jobs	~90 min (some run in parallel)

With 30 jobs, 6 levels and a widest level of 8, that is a 40-minute critical path against 90 minutes of serial work — so parallelism saves more than half, and **more than 8 workers buys nothing**. Both numbers come free from the topological levels.

The backfill:

180 days x 40 min sequential	= 120 hours = 5 days
180 days, 10 in parallel	= 12 hours
180 days, 30 in parallel	= 4 hours

A backfill is embarrassingly parallel because each day's partition is independent — that is the payoff for the partitioning discipline, and it turns “we cannot recompute history” into an afternoon.

But the parallelism has a limit that is not the orchestrator's:

30 concurrent backfill runs on a shared warehouse cluster
 → the cluster is saturated
 → the DAILY pipeline, running at the same time, misses its SLA

So backfills get their own compute or a concurrency limit, and that is a real operational decision rather than a detail.

Cost:

warehouse compute, medium cluster	~\$4/hour
daily pipeline, 90 min of compute	~\$6/day = \$180/month
a 180-day backfill, 270 hours	~\$1,080 one-off

Incremental versus full refresh:

```
FULL refresh of fact_orders (2 years, 100M rows)
  scan + rewrite                ~12 min, every night
  by year 5                     ~30 min, growing forever

INCREMENTAL (one day, 137k rows)
  ~30 seconds

full refresh monthly + incremental daily
  daily cost 30 s, monthly cost 12 min
  → bounded drift, and the daily runtime stops growing
```

Data quality thresholds, sized from history:

```
trailing 28-day average row count    137,000
alert if today is                    < 68,500 or > 274,000 (50% / 200%)

why so wide: a genuine sale day can double the volume
              a tighter bound pages someone every Diwali
```

Late data, quantified:

```
observed lateness of a partner's file
p50    2 hours
p95    14 hours
p99    3 days
max    9 days (a one-off outage)

reprocessing window = 3 days
  → recompute 3 partitions nightly instead of 1
  → 3x the compute for those tables, ~5 extra minutes
  → catches 99% of late rows
the 1% beyond that is caught by the monthly full refresh
```

That is the pattern for choosing any of these numbers: measure the distribution, cover p99 with the routine mechanism, and cover the tail with a periodic full recompute.

7 The trade-offs

Batch is simple and it is always behind. A nightly pipeline means the dashboard is up to 24 hours stale plus the runtime. Hourly reduces it and multiplies the compute and the number of runs that can fail. **The staleness is a chosen number and it must be stated,** because somebody will eventually make an operational decision on a figure they assume is current.

Idempotency costs you write efficiency. Delete-and-insert rewrites the whole partition even when one row changed, and a partition swap needs the full result materialised before the swap. Appending is cheaper and it is unrepeatable — **and you will re-run, because retries and backfills are not optional.** It is not a close trade.

Incremental is fast and drifts; full refresh is correct and grows. Incremental never notices that a partition written six months ago was wrong. Full refresh gets slower every year until it does not fit in the window. **The compromise — incremental daily, full monthly — bounds the drift and caps the runtime,** and it is what most mature pipelines do.

Reprocessing late data changes published numbers. Correct per-day figures, and anyone who screenshotted yesterday's dashboard now has a different number from today's. Folding forward keeps every published number stable and makes individual days slightly wrong. **Neither is right in general** — it depends on whether the audience cares about per-day precision or about the total, and it is a conversation with the audience rather than an engineering choice.

More jobs means more granular resumption and more orchestration overhead. One enormous job restarts from the beginning when it fails at 90%. Fifty small jobs resume precisely and give you fifty things to name, monitor, and get the dependencies right between. **The unit should be “the smallest thing worth re-running alone.”**

Data quality checks cost runtime and are the only thing that catches silent wrongness. Bhaskar's nine days of 4%-light weighings had a 100% job success rate. **“Did the job run?” and “is the number right?” are different questions,** and only the second one matters to the person reading the dashboard.

When would I not build a batch pipeline? When the requirement is genuinely real-time — a fraud decision, a live counter — which is streaming, and [day 132's](#) subject. When the transformation is small enough to be a view or a materialised view in the warehouse, which needs no orchestrator at all. And when there are three jobs and one schedule, where a cron entry and a shell script are honest and an Airflow deployment is not.

8 In the interview

How it gets asked

- “How does yesterday's revenue number get onto the dashboard?” — the direct version.
- “A job failed at 3 a.m. What happens?”
- “How do you re-run yesterday without double-counting?” — the idempotency question.

- “We fixed a bug. Recompute the last six months.” — the backfill question.
- “Every job succeeded and the number is wrong. How would you find out?”
- “Design the data pipeline for this system.”

The first ninety seconds

“It is a chain of jobs with dependencies, run on a schedule, and the dependencies form a DAG — so the orchestrator runs them in topological order, with everything at the same level in parallel.

The path: production writes go to the warehouse continuously via change data capture. At 4 a.m. a run starts for the previous day. It cleans and types the raw data, builds the fact and dimension tables, computes the aggregates, runs quality checks, and writes the dashboard tables. About forty minutes for the critical path.

Two design properties make it survivable, and I would build both in from the start.

Every job is partitioned by time and owns exactly one partition. The run for the 11th reads the 11th’s slice and writes the 11th’s partition, and touches nothing else. Crucially it takes the date as a **parameter** — the execution date — never `now()`. A job that uses `now()` cannot be backfilled and cannot be re-run, and that is the most common bug in badly built pipelines.

Every write is idempotent, which here means overwrite rather than append. Delete the partition and insert, in one transaction, or swap the partition atomically. If the job appends, running it twice doubles the day — and you *will* run it twice, because retries exist and backfills exist. Idempotency is also what makes resumption safe: a job that failed halfway through an append has already corrupted its output.

Those two properties give you the backfill for free. ‘Recompute the last six months’ is a hundred and eighty independent runs, one per partition, and they parallelise — five days sequentially becomes four hours at thirty at a time.

And the thing I would raise before you ask: a pipeline that runs successfully can still produce wrong numbers. Every job green, every figure wrong, because a source changed shape. So the pipeline asserts on the *data* — row counts within bounds of the trailing average, no nulls in keys, totals reconciling — and a failed assertion stops the run rather than warning, because a wrong number on a dashboard is worse than a missing one.

How fresh does the number need to be? Because if the answer is minutes rather than hours, this is the wrong architecture.”

The follow-ups

“A job fails at 3 a.m. Walk me through it.”

“First, what does *not* happen: nobody gets paged. A transient failure at 3 a.m. with three hours of buffer before the SLA is not an incident, and paging on it is how alerts get muted.

The orchestrator retries with backoff — three attempts, exponential — and **the retry starts from the failed job, not from the beginning**, because each job’s success is recorded. The four jobs that already succeeded are not re-run. That is only safe because their writes were idempotent; with an append, a half-completed job has already written rows and the retry adds more.

If all retries fail, the orchestrator marks the run failed and everything downstream is marked skipped rather than left pending — which is a forward traversal from the failed vertex, the same graph operation as everything else here. ‘Skipped 14 jobs because build_fact_orders failed’ is a much better report than fourteen timeouts.

The alert fires on the SLA, not on the failure. If it is 07:45 and yesterday’s revenue does not exist, that pages someone. The distinction matters: I want the on-call person woken for ‘the number will not be there’, not for ‘a job needed a second attempt’.

And the failure mode I would design against specifically is the **partial write** — a job that crashed after writing some rows. Delete-and-insert in one transaction makes that impossible: either the partition is the old version or the new one, never a mixture.”

“We found a bug in the revenue calculation. Recompute six months.”

“In a well-built pipeline that is a command, and in a badly built one it is a project — so let me say what makes the difference.

Because each run is parameterised by date and overwrites only its own partition, a backfill is a hundred and eighty independent runs. In Airflow that is `catchup` or a `days backfill` command. They parallelise perfectly, since no two touch the same data.

The arithmetic: forty minutes each, so a hundred and twenty hours sequentially — five days. Ten in parallel is twelve hours; thirty is four.

But the limit is not the orchestrator, it is the warehouse. Thirty concurrent backfill runs saturate the cluster, and the *daily* pipeline running at the same time misses its SLA. So the backfill either gets its own compute — which separated storage and compute makes easy — or a concurrency limit that leaves headroom. I would set that deliberately rather than discover it.

Three practical things I would insist on. Backfill into a **separate table** first and compare against the existing one, so I can see exactly what changed before publishing — a bug fix that changes revenue by 40% needs a human to look at it. Run it **newest first**, because recent data matters most and I want the fix visible today rather than in four hours. And **announce it**, because anyone who reported last quarter’s figure from this table is about to have a different number.

What makes this impossible in a bad pipeline: jobs that use `now()`, jobs that append, and aggregates maintained as running totals. All three mean history cannot be recomputed, only rebuilt from nothing.”

“Every job succeeded and the number is wrong. How would you have caught it?”

“Job success and data correctness are different questions, and only the second one matters to the person reading the dashboard. So the answer is assertions on the data, and I would place them at three points.

On the source, before transforming. Row count within, say, fifty to two hundred percent of the trailing 28-day average — wide, deliberately, because a genuine sale day doubles the volume and a tight bound pages someone every festival. No nulls in the primary key. And the maximum timestamp is recent, which catches a source that has silently stopped updating — a job reading a stale table succeeds perfectly and produces yesterday’s answer again.

After each join. Compare the row count before and after. **A join whose right side has duplicate keys silently multiplies rows**, and the total quietly doubles. This is the most common transformation bug and almost nobody checks for it, and it is two lines.

On the output, before publishing. Reconcile the total against something independent — the sum in the warehouse against the sum in production for the same window, allowing for a small tolerance. And a day-over-day change beyond a threshold requires a flag rather than passing silently.

A failed check stops the pipeline, and downstream is skipped rather than fed bad input. That is the whole point: a missing number gets investigated, a wrong number gets acted on.

And the honest caveat: none of this catches a source that is wrong in a plausible way — Bhaskar’s scale reading 4% light for nine days passes every one of those checks. That is what the reconciliation against an independent source is for, and if there is no independent source, then the answer is that this number cannot be verified and everyone should know that.”

“How do you handle a source that arrives four hours late, sometimes three days late?”

“Measure the distribution first, then choose a mechanism that covers p99 and a periodic full recompute for the tail. I would not pick a number by feel.

Say p50 is two hours, p95 is fourteen, p99 is three days. Then the routine mechanism is a **three-day reprocessing window**: every night, recompute the last three partitions rather than only yesterday’s. That costs three times the compute for the affected tables — about five extra minutes here — and catches 99% of late rows. The remaining 1% is caught by the monthly full refresh.

Two design details that matter more than the window. Partition by **event time**, not arrival time — a row that happened on the 11th belongs in the 11th’s partition even if it arrives on the 14th, otherwise every question about ‘what happened on the 11th’ is wrong. And this only works because partitions are overwriteable, which is the payoff for the idempotency discipline.

The consequence I would state explicitly is that published numbers change. Yesterday’s revenue as displayed this morning may not equal yesterday’s revenue as displayed on Thursday. For an internal dashboard that is fine if people know. For a figure that goes into a board pack or an external report, it is not — so those read from a **snapshot table**, written once when the period closes and never modified. That separation between ‘the live view, which may be revised’ and ‘the number of record, which is frozen’ is the design, and it is a governance decision as much as a technical one.

The alternative — fold late rows into the day they arrived — is simpler and keeps every published number stable, at the cost of individual days being slightly wrong. Right when the monthly aggregate is what matters and nobody looks at single days.”

The model answer

“Design the pipeline that produces the daily business dashboard for an e-commerce platform: revenue, orders, conversion rate, and cohort retention. It must be ready by 8 a.m.”

“Let me work backwards from the 8 a.m. deadline, because that constrains everything else.

The shape. Production Postgres feeds the warehouse continuously via CDC, so extraction is not part of the nightly window at all — the raw data is already there when the pipeline starts. That matters: an extraction step that pulls from production at 4 a.m. is both a load on production and the most likely thing to make me miss the SLA.

Schedule 04:00, SLA 08:00, expected runtime 40 minutes. That leaves over three hours of buffer for retries and for an upstream source being late, which is where the buffer actually gets spent.

The DAG, in layers. Clean and type the raw tables — orders, customers, events — in parallel, since they are independent. Then build `fact_orders`, `fact_events`, and the dimension tables. Then the aggregates: `daily_revenue`, `daily_orders`, `conversion_funnel`, `cohort_retention`. Then quality checks. Then publish to the dashboard tables. Six levels, about thirty jobs, widest level around eight — so eight workers, and more buys nothing. Those two numbers come free from the topological levels and I would put them in the design doc.

Every job takes the execution date as a parameter and overwrites exactly its own partition, with a delete-and-insert in one transaction. That gives me safe retries, safe backfills, and safe reprocessing of late data with no additional machinery.

Cohort retention is the one that does not fit the pattern, and I would call that out. It is inherently a full-history computation — a cohort’s retention curve changes as time passes — so it cannot be a per-day partition. I would rebuild it incrementally where possible, and do a full recompute weekly, and be explicit that this table is the slowest job and the one most likely to grow past its window. **Every pipeline has one job like this and pretending it fits the general pattern is how the runtime quietly triples over two years.**

Late data: I would measure the arrival distribution for each source. Payment settlement files are usually the worst offender — often a day or two behind — so a three-day reprocessing window on the revenue tables, which costs a few extra minutes.

Quality gates before publishing, and these are the part I would defend hardest in review: order count within 50–200% of the trailing 28-day average; no nulls in `order_id` or `customer_id`; row counts checked before and after every join, because a duplicate dimension key silently doubling revenue is the failure that actually happens; and revenue reconciled against the sum in production for the same

window, within a tolerance. **Failure stops the pipeline and the dashboard shows yesterday's data with a stale marker**, because a visibly stale number is much better than a confidently wrong one.

Alerting on the SLA, not on jobs. A retry at 04:12 that succeeds is not an incident. 'It is 07:45 and revenue does not exist' pages someone.

And the two things I would agree with the business before building. First, which numbers are allowed to change after publication and which are frozen — the live dashboard can be revised as late data arrives, but the month-end figures go into a snapshot table written once and never touched, or finance will find two different answers to the same question and stop trusting all of it. Second, that **revenue is defined once**, in one model, and every dashboard reads that definition. Two teams computing revenue slightly differently is the most common data problem in any company, and it is not a technical failure — it is a failure to decide.”

9 Recall card

A pipeline is a DAG and running it is a topological sort — levels give the critical path and the useful worker count. ELT (load raw, transform in the warehouse) over ETL, because you cannot re-transform what you threw away.

Two properties make it survivable: partition by time, and make every write idempotent — overwrite, never append. Jobs take the **execution date as a parameter**, never `now()` ; a job using `now()` can be neither re-run nor backfilled.

Those two give you backfills for free: 180 days is 180 independent runs, parallelisable — 5 days sequentially, 4 hours at thirty at a time. The limit is warehouse capacity, not the orchestrator.

Late data needs a stated policy: reprocess a window sized from the measured p99 (correct per-day, but published numbers change) or fold forward (stable, individual days wrong). **Partition by event time, not arrival time.**

A pipeline that runs successfully can still be wrong. Assert on the data — row counts against the trailing average, no nulls in keys, **counts before and after every join** (a duplicate key silently doubles revenue), totals reconciled against the source — and **fail the run rather than warn**. Alert on the SLA, not on jobs.

Practice — Minimum spanning trees: Kruskal and Prim

DSA topic: Minimum spanning trees: Kruskal and Prim **System design topic:** ETL, batch pipelines, and where data goes to be counted

Code these, in this order

One rule for the whole set: store edges as `(weight, a, b)` so a bare `sort()` is correct, and check `len(chosen) == n - 1` before returning a total. The first removes a place to make a mistake; the second is the line almost everybody forgets, and without it a disconnected graph returns a plausible number.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Min Cost to Connect All Points	LeetCode 1584 (Medium)	An implicit complete graph — dense Prim's, and never building the edge list.
2	Connecting Cities With Minimum Cost	LeetCode 1135 (Medium)	Plain Kruskal's, plus the disconnected check.
3	Optimize Water Distribution in a Village	LeetCode 1168 (Hard)	The virtual-vertex trick.
4	Find Critical and Pseudo-Critical Edges	LeetCode 1489 (Hard)	Kruskal's run repeatedly, with an edge forced in or excluded.
5	Min Cost to Connect All Points, again	—	Solve with Kruskal's too, and compare the memory.

On problem 1, do the arithmetic before coding

n points means $n(n-1)/2$ edges. Compute that for $n = 1,000$. Then write the $O(n^2)$ dense Prim's that never materialises the edge list, and separately write the Kruskal's version that does. Record the peak memory of each. One of them is $O(n)$ and the other is $O(n^2)$.

On problem 3, write the reframing down first

Before any code, write two sentences: what the extra vertex is, and what a well becomes. Then say why the graph is now guaranteed connected and what that means for the -1 check. Then solve it.

On problem 4, be careful with “forced in”

Forcing an edge into the tree means taking it *before* sorting the rest and starting the total from its weight. Excluding one means skipping it entirely, not deprioritising it. Write both helpers, and test each on a graph where you know the answer by hand.

Then the Prim’s-versus-Dijkstra experiment

Take your Dijkstra from [day 136](#) and change exactly one line — the value pushed into the heap — to turn it into Prim’s. Then build a graph where the two give different trees:

```
A --1-- B --5-- C
A -----4----- C
```

Run both from A. Record the edge sets. Write one sentence saying which objective each one minimised.

Then the disconnected check

Run Kruskal’s on four vertices and two edges, with and without the `len(chosen) == n - 1` check. Record both returns. Say which one a caller would notice.

Then the tie experiment

Build a graph where three edges share the same weight and there are several minimum spanning trees. Run Kruskal’s and Prim’s and compare the edge sets and the totals. Write down which one is safe to assert in a test.

The definitions drill

1. Define a spanning tree and say how many edges it has.
2. Say why a minimum spanning tree never contains a cycle.
3. Say what happens on a disconnected graph and what you return.
4. Say when the minimum spanning tree is unique.

The two-algorithms drill

1. Write Kruskal’s from memory in eight lines.
2. Write Prim’s with a heap from memory.
3. Point at the one line that differs from Dijkstra and say what each version minimises.
4. Write the dense `O(V2)` Prim’s and say what `best[v]` holds.
5. Give the rule for choosing between them, with the density arithmetic.

The proof drill

1. State the cut property.

2. Give the swap argument in three sentences.
3. Say how Prim's is an application of it.
4. Say how Kruskal's is an application of it.
5. State the cycle property and say what it justifies.

The not-shortest-path drill

1. Give the triangle counter-example and both answers.
2. State the two objectives in one sentence each.
3. Say what happens if you write Prim's with Dijkstra's key.
4. Say why that bug agrees with the correct version surprisingly often.

The costs drill

1. Give Kruskal's cost and say what dominates.
2. Give heap-Prim's and dense-Prim's costs.
3. Compute all three at $V = 100,000$, $E = 300,000$ and at $V = 5,000$ complete.
4. Say what Kruskal's memory constraint is and when it binds.
5. Say what happens to Kruskal's cost if the edges are pre-sorted.

The break-it drill

Trigger each and record the exact output or error:

1. Prim's with the accumulated key, on a chain-versus-single-edge graph.
2. No `len(chosen) == n - 1` check, on a disconnected graph.
3. Prim's without the `in_tree` stale-entry skip.
4. Kruskal's on a directed edge list.
5. A test asserting a specific edge set on a graph with tied weights.
6. Building the full edge list for 100,000 points on a plane.

The path drill

1. Trace a number from a production write to a dashboard, naming every stage.
2. Say what a pipeline's dependency structure is, in graph terms.
3. Say what the number of levels and the widest level tell you.
4. Say what ELT is and give the two reasons it replaced ETL.

The two-properties drill

1. State both properties that make a pipeline survivable.

2. Say what “the job takes the execution date as a parameter” prevents.
3. Give three idempotent write patterns and say when each is right.
4. Say what `INSERT` alone costs you.
5. Say why resumability requires idempotency.

The backfill drill

1. Say why a backfill is possible at all, in terms of the two properties.
2. Compute the time for 180 days at 40 minutes, sequential and at 10 and 30 parallel.
3. Say what limits the parallelism, and it is not the orchestrator.
4. Give three operational precautions for a backfill.
5. Name three things that make a backfill impossible.

The late-data drill

1. Give the two policies and what each costs.
2. Say how you would size the reprocessing window.
3. Say why you partition by event time and not arrival time.
4. Say what “published numbers change” means for downstream consumers.
5. Give the design that keeps some numbers frozen.

The quality drill

1. Name the three places checks belong and give two checks for each.
2. Say why a failed check must stop rather than warn.
3. Give the join check and say what bug it catches.
4. Say why row-count bounds should be wide.
5. Name the failure that none of these checks catches.

The incremental drill

1. Compare incremental and full refresh on correctness and on runtime.
2. Give the compromise and say what it bounds.
3. Compute the runtime difference for a 100M-row fact table.

The numbers drill

1. Compute daily volume for 50M orders and 5B events a year, and give the ratio.
2. Break a 40-minute critical path into its stages.
3. Compute backfill time at three parallelism levels.
4. Compute the monthly compute cost of a 90-minute daily pipeline.

5. Size a reprocessing window from a lateness distribution.
6. Set a row-count alert threshold from a trailing average and justify the width.

The failure drill

For each, say what happens and what you would build:

1. A job fails at 03:12 with a three-hour SLA buffer.
2. A job crashed halfway through an `INSERT`.
3. A job that computes “today” using `now()`, re-run tomorrow.
4. A source table stops updating but the job still succeeds.
5. A join whose right side gained a duplicate key.
6. Thirty backfill runs launched while the nightly pipeline is running.
7. A scale reading 4% light for nine days, with every job green.

Two of the seven are caught by no check you can write. Name them and say what does catch them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Connect all the cities at minimum cost.* Spanning tree with $V - 1$ edges and no cycles, Kruskal’s with Union-Find, Prim’s as Dijkstra with one line changed, the density rule for choosing, the disconnected check, and “this is not a shortest-path tree” said before being asked.
 2. *Why does the greedy choice work here?* The cut property stated precisely, then the swap argument in three sentences, then how each algorithm is an application of it — plus the cycle property for why skipping is safe.
 3. *How does yesterday’s revenue number get onto the dashboard?* The DAG and topological order, partitioned idempotent jobs with the execution date as a parameter, resumption, backfills falling out for free, late-data policy, and data quality checks that stop the run.
-

Before you move on

- [] I store edges as `(weight, a, b)` and use a bare `sort()`.
- [] I check `len(chosen) == n - 1` before returning a total.
- [] I can write Kruskal’s in eight lines from memory.

- [] I can write heap-Prim's and the dense $O(V^2)$ version.
- [] I can point at the one line that differs from Dijkstra.
- [] I can give the density rule with numbers.
- [] I can state the cut property and give the swap argument.
- [] I can state the cycle property and what it justifies.
- [] I have the triangle counter-example ready for MST-is-not-shortest-path.
- [] I know Kruskal's cost is dominated by the sort.
- [] I know Kruskal's memory constraint and when it binds.
- [] I know the tree is not unique with tied weights, so I assert the total.
- [] I know the virtual-vertex trick and the general pattern behind it.
- [] I know Union-Find has no direction and what the directed analogue is called.
- [] I can trace a number from a production write to a dashboard.
- [] I know a pipeline is a DAG and running it is a topological sort.
- [] I can state both survivability properties.
- [] I know why `now()` in a job is the most common pipeline bug.
- [] I can give three idempotent write patterns.
- [] I know resumability requires idempotency.
- [] I can explain why backfills are free, with the arithmetic.
- [] I know what limits backfill parallelism.
- [] I can give both late-data policies and what each costs.
- [] I know to partition by event time, not arrival time.
- [] I know where the three kinds of quality check belong.
- [] I know the before-and-after-join row count check and what it catches.
- [] I know a failed check must stop the run.
- [] I alert on the SLA, not on job failures.
- [] I know what none of the checks catch, and what does.
- [] I answered all three questions above out loud.

Bipartite graphs and two-colouring

DATA STRUCTURES AND ALGORITHMS

Bipartite graphs and two-colouring

SYSTEM DESIGN

Websockets, long polling, and server-sent events

1 What this is, and why they ask it

A graph is **bipartite** if you can split its vertices into two groups such that every edge goes *between* the groups and none stays inside one. Two teams with no conflicts within a team. Two tables where nobody sits opposite someone they cannot bear.

Checking it is a traversal with one extra field: give the starting vertex a colour, give every neighbour the opposite colour, and if you ever meet a vertex that already has the *same* colour as the one you came from, no such split exists.

They ask it because it is the cleanest example of a whole category — **a constraint that propagates**. You do not get to choose colours; the first choice forces everything reachable from it. And because the answer to “when is it impossible?” is a single crisp fact worth knowing: **a graph is bipartite exactly when it contains no cycle of odd length**.

It also hides in problems that never say “bipartite”: splitting people into two groups, detecting a contradiction in a set of “these two differ” statements, checking whether a matching problem is even well-posed. **The tell is any problem about two sides, or about a relation that must never hold within a group**.

By the end of this lesson you can two-colour with BFS and DFS, state and justify the odd-cycle rule, handle a disconnected graph, return the two groups rather than just a boolean, and say what changes when the question becomes three colours.

2 The story

There are two long tables in the hall and Kamala has ninety-four people to seat.

The list her mother-in-law gave her is not a seating plan. It is a list of pairs who must not be at the same table, and it runs to about thirty lines, and every one of them has a history that Kamala has heard at least twice.

Her aunt Sushila and Sushila’s brother-in-law: not together. That one is from 1998 and concerns a scooter. Ramesh and his cousin Prakash: not together. Prakash and his wife’s brother: also not together. Kamala’s mother and one particular neighbour: absolutely not, and the reason was never explained to her.

She started on Wednesday evening expecting it to take an hour.

What she found straight away was that she had almost no choices. She put Sushila at the first table because somebody had to go first. That immediately put the brother-in-law at the second table. But the brother-in-law is also on the list with his own nephew, so the nephew goes back to the first table. And the nephew does not speak to Kamala's mother's neighbour, so that neighbour goes to the second.

One decision at the start, and eleven people placed with no further thought. She was not solving anything after the first name; she was just following.

The unrelated groups were easy. The people from her husband's office had no feuds at all, so they went wherever there was room. And Ramesh's chain was separate from Sushila's chain entirely, so she started that one fresh, with another arbitrary first choice.

Then she reached the five.

Her father-in-law's brothers, and it took her a long time to see what was wrong. The eldest will not sit with the second. The second will not sit with the third. The third will not sit with the fourth, the fourth will not sit with the fifth, and — this is the part — **the fifth will not sit with the eldest**. Five men, five refusals, in a ring.

She placed the eldest at table one, so the second went to table two, the third back to table one, the fourth to table two, and the fifth to table one. And the fifth and the eldest were now at the same table, which is the one thing the ring said could not happen.

She tried it starting from a different brother. Same result. She tried it starting from the second table. Same result.

At about eleven o'clock she understood that it was not that she was doing it badly. **Five people in a ring, each refusing the next, cannot be split into two tables.** Six could. Four could. Five cannot, and no amount of rearranging changes it.

She put a small third table by the window for two of them and told her mother-in-law it was for the elders.

3 The idea in plain English

Kamala's evening is two-colouring, and the five brothers are the odd cycle.

A bipartite graph splits into two groups with no edge inside either group. Two tables, and every pair on the list ends up separated. The word means "two parts", and the groups are usually called the two **sides**.

Colouring is the way to check it. Give a vertex a colour — call them 0 and 1, or red and blue — and give every neighbour the opposite. **You are not choosing; you are propagating.** Kamala placed Sushila and then eleven people followed with no decision

at all.

The first choice in each component is arbitrary and does not matter. Starting Sushila at the second table just swaps the two sides. So the algorithm picks a colour for each unvisited start vertex and follows.

A conflict is a neighbour that already has the same colour as you. Not merely “already coloured” — already coloured **the same**. A neighbour with the opposite colour is exactly what you wanted; that is the constraint being satisfied, not violated. This is the check people get wrong, and it is the direct parallel of [day 132](#)’s “seen is not enough”.

When a conflict happens, no split exists, and the reason is worth having: the conflict means you reached a vertex two ways, along paths whose lengths have the same parity, which closes a cycle of **odd** length.

The rule, in one line: a graph is bipartite if and only if it has no odd-length cycle.

The argument in both directions is short. **If it is bipartite**, then walking along any cycle alternates sides — side A, side B, side A — so returning to your starting vertex means you took an even number of steps. So every cycle is even. **If every cycle is even**, then colouring by the parity of the distance from a start vertex is consistent: two different routes to the same vertex must have the same parity, because otherwise gluing them together would form an odd cycle.

Kamala’s five brothers are a cycle of length five. Colours alternate 1, 2, 1, 2, 1 — and the fifth meets the first, both at table one. Six brothers in a ring would work; five cannot.

Trees are always bipartite, because a tree has no cycles at all. So are grids, and any graph you can lay out in alternating layers. **Any graph containing a triangle is not**, because a triangle is a cycle of length three.

The graph is probably not connected, and each piece is coloured independently. Kamala’s office group had no constraints and Ramesh’s chain was separate from Sushila’s. So the algorithm needs the outer loop over all vertices — the same requirement as [connected components](#) — and **a graph is bipartite only if every component is**.

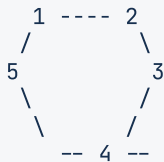
And the last thing: usually you want the two groups, not a boolean. The colour array already holds them — all the 0s and all the 1s — so returning the split costs one extra line and is what most real problems actually ask for.

Two colours is easy and three is not. Deciding whether a graph can be coloured with three colours is NP-complete: no known efficient algorithm, and nobody expects one. **Two-colouring is a linear-time traversal; three-colouring is a research prob-**

lem. That cliff between two and three is worth knowing, because an interviewer may ask.

4 The picture

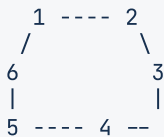
Kamala's five brothers:



```
colour 1 as A
2 must be B
3 must be A
4 must be B
5 must be A
... and 5 is adjacent to 1, which is also A.  CONFLICT.

cycle length 5 → ODD → not bipartite
```

Six brothers, same shape, one more person:



```
1=A 2=B 3=A 4=B 5=A 6=B
6 is adjacent to 1: B and A.  Fine.

cycle length 6 → EVEN → bipartite
tables:  A = {1, 3, 5}  B = {2, 4, 6}
```

What to notice. One extra person changes it from impossible to trivial. Nothing else about the structure differs — it is entirely the parity of the ring.

The propagation, traced with BFS:

```
graph: 0-1, 0-3, 1-2, 2-3, 3-4
```

step	queue	pop	neighbours	action	colours
1	[0]	0	1, 3	colour both 1	0:0 1:1 3:1
2	[1,3]	1	0, 2	0 is 0, differs: ok colour 2 with 0	2:0
3	[3,2]	3	0, 2, 4	0 is 0, differs: ok 2 is 0, differs: ok colour 4 with 0	4:0
4	[2,4]	2	1, 3	both 1, both differ: ok	
5	[4]	4	3	1, differs: ok	

result: side A = {0, 2, 4} side B = {1, 3} bipartite

What to notice at step 3. Vertex 3 looks at vertex 2, which is already coloured — and that is **fine**, because it is coloured differently. “Already coloured” is not a conflict; “already coloured the same” is.

And the conflict, on a triangle:

```
graph: 0-1, 1-2, 2-0
```

```
colour 0 → A
colour 1 → B      (neighbour of 0)
colour 2 → A      (neighbour of 1)
now 2 looks at 0: both A.  CONFLICT.
```

```
triangle = cycle of length 3 = odd → never bipartite
```

The two-sides shape, which is what bipartite graphs usually look like when drawn deliberately:

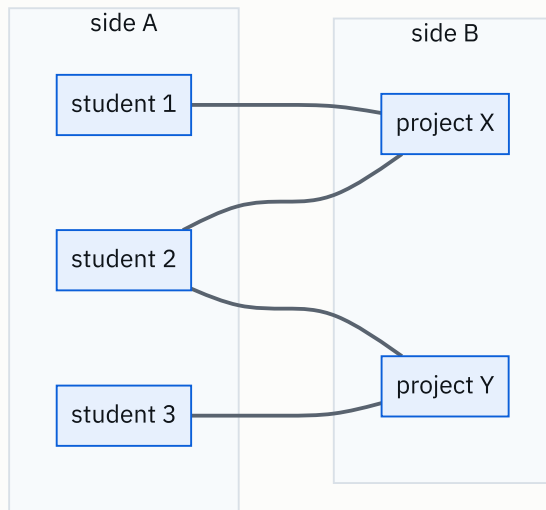


FIGURE 140.1

What to notice. Many real bipartite graphs are bipartite *by construction* — students and projects, buyers and sellers, jobs and machines — and nobody has to check. **The interesting problems are the ones where the two sides are not given and you have to find out whether they exist,** which is Kamala’s list.

5 The code, built step by step

BFS first, because the queue makes the propagation obvious.

```

from collections import deque

def is_bipartite(graph: dict[int, list[int]], n: int) → bool:
    colour = [-1] * n          # -1 means uncoloured
    for start in range(n):     # every component
        if colour[start] != -1:
            continue
        colour[start] = 0      # arbitrary, and it does not matter
        queue = deque([start])
        while queue:
            vertex = queue.popleft()
            for neighbour in graph[vertex]:
                if colour[neighbour] == -1:
                    colour[neighbour] = 1 - colour[vertex]    # the opposite
                    queue.append(neighbour)
                elif colour[neighbour] == colour[vertex]:
                    return False    # same colour: odd cycle
        return True
  
```

Twelve lines, and three of them carry it.

`1 - colour[vertex]` flips 0 to 1 and 1 to 0. Clean, and it makes “the opposite” one operation.

`elif colour[neighbour] == colour[vertex]` is the entire check, and note what it is *not*: it does not test whether the neighbour has been coloured. Being coloured differently is the desired outcome.

The outer loop over `range(n)` handles disconnection, and the graph must be bipartite in *every* component. `colour` doubles as the seen set, so there is no separate structure.

Returning the sides instead of a boolean is one line:

```
def two_sides(graph: dict[int, list[int]], n: int) → tuple[list[int], list[int]] | None:
    colour = _colour(graph, n)
    if colour is None:
        return None
    return ([v for v in range(n) if colour[v] == 0],
            [v for v in range(n) if colour[v] == 1])
```

This is what most real problems want — “split these into two groups” rather than “can they be split” — and the colour array already holds the answer.

The DFS version, for comparison:

```
def is_bipartite_dfs(graph: dict[int, list[int]], n: int) → bool:
    colour = [-1] * n

    def visit(vertex: int, c: int) → bool:
        colour[vertex] = c
        for neighbour in graph[vertex]:
            if colour[neighbour] == -1:
                if not visit(neighbour, 1 - c):
                    return False
            elif colour[neighbour] == c:
                return False
        return True

    return all(colour[v] != -1 or visit(v, 0) for v in range(n))
```

Both work and both are $O(V + E)$. I would write BFS on a graph that might be deep, for the usual reason: a chain of a hundred thousand vertices overflows the recursion stack.

BFS also gives you something DFS does not, cheaply: the odd cycle itself. Keep parent pointers, and when a conflict is found, walk both endpoints back to their common ancestor:

```
def find_odd_cycle(graph: dict[int, list[int]], n: int) → list[int] | None:
    colour = [-1] * n
    parent = [-1] * n
    for start in range(n):
        if colour[start] ≠ -1:
            continue
        colour[start] = 0
        queue = deque([start])
        while queue:
            vertex = queue.popleft()
            for neighbour in graph[vertex]:
                if colour[neighbour] = -1:
                    colour[neighbour] = 1 - colour[vertex]
                    parent[neighbour] = vertex
                    queue.append(neighbour)
                elif colour[neighbour] = colour[vertex]:
                    return _build_cycle(parent, vertex, neighbour)
    return None
```

```
def _build_cycle(parent: list[int], a: int, b: int) → list[int]:
    """Walk both back to their meeting point; the two halves plus the edge form the cycle."""
    seen_a, node = set(), a
    while node ≠ -1:
        seen_a.add(node)
        node = parent[node]
    meet = b
    while meet not in seen_a:
        meet = parent[meet]
    left, node = [], a
    while node ≠ meet:
        left.append(node)
        node = parent[node]
    right, node = [], b
    while node ≠ meet:
        right.append(node)
        node = parent[node]
    return [meet] + left[::-1] + right + [meet]
```

Being able to show the offending cycle turns “impossible” into “here is why”, which for Kamala’s problem is the difference between a useless answer and an actionable one.

And the variation that most interview problems actually are — the constraint is given as pairs of people who must be separated, and the vertices are not numbered:

```
def possible_split(n: int, dislikes: list[list[int]]) → bool:
    """People are 1..n. dislikes[i] = [a, b] means a and b cannot share a group."""
    graph: dict[int, list[int]] = {v: [] for v in range(1, n + 1)}
    for a, b in dislikes:
        graph[a].append(b)
        graph[b].append(a)
    ...
```

undirected: BOTH directions

The two appends are not optional, and forgetting one gives a graph where the constraint only holds in one direction — which produces a confident, wrong `True`.

The complete solution

```
"""Bipartite checking: two-colouring, the two sides, and the odd cycle."""

from __future__ import annotations
from collections import deque

def build(n: int, edges: list[tuple[int, int]]) → dict[int, list[int]]:
    graph: dict[int, list[int]] = {v: [] for v in range(n)}
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)  # undirected: both directions
    return graph

def colour_graph(graph: dict[int, list[int]], n: int) → list[int] | None:
    """Two-colour every component. Returns the colours, or None if impossible."""
    colour = [-1] * n
    for start in range(n):
        if colour[start] ≠ -1:
            continue
        colour[start] = 0  # arbitrary; it only swaps the sides
        queue = deque([start])
        while queue:
            vertex = queue.popleft()
            for neighbour in graph[vertex]:
                if colour[neighbour] = -1:
                    colour[neighbour] = 1 - colour[vertex]
                    queue.append(neighbour)
                elif colour[neighbour] = colour[vertex]:
                    return None  # same colour across an edge
    return colour

def is_bipartite(graph: dict[int, list[int]], n: int) → bool:
    return colour_graph(graph, n) is not None

def two_sides(graph: dict[int, list[int]], n: int) → tuple[list[int], list[int]] | None:
    colour = colour_graph(graph, n)
    if colour is None:
        return None
    return ([v for v in range(n) if colour[v] = 0],
            [v for v in range(n) if colour[v] = 1])

def find_odd_cycle(graph: dict[int, list[int]], n: int) → list[int] | None:
    """The offending cycle, so 'impossible' can be explained."""
    colour = [-1] * n
    parent = [-1] * n
    for start in range(n):
        if colour[start] ≠ -1:
            continue
        colour[start] = 0
        queue = deque([start])
        while queue:
            vertex = queue.popleft()
            for neighbour in graph[vertex]:
                if colour[neighbour] = -1:
                    colour[neighbour] = 1 - colour[vertex]
                    parent[neighbour] = vertex
                    queue.append(neighbour)
                elif colour[neighbour] = colour[vertex]:
                    ancestors, node = set(), vertex
                    while node ≠ -1:
```

```

        ancestors.add(node)
        node = parent[node]
        meet = neighbour
        while meet not in ancestors:
            meet = parent[meet]
        left, node = [], vertex
        while node != meet:
            left.append(node)
            node = parent[node]
        right, node = [], neighbour
        while node != meet:
            right.append(node)
            node = parent[node]
        return [meet] + left[::-1] + right

    return None

if __name__ == "__main__":
    # Kamala's five brothers in a ring: 0-1-2-3-4-0
    five = build(5, [(0, 1), (1, 2), (2, 3), (3, 4), (4, 0)])
    print("five brothers :", is_bipartite(five, 5), find_odd_cycle(five, 5))

    # Six in a ring.
    six = build(6, [(0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (5, 0)])
    print("six brothers :", is_bipartite(six, 6), two_sides(six, 6))

    # Two separate components, one of them a triangle.
    mixed = build(7, [(0, 1), (1, 2), (2, 0), (3, 4), (4, 5), (5, 6)])
    print("mixed :", is_bipartite(mixed, 7))

    # A tree: always bipartite.
    tree = build(5, [(0, 1), (0, 2), (1, 3), (1, 4)])
    print("tree :", is_bipartite(tree, 5), two_sides(tree, 5))

```

Running it:

```

five brothers : False [0, 1, 2, 3, 4]
six brothers  : True  ([0, 2, 4], [1, 3, 5])
mixed         : False
tree          : True  ([0, 3, 4], [1, 2])

```

Three things to look at. The five-brother ring is `False`, and `find_odd_cycle` names the five people involved — which is the answer Kamala actually needed.

The six-ring splits cleanly into evens and odds, which is what alternating around an even cycle gives you.

`mixed` is `False` even though one of its two components — the path 3-4-5-6 — is perfectly bipartite. **The whole graph is bipartite only if every component is**, and a single triangle anywhere ruins it.

And the tree is bipartite with sides `{0, 3, 4}` and `{1, 2}` — the even and odd depths. **Every tree is bipartite**, because there are no cycles at all, let alone odd ones.

6 What it costs

Time and space are a plain traversal.

each vertex coloured once	V
each edge examined twice (once per end)	2E

	$O(V + E)$ time
colour array	$O(V)$
queue or recursion stack	$O(V)$

	$O(V)$ space

No extra factor over BFS at all. The colour array replaces the `seen` set rather than adding to it, so this genuinely costs the same as a bare traversal.

Concretely:

```
V = 100,000, E = 300,000
100,000 + 600,000 = 700,000 steps → milliseconds
```

Early exit is large in practice. The moment a conflict is found you return, and a graph with a triangle near the start is decided in a handful of steps. The worst case is a graph that is bipartite, where every vertex and edge must be examined to be sure.

The odd-cycle extraction adds:

parent array	$O(V)$ extra space
walking back to the meeting point	$O(V)$ once, only on failure
	→ same $O(V + E)$ overall

Memory at scale:

```
V = 1,000,000
colour as a list of small ints    ~8 MB
parent as a list of ints         ~8 MB
queue at peak (widest level)     up to V
```

Use a list indexed by vertex, not a dictionary, when vertices are `0..n-1` — a Python dict of a million entries is roughly ten times the memory of a list.

Against the alternatives, because “split into two groups” can be attacked other ways:

two-colouring (BFS/DFS)	$O(V + E)$	exact, linear
Union-Find with parity	$O(E \times \alpha)$	exact, and works as edges ARRIVE
brute force over subsets	$O(2^V \times E)$	hopeless past $V = 25$

The Union-Find version is worth knowing — keep, alongside each element, the parity of its distance to its representative, and an edge joining two elements already in the same group with the *same* parity is a contradiction. Same idea, and it works online as constraints arrive, which a traversal does not.

```
V = 100,000 constraints arriving one at a time
re-running the traversal per constraint    100,000 x 400,000 = 4 x 10^10
Union-Find with parity                    100,000 x 4      = 400,000
```

And the cliff at three colours:

2-colouring	$O(V + E)$	a traversal
3-colouring	NP-complete	no known polynomial algorithm
		backtracking, $\sim O(3^V)$ worst case
		practical only for small V or special graphs

Two is linear and three is intractable, with nothing in between, and that is a genuinely surprising fact worth being able to state.

7 The traps

Treating “already coloured” as a conflict

The near-miss, and it fails on almost every bipartite graph:

```
if colour[neighbour] != -1:
    return False                # "already coloured, so conflict"
```

```
>>> is_bipartite_broken(build(4, [(0,1),(1,2),(2,3),(3,0)]), 4)
False                               # a 4-cycle IS bipartite
```

A neighbour coloured the **opposite** is the constraint being satisfied. Only the **same** colour is a conflict. This is exactly the “seen is not enough” mistake from [day 132](#), in a new costume.

Only colouring one component

```
colour[0] = 0
bfs_from(0)
return True
```

```
>>> mixed = build(7, [(0,1),(1,2),(2,0),(3,4),(4,5),(5,6)])
>>> single_component_version(mixed, 7)
# colours 0,1,2 → finds the triangle → False. Correct here by luck.

>>> other = build(7, [(0,1),(1,2),(3,4),(4,5),(5,3)])
>>> single_component_version(other, 7)
True # the triangle at 3-4-5 was never visited
```

A graph is bipartite only if every component is, and the sample input is usually connected so nothing warns you. The outer loop over `range(n)` is required.

Building the graph one-directionally

```
for a, b in dislikes:
    graph[a].append(b) # forgot graph[b].append(a)
```

The relation “cannot sit together” is symmetric, so the graph must be. With one direction, `b` never sees `a` as a neighbour and the constraint is only half-enforced:

```
>>> possible_split_one_way(3, [[1,2],[1,3],[2,3]])
True # a triangle. The answer is False.
```

No error, and a confident wrong `True` — which for a “can this be done” question is the worst direction to be wrong in.

Assuming the sides are balanced

```
sideA, sideB = two_sides(graph, n)
assert len(sideA) == len(sideB)
```

Nothing requires the sides to be the same size. A star — one centre and nine leaves — is bipartite with sides of size 1 and 9. **If the problem *also* requires equal sizes, that is a different and much harder problem** (it becomes a partition problem, which is NP-hard), and noticing the difference is worth stating.

Recursion depth on a long chain

```
Traceback (most recent call last):
  File "bipartite.py", line 9, in visit
    if not visit(neighbour, 1 - c):
[Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

A path graph of a hundred thousand vertices is bipartite and takes a hundred thousand frames. `n ≤ 105` in the constraints means write the BFS version.

Trying to two-colour a directed graph

Bipartiteness is defined for undirected graphs. If you are handed directed edges, the question almost certainly means “ignore the directions”, and you should say so and build the graph symmetrically. Colouring while respecting direction is a different and usually meaningless computation.

Reaching for it when the problem needs three groups

```
is_bipartite(graph, n)          # "can these be split into groups with no conflicts?"
```

If the problem allows three or more groups, this answers the wrong question — a triangle needs three and is perfectly splittable. **And the three-colour version is NP-complete**, so if the problem genuinely asks for three, either `n` is small enough for backtracking or the problem has extra structure you are meant to exploit.

8 In the interview

How it gets asked

- “Can these people be split into two teams with no conflicts?” — LeetCode 886, the direct version.
- “Is this graph bipartite?” — LeetCode 785.
- “Given statements that certain pairs are different, is there a contradiction?”
- “Split the students into two classrooms so that no two who fight are together.”
- “Why is it impossible?” — the odd-cycle question.
- “Now do it with three groups.” — the NP-completeness question.

The first ninety seconds

“This is a bipartite check — can the vertices be split into two groups with every edge crossing between them — and it is a traversal with one extra array.

Colour a start vertex 0. Colour every neighbour the opposite. **The check is: if a neighbour is already coloured the *same* as the vertex I am standing on, no split exists.** I want to be precise about that, because the common bug is treating ‘already coloured’ as the conflict — a neighbour coloured the opposite is exactly the constraint being satisfied, not violated.

The first colour in each component is arbitrary — starting the other way round just swaps the two sides — so there is no searching or backtracking anywhere. Once one vertex is placed, everything reachable from it is forced.

The loop is over every vertex, because the graph is probably disconnected and it is bipartite only if every component is. A single triangle in a component nobody visited makes the whole answer wrong, and the sample input is almost always connected so nothing warns you.

The reason a conflict means impossible is worth stating: a graph is bipartite exactly when it has no odd-length cycle. Walking around a cycle alternates sides, so returning to where you started takes an even number of steps — an odd cycle cannot alternate, and a triangle is the smallest example.

$O(V + E)$ time and $O(V)$ space, and the colour array replaces the seen set rather than adding to it, so it costs exactly what a plain BFS costs.

I would write BFS rather than DFS here, because a path of a hundred thousand vertices is a legal input and would overflow the recursion stack.

And I would return the two groups rather than a boolean, because that is usually what the problem actually wants and the colour array already holds it. Do you want the split, or just whether one exists?”

The follow-ups

“Why does an odd cycle make it impossible? Prove it.”

“Both directions, and each is two sentences.

Bipartite implies no odd cycle. In a bipartite graph every edge crosses between the two sides, so walking along a path alternates: A, B, A, B. To return to the vertex you started from you must be back on its original side, which takes an even number of steps. So every cycle has even length.

No odd cycle implies bipartite. Colour each vertex by the parity of its distance from a chosen start. That is consistent as long as no two routes to the same vertex have different parities — and if they did, gluing those two routes together would form a cycle of odd length, which we assumed does not exist. So the colouring is well defined, and every edge joins vertices of different parity.

The algorithm is that second argument, executed. BFS assigns exactly the parity of the distance, and the conflict check is the moment it discovers two routes of the same parity — which is the odd cycle.

And I can produce the cycle itself if it helps: keep parent pointers, and on a conflict walk both endpoints back to their common ancestor. The two halves plus the offending edge form the odd cycle. **For a real problem that is much more useful than a boolean** — ‘these five people are the reason’ is actionable and ‘impossible’ is not.”

“The constraints arrive one at a time. After each, say whether a split still exists.”

“Then a traversal is the wrong tool — re-running it after each constraint is m traversals, so at a hundred thousand constraints on a hundred thousand people that is around forty billion steps.

The right structure is **Union-Find with parity**. Alongside each element I store the parity of its distance to its representative — 0 if it is on the same side as the root, 1 if the opposite. Then:

A constraint ‘a and b differ’ means: find both roots. **If the roots differ**, merge, adjusting the parity so that **a** and **b** end up opposite. **If the roots are the same**, check the parities — if they are already opposite, the constraint is consistent and there is nothing to do; if they are the same, this constraint contradicts the existing ones and no split exists from here on.

That is effectively constant per constraint, so a hundred thousand of them is four hundred thousand operations rather than forty billion.

It is the same idea as the traversal, just carried incrementally: parity relative to a root instead of parity relative to a start vertex. And it is genuinely online, which the traversal cannot be — which is the same static-versus-dynamic rule as [day 138](#)’s connected components.”

“Now split them into three groups.”

“That is a completely different problem and I would say so immediately rather than attempt to extend the algorithm.

Two-colouring is linear because there is never a choice. Once one vertex is placed, every neighbour is forced, and every neighbour of those, and so on — there is no branching, so a single traversal settles it.

With three colours there is a choice at every step. A neighbour of a red vertex can be green or blue, and the consequences of that choice can only be discovered much later. So the algorithm becomes a search with backtracking, and **3-colourability is NP-complete** — one of the classic ones. There is no known polynomial algorithm and it is not expected that there is one.

Practically: backtracking with constraint propagation solves realistic instances up to a few hundred vertices quickly, because good heuristics — colour the most constrained vertex first, order values to leave the most options — prune enormously. But the worst case is exponential.

The cliff between two and three is the interesting fact, and it shows up elsewhere: 2-SAT is linear and 3-SAT is NP-complete, for essentially the same reason — with two options each constraint forces, and with three it merely restricts.

If the problem genuinely needs three groups and n is large, I would look for extra structure — planar graphs are always four-colourable and there are efficient algorithms; interval graphs colour greedily — or ask whether an approximate answer is acceptable.”

“What if the two groups must be the same size?”

“Then it is a different and much harder problem, and I would separate the two requirements explicitly.

Bipartiteness says a valid split into two sides exists. It says nothing about their sizes — a star with one centre and nine leaves is bipartite with sides of 1 and 9, and no rearrangement balances it, because the colouring is forced.

Where there is freedom is between components. Each connected component has exactly two valid colourings — its own sides, and the same sides swapped — so with k components there are 2^k ways to assign them, and the question becomes: can I choose an orientation for each component so that the totals balance?

That is a subset-sum problem. Each component contributes a difference of $+d$ or $-d$ where d is the gap between its two sides, and I need those to sum to zero. Subset sum is NP-hard in general but has a pseudo-polynomial dynamic-programming solution in $O(k \times \text{total})$, which for realistic sizes is fine — and it is [day 149](#).

So the answer is: check bipartiteness first, $O(V + E)$; then, if balance is required, solve a subset-sum over the component differences. **Two separate problems, and noticing that the second one exists is the whole point of the question.**“

The model answer

“You have n people and a list of pairs who dislike each other. Split everyone into two groups so that no group contains a disliking pair. Return the groups, or say it is impossible and explain why.”

“This is a bipartite check, and the ‘explain why’ at the end changes what I build, so let me take that seriously rather than returning a boolean.

The model. A vertex is a person. An edge joins two people who dislike each other, **undirected** — the relation is symmetric, so I add both directions when building. That is one line and forgetting it gives a confident wrong `True`, which for a ‘can this be done’ question is the worst way to be wrong.

The algorithm is BFS two-colouring. Colour a start vertex 0, colour every neighbour the opposite, and report failure when a neighbour already carries the *same* colour. I would say precisely that, because checking ‘already coloured’ rather than ‘coloured the same’ fails on every even cycle — a four-person ring is perfectly splitable.

The loop runs over every person, because a dislike list produces a very disconnected graph — most people have no constraints at all and form singleton components. A contradiction hiding in a component nobody visited is the realistic failure here, not a theoretical one.

BFS rather than DFS, because the constraint chains can be long and I do not want a recursion limit deciding whether my answer is correct.

For the explanation, I keep parent pointers and extract the odd cycle on failure. Then the answer is not ‘impossible’ but ‘these five people form a ring of mutual dislikes, and a ring of odd length cannot be split into two’ — which is something a person can act on, by adding a third group or by resolving one specific pair. **A boolean would be technically correct and useless**, and I think that is what the question is testing.

Cost: $O(V + E)$ time, $O(V)$ space, with `V` people and `E` dislike pairs. For a thousand people and ten thousand pairs that is instant. I would use lists indexed by person rather than dictionaries, since people are numbered.

Two things I would clarify before coding. Are people numbered from 0 or from 1 — LeetCode’s version of this uses 1-based, which is an off-by-one waiting to happen. And do the groups need to be the same size? **If they do, that is a genuinely different problem:** bipartiteness gives me a valid split but says nothing about balance, and choosing which way round to orient each independent component to balance the totals is a subset-sum problem. I would want to know that before promising anything.

And if it turns out the constraints arrive over time — people added to the list as the event is planned — I would switch to Union-Find with parity rather than re-running the traversal, because that is effectively constant per constraint against a full

traversal each time.”

9 Recall card

Bipartite = split into two groups with every edge crossing between them. Two-colour with BFS: colour the start 0, neighbours get $1 - \text{colour}$, and **the conflict is a neighbour with the same colour** — already coloured is not a conflict.

A graph is bipartite $\Leftrightarrow$ it has no odd-length cycle. Walking a cycle alternates sides, so returning takes an even number of steps. A triangle is the smallest counter-example; **every tree is bipartite.**

Loop over every vertex — the graph is bipartite only if *every* component is, and a triangle in an unvisited component is the realistic bug. Build the graph in **both directions**; the relation is symmetric.

$O(V + E)$ time and $O(V)$ space — the colour array replaces the seen set, so it costs exactly what a plain BFS costs. Return the two sides, not a boolean, and keep parents so you can show the odd cycle.

Constraints arriving over time $\rightarrow$ Union-Find with parity (effectively constant, versus a traversal each time). **Three colours is NP-complete** — the cliff between 2 and 3 is real, and equal-sized groups is a separate subset-sum problem.

1 What this is, and why they ask it

The web is built on the browser asking and the server answering. That works for everything except the case where **the server knows something first** — a new message, a price change, a delivery moving, another player's move — and needs to tell a browser that did not ask.

There are four ways to bridge that gap, and they are not four equivalent options. **Polling** asks repeatedly. **Long polling** asks once and the server holds the request open until it has something. **Server-sent events** keeps one connection open and streams updates down it. **WebSockets** upgrade the connection to a genuine two-way channel.

They ask this because “how does the browser find out about a new message?” is a question in almost every chat, feed, dashboard or collaborative design, and because the tempting answer — WebSockets, because it is the most capable — is often the wrong one. **The correct answer usually depends on whether the client needs to send anything back**, and on what holding a hundred thousand open connections does to your architecture.

The scaling half is what separates a good answer: an open connection is server state, and server state means your stateless fleet is no longer stateless.

By the end of this lesson you can pick between the four from the requirement, quantify what polling costs at scale, describe what changes in the architecture when connections are persistent, handle reconnection and missed messages, and say why a chat system and a live dashboard get different answers.

2 The story

The tap in the bathroom has been dripping since Sunday and the plumber said he would come on Tuesday.

By eleven o'clock on Tuesday Anjali has rung the shop three times.

The first time the boy said he had gone to Kandivali and would come after. The second time, forty minutes later, the same boy said the same thing in a slightly shorter way. The third time he said “he has not come back yet, madam” before she had finished saying who she was.

This is the whole morning. She is working from home and she cannot go out, because if he arrives and the door is not answered he will not come back until Thursday. So every twenty minutes or so she stops what she is doing and rings, and every time the answer is no, and every one of those calls costs her about three minutes of concentration and costs the boy at the shop his patience.

At half past twelve she tries something different. She rings and says: do not put the phone down. I will wait. Tell me the moment he walks in.

The boy is not unwilling but it does not really work. He puts the receiver down on the counter and goes back to what he was doing, and she can hear the shop but she cannot hear anything useful, and after eleven minutes somebody else needs to make a call and he has to hang up. She rings again and they start over.

At twenty past one his father takes the phone, and being a practical man he says the obvious thing, which nobody had said all morning.

“Give me your number. I will call you when he leaves.”

And that is the end of it. She goes back to work and forgets about the tap. At four minutes past three the phone rings, the father says “he is coming now, ten minutes”, and she puts the kettle on.

The thing Anjali noticed afterwards, and mentioned to her husband, is that she had rung nine times and every single one of those calls was wasted except the last one, which she did not even make. **Nine calls to find out nothing, and one call in the other direction to find out everything.**

The thing the boy at the shop noticed is different and he mentioned it too. When four different people are all ringing every twenty minutes, he does nothing else all day.

3 The idea in plain English

Anjali's morning is the four mechanisms, in the order she tried them.

Polling is ringing every twenty minutes. The client asks on a timer. Simple, works everywhere, needs nothing special on either side.

And almost every request is wasted. Nine calls, one useful answer. That is the entire cost of polling, and it scales badly in a specific way: **the load is set by the number of clients and the interval, not by how often anything actually happens.** Ten thousand users polling every five seconds is two thousand requests a second whether or not there is any news at all.

It also decides your latency in advance. Poll every five seconds and the average delay before a user sees something is two and a half seconds. **Halving the delay doubles the load**, always, and that trade never gets better.

Long polling is “do not put the phone down”. The client asks, and the server **holds the request open** — does not answer — until it has something to say or until a timeout. Then the client immediately asks again.

It gives near-instant delivery with no wasted requests, which is real. What it costs is exactly what Anjali found: **the line is tied up**. Every waiting client occupies a connection and, on a traditional thread-per-request server, a thread. And the connection gets dropped by something in the middle — a proxy, a load balancer, a mobile network — after some number of seconds, which is why the timeout exists and why the client must re-establish constantly.

Server-sent events is the father’s answer, one way. One connection, opened once, and the server pushes messages down it as they happen. It is ordinary HTTP with a `text/event-stream` response that never ends. The browser’s `EventSource` handles it, **including automatic reconnection**, which is a genuine advantage.

It is one-directional: server to client only. If the client needs to send something, it makes an ordinary request, which is fine — that is what browsers do anyway.

WebSockets is a phone line left open in both directions. The connection starts as HTTP and is **upgraded** to a persistent two-way channel over which either side can send at any time, with much lower per-message overhead than HTTP.

The choice is mostly about direction and frequency, and the honest rule is:

- **Does the client need to send frequently, with low latency?** → WebSockets. Chat, collaborative editing, multiplayer games.
- **Only the server sends?** → Server-sent events. Live scores, notifications, dashboards, progress bars, streamed AI responses.
- **Updates are rare and latency does not matter much?** → Polling. Simplest thing that works, and often correct.
- **Need push and cannot use the others?** → Long polling as a fallback.

Now the half that decides whether the design is any good: an open connection is server state.

Your API servers were stateless — any request could go to any machine, and a machine dying cost one request. **With persistent connections, a specific user is attached to a specific machine.** That changes four things at once:

- **A deploy disconnects everybody.** Restarting a server drops every connection it holds, and they all reconnect at once — a [thundering herd](#), from your own deploy.
- **Sending a message means finding the right machine.** User B's connection is on server 7 and the message arrived at server 3. Server 3 has to get it to server 7, which needs a shared channel between servers.
- **Load balancing becomes uneven**, because connections are long-lived: a machine that came up early accumulates connections and never sheds them.
- **The connection count, not the request rate, becomes the capacity limit.**

The standard answer to the second point is a **pub-sub backplane**. Every server subscribes to the channels for the users it holds; a message published anywhere reaches the server holding that user. That is [day 131](#)'s fan-out, applied inside your own fleet.

And the last idea, which is the one people forget: **connections drop, so messages get missed**. A user on a train loses the connection for forty seconds. **Push alone is not delivery**. The design needs a sequence number or a cursor, so that on reconnect the client says "I last saw message 8814" and the server sends what it missed. Without that, a WebSocket is a beautifully engineered way to silently lose messages.

4 The picture

The four mechanisms on a timeline, with the same event happening at $t = 7$:

	t=0	2	4	6	8	10	
POLLING (every 2s)							
client:	?	?	?	?	?	?	
server:	no	no	no	no	YES	no	
					^		
							event at t=7, delivered at t=8
							5 wasted requests
LONG POLLING							
client:	?						
server:					YES		← held open until t=7
client:					?		← immediately re-asks
					^		
							delivered at t=7. Zero waste.
							But the connection was held for 7s.
SERVER-SENT EVENTS							
client:	open	=====					
server:					data: ...		
					^		
							one connection, held forever,
							server pushes when it likes
WEBSOCKET							
client:	open	<=====					
client:		→ typing...					
server:					← data		
							both directions, any time

What to notice. Long polling and SSE deliver at the same moment. The difference is that long polling re-establishes the connection after every message and SSE does not — so for a chatty stream, SSE is far cheaper, and for a rare event they are nearly identical.

The architecture change that persistent connections force:

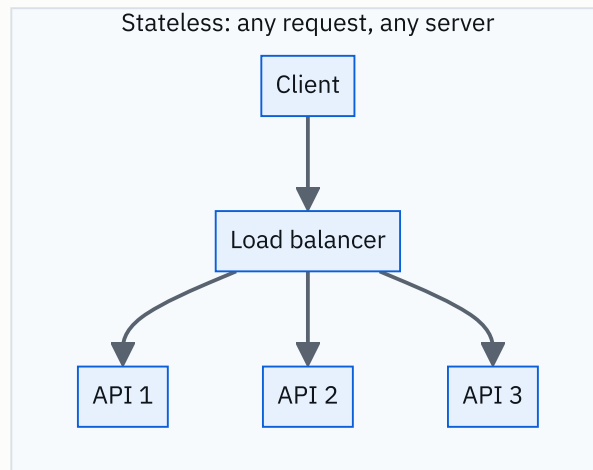


FIGURE 140.2

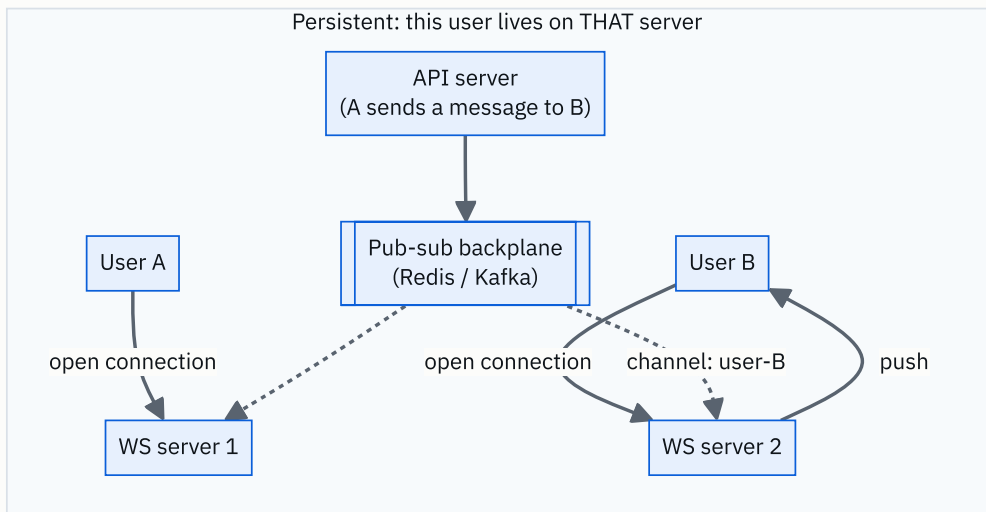


FIGURE 140.3

What to notice. The API server that received A’s message has no connection to B and no way to reach them directly. It publishes to a channel; the server that *does* hold B’s connection is subscribed and delivers. **Every server subscribes only to the users it holds**, which is what keeps the backplane traffic proportional to messages rather than to servers.

And the reconnection gap, which is the failure people design around last:

```
t=0    connected, received up to message 8814
t=12   train enters a tunnel. Connection drops.
       messages 8815, 8816, 8817 are published. The server pushes into a dead socket.
t=52   connection re-established

WITHOUT a cursor:  the client is now missing three messages, forever, silently.
WITH a cursor:     client reconnects saying "last seen 8814"
                   server replies with 8815-8817, then resumes live
```

5 How it actually works

Polling

```
setInterval(async () => {
  const res = await fetch(`/api/messages?since=${lastId}`);
  const messages = await res.json();
  if (messages.length) render(messages);
}, 5000);
```

Two things make polling much less bad than its reputation. Sending `since=lastId` means the response is usually empty rather than a full re-fetch. And an HTTP `304 Not Modified` with an `ETag` costs a few hundred bytes rather than a full body.

It is also the only mechanism that survives everything — every proxy, every corporate firewall, every ancient browser — and it is stateless, so it needs no architectural change at all. **For a notification badge checked every thirty seconds, it is the correct answer**, and saying so is better than reaching for WebSockets.

Long polling

```
async function poll() {
  try {
    const res = await fetch(`/api/messages?since=${lastId}`, { signal: timeout(30000) });
    const messages = await res.json();
    if (messages.length) { render(messages); lastId = last(messages).id; }
  } catch (e) { /* timeout or drop */ }
  poll();      // immediately ask again
}
```

Server side, the request is held until there is something or the timeout expires — typically 20 to 30 seconds, chosen to be shorter than whatever the intermediaries will tolerate.

On a thread-per-request server this is a disaster: ten thousand waiting clients is ten thousand blocked threads, and each is about a megabyte of stack. **On an async server it is fine** — a waiting request is a suspended coroutine costing a few kilobytes. That difference is why long polling was painful in 2010 and is merely unfashionable now.

The gap is the subtle bug. Between the server answering and the client re-asking there is a window of a few milliseconds. A message published in that window is missed unless the client sends `since=lastId` and the server serves from a buffer rather than only from live events. `since` is not an optimisation here, it is correctness.

Server-sent events

```
const es = new EventSource("/api/stream");
es.onmessage = (e) => render(JSON.parse(e.data));
```

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
Cache-Control: no-cache

id: 8815
data: {"from":"anjali","text":"he is coming"}

: heartbeat
```

Four things it gives you free, and they are why it is underused:

- **Automatic reconnection.** `EventSource` retries with a delay the server can set via a `retry` field.
- **The `Last-Event-ID` header.** On reconnect the browser automatically sends the id of the last event it received, so **the resume-from-cursor mechanism is built into the protocol** rather than something you invent.
- **Plain HTTP.** Proxies, compression, authentication cookies and HTTP/2 multiplexing all work unchanged.
- **Text framing** with no library needed on either side.

Its limits: one direction only; text only, so binary must be encoded; and over HTTP/1.1 it consumes one of the browser's six connections per domain, which HTTP/2 fixes by multiplexing.

The colon-prefixed heartbeat line matters more than it looks. Proxies close idle connections, so an otherwise-silent stream must emit a comment every 15–30 seconds to stay alive.

WebSockets

```
const ws = new WebSocket("wss://example.com/socket");
ws.onmessage = (e) => render(JSON.parse(e.data));
ws.send(JSON.stringify({ type: "typing" }));
```

The handshake is an HTTP request with `Upgrade: websocket`; after the `101 Switching Protocols` response the connection is a bidirectional frame-based channel.

Per-message overhead is 2 to 14 bytes, against several hundred for an HTTP request with headers and cookies. **That is the real argument for WebSockets on a chatty channel** — not “it is real-time”, but that a typing indicator costing 6 bytes instead of 700 changes what is affordable.

What you take on: heartbeats (ping/pong) to detect dead connections, because TCP will happily keep a dead connection open for minutes; reconnection with backoff and jitter, written by you; message ordering and delivery guarantees, also yours; and `wss://` in production, without which corporate proxies mangle the upgrade.

The scaling shape

Sticky sessions or a connection-aware router. The load balancer must send a reconnecting client’s traffic to a server that can serve it — or, better, any server, with a backplane doing the routing.

The pub-sub backplane is the standard pattern. Redis pub-sub is the common choice for chat because messages are ephemeral; Kafka when the stream must be durable and replayable.

```
message arrives at API server 3, for user B
  → PUBLISH channel:user-B {...}
  → WS server 7 is SUBSCRIBED to channel:user-B (it holds B's connection)
  → pushes down B's socket
```

Each server subscribes only to its own users’ channels, so backplane traffic scales with messages, not with `messages × servers`.

And the deploy problem needs a deliberate answer. Restarting a fleet of Web-Socket servers disconnects everyone at once, and they all reconnect within a second. The mitigations are: **jittered reconnect in the client**, shipped from day one because you cannot deploy a fix to disconnected clients; **draining** — stop accepting new connections, then close existing ones gradually over minutes; and a rolling restart that takes one server at a time.

Push notifications are a different thing

When the app is closed, none of this applies — there is no connection to hold. That is APNs and FCM, going through the operating system, and it is [tomorrow’s](#) lesson. **“Real-time while the app is open” and “notify when it is closed” are two separate systems**, and a design that conflates them is incomplete.

6 The numbers

What polling costs. 100,000 concurrent users, polling every 5 seconds:

100,000 / 5 s	= 20,000 requests/second
each request: headers + cookies	~800 bytes up, ~300 down
bandwidth	20,000 x 1.1 KB = 22 MB/s = 176 Mbps

if 1% of polls return anything:		
useful responses	200/s	
wasted requests	19,800/s	→ 99% waste

And halving the latency doubles all of it:

poll every 5 s	→ 20,000 req/s, average delay 2.5 s
poll every 1 s	→ 100,000 req/s, average delay 0.5 s
poll every 0.5 s	→ 200,000 req/s, average delay 0.25 s

With WebSockets:

100,000 open connections	
memory per connection (async server)	~10-50 KB
	100,000 x 30 KB = 3 GB
per message overhead	~6 bytes + payload
actual message rate	200/s
bandwidth	200 x 200 bytes = 40 KB/s

176 Mbps against 40 KB/s — a factor of about four thousand — because the traffic is now proportional to events rather than to clients times frequency.

Connections per server:

Node.js / Go / async Python, tuned	~50,000-100,000 connections per machine (file descriptors, memory, and the ephemeral port range are the limits)
100,000 users	2-4 machines for connections alone
1,000,000 users	20-40 machines

Compare that with a stateless API fleet, where a machine handles thousands of requests a second and there is no per-user cost at all. **Persistent connections make your capacity a function of *users online*, not of *requests***, and that is a different sizing conversation.

The backplane:

```
1,000,000 users, average 1 message received per user per minute
messages/s                               1,000,000 / 60 = 16,667
each published once to the backplane     16,667 publishes/s
Redis pub-sub capacity                    ~1,000,000 messages/s → comfortable
```

But if every server subscribed to everything:

```
16,667 messages/s x 40 servers           = 666,667 deliveries/s
                                           and every server filters out 97.5%
```

Per-user channels rather than a firehose is the difference between comfortable and impossible, and it is the design detail that matters most in the backplane.

The deploy herd:

```
40 servers x 25,000 connections          = 1,000,000 clients
rolling restart, all at once              1,000,000 reconnects in ~1 second
                                           → the replacement servers are overwhelmed
with 30 s of jitter                       ~33,000 reconnects/s → survivable
rolling one server at a time              25,000 at a time → invisible
```

Long polling on a threaded server:

```
10,000 waiting clients x 1 MB thread stack = 10 GB
                                           → not viable

on an async server: 10,000 suspended coroutines x ~10 KB = 100 MB
                                           → fine
```

Latency comparison:

polling, 5 s interval	0-5 s, average 2.5 s
long polling	~network round trip, ~50 ms
SSE	~50 ms
WebSocket	~50 ms, and much lower per-message cost

All three push mechanisms deliver at essentially the same speed. The choice is about direction, overhead and operational complexity — not about latency, which is a common misconception.

7 The trade-offs

Polling wastes requests and buys simplicity. Ninety-nine percent waste at a five-second interval, and in exchange: no server state, no connection management, no deploy problem, no reconnection logic, works through every proxy in existence, and any server

can serve any request. **For infrequent updates that is a good trade and reaching past it is over-engineering.**

Persistent connections make your fleet stateful, and that is the real cost. A user is bound to a machine. Deploys disconnect everyone. Load balancing goes uneven because connections never rebalance on their own. Capacity is measured in concurrent users rather than requests a second. **None of that is hard, and all of it is work you did not have before.**

WebSockets give you two directions and hand you everything else. Heartbeats, reconnection, backoff, ordering, delivery guarantees, message framing — all yours to write. SSE gives you reconnection and event ids for free and takes away one direction. **If the client does not need to send frequently, SSE is a much smaller amount of code for the same user experience,** and it is chronically under-chosen.

Push is not delivery. Every one of these mechanisms drops messages when the connection drops, and it drops constantly on mobile. **A cursor or sequence number is not optional** — without it, a message published during a forty-second tunnel is gone with no error anywhere. SSE has this built in as `Last-Event-ID` ; with WebSockets you build it.

The backplane is another system in the critical path. Redis pub-sub is fast and fire-and-forget, so a message published while a server is momentarily disconnected is lost — which is [day 131](#)'s warning, and it matters here because chat messages are exactly the thing you must not lose. **Store the message durably first, then publish the notification,** so the push is an optimisation and the store is the source of truth.

And more real-time than the product needs is a cost with no benefit. A dashboard refreshed every ten seconds is indistinguishable from one refreshed continuously, to a human looking at a number. **The question is not “can we push?” but “does anyone notice the difference?”**

When would I not use any of them? When updates are rare and the user can pull to refresh — most content apps. When the “real-time” requirement came from a diagram rather than a user. And when the app is closed, where none of this works and the answer is a platform push notification.

8 In the interview

How it gets asked

- “How does the browser find out about a new message?” — the direct version.
- “Design a live dashboard / notification system / chat.”
- “Why not just poll?” — the arithmetic question.

- “You have a million concurrent connections. What breaks?” — the scaling question.
- “A deploy disconnected every user at once. What now?”
- “WebSockets or server-sent events?”

The first ninety seconds

“Four options, and the choice comes down to two questions: does the client need to send, and how often does anything actually happen.

Polling is the client asking on a timer. Simple, stateless, works through every proxy — and at a five-second interval with a hundred thousand users that is twenty thousand requests a second, of which about ninety-nine percent return nothing. And the latency is fixed in advance: halving it doubles the load, forever.

Long polling holds the request open until there is something. Near-instant delivery with no waste, at the cost of an occupied connection per waiting client — fine on an async server, ruinous on a thread-per-request one.

Server-sent events is one HTTP connection held open with the server streaming down it. One direction only, and it gives me two things free that I would otherwise write: **automatic reconnection**, and the **Last-Event-ID** header, so resuming from where the client left off is built into the protocol.

WebSockets upgrade to a genuine two-way channel. Per-message overhead of a few bytes against several hundred for an HTTP request, which is what makes something like a typing indicator affordable.

My rule: if the client sends frequently, WebSockets. If only the server sends, SSE. If updates are rare, polling — and I would defend that rather than treat it as a failure.

The part I would flag before you ask is that persistent connections make the fleet stateful. A user is now bound to a specific machine, so sending them a message from a different machine needs a pub-sub backplane; a deploy disconnects everyone at once and they all reconnect together; load balancing goes uneven because long-lived connections never rebalance; and capacity becomes a function of users online rather than requests per second.

And push is not delivery. Connections drop constantly on mobile, so every design needs a cursor — ‘I last saw message 8814’ — or a forty-second tunnel silently loses three messages.

Does the client need to send, and how often do updates actually happen?”

The follow-ups

“Why not just poll? Give me the numbers.”

“Sometimes just poll — I would say that first, because for infrequent updates it is the right answer and everything else is over-engineering.

The arithmetic for when it stops working: a hundred thousand concurrent users at a five-second interval is twenty thousand requests a second. Each request carries headers and cookies — call it 800 bytes up and 300 down — so about 22 megabytes a second, 176 megabits, of traffic that mostly says ‘nothing new’. If one percent of polls return something, ninety-nine percent of that is waste.

And it does not improve as you tune it. The load is set by clients times frequency, not by how often anything happens. Poll every second instead and it is a hundred thousand requests a second. **The latency and the load are locked together and you can only trade one for the other.**

The same workload over WebSockets is a hundred thousand open connections — about three gigabytes of memory across a few machines — carrying two hundred actual messages a second at a few hundred bytes each. Forty kilobytes a second against twenty-two megabytes.

Where polling stays correct: a notification badge checked every thirty seconds, a status page, anything where a minute of delay is invisible. And I would make it a *conditional* poll — send the last-seen id or an `ETag`, so the usual response is empty or a 304 rather than a full payload. That alone often makes polling viable at ten times the scale people assume.”

“A million concurrent connections. What breaks first?”

“Not bandwidth — the messages are small. Four things, roughly in this order.

Memory per connection. Ten to fifty kilobytes each on a tuned async server, so a million connections is tens of gigabytes across the fleet. At fifty thousand connections per machine that is twenty machines doing nothing but holding sockets.

File descriptors and ephemeral ports. Both need raising from their defaults, and the port range limits outbound connections per source address — which matters for the backplane rather than for clients.

The backplane, if it is designed wrong. If every server subscribes to every message, sixteen thousand messages a second across forty servers is six hundred and sixty thousand deliveries a second, of which each server discards 97 percent. **Per-user or per-room channels fix that** and it is the single most important design decision in the backplane.

And deploys, which is the one that actually causes incidents. Restarting forty servers holding twenty-five thousand connections each means a million clients reconnecting within about a second. The replacement servers fall over, clients retry, and it becomes self-sustaining. **Three mitigations, all required:** jittered reconnect in the client — shipped from day one, because you cannot deploy a fix to disconnected clients; connection draining, so a server stops accepting new connections and closes existing ones over minutes; and a rolling restart, one machine at a time, so only twenty-five thousand reconnect at once.

And I would question the requirement. A million *concurrent* connections is a large product. If it is really a million registered users with fifty thousand online, the answer is much smaller.”

“WebSockets or server-sent events?”

“SSE unless the client needs to send frequently, and I think SSE is chronically under-chosen.

What SSE gives me free: automatic reconnection in the browser, with a retry interval the server can set; the `Last-Event-ID` header sent automatically on reconnect, so **resume-from-cursor is part of the protocol** rather than something I design; and plain HTTP, so proxies, compression, auth cookies and HTTP/2 multiplexing all work with no special handling.

What it costs: one direction, and text only. But the client can still send — with an ordinary HTTP request, which is what browsers are for. For notifications, live scores, a dashboard, a progress bar, or streaming a model’s output token by token, that is completely sufficient and it is a fraction of the code.

WebSockets earn their place when the client sends often and latency matters on that path too. Chat with typing indicators, collaborative editing, multiplayer games. The per-message overhead is a few bytes against several hundred for an HTTP request, so a typing indicator every keystroke is affordable over a socket and absurd over HTTP.

The hidden cost of WebSockets is everything the protocol does not give you: heartbeats, because TCP will hold a dead connection open for minutes; reconnection with backoff and jitter; message ordering; delivery guarantees; and framing. Every one of those is code you write and get wrong once.

One practical note: use `wss://` in production regardless. Plain `ws://` gets mangled by corporate proxies often enough that it is not worth debugging.”

“How do you make sure no message is lost?”

“By not relying on the push for delivery, and I would state that as the principle before the mechanism.

The message is written durably first — to the database — and the push is a notification that something changed. If the push is lost, the message still exists and the client will get it on its next fetch. If I invert that and treat the socket as the delivery mechanism, then a dropped connection is lost data.

Then the cursor. Every message has a monotonically increasing id per conversation. The client remembers the last id it rendered. On connect or reconnect it says ‘I last saw 8814’, and the server sends everything since, then switches to live push. **SSE gives me exactly this via `Last-Event-ID` ; with WebSockets I send it in the first frame after connecting.**

The subtle gap is between the catch-up and the live stream. A message published while the server is sending the backlog can be missed or duplicated. The clean way is: subscribe to the live stream **first**, buffer what arrives, then fetch the backlog, then replay the buffer, discarding anything at or below the last id delivered. Ordering by id makes deduplication trivial.

And the client must deduplicate anyway, because at-least-once is the only guarantee available — the client can receive a message, fail to render it, and reconnect asking for the same range. Rendering keyed by message id makes a duplicate a no-op.

The related failure I would name: Redis pub-sub is fire-and-forget, so a message published while a WebSocket server is briefly disconnected from Redis is gone. That is fine precisely *because* the message is already in the database and the cursor mechanism will catch it. **Fire-and-forget is acceptable for the notification and never for the data.**“

The model answer

“Design the real-time layer for a chat application: one-to-one and group chats, typing indicators, read receipts, presence, and it must work on mobile networks.”

“Chat is the case where WebSockets genuinely win, so let me say why rather than assume it, and then spend most of the time on the parts that are not the protocol choice.

WebSockets, because the client sends constantly. Typing indicators, read receipts and presence pings all go client-to-server, frequently, and at a few bytes of overhead per frame against several hundred for an HTTP request. A typing indicator every keystroke is affordable over a socket and absurd otherwise. **If this were notifications only, I would use SSE and write half as much code.**

The write path, and this is the most important decision: the message is persisted before it is pushed. Client sends over the socket; the server writes it to the message store with a monotonic per-conversation id; then it publishes to the backplane; then it acknowledges to the sender. **The socket is a delivery optimisation and the database is the source of truth.** If I invert that, a dropped connection is lost data rather than delayed data.

Fan-out via a pub-sub backplane, on per-conversation channels. The server holding the sender’s connection publishes to `conversation:4471`; every server holding a participant of that conversation is subscribed and pushes down the relevant sockets. **Per-conversation rather than a firehose**, or every server processes every message in the system and discards almost all of it.

Mobile is the requirement that shapes the rest, because connections drop constantly — tunnels, lifts, handovers between towers, and the operating system suspending the app.

So: a cursor, and it is not optional. The client stores the last message id it rendered per conversation. On every connect it sends that; the server replies with everything since, then switches to live. Subscribe first, buffer, fetch the backlog, replay the buffer discarding anything already delivered — otherwise there is a gap between the catch-up and the live stream. And the client deduplicates by message id, because at-least-once is the only guarantee on offer.

Heartbeats both ways, because TCP holds dead connections open for minutes and a phone that walked into a lift looks perfectly connected. Ping every thirty seconds, close after two misses — and the client uses those same misses to trigger reconnection with backoff **and jitter**, which must ship in the first release because I cannot deploy a fix to clients that are not connected.

Presence is the part I would deliberately make cheap. ‘Online’ is the existence of a connection plus a recent heartbeat, held in Redis with a TTL of about a minute, refreshed by the heartbeat. **I would not push presence changes for everyone to**

everyone — that is n^2 traffic in a large group and it is the classic way chat systems melt. Presence is fetched when a conversation is opened, and pushed only for the small set of conversations currently on screen.

Typing indicators are fire-and-forget and never persisted. They expire after a few seconds client-side, they are throttled to at most one per second per user, and losing one costs nothing. **Being explicit about which data is durable and which is ephemeral is most of what keeps this affordable.**

Deploys: rolling, one server at a time, with draining — stop accepting new connections, then close existing ones over a few minutes rather than all at once. Twenty-five thousand clients reconnecting is invisible; a million is an outage of your own making.

And when the app is closed, none of this works, which is the boundary I would draw explicitly. There is no socket to push down, so undelivered messages go to APNs or FCM as a platform push notification — a completely separate path, with its own delivery semantics, and the two systems have to agree on what has already been seen so the user is not notified twice for a message they read on the web.”

9 Recall card

Four mechanisms: polling (simple, stateless, ~99% waste, latency and load locked together), long polling (instant, holds a connection per waiting client), **SSE** (one HTTP stream, server→client only, and it gives you **automatic reconnect plus Last-Event-ID** free), **WebSockets** (two-way, ~6 bytes per message, and everything else is yours to write).

Choose on direction and frequency: client sends often → WebSockets; server only → SSE; rare updates → polling, and defend it.

Persistent connections make a stateless fleet stateful. A user is bound to a machine, so cross-server delivery needs a **pub-sub backplane on per-user or per-room channels** — a firehose makes every server discard 97% of everything.

Push is not delivery. Persist the message first and treat the push as a notification; every client carries a **cursor** so a reconnect resumes from the last id. Subscribe-then-backfill-then-replay closes the gap, and clients deduplicate by id.

Deploys are the incident. A million clients reconnecting in one second takes down the replacements — **jittered reconnect shipped from day one**, connection draining, and rolling one server at a time. And when the app is closed, none of this applies: that is a platform push notification.

DSA topic: Bipartite graphs and two-colouring **System design topic:** Websockets, long polling, and server-sent events

Code these, in this order

One rule for the whole set: the conflict test is `colour[neighbour] == colour[vertex]`, not `colour[neighbour] != -1`. Write that line first and read it back before filling anything else in. Checking “already coloured” fails on every even cycle, and a four-vertex ring is the smallest example.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Is Graph Bipartite?	LeetCode 785 (Medium)	The colour check, and the loop over every component.
2	Possible Bipartition	LeetCode 886 (Medium)	Building the graph in both directions from a dislike list.
3	Satisfiability of Equality Equations	LeetCode 990 (Medium)	The same idea via Union-Find, with <code>=</code> before <code>!=</code> .
4	Maximum Number of Edges to Keep Graph Fully Traversable	LeetCode 1579 (Hard)	Union-Find again — contrast with two-colouring.
5	Flower Planting With No Adjacent	LeetCode 1042 (Medium)	Four colours, greedy — and why two-colouring does not apply.

On problem 1, break the check on purpose

Write the version that returns `False` whenever a neighbour is already coloured. Run it on a four-vertex ring `0-1-2-3-0`. Record the answer and the correct one. One sentence on what “already coloured” fails to distinguish.

On problem 1, break the outer loop on purpose

Build a graph whose first component is a path and whose second is a triangle. Run a version that only colours from vertex 0. Record the answer.

On problem 2, break the graph build

Build the adjacency one-directionally and run it on a triangle `[[1,2],[1,3],[2,3]]`. Record the answer. Say which direction of wrongness that is — a false yes or a false no — and why that is the worse one here.

On problem 5, say why the phase's tools do not apply

Four colours, and the graph is guaranteed to have degree at most three. Write one sentence saying why greedy works here and why a two-colouring check would answer the wrong question. Then say what makes the general three-colour problem hard.

Then the odd-cycle extractor

Add parent pointers to your problem 1 solution and return the actual odd cycle on failure. Test it on a triangle, on a five-ring, and on a graph where the cycle is buried three levels deep and reached from two different branches — that last one is where the common-ancestor walk earns its keep.

Then the parity Union-Find

Solve problem 2 again with Union-Find plus parity: alongside each element, store the parity of its distance to its root. Then time both versions on 100,000 constraints delivered one at a time, reporting after each. Two numbers.

The check drill

1. Write the BFS version from memory.
2. State the conflict condition precisely, and say what it is not.
3. Say why the first colour in a component is arbitrary.
4. Say why the loop runs over every vertex.
5. Say why the colour array replaces the seen set rather than adding to it.

The odd-cycle drill

1. State the rule in one line.
2. Prove bipartite $\Rightarrow$ no odd cycle.
3. Prove no odd cycle $\Rightarrow$ bipartite.
4. Say which of those two the algorithm actually executes.
5. Give the smallest non-bipartite graph, and say why every tree is bipartite.

The variants drill

1. Return the two sides instead of a boolean.
2. Return the odd cycle, and describe the common-ancestor walk.
3. Describe the Union-Find-with-parity version and when you would use it.
4. Say what changes if the two sides must be equal in size.
5. Say what changes at three colours, and why.

The costs drill

1. Give time and space, and say why there is no overhead over a plain BFS.
2. Say what the worst case is and when early exit helps.
3. Compare traversal-per-constraint with parity Union-Find at 100,000 constraints.
4. Say why lists beat dictionaries here when vertices are numbered.

The break-it drill

Trigger each and record the exact output or error:

1. `colour[neighbour]  $\neq$  -1` as the conflict test, on a four-ring.
 2. Colouring only from vertex 0, with a triangle in another component.
 3. A one-directional graph build, on a triangle.
 4. Recursive DFS on a path of 100,000 vertices.
 5. Asserting the sides are equal in size, on a star.
 6. Two-colouring a graph where the problem allows three groups.
-

The four-mechanisms drill

1. Describe each of the four in one sentence.
2. Give the two questions that decide between them.
3. Say which two deliver at the same speed and why that surprises people.
4. Say what SSE gives you free that WebSockets do not.
5. Say what WebSockets give you that SSE cannot.

The polling drill

1. Compute requests per second for 100,000 users at a 5-second interval.
2. Compute the bandwidth, and the waste fraction at a 1% hit rate.
3. Say what happens to load when you halve the latency, and why that never improves.
4. Give two things that make polling much cheaper than its reputation.
5. Name three cases where polling is the correct answer.

The state drill

1. Say what changes when a fleet holds persistent connections — four things.
2. Explain why cross-server delivery needs a backplane.
3. Say why per-user channels beat a firehose, with the arithmetic.
4. Say why load balancing goes uneven with long-lived connections.
5. Say what capacity is now measured in.

The delivery drill

1. Say why push is not delivery.
2. Describe the cursor mechanism.
3. Say what SSE provides for this automatically.
4. Describe the gap between catch-up and live, and the fix.
5. Say why the client must deduplicate regardless.
6. Say why Redis pub-sub being fire-and-forget is acceptable here.

The deploy drill

1. Compute the reconnect rate for 40 servers $\times$ 25,000 connections restarted at once.
2. Recompute with 30 seconds of jitter and with a rolling restart.
3. Name the three mitigations and say which must ship first, and why.
4. Say what draining is.

The numbers drill

1. Compare polling bandwidth against WebSocket bandwidth for the same workload.
2. Give connections per machine and memory per connection.
3. Size the machines for 100,000 and for 1,000,000 concurrent users.
4. Compute backplane load with per-user channels and with a firehose.
5. Compute long polling's memory on a threaded server and on an async one.
6. Give the latency of all four mechanisms.

The failure drill

For each, say what happens and what you would build:

1. A user's phone enters a tunnel for forty seconds.
2. A deploy restarts every WebSocket server simultaneously.
3. A message is published to Redis while a server is briefly disconnected from it.
4. Every server subscribes to every message.
5. Long polling on a thread-per-request server with 10,000 waiting clients.
6. A WebSocket held open to a device that has been switched off.
7. The app is closed and a message arrives.

Two of the seven are not solvable by any of today's mechanisms. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Can these people be split into two teams with no conflicts?* Two-colouring with the precise conflict condition, the arbitrary first colour, the loop over every component, the odd-cycle rule with its proof, and returning the sides plus the offending cycle.
 2. *Why is it impossible?* The odd cycle, both directions of the proof, the five-ring made concrete — and the cliff at three colours if they push.
 3. *How does the browser find out about a new message?* The four mechanisms and the two questions that choose between them, the polling arithmetic, the statefulness that persistent connections force, the backplane, and “push is not delivery” with the cursor.
-

Before you move on

- ☐ ☐ My conflict test compares colours, not “is it coloured”.
- ☐ ☐ I can write the BFS two-colouring from memory.
- ☐ ☐ I know the first colour is arbitrary and why.
- ☐ ☐ I loop over every vertex and know what a hidden triangle costs.
- ☐ ☐ I build undirected graphs in both directions.
- ☐ ☐ I can state the odd-cycle rule and prove both directions.
- ☐ ☐ I know every tree is bipartite and why.
- ☐ ☐ I return the two sides, not a boolean.
- ☐ ☐ I can extract the odd cycle with parent pointers.
- ☐ ☐ I know the parity Union-Find version and when it wins.
- ☐ ☐ I know equal-sized groups is a separate, harder problem.
- ☐ ☐ I know three-colouring is NP-complete and why two is not.
- ☐ ☐ I can give the cost and say why there is no overhead over BFS.
- ☐ ☐ I can describe all four real-time mechanisms.
- ☐ ☐ I know the two questions that choose between them.
- ☐ ☐ I can compute polling’s request rate and waste fraction.
- ☐ ☐ I know halving polling latency doubles the load, always.
- ☐ ☐ I know what makes polling cheaper than its reputation.
- ☐ ☐ I can name three cases where polling is right.
- ☐ ☐ I know what SSE gives free and what it cannot do.
- ☐ ☐ I know what WebSockets hand back to me to implement.

- [] I can name the four things persistent connections change.
- [] I can explain the backplane and why per-user channels matter.
- [] I know push is not delivery, and can describe the cursor.
- [] I know the catch-up-to-live gap and the fix.
- [] I can compute the deploy reconnect herd and name three mitigations.
- [] I know jittered reconnect must ship in the first release, and why.
- [] I know the boundary where platform push notifications take over.
- [] I answered all three questions above out loud.

Multi-source BFS and 0-1 BFS

DATA STRUCTURES AND ALGORITHMS

Multi-source BFS and 0-1 BFS

SYSTEM DESIGN

Push notifications, end to end

1 What this is, and why they ask it

Two small variations on breadth-first search, and each one collapses a whole family of problems that look hard into problems that are not.

Multi-source BFS starts from many vertices at once instead of one. You push *all* the sources into the queue before the loop begins, and the search spreads outward from every one of them simultaneously. Every vertex ends up with its distance to the **nearest** source — with no comparison, no minimum, no repeated runs.

0-1 BFS handles the case where edges cost either 0 or 1. Plain BFS is wrong there and Dijkstra is more than you need; the answer is a deque, pushing zero-cost neighbours to the **front** and one-cost neighbours to the **back**. Still $O(V + E)$, still no heap.

They ask these because both are one-line changes to code you already have, and because both have a recognisable tell in the problem statement. “How long until every orange rots”, “distance from each cell to the nearest gate”, “the shortest path if you may break at most one wall” — those are three different-looking problems and two small variations.

The multi-source one in particular is the difference between an $O(k \times (V + E))$ solution and an $O(V + E)$ one, where k is the number of sources. **With 500 sources on a million-cell grid that is five hundred million steps against one million**, and candidates who do not see it write the slow version and cannot explain why it times out.

By the end of this lesson you can spot both from the phrasing, write each from memory, say precisely why multi-source costs the same as single-source, and explain why the deque keeps 0-1 BFS linear.

2 The story

The water in Sarita’s part of the city comes for ninety minutes in the morning and it has never been enough, so from March to June the society hires tankers.

Three of them, and where they park is Sarita’s decision because she is the secretary and because the first year nobody decided and it was a disaster.

There are about two hundred and ten flats spread over six buildings and a lane that goes round the back, and everybody has to walk to a tanker with whatever they can carry. So the question is simple to say and was hard the first time: given three tankers parked at three places, how far does each flat have to walk?

The way she did it in the first year was one tanker at a time. She walked the whole colony imagining the tanker at the front gate and worked out how far every flat was from it. Then she did the whole colony again for the second tanker. Then again for the third. Three complete rounds, and at the end she had three numbers for every flat and had to take the smallest.

It took her most of a Sunday and she got two of the numbers wrong.

The second year she did something different and it took twenty minutes.

She did not think about the tankers separately at all. She stood the three of them in her head at their three spots, and then asked: which flats are right next to *a* tanker — any of them? Those are at distance one. Then: which flats are next to *those*? Those are at distance two, and it does not matter which tanker they trace back to. Then the next ring, and the next.

Each flat is claimed by whichever tanker's ring reaches it first, and she never once compared three numbers, because a flat that has already been claimed is not looked at again.

The whole colony was covered in five rings.

The other thing that year taught her was about the back lane. There is a gate between the second building and the back lane, and it is usually open, so from the second building the back lane is essentially free — you walk through. When it is padlocked you have to go all the way round the front, which is a proper walk.

So the distances are not really all the same. Most steps between two adjacent points cost something and a few of them cost nothing at all, and if you work with the gate open you get quite a different set of numbers from the ones you get with it shut.

3 The idea in plain English

Sarita's second year is multi-source BFS, and the gate is 0-1 BFS.

Ordinary BFS starts from one vertex. Push the start, then repeatedly pop and push unseen neighbours, and every vertex ends up with its distance from that one start — [day 127](#), unchanged.

Multi-source BFS starts from all of them. Push **every** source into the queue before the loop begins, each at distance zero. Everything else is identical.

And each vertex ends up with the distance to its nearest source, with no comparison anywhere. That is the part worth understanding rather than memorising. BFS marks a vertex the first time it reaches it, and the first ring to reach a vertex is the one

from whichever source is nearest. **The minimum falls out of the order, exactly as Sarita's flats were claimed by whichever ring got there first.**

It costs the same as one source. Every vertex is still dequeued exactly once and every edge examined once, so it is $O(V + E)$ regardless of whether there is one source or five hundred. Running a separate BFS per source and taking the minimum is $O(k \times (V + E))$, and on a million-cell grid with 500 gates that is five hundred million steps against one million.

The tell in a problem statement is the word “nearest”, or “any of these”, or “how long until everything”. Rotting oranges — every rotten orange is a source. Walls and gates — every gate is a source. Distance from each cell to the nearest zero — every zero is a source. **They are all the same eight lines.**

Counting rounds rather than steps is the small extra move that these problems usually need. Process the queue **one whole level at a time** — take its current length, pop exactly that many — and each pass is one minute, one round, one step of the simulation.

And “what never gets reached” needs an explicit answer. A fresh orange walled off from every rotten one never rots. The problem wants -1 , not the time the reachable ones took, and the check is a counter examined after the loop, not something inside it.

Now the gate, which is the second idea.

BFS's guarantee depends on every edge costing the same — that is why the rings are rings. The moment an edge is free, a vertex two edges away can be closer than a vertex one edge away, and BFS's ordering is wrong.

Dijkstra fixes it and is more machinery than the problem needs. With only two distinct costs, a priority queue is doing work that a much simpler structure can do.

0-1 BFS uses a deque and one rule: a zero-cost edge pushes to the front, a one-cost edge pushes to the back.

Why that works is worth saying in one sentence: **the deque always holds at most two distinct distance values — d and $d + 1$ — with all the d s in front.** A zero-cost neighbour has the same distance as the vertex you are standing on, so it belongs with the current group at the front. A one-cost neighbour belongs with the next group at the back. **That is exactly the invariant a plain queue maintains for uniform weights,** and maintaining it by hand is what removes the need for a heap.

So it stays $O(V + E)$ — no $\log$ factor. Against Dijkstra's $O((V + E) \log V)$ that is a real saving, and more importantly it is much less code.

The tell for 0-1 BFS is a problem where a move is either free or costs one: “minimum walls to break”, “minimum sign flips”, “you may reverse at most some edges”, “cells of two types”. The moment there are three distinct costs the invariant breaks and it is Dijkstra.

One more thing about multi-source that is worth carrying: the sources do not have to be real vertices. A common trick is to add a **virtual source** connected to all the real sources with zero-cost edges, run ordinary single-source BFS from it, and get the same answer. That is the same reframing as [day 139](#)’s wells, and it is useful when a library or an existing function only accepts one start.

4 The picture

Sarita’s two approaches, on a small grid. T marks a tanker.

```
col:  0    1    2    3    4
row 0  T    .    .    .    T
row 1  .    .    .    .    .
row 2  .    .    T    .    .
row 3  .    .    .    .    .
```

ONE BFS PER TANKER, then take the minimum:

```
from (0,0):      from (0,4):      from (2,2):
0 1 2 3 4      4 3 2 1 0      4 3 2 3 4
1 2 3 4 5      5 4 3 2 1      3 2 1 2 3
2 3 4 5 6      6 5 4 3 2      2 1 0 1 2
3 4 5 6 7      7 6 5 4 3      3 2 1 2 3
```

then, cell by cell, min of the three:

```
0 1 2 1 0
1 2 1 2 1
2 1 0 1 2
3 2 1 2 3
```

cost: 3 full traversals + 20 comparisons

MULTI-SOURCE, all three pushed before the loop:

```
0 1 2 1 0
1 2 1 2 1
2 1 0 1 2
3 2 1 2 3
```

cost: ONE traversal. No comparisons at all.

What to notice. The answers are identical, and the second version never computed the three separate distances that the first version then had to compare. **The minimum was never taken; it fell out of the order in which the rings arrived.**

The rings, drawn:

```
ring 0:  the three tankers themselves
ring 1:  everything adjacent to any tanker
ring 2:  everything adjacent to ring 1 and not already claimed
...
```

0	1	2	1	0	← 0s are sources, 1s are ring 1
1	2	1	2	1	
2	1	0	1	2	
3	2	1	2	3	← ring 3, the furthest anything is from water

the largest number in the grid is how far the worst-off flat must walk.
For "how long until everything is covered", that IS the answer.

And the gate, which is 0-1 BFS:

```
A --0-- B --1-- D      A to D:  via B, cost 0 + 1 = 1
|                               |
+-----1-----+      A to D:  direct,   cost 1
```

plain BFS sees TWO edges via B and ONE direct
→ it reports the direct route as "closer"
→ both cost 1 here, but change the direct edge to cost 1
and the two-edge route to 0+0 and BFS is simply wrong

THE DEQUE, traced on: S --0-- A --1-- B, S --1-- C

deque	pop	relax	deque after
-----	---	-----	-----
[S(0)]	S	A via 0-edge → dist 0, FRONT	[A(0), C(1)]
		C via 1-edge → dist 1, BACK	
[A(0), C(1)]	A	B via 1-edge → dist 1, BACK	[C(1), B(1)]
[C(1), B(1)]	C	...	[B(1)]
[B(1)]	B	...	[]

at every moment the deque holds only 0s then 1s, or only 1s then 2s.
Two distinct values, in order. That is the whole invariant.

5 The code, built step by step

Multi-source first. Start from the ordinary version and change one thing.

```

from collections import deque

def distance_to_nearest(graph: dict[int, list[int]], sources: list[int]) → dict[int, int]:
    """Distance from every vertex to its nearest source.  $O(V + E)$ , any number of sources."""
    distance = {s: 0 for s in sources}          # ALL sources, distance zero
    queue = deque(sources)                      # ALL sources, before the loop
    while queue:
        vertex = queue.popleft()
        for neighbour in graph[vertex]:
            if neighbour not in distance:
                distance[neighbour] = distance[vertex] + 1
                queue.append(neighbour)
    return distance

```

Two lines differ from single-source BFS, and both are on the way in: the dictionary is seeded with every source rather than one, and the queue starts holding all of them.

Nothing inside the loop changed at all. No minimum, no comparison, no tracking of which source a vertex came from. The marking-on-first-arrival that BFS already does is what produces the nearest-source answer.

On a grid, which is where this appears most often:

```

def nearest_on_grid(grid: list[list[int]], source_value: int, passable: int) → list[list[int]]:
    rows, cols = len(grid), len(grid[0])
    dist = [[-1] * cols for _ in range(rows)]    # -1 means unreachable
    queue = deque()
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == source_value:
                dist[r][c] = 0
                queue.append((r, c))              # every source, before the loop
    while queue:
        r, c = queue.popleft()
        for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
            nr, nc = r + dr, c + dc
            if 0 ≤ nr < rows and 0 ≤ nc < cols and grid[nr][nc] == passable and dist[nr][nc] == -1:
                dist[nr][nc] = dist[r][c] + 1
                queue.append((nr, nc))
    return dist

```

The double loop that finds the sources runs **before** the BFS loop, not inside it. That placement is the entire algorithm.

Now the level-counting version, for “how many rounds”:

```
def rounds_until_done(grid: list[list[int]]) → int:
    """2 = rotten, 1 = fresh, 0 = empty. Minutes until nothing fresh remains, or -1."""
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                queue.append((r, c))
            elif grid[r][c] == 1:
                fresh += 1

    minutes = 0
    while queue and fresh:
        for _ in range(len(queue)):
            # exactly one whole level
            r, c = queue.popleft()
            for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
                nr, nc = r + dr, c + dc
                if 0 ≤ nr < rows and 0 ≤ nc < cols and grid[nr][nc] == 1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    queue.append((nr, nc))
            minutes += 1
    return -1 if fresh else minutes
```

Three details carry it. `for _ in range(len(queue))` freezes the level size before the loop begins, so newly pushed vertices are processed in the *next* round. `while queue and fresh` stops as soon as nothing fresh remains, so the count is not inflated by a final empty round. And `return -1 if fresh else minutes` is the unreachable check, after the loop.

Now 0-1 BFS.

```
def zero_one_bfs(graph: dict[int, list[tuple[int, int]]], n: int, source: int) → list[float]:
    """Shortest paths when every edge costs 0 or 1. O(V + E) – no heap."""
    INF = float("inf")
    distance = [INF] * n
    distance[source] = 0
    deq = deque([source])
    while deq:
        vertex = deq.popleft()
        for neighbour, weight in graph[vertex]:
            candidate = distance[vertex] + weight
            if candidate < distance[neighbour]:
                distance[neighbour] = candidate
                if weight == 0:
                    deq.appendleft(neighbour)
                    # same distance: the FRONT
                else:
                    deq.append(neighbour)
                    # one more: the BACK
    return distance
```

The `if weight == 0` is the whole algorithm. A zero-cost neighbour has the same distance as the vertex you are standing on, so it belongs at the front with the current group; a one-cost neighbour belongs at the back with the next group.

Note there is no `visited` set, and that is deliberate. A vertex can be improved after it has been popped — reached again by a cheaper route — so the guard is the relaxation `candidate < distance[neighbour]`, exactly as in Dijkstra. A vertex can enter the deque

more than once, and it is bounded because each entry requires a strict improvement.

The grid version, which is how it usually appears:

```
def min_walls_to_break(grid: list[list[int]]) → int:
    """0 = open, 1 = wall. Fewest walls to break walking from the top-left to the bottom-right."""
    rows, cols = len(grid), len(grid[0])
    INF = float("inf")
    dist = [[INF] * cols for _ in range(rows)]
    dist[0][0] = grid[0][0] # breaking into the start cell, if it is a wall
    deq = deque([(0, 0)])
    while deq:
        r, c = deq.popleft()
        for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
            nr, nc = r + dr, c + dc
            if not (0 ≤ nr < rows and 0 ≤ nc < cols):
                continue
            cost = grid[nr][nc] # 1 to enter a wall, 0 otherwise
            if dist[r][c] + cost < dist[nr][nc]:
                dist[nr][nc] = dist[r][c] + cost
                if cost == 0:
                    deq.appendleft((nr, nc))
                else:
                    deq.append((nr, nc))
    return dist[rows - 1][cols - 1]
```

The complete solution

```
"""Multi-source BFS and 0-1 BFS: two one-line variations on breadth-first search."""

from __future__ import annotations
from collections import deque

INF = float("inf")

def multi_source(graph: dict[int, list[int]], sources: list[int]) → dict[int, int]:
    """Distance to the NEAREST source. Costs the same as one source:  $O(V + E)$ ."""
    distance = {s: 0 for s in sources}
    queue = deque(sources)  # every source, before the loop
    while queue:
        vertex = queue.popleft()
        for neighbour in graph[vertex]:
            if neighbour not in distance:
                distance[neighbour] = distance[vertex] + 1
                queue.append(neighbour)
    return distance

def nearest_on_grid(grid: list[list[int]], source: int, passable: int) → list[list[int]]:
    """Grid version. -1 for anything the sources cannot reach."""
    rows, cols = len(grid), len(grid[0])
    dist = [[-1] * cols for _ in range(rows)]
    queue = deque()
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == source:
                dist[r][c] = 0
                queue.append((r, c))
    while queue:
        r, c = queue.popleft()
        for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
            nr, nc = r + dr, c + dc
            if 0 ≤ nr < rows and 0 ≤ nc < cols and grid[nr][nc] == passable and dist[nr][nc] == -1:
                dist[nr][nc] = dist[r][c] + 1
                queue.append((nr, nc))
    return dist

def rounds_until_done(grid: list[list[int]]) → int:
    """Level-by-level multi-source: 2 rotten, 1 fresh, 0 empty. Minutes, or -1."""
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                queue.append((r, c))
            elif grid[r][c] == 1:
                fresh += 1
    minutes = 0
    while queue and fresh:
        for _ in range(len(queue)):  # freeze the level size
            r, c = queue.popleft()
            for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
                nr, nc = r + dr, c + dc
                if 0 ≤ nr < rows and 0 ≤ nc < cols and grid[nr][nc] == 1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    queue.append((nr, nc))
        minutes += 1
    return -1 if fresh else minutes

def zero_one_bfs(graph: dict[int, list[tuple[int, int]]], n: int, source: int) → list[float]:
    """Weights of 0 or 1 only. Deque: 0-edges to the front, 1-edges to the back.  $O(V + E)$ ."""
    distance = [INF] * n
    distance[source] = 0
    deq = deque([source])
    while deq:
        vertex = deq.popleft()
```

```

        for neighbour, weight in graph[vertex]:
            candidate = distance[vertex] + weight
            if candidate < distance[neighbour]:
                distance[neighbour] = candidate
            (deq.appendleft if weight == 0 else deq.append)(neighbour)
    return distance

if __name__ == "__main__":
    # Sarita's colony: T = 2 (tanker), . = 0 (walkable)
    colony = [
        [2, 0, 0, 0, 2],
        [0, 0, 0, 0, 0],
        [0, 0, 2, 0, 0],
        [0, 0, 0, 0, 0],
    ]
    for row in nearest_on_grid(colony, source=2, passable=0):
        print(" ", row)
    print("worst walk:", max(max(r) for r in nearest_on_grid(colony, 2, 0)))
    print()

    oranges = [[2, 1, 1], [1, 1, 0], [0, 1, 1]]
    print("rot time :", rounds_until_done([row[:] for row in oranges]))
    sealed = [[2, 1, 1], [0, 1, 1], [1, 0, 1]]
    print("impossible:", rounds_until_done([row[:] for row in sealed]))
    print()

    # 0-1 BFS: S→A free, A→B costs 1, S→C costs 1, C→B free
    zero_one = {0: [(1, 0)], (2, 1)], 1: [(3, 1)], 2: [(3, 0)], 3: []}
    print("0-1 dists :", zero_one_bfs(zero_one, 4, 0))

```

Running it:

```

    [0, 1, 2, 1, 0]
    [1, 2, 1, 2, 1]
    [2, 1, 0, 1, 2]
    [3, 2, 1, 2, 3]
worst walk: 3

rot time : 4
impossible: -1

0-1 dists : [0, 0, 1, 1]

```

Three things to look at. The colony grid matches the hand-computed one in section 4 exactly, from **one** traversal — and `worst walk: 3` is the number Sarita actually wanted, which is just the maximum of the grid.

`impossible` returns `-1`: there is a fresh orange the rot cannot reach, and the answer is not “the time the reachable ones took”.

And the 0-1 result: vertex 1 is at distance **0** because the edge to it is free, and vertex 3 is at distance 1 — reachable either as `0→1→3` (`0 + 1`) or `0→2→3` (`1 + 0`). Plain BFS would have called vertex 1 “one step away” and got both of those wrong.

6 What it costs

Multi-source BFS.

every vertex is dequeued at most once	V
every edge is examined once from each end	$2E$

	$O(V + E)$ time
distance map + queue	$O(V)$ space

The number of sources does not appear. That is the whole point, and it is worth stating explicitly rather than letting it be inferred: k sources cost the same as one, because seeding the queue with k entries does not change how many times each vertex is dequeued.

Against the naive version:

k separate BFS runs, then a minimum per vertex	$O(k \times (V + E) + k \times V)$
--	------------------------------------

1,000 x 1,000 grid, 500 gates	
$V = 1,000,000$, $E \sim 2,000,000$	
multi-source	3,000,000 steps
one per gate	$500 \times 3,000,000 = 1,500,000,000$ steps
	$\rightarrow 500x$

Five hundred times, from moving one loop before another. On a judge that is `Time Limit Exceeded` against a comfortable pass.

The level-counting version costs the same — freezing `len(queue)` and looping does not visit anything twice:

sum over all levels of (level size) = V	$\rightarrow$ still $O(V + E)$
---	--------------------------------

0-1 BFS.

each vertex can enter the deque more than once, but only on a strict improvement, and distances only ever decrease through at most 2 distinct values	$O(V + E)$ time
distance array + deque	$O(V)$ space

Against Dijkstra on the same graph:

$V = 100,000$, $E = 300,000$	
0-1 BFS	400,000 steps
Dijkstra	$300,000 \times 17 = 5,100,000$ steps
	$\rightarrow \sim 13x$ more work, for the same answer

And the reason the `log` disappears is worth being able to say: Dijkstra's heap exists to find the smallest tentative distance among many different values. With only two distinct values in flight, "smallest" is just "whatever is at the front", and a deque maintains that in constant time.

The whole family, side by side:

BFS, one source	$O(V + E)$	
BFS, k sources	$O(V + E)$	-- k does not appear
k separate BFS runs	$O(k \times (V + E))$	-- what not to write
0-1 BFS	$O(V + E)$	
Dijkstra	$O((V + E) \log V)$	

Space, on a large grid:

1,000 x 1,000 grid	
distance as a list of lists of ints	~8 MB
queue at peak (one ring)	~1,000-4,000 entries with many sources

With many sources the first ring can be large — 500 gates means 500 entries before a single pop — but it is still bounded by `V`, and in practice a multi-source BFS has a *shallower* and *wider* frontier than a single-source one, which is if anything better for memory locality.

And the number that justifies the level-counting form:

```
"how many minutes" from a distance grid = max(dist)
"how many minutes" from level counting = the loop counter

both  $O(V + E)$ ; the level version avoids materialising the grid
when you only want the number
```

7 The traps

Running one BFS per source

The correct-but-fatal version:

```
best = [[INF] * cols for _ in range(rows)]
for gate in gates:
    d = bfs_from(grid, gate)
    for r in range(rows):
        for c in range(cols):
            best[r][c] = min(best[r][c], d[r][c])
```

```
500 gates on a 1,000 x 1,000 grid
500 x 1,000,000 = 500,000,000 cell visits
```

Time Limit Exceeded

The fix is to move the source-collection loop above the BFS loop. The tell in the statement is “nearest”, “any of these”, or “until everything is”.

Pushing sources inside the loop

```
queue = deque([sources[0]])
while queue:
    ...
    for s in sources:                # pushing the rest as you go
        queue.append(s)
```

This does not produce nearest-source distances — later sources enter the queue behind vertices that are already at distance 2 or 3, so their rings start at the wrong offset. **All sources must be at distance zero before anything is popped.**

Not freezing the level size

```
while queue:
    for _ in range(len(queue)):    # correct
        ...
```

```
while queue:
    while queue:                  # WRONG: consumes newly pushed vertices too
        ...
    minutes += 1
```

The inner `while` drains everything, including the vertices this round just pushed, so the whole search finishes in one “minute”. `len(queue)` **must be evaluated once, before the inner loop.**

Counting an extra round

```
while queue:                    # no `and fresh`
    for _ in range(len(queue)):
        ...
    minutes += 1
```

The last level pushes nothing, but the loop still runs once more for it and increments the counter:

```
>>> rounds_until_done_no_fresh_check([[2, 1]])
2                                # the answer is 1
```

`while queue and fresh` stops as soon as there is nothing left to spread to.

Forgetting what was never reached

```
return minutes
```

A fresh orange sealed off from every rotten one never rots, and the answer must be `-1`. The counter examined **after** the loop is the check, and it is the hidden test on every problem in this family.

0-1 BFS with a `visited` set

```
if neighbour not in visited:
    visited.add(neighbour)
    ...
```

Wrong. In 0-1 BFS a vertex can be reached again by a strictly cheaper route after it has been popped — that is the whole reason the distances are not uniform. The guard must be the relaxation `candidate < distance[neighbour]`, exactly as in Dijkstra. Adding a `visited` set silently locks in the first distance found and produces answers that are too large.

0-1 BFS with three distinct weights

```
graph = {0: [(1, 0), (2, 1), (3, 2)]}    # weights 0, 1 and 2
```

The deque invariant is that at most two distinct distance values are ever in flight. With a weight of 2, a neighbour belongs neither at the front nor at the back of the current group, the deque stops being sorted, and the answers are wrong with no error.

Two weights, and only 0 and 1. Otherwise it is Dijkstra — or, if the weights are small integers `0..k`, a **dial** of `k+1` buckets, which is the generalisation and is worth naming.

The mutated grid

```
rounds_until_done(grid)           # overwrites 1s with 2s
rounds_until_done(grid)           # second call sees an all-rotten grid
```

The rot version marks visited cells by changing them, which destroys the caller's input. Copy it, or use a separate distance grid, and **say which you are doing** — it is the same trade as [day 130's](#) islands.

8 In the interview

How it gets asked

- *“How long until every orange rots?”* — LeetCode 994, the canonical multi-source problem.
- *“Fill each empty room with the distance to the nearest gate.”* — LeetCode 286.
- *“01 Matrix: distance from each cell to the nearest 0.”* — LeetCode 542.
- *“What is the largest distance from any land cell to the nearest water?”* — LeetCode 1162, “As Far from Land as Possible”.
- *“Minimum number of walls to break to cross the grid.”* — 0-1 BFS.
- *“You may reverse at most some edges. Fewest reversals to reach the target?”* — 0-1 BFS on a directed graph.

The first ninety seconds

“This is a nearest-source question, so it is multi-source BFS, and the change from ordinary BFS is one line: **push every source into the queue before the loop starts**, each at distance zero.

The reason that works — and it is worth being precise, because it looks like it should need a minimum somewhere — is that BFS marks a vertex the first time it reaches it, and the first ring to reach a vertex comes from whichever source is nearest. **The minimum falls out of the ordering. There is no comparison anywhere in the code.**

And it costs the same as a single source. Every vertex is still dequeued exactly once and every edge examined once, so it is $O(V + E)$ whether there is one source or five hundred. The alternative — a separate BFS per source, then a minimum per cell — is $O(k \times (V + E))$, and with 500 gates on a million-cell grid that is five hundred million steps against three million. **That factor of k is the whole point of recognising the shape.**

If the question is ‘how many rounds’ rather than ‘what is each distance’, I process one whole level at a time — freeze `len(queue)` before the inner loop, pop exactly that many — and each pass is one minute.

Two things I would state before coding. The tell for this shape is ‘nearest’, or ‘any of these’, or ‘until everything is’. And whatever is never reached needs an explicit answer — a fresh orange sealed off never rots, and the expected result is `-1`, not the time the reachable ones took. That is a counter checked after the loop, and it is the hidden test on every problem in this family.

Are all the moves the same cost? Because if some are free and some are not, plain BFS is wrong and I would use a deque instead of a queue.”

The follow-ups

“Why does multi-source not need a minimum?”

“Because BFS’s ordering already computes it.

The invariant is that BFS pops vertices in non-decreasing order of distance and marks each one the first time it is reached. With several sources all seeded at distance zero, ring one is ‘everything adjacent to *any* source’, ring two is ‘everything adjacent to ring one and not already claimed’, and so on. **A vertex is claimed by whichever ring reaches it first, and that ring came from the nearest source.**

So the minimum is not computed, it is a consequence of the order. If I later reach the same vertex from a different source, that route is by definition no shorter — the ring structure guarantees it — and the already-marked check discards it.

The same argument tells me what it costs: each vertex is dequeued once and each edge examined once, and seeding `k` entries instead of one does not change either count. So `k` never appears in the complexity.

And if I need to know *which* source claimed each vertex — ‘which tanker should this flat walk to’ rather than ‘how far’ — that is one extra array carrying the source id along with the distance, and it is still one pass.”

“Why does plain BFS break when an edge is free, and why is Dijkstra overkill?”

“BFS’s correctness rests on every edge costing the same — that is why the rings are rings, and why the first arrival is the cheapest. With a free edge, a vertex two edges away can be genuinely closer than a vertex one edge away, so ‘fewest edges’ and ‘cheapest’ stop being the same question, and BFS optimises the wrong one.

Dijkstra fixes it and pays for generality it does not need here. Its heap exists to find the smallest tentative distance among many different values. With only two distinct edge costs, at any moment the frontier contains only two distinct distances — d and $d + 1$. So ‘the smallest’ is just ‘whatever is at the front’, if I keep the front sorted by hand.

That is the deque rule: a zero-cost edge pushes to the front, a one-cost edge pushes to the back. A zero-cost neighbour has the same distance as where I am standing, so it joins the current group; a one-cost neighbour joins the next group. The invariant — all the d s in front, then the $d+1$ s — is exactly what a plain queue maintains for uniform weights, maintained manually.

$O(V + E)$ with no log factor, against Dijkstra’s $O((V + E) \log V)$ — about thirteen times less work on a hundred-thousand-vertex graph, and considerably less code.

The one thing I would flag: there is no `visited` set. A vertex can be popped and then improved by a cheaper route, so the guard is the relaxation `new < distance[v]`, as in Dijkstra. A `visited` set locks in the first distance and gives answers that are too large, with no error.”

“The costs are 0, 1 and 2. Now what?”

“The deque invariant breaks, so 0-1 BFS is out — a weight-2 neighbour belongs neither at the front nor at the back of the current group, and the deque stops being sorted with no warning.

Two answers depending on the range.

If the weights are small integers $0..k$, use a dial — sometimes called dial’s algorithm or a bucket queue. Keep $k+1$ buckets indexed by $\text{distance} \bmod (k+1)$, process them in cyclic order, and each relaxation puts the neighbour in the bucket for its new distance. That is $O(V + E + k \times V)$ — still effectively linear for small k , and it is the natural generalisation of the deque, which is just the case $k = 1$.

If the weights are arbitrary, it is Dijkstra, and I would not try to be clever.

And there is a third option worth knowing when there are exactly two costs but they are not 0 and 1 — say 3 and 7. Subtracting the smaller from every edge does not preserve shortest paths, because paths with different edge counts shift by different amounts. So that transformation is wrong, and it is a tempting mistake. It is Dijkstra, or a dial if the range is small.”

“Find the land cell furthest from any water.”

“Multi-source BFS from all the water cells, and then the answer is the **maximum** of the resulting distance grid — which is one of the nicest examples of this shape, because the naive reading suggests something much harder.

The naive version is: for each land cell, find its nearest water. That is one BFS per land cell — $O(V^2)$ — and on a hundred-by-hundred grid it is a hundred million cell visits.

Inverting it is the whole trick. Instead of ‘for each land cell, find the nearest water’, do ‘from all water at once, spread outward’. Every land cell gets its distance to the nearest water in one pass, and the answer is the largest of them. $O(V + E)$.

Two edge cases the problem is actually testing. If there is no water at all, or no land at all, the answer is -1 — nothing to spread from, or nothing to reach. And the grid may be entirely one or the other, so both checks are needed before the traversal.

The general pattern is worth naming, because it recurs: **when a question is ‘for every X, find the nearest Y’, flip it and spread from all the Ys at once.** That turns k traversals into one, and it is the same move as flooding from the border on the surrounded-regions problem — start from the things you have few of.”

The model answer

“A city grid where some cells are hospitals and some are blocked. For every residential cell, find the walking distance to the nearest hospital. Then tell me where to put one more hospital to minimise the worst distance.”

“Two parts, and the first is a straightforward multi-source BFS while the second is where the interesting reasoning is. Let me do them in order.

Part one. Vertices are cells, edges are ‘adjacent and both passable’, and every hospital is a source. Push all of them into the queue at distance zero before the loop, then ordinary BFS. Every residential cell ends up with its distance to the nearest hospital, in one pass, $O(\text{rows} \times \text{cols})$.

I would say explicitly why there is no minimum in the code, because it is the part that looks wrong: BFS claims each cell on first arrival, and the first ring to arrive comes from the nearest hospital. The alternative — one BFS per hospital and then a per-cell minimum — is $O(h \times \text{rows} \times \text{cols})$, so with fifty hospitals on a thousand-by-thousand grid that is fifty billion cell visits against a million.

Blocked cells are simply not passable, so they are never enqueued, and a residential cell that no hospital can reach keeps its -1 . That case needs a decision from the product rather than from me — is an unreachable neighbourhood an error, or is it expected? — and I would ask rather than assume.

Part two is a different problem and I would say so. ‘Where to put one more to minimise the worst distance’ is a minimax question, and the brute-force reading is: for every candidate cell, add a hospital there and recompute. That is $\text{rows} \times \text{cols}$ multi-source BFS runs — a million runs of a million cells on a thousand-by-thousand grid, which is 10^{12} and impossible.

So I would narrow the search before optimising the search. Three observations, in order of how much they buy.

First, **only the currently-worst cells matter**. Adding a hospital can only reduce the maximum if it helps the cells that are currently at the maximum. So compute the distance grid once, find the worst distance D and the set of cells achieving it, and only consider placements that improve at least one of those.

Second, **binary search on the answer**. ‘Can the worst distance be brought down to d ?’ is a much easier question than ‘what is the best achievable?’. For a candidate d , every cell currently worse than d must be within d of the new hospital — so the new hospital must lie in the intersection of the d -radius neighbourhoods of all those cells. That intersection is computed with one more multi-source BFS, from the currently-too-far cells, and it is either empty or not. Binary searching d over $0..D$ is $\log D$ such checks, so about ten or twelve passes rather than a million.

Third, and I would offer this as the practical fallback: **if the grid is small, or if an approximate answer is acceptable, just try the top few hundred candidates** — cells near the centroid of the worst region — and take the best. That is a heuristic, it

is honest about being one, and for a real city-planning tool it is probably what ships.

The thing I would not do is pretend part two is the same problem as part one.

Part one is a linear-time classic. Part two is an optimisation over placements, and recognising that the naive version is 10^{12} before writing it is most of the answer.”

9 Recall card

Multi-source BFS: push every source into the queue before the loop, each at distance zero. One line, and every vertex gets its distance to the **nearest** source — the minimum falls out of the ring order, so there is no comparison anywhere.

It costs $O(V + E)$ regardless of the number of sources. One BFS per source is $O(k \times (V + E))$ — 500 gates on a million-cell grid is $500 \times$ more work. **The tell is “nearest”, “any of these”, or “until everything is”.**

For “how many rounds”, freeze `len(queue)` and process one whole level per pass — and use `while queue and fresh`, or the last empty round inflates the count. **Whatever is never reached needs an explicit `-1`, checked after the loop.**

0-1 BFS: a deque, zero-cost edges to the front and one-cost edges to the back. The frontier only ever holds two distinct distances, which is what a plain queue gives you for uniform weights — so it stays $O(V + E)$, about $13 \times$ cheaper than Dijkstra.

No `visited` set in 0-1 BFS — a vertex can be improved after being popped, so the guard is the relaxation. And with three or more distinct weights the invariant breaks: use a bucket dial for small integers, Dijkstra otherwise.

1 What this is, and why they ask it

Yesterday's mechanisms all require the app to be open and connected. A push notification is what happens when it is not — the phone is in a pocket, the app was killed three days ago, and a message still has to appear on the lock screen.

You cannot reach the device. Your servers have no route to a phone behind a mobile carrier's network, and even if they did, the operating system would not let a sleeping app receive anything. **The only channel is Apple's APNs or Google's FCM**, which maintain their own persistent connection to every device and are the only software permitted to wake an app.

So the whole design is: get a token, keep it fresh, hand your message to the platform, and accept that you have no idea whether it arrived.

They ask this because almost every consumer product needs it, because “how does a notification actually reach the device?” has a specific answer most people have never traced, and because the interesting parts are the ones nobody thinks about until production — **token invalidation, fan-out at scale, and the fact that delivery is not guaranteed and not reported.**

By the end of this lesson you can trace a notification from a database write to a lock screen, name every component, handle token lifecycle properly, size a fan-out to ten million devices, and say what you do about the delivery guarantee you do not have.

2 The story

Lakshmi's son is in the hostel at Manipal and she cannot ring his room, because there is no phone in the room and there never was.

What there is, is a number for the hostel office, and a man called Devaraj who sits at the desk from six in the morning until ten at night, and a board behind him with three hundred and eighty pegs on it.

The arrangement is that she rings the office, says the room number, and Devaraj decides what happens next.

Most of the time it is fine. She rings at half past seven, he sends the boy up, and her son comes down and rings her back from the desk phone. Perhaps ten minutes.

But the things she has learnt over two years are all about what Devaraj will and will not do, and none of them were explained to her at the start.

He will not knock after ten. Unless she says the word “urgent”, in which case he will, once, and if there is no answer that is the end of it. She used the word for a real emergency in the first month and he came straight up, and she has been careful with it since.

He does not tell her whether the message got through. She rings, she says the room number, he says “right”, and the call ends. Whether her son was in the room, whether he was asleep, whether the boy actually went up — none of that comes back to her. She finds out when he rings, or she does not find out.

And the room number goes stale. Her son changed rooms in the second year and did not think to tell her, and for eleven days she rang and gave the old number and Devaraj sent the boy to the old room, where a different boy said he did not know any Prakash. Nobody rang her to say the number was wrong. She only found out because she happened to mention the room number in a conversation and her son said that was not his room any more.

And he will not carry the same message four times. She rang three times in one afternoon once, worried about something, and the third time he said, quite kindly, that he had already sent the boy twice and would send him once more at six when the mess opened and everybody would be downstairs anyway.

Her sister, whose daughter is in a hostel with two hundred and forty girls, described a Sunday evening when thirty parents rang within about twenty minutes because the exam results had come out. The woman at that desk took every call and then sent one boy round the whole building with a list, floor by floor, which took seventy minutes and was the only way it could have been done.

3 The idea in plain English

Devaraj is APNs and FCM, and every one of Lakshmi’s four lessons is a property of push notification systems.

You cannot reach the device directly. A phone on a mobile network has no public address, moves between networks, and sleeps. **The operating system holds one persistent connection to Apple’s or Google’s servers** and everything for every app on that phone comes down that single connection — which is why battery life is tolerable at all, and why you cannot have your own.

So the flow is always three hops: your server → the platform (APNs or FCM) → the device's operating system → your app. **You only control the first hop.**

A device token is the room number. When your app first runs, it asks the operating system for permission and receives a **token** — an opaque string identifying this app on this device. The app sends it to your server, and your server stores it against the user. **That token is the only way anyone can address that device**, and every notification you send names it.

Tokens go stale, and nobody tells you. The user reinstalls, restores from a backup, updates the OS, or deletes the app. **The old token stops working and no message arrives to say so.** Lakshmi rang the old room for eleven days.

What you get instead is a rejection, and only when you send. APNs returns `410 Unregistered`; FCM returns `UNREGISTERED` or `INVALID_ARGUMENT`. **That response is the only notification-of-invalidation you will ever receive, and you must act on it by deleting the token** — because if you do not, you accumulate dead tokens forever and eventually most of what you send goes nowhere.

Delivery is not guaranteed and not reported. Devaraj says “right” and the call ends. APNs accepts your request and returns success, and that success means *accepted for delivery*, not delivered. The phone may be off, out of coverage for a week, or the user may have muted your app. **There is no acknowledgement from the device**, and any design that assumes one is wrong.

The consequence is the important one: a push is a nudge, not a delivery mechanism. The message itself lives in your database. The push says “there is something new”, and the app fetches the real content when it opens. **If the push is lost, the user still sees the message when they next open the app.** Invert that — put the only copy of the content in the notification — and a lost push is lost data.

Priority is the “urgent” word. Both platforms have a high and a normal priority. **High priority wakes the device immediately** and is for something the user genuinely wants now — a message, a call, a delivery arriving. **Normal priority is batched by the OS** and may be delayed to save battery. Marking everything high is abuse, and both platforms throttle apps that do it.

Collapsing is Devaraj refusing to carry the same message four times. Both platforms support a **collapse key**: a new notification with the same key replaces any undelivered one. Three unread-count updates while the phone was off become one. **Without it, a user who was offline for a day opens their phone to forty notifications**, which is how apps get their notifications turned off permanently.

And the exam-results Sunday is fan-out. One event, thirty parents — or ten million devices. The sending itself is a queue and a pool of workers, and the platform's own rate limits are usually not the constraint; **your ability to read ten million tokens and make ten million HTTP calls is.**

4 The picture

The path, end to end:

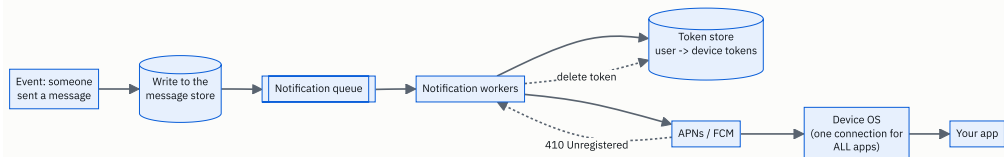


FIGURE 141.1

What to notice. The message is written to the store **before** anything is queued — the push is triggered by a durable fact, not the other way round. And the only feedback loop is the dotted one: a rejection telling you a token is dead. **There is no arrow from the device back to you saying “received”.**

The three hops, and who controls each:

YOUR SERVER	---(1)---	APNs / FCM	---(2)---	DEVICE OS	---(3)---	YOUR APP
(1) you control this.		HTTP/2 request with a token and a payload. You get a response: accepted, or a token error.				
(2) you control NOTHING.		Platform's persistent connection to the device. May be delayed, batched, or dropped entirely. No feedback of any kind.				
(3) the OS decides.		Shows it, or wakes your app, or does neither if the user muted you or is in Focus mode.				

What to notice. Two of the three hops are outside your system, and the two you do not control are the ones that decide whether anyone sees it.

The token lifecycle, which is where the bugs live:

```

app installed
|
v
ask permission ----> DENIED ----> no token, ever. Handle this.
|
v GRANTED
OS issues a token
|
v
app sends token to your server ---> store (user, token, platform, updated_at)
|
... user reinstalls / restores a backup / updates the OS
v
token silently becomes INVALID
|
v
you send          ---> APNs returns 410 Unregistered
|
v
DELETE the token. This is the ONLY signal you will get.

```

And the fan-out shape, for a broadcast:

```

one event → 10,000,000 devices

BAD:  one worker loops over 10 million tokens
      at 50 ms per HTTP call = 138 hours

GOOD: chunk the token list, push chunks onto a queue,
      N workers each hold a persistent HTTP/2 connection
      and pipeline many requests down it

      500 workers x 200 requests/s = 100,000/s
      10,000,000 / 100,000 = 100 seconds

```

5 How it actually works

APNs and FCM, concretely

APNs (Apple Push Notification service) speaks HTTP/2. You authenticate with a JWT signed by a key from your developer account, and you send one request per device token:

```

POST /3/device/<device-token>
authorization: bearer <jwt>
apns-topic: com.example.app
apns-priority: 10           # 10 = immediate, 5 = power-efficient
apns-push-type: alert       # alert | background | voip | ...
apns-collapse-id: chat-4471 # replaces an undelivered one with the same id
apns-expiration: 1767225600 # discard after this time

{"aps": {"alert": {"title": "Prakash", "body": "on my way"}, "badge": 3, "sound": "default"}}

```


The connection is the thing to get right. HTTP/2 lets you multiplex many requests over one connection, so you open a small number of persistent connections and pipeline down them. Opening a connection per notification is the classic mistake and it caps you at a fraction of the throughput.

FCM (Firebase Cloud Messaging) is Google's, and it also fronts Android's own transport. Its HTTP v1 API takes one message per request, with a `token` and separate `android`, `apns` and `webpush` blocks so one call can target several platforms.

FCM can also relay to APNs, which is why many teams use FCM as a single interface for both — one API, one integration, at the cost of an extra hop and a dependency on Google for iOS delivery.

The payload limits, which shape the design

APNs	4 KB	(5 KB for some types)
FCM	4 KB	

Four kilobytes is not much, and it is the technical reason the “push is a nudge” rule is not merely good practice. A message body, a title, an image URL, a deep link and some metadata fill it quickly. **Send the ids, not the content**, and let the app fetch.

Silent pushes

A **background** or **data-only** push wakes your app without showing anything, so it can fetch and update before the user opens it.

```
{"aps": {"content-available": 1}}
```

Both platforms treat these as a privilege and throttle them heavily. iOS in particular decides when — or whether — to deliver a background push based on battery, network, and how the user actually uses your app. **A silent push is a hint, not a scheduled task**, and any design that relies on one firing promptly will fail in the field.

Token management, which is most of the work

```
CREATE TABLE device_tokens (  
  user_id      BIGINT      NOT NULL,  
  token        TEXT        NOT NULL,  
  platform     TEXT        NOT NULL,          -- ios | android | web  
  app_version  TEXT,  
  created_at   TIMESTAMPTZ NOT NULL,  
  last_seen    TIMESTAMPTZ NOT NULL,          -- refreshed on every app open  
  PRIMARY KEY (token)  
);  
CREATE INDEX ON device_tokens (user_id);
```

Token, not user, is the primary key, because one user has several devices and a device can change hands.

Four rules that keep this healthy:

- **Refresh** `last_seen` **whenever the app opens** and re-register the token, because tokens change silently.
- **Delete on a** `410` **or** `UNREGISTERED` **response**. This is the only invalidation signal in existence.
- **Prune tokens unseen for, say, six months**. They are almost certainly dead and you are paying to send to them.
- **Handle the same token appearing for a different user** — a shared or resold device. The new registration wins, and the old association must be removed, or you will send one user's private messages to another person's phone. **That is the token bug that becomes a security incident**.

Fan-out at scale

For a broadcast to millions, the shape is a queue and a pool:

```
1. select token ranges (by id, not OFFSET) into chunks of ~1,000
2. push each chunk onto a queue
3. workers pull chunks, hold persistent HTTP/2 connections, pipeline requests
4. record failures; delete invalidated tokens
```

Chunk by primary-key range, never `LIMIT/OFFSET` — deep offsets over ten million rows get quadratically slower, and rows shifting under a paginated scan cause both duplicates and misses.

And the notification itself should be built once and shared, not rendered per device, unless it is personalised. Rendering ten million payloads when one would do is a surprising amount of CPU.

Deduplication and rate limiting the user

Per-user throttling is a product requirement, not a nicety. A user who receives forty notifications in an evening turns them off, and turning them off is permanent in practice.

```
per user:  at most N notifications per hour, per category
           collapse repeated updates of the same object
           respect quiet hours in the user's timezone
           batch low-priority ones into a digest
```

The collapse key does part of this at the platform — replacing an undelivered notification with a newer one of the same key — but only for undelivered ones. **Anything already delivered is delivered**, so the throttling has to happen on your side as well.

Web push

Browsers use the **Web Push protocol** with **VAPID** keys, via a service worker. Different API, same shape — a subscription endpoint instead of a device token, the same lack of delivery guarantee, and the same invalidation-on-410 rule. **Payloads are encrypted end-to-end**, so the push service cannot read them, which is a genuine difference from APNs and FCM.

What you can and cannot measure

sent	you know: you made the request
accepted	you know: the platform returned success
delivered	you DO NOT KNOW
opened	you know: IF the app reports it back to you

“**Delivery rate**” in a push dashboard is almost always “**acceptance rate**”, and being clear about that is worth saying. The only real signal is **opens**, reported by your own app when the notification is tapped — and that under-counts, because a user may read the content in the notification without opening anything.

6 The numbers

Fan-out to ten million devices.

tokens	10,000,000
per-request latency to APNs	~20-50 ms
requests per persistent connection	HTTP/2 multiplexing, many in flight
realistic per worker	~200 requests/second

one worker	$10,000,000 / 200 = 50,000 \text{ s} = 14 \text{ hours}$
100 workers	$500 \text{ s} = 8 \text{ minutes}$
500 workers	100 s

The constraint is your outbound capacity, not the platform’s limits. APNs has no published per-app rate limit for ordinary alerts; FCM’s quotas are generous. **What runs out is your workers, your connections, and your token-reading throughput.**

Reading the tokens is not free either:

10,000,000 rows at ~200 bytes	2 GB
scanned by primary-key range in 10,000 chunks of 1,000	
at ~5 ms per chunk query	50 seconds of database time

with LIMIT/OFFSET instead:
chunk 10,000 is OFFSET 9,999,000 → the database scans 10M rows to skip them
total work ~ $O(n^2 / \text{chunk})$ → hours

Token decay, which decides how much of your fan-out is wasted:

typical annual token churn 30-50%
reinstalls, device changes, uninstalls, OS restores

10,000,000 tokens, never pruned, after 2 years
live ~4,000,000
dead ~6,000,000
→ 60% of every send goes nowhere

with deletion on 410 and a 6-month unseen prune:
dead tokens <5%

Sixty percent waste against five percent is the entire argument for taking token hygiene seriously, and it is also a 60% reduction in fan-out time.

Payload sizes:

limit 4 KB
a full message body + metadata ~1-2 KB
ids and a short preview only ~300 bytes

10,000,000 x 1.5 KB = 15 GB of outbound per broadcast
10,000,000 x 0.3 KB = 3 GB

Engagement, which is the number the product actually cares about:

push opt-in rate, iOS ~40-60% (explicit permission)
push opt-in rate, Android higher historically; Android 13+ also asks
open rate, well-targeted ~5-10%
open rate, generic broadcast ~1-2%
uninstall / disable after over-sending measurable within weeks

Those numbers are why per-user throttling is an engineering requirement. Sending twice as much does not double engagement; it reduces the audience permanently.

Latency, end to end:

event → your queue ~10 ms
queue → worker picks it up ~100 ms - 2 s (depends on backlog)
worker → APNs accepted ~30 ms
APNs → device (high priority, awake) ~1-3 s
APNs → device (normal priority) seconds to minutes, OS-batched
APNs → device (off / no coverage) stored and retried, or DISCARDED at expiry

Expiry matters and is under-used:

```
apns-expiration: now + 3600
apns-expiration: 0
```

```
a chat message: pointless after an hour
deliver now or discard entirely
```

Without an expiry, a phone that was off for three days receives a stale flood on being switched on — which is the other half of the collapse-key problem.

7 The trade-offs

You have no delivery guarantee and no way to get one. The platform tells you it accepted the request and nothing else. **So the push can never be the delivery mechanism** — the content lives in your store and the push is a nudge. Every design that puts the only copy in the notification loses data the first time a phone is switched off.

You depend on two companies you cannot influence. APNs and FCM outages happen, their policies change, and their rate limits and throttling behaviour are theirs to adjust. There is no alternative channel: **this is a genuine, unavoidable single point of dependency** and the honest mitigation is a fallback to email or SMS for anything critical, not a clever architecture.

Priority is a shared resource you can abuse and lose. Marking everything high-priority works until the platform starts throttling your app, at which point your genuinely urgent notifications are delayed too. **The cost of over-using priority is paid later and by the notifications you care most about.**

Token storage grows and rots. Millions of rows, thirty to fifty percent churning annually, and no signal of invalidation except a rejection when you send. Without the delete-on-410 discipline and an unseen-prune, a majority of your fan-out eventually goes nowhere — and you pay for all of it in time and workers.

Fan-out to millions takes minutes, not seconds, and that is a product decision. Ten million devices at a hundred thousand a second is a hundred seconds. For a breaking-news alert that spread is acceptable; for “the auction closes now” it is not, and the answer is to prioritise — send to the users for whom the timing matters first — rather than to try to make the whole thing instant.

And the biggest cost is not technical. Over-notifying is the fastest way to lose a channel permanently: users disable notifications and do not re-enable them. **Every notification should have a reason a user would agree with**, and per-user throttling, collapsing, quiet hours and digests are engineering work in service of that.

When would I not use push? When the message is not time-sensitive — an in-app inbox the user checks is better and costs nothing. When it is critical and must be received, where push is the wrong channel entirely and SMS or email with a delivery

receipt is the honest answer. And when the app is open, where you already have a connection and yesterday's mechanisms deliver instantly and free.

8 In the interview

How it gets asked

- *"How does a push notification actually reach the device?"* — the direct version.
- *"Design a notification system."* — [day 152](#) is the full case study; this is the mechanism inside it.
- *"A user says they did not get a notification. How would you investigate?"*
- *"Send a notification to ten million users. How long does it take?"*
- *"How do you handle a user who reinstalls the app?"* — the token question.
- *"How do you stop notification fatigue?"*

The first ninety seconds

“The key fact is that **I cannot reach the device**. A phone on a mobile network has no address I can call, and even if it did, the operating system would not let a sleeping app receive anything. The OS holds one persistent connection to Apple’s APNs or Google’s FCM, shared by every app on the phone — which is why battery life is tolerable — and that connection is the only channel.

So there are three hops: my server to the platform, the platform to the device OS, the OS to my app. **I control the first one and nothing else.**

The addressing is a device token. On first run the app asks for permission and the OS issues an opaque token; the app sends it to me and I store it against the user, keyed by token because one user has several devices.

Then the flow: an event happens, I write the actual content to my database, and *then* I enqueue a notification job. Workers read the tokens for that user and make one HTTP/2 request per token, over persistent multiplexed connections.

Two things I would state up front, because they shape the whole design.

There is no delivery guarantee and no delivery report. The platform returns ‘accepted’, which means accepted for delivery, not delivered. The phone may be off for a week. **So the push is a nudge, not the message** — the content lives in my store, the notification says ‘something new’, and the app fetches on open. If I put the only copy in the payload, a phone that was switched off has lost it, and the 4 KB payload limit is a hint that this is not what it is for.

And tokens go stale silently. Reinstall, restore from backup, uninstall — the token dies and nobody tells me. **The only signal I will ever get is a 410 Unregistered when I try to send**, and I must delete on it. Without that discipline, thirty to fifty percent annual churn means a majority of my sends eventually go nowhere.

Is this per-user notifications, or a broadcast to everyone? Because the fan-out arithmetic is the interesting part of the second one.”

The follow-ups

“A user says they did not receive a notification. Walk me through the investigation.”

“Down the chain, and the useful thing to say first is that **I can only observe the first hop**, so most of the diagnosis is elimination.

Did the event happen and did we enqueue it? Check the message exists in the store and the notification job was created. This catches the majority of real cases and it is not really a push problem at all.

Did we have a token for that user? Very common: permission never granted, or the token was deleted after a 410 and never re-registered because the app has not been opened since. If there is no token, nothing was ever going to arrive.

Did we send, and what did the platform say? This is the last thing I can actually see. Accepted, or a token error, or a rate-limit response. If it was accepted, my system did everything it could.

After that, I am guessing, and I would say so. The phone may have been off, the user may have muted the app or the category, they may be in a Focus mode, the notification may have been collapsed by a newer one with the same key, or it may have expired before the device came back online. Normal-priority notifications are batched by the OS and can be delayed for a long time on a device that is idle.

What I would build to make this answerable: a per-notification audit record — event id, user, token, platform response, timestamp — retained for a few weeks. And an `opened` event reported by the app when the notification is tapped, which is the only real signal that anything arrived, while acknowledging it under-counts because people read the text without tapping.

The honest summary is that ‘delivered’ is not a state I can observe, and a dashboard that shows a delivery rate is showing an acceptance rate.”

“Send to ten million users. How long, and what breaks?”

“About a hundred seconds with five hundred workers, and the constraints are mine rather than the platform’s.

The arithmetic: each request to APNs is twenty to fifty milliseconds, but with HTTP/2 I multiplex many in flight over a persistent connection, so a worker realistically sustains around two hundred a second. Ten million over five hundred workers at two hundred a second is a hundred seconds. One worker would take fourteen hours.

What breaks first, in order.

Reading the tokens, if I paginate wrongly. Chunking with `LIMIT/OFFSET` means chunk ten thousand is `OFFSET 9,999,000`, and the database scans ten million rows to skip them — so the scan alone becomes quadratic and takes hours. **Chunk by primary-key range**, which is constant per chunk and also immune to rows shifting under the scan and causing duplicates or misses.

Connection handling. Opening a connection per notification instead of reusing persistent HTTP/2 connections caps throughput at a fraction and burns CPU on TLS handshakes.

Dead tokens. With thirty to fifty percent annual churn and no pruning, sixty percent of ten million requests after two years go to tokens that no longer exist — so I am spending sixty percent of my hundred seconds on nothing, and generating six million error responses to process.

And the thing I would raise as a product question: a hundred seconds of spread means some users get it a minute and a half after others. For breaking news that is fine. For ‘the auction closes now’ it is not, and the answer is to order the fan-out by who cares — active users, users in the relevant timezone — rather than to attempt to make ten million requests instantaneous.”

“How do you handle a user who reinstalls the app?”

“The old token becomes invalid silently and a new one is issued, and there are three separate things to get right.

Re-register on every app open, not only on install. Tokens can change without a reinstall — OS updates, backup restores — so the app should send its current token each time it starts, and I upsert it. That is cheap and it repairs most drift without any signal.

Delete on rejection. When APNs returns `410 Unregistered` or FCM returns `UNREGISTERED`, that token is permanently dead and I remove it immediately. **This is the only invalidation signal that exists**, and ignoring it is how a token table becomes mostly garbage.

Prune by `last_seen`. A token not seen for six months belongs to an app that has not been opened for six months, and sending to it is almost certainly wasted. I would prune rather than keep it forever.

And the case that is a security problem rather than an efficiency one: the same token appearing for a different user. That happens when a device is sold, or shared, or when someone logs out and someone else logs in. **If I key the table by user and let the old row survive, I will send one person’s private messages to another person’s phone.** So the token is the primary key, a new registration reassigns it, and logout deletes it — and that last one is the step people forget.

The related detail: on logout I should also tell the app to discard its local copy, because otherwise it may re-register the same token on the next launch under the previous session.”

“How do you stop notification fatigue?”

“It is the most important engineering constraint in the system and it does not look like one, because the failure is a permanent loss of the channel rather than an outage.

Collapse keys, at the platform. A new notification with the same collapse id replaces an undelivered one. Three unread-count updates while the phone was off become one. That handles the specific case of a device that was offline, which is where the worst floods come from.

Per-user, per-category rate limits, on my side. The collapse key only affects undelivered notifications; anything already on the lock screen stays. So I enforce a cap — at most so many per hour per category — with a counter per user, and anything over it is dropped or deferred into a digest.

Quiet hours in the user’s own timezone. Which requires storing the timezone, and defaulting it sensibly from the device rather than from the server’s clock.

Digests for anything low-value. ‘Six people liked your photo’ rather than six notifications. That is a batching job on a schedule rather than a per-event push, and it usually removes the majority of volume.

Expiry, which is under-used: a chat notification is pointless four hours later, so `apns-expiration` an hour out means a phone switched on tomorrow does not receive a flood of stale ones.

And the numbers that make the case: opt-in is roughly forty to sixty percent on iOS, a well-targeted notification gets maybe five to ten percent opens and a generic broadcast one to two, and disabling is effectively permanent. **Sending twice as much does not double engagement; it shrinks the audience forever.** That is the argument I would make to a product manager who wants a daily broadcast.”

The model answer

“Design push notifications for a messaging app: one-to-one and group chats, ten million users, and a notification must appear promptly when the app is closed.”

“Let me establish the principle first, because it determines the rest: **the push is a nudge and the message lives in my store.** The 4 KB payload limit and the absence of any delivery guarantee both point the same way, and inverting this — putting the only copy of the content in the notification — loses messages the first time a phone is switched off.

The write path. A message arrives over the WebSocket from yesterday’s lesson, is persisted with a monotonic per-conversation id, and is published to the real-time backplane for anyone currently connected. **Only then** is a notification job enqueued, for the recipients who are *not* currently connected — which is a cheap check against the presence store, and it avoids notifying someone who is looking at the message as it arrives.

The notification workers read the recipient’s tokens — token as primary key, so several devices per user — and send one HTTP/2 request each over persistent multiplexed connections.

Payload: sender name, a short preview, the conversation id, the message id, and the unread badge count. Ids, not content. The app fetches the conversation on open, which also means the notification cannot go stale in a way the user sees.

High priority, because a chat message is exactly the case high priority exists for — and precisely *because* I use it here, I would use normal priority for everything else in the product, so the platform does not start throttling the one thing that matters.

Collapse key per conversation. A phone that has been off for two hours receives one notification per conversation, not forty. `apns-collapse-id: conv-4471`, and the payload carries the unread count so the replaced notification still conveys ‘you have eleven messages here’. **This single setting is the difference between a usable product and one people mute.**

Expiry of about an hour. A chat notification from yesterday afternoon is noise, and without an expiry the platform stores and delivers it when the device reconnects.

Group chats are the fan-out case. A message in a group of two hundred is two hundred notification jobs. I would enqueue one job per recipient rather than one job that loops, so that a failure retries one recipient rather than the group, and so the work spreads across workers. For very large groups I would also apply a per-user rate limit, because a busy group is the single most common cause of someone disabling notifications for an entire app.

Token hygiene, because at ten million users it is a real cost. Re-register on every app open. Delete on `410` or `UNREGISTERED`. Prune tokens unseen for six months. And **delete the token on logout** — otherwise a sold or shared device

receives the previous owner's private messages, which is a security incident rather than a bug.

What I would monitor: notifications enqueued, accepted, and rejected by reason; token deletions per day, because a spike means an app release broke registration; the queue's oldest-message age, because a backlog here is directly visible to users; and open rate per category, which is the only real signal that anything arrives.

And the boundary I would draw explicitly: when the app is open, this path is not used at all — the WebSocket delivers, and sending a push as well produces a notification for a message the user is currently reading. That check against presence is small and it is the difference between a polished product and an irritating one.”

9 Recall card

You cannot reach the device. The OS holds one connection to APNs/FCM shared by every app, and that is the only channel. Three hops — your server → platform → OS → app — and **you control only the first**.

A push is a nudge, not the message. Content lives in your store; the payload carries ids and a preview (4 KB limit). There is **no delivery guarantee and no delivery report** — “accepted” means accepted, and `delivered` is not a state you can observe.

Tokens go stale silently. The **only** invalidation signal is a `410 Unregistered / UNREGISTERED` when you send — delete on it, re-register on every app open, prune by `last_seen`, and **delete on logout**, or a resold device gets someone else's messages.

Fan-out is your capacity, not the platform's: ~200 req/s per worker with persistent HTTP/2, so 10M devices is ~100 s at 500 workers. **Chunk tokens by key range, never LIMIT/OFFSET**. Untended tokens churn 30–50% a year, so most of a send eventually goes nowhere.

Fatigue is the real failure mode: collapse keys, per-user per-category caps, quiet hours, digests, and an expiry. Opt-in is ~40–60%, disabling is permanent, and **sending more shrinks the audience rather than growing engagement**.

DSA topic: Multi-source BFS and 0-1 BFS **System design topic:** Push notifications, end to end

Code these, in this order

One rule for the whole set: **the loop that finds the sources goes above the BFS loop, never inside it.** Type the source-collection loop, then a blank line, then `while queue:`. Every problem below is a one-line change from ordinary BFS, and the line is on the way in.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	01 Matrix	LeetCode 542 (Medium)	The plainest multi-source: every 0 is a source.
2	Rotting Oranges	LeetCode 994 (Medium)	Level counting, and the <code>-1</code> for what is never reached.
3	Walls and Gates	LeetCode 286 (Medium)	Many sources, and why one BFS per gate times out.
4	As Far from Land as Possible	LeetCode 1162 (Medium)	Inverting “for each X find the nearest Y”, then <code>max</code> .
5	Minimum Obstacle Removal to Reach Corner	LeetCode 2290 (Hard)	0-1 BFS with a deque.
6	Minimum Cost to Make at Least One Valid Path	LeetCode 1368 (Hard)	0-1 BFS again, with the cost on direction changes.

On problem 3, write the slow version and time it

Solve it once with one BFS per gate and a per-cell minimum, and once multi-source. Run both on a $1,000 \times 1,000$ grid with 500 gates. Record both times and both cell-visit counts. The ratio should be about 500.

On problem 2, break it three ways

1. Use `while queue:` for the inner loop instead of `for _ in range(len(queue))`. Record the answer.
2. Drop the `and fresh` from the outer condition. Run on `[[2, 1]]` and record the answer.
3. Drop the `-1` check. Run on a grid with a sealed-off fresh orange and record what comes back.

Three wrong answers, all plausible.

On problem 5, add a `visited` set on purpose

Solve it correctly, then add `if neighbour in visited: continue` to the 0-1 BFS. Find an input where the two disagree and record both answers. Write one sentence saying why a vertex may need to be processed twice here and never does in plain BFS.

On problem 6, count the deque operations

Instrument `appendleft` and `append`. Run on the largest input and record both counts. Then solve it with Dijkstra and count heap pushes and pops. Compare the totals and say where the `log` factor went.

Then the three-weights experiment

Take your 0-1 BFS and feed it a graph with weights 0, 1 and 2. Compare its answer against Dijkstra's on the same graph. Find an input where they differ, and write one sentence naming the invariant that broke.

The multi-source drill

1. Say what the one-line change from ordinary BFS is.
2. Say why no minimum appears anywhere in the code.
3. Say why the cost does not depend on the number of sources.
4. Give the three phrasings that signal this shape.
5. Describe the virtual-source alternative and when you would use it.

The level-counting drill

1. Write the level-by-level loop from memory.
2. Say why `len(queue)` must be frozen before the inner loop.
3. Say why the outer condition needs `and fresh`.
4. Say where the unreachable check goes and why not inside the loop.

The 0-1 drill

1. Say why plain BFS breaks when an edge is free.
2. State the deque rule in one line.
3. State the invariant the deque maintains, and why it is the same one a plain queue gives.
4. Say why there is no `visited` set and what the guard is instead.
5. Say what breaks with three distinct weights, and give the two alternatives.

The costs drill

1. Give multi-source BFS's cost and say what does not appear in it.
2. Compute multi-source against one-BFS-per-source for 500 gates on a million cells.
3. Give 0-1 BFS against Dijkstra at `V = 100,000, E = 300,000`.
4. Say where Dijkstra's `log` factor comes from and why it disappears here.
5. Give the whole family's costs in one list.

The break-it drill

Trigger each and record the exact output or error:

1. One BFS per source on a $1,000 \times 1,000$ grid with 500 sources.
 2. Sources pushed inside the loop rather than before it.
 3. `while queue:` as the inner level loop.
 4. The `and fresh` condition removed, on `[[2, 1]]`.
 5. The `-1` check removed, with a sealed-off cell.
 6. A `visited` set added to 0-1 BFS.
 7. 0-1 BFS on a graph with a weight of 2.
-

The three-hops drill

1. Name the three hops and say which you control.
2. Say why you cannot reach the device directly — two reasons.
3. Say why the OS holds one connection for all apps.
4. Name the two platforms and what FCM can additionally do.

The token drill

1. Say what a device token is and what it identifies.
2. Give the table's primary key and say why it is not the user.
3. Name the four rules that keep a token store healthy.
4. Say what the only invalidation signal is.
5. Give the token bug that is a security incident, and the step people forget.

The guarantee drill

1. Say what "accepted" means and what it does not.
2. Say why the push must be a nudge rather than the message.
3. Give the payload limit and say why it reinforces the same rule.
4. List what you can measure and what you cannot.

5. Say what a “delivery rate” dashboard is really showing.

The controls drill

1. Say what priority does, on both settings, and what abusing it costs.
2. Say what a collapse key does and exactly which notifications it affects.
3. Say what expiry is for and give a case where omitting it hurts.
4. Say what a silent push is and why you cannot rely on it.

The fan-out drill

1. Compute the time for 10M devices at 200 requests/s per worker, for 1, 100 and 500 workers.
2. Say what actually limits the throughput.
3. Say why you chunk by key range and what `LIMIT/OFFSET` costs.
4. Compute the effect of 40% dead tokens on the same fan-out.
5. Say what you would do about a hundred-second spread on a time-critical alert.

The fatigue drill

1. Name the five mechanisms for controlling notification volume.
2. Say which one the platform provides and exactly what it does not cover.
3. Give opt-in and open rates, and say why disabling is effectively permanent.
4. Give the sentence you would say to a product manager who wants a daily broadcast.

The numbers drill

1. Give annual token churn and the two-year effect without pruning.
2. Compute outbound bytes for 10M devices at 1.5 KB and at 0.3 KB.
3. Break end-to-end latency into its stages and name the variable one.
4. Give the database time for scanning 10M tokens by key range.

The failure drill

For each, say what happens and what you would build:

1. A user reinstalls the app and never opens it again.
2. A phone is switched off for three days with no expiry set.
3. A group of 200 people, one chatty conversation, no per-user cap.
4. A device is sold and the new owner logs in.
5. A silent push relied on for a scheduled refresh.
6. A notification sent while the app is open and connected.

7. Chunking 10M tokens with `LIMIT/OFFSET` .

Two of the seven are security or privacy problems rather than performance. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *How long until every orange rots?* Multi-source BFS with all sources pushed before the loop, why no minimum appears, the level-counting form, the `-1` case, and the arithmetic against one BFS per source.
 2. *You may break at most some walls. Fewest walls to cross the grid?* Why plain BFS is wrong, why Dijkstra is more than needed, the deque rule, the two-distinct-distances invariant, no `visited` set — and what changes at three weights.
 3. *How does a push notification actually reach the device?* The three hops and which you control, the token and its silent invalidation, “accepted is not delivered”, push-as-a-nudge with the 4 KB limit as evidence, and collapse keys plus per-user throttling.
-

Before you move on

- ☐ I put the source-collection loop above the BFS loop.
- ☐ I can say why multi-source needs no minimum.
- ☐ I know the cost does not depend on the number of sources.
- ☐ I can give the three phrasings that signal this shape.
- ☐ I can write the level-counting loop and know why `len(queue)` is frozen.
- ☐ I know why the outer condition needs `and fresh` .
- ☐ I always answer for what was never reached.
- ☐ I know the virtual-source alternative.
- ☐ I can say why plain BFS breaks on a free edge.
- ☐ I can state the deque rule and the invariant behind it.
- ☐ I know 0-1 BFS has no `visited` set, and what the guard is.
- ☐ I know what breaks at three weights, and both alternatives.
- ☐ I can compute multi-source against per-source with real numbers.
- ☐ I can compute 0-1 BFS against Dijkstra and say where the log went.
- ☐ I know the three hops and which one I control.
- ☐ I know why the device cannot be reached directly.

- [] I can describe the token lifecycle end to end.
- [] I know the token table is keyed by token, not user.
- [] I know the only invalidation signal, and to delete on it.
- [] I know the logout step and why omitting it is a security incident.
- [] I know “accepted” is not “delivered”, and what I cannot measure.
- [] I treat the push as a nudge and keep the content in my store.
- [] I know what priority does and what abusing it costs.
- [] I know exactly which notifications a collapse key affects.
- [] I know what expiry prevents.
- [] I can compute fan-out time and say what limits it.
- [] I chunk by key range, never `LIMIT/OFFSET`.
- [] I can quote token churn and its effect on a fan-out.
- [] I can name five ways to control notification volume.
- [] I answered all three questions above out loud.

Graphs revision and mock round

DATA STRUCTURES AND ALGORITHMS

Graphs revision and mock round

SYSTEM DESIGN

Geospatial indexing: geohash and quadrees

1 What this is, and why they ask it

Eighteen days ago you did not know what a vertex was. Today the phase closes, and the way to close it is not to reread it — it is to solve two problems you have not seen, out loud, with a clock running.

This lesson has two halves. The first compresses the phase into what it actually is: **one modelling question, one traversal, and about eight variations on it**. The second is a mock round — two unseen problems worked through as an interview, including the part where you are stuck and have to keep talking.

The reason a revision day exists is that graph interviews are not lost at the algorithm. They are lost in the first ninety seconds, when a candidate reads “given a list of accounts with email addresses” and does not see a graph. **Every algorithm in this phase is twenty lines you can write from memory. The gap is recognition**, and recognition is a different skill that has to be practised on its own.

By the end of today you can state the modelling question in two sentences, choose the right traversal from a decision table without hesitating, quote every cost, name the eight recurring bugs, and take two unseen problems from statement to working code while explaining yourself.

2 The story

Bhima has decorated for weddings, political rallies, a hospital’s opening, three school annual days and one funeral he would rather not discuss, and he will tell you they are all the same job.

People find that annoying when he says it, especially the families, because a wedding is not a rally. But he has been doing it since 1996 and what he means is quite specific.

Whatever the event, he stands in the empty space on the first visit and answers the same six questions, in the same order, before he opens his mouth about colours or flowers or anything anybody actually wants to talk about.

Where do people come in, and where does that push them once they are through the gate. Where is the thing everyone will look at, and can it be seen from the back. Where does the food go, because that is where the crowd will actually stand regardless of where

you want them. Where do the cables run, and can somebody trip over them. Where do the people who are working move, and does that cross where the guests move. And what happens if it rains.

Six answers, and after that the job is mostly arranging.

The families think their event is unique. And in the details it is — a rally needs the stage high and a wedding needs it low, a school function has four hundred children who will not stay where you put them. But the six questions do not change, and Bhima has never once found an event where they did.

The young man who worked with him for two years never got this. He was better than Bhima with his hands and he had a much better eye for colour. What he did every single time was start with the decoration. He would work out something beautiful for the entrance, and then discover that the entrance was where the catering van had to reverse in, and everything had to be moved on the morning.

Bhima said the same sentence to him for two years and it never took. **The job is not the decorating. The job is the six questions, and then the decorating is easy.**

3 The idea in plain English

Bhima’s six questions are what this phase actually was.

The whole phase is one modelling question and one traversal. Everything else is a variation.

The modelling question, in two sentences you say out loud before writing anything:

“A vertex is . An edge from A to B means .”

If you cannot finish both, you do not understand the problem yet and no amount of code will help. If you can, the code is mechanical.

Then four classification words, and each rules something in or out:

QUESTION	WHY IT MATTERS
Directed or undirected?	Decides cycle detection entirely — two different algorithms.
Weighted or unweighted?	Decides shortest path: BFS or Dijkstra or Bellman-Ford.
Cyclic or acyclic?	A DAG allows topological sort and one-pass DP with any weights.
Connected?	Almost never. Decides whether you need the outer loop.

The recognition question, in one line: *are there things, and pairwise relationships between them, and is the question about following those relationships?* If yes, it is a graph, whatever the story says.

And the tells that mean a specific algorithm:

THE PROBLEM SAYS	THE ANSWER IS
“fewest steps”, “minimum number of moves”	BFS
“shortest path” with numbers on the edges	Dijkstra
“shortest path” with negative numbers	Bellman-Ford
“in what order”, “can all X be finished”	Topological sort
“how many groups”, “count the islands”	Connected components
“are these connected?” as edges arrive	Union-Find
“connect everything at minimum cost”	Kruskal or Prim
“split into two teams”	Bipartite check
“nearest”, “until everything is”	Multi-source BFS
“is there a circular dependency”	Directed cycle detection

Three modelling moves that turn awkward problems into ordinary ones, and these are the ones worth carrying:

One: the implicit graph. Grids, puzzle states and word ladders have no edge list — neighbours are computed from the coordinates or the state. **You never build the graph**, you write a neighbour function, and every algorithm works unchanged. Candidates who wait to be handed an edge list do not recognise these.

Two: make the shared property a vertex. “Accounts that share an email” as a graph over accounts needs n^2 comparisons to find the edges. Put the *emails* in the graph as vertices too, and it is one linear pass. The same move handles “employees on a shared project”, “stones in the same row or column”.

Three: the virtual source. “Dig a well here, or lay a pipe to a neighbour” looks like two kinds of decision. Add one extra vertex representing the water supply, make each well an edge from it, and there is only one kind of decision — a minimum spanning tree. The same trick turns multi-source BFS into single-source.

And the shape of every traversal in the phase is the same eight lines:

```

seen = {start}
frontier = [start]
while frontier:
    current = take from frontier
    for neighbour in neighbours(current):
        if neighbour not in seen:
            seen.add(neighbour)          # mark on PUSH, always
            add neighbour to frontier

```

Change what “take from frontier” means and you have a different algorithm. Take from the front — BFS. Take from the back — DFS. Take the cheapest — Dijkstra. Take from the front but push zero-cost edges to the front — 0-1 BFS. **That substitution is the whole phase, and saying it out loud shows you see the structure rather than four memorised procedures.**

4 The picture

The decision path, which is the thing to have automatic:

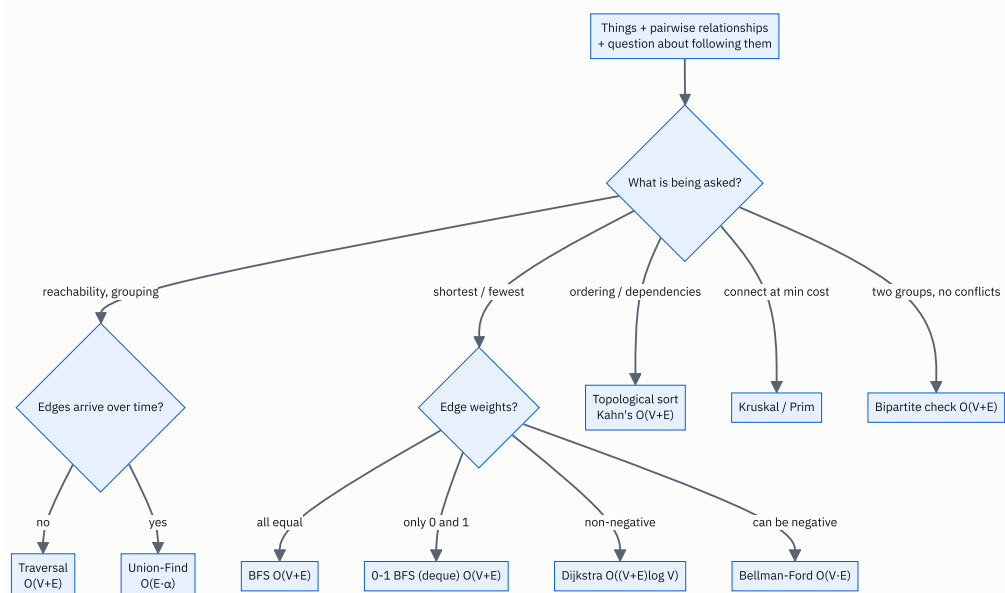


FIGURE 142.1

What to notice. Every leaf is $O(V + E)$ except two, and both exceptions are earned: Dijkstra pays a $\log$ for arbitrary weights, Bellman-Ford pays a factor of V for negative ones. **If your answer is more expensive than $O(V + E)$, you should be able to say which property forced it.**

The one shape, four algorithms:

```
seen = {start}; frontier = [start]
while frontier:
    current = TAKE(frontier)
    for nb in neighbours(current):
        if not seen: mark, PUSH(frontier, nb)

TAKE = popleft, PUSH = append          → BFS          0(V+E)
TAKE = pop, PUSH = append              → DFS          0(V+E)
TAKE = heappop, PUSH = heappush        → Dijkstra     0((V+E) log V)
TAKE = popleft, PUSH = appendleft/append by cost
                                         → 0-1 BFS     0(V+E)
```

The eight recurring bugs, on one page:

- | | |
|-----------------------------------|---|
| 1. edge direction reversed | → a valid answer to the WRONG graph. Silent. |
| 2. built from the edge list | → isolated vertices vanish. PHANTOM CYCLE. |
| 3. only one component searched | → undercount. Sample inputs are connected. |
| 4. marked on pop, not on push | → queue holds E entries. MemoryError / TLE. |
| 5. undirected edge added once | → reachable one way only. Silent. |
| 6. "seen" used as the cycle test | → directed: fails on a diamond
undirected: fails on ONE edge |
| 7. recursion on a long chain | → RecursionError; raising the limit → segfault |
| 8. missing bounds check on a grid | → grid[-1] WRAPS. Wrong answer, no error. |

Five of the eight produce no error at all. That is the whole reason to keep this list.

5 The code, built step by step

Six templates. Learn these and you can write any graph problem in an interview without designing anything under pressure.

Template 0 — build, and the neighbour function

```
from collections import defaultdict

def build(n: int, edges, directed: bool = False):
    graph = {v: [] for v in range(n)}          # from range(n), NOT from the edges
    for a, b in edges:
        graph[a].append(b)
        if not directed:
            graph[b].append(a)                  # BOTH directions, or bug 5
    return graph
```

```

FOUR = ((-1, 0), (1, 0), (0, -1), (0, 1))

def neighbours(r, c, rows, cols):
    for dr, dc in FOUR:
        nr, nc = r + dr, c + dc
        if 0 ≤ nr < rows and 0 ≤ nc < cols:    # all four comparisons
            yield nr, nc

```

`range(n)` and not the edge list — that one choice prevents bugs 2 and 3. **All four bounds comparisons** — `grid[-1]` is the last row, so a missing check wraps the grid silently.

Template 1 — BFS, distances and the path

```

from collections import deque

def bfs(graph, start, goal=None):
    dist = {start: 0}
    parent = {start: None}
    queue = deque([start])
    while queue:
        v = queue.popleft()
        if v == goal:
            break
        for nb in graph[v]:
            if nb not in dist:
                dist[nb] = dist[v] + 1
                parent[nb] = v
                queue.append(nb)
    return dist, parent

```

`dist` is also the seen set. Mark on push. `deque`, never `list.pop(0)`.

Multi-source is one change: seed `dist` and `queue` with every source before the loop.

Template 2 — DFS with three colours (directed cycles, topological order)

```
WHITE, GREY, BLACK = 0, 1, 2

def has_cycle(graph, n):
    colour = [WHITE] * n
    def visit(v):
        colour[v] = GREY                # on the CURRENT PATH
        for nb in graph[v]:
            if colour[nb] == GREY:
                return True
            if colour[nb] == WHITE and visit(nb):
                return True
        colour[v] = BLACK                # after the loop
        return False
    return any(colour[v] == WHITE and visit(v) for v in range(n))
```

Grey means ancestor. Black is a finished region and is fine. Two states report a cycle on a diamond.

Template 3 — Kahn's (order, cycle check, parallel schedule)

```
def kahn(graph, n):
    indeg = [0] * n
    for v in range(n):
        for nb in graph[v]:
            indeg[nb] += 1
    queue = deque(v for v in range(n) if indeg[v] == 0)
    order = []
    while queue:
        v = queue.popleft()
        order.append(v)
        for nb in graph[v]:
            indeg[nb] -= 1
            if indeg[nb] == 0:
                queue.append(nb)
    return order if len(order) == n else None    # None = cycle
```

Prefer this to the DFS version: it detects cycles structurally, never recurses, and processing one whole level at a time gives you a parallel schedule for free.

Template 4 — Dijkstra (and Prim, one line apart)

```
import heapq

def dijkstra(graph, n, source):
    dist = [float("inf")] * n
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        cost, v = heapq.heappop(heap)
        if cost > dist[v]:
            continue # stale: lazy deletion
        for nb, w in graph[v]:
            if cost + w < dist[nb]:
                dist[nb] = cost + w
                heapq.heappush(heap, (cost + w, nb))
    return dist
```

(cost, vertex) — cost first. Return on the **pop**, not the discovery. And Prim's is this with `heappush(heap, (w, nb))` instead of `(cost + w, nb)` — **one term**.

Template 5 — Union-Find

```
class DSU:
    def __init__(self, n):
        self.p = list(range(n)); self.sz = [1]*n; self.groups = n
    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]] # path halving
            x = self.p[x]
        return x
    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb: return False # already together → cycle / redundant
        if self.sz[ra] < self.sz[rb]: ra, rb = rb, ra
        self.p[rb] = ra; self.sz[ra] += self.sz[rb]; self.groups -= 1
        return True
```

Both optimisations, always. The boolean return is what makes it reusable, and Kruskal's is this plus a sort.

The complete solution

```
"""The graph phase on one page: build, traverse, order, connect."""

from __future__ import annotations

import heapq
from collections import deque

INF = float("inf")
WHITE, GREY, BLACK = 0, 1, 2


def build(n, edges, directed=False):
    graph = {v: [] for v in range(n)}
    for a, b in edges:
        graph[a].append(b)
        if not directed:
            graph[b].append(a)
    return graph


def bfs_dist(graph, sources):
    """Single- or multi-source. Seed with every source before the loop."""
    dist = {s: 0 for s in sources}
    queue = deque(sources)
    while queue:
        v = queue.popleft()
        for nb in graph[v]:
            if nb not in dist:
                dist[nb] = dist[v] + 1
                queue.append(nb)
    return dist


def components(graph, n):
    """The outer loop with a SHARED seen set is the whole algorithm."""
    seen, pieces = set(), []
    for start in range(n):
        if start in seen:
            continue
        piece, stack = [], [start]
        seen.add(start)
        while stack:
            v = stack.pop()
            piece.append(v)
            for nb in graph[v]:
                if nb not in seen:
                    seen.add(nb)
                    stack.append(nb)
        pieces.append(piece)
    return pieces


def kahn(graph, n):
    indeg = [0] * n
    for v in range(n):
        for nb in graph[v]:
            indeg[nb] += 1
    queue = deque(v for v in range(n) if indeg[v] == 0)
    order = []
    while queue:
        v = queue.popleft()
        order.append(v)
        for nb in graph[v]:
            indeg[nb] -= 1
            if indeg[nb] == 0:
                queue.append(nb)
    return order if len(order) == n else None
```

```

def dijkstra(graph, n, source):
    dist = [INF] * n
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        cost, v = heapq.heappop(heap)
        if cost > dist[v]:
            continue
        for nb, w in graph[v]:
            if cost + w < dist[nb]:
                dist[nb] = cost + w
                heapq.heappush(heap, (cost + w, nb))
    return dist

def is_bipartite(graph, n):
    colour = [-1] * n
    for start in range(n):
        if colour[start] != -1:
            continue
        colour[start] = 0
        queue = deque([start])
        while queue:
            v = queue.popleft()
            for nb in graph[v]:
                if colour[nb] == -1:
                    colour[nb] = 1 - colour[v]
                    queue.append(nb)
                elif colour[nb] == colour[v]:
                    # SAME colour, not merely coloured
                    return False
    return True

class DSU:
    def __init__(self, n):
        self.p, self.sz, self.groups = list(range(n)), [1] * n, n

    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]]
            x = self.p[x]
        return x

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.sz[ra] < self.sz[rb]:
            ra, rb = rb, ra
        self.p[rb] = ra
        self.sz[ra] += self.sz[rb]
        self.groups -= 1
        return True

def kruskal(n, weighted_edges):
    dsu, total, chosen = DSU(n), 0, []
    for w, a, b in sorted(weighted_edges):
        if dsu.union(a, b):
            total += w
            chosen.append((a, b, w))
    return (total, chosen) if len(chosen) == n - 1 else (-1, chosen)

if __name__ == "__main__":
    g = build(7, [(0, 1), (0, 2), (1, 3), (4, 5), (5, 6), (4, 6)])
    print("dist from 0 :", bfs_dist(g, [0]))
    print("multi-source:", bfs_dist(g, [0, 4]))
    print("components :", components(g, 7))
    print("bipartite? :", is_bipartite(g, 7))
    # 4-5-6 is a triangle

    dag = build(6, [(0, 2), (1, 2), (2, 3), (2, 4), (3, 5), (4, 5)], directed=True)

```

```

print("topo order  :", kahn(dag, 6))
cyc = build(3, [(0, 1), (1, 2), (2, 0)], directed=True)
print("cyclic      :", kahn(cyc, 3))

wg = {0: [(1, 4), (2, 1)], 1: [(3, 1)], 2: [(1, 2), (3, 5)], 3: []}
print("dijkstra    :", dijkstra(wg, 4, 0))
print("kruskal     :", kruskal(4, [(1, 0, 2), (4, 0, 1), (2, 2, 1), (5, 2, 3), (1, 1, 3)]))

```

Running it:

```

dist from 0 : {0: 0, 1: 1, 2: 1, 3: 2}
multi-source: {0: 0, 4: 0, 1: 1, 2: 1, 5: 1, 6: 1, 3: 2}
components  : [[0, 2, 1, 3], [4, 6, 5]]
bipartite?   : False
topo order   : [0, 1, 2, 3, 4, 5]
cyclic       : None
dijkstra     : [0, 3, 1, 4]
kruskal      : (4, [(0, 2, 1), (1, 3, 1), (2, 1, 2)])

```

Three things to look at. `dist from 0` reaches only four of seven vertices — the other component is unreachable, which is why `components` exists and why the outer loop is not optional.

`bipartite?` is `False` because of the triangle at 4-5-6, **even though the component containing vertex 0 is perfectly bipartite**. A graph is bipartite only if every component is.

And `dijkstra` gives vertex 1 a distance of **3**, via vertex 2 at cost 1 plus 2 — not the direct edge of weight 4. That is the whole reason Dijkstra exists rather than BFS.

6 What it costs

Every cost in the phase, in one table.

ALGORITHM	TIME	SPACE	WHEN
BFS / DFS	$O(V + E)$	$O(V)$	reachability, components, fewest steps
Multi-source BFS	$O(V + E)$	$O(V)$	nearest source — <code>k</code> does not appear
0-1 BFS	$O(V + E)$	$O(V)$	weights are only 0 and 1
Bipartite check	$O(V + E)$	$O(V)$	two groups; colour array is the seen set
Topological sort	$O(V + E)$	$O(V)$	ordering; also detects cycles
DAG shortest/longest path	$O(V + E)$	$O(V)$	any weights, including negative
Dijkstra	$O((V + E) \log V)$	$O(V + E)$	non-negative weights
Bellman-Ford	$O(V \times E)$	$O(V)$	negative weights; detects negative cycles
Floyd-Warshall	$O(V^3)$	$O(V^2)$	all pairs, $V \lesssim 500$
Union-Find	$O(E \times \alpha) \approx O(E)$	$O(V)$	grouping as edges arrive
Kruskal	$O(E \log E)$	$O(V + E)$	MST — the sort dominates
Prim (heap)	$O(E \log V)$	$O(V + E)$	MST on a sparse graph
Prim (dense)	$O(V^2)$	$O(V)$	MST on a dense graph or a matrix

The numbers that make those concrete, on $V = 100,000$ and $E = 300,000$ — a typical sparse graph:

BFS / DFS / topological	700,000 steps	
Dijkstra	5,100,000	~7x BFS
Bellman-Ford	30,000,000,000	~40,000x BFS
Floyd-Warshall	10^{15}	impossible

And the two comparisons worth having ready:

```

multi-source vs one BFS per source
  500 sources, 1,000,000 cells
  multi-source      3,000,000
  per-source        1,500,000,000      → 500x

traversal vs Union-Find, edges arriving one at a time
  m = 100,000 edges, V = 100,000
  traversal per edge 30,000,000,000
  Union-Find         400,000          → 75,000x

```


A **phantom cycle**, because `len(order) = n` cannot hold. Build from `range(n)`.

3. Only one component searched

```
return len(bfs_dist(graph, [0]))      # "the number of vertices"
```

Correct on a connected sample, wrong on the hidden test. “All”, “every”, “how many groups” means the outer loop.

4. Marked on pop, not on push

```
v = queue.popleft()
if v in seen: continue
seen.add(v)
for nb in graph[v]: queue.append(nb)      # pushed unchecked
```

Answer correct, queue holds **E** entries instead of **V**.

MemoryError

Say “each vertex enters the queue at most once” as you write it. If that sentence is not true, the code is wrong.

5. The undirected edge added once

```
graph[a].append(b) # graph[b].append(a) forgotten
```

```
>>> reachable(g, "x", "y"), reachable(g, "y", "x")
(True, False)
```

Silent, and on a bipartite or “possible partition” problem it produces a confident wrong True .

6. “Seen” used as the cycle test

Directed, two states instead of three:

```
>>> has_cycle_two_state(diamond, 4)      # 0→1→3, 0→2→3
True                                     # there is no cycle
```

Undirected, no parent check:

[illegible]

Two different problems that share a word. Directed needs grey/black; undirected needs the parent exclusion.

7. Recursion on a long chain

```
RecursionError: maximum recursion depth exceeded
```

and the “fix”:

```
sys.setrecursionlimit(10**6)
```

```
Segmentation fault (core dumped)
```

$n \leq 10^5$ in the constraints means write it iteratively, or use Kahn’s, which never recurses.

8. The missing bounds check on a grid

```
if grid[r + dr][c + dc] == "1":           # no bounds check
```

`grid[-1]` is the **last row**, so the top of the grid connects to the bottom. Falling off the right edge raises `IndexError`; falling off the top does not. **Half the failures crash and half are silent**, which is the worst combination.

8 In the interview

How it gets asked

Never as “traverse this graph”. These:

- “Given courses and prerequisites, can they all be finished? Now give me an order.”
- “Count the islands.” / “How many provinces?”
- “Given accounts with emails, merge the ones belonging to the same person.”
- “Find the shortest transformation from one word to another.”
- “Cheapest flight from A to B with at most k stops.”
- “Connect all the points at minimum cost.”
- “Can these people be split into two teams?”
- “How long until every orange rots?”

The tell, in one line: things, plus pairwise relationships, plus a question about following them.

The mock round

Two problems. Twenty minutes each. Say everything out loud, including the parts where you are stuck.

Problem 1. “You are given n stones on a 2D plane, each at some (row, col) . A stone can be removed if it shares a row or a column with another stone that is still on the plane. Return the maximum number of stones you can remove.”

Minute 0 to 4 — model it, and do not code.

“Let me not code, because the modelling here is the entire problem and the first model I would reach for is wrong.

The obvious model: a vertex is a stone, and an edge joins two stones sharing a row or a column. Finding those edges means comparing every pair — n^2 . For a thousand stones that is a million comparisons, which is survivable, but it is the wrong shape and I think there is better.

The insight about the answer, first. Take any connected group of stones. I claim I can remove all but one of them: repeatedly remove a stone that is a *leaf* in a spanning tree of the group, and it always shares a row or column with something still present. So **from a component of size k I remove $k - 1$.**

So the answer is $n - (\text{number of connected components})$. That reduces the whole problem to counting components.

Now the better model, so I do not pay n^2 . Instead of connecting stones to stones, I make the **rows and columns** the vertices, and each stone an edge joining its row to its column. Two stones in the same row are then automatically connected through that row vertex, and I never compared any pair of stones.

A vertex is a row index or a column index. An edge is a stone. Undirected, unweighted.”

Minute 4 to 12 — write it.

```
def remove_stones(stones: list[list[int]]) → int:
    dsu = DSU(20000)          # rows 0..9999, columns offset by 10000
    seen = set()
    for r, c in stones:
        dsu.union(r, c + 10000)    # a stone joins its ROW to its COLUMN
        seen.add(r)
        seen.add(c + 10000)
    roots = {dsu.find(x) for x in seen}
    return len(stones) - len(roots)
```

Say while typing: “The offset is so row 3 and column 3 are different vertices — without it, a stone at (3, 3) would join a vertex to itself and, worse, rows and columns would merge wrongly. And I count roots only over the indices that actually appear, not over all twenty thousand, or every unused row is its own component.”

Minute 12 to 16 — cost.

“One pass over the stones, two `find`s and a `union` each, so $O(n \times \alpha)$ — effectively $O(n)$. Space is the Union-Find array, $O(\text{rows} + \text{cols})$.

Against the naive model: n^2 comparisons to build the edges, then a traversal. At a thousand stones that is a million against a thousand. **Making the shared property a vertex is what removed the quadratic step**, and it is the same move as the accounts-and-emails problem.”

Minute 16 to 20 — the follow-up.

“Prove that you can always remove all but one from a component.”

“Take a spanning tree of the component. Repeatedly remove a leaf: a leaf has a neighbour still present, and being adjacent means sharing a row or a column, so the removal is legal. Removing a leaf leaves a smaller tree, so I can repeat until one vertex remains. Hence $k - 1$ from a component of k , and the total is $n - \text{components}$.”

Problem 2. “A network of n servers connected by cables. Some cables are optional and some are critical. Find the cheapest set of cables that keeps every server connected, but the answer must also tell me which cables could be removed later without disconnecting anything.”

Minute 0 to 5 — recognise, and ask.

“Two questions bundled together, and they are different algorithms, so let me separate them.

‘Cheapest set that keeps everything connected’ is a minimum spanning tree.

Vertices are servers, edges are cables with costs, undirected. Kruskal’s, because the input is an edge list and because Union-Find gives me the cycle check for free.

‘Which could be removed without disconnecting anything’ is a different question, and I want to check what it means. Removed *from the original network*, or *from the spanning tree*? Because if it is from the spanning tree, the answer is none — every edge of a tree is a bridge, by definition. So I think it means: in the original graph, which cables lie on a cycle, since those can go.

Is that the right reading?”

Minute 5 to 14 — write it.

```
def cheapest_and_removable(n, cables):
    """cables: (cost, a, b). Returns (total, kept, removable)."""
    dsu = DSU(n)
    total, kept, removable = 0, [], []
    for cost, a, b in sorted(cables):
        if dsu.union(a, b):
            total += cost
            kept.append((a, b, cost))
        else:
            removable.append((a, b, cost))    # both ends already connected → a cycle
    if len(kept) != n - 1:
        return -1, kept, removable          # the network is not connected at all
    return total, kept, removable
```

Say while typing: “The `else` branch is free — an edge Kruskal’s rejects is exactly an edge whose endpoints were already connected, which means it closes a cycle, which means it is removable. One loop answers both questions. And the `len(kept) != n - 1` check catches a network that was already disconnected, which returns a plausible total otherwise.”

Minute 14 to 18 — cost and correctness.

“ $O(E \log E)$ dominated by the sort; the Union-Find part is effectively free. Space $O(V + E)$.”

Why greedy is correct here — the cut property. Split the vertices any way you like; the cheapest edge crossing between the two halves belongs to some minimum spanning tree, because if a tree used a different crossing edge you could swap it for the cheaper one and still have a spanning tree of no greater weight. Kruskal’s is repeated application of that.

One thing I should be honest about: with tied costs there are several minimum spanning trees, so the `kept` and `removable` sets are not unique — only the total is. If the operator needs a specific answer, I need a tie-break rule.”

Minute 18 to 20 — when it goes wrong.

If you have not seen the “rejected edge is removable” trick, say so and keep working:

“I do not immediately see how to get the removable set out of the same pass. Let me solve the two separately first — Kruskal’s for the tree, and then for each edge test whether removing it disconnects the graph, which is $O(E \times (V + E))$ and too slow but correct — and then look for something better.”

That is a passing answer. Being stuck out loud with a correct fallback beats silence, and the interviewer will usually offer the hint once they see a working baseline.

The model answer

“*Why did you choose that algorithm?*” — the question that ends most graph interviews.

“Four things, in this order.

What is being asked. Reachability and grouping is a traversal. Fewest steps is BFS. Ordering is topological sort. Cheapest total connection is a minimum spanning tree. Those are different questions and they do not share an answer.

What the edges look like. Unweighted means BFS and I would say so rather than reach for Dijkstra, which is about seven times more work on a sparse graph for an answer BFS already has. Non-negative weights means Dijkstra. Negative means Bellman-Ford, and I would name the exact clause of Dijkstra’s proof that fails. And if it is a DAG, one pass in topological order handles any weights, including negative — faster than all of them, and the answer people forget.

Whether the graph changes. Static and asked once, a traversal. Edges arriving over time, Union-Find — that is $m \times \alpha$ against $m \times (V + E)$, which at a hundred thousand edges is four hundred thousand steps against thirty billion.

And the size. V under a couple of hundred makes an adjacency matrix and even Floyd-Warshall reasonable. Above ten thousand, a matrix does not fit and the representation choice is forced.

The thing I check before all of that is whether the graph even needs building. Grids, word ladders and puzzle states are implicit — I write a neighbour function and never materialise an edge list — and problems about a shared property get an extra vertex for the property, which turns an n^2 edge-building step into one linear pass.”

9 Recall card

Two sentences before any code: “A vertex is . *An edge from A to B means* .” Then four words: **directed? weighted? cyclic? connected?** — and connected is almost never.

One shape, four algorithms. `seen`, a frontier, mark on push. Take from the front → BFS; from the back → DFS; cheapest → Dijkstra; front-with-zero-cost-edges-pushed-front → 0-1 BFS.

Everything is $O(V + E)$ except two earned exceptions: Dijkstra pays $\log V$ for arbitrary weights, Bellman-Ford pays V for negative ones. **On a DAG, one topological pass handles any weights** — the cheapest option and the one most often missed.

Three modelling moves: implicit graphs (never build the edges); **make the shared property a vertex** (removes an n^2 step); and the virtual source (turns two kinds of decision into one).

Eight bugs, five of them silent: reversed direction; built from the edge list (phantom cycle); one component only; marked on pop; undirected edge added once; “seen” as the cycle test (diamond for directed, one edge for undirected); recursion on a long chain; and a missing grid bounds check (`grid[-1]` wraps).

1 What this is, and why they ask it

“Find every driver within two kilometres of me” sounds like a filter and is not. A `WHERE` clause on latitude and longitude cannot use an ordinary index, because a B-tree sorts on one dimension and a location is two. Index the latitude and you narrow a horizontal band across the entire planet; index both and the database still has to intersect two enormous sets.

The fix is to turn two dimensions into one, in a way that preserves nearness: **give every location a name such that nearby locations have similar names**. Then “near me” becomes a prefix match or a small set of key lookups, which a B-tree does very well.

There are two families. **Geohash** interleaves the bits of latitude and longitude into a single string, so a shared prefix means a shared area. **Quadtrees** recursively split space into four, subdividing only where things are dense, so a busy city gets fine cells and an empty desert gets one enormous one.

They ask this because it appears in every location design — ride-hailing, food delivery, dating, maps, store locators — and because the naive answer has a specific failure that you can quantify. **It is also one of the few areas where the standard solution has a well-known bug** — the boundary problem — and knowing about it is the difference between reciting a technique and having used one.

By the end of this lesson you can explain why a two-column index fails, describe geohashes and quadtrees and choose between them, handle the boundary problem, size a real system, and say which parts you would not build yourself.

2 The story

Ramnath has delivered in the same three wards for thirty-four years, and the thing that makes the job possible is that an address narrows.

He gets a sack in the morning and the first thing he does is sort it, and the sorting is not by name or by street. It is by the number.

The first two digits get him to the region. The next one narrows it further. The last three name one office — his office. Anything that does not start with those six digits is somebody else’s problem and goes back in the bin at the counter, which takes him about eleven minutes and removes most of the sack.

The index on `lat` narrows to a **horizontal band around the entire planet** at that latitude, which for a global system is millions of rows, and then every one of them is checked against the longitude. A **composite index on `(lat, lng)` does not fix it either**, because a composite index only narrows on the second column after the first is fixed to a single value — and latitude is a range, not a value.

So: reduce two dimensions to one, preserving nearness. That is the whole idea, and both families are ways of doing it.

A geohash interleaves the bits. Take the latitude's bits and the longitude's bits and alternate them — `lat, lng, lat, lng` — then encode the result in base 32. The result is a short string like `te7ud`.

And the property that makes it useful is exactly Ramnath's: a shared prefix means a shared area. Every location whose geohash starts with `te7ud` is inside one box on the earth's surface. Longer prefix, smaller box:

1 character	~5,000 km x 5,000 km
4 characters	~39 km x 20 km
6 characters	~1.2 km x 0.6 km
8 characters	~38 m x 19 m

So “find everything near me” becomes “find everything whose geohash starts with my first six characters” — which is a prefix range scan on an ordinary B-tree, and B-trees are extremely good at that. **Two dimensions became one, and a standard index works.**

The boundary problem is Ambedkar Road, and it is the thing to raise unprompted. Two points can be metres apart and share no prefix at all, because a cell boundary runs between them. A driver twenty metres away, across the line, is invisible to a pure prefix search.

The fix is to query the cell and its eight neighbours. Geohash libraries compute a cell's neighbours directly, so a search is nine prefix scans rather than one. **That is not a workaround, it is the standard way to do it**, and a design that does one prefix scan is wrong.

The second problem is that the boxes are a fixed size and the world is not evenly full. A six-character cell in central Mumbai contains thousands of drivers; the same size cell over the Arabian Sea contains none. Query a fixed precision and you either get a useless number of results or none at all.

Quadtrees solve that by subdividing only where it is needed. Start with one box covering everything. When a box holds more than some number of points — say a hundred — split it into four quadrants and push the points down. Repeat. **Dense areas end**

up deeply subdivided and empty areas stay as one large cell, which is Ramnath’s cousin’s enormous rural ward and his own small urban one.

So the trade is: geohash cells are fixed-size and dead simple to store; quadtree cells are adaptive and need a real structure that changes as points move.

And the third idea, which is what most modern systems actually use: cells on a sphere. Geohashes are computed on a flat latitude/longitude rectangle, so cells become badly distorted near the poles and a cell’s “size” varies with where you are. Google’s S2 and Uber’s H3 map the sphere properly — S2 by projecting onto a cube, H3 by tiling with hexagons — so cells have consistent area, and neighbours behave sensibly. H3’s hexagons have one nice property worth knowing: every neighbour is the same distance away, whereas a square’s diagonal neighbours are further than its edge neighbours.

Whatever the indexing, the last step is always the same: the index narrows, and then you compute real distances. A cell is a box, and “within two kilometres” is a circle. So the index gives you a candidate set of the right order of magnitude, and then you filter it exactly. The index’s job is to avoid scanning everything, not to give the final answer.

4 The picture

Why a two-column index does not work:

index on lat, then filter on lng:

lng →

lat ^

+-----+
|
|#####|
|#####|
|#####|
|
| +---+
| | X |
| +---+
+-----+

← the lat index gives you
THIS BAND, around the
whole planet

← you wanted this box

millions of rows scanned to find a few hundred.

Geohash: interleaving, and the prefix property:

```

lat bits:      1 0 1 1 0 ...
lng bits:      0 1 1 0 0 ...
interleaved:   1 0 0 1 1 1 1 0 0 0 ...    → base32 → "te7ud..."

```

the tree of cells:

```

"t"           one huge box
|
"te"          1/32 of it
|
"te7"         1/32 again
|
"te7u"  "te7v"  "te7w"  ...
|
"te7ud"      ~1.2 km x 0.6 km

```

shared prefix = shared box. Longer prefix = smaller box.

The boundary problem, drawn:

```

geohash cell "te7ud" | geohash cell "te7uf"
                    |
                    | . B
                    |
                20 metres apart
                prefixes share NOTHING useful

```

prefix search from A's cell finds A and misses B.

THE FIX: query the cell AND its 8 neighbours

```

+-----+-----+-----+
| NW  |  N  | NE  |
+-----+-----+-----+
|  W  | ME  |  E  |
+-----+-----+-----+
| SW  |  S  | SE  |
+-----+-----+-----+

```

9 prefix scans instead of 1

Quadtree adaptivity, which is the other half:

FIXED GRID (geohash, precision 6)

```
+---+---+---+---+
|   |   |####|####|
|   |   |####|####|  ← city
+---+---+---+---+
|   |   |####|####|
|   |   |####|####|
+---+---+---+---+
|   |   |   |   |   |  ← sea
|   |   |   |   |   |
+---+---+---+---+
```

every cell the same size:
4 cells with 3,000 each
12 cells with 0

QUADTREE (capacity 100)

```
+-----+-----+
|           | +---+---+
|           | |##|##|
|           | +---+---+
| (empty,   | |##|##|
| ONE      | +---+---+
| cell)    |           |
+-----+-----+
|           |           |
|           |           |
+-----+-----+
```

cells sized by DENSITY:
the city is subdivided 4 more
levels; the sea is one cell

What to notice. The geohash grid does the same amount of dividing everywhere, so a query in the city returns thousands and a query at sea returns nothing. The quadtree divides where the points are, so **every leaf holds roughly the same number of points** — which is Ramnath’s cousin’s sixty square kilometres with the same number of addresses as Ramnath’s two.

5 How it actually works

Geohash, concretely

```
import geohash                                     # python-geohash, or pygeohash

h = geohash.encode(19.0760, 72.8777, precision=6)   # → 'te7ud5'
lat, lng = geohash.decode(h)
neighbours = geohash.neighbors(h)                  # the 8 surrounding cells
```

Storage is a plain indexed column:

```
CREATE TABLE driver_locations (
  driver_id BIGINT PRIMARY KEY,
  lat       DOUBLE PRECISION NOT NULL,
  lng       DOUBLE PRECISION NOT NULL,
  geohash6  TEXT NOT NULL,
  updated_at TIMESTAMPTZ NOT NULL
);
CREATE INDEX ON driver_locations (geohash6);
```

And the query is the nine-cell scan, then an exact filter:

```
SELECT driver_id, lat, lng
FROM driver_locations
WHERE geohash6 IN ('te7ud5','te7ud4','te7ud6','te7ud7','te7udh', ...) -- 9 cells
    AND updated_at > now() - interval '1 minute'
    AND earth_distance(ll_to_earth(lat, lng), ll_to_earth(:lat, :lng)) < 2000
ORDER BY earth_distance(...) LIMIT 20;
```

Three things in that query matter. The nine cells, not one. The freshness filter, because a driver's location from an hour ago is not a driver. And the exact distance calculation at the end, because a cell is a box and the requirement is a circle.

Choosing the precision is the design decision, and it comes from the radius you want:

```
radius 500 m    → precision 7 (~150 m cells), 9 cells covers ~450 m
radius 2 km     → precision 6 (~1.2 km cells), 9 cells covers ~3.6 km
radius 10 km    → precision 5 (~5 km cells)
```

Pick the precision so that nine cells comfortably cover the radius, then filter exactly. Too fine and you need far more than nine cells; too coarse and you scan a huge candidate set.

Redis GEO, which is what many systems actually use

Redis has this built in, and it is a sorted set with geohash values as scores:

```
GEOADD drivers 72.8777 19.0760 driver:8814
GEOSEARCH drivers FROMLONLAT 72.8777 19.0760 BYRADIUS 2 km ASC COUNT 20
```

$O(\log n + m)$ per query, in memory, with the boundary handling and the distance maths done for you. **For live location on a fleet of moderate size this is the whole answer,** and reaching past it needs a reason — usually scale, durability, or wanting the data queryable alongside other columns.

Quadtrees

```
insert(point):
    find the leaf containing it
    add the point
    if the leaf now holds more than CAPACITY:
        split into 4 children
        redistribute the points

query(circle):
    start at the root
    for each child whose box INTERSECTS the circle:
        recurse
    at leaves, test each point exactly
```


The **capacity** is the **tuning knob** — usually a few tens to a few hundred. Small capacity means a deeper tree and more nodes; large capacity means more points to filter at each leaf.

What quadtrees cost you is mutation. A ride-hailing system updates every driver's location every few seconds; every update potentially removes a point from one leaf, inserts into another, and may trigger a split or a merge. **That is real work and it needs locking**, which is exactly what geohashes avoid — updating a geohash is writing one string.

So the honest rule: quadtrees for relatively static data with wildly uneven density — points of interest, shops, property listings. **Geohash or S2/H3 for high-churn moving objects.**

S2 and H3

S2 (Google) projects the sphere onto the six faces of a cube and uses a Hilbert curve within each face, so that nearby cells have nearby 64-bit ids. Thirty levels of subdivision, from the whole earth down to about a square centimetre. **The Hilbert curve preserves locality better than geohash's simple interleaving**, which reduces — though does not eliminate — the boundary problem.

H3 (Uber) tiles the sphere with **hexagons**. Two properties matter:

- **All six neighbours are equidistant** from a cell's centre, unlike a square where diagonal neighbours are 1.41 times as far. That makes “expand the search by one ring” behave uniformly.
- **Cells are close to equal area** across the globe, which geohash cells are not — a geohash cell at 60° latitude is about half the width of one at the equator.

Hexagons cannot tile perfectly at every level, so H3's hierarchy is approximate rather than exact — a child is not exactly one seventh of its parent. **That is the trade, and it rarely matters in practice.**

PostGIS and R-trees

```
CREATE EXTENSION postgis;
ALTER TABLE places ADD COLUMN geom geography(Point, 4326);
CREATE INDEX ON places USING GIST (geom);

SELECT * FROM places
WHERE ST_DWithin(geom, ST_MakePoint(:lng, :lat)::geography, 2000)
ORDER BY geom <=> ST_MakePoint(:lng, :lat)::geography
LIMIT 20;
```

PostGIS uses an **R-tree** (via GiST): a tree of bounding rectangles, where each node's rectangle contains its children's. It handles points, lines and polygons, does real spherical distance, and there is **no boundary problem and no cell-size tuning** — the index is genuinely two-dimensional rather than a projection to one.

For anything already in Postgres and not at extreme write rates, this is the right answer, and reaching for a hand-rolled geohash scheme instead is usually a mistake. Where it stops being right is very high write volume — R-tree updates are more expensive than writing a string — and horizontal scale.

The read/write asymmetry, which shapes the architecture

```
ride-hailing:
  writes    every driver, every 4 seconds    → enormous, tiny, and disposable
  reads     every rider request              → fewer, and must be fast
```

Driver locations are ephemeral and do not belong in the durable database at all. Redis with a TTL, or an in-memory service, and the source of truth is “wherever the driver last reported”. Losing the whole store means every driver re-reports within four seconds. **Recognising that the write-heavy part needs no durability is the main architectural insight in these designs.**

6 The numbers

Why the naive query fails:

```
1,000,000 drivers worldwide
index on lat, band of 0.04 degrees (~4.4 km)
fraction of the planet's latitude range covered    0.04 / 180 = 0.02%
but drivers are NOT uniform: in a city, that band
crosses many dense areas globally
realistic rows examined                            50,000 - 200,000
then filtered on lng to                             ~200
```

```
with a geohash-6 index:
  9 cells x ~1.2 km each
  drivers in ~3.6 km^2 of a dense city              ~500
  exact distance filter                             → ~200
                                                    → 250x fewer rows
```

Geohash cell sizes, worth memorising roughly:

precision	width x height (at the equator)	
4	39 km	x 20 km
5	5 km	x 5 km
6	1.2 km	x 0.6 km
7	153 m	x 153 m
8	38 m	x 19 m

Sizing a ride-hailing system. 100,000 active drivers in a city, reporting every 4 seconds:

writes	100,000 / 4 s	= 25,000 writes/second
each	driver_id + lat + lng + time	= ~50 bytes
		= 1.25 MB/s

reads	50,000 ride requests/hour peak = ~14/second
	each: 9 cell lookups + distance filter on ~500 candidates

Twenty-five thousand writes a second against fourteen reads. That asymmetry is the whole design: **the write path must be cheap and the data must be disposable.**

Redis GEOADD	~100,000 ops/s per instance	→ comfortable
Postgres with a GiST index	~5,000-20,000 writes/s	→ needs sharding

Memory for the live set:

100,000 drivers x ~100 bytes in a Redis sorted set	= 10 MB
1,000,000 drivers	= 100 MB

Trivial, which is the other reason this lives in memory.

Quadtree sizing for 10 million static points of interest, capacity 100:

leaves	10,000,000 / 100	= 100,000 leaves
internal nodes	~ leaves / 3	= ~33,000
total nodes	~133,000	
per node	~100 bytes	= ~13 MB of structure
depth	log ₄ (100,000)	= ~8 levels
query	8 node hops + ~100 point tests	

Query cost, compared:

full scan, 10M points	10,000,000 distance calculations
geohash, 9 cells	~500 candidates + 500 distance calcs
quadtree	~8 hops + ~100-400 distance calcs
PostGIS GiST	~log(n) index descent + candidates

The exact-distance step is not free and is often forgotten in estimates: 500 haversine calculations is about 50 microseconds, which is fine, and 50,000 would not be.

Boundary problem, quantified:

```
precision-6 cell: 1.2 km x 0.6 km, perimeter ~3.6 km
search radius 500 m
fraction of a cell's area within 500 m of an edge: ~70%

→ for a MAJORITY of query points, the answer requires neighbouring cells
```

That is why nine cells is the standard and one cell is a bug — it is not a rare edge case, it is most queries.

Cell distortion by latitude, which is the argument for S2 and H3:

```
geohash precision 6, cell width
  at the equator (0 deg)      1.2 km
  at Mumbai (19 deg)         1.1 km
  at London (51 deg)         0.75 km
  at Reykjavik (64 deg)      0.53 km

→ the same precision means different real areas
→ a fixed precision tuned for one city is wrong in another
```

7 The trade-offs

Geohash is simple and its cells are fixed. One indexed string column, ordinary B-tree, works in any database, trivially shardable by prefix. What you give up: cells do not adapt to density, so a precision tuned for Mumbai returns nothing useful in a rural area and thousands in a city centre; cells are distorted by latitude; and **the boundary problem means every query is nine lookups rather than one.**

Quadtrees adapt to density and cost you mutation. Every leaf holds a similar number of points regardless of how crowded the area is, which makes query cost predictable. In exchange, moving a point may trigger a split or a merge, the structure needs locking under concurrent updates, and it is a real data structure to implement and operate rather than a string. **Static-ish data with uneven density: quadtree. Constantly moving objects: not a quadtree.**

S2 and H3 fix the geometry and add a dependency. Proper spherical cells, consistent areas, better locality from the Hilbert curve, and H3's equidistant neighbours. The cost is a library and a set of concepts your team has to learn, and ids that are opaque 64-bit integers rather than human-readable strings.

PostGIS is the least work and the least scalable. A real two-dimensional index, no boundary problem, no cell tuning, exact spherical distances, and polygons and lines as well as points. It tops out on write throughput far earlier than a Redis-based approach, and scaling it horizontally is a project.

Precision is a permanent trade. Finer cells mean a smaller candidate set and more cells to query; coarser means fewer lookups and more filtering. **And it interacts with density**, so any single global precision is wrong somewhere — which is the argument for storing several precisions, or for a quadtree, or for accepting that the numbers are tuned for the cities that matter.

The index never gives the final answer. A cell is a box and the requirement is a circle, so the exact distance filter always runs. Estimates that omit it under-count the work, and designs that omit it return drivers 2.8 km away for a 2 km query.

When would I not build any of this? When the data fits in memory and the query rate is low — a few thousand points and a linear scan with a distance calculation is a millisecond and needs no index at all. When PostGIS is available and the write rate is modest, where a GiST index is one line and has none of the boundary or tuning problems. And when the requirement is really “the nearest one” rather than “everything within a radius”, where a managed service or a routing engine may be a better fit than an index you maintain.

8 In the interview

How it gets asked

- “How do you find all drivers within two kilometres?” — the direct version.
- “Why can’t you just index latitude and longitude?” — the arithmetic question.
- “Design Uber / a food delivery app / a store locator.” — where this appears as a component.
- “What is a geohash?”
- “A driver twenty metres away is not being returned. Why?” — the boundary question, and the best one.
- “Drivers report their location every four seconds. Where does that go?”

The first ninety seconds

“The reason this is not a simple `WHERE` clause is that a B-tree indexes one dimension and a location is two. An index on latitude narrows me to a horizontal band around the entire planet, and then every row in that band is checked against longitude — hundreds of thousands of rows to find a couple of hundred. **And a composite index on `(lat, lng)` does not help**, because it only narrows on the second column once the first is a single value, and latitude here is a range.

So the move is to reduce two dimensions to one while preserving nearness: give every location a name such that nearby locations share a prefix.

A **geohash** does that by interleaving the bits of latitude and longitude and encoding them in base 32. A shared prefix means a shared box: six characters is about 1.2 by 0.6 kilometres, seven is about 150 metres. So ‘near me’ becomes a prefix range scan on an ordinary B-tree, which is exactly what B-trees are good at.

The thing I would raise before you ask is the boundary problem. Two points twenty metres apart can share no prefix at all if a cell edge runs between them. So a query is **the cell plus its eight neighbours** — nine prefix scans — and the libraries compute neighbours for you. **That is not an edge case: for a 500-metre radius on precision-6 cells, roughly 70% of query points are within 500 metres of a cell edge, so most queries need it.**

And the index never gives the final answer. A cell is a box and the requirement is a circle, so the last step is always an exact distance filter over the candidates. The index’s job is to get me from ten million rows to five hundred, not to five hundred to twenty.

The other question I would ask is about the write pattern, because for something like ride-hailing it is a hundred thousand drivers reporting every four seconds — twenty-five thousand writes a second against maybe fourteen reads. **That data is ephemeral and does not belong in the durable database at all**, and recognising that changes the whole architecture.”

The follow-ups

“A driver twenty metres away is not being returned. What is wrong?”

“Almost certainly the boundary problem, and I would explain the mechanism rather than just name it.

A geohash cell is a box, and two points on opposite sides of a cell edge share no useful prefix even if they are metres apart — one might be `te7ud5` and the other `te7uf0`. A pure prefix search on my own cell simply does not see the other one.

The fix is to search the cell and its eight neighbours, which the geohash library gives me directly. Nine lookups instead of one.

And the reason I would call this a design error rather than a rare edge case is the arithmetic. With precision-6 cells at roughly 1.2 by 0.6 kilometres, and a 500-metre search radius, about seventy percent of possible query positions are within 500 metres of some edge. So a majority of queries need at least one neighbouring cell. Anyone who tested with a point in the middle of a cell would never see it.

The other candidates I would check, because it might not be the boundary: a freshness filter excluding a driver whose last report is stale — which is correct behaviour, not a bug; the precision being too fine so that nine cells do not cover the radius; and the exact distance filter using a flat-earth approximation that is wrong at that scale.

And the structural fix, if this keeps happening: move to H3 or S2, where the neighbour handling is cleaner — H3’s hexagons have all six neighbours equidistant, so ‘expand by one ring’ is uniform, whereas a square’s diagonal neighbours are 1.41 times further and the ring is lopsided.”

“Geohash or quadtree? What decides it?”

“How much the data moves, and how uneven the density is.

Geohash cells are a fixed grid. Simple — one indexed string, ordinary B-tree, works in any database, and shards naturally by prefix. Updating a point’s location is writing one string, which is why it survives a heavy write load. What it does not do is adapt: precision-6 in central Mumbai returns thousands and the same precision over farmland returns nothing, and cells are noticeably distorted by latitude — a precision-6 cell is 1.2 km wide at the equator and 0.53 km in Reykjavik.

A quadtree subdivides only where points are dense, so every leaf holds roughly the same number of points and query cost is predictable everywhere. That is exactly what you want for shops, property listings, or points of interest — data that is wildly uneven and does not move.

The cost is mutation. A moving point may leave one leaf and enter another, which can trigger a split or a merge, and under concurrent updates that needs locking. For a hundred thousand drivers reporting every four seconds — twenty-five thousand updates a second — that is the wrong structure.

So: quadtree for static-ish, uneven data. Geohash or H3 for moving objects. And in practice, for live driver locations I would reach for Redis GEO first, which is a sorted set with geohash scores and does the boundary and distance handling for me — a hundred thousand operations a second on one instance, and about ten megabytes for a hundred thousand drivers.”

“Where do twenty-five thousand location updates a second actually go?”

“Not into the durable database, and I think recognising that is the main architectural point in these designs.

The data is ephemeral. A driver’s location four seconds ago is worthless — the current one supersedes it entirely. There is no query that needs history for dispatch, and if the entire store were lost, every driver re-reports within four seconds and the system is whole again.

So: Redis, keyed by city or region, with a TTL of maybe thirty seconds.

`GEOADD` per update, `GEOSEARCH` per rider request. A hundred thousand drivers is about ten megabytes and well within one instance’s throughput; sharding by city is natural and never needs a cross-shard query, because nobody requests a ride across cities.

The TTL is doing real work. A driver who goes offline stops reporting, and their entry expires automatically — so ‘is this driver still available’ needs no separate mechanism and no cleanup job. That is a nice property to get for free.

What does go to durable storage is the trip, not the pings — start, end, route, fare — which is a few writes per trip rather than tens of thousands a second. And if location history is needed for analytics or disputes, that is a separate, batched, append-only stream into object storage, sampled rather than complete.

The general principle: separate the high-churn ephemeral state from the low-volume durable state, and give them completely different homes. Trying to serve both from one database is what makes these systems expensive.”

“Would you use PostGIS instead?”

“For most systems, yes, and I would say that rather than reflexively building a geohash scheme.

PostGIS gives me a genuine two-dimensional index — a GiST R-tree over bounding boxes — so there is **no boundary problem and no cell precision to tune.**

`ST_DWithin` does proper spherical distance, `ORDER BY geom ↔ point` does an indexed nearest-neighbour search, and it handles polygons and lines as well as points, which matters the moment somebody asks ‘which delivery zone is this address in’.

And it is one extension and one index on data that is already in the database, query-able alongside every other column — ‘restaurants within 2 km that are open now and rated above four’ is one query, whereas with an external geo store it is a join across two systems.

Where it stops being right is write throughput. An R-tree update is considerably more expensive than writing a string, so twenty-five thousand location updates a second is beyond a single Postgres instance — realistically five to twenty thousand writes a second with a GiST index. For read-heavy location data — shops, listings, addresses — that ceiling is irrelevant.

So my default is: PostGIS for anything that is mostly read, Redis GEO for live moving objects, and H3 or S2 when I need consistent cell geometry across the globe or want to shard by cell across many machines. Hand-rolling geohash prefixes is the option I would justify rather than assume.”

The model answer

“Design the driver-matching component of a ride-hailing service: 100,000 active drivers in a city, drivers report location every four seconds, riders request a car and want the nearest few.”

“Let me lead with the asymmetry, because it decides everything else.

Twenty-five thousand writes a second against maybe fourteen reads a second at peak. A hundred thousand drivers reporting every four seconds is twenty-five thousand updates; fifty thousand ride requests an hour is about fourteen a second. **This is one of the most write-skewed workloads in any common design**, and the write data is completely disposable — a location from four seconds ago has no value.

So the live location store is Redis, not the database. A geospatial sorted set per city, `GEOADD` on every driver ping, with a thirty-second TTL on each driver's entry. A hundred thousand drivers is about ten megabytes and twenty-five thousand operations a second is comfortable for one instance. **If the whole store is lost, it rebuilds itself in four seconds**, so it needs no replication for durability — only for availability.

The TTL is load-bearing. A driver who closes the app stops reporting and disappears automatically, so ‘currently available’ needs no separate heartbeat mechanism and no reaper job.

The read path. A rider requests a car at a point. `GEOSEARCH ... BYRADIUS 2 km ASC COUNT 20` gives me the nearest twenty candidates with distances, boundary handling and spherical maths already done. **I would not hand-roll geohash prefixes here** — Redis's implementation is exactly that, done correctly, and reimplementing it is how you get the boundary bug.

Then the part the index does not do: ranking. Straight-line distance is not the right answer — a driver 800 metres away across a river is worse than one 1.5 kilometres away on the same road. So the candidate set from Redis is scored on estimated time of arrival, driver rating, and whether they are already finishing a trip nearby. **The index narrows ten thousand to twenty; the business logic picks one of the twenty**, and conflating those two steps is the common mistake.

Sharding is by city, and that is the nice property of this domain: nobody requests a ride across cities, so there is never a cross-shard query. A geo-partitioned Redis cluster keyed by city id, with the API routing on the request's coordinates.

What goes to the durable database is the trip — request, matched driver, start, end, route, fare — which is a handful of writes per trip, so a few hundred a second. Completely different volume, completely different durability requirement, and a completely different store.

Two things I would flag as harder than they look. First, **matching is not just search** — two riders must not be offered the same driver, so there is a reservation step with a short lock or a conditional write, which is the [day 127](#) distributed-lock problem and needs the fencing discussion. Second, **surge and supply balance are the actual product**, and they need aggregate density per cell rather than per-driver lookups — which is where H3 earns its place, because equal-area hexagons make ‘demand per cell’ comparable across the city in a way that latitude-distorted geohash cells do not.

And what I would not build: a quadtree. The data moves constantly, and a structure that splits and merges on every update is the wrong shape for twenty-five thousand writes a second. Quadtrees are for the static half of this problem — the restaurants, not the drivers.”

9 Recall card

A B-tree indexes one dimension and a location is two — an index on latitude gives you a band round the planet, and a composite `(lat, lng)` index does not help because latitude is a range, not a value.

The move is to reduce 2D to 1D preserving nearness. A geohash interleaves the bits so a **shared prefix = a shared box** (precision 6 $\approx 1.2 \times 0.6$ km, precision 7 ≈ 150 m), and “near me” becomes a prefix range scan.

The boundary problem is not an edge case. Points metres apart can share no prefix, so you query the cell **and its eight neighbours** — for a 500 m radius on precision-6 cells, $\sim 70\%$ of query points need it.

Geohash: fixed cells, trivial updates, distorted by latitude $\rightarrow$ moving objects. Quadtree: cells adapt to density, mutation is expensive $\rightarrow$ static uneven data. S2/H3: proper spherical cells, H3’s six neighbours equidistant. PostGIS/R-tree: no boundary problem, no tuning, lower write ceiling.

The index narrows; it never answers. A cell is a box and the query is a circle, so an exact distance filter always follows. And for live locations, **the write path is ephemeral** — Redis with a TTL, rebuilt in seconds if lost, while trips go to the durable store.

DSA topic: Graphs revision and mock round **System design topic:** Geospatial indexing: geohash and quadrees

Code these, in this order

This is a closing day, so the rule is different: **write every one of these from an empty file, with the lesson closed.** If you have to look something up, finish the problem, then write that template out three times from memory before starting the next one.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Number of Islands	LeetCode 200 (Medium)	Grid as a graph, iterative, mark on push.
2	Course Schedule II	LeetCode 210 (Medium)	Kahn's, edge direction, build from <code>range(n)</code> .
3	Network Delay Time	LeetCode 743 (Medium)	Dijkstra, <code>(cost, vertex)</code> , <code>max</code> of the distances.
4	Accounts Merge	LeetCode 721 (Medium)	The shared-property-as-vertex move.
5	Most Stones Removed	LeetCode 947 (Medium)	Mock problem 1, timed.
6	Min Cost to Connect All Points	LeetCode 1584 (Medium)	Mock problem 2's shape — dense Prim's.

Run the mock round properly

Twenty minutes each for problems 5 and 6, with a timer, standing up, talking through-out. Then listen back for three things:

1. Did you say the two modelling sentences before writing anything?
2. Did you give the cost without being asked?
3. When you were stuck, did you keep talking and offer a correct-but-slow fallback?

The third one is what actually fails interviews.

Then the recognition drill, cold

Ten problem statements, no solving. For each, write the two sentences, the four classification words, and the algorithm — in under ninety seconds each.

1. “Given a list of dominoes, can they be arranged in a line so touching ends match?”
2. “Given exchange rates, is there a sequence of trades that ends with more than you started?”
3. “Given people and who reports to whom, how long until a message reaches everyone?”
4. “Fewest single-letter changes from one dictionary word to another.”
5. “Given a set of tasks with durations and dependencies, when is everything done?”
6. “Given a grid of costs, cheapest path from corner to corner.”
7. “Split the students into two classrooms so no two who fight are together.”
8. “After each new cable is laid, how many separate networks are there?”
9. “Which cables could be removed without disconnecting anything?”
10. “How far is every house from the nearest fire station?”

Two of the ten are the same problem in different clothes. Say which.

Then the eight-bugs drill

Write a small graph program and deliberately introduce each of the eight bugs, one at a time. Record the exact output or error for each. **Five produce no error**, and seeing that yourself is worth more than reading it.

Then the templates, timed

From an empty file:

1. `build` plus the grid `neighbours` function. Target: two minutes.
 2. BFS with distances and parents. Target: three minutes.
 3. Three-colour cycle detection. Target: three minutes.
 4. Kahn’s. Target: three minutes.
 5. Dijkstra with lazy deletion. Target: four minutes.
 6. Union-Find with both optimisations. Target: four minutes.
 7. Kruskal’s on top of it. Target: two minutes.
-

The modelling drill

1. Give the two sentences you say before any code.
2. Give the four classification words and say what each decides.
3. State the recognition question in one line.
4. Recite the tells table: ten phrasings and their algorithms.
5. Name the three modelling moves and give a problem for each.

The one-shape drill

1. Write the eight-line traversal skeleton from memory.
2. Give the four `TAKE` / `PUSH` substitutions and the algorithm each produces.
3. Say where the mark goes and why.
4. Say what changes to make it multi-source.

The costs drill

Recite the whole table from memory, then check:

1. BFS, DFS, multi-source, 0-1 BFS, bipartite.
2. Topological sort, and DAG shortest path with any weights.
3. Dijkstra, Bellman-Ford, Floyd-Warshall.
4. Union-Find, Kruskal, both Prims.
5. Then compute all of them on `V = 100,000, E = 300,000`.

The comparisons drill

1. Multi-source against one-BFS-per-source, with numbers.
2. Traversal against Union-Find for arriving edges, with numbers.
3. Adjacency list against matrix, with numbers.
4. Dijkstra against BFS on an unweighted graph.
5. Say why `E` is the memory and `V` is not.

The eight-bugs drill

For each: the symptom first, then the fix, then whether it errors.

1. Reversed edge direction.
 2. Built from the edge list.
 3. One component only.
 4. Marked on pop.
 5. Undirected edge added once.
 6. “Seen” as the cycle test — both flavours.
 7. Recursion on a long chain.
 8. Missing grid bounds check.
-

The why-not-a-B-tree drill

1. Say why an index on latitude does not work.
2. Say why a composite `(lat, lng)` index does not fix it.

3. Give rows-examined figures for the naive version and the geohash version.
4. State the general move in one sentence.

The geohash drill

1. Say how a geohash is constructed.
2. State the prefix property.
3. Give cell sizes for precisions 4 through 8.
4. Say how you choose a precision from a search radius.
5. Say what the query looks like, including the two filters after the cell lookup.

The boundary drill

1. Explain the boundary problem in two sentences.
2. Give the standard fix and how many lookups it costs.
3. Compute the fraction of query points that need it, for a 500 m radius on precision-6 cells.
4. Say why it is a design error rather than an edge case.
5. Say what H3 does better here, and why.

The quadtree drill

1. Describe insertion and the capacity parameter.
2. Say what property every leaf has, and why that matters.
3. Say what quadtrees cost that geohashes do not.
4. Give the rule for choosing between them.
5. Size a quadtree for 10M points at capacity 100.

The alternatives drill

1. Say what S2 does differently and what the Hilbert curve buys.
2. Give H3's two properties and its one compromise.
3. Say what PostGIS uses and what it removes entirely.
4. Say where PostGIS stops being the right answer.
5. Say what Redis GEO is, underneath.

The numbers drill

1. Compute writes and reads per second for 100,000 drivers reporting every 4 s.
2. Size the Redis memory for that.
3. Give write throughput for Redis GEO and for Postgres with GiST.
4. Give geohash cell width at four different latitudes.

5. Compute candidates and distance calculations for a 9-cell query in a dense city.

The architecture drill

1. Say why driver locations do not belong in the durable database.
2. Say what the TTL does for you, beyond cleanup.
3. Say what does go to durable storage, and at what volume.
4. Say what the index does and does not decide, on the read path.
5. Say why sharding by city is unusually clean here.

The failure drill

For each, say what happens and what you would build:

1. A driver twenty metres away is not returned.
2. A precision tuned for Mumbai, used in a rural district.
3. A quadtree under 25,000 location updates a second.
4. A query that returns candidates but skips the exact distance filter.
5. Location updates written to Postgres with a GiST index at 25,000/s.
6. Two riders matched to the same driver.
7. The entire Redis location store is lost.

Two of the seven are working as designed. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Is this a graph problem? What are the vertices and edges?* The two sentences, the four words, the recognition question, and the three modelling moves — offered as things you check rather than recited.
 2. *Why did you choose that algorithm?* What is being asked, what the edges look like, whether the graph changes, and the size — in that order, with the DAG case named as the one people forget.
 3. *How do you find all drivers within two kilometres?* Why a B-tree fails, the 2D-to-1D move, geohash prefixes with cell sizes, the boundary problem with its 70% figure, the exact distance filter, and the write/read asymmetry that decides where the data lives.
-

Before you move on

- ☐ ☐ I say the two modelling sentences before writing any code.
- ☐ ☐ I can give the four classification words and what each decides.
- ☐ ☐ I can recite the tells table.
- ☐ ☐ I can name the three modelling moves with a problem for each.
- ☐ ☐ I can write the eight-line skeleton and its four substitutions.
- ☐ ☐ I can recite the whole costs table from memory.
- ☐ ☐ I can compute every algorithm's cost on a sparse graph.
- ☐ ☐ I know multi-source against per-source, with numbers.
- ☐ ☐ I know traversal against Union-Find, with numbers.
- ☐ ☐ I know **E** is the memory, not **V**.
- ☐ ☐ I can name all eight bugs and which five are silent.
- ☐ ☐ I wrote all seven templates from an empty file, timed.
- ☐ ☐ I ran both mock problems out loud with a clock.
- ☐ ☐ I kept talking when I was stuck.
- ☐ ☐ I can say why an index on latitude does not work.
- ☐ ☐ I can say why a composite index does not fix it.
- ☐ ☐ I can explain geohash construction and the prefix property.
- ☐ ☐ I know cell sizes for precisions 4 to 8.
- ☐ ☐ I can choose a precision from a radius.
- ☐ ☐ I can explain the boundary problem and quantify it.
- ☐ ☐ I know the nine-cell query is the standard, not a workaround.
- ☐ ☐ I can describe a quadtree and its capacity parameter.
- ☐ ☐ I know the rule for geohash against quadtree.
- ☐ ☐ I know what S2 and H3 fix, and H3's two properties.
- ☐ ☐ I know what PostGIS removes and where it stops scaling.
- ☐ ☐ I know the index narrows and never answers.
- ☐ ☐ I can compute the write/read asymmetry for ride-hailing.
- ☐ ☐ I know why live locations are ephemeral and what the TTL buys.
- ☐ ☐ I answered all three questions above out loud.

The nine sections, and what each is for

Every lesson in this book, on both tracks, carries the same nine sections in the same order. This is what each one is for, and how to read it.

1. What this is, and why they ask it

The topic in plain words, and the reason an interviewer spends time on it. This section exists so you never learn something without knowing what it buys you.

2. The story

A scene with a person in it, told in ordinary language, with no technical words at all. Two hundred to four hundred words about matching socks, or a canteen queue, or laying tables.

This is the most important section for a beginner and the easiest to skip. The story gives you a place to put the idea before you have the vocabulary for it. Later, when you are asked the question under pressure, the story is what you will reach for first.

3. The idea in plain English

The story, translated. Every term is defined the first time it appears, and defined terms are set in **bold** so you can find them again. Appendix D collects them all.

4. The picture

A diagram. Arrays, memory and pointers are drawn as ASCII boxes with indices above and values below, because adjacency matters and a box drawing keeps it. Trees, graphs, architectures and request flows are drawn as proper figures.

Every diagram is followed by a line telling you what to notice in it.

5. The code, built step by step

Fragments of ten lines or fewer, each one followed by prose explaining it. Then the complete solution in one block, ready to run.

The full answer is always here. This book does not withhold solutions.

6. What it costs

The arithmetic, done where you can see it.

On the DSA track that means counting the loop iterations out loud: not “this is quadratic” but “the inner loop runs sixty times for each of sixty items, so about three thousand comparisons”. On the system design track it means doing the multiplication: not “that is a lot of storage” but $50\text{M} \times 20 \times 2\text{KB} = 2\text{TB}$.

7. The traps

The mistakes people actually make, with the real error text printed exactly as it appears on screen. When your own program breaks this way, you will recognise it.

8. In the interview

The point of the whole document.

Real interviewer phrasings. A script for the first ninety seconds, because that is when candidates either take control of the question or lose it. The three follow-ups that usually come next. And a model answer, written out in full, so you have something to measure yours against.

9. Recall card

The two or three sentences that are the entire lesson. Say them aloud, from memory, in full sentences.

If you can do that, you have finished the lesson. If you cannot, you have read it, which is a different thing.

Recall cards

Section 9 of every lesson in this volume, collected. Cover the card, say it aloud, then check. Cards are grouped by the phase they belong to, on both tracks.

Building blocks of big systems

Day 129 · system-design — Message queues: why async changes everything

Async means the caller hands over the work and leaves. Latency goes from the sum of everything to the time to write one message — 2,400 ms to 86 ms on a typical signup — and availability stops being the product of every dependency's.

A queue absorbs bursts; it does not create capacity. If arrivals exceed service rate the queue grows without limit. Size consumers at about $2 \times$ **average**, because drain rate is $M - L$, not M . Alert on **oldest-message age**, not depth.

Four things you give up: the answer (so design what the user sees); ordering (per key only — SQS FIFO caps at 300/s per group); at-most-once (delivery is at-least-once, so **every consumer must be idempotent**); and loud failures (so correlation IDs and a dead-letter queue from day one).

Visibility timeout is the classic bug: a job longer than the timeout gets redelivered while still running, and the work happens twice with no crash. Extend it while working, or rely on idempotency.

Work → task queue (SQS, RabbitMQ). Events → log (Kafka), when several consumers want the same thing or replay matters. And the rule for what goes in a queue: *does the user need the result to continue?*

Day 130 · system-design — Kafka, explained

Kafka is a log, not a queue. Messages are appended, kept for a retention period, and **reading does not remove them** — each consumer group tracks its own offset, so many teams read the same data and any of them can rewind.

Ordering is per partition only. The message key is hashed to choose a partition, so key by entity to get per-entity ordering. **Ordering and parallelism trade directly**, and a skewed key gives a hot partition that no amount of extra brokers fixes.

A consumer group is one logical reader. Each partition goes to exactly one member, so **more consumers than partitions means idle consumers** — the partition count is a hard ceiling, chosen early and awkward to raise.

`acks=all`, `replication.factor=3`, `min.insync.replicas=2` for anything that matters, plus an idempotent producer. `min.insync.replicas=1` looks safe and silently permits data loss.

Monitor consumer lag as a time, and alert long before it reaches retention — a consumer that falls outside the window loses messages with no error at all. And Kafka gives you no per-message retry or DLQ: you build retry topics yourself, or use a task queue for work.

Day 131 · system-design — Publish-subscribe versus point-to-point

Point-to-point: one message, one consumer, consumers compete. Pub-sub: one message, every subscriber gets a copy, and **the publisher does not know who is listening** — which is the whole point.

The smell of the wrong choice: the producer holds a list of its consumers, so adding a consumer means deploying the producer. Adding a consumer to a queue makes the system *faster*; adding a subscriber to a topic makes it do *more work*.

Fan-out is the production shape: publish once to a topic, one queue per consumer. Pub-sub decoupling plus per-consumer retries, DLQs and scaling. SNS → SQS, or a Kafka topic with one consumer group per team.

Kafka does both with consumer groups: *within* a group consumers compete; *between* groups every group sees everything — and it stores one copy where SNS→SQS stores five.

Redis pub-sub is fire-and-forget — delivered only to whoever is connected right now, lost silently otherwise. Fine for cache invalidation, wrong for anything that matters; Redis Streams is the durable version. And: **publish events (facts, past tense, many listeners), send commands (imperatives, one handler).**

Day 132 · system-design — Stream processing basics

Stream processing is computing over data that never ends, and the whole difficulty is that events arrive out of order. Bucket by **event time** (when it happened, reproducible), never processing time (when you found out, depends on your network).

A watermark is a promise: “I have seen essentially everything before time T.” Usually `max event time seen - a fixed allowance`, and the allowance comes from measuring the actual delay distribution — set it above p99. It only advances when events

arrive, so **one idle partition freezes the whole job's output**.

Late events need a stated policy: drop with a counter, fold into the current window, or keep state for an **allowed lateness** and emit a correction — which forces every sink to upsert by window rather than append. Send them to a side output either way, so you can count them.

Windows: tumbling (one window per event, cheapest), sliding (multiplies state and CPU by the overlap — a 5-minute window every 10 seconds is $30\times$), session (ends on a gap).

Latency is dominated by the watermark lag you chose, not by the engine. And the way you know the numbers are right is a nightly batch recount over the raw log, published as a percentage difference.

Day 133 · system-design — Object storage, S3-style

Object storage is a cloakroom, not a file system. Whole objects under a flat key namespace — the slashes in `photos/2026/a.jpg` are just characters. **No append, no partial write, no rename;** changing anything means rewriting the whole object.

Durability 11 nines; availability 4 nines; first byte in 20–100 ms. Durable $\neq$ available, and it is not a backup — versioning and cross-account replication protect against *your own* mistakes.

Presigned URLs so bytes never touch your servers; multipart for large files; a CDN in front, always. Drive post-upload work from the store's own event, not from the client.

Storage is cheap and egress is the bill: $\sim \$0.023/\text{GB}/\text{month}$ to store, $\$0.09/\text{GB}$ to send. A photo app at 40 TB pays $\sim \$900$ for storage and $\sim \$54,000$ for egress before a CDN.

Database holds the reference, store holds the bytes — and a reconciliation job finds the orphaned objects and the broken links, because there is no transaction across the two.

Day 134 · system-design — Blob storage versus storing files in the database

Database holds the reference; object storage holds the bytes. The four numbers: **backup/restore** (4 TB means ~ 6 hours instead of ~ 2 minutes), **buffer pool** (one 4 MB read evicts $\sim 1,000$ row pages, so *other* queries get slower), **connections** (a 50 MB stream holds one for seconds; presigned URLs use zero), and **cost** ($\sim \$0.115/\text{GB}$ vs $\sim \$0.023/\text{GB}$, before the bigger instance).

What you give up is atomicity across two systems — orphans and broken links, plus non-atomic deletes. The fix: **create the row first in PENDING**, drive completion from the store's event, and run a reconciliation job where "READY row with no object" is an alert.

The minority answer is real: files under ~100 KB with a few GB total and a genuine consistency requirement — and SQLite, where blobs under 100 KB actually read faster from the database than from files.

Postgres TOAST stores large values out-of-line and compressed and does not read them unless selected — so it weakens the cache argument and **nothing else**. Know that, and you know which arguments survive.

Serving is the argument that settles it: bytes in a database go through your app; bytes in a store go browser → CDN → store, using none of your connections or bandwidth.

Day 135 · system-design — Search: how a search index actually works

An inverted index maps terms to sorted lists of document ids (postings), so a query is a lookup and a multi-word query is a linear merge of sorted lists. `LIKE '%x%'` cannot use a B-tree at all — leading wildcard — so it scans everything: ~500 ms per query on a million rows against ~2 ms.

The analysis pipeline must be identical at index time and query time: tokenise, lowercase, stop words, stem. A mismatch returns nothing for correct queries, with no error.

Ranking is what makes it search rather than filtering. BM25 = rare terms count more (IDF), term frequency **saturates**, shorter fields count more. Tune field boosts, not the parameters — and layer business rules on top, because that is where the product is.

The index is a second copy and it goes stale. Elasticsearch refreshes ~1 s by default. Sync via the **outbox** or CDC, never dual write; **search returns ids and you hydrate from the database**; and run a reconciliation job, because it will drift.

Postgres full-text search (`tsvector` + GIN) up to a few million documents — transactionally consistent, one fewer system. Beyond that, or for facets, typo tolerance and real BM25, a dedicated engine — and budget memory comparable to the index size.

Day 136 · system-design — Elasticsearch in a design

Elasticsearch is a derived copy, never the source of truth, and must be rebuildable from the database in an afternoon. If something lives only there, the design is wrong.

Write path: row + outbox in one transaction → consumer → bulk index. Never dual-write from the application — no transaction across two systems, and the inconsistency is permanent and silent. CDC is the alternative.

Read path: search returns ids; the application hydrates from the database. A stale index then shows a slightly wrong *set* but never wrong *content*, and deleted items filter themselves out.

Shards are fixed at creation and are read-path overhead (every query touches all of them); replicas are the read-throughput lever and can change any time. Size shards at 10–50 GB, keep heap ≤31 GB and ≤50% of RAM — **the other half is page cache, which is where search speed comes from.**

Query the alias, never a concrete index, so a mapping change is a reindex plus an atomic switch instead of an outage. `text` for searching, `keyword` for filtering and aggregating; put non-relevance conditions in `filter`; and run a reconciliation job, because the index *will* drift.

Day 137 · system-design — Time-series and metrics stores

Time-series data is append-only, arrives in order, has similar consecutive values and regular timestamps, and is queried in ranges. Each fact buys an optimisation: immutable time-chunks, delta-of-delta timestamps (one bit on a fixed interval), XOR-compressed floats. **16 bytes per point becomes ~1.5.**

Cardinality kills these systems, not point volume. Every distinct label combination is a series, and memory scales with **series count**. Never label with a user id, request id, or a path containing ids — that detail belongs in traces and logs.

Downsample and tier: raw for ~15 days, 1-minute for 90, 1-hour for 2 years — about **18× less storage**, and it is what makes long-range queries possible at all. **Store min/max/sum/count, not just the mean.**

Percentiles do not average. Use histograms, sum the buckets across hosts, then compute the quantile — and remember each bucket is a series, so histograms multiply cardinality by the bucket count.

Plain Postgres tops out around 10–50k rows/s against ~1M for a purpose-built store, from index maintenance and ~24-byte row headers. Fine at small volume with time partitioning; **TimescaleDB when you need to join metrics against relational data.**

Day 138 · system-design — Data warehouses: OLAP versus OLTP

OLTP: many small operations, few rows, most columns, milliseconds. OLAP: few huge operations, millions of rows, two or three columns, seconds. Opposite in every dimension, and the difference is the **physical layout**, not tuning.

Row store reads every field to get one: $40\text{M rows} \times 400 \text{ B} = 16 \text{ GB}$ to sum a column that needs 320 MB. **Column store reads only that column, compressed 5–10×** → ~80 MB, plus vectorised execution and zone maps. ~100× less I/O, and partition pruning often beats all of it.

A read replica fixes the *harm (CPU, locks, buffer-pool eviction)* and not the query — still a row store, still minutes, and now it lags. It is the right first step and the wrong final answer.

Star schema: one enormous thin fact table plus small wide dimensions, denormalised on purpose. Slowly changing dimensions when history must be right. **ELT with CDC** is the modern pipeline; transformations form a DAG run in topological order.

Storage is never the cost — compute is (\$0.06/month for the data, ~\$2,900 for a cluster left running; a 5 TB unpartitioned hourly query is \$18,000/month on per-byte pricing). **And under ~100 GB, a replica with indexes or DuckDB over Parquet is the honest answer.**

Day 139 · system-design — ETL, batch pipelines, and where data goes to be counted

A pipeline is a DAG and running it is a topological sort — levels give the critical path and the useful worker count. **ELT** (load raw, transform in the warehouse) over **ETL**, because you cannot re-transform what you threw away.

Two properties make it survivable: partition by time, and make every write idempotent — overwrite, never append. Jobs take the **execution date** as a **parameter**, never `now()` ; a job using `now()` can be neither re-run nor backfilled.

Those two give you backfills for free: 180 days is 180 independent runs, parallelisable — 5 days sequentially, 4 hours at thirty at a time. The limit is warehouse capacity, not the orchestrator.

Late data needs a stated policy: reprocess a window sized from the measured p99 (correct per-day, but published numbers change) or fold forward (stable, individual days wrong). **Partition by event time, not arrival time.**

A pipeline that runs successfully can still be wrong. Assert on the data — row counts against the trailing average, no nulls in keys, **counts before and after every join** (a duplicate key silently doubles revenue), totals reconciled against the source — and

fail the run rather than warn. Alert on the SLA, not on jobs.

Day 140 · system-design — Websockets, long polling, and server-sent events

Four mechanisms: polling (simple, stateless, ~99% waste, latency and load locked together), long polling (instant, holds a connection per waiting client), **SSE** (one HTTP stream, server→client only, and it gives you **automatic reconnect plus Last-Event-ID** free), **WebSockets** (two-way, ~6 bytes per message, and everything else is yours to write).

Choose on direction and frequency: client sends often → WebSockets; server only → SSE; rare updates → polling, and defend it.

Persistent connections make a stateless fleet stateful. A user is bound to a machine, so cross-server delivery needs a **pub-sub backplane on per-user or per-room channels** — a firehose makes every server discard 97% of everything.

Push is not delivery. Persist the message first and treat the push as a notification; every client carries a **cursor** so a reconnect resumes from the last id. Subscribe-then-backfill-then-replay closes the gap, and clients deduplicate by id.

Deploys are the incident. A million clients reconnecting in one second takes down the replacements — **jittered reconnect shipped from day one**, connection draining, and rolling one server at a time. And when the app is closed, none of this applies: that is a platform push notification.

Day 141 · system-design — Push notifications, end to end

You cannot reach the device. The OS holds one connection to APNs/FCM shared by every app, and that is the only channel. Three hops — your server → platform → OS → app — and **you control only the first**.

A push is a nudge, not the message. Content lives in your store; the payload carries ids and a preview (4 KB limit). There is **no delivery guarantee and no delivery report** — “accepted” means accepted, and `delivered` is not a state you can observe.

Tokens go stale silently. The **only** invalidation signal is a `410 Unregistered / UNREGISTERED` when you send — delete on it, re-register on every app open, prune by `last_seen`, and **delete on logout**, or a resold device gets someone else’s messages.

Fan-out is your capacity, not the platform’s: ~200 req/s per worker with persistent HTTP/2, so 10M devices is ~100 s at 500 workers. **Chunk tokens by key range, never LIMIT/OFFSET**. Untended tokens churn 30–50% a year, so most of a send eventually goes nowhere.

Fatigue is the real failure mode: collapse keys, per-user per-category caps, quiet hours, digests, and an expiry. Opt-in is ~40–60%, disabling is permanent, and **sending more shrinks the audience rather than growing engagement.**

Day 142 · system-design — Geospatial indexing: geohash and quadtrees

A B-tree indexes one dimension and a location is two — an index on latitude gives you a band round the planet, and a composite `(lat, lng)` index does not help because latitude is a range, not a value.

The move is to reduce 2D to 1D preserving nearness. A geohash interleaves the bits so a **shared prefix = a shared box** (precision 6 $\approx 1.2 \times 0.6$ km, precision 7 ≈ 150 m), and “near me” becomes a prefix range scan.

The boundary problem is not an edge case. Points metres apart can share no prefix, so you query the cell **and its eight neighbours** — for a 500 m radius on precision-6 cells, ~70% of query points need it.

Geohash: fixed cells, trivial updates, distorted by latitude → moving objects. Quadtree: cells adapt to density, mutation is expensive → static uneven data. S2/H3: proper spherical cells, H3’s six neighbours equidistant. PostGIS/R-tree: no boundary problem, no tuning, lower write ceiling.

The index narrows; it never answers. A cell is a box and the query is a circle, so an exact distance filter always follows. And for live locations, **the write path is ephemeral** — Redis with a TTL, rebuilt in seconds if lost, while trips go to the durable store.

Distributed systems core

Day 125 · system-design — Retries, backoff, and thundering herds

Retry only transient failures. Timeouts, `429`, `5xx`, connection resets. Never `400` / `401` / `403` / `404`. And never retry at all unless the operation is idempotent — a timeout does not mean it did not happen.

Amplification compounds through layers: $3 \times 3 \times 3 = 27$ calls to the thing that was already failing. Retry at exactly one layer and set the others to zero.

Backoff spaces the herd out; only jitter breaks it up. Plain exponential backoff still lands a thousand clients in the same instant. Full jitter — `random.uniform(0, min(cap, base × 2attempt))` — cuts peak load by hundreds of times, and AWS measured it beating every cleverer scheme.

A retry budget is the control that scales: retries capped at ~10% of the request rate. $1.1 \times$ load in an incident instead of $4 \times$, with no tuning and no operator.

The other herds are the same bug: cache stampede (jittered TTL + coalescing), restart herd (staggered rollout), cron herd (`hash(id) % 60`), reconnect herd (jitter in the client, shipped from day one).

Day 126 · system-design — Circuit breakers and bulkheads

The failure is the caller, not the dependency. 200 workers, a 30-second dependency used by 10% of a 1,000/s stream, and the pool is exhausted in **2 seconds** — including the 90% of requests that never touch it.

Breaker = three states. Closed (counting), open (fail instantly), half-open (probe after cooldown; one failure re-opens). Threshold is a **rate over a recent window with a minimum call volume**, never an absolute count. Count slow calls as failures.

Bulkhead = a bounded pool per dependency, and it must reject rather than queue. Size with Little's Law — arrival rate $\times$ p99 latency — never for the failure case. It costs utilisation and buys isolation.

Order in the request path: bulkhead $\rightarrow$ breaker $\rightarrow$ retries $\rightarrow$ call. Retries live inside the breaker, so attempts count towards opening it and an open breaker stops them running.

A breaker is not finished until you have decided what it returns. Stale data, degraded response, queued write, sensible default, fast error — in that order. Never fall back on the answer itself.

Day 127 · system-design — Distributed locks

Acquire atomically with a lease in one command: `SET key <unique-token> NX PX 300000`. Separate `SET` then `EXPIRE` leaves a permanent lock on a crash; `GET` then `SET` is not a lock at all.

Release must check ownership, atomically — a Lua script, not `GET` then `DEL` — or an expired holder deletes somebody else's lock.

The unfixable failure: a paused holder. The lease expires, another client acquires, the first wakes up still believing it holds the lock. You cannot distinguish paused from dead, and no timeout setting avoids both failure directions.

So fence instead of preventing. Monotonic token per grant, carried on every write, and **the resource** rejects anything below the highest token it has seen. A wasted handover instead of corrupted data.

Redis for efficiency locks (fast, $\sim 100k$ ops/s, can double-grant); **ZooKeeper or etcd for correctness locks** (ephemeral nodes release on crash, sequence numbers are free fencing tokens). And before any of it, look for a unique constraint, a condi-

tional update, or a single owner per key — a lock is what you use when the storage layer cannot express the rule.

Day 128 · system-design — Distributed systems revision and interview questions

One problem: partial failure, unobservable. Dead, slow, and unreachable look identical from outside. Every other idea in the phase is a response to that.

Eight responses: suspicion not truth (detection); a timeout is an unknown (idempotency); causality not clocks; CAP only bites during a partition; agreement costs a majority (quorum, Raft); atomicity across services becomes compensation (saga, outbox); backoff with **full jitter** and a budget; bound everything (bulkhead) and fail fast (breaker).

The stance that matters more than the mechanisms: do not prevent failure, make it survivable. Fencing, idempotency, compensation, expiry jobs, reconciliation.

Reach in this order: remove the coordination (one owner, one constraint) → bound the damage → make being wrong survivable. Consensus is a strong move, not a default.

Have the numbers ready: 0.5 ms same-datacentre / 80 ms cross-region, skew 0.1–5 ms local and 10–100 ms remote, Raft 150–300 ms, 3 retries at 10% budget with full jitter, breaker at 50% over ≥ 20 calls, $R + W > N$, $27 \times$ amplification, and the two-second pool exhaustion.

Graphs

Day 125 · dsa — What a graph is, and where graphs hide

A graph is things plus pairwise connections. Vertices and edges. V and E , and you state both.

Four questions, always, before any code: directed or undirected? weighted or unweighted? can it have cycles? is it connected? Getting the first one wrong gives a confidently backwards answer, not a crash.

Say two sentences out loud before writing a line: “A vertex is . *An edge from A to B means* .” If you cannot finish them, you do not understand the problem yet.

Graphs hide in grids (cells and adjacency), prerequisites, word ladders, puzzle states and shared properties. An **implicit** graph computes its neighbours instead of storing them — that is the form people fail to spot.

Every traversal is $O(V + E)$ — a sum, not a product — and every one needs a **seen** set, because unlike a tree, a graph has cycles. E is the memory, not V .

Day 126 · dsa — Representing a graph: adjacency matrix versus adjacency list

Matrix is $V \times V$ cells, always. **List** is $V + 2E$ entries. For $V = 1,000$ and $E = 3,000$ that is a million against seven thousand — $140\times$ — and the gap grows with V .

Three operations decide it: edge test is $O(1)$ on a matrix and $O(\text{degree})$ on a list; neighbours are $O(V)$ on a matrix and $O(\text{degree})$ on a list; memory is $O(V^2)$ against $O(V + E)$. Traversals only ever ask the second one, so **traversal is $O(V^2)$ on a matrix and $O(V + E)$ on a list.**

Use a list by default. Use a matrix when the graph is genuinely dense, when V is under a couple of hundred, or when the algorithm is defined on a matrix (Floyd-Warshall).

Need $O(1)$ edge tests on a sparse graph? Store neighbours as sets, not a matrix. $O(1)$ membership at $O(V + E)$ memory.

The bug: `[[0] * n] * n` makes one row referenced n times. One assignment changes every row, and the symptom looks like a traversal bug. Always `[[0] * n for _ in range(n)]`.

Day 127 · dsa — Breadth-first search on a graph

BFS visits in rings, and the first time it reaches a vertex, that is the fewest edges. Because ring k finishes entirely before ring $k+1$ begins. **True only when every edge costs the same** — weights mean Dijkstra.

Nine lines: `deque`, a `seen` set, **pop the front, push unseen neighbours**. Never `list.pop(0)` — that is $O(n)$ and makes the whole thing quadratic.

Mark on push, not on pop. It is what makes “each vertex enters the queue at most once” true, and that sentence is the complexity proof. Marking on pop gives a queue of size E and a memory error on a dense graph.

One dictionary does three jobs: `distance` (or `parent`) is also the seen set, and its keys are the reachable set. Walk `parent` backwards and reverse for the actual path.

$O(V + E)$ time — a sum — and $O(V)$ space, where the queue’s peak is the widest level. And if the question says “all” or “how many groups”, wrap it in an outer loop over every vertex, because the graph is probably not connected.

Day 128 · dsa — Depth-first search on a graph

DFS goes deep, then backs up. Recursively: mark on entry — first line, not after the loop — then recurse into unseen neighbours. Iteratively: it is BFS with `stack.pop()` instead of `queue.popleft()`.

Same $O(V + E)$ as BFS. The difference is what you know while running: DFS has a **current path**, BFS has rings. Fewest steps $\rightarrow$ BFS. Cycles, topological order, all-paths, structure $\rightarrow$ DFS.

Three colours for directed cycles. White unvisited, **grey = on the current path**, black finished. An edge to grey is a cycle; an edge to black is a shared dependency and is fine. Two states report a cycle on a plain diamond.

Undirected is a different check: every edge appears in both adjacency lists, so exclude the vertex you came from rather than tracking colours.

Write it iteratively when `n` can exceed a few thousand. ~ 960 usable frames by default, and `setrecursionlimit` turns a `RecursionError` into a segfault. And always loop over every vertex — a directed cycle can hide in a region nothing points into.

Day 129 · dsa — Connected components

Components are the traversal plus four lines: loop over **every** vertex, and when one is unseen, count a new component and flood from it. The `seen` set lives outside the loop — resetting it inside returns `V`.

Cost is $O(V + E)$, the same as one traversal, not V of them, because the shared `seen` set means every vertex and edge is touched once across all the starts.

Label instead of counting when you need sizes or “same group?” — one dictionary does the seen set, the labels and the count.

Build from `range(n)`, not from the edge list. A vertex with no edges is a component of size one, and a `defaultdict` built from edges silently loses it.

Static graph $\rightarrow$ traversal. Edges arriving over time $\rightarrow$ Union-Find ($m \times 4$ instead of $m \times (V + E)$). And on a directed graph, “components” is ambiguous — ask whether they mean weakly or strongly connected.

Day 130 · dsa — Grids are graphs: islands and flood fill

A grid is a graph: a cell is a vertex, touching cells are edges, and nothing is stored — the neighbours are computed from the coordinates. Substitute `neighbours(r, c)` for `graph[v]` and every graph algorithm works unchanged.

The bounds check is four comparisons and all four matter. `grid[-1]` is the last row, so a missing check wraps the grid instead of crashing — a wrong answer, not an error.

Write it iteratively and mark on push. A million-cell island is a million frames deep (`RecursionError` , or a segfault if you raise the limit), and marking on pop lets the stack reach four times the cell count.

Cost is $O(\text{rows} \times \text{cols})$. On an open grid, BFS's queue holds one ring ($\sim \text{rows}$ cells) and DFS's stack can hold the whole snaking path — the opposite of the general graph case.

Multi-source BFS is one line — push every source before the loop — and costs the same as one source. The tells are “nearest”, “any of these”, or “how long until everything”. And when the condition is about reaching the edge, **start from the edge**.

Day 131 · dsa — Shortest path in an unweighted graph

Every edge costs the same → shortest means fewest edges → BFS, and the first time BFS reaches a vertex is by the fewest possible edges.

The proof in one sentence: ring `k` finishes before ring `k+1` begins, so a shorter route would already have been found. That step is exactly what breaks when edges have different costs.

Path: keep a `parent` map — it is also the seen set — and walk it backwards from the goal, then reverse. Say “a shortest path”, because ties are resolved by adjacency order.

Counting all shortest paths is one `elif`: when an already-seen vertex has `distance = distance[current] + 1`, add the counts. Still $O(V + E)$; *listing* them can be exponential.

Bidirectional halves the exponent — $2 \times b^{(d/2)}$ instead of b^d , four million times fewer on `b=200, d=6`. Always expand the smaller frontier, and on a directed graph the backward search needs **reversed** edges.

Day 132 · dsa — Cycle detection in an undirected graph

A cycle returns to a vertex without reusing an edge. In an undirected graph every edge is stored twice, so “already seen” fires once per edge and is not a cycle — it is the edge you walked in on, viewed from the other end.

The condition is `seen AND not the parent`. Pass the current vertex down as the child's parent (DFS) or keep a parent map (BFS). Test with two vertices and one edge: the answer must be `False`.

Directed is a different algorithm. No mirror edges there, so the parent check does nothing; you need three colours, where **grey means on the current path**. Establish which kind of graph you have first.

Union-Find detects a cycle without traversing: an edge whose endpoints are already in the same group closes one. Effectively $O(E)$, smaller constant memory, works **online** as edges arrive — and it catches parallel edges that the parent check misses.

The free check: a connected acyclic graph has exactly $V - 1$ edges. $E > V - 1$ means a cycle, with no traversal. And once $E = V - 1$, checking connectivity alone decides “is this a tree”.

Day 133 · dsa — Cycle detection in a directed graph

Three colours: white untouched, grey = started and not finished, so on the current path, **black = finished.** An edge to grey is a cycle; an edge to black is a shared dependency and is fine.

Two states report a cycle on a diamond — $0 \rightarrow 1 \rightarrow 3$, $0 \rightarrow 2 \rightarrow 3$ — which is the shape every real dependency graph is full of. If you use sets instead of colours, the missing line is `on_path.remove(vertex)` on the way out.

Yesterday's parent check is wrong here, in both directions: there are no mirror edges for it to suppress, and it silently misses a genuine $A \rightarrow B \rightarrow A$.

Kahn's is usually the better answer: repeatedly take a vertex with in-degree zero; if fewer than n come out, the leftovers are stuck in or behind a cycle — and the emitted order is a valid schedule, which is the follow-up. Same $O(V + E)$, no recursion.

Loop over every vertex (a directed cycle hides easily in an unreachable region), **build from** `range(n)`, and write it iteratively or use Kahn's when n can exceed a few thousand.

Day 134 · dsa — Topological sort

A topological order puts every vertex before everything it points to. It exists **iff** the graph is acyclic, and there are usually **many** valid orders — verify the property in tests, never a specific list.

Kahn's: in-degree zero into a queue; pop, output, decrement; anything reaching zero is queued. If fewer than n come out, the leftovers are in a cycle. **Prefer it:** structural cycle detection, no recursion, parallel schedule for free, and it reads like the domain.

DFS version: append on *finish*, then reverse — and it needs the three-colour check bolted on, or it returns a confident wrong answer on a cyclic graph.

Process one whole level at a time and you get the parallel schedule. Number of levels = critical path; widest level = maximum useful workers. Same $O(V + E)$, strictly more information.

Two silent bugs: the edge direction reversed (valid order of the wrong graph), and building the structure from the edge list so isolated vertices vanish — which reports a **phantom cycle**. Build from `range(n)` and say the edge's meaning as a sentence first.

Day 135 · dsa — Course schedule and the dependency family

The recognition question: things, plus rules that one must come before another. Then two sentences before any code: “A vertex is ___” and “An edge from A to B means A must happen before B.”

Six shapes, one Kahn's: possible? / an order / the *smallest* order (heap) / a **schedule** (levels, or `finish[v] = duration[v] + max(deps)`) / is it **unique**? (queue never holds more than one) / derive the edges then sort.

Two modelling moves: edges come from **adjacent pairs and the first difference only** — every other position is a false edge; and **two levels of dependency need two nested sorts**.

Any DP over a DAG is one pass in topological order — longest path, shortest path, earliest finish, path counts — because every dependency is final by the time you reach a vertex. Handles negative weights, unlike Dijkstra.

Three silent bugs: reversed edge direction (a valid order of the wrong graph, and identical for the boolean version); building from the pairs so isolated vertices vanish (a **phantom cycle**); and using level count as “minimum time” when durations vary.

Day 136 · dsa — Dijkstra's algorithm

Dijkstra is BFS with a priority queue instead of a queue — expand the cheapest-so-far rather than the oldest. Relaxation `if dist[u] + w < dist[v]` is the only place distances change.

Popped means settled means final, and the three-sentence proof is: a cheaper route would have to leave the settled set through a vertex costing at least as much, and **since no edge is negative**, the remainder cannot bring it below. That last clause is the whole non-negative requirement.

Lazy deletion: no decrease-key in `heapq`, so push again and `if cost > distance[vertex]: continue` on pop. The heap holds up to `E` entries; $O(E \log E) = O(E \log V)$.

Return on the pop, not on the discovery. Discovery gives you a route; popping gives you the cheapest. And `parent[v] = u` goes **inside** the relaxation.

$O((V + E) \log V)$ — about $13 \times$ BFS, so use BFS when weights are equal. Negative weights $\rightarrow$ Bellman-Ford. A stop limit or any extra condition $\rightarrow$ expand the *state*, not the algorithm. And the proof only needs “extending a path never improves it”, so `max` instead of `+` solves the minimise-the-worst-edge family unchanged.

Day 137 · dsa — Bellman-Ford, and what negative edges break

Dijkstra settles a vertex permanently; a negative edge is exactly a reason to unsettle one. That is the proof clause that fails, and it is the answer to “why not Dijkstra”.

Bellman-Ford: relax every edge, $V - 1$ times. No data structure, and an edge list is the right representation. $V - 1$ because a shortest path has at most $V - 1$ edges, and after round k every k -edge shortest path is correct.

One extra round detects a negative cycle — anything still improving can only come from one. Only vertices **reachable from** the cycle are $-\infty$; the rest have valid answers.

$O(V \times E)$ — about $700 \times$ Dijkstra at $V=10^4$, $E=5 \times 10^4$ — so use it only when weights can be negative. Early exit when a pass changes nothing usually terminates in a handful of rounds. SPFA is a practical speed-up with the **same worst case**.

The decision order: DAG $\rightarrow$ one topological pass (any weights); equal weights $\rightarrow$ BFS; 0/1 $\rightarrow$ deque BFS; non-negative $\rightarrow$ Dijkstra; negative $\rightarrow$ Bellman-Ford; all pairs and small $V \rightarrow$ Floyd-Warshall. And with an integer sentinel, the `distance[u]  $\neq$  INF` guard is mandatory.

Day 138 · dsa — Union-Find: the disjoint set union

Two arrays: `parent` (a root points at itself) and `size`. Same group $\Leftrightarrow$ same `find root`. `union` finds both roots and points one at the other.

Two optimisations, and you need both. Path compression — point each node at its grandparent while walking up (path halving, iterative, three lines). **Union by size** — attach the smaller tree under the larger root. Either alone is $O(\log n)$; together it is $O(\alpha(n))$, at most 4 for any real input.

`union` returns a boolean, and that is why it is so reusable: `False` means already connected, which is “cycle”, “redundant edge”, or “skip in Kruskal’s” depending on the problem. Keep a `groups` counter for free component counts.

Static graph asked once $\rightarrow$ traversal. Edges arriving over time $\rightarrow$ Union-Find ($m \times \alpha$ against $m \times (V + E)$ — $75,000 \times$ at $m = V = 10^5$).

It cannot un-merge, cannot give you a path, has no direction, and does not list a group's members. If edges are *removed* and the sequence is known, process it in reverse. And `size` is only valid at the root — read it through `find`.

Day 139 · dsa — Minimum spanning trees: Kruskal and Prim

A spanning tree is $V - 1$ edges, connected, no cycles. Minimum = cheapest total. A cycle is wasted weight, which is why both algorithms are about avoiding one.

Kruskal's: sort all edges, take each one where `union` returns `True`. $O(E \log E)$ — the **sort** dominates; Union-Find is free. Fragments stay disconnected for most of the run, and that is fine.

Prim's: grow one blob, always taking the cheapest edge leaving it. Structurally identical to Dijkstra except for **one line** — Dijkstra pushes `cost_so_far + w`, Prim pushes `w`. Sparse $\rightarrow$ either; dense or a matrix $\rightarrow$ the $O(V^2)$ Prim with no heap.

Why greedy is correct: the cut property. Split the vertices any way; the cheapest crossing edge is in some MST — proved by swapping it for whatever crossing edge the tree used. Both algorithms are repeated applications of it.

Three things to state: an MST is **not** a shortest-path tree (triangle of 1s: MST makes two vertices distance 2 apart); fewer than $V - 1$ edges means **disconnected**, return `-1`; and with tied weights the tree is not unique, so assert the **total**.

Day 140 · dsa — Bipartite graphs and two-colouring

Bipartite = split into two groups with every edge crossing between them. Two-colour with BFS: colour the start 0, neighbours get `1 - colour`, and the conflict is a neighbour with the *same* colour — already coloured is not a conflict.

A graph is bipartite $\Leftrightarrow$ it has no odd-length cycle. Walking a cycle alternates sides, so returning takes an even number of steps. A triangle is the smallest counter-example; every tree is bipartite.

Loop over every vertex — the graph is bipartite only if *every* component is, and a triangle in an unvisited component is the realistic bug. Build the graph in **both directions**; the relation is symmetric.

$O(V + E)$ time and $O(V)$ space — the colour array replaces the seen set, so it costs exactly what a plain BFS costs. Return the two sides, not a boolean, and keep parents so you can show the odd cycle.

Constraints arriving over time $\rightarrow$ Union-Find with parity (effectively constant, versus a traversal each time). **Three colours is NP-complete** — the cliff between 2 and 3 is real, and equal-sized groups is a separate subset-sum problem.

Day 141 · dsa — Multi-source BFS and 0-1 BFS

Multi-source BFS: push every source into the queue before the loop, each at distance zero. One line, and every vertex gets its distance to the **nearest** source — the minimum falls out of the ring order, so there is no comparison anywhere.

It costs $O(V + E)$ regardless of the number of sources. One BFS per source is $O(k \times (V + E))$ — 500 gates on a million-cell grid is $500 \times$ more work. The tell is “nearest”, “any of these”, or “until everything is”.

For “how many rounds”, freeze `len(queue)` and process one whole level per pass — and use `while queue and fresh`, or the last empty round inflates the count. Whatever is never reached needs an explicit `-1`, checked after the loop.

0-1 BFS: a deque, zero-cost edges to the front and one-cost edges to the back. The frontier only ever holds two distinct distances, which is what a plain queue gives you for uniform weights — so it stays $O(V + E)$, about $13 \times$ cheaper than Dijkstra.

No `visited` set in 0-1 BFS — a vertex can be improved after being popped, so the guard is the relaxation. And with three or more distinct weights the invariant breaks: use a bucket dial for small integers, Dijkstra otherwise.

Day 142 · dsa — Graphs revision and mock round

Two sentences before any code: “A vertex is . *An edge from A to B means* .” Then four words: **directed?** **weighted?** **cyclic?** **connected?** — and connected is almost never.

One shape, four algorithms. `seen`, a frontier, mark on push. Take from the front $\rightarrow$ BFS; from the back $\rightarrow$ DFS; cheapest $\rightarrow$ Dijkstra; front-with-zero-cost-edges-pushed-front $\rightarrow$ 0-1 BFS.

Everything is $O(V + E)$ except two earned exceptions: Dijkstra pays $\log V$ for arbitrary weights, Bellman-Ford pays V for negative ones. **On a DAG, one topological pass handles any weights** — the cheapest option and the one most often missed.

Three modelling moves: implicit graphs (never build the edges); **make the shared property a vertex** (removes an n^2 step); and the virtual source (turns two kinds of decision into one).

Eight bugs, five of them silent: reversed direction; built from the edge list (phantom cycle); one component only; marked on pop; undirected edge added once; “seen” as the cycle test (diamond for directed, one edge for undirected); recursion on a long chain; and a missing grid bounds check (`grid[-1]` **wraps**).

The complete 180-day map

All one hundred and eighty days, both tracks, in order. Days in **bold** are in this volume.

DAY	DATA STRUCTURES AND ALGORITHMS	SYSTEM DESIGN
125	What a graph is, and where graphs hide	Retries, backoff, and thundering herds
126	Representing a graph: adjacency matrix versus adjacency list	Circuit breakers and bulkheads
127	Breadth-first search on a graph	Distributed locks
128	Depth-first search on a graph	Distributed systems revision and interview questions
129	Connected components	Message queues: why async changes everything
130	Grids are graphs: islands and flood fill	Kafka, explained
131	Shortest path in an unweighted graph	Publish-subscribe versus point-to-point
132	Cycle detection in an undirected graph	Stream processing basics
133	Cycle detection in a directed graph	Object storage, S3-style
134	Topological sort	Blob storage versus storing files in the database
135	Course schedule and the dependency family	Search: how a search index actually works
136	Dijkstra's algorithm	Elasticsearch in a design
137	Bellman-Ford, and what negative edges break	Time-series and metrics stores
138	Union-Find: the disjoint set union	Data warehouses: OLAP versus OLTP
139	Minimum spanning trees: Kruskal and Prim	ETL, batch pipelines, and where data goes to be counted
140	Bipartite graphs and two-colouring	Websockets, long polling, and server-sent events
141	Multi-source BFS and 0-1 BFS	Push notifications, end to end
142	Graphs revision and mock round	Geospatial indexing: geohash and quadtrees

Glossary

Every term this book defines, with the day it first appears. Definitions are the sentence that introduced the term.

4

4 kb [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

A

a replicated log [Day 119](#) — Real systems need a never-ending sequence of decisions, which is why every practical consensus system is built around a replicated log: an ordered, numbered list of entries, identical on every machine.

abstract class [Day 052](#) — Definition. An abstract class is a partly-built class: it can hold fields, constructors and working methods, and it declares some methods that subclasses must fill in.

acid [Day 025](#) — When Postgres says COMMIT succeeded, the data is on disk, and that promise is the D in ACID — Atomicity, Consistency, Isolation, Durability — the four letters you will meet properly on [\[day 033\]\(../day-033-window-with-a-map/README.md\)](#).

adaptee [Day 068](#) — The adaptee — the washing machine, which has a different shape and cannot be changed.

adapter [Day 059](#) — An adapter is an implementation that translates to a specific technology (PostgresOrderRepository).

add a tie-breaker [Day 027](#) — And add a tie-breaker — without one, two customers with the same total may swap places between runs, which makes paginated results duplicate and skip.

algorithm [Day 076](#) — A strategy is an algorithm plugged into a caller; a command is a request with its receiver and arguments already bound, so it can be stored, queued and undone.

always comparable [Day 115](#) — Two properties, and it needs both: it is always comparable (integers), and it is never equal (strictly increasing).

amortised [Day 005](#) — The word for that is amortised: `list.append` is $O(1)$ amortised, meaning any single append might be expensive but a long run of them averages out to constant time.

anaemic model [Day 044](#) — A design where classes hold only data and all the rules live in some `IdolService` is called an anaemic model — objects in name only — and it is the most common way candidates lose this round.

and a direction [Day 079](#) — It carries a floor and a direction — "I am on 4 and I want to go up".

anti-diagonals [Day 016](#) — To collect the down-left diagonals — usually called the anti-diagonals — group cells by the value of $r + c$.

anti-sniping [Day 095](#) — Starting the count again when somebody speaks at the last moment is anti-sniping: extending the deadline whenever a bid arrives near the end.

approximate [Day 102](#) — Also: Redis's LRU is approximate — it samples a handful of keys and evicts the least recently used among them, rather than maintaining a true global ordering, because exact LRU would cost a linked-list update on every access.

array [Day 015](#) — Padma's shelf is an array — the fixed row of numbered boxes you met on [\[day 009\]\(../day-009-what-an-array-is/README.md\)](#).

artefact [Day 012](#) — The output is one immutable artefact — most commonly a container image — containing your code, its dependencies, and the exact runtime version, built the same way every time.

assignment rule [Day 108](#) — "Walk forwards until you reach a point" is the assignment rule: a key belongs to the first node clockwise.

at-least-once [Day 129](#) — Two: you gain duplicates. Queues deliver at-least-once: if the consumer does not acknowledge in time, the message comes back.

atomic [Day 008](#) — The fix is to make check-and-act atomic — indivisible, so that no other thread can act in the middle of it.

atomicity [Day 025](#) — That is atomicity — the A in ACID — and it is impossible to get right with a file unless you write the whole log-and-replay machinery yourself, at which point you have written a database.

availability [Day 114](#) — The words also do not mean what they normally mean: consistency here is a very strong guarantee about a single value, and availability is a very strong claim that **every** non-failing node answers.

aws ecs [Day 012](#) — That something is an orchestrator — Kubernetes, AWS ECS, Nomad — or, on a smaller system, systemd on a virtual machine.

B

backend [Day 002](#) — The backend is the code that never leaves the kitchen.

bearer token [Day 019](#) — The token is a bearer token: whoever bears it is treated as the user.

best case [Day 002](#) — The best case is 1, the worst case is n.

binary tree [Day 098](#) — A binary tree is a tree where every node has at most two children, called left and right.

blast radius [Day 111](#) — The eleven days revealing water, diesel and medicine is blast radius: what actually stops.

block [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

blocked [Day 007](#) — When the barber stood there holding a head while the boy fetched hair colour, he was blocked — occupying a chair while doing no work.

both [Day 030](#) — Once both are inside the loop, look at the gap between them, measured the way round the loop from fast to slow.

boundary [Day 118](#) — The unifying idea worth stating: **two heaps facing each other let you maintain a boundary in a changing set — and the boundary is the answer.**

bounded context [Day 051](#) — The name for the boundary between those worlds is a bounded context: each context gets its own model of "customer", and they are linked by an id rather than merged.

bounded optimism [Day 116](#) — Spending up to two thousand is bounded optimism — act early where the downside is small.

breadth-first [Day 100](#) — The other family is breadth-first — one whole level at a time — which needs a queue instead of a stack and is [\[tomorrow\]\(../day-101-bfs-level-order/README.md\)](#).

break point [Day 045](#) — The result has one break point — one place where a value is followed by a smaller value.

broken links [Day 134](#) — You get orphans — objects nobody references — and broken links — rows pointing at nothing.

bucket by time [Day 157](#) — The one thing that breaks this model is a channel that never stops growing. A partition with ten million messages is too large for most stores, so bucket by time: the partition key becomes (channel_id, month).

bucket counts [Day 171](#) — The fix is histograms. Each machine exports bucket counts — "how many requests fell in 0–10 ms, 10–50 ms, 50–100 ms" — and you sum the buckets across machines and compute the percentile from the sum. That is exactly what Prometheus's `histogram_quantile` does, and it is why counters and histograms are the metric types that aggregate and...

bulkhead [Day 111](#) — A bulkhead is a partition that stops a failure spreading — from ships, where a holed compartment does not sink the vessel.

burst size [Day 023](#) — | The tank | The algorithm | |---|---| | 500-litre capacity | burst size — the most you can spend at once | | 20 litres an hour going in | refill rate — the sustained rate you are allowed | | taking water to wash | spending a token; one request costs one token | | the tank being empty | no tokens left → the request is refused, 429 | |...

C

cache [Day 003](#) — Saving it in his phone is caching. A cache is a local copy of an answer, kept so that you do not have to ask again.

cache it [Day 107](#) — Fix: you cannot split a single key across shards, so cache it — the photocopy — or replicate it to every shard, or give it its own shard.

cache stampede [Day 090](#) — And the first cold morning is a cache stampede.

candidates [Day 093](#) — The trays are the candidates — the values you are allowed to pick from.

capacity [Day 005](#) — A list keeps a capacity — how many slots it has room for — which is bigger than its length, the number of slots actually used.

cdn [Day 070](#) — CDN — a geographically distributed caching proxy.

certificate [Day 006](#) — When your browser connects to `google.com`, the machine sends back a certificate — a small document containing the name it claims (`google.com`), a public key, an expiry date, and a signature from a Certificate Authority.

chaining [Day 060](#) — The standard fix is chaining: each bucket holds a small list of the (key, value) pairs that landed there, and a lookup checks the two or three entries in that one bucket.

channels [Day 092](#) — The group message, the direct message and the phone call are channels — the different ways a message can physically reach a person.

checks permission [Day 011](#) — The kernel checks the arguments and checks permission — this is Balan saying no.

chunks [Day 160](#) — A file is not stored as a file. It is split into chunks — fixed or variable-size blocks, typically around 4 MB — and each chunk is identified by the hash of its contents. The file itself becomes a small piece of metadata: a name, a version, and an ordered list of chunk hashes.

circular buffer [Day 073](#) — That is a circular buffer, also called a ring buffer: a fixed block of slots where position $7 + 1$ is position 0.

class [Day 043](#) — A class is the description; an object is one thing made from that description.

client [Day 068](#) — The client — the wall socket, which will only accept one shape.

collapse key [Day 141](#) — Collapsing is Devaraj refusing to carry the same message four times. Both platforms support a collapse key: a new notification with the same key replaces any undelivered one.

collapsing writes [Day 102](#) — Its real advantage is collapsing writes — a video watched ten thousand times in a minute becomes one database write instead of ten thousand.

collection [Day 016](#) — A collection is the set of things: `/bags`.

column [Day 026](#) — A column is one attribute of it, with a type that constrains what can go there.

compact [Day 011](#) — The price is that you now need a separate way to know which slots are real, and you must eventually compact: one pass that closes all the holes at once.

comparator [Day 058](#) — And the two uncles are the one case that needs a comparator — a rule about a pair, not a fact about an item.

comparison [Day 052](#) — A comparison is one question: *is this one bigger than that one?* A swap is exchanging two values so each sits where the other was.

complete [Day 114](#) — Insert appends first, and that is not a detail. Putting the new value at the end of the array is the only place that keeps the tree complete — any other position leaves a gap and destroys the array layout.

component [Day 069](#) — The tumblers are the component — the real object that does the actual job.

composite [Day 029](#) — This only ever applies when the primary key is composite — two or more columns together.

composite key [Day 026](#) — Its primary key is usually the two columns together — a composite key — which also enforces that the same pair cannot appear twice.

composition [Day 071](#) — Strategy uses composition — the varying part is an object passed in.

composition root [Day 053](#) — The composition root is the one place in the program where the real concrete classes are named and wired together — usually `main()` or a startup module.

connection [Day 041](#) — Under all of this, every query needs a connection — and opening one is genuinely expensive: TCP, then TLS ([[day 006](#)](../day-006-python-strings-dicts-sets/README.md)'s handshake), authentication, and in Postgres a whole operating-system process per connection ([[day 008](#)](../day-008-reading-a-problem/README.md)'s processes, ~5–10 MB each).

connection-oriented [Day 004](#) — This is why TCP is called connection-oriented — there is a real, agreed-upon connection with state on both sides.

consequence [Day 109](#) — The lorries are the consequence — the number that changes a decision, which is the only number that actually mattered.

consistency [Day 114](#) — The words also do not mean what they normally mean: consistency here is a very strong guarantee about a single value, and availability is a very strong claim that *every* non-failing node answers.

constraints [Day 008](#) — A problem statement has four things in it: what you are given, what you must return, the constraints that bound the input, and the edge cases that the ordinary solution gets wrong.

constructor [Day 044](#) — `__init__` is the constructor: the code that runs when a new object is made, whose job is to leave the object in a valid state.

container [Day 013](#) — A container is an ordinary process — the same kind of process you met on [[day 008](#)](../day-008-reading-a-problem/README.md) — that has been given a restricted view of the machine it is running on and a hard limit on what it may consume.

container image [Day 012](#) — The output is one immutable artefact — most commonly a container image — containing your code, its dependencies, and the exact runtime version, built the same way every time.

containerd [Day 013](#) — Docker's contribution was making it usable, and the format is now standardised by the OCI (Open Container Initiative) so that other tools — Podman, containerd, CRI-O — run the same images.

context [Day 073](#) — The object — the context — holds one of those classes and forwards every request to it.

coordinator [Day 120](#) — The roles. One coordinator — Kalpana — and several participants, the machines holding the data.

correlation id [Day 091](#) — So every record carries a correlation id — generated at the edge, stored in a context variable, and attached automatically by the framework rather than passed through every function signature.

cost [Day 032](#) — It enumerates the plausible combinations, assigns each a cost — an abstract number, not milliseconds — and picks the smallest.

count map [Day 063](#) — Going through the messages once and bumping a number is building a frequency map, also called a count map or a counter — all three names mean the same thing and interviewers use all three.

counter [Day 063](#) — Going through the messages once and bumping a number is building a frequency map, also called a count map or a counter — all three names mean the same thing and interviewers use all three.

cri-o [Day 013](#) — Docker's contribution was making it usable, and the format is now standardised by the OCI (Open Container Initiative) so that other tools — Podman, containerd, CRI-O — run the same images.

critical path [Day 134](#) — The number of levels is the critical path — the minimum time to finish everything if you had unlimited workers.

cycle [Day 125](#) — Cyclic or acyclic is the third. A cycle is a path that starts and ends at the same vertex without reusing an edge.

D

dangerous [Day 104](#) — Compact, and dangerous: any statement that is not deterministic gives a different result on the follower.

dashed [Day 050](#) — Four: implements an interface — the same triangle, but a dashed line:

dau [Day 097](#) — DAU — daily active users.

deep-copied [Day 067](#) — The cooker and the dabbas were deep-copied — genuinely new things that happen to look the same.

deleted [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

delta of delta [Day 137](#) — So you store the difference of the differences — the delta of delta — which is almost always zero, and zero costs one bit.

dependency [Day 053](#) — A dependency is anything a class needs in order to do its job but does not own — a repository, a mailer, a clock, a payment gateway.

depth [Day 046](#) — First, depth: with Vehicle → MotorVehicle → Car → ElectricCar → LeasedElectricCar, understanding one method means reading five files, and a change at the top can break anything below it, at any depth.

depth-first [Day 100](#) — Going depth-first means you follow one branch all the way to the bottom before backing up and trying the next one, and there are exactly three places you can do the visiting: before the children, between them, or after them.

deque [Day 031](#) — The fix for max is to keep a deque — a double-ended queue, from [day 074](../day-074-deques-and-window-max/README.md) — holding indices, arranged so that their values are decreasing from front to back.

diminishing returns [Day 135](#) — But with diminishing returns: the eighth mention adds much less than the second.

direct addressing [Day 066](#) — The wall of numbered holes is direct addressing: because the keys are the numbers 1 to 120, you do not need to hash anything.

domain name [Day 003](#) — "Anjali Sharma" is the domain name. A domain name is a human-readable label for a machine or a group of machines: google.com, leetcode.com, api.github.com.

draw two diagrams [Day 050](#) — If your design genuinely has fifteen classes, draw two diagrams: the core five or six, and then a second showing one subsystem in detail.

duck typing [Day 047](#) — This is duck typing: if it has a send method that takes a string, it is a notifier.

dummy node [Day 080](#) — Sister Mary is a dummy node — sometimes called a sentinel head.

E

earlier [Day 118](#) — The safe pattern: the leader treats its lease as expiring earlier than the grantor does, so there is a gap in which nobody believes they are leader.

edge cases [Day 008](#) — A problem statement has four things in it: what you are given, what you must return, the constraints that bound the input, and the edge cases that the ordinary solution gets wrong.

edge compute [Day 103](#) — There is a real modern exception, and mentioning it is a good signal: edge compute — Cloudflare Workers, Lambda@Edge — runs your code at the edge, so some personalisation and some logic can happen there without a trip to the origin.

effect [Day 113](#) — Say it that way: "exactly-once delivery is impossible; exactly-once effect is achievable, with at-least-once delivery and an idempotency key." Systems that advertise exactly-once are doing precisely this.

elastic scaling [Day 100](#) — The locum starting with no handover is elastic scaling: add a machine and it is immediately useful.

election timeout [Day 118](#) — "By half past five" is the election timeout — the guess that the leader is gone.

empty [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

etl [Day 139](#) — ETL versus ELT is about where the reshaping happens. ETL — extract, transform, load — transforms on a separate machine before loading, which is what you did when warehouse compute was scarce.

event object [Day 072](#) — In practice you push a small, immutable event object — a plain record describing what happened, like `OrderPlaced(order_id, user_id, total, items)`.

eventual consistency [Day 115](#) — At the bottom is eventual consistency: if writes stop, the copies will agree — eventually, with no promise about when or what you see meanwhile.

exactly [Day 038](#) — The map earns its space exactly when negatives are possible or the question is exactly — the two places monotonicity fails.

exactly correct [Day 089](#) — This is exactly correct — no boundary effect, no approximation — and it is the version you already wrote on [\[day 077\]\(../day-077-stacks-queues-revision/README.md\)](#) as a deque of timestamps.

exactly one [Day 171](#) — XOR is exactly one — which is the same as "different", and that is the property the whole of the next XOR day rests on.

exclusive lock [Day 035](#) — When a transaction updates a row, the database gives it an exclusive lock on that row: nobody else may change the row until the transaction commits or rolls back.

expiry [Day 090](#) — The dates are expiry — a completely separate mechanism.

F

facts [Day 082](#) — So: a FinePolicy. The loan holds the facts — who, what, when it went out, when it was due, when it came back.

false negative [Day 124](#) — A false negative is failing to notice a machine that really has died.

false positive [Day 124](#) — A false positive is declaring a healthy machine dead.

fanout [Day 030](#) — The critical property is the fanout: because a page is 8 KB and an entry is small, one node holds hundreds of entries.

fee [Day 044](#) — If a copy is > returned late, a fee is charged at ₹2 a day."

fifo [Day 090](#) — FIFO — evict the oldest inserted.

file [Day 093](#) — A file is a thing that is not a box: a plate, a lamp.

filled diamond [Day 050](#) — A filled diamond means *composition* — the parts die with the whole, so Order ♦ — OrderLine because a line item has no meaning without its order.

finite state machine [Day 073](#) — This is a finite state machine — a fixed set of states, a fixed set of events, and a table saying which event moves you from which state to which.

float [Day 081](#) — Devi's second tin is a float: coins reserved so tomorrow can start.

four-direction move [Day 096](#) — "Each one immediately beside the one before" is the four-direction move: up, down, left, right.

fragile [Day 021](#) — It is also fragile: it silently breaks on capital letters, spaces, digits and anything non-English, and §7 shows exactly how silently.

frontend [Day 002](#) — The frontend is the code that crosses the counter.

G

galloping mode [Day 057](#) — When merging, if one run keeps winning, Timsort switches to galloping mode — it binary-searches ahead to find how many elements it can take in one go instead of comparing one at a time.

gap [Day 047](#) — The gap is the answer she is searching for — not something in the array, a number she is choosing.

generator expression [Day 020](#) — That is a generator expression — it produces each item as join asks for it.

greedy simulation [Day 046](#) — It is a greedy simulation: load as much as fits, then start a new trip.

guest operating system [Day 013](#) — A hypervisor — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete guest operating system: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

H

hash map [Day 066](#) — Kiran's phone is a hash map: a general system that can look up anything by any key, at the price of storing the keys and computing where they go.

hash ring [Day 108](#) — The eleven-kilometre loop is the hash ring — the space of all hash values, joined end to end.

hateoas [Day 016](#) — And then there is the seventh idea, part of the uniform interface, that everyone quotes and almost nobody implements: HATEOAS — Hypermedia As The Engine Of Application State.

header interface [Day 058](#) — A header interface is named after the implementation and lists everything it can do: Repository, Machine, UserService.

heap [Day 117](#) — The six letters on the table are the heap: one candidate per list.

heapsort [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A* — the nearest unvisited node - top-k problems — [day 116](../day-116-top-k/README.md) - merging k sorted lists — [day 117](../day-117-merge-k-sorted/README.md) - the running median — [day 118](../day-118-two-heaps/README.md), with two heaps -...

heartbeat [Day 124](#) — The whistle is a heartbeat. A heartbeat is a small, regular message that says "I am alive".

hidden spof [Day 111](#) — The shared approach road is a hidden SPOF — two components that are not independent.

hiding mechanism [Day 052](#) — Abstraction is about hiding mechanism: the caller says "charge this card" and does not learn which payment provider ran it.

highest node [Day 104](#) — Every path has exactly one highest node — the point where it turns around.

historical snapshot [Day 029](#) — | Copy | Instead of | Why | |---|---|---| | posts.comment_count | COUNT(*) on comments | counting on every read is unaffordable at a high read ratio | | orders.customer_name | joining to customers | a historical snapshot: the name at the time of the order | | order_items.unit_price | joining to products | the price *then*, which must...

hollow triangle [Day 050](#) — Three: inheritance — a line with a hollow triangle pointing at the parent:

hook [Day 075](#) — award is a hook: the base class implements it as `*doing nothing*`, so a subclass may override it and most do not.

horizontal scaling [Day 098](#) — Horizontal scaling means putting more machines beside it and dividing the work.

hypervisor [Day 013](#) — A hypervisor — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete guest operating system: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

I

image [Day 013](#) — An image is a read-only stack of filesystem layers — a base operating system's files, then your dependencies, then your code.

implicit graph [Day 130](#) — This is an implicit graph — the edges exist by rule rather than by record — and you met the idea on [\[day 125\]\(../day-125-what-a-graph-is/README.md\)](#).

in parallel [Day 111](#) — Components in parallel — any one will do — multiply their `*failure*` probabilities, which is a completely different shape.

in series [Day 111](#) — Components in series — every one is needed — multiply their availabilities, so the total is always worse than the worst one.

in-degree [Day 125](#) — In a directed graph you count separately: in-degree is arrows coming in, out-degree is arrows going out.

index [Day 025](#) — A database keeps an index — a separate structure, usually a B-tree, that maps a value to the location of the matching rows.

indexed heap [Day 114](#) — Hence the indexed heap: a hash map from value to index, updated on every swap.

inheritance [Day 071](#) — Template Method uses inheritance — an abstract base with the skeleton and subclasses filling in the holes.

inorder [Day 110](#) — The standing order is inorder: left subtree, then root, then right subtree.

input [Day 008](#) — In a problem: the input — its type, its size, and what is in it.

integrity [Day 006](#) — TLS also guarantees integrity: each piece carries a check value, so if anything in transit is altered by even one bit, the other end notices and drops the connection.

intent plus multiplicity [Day 070](#) — The honest distinction is intent plus multiplicity: you stack decorators and expect several; a proxy is usually one and is about access rather than behaviour.

interface [Day 052](#) — An interface is a pure list of method signatures with no state and no implementation — a contract that says what a type can do.

invariant [Day 028](#) — That is called an invariant — something true before the loop starts, still true after every turn, and therefore true when the loop ends.

is possible [Day 046](#) — `> *Find the smallest X such that some task is possible.* > > *Find the largest X such that some condition still holds.*`

item [Day 016](#) — An item is one of them: `/bags/213`.

iterator [Day 075](#) — Iterator is the pattern where "go through this collection one item at a time" is separated from "this collection".

J

json [Day 015](#) — JSON — JavaScript Object Notation — is the plain-text format most web APIs use for both request bodies and responses, and you met it on [\[day 007\]\(../day-007-space-complexity/README.md\)](#); it looks like `{"id": 42, "name": "Zoya"}`.

jump [Day 094](#) — A snake and a ladder are both a jump: a pair (from, to).

jwt [Day 020](#) — a session id — an opaque reference, meaningless by itself, that the server looks up; - a JWT — a self-contained signed token that carries the facts inside it, so no lookup is needed; - OAuth 2.0 and OpenID Connect — a protocol for getting one of the above from **somebody else**, so a user can log in with Google without your service ever...

K

k closest points [Day 116](#) — k closest points — the same shape with a computed key.

k most frequent [Day 116](#) — k most frequent — count first, then top-k on the counts.

k-th largest specifically [Day 116](#) — k-th largest specifically — quickselect is the natural fit, because you want one element rather than a set, and quickselect gives it directly.

key [Day 023](#) — The key is as much of the design as the algorithm.

key property [Day 108](#) — Adding the temple point disturbing only sixty houses is the key property: adding a node moves only the keys between it and its predecessor.

L

lag [Day 112](#) — | Move | Price | ---|---| | Scale up | a ceiling, downtime to resize, and still one machine — no availability gain | | Cache | staleness, invalidation, and a stampede when a hot key expires | | CDN | purges are slow; nothing personalised; cache-key explosion from query strings | | Stateless | a shared session store becomes a hot...

largest [Day 055](#) — the 1st smallest is at position 0, so the k-th smallest is at position $k - 1$; - the 1st largest is at position $n - 1$, so the k-th largest is at position $n - k$.

latency [Day 089](#) — The cost is latency — a request may wait in the queue — and that makes it wrong for a user-facing API, where you would rather refuse quickly than hold a connection.

lazy [Day 117](#) — Merging sorted streams — `heapq.merge(*iterables)` does exactly this and is lazy, so it works on inputs larger than memory.

lazy deletion [Day 114](#) — Python has neither. The idiomatic workaround is lazy deletion: push a new entry and mark the old one stale, skipping stale entries as they surface.

lazy loading [Day 041](#) — That is lazy loading: the ingredient bought at the moment it is needed.

lazy refill [Day 089](#) — The lazy refill is the implementation trick: no timer, no background job.

leader [Day 105](#) — Counter two is the leader: writes go there, and reads from it are always current.

leaf [Day 098](#) — He is a leaf — an item with no children.

lease [Day 118](#) — A lease is leadership with an expiry: you are the leader until time T unless you renew.

ledger entries [Day 085](#) — An expense produces a set of ledger entries: the payer is owed, each participant owes their share.

left [Day 040](#) — Remove the strip to the left: `prefix[r2 + 1][c1]`.

lfu [Day 090](#) — LFU — evict the least frequently used.

linear scan [Day 062](#) — Scrolling the whole list of names is a linear scan: to answer one question you touch everything.

linearizability [Day 114](#) — C — Consistency. Specifically linearizability: every read sees the most recent write, and the system behaves as if there were one copy of the data.

list [Day 001](#) — heap is a list: a row of values kept in order, like the socks piled on the bed.

list of lists [Day 016](#) — A matrix is a list of lists: the outer list holds the rows, and each row is itself a list holding that row's values.

load balancer [Day 098](#) — Sending students to the right counter is a load balancer.

load factor [Day 060](#) — The number that controls this is the load factor: the number of items divided by the number of buckets.

load shedding [Day 007](#) — This idea has a name — load shedding — and it is one of the things that separates a system that degrades from one that collapses.

log-structured merge tree [Day 031](#) — A log-structured merge tree — used by Cassandra, RocksDB, LevelDB and ScyllaDB — takes the opposite trade.

logical cohesion [Day 061](#) — Anjali's first kitchen is logical cohesion — grouped by kind, which looks tidy and is wrong, because the unit of use is the job, not the kind.

logical time [Day 113](#) — The replacement is logical time — Lamport clocks and vector clocks — which order events by *causality* rather than by wall time.

loop [Day 001](#) — A loop is a statement that says, "do the following again and again".

loop body [Day 002](#) — A loop body is the indented block underneath the loop header — the part that runs again and again.

low cohesion [Day 061](#) — The kitchen arranged by size is low cohesion — things used together kept apart, so every job takes trips.

lru [Day 090](#) — LRU — evict the least recently used.

M

manifest [Day 158](#) — Adaptive bitrate streaming. The client downloads a manifest — an HLS .m3u8 or DASH .mpd — listing every rendition and every segment.

many times [Day 149](#) — Virtual nodes fix that. Each physical machine is placed on the circle many times — 100 to 256 positions each, by hashing machine-1#0, machine-1#1, and so on.

many-to-many [Day 026](#) — | Shape | Example | Where the key goes | |---|---|---| | one-to-many | a user has many posts | on the many side: posts.author_id | | many-to-many | a post has many tags, a tag has many posts | a third table: post_tags(post_id, tag_id) | | one-to-one | a user has one profile | either side; usually the optional one |

matrix [Day 016](#) — A matrix is a rectangle of values addressed by two numbers instead of one: which row, then which column.

mau [Day 097](#) — MAU — monthly active users.

max-heap [Day 116](#) — The mirror rule: to find the k smallest, keep a max-heap of size k — in Python, a min-heap of negated values.

method resolution order [Day 052](#) — The Python answer. The method resolution order — D._mro_ — is computed by a rule called C3 linearisation, and super() follows that order rather than jumping straight to a parent.

milliseconds [Day 009](#) — About 10 milliseconds — a hundred times slower again than SSD, and a hundred thousand times slower than RAM.

min-heap [Day 118](#) — Each half is a heap, and they face each other: a max-heap holding the lower half so its largest is at the top, and a min-heap holding the upper half so its smallest is at the top — the two elements at the boundary are exactly the ones you need. And the only real work is keeping the two halves the same size, which is two lines and is...

minimum spanning forest [Day 139](#) — Both algorithms then produce a minimum spanning forest — one tree per component — and you detect it by counting: fewer than $V - 1$ edges taken means the graph was disconnected.

mixin [Day 049](#) — A mixin is a small class carrying one piece of behaviour, meant to be inherited alongside others:

monotone [Day 043](#) — A yes-or-no question with that shape is called monotone: once the answer flips from no to yes, it stays yes.

monotonic deque [Day 074](#) — The good solution keeps a small monotonic deque of "elements that could still be the maximum" and is $O(n)$ overall.

N

name mangling [Day 045](#) — Two underscores triggers name mangling: inside the class the attribute is rewritten as `_Account_key`, so a subclass defining its own `_key` does not collide.

namespace [Day 013](#) — A namespace is a kernel feature that gives a process its own private version of one part of the system.

negative [Day 035](#) — A sum over values that can be negative is not monotonic — the window is the wrong room, and the case goes to prefix sums, which arrive on [\[day 037\]\(../day-037-prefix-sums/README.md\)](#).

negative infinity [Day 049](#) — LeetCode 162 states it: treat `nums[-1]` and `nums[n]` as negative infinity — imaginary values smaller than anything real, just off each end.

negatives [Day 038](#) — The map earns its space exactly when negatives are possible or the question is exactly — the two places monotonicity fails.

nested loop [Day 001](#) — A nested loop is a loop inside another loop.

network partition [Day 114](#) — The exchange fault is a network partition — the nodes are alive but cannot talk.

never equal [Day 115](#) — Two properties, and it needs both: it is always comparable (integers), and it is never equal (strictly increasing).

never forgets [Day 090](#) — Needs a third structure and a `min_frequency` counter to stay $O(1)$, and it never forgets: an entry that was hot last month holds a huge count and will not leave.

never persisted [Day 157](#) — Typing indicators are presence with a shorter fuse. Same mechanism, a two-to-three second TTL, throttled so a keystroke does not become a network message, and never persisted — a typing event that arrives late is worse than one that is dropped.

no children [Day 121](#) — It has no children — nothing continues past it.

node [Day 125](#) — The word node means the same thing and both are used; this course will say vertex when talking about the structure and node when talking about code.

not a map [Day 067](#) — | not a map — heap or sorted array |

O

$o(1)$ [Day 079](#) — tail — a reference to the last node, which makes append $O(1)$ instead of $O(n)$.

object [Day 043](#) — An object is data and the behaviour that belongs to that data, kept together in one place.

observers [Day 072](#) — The objects on the list are the observers.

occupied [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

offset [Day 130](#) — Each viewer's position is an offset. An offset is just a number — the position in the log that a particular reader has got to.

on first touch [Day 041](#) — Each `post.author` is a `*different table*`, not yet loaded — so the ORM, helpfully and silently, fetches it on first touch.

one-shot [Day 075](#) — A list can be walked many times; a generator is one-shot — walk it twice and the second walk is empty, which is a bug people hit constantly.

opens [Day 126](#) — When failures exceed a threshold, it opens: from then on, calls to that dependency fail instantly without being attempted.

orchestrator [Day 012](#) — That something is an orchestrator — Kubernetes, AWS ECS, Nomad — or, on a smaller system, `systemd` on a virtual machine.

origin [Day 101](#) — The storeroom is the origin — the source of truth.

origin pull [Day 103](#) — Sending the finished pages down the wire is origin pull: the edge fetches from the origin on demand.

orphan rows [Day 026](#) — Without it you get orphan rows — a post whose author does not exist — and they are impossible to clean up later, because you cannot tell which ones were mistakes.

orphans [Day 134](#) — You get orphans — objects nobody references — and broken links — rows pointing at nothing.

out-degree [Day 125](#) — In a directed graph you count separately: in-degree is arrows coming in, out-degree is arrows going out.

overloading [Day 047](#) — Compile-time polymorphism, or overloading: several methods with the same name and different argument types, and the compiler picks.

overriding [Day 047](#) — Runtime polymorphism, or overriding: a subclass replaces a parent's method, and which one runs is decided by the object at run time.

P

page [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

parallel edges [Day 132](#) — And the case where excluding by vertex is not enough. If the graph can have two separate edges between the same pair — parallel edges — then `A—B` twice genuinely `*is*` a cycle, and skipping "the parent vertex" wrongly ignores it.

parameters [Day 015](#) — In an API those are the parameters — the values you must supply with a request — and leaving a required one out gets you an error, not a guess.

parent [Day 132](#) — The fix: pass down where you came from, and skip it. Each recursive call carries the vertex that called it — its parent — and the check becomes "seen, and not my parent".

partial failure [Day 113](#) — The one that matters most is partial failure: a call over a network has a third outcome that does not exist locally, `*"I do not know"`, and almost every difficult problem in the rest of this phase is a consequence of it.

participants [Day 120](#) — The roles. One coordinator — Kalpana — and several participants, the machines holding the data.

partition key [Day 039](#) — A wide-column store — Cassandra is the name to know — organises data by two keys: a partition key that decides which machine (and which bucket on it) a row belongs to, and clustering columns that keep rows `*sorted inside*` that bucket.

partitions [Day 130](#) — It is split into partitions — independent logs that together make up the topic.

path [Day 005](#) — A request has four parts: a method saying what you want done, a path saying what you want it done to, headers carrying facts about the request itself, and an optional body carrying the content.

port [Day 003](#) — A port is a second number that says which program on that machine you want, the way a flat number says which door inside the building.

prefix sums [Day 037](#) — His noted meter readings are the prefix sums — and his godown reading, taken before anything was driven, is the extra zero at the front that today's code lives or dies by.

preorder [Day 110](#) — The calling order is preorder: root, then left subtree, then right subtree.

priority queue [Day 139](#) — Finding "the cheapest edge leaving the tree" efficiently means a priority queue — which makes Prim's structurally almost identical to [Dijkstra's](../day-136-dijkstra/README.md).

procedure [Day 062](#) — It is a procedure: a fixed order in which to read a piece of code you have never seen, so that by the end of it you can say what is wrong, what evidence you have, and what you would change first.

process [Day 008](#) — A process is a running program with its own private memory.

program [Day 001](#) — A program is a sequence of statements.

protecting state [Day 052](#) — Definition. Encapsulation is about protecting state: the balance cannot go negative because the only way to change it runs a check.

Q

qps [Day 097](#) — QPS stands for *queries per second* — the number of requests arriving at your system each second.

queue [Day 127](#) — The queue holds the frontier in order. A queue is a line where things come out in the order they went in — first in, first out — which you met on [day 73](../day-073-queues/README.md).

quorum [Day 118](#) — "Three of the four must agree" is a quorum — the majority from [day 117](../day-117-merge-k-sorted/README.md).

R

race condition [Day 008](#) — A race condition is when the result depends on the exact timing of two things running at once.

ram [Day 009](#) — The suitcase on the almirah — main memory, RAM. RAM — random access memory — is where your running program's data lives.

rank [Day 138](#) — Union by rank is the same idea using an upper bound on height rather than the exact size; both work, size is easier to reason about, and size gives you group sizes for free, which problems often want.

rebalancing [Day 107](#) — Moving twenty bundles is rebalancing — moving whole logical shards, not rehashing keys.

rebuilding the tree [Day 109](#) — Repacking the whole cart is rebuilding the tree — $O(n)$.

recursion [Day 053](#) — That is recursion: a function whose body calls the same function on a smaller piece of the problem.

references [Day 009](#) — A Python list stores references — memory addresses of objects that live elsewhere:

refill rate [Day 023](#) — | The tank | The algorithm | |---|---| | 500-litre capacity | burst size — the most you can spend at once | | 20 litres an hour going in | refill rate — the sustained rate you are allowed | | taking water to wash | spending a token; one request costs one token | | the tank being empty | no tokens left → the request is refused, 429 | |...

register [Day 009](#) — The chair beside the bed — registers. A register is a tiny store inside the processor itself, holding one value.

relative epsilon [Day 048](#) — The fix is a relative epsilon — stop when the gap is small *compared to the value*:

removal [Day 118](#) — The same two heaps, plus removal, which a heap cannot do.

replica [Day 136](#) — A replica is a copy of a shard on another node, serving reads and taking over if the primary fails.

replica divergence [Day 115](#) — Different people knowing different things is replica divergence — and none of them is wrong.

request-aware routing [Day 099](#) — The joint-family card going to counter one is request-aware routing — some requests are much more expensive than others.

resolver [Day 021](#) — Each field in the schema has a resolver — a function that knows how to produce that field.

resources [Day 017](#) — Before writing a single path, list the resources — the things the feature is about.

returned [Day 044](#) — If a copy is > returned late, a fee is charged at ₹2 a day.*

ring buffer [Day 073](#) — That is a circular buffer, also called a ring buffer: a fixed block of slots where position 7 + 1 is position 0.

role interface [Day 058](#) — A role interface is named after what a client needs: UserReader, Printer, PriceQuoter.

root [Day 093](#) — The root is the root itself, written /.

rotation [Day 109](#) — The fix is a rotation: a local rearrangement of three or four pointers that reduces the height by one without touching the rest of the tree or changing the inorder order.

routine [Day 109](#) — Counting paces and multiplying is the routine — the same sequence every time, said out loud.

row [Day 026](#) — A row is one of those things.

run [Day 044](#) — So the copies of a value form one run: a contiguous stretch, described completely by its two ends.

S

same remainder [Day 038](#) — A stretch is divisible by k when the prefixes at its two ends leave the same remainder — so carry running % k and count matching remainders.

schema [Day 021](#) — A GraphQL API is defined by a schema: every type, every field, and every field's type, written down and strongly typed.

scoping [Day 096](#) — The two or three narrowing questions are scoping — deciding out loud what you are *not* building.

script [Day 077](#) — Today is the script — the fixed order of moves you run on every one of these prompts for the next twenty days.

seam [Day 053](#) — A seam is a place where you can change behaviour without editing the class.

search space [Day 042](#) — The stretch `nums[low..high]` is called the search space — the part of the array you have not ruled out yet.

selectivity [Day 032](#) — From those numbers the planner estimates selectivity: what fraction of rows a condition keeps.

sentinel [Day 037](#) — That extra entry is called a sentinel: a value placed at the boundary so that the boundary needs no special treatment.

separator [Day 019](#) — The string before the `.join` is the separator — `""`.join glues with nothing between, `" "`.join puts a comma and a space between each pair.

sequence diagram [Day 050](#) — That is a sequence diagram: participants as columns, time running down, and each message an arrow from one column to the next.

serialisable [Day 033](#) — The strongest form is serialisable: the result is as if the transactions had run one after another, in some order.

service credits [Day 173](#) — An SLA is a contract with a customer: a promise, and a consequence when you break it. The consequence is almost always service credits — money back off the next bill — rather than damages.

session [Day 020](#) — A session is a record the server keeps about a logged-in user.

session guarantees [Day 115](#) — And the interesting rungs are in the middle, particularly causal consistency and the four session guarantees, because they are what users actually notice and they are much cheaper than the top.

set [Day 006](#) — A set stores values and answers "is this in here?" in a single step, no matter how many it holds.

shallow-copied [Day 067](#) — The milk and the newspaper were shallow-copied — the new kitchen got a *reference* to the same arrangement, and changing it from either end changed it for both.

shape [Day 003](#) — It throws away every detail that does not change as the input grows — the exact number of steps, how fast your laptop is, whether one line takes two nanoseconds or twenty — and keeps only the shape of the growth.

shard [Day 106](#) — Each room is a shard — a machine holding a subset of the rows.

shared lock [Day 035](#) — A shared lock is for reading: many transactions can hold one on the same row at once, because readers do not disturb each other.

shared store [Day 100](#) — The shared cabinet at the front desk is a shared store — a database, or Redis.

singleton [Day 064](#) — Singleton is the pattern that guarantees a class has exactly one instance, and gives everybody a global way to reach it.

singly linked list [Day 076](#) — A singly linked list — where each node knows only the *next* one — cannot do that.

slice [Day 019](#) — `s[1:4]` is a slice — a new string made of part of the old one.

smallest [Day 046](#) — `>` *Find the smallest X such that some task is possible.* `>` *Find the largest X such that some condition still holds.*

snapshot [Day 034](#) — The transaction takes a snapshot at its first read and every statement in it sees that snapshot, however long it runs.

snipe [Day 095](#) — A snipe is a bid placed in the last second, leaving nobody time to respond.

sorted region [Day 052](#) — And the sorted region is the part of the list you have already finished; every one of these methods grows a sorted region and shrinks an unsorted one, and they differ in *which end* the region grows from and *how* the next value gets there.

sorting everything [Day 116](#) — The nephew laying out the whole load is sorting everything — correct, and far more work than asked for.

splits [Day 031](#) — If the leaf is full, it splits: half the keys move to a new page, and a key is pushed up to the parent.

spof [Day 111](#) — The bridge is an SPOF — one component, and everything depends on it.

squares [Day 003](#) — | Example | `|---|---|---|` | $O(1)$ | constant | does not change | reading items[5] | | $O(\log n)$ | logarithmic | goes up by one step | halving until you reach 1 | | $O(n)$ | linear | doubles | one loop over the list | | $O(n \log n)$ | linearithmic | slightly more than doubles | sorting | | $O(n^2)$ | quadratic | goes up four times | every...

stable [Day 051](#) — Python's sort is Timsort: $O(n \log n)$ worst case, stable, and adaptive — it finds runs that are already ordered and merges them, so nearly-sorted input runs close to $O(n)$.

stack [Day 075](#) — The fridge is a stack: last in, first out.

stack frame [Day 068](#) — When a function calls another function, the machine pushes a stack frame — the local variables and the place to return to — onto a stack.

stack overflow [Day 068](#) — And the bad Sunday is a stack overflow: too many unfinished things, and the one at the bottom is forgotten.

state [Day 143](#) — And the state is the thing to get right. The state is what identifies a subproblem — the arguments to the recursive function.

statements [Day 001](#) — A program is a sequence of statements.

status [Day 086](#) — It has a status — available, locked, booked — and a price, because the same seat costs more on a Saturday night.

still one machine [Day 112](#) — | Move | Price | |---|---| | Scale up | a ceiling, downtime to resize, and still one machine — no availability gain | | Cache | staleness, invalidation, and a stampede when a hot key expires | | CDN | purges are slow; nothing personalised; cache-key explosion from query strings | | Stateless | a shared session store becomes a hot...

stonith [Day 118](#) — STONITH — "shoot the other node in the head": the new leader forcibly disables the old one, by cutting power or revoking its storage lease, before accepting any write.

strategy [Day 071](#) — Strategy turns an algorithm into a value.

strictly shorter [Day 072](#) — left[i] — the index of the nearest bar to the left that is strictly shorter.

structural [Day 058](#) — In Python, typing.Protocol is structural — it matches on the shape of the methods, and the implementing class inherits nothing and imports nothing ([[day 052](#)](../day-052-quadratic-sorts/README.md)).

subarray [Day 024](#) — There is a third word you will meet: a subarray is the same thing as a substring, for arrays.

subject [Day 072](#) — Observer is the pattern where one object — the subject — keeps a list of other objects that want to be told when something happens to it, and tells all of them with one call.

subscription list [Day 072](#) — The group itself is the subscription list, and the one message is the notification.

substring [Day 155](#) — First the definitions. A substring is contiguous — "aba" is a substring of "xabay", but "aa" is not.

subtree [Day 122](#) — If you get to the end, the node you are standing on is the root of a subtree — the whole set of words that begin with what the user typed.

surge multiplier especially [Day 088](#) — The surge multiplier especially is frozen.

swap [Day 052](#) — A comparison is one question: *is this one bigger than that one?* A swap is exchanging two values so each sits where the other was.

system call [Day 011](#) — The mechanism for asking is a system call, and the boundary it crosses is the most important line in a computer.

T

table [Day 026](#) — A table holds one kind of thing.

tail call [Day 088](#) — A tail call is a recursive call whose result is returned directly, with nothing left to do afterwards:

tail-recursive [Day 090](#) — This version is also tail-recursive — nothing happens after the call returns — which a language with tail-call elimination would turn into a loop for free.

tcp [Day 004](#) — TCP and UDP are the two ways one machine sends data to another.

template [Day 092](#) — The saved wording with the date and the hours swapped in is a template — a fixed piece of text with holes in it, filled in per message.

test double [Day 053](#) — A test double is a stand-in you pass in during a test.

the return address [Day 088](#) — the arguments it was called with, - the local variables it creates, - the return address — the exact point in the caller to resume at, - and a link to the frame below it.

the running median [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A* — the nearest unvisited node - top-k problems — [[day 116](#)](../day-116-top-k/README.md) - merging k sorted lists — [[day 117](#)](../day-117-merge-k-sorted/README.md) - the running median — [[day 118](#)](../day-118-two-heaps/README.md), with two heaps -...

third table [Day 026](#) — | Shape | Example | Where the key goes | |---|---|---| | one-to-many | a user has many posts | on the many side: posts.author_id | | many-to-many | a post has many tags, a tag has many posts | a third table: post_tags(post_id, tag_id) | | one-to-one | a user has one profile | either side; usually the optional one |

thread [Day 008](#) — A thread is one line of execution inside a process, and all the threads in a process share that memory.

three distinct orderings [Day 092](#) — = 6 decision paths — but only three distinct orderings: [1,1,2], [1,2,1], [2,1,1]`.

three-way partition [Day 054](#) — The fix is a three-way partition: split into *less than*, *equal to*, and *greater than*, and recurse only into the outer two.

tight coupling [Day 061](#) — The balcony light on the reading lamp's switch is tight coupling — two things wired together that have no reason to be, so you cannot change one without changing the other.

time-to-live [Day 037](#) — Bhaskar's clear-out is time-to-live: attach an expiry to a key and the store deletes it, automatically.

tiny constraint [Day 097](#) — A tiny constraint: $n \leq 8$ means factorial, $n \leq 20$ means 2^n , target ≤ 500 with small candidates means combination sum.

toast [Day 134](#) — And there is a middle option worth knowing. Postgres stores large values out-of-line automatically — TOAST — in a side table, compressed, and only reads them when the column is selected.

token [Day 019](#) — The server verifies them and issues a token — a session id or a signed token.

tombstone [Day 011](#) — There is a real technique that does leave holes — marking a slot as deleted instead of removing it, called a tombstone — and databases and hash tables use it constantly.

top-k problems [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A* — the nearest unvisited node - top-k problems — [[day 116](#)](../day-116-top-k/README.md) - merging k sorted lists — [[day 117](#)](../day-117-merge-k-sorted/README.md) - the running median — [[day 118](#)](../day-118-two-heaps/README.md), with two heaps -...

total order [Day 123](#) — It gives you a total order — sort by the number, break ties by machine name — that never contradicts causality.

transitions itself [Day 071](#) — In State, the object transitions itself: a Draft order moves itself to Submitted, and the states know about each other.

transitive dependency [Day 029](#) — This is a transitive dependency: id → department_id → department_name.

tree [Day 040](#) — A domain of independent records — a tree — tolerates anything.

ttl [Day 003](#) — The family moving out is the staleness problem, and it is the reason for TTL. Every DNS answer comes with a TTL — time to live — which is a number of seconds saying how long the answer may be trusted before it must be looked up again.

tuple [Day 005](#) — A tuple is written with round brackets — (3, 7, 2) — and is a list you cannot change.

two [Day 087](#) — In fib, each call makes two — so the work explodes.

two implementations [Day 079](#) — And, as always, two implementations: NearestSuitable as above, and ZonedDispatcher where each car owns a band of floors.

U

udp [Day 004](#) — TCP and UDP are the two ways one machine sends data to another.

under-fetching [Day 021](#) — And from [day 015](../day-015-the-write-pointer/README.md)'s arithmetic: a screen needing four different resources makes four round trips, which on a mobile network is roughly 480 ms instead of 120 — under-fetching, usually called the N + 1 problem when it happens in a list.

union filesystem [Day 013](#) — The stack of layers is a union filesystem — several directories presented as one merged tree.

upper bound [Day 044](#) — The upper bound is the first index where `nums[i] > 4`, which is index 4 — one past the run.

uri [Day 016](#) — In REST the thing is called a resource, and its address is a URI — the path in the URL.

url [Day 001](#) — A URL is an address for something on the internet.

V

value object [Day 051](#) — A value object is defined entirely by its values.

version [Day 117](#) — And the second half, which people forget: you must be able to tell which value is newest. The overlapping replica has the new value, but the others may return an old one, so every record carries a version — a counter, a timestamp, or a vector clock — and the reader takes the highest.

vertex [Day 125](#) — The places are the vertices. A vertex is one of the things in your collection — a bus stop, a person, a city, a course, a web page, a cell in a grid.

vertical scaling [Day 098](#) — Vertical scaling means replacing the machine with a bigger one — more cores, more memory, faster disk.

victim [Day 035](#) — It watches who waits for whom, and when it finds a circle, it picks a victim — one transaction in the cycle — and kills it with an error.

virtual address space [Day 011](#) — Memory. Each process gets its own virtual address space — its own set of addresses that the OS maps onto real physical memory.

virtual machine [Day 013](#) — A virtual machine is an emulated computer.

volatile [Day 009](#) — It is also volatile: switch the power off and it is gone.

W

weight [Day 122](#) — To do that in code, every word needs a weight — a number saying how often it is chosen.

which class [Day 076](#) — Factory vs Builder — a factory decides which class; a builder assembles one complicated object step by step.

which is hours [Day 166](#) — The cost is that it must be rebuilt, and that is the trade to state. New queries do not appear until the next rebuild, which is hours — and there is a separate fast path for trending terms, which is the third idea below.

within [Day 131](#) — Consumer group — Kafka's answer, and it is genuinely both: within a group, consumers compete like a queue; between groups, each gets everything like a topic.

worst case [Day 002](#) — The best case is 1, the worst case is n.

write skew [Day 034](#) — It appears in backend interviews at every product company, usually right after ACID, and the follow-up about write skew is where senior candidates are separated from the table-reciters.

write-around with invalidation [Day 102](#) — Wiping a line when the price moves is write-around with invalidation: update the truth, delete the cache entry, and let the next reader repopulate it.

write-through [Day 102](#) — Changing the board at four fifteen is write-through: update the cache when you update the truth.

Y

yes [Day 093](#) — | Problem | Recurse on | Duplicates in input | Extra rule | |---|---|---|---| | Combinations (LC 77) | $i + 1$ | no | stop at size k | | Combination Sum (LC 39) | i — reuse allowed | no | stop when remaining $= 0$ | | Combination Sum II (LC 40) | $i + 1$ — each used once | yes | sort, and skip when $i > \text{start}$ and $c[i] == c[i-1]$ | |...

References and further reading

Further reading, arranged by the part of the book it belongs to. Everything listed is a primary source, an official specification, or reference material that is free to read. None of this book is copied from any of them. They are here for the reader who wants the long version, the formal proof, or the original paper.

FOR THE WHOLE BOOK

Python Software Foundation. *The Python Language Reference and Standard Library*. <https://docs.python.org/3/> [PSF Licence, free to read]

The authority for every language behaviour this book relies on.

Python Wiki. *TimeComplexity - cost of the built-in container operations*. <https://wiki.python.org/moin/TimeComplexity> [Free to read]

The table behind almost every complexity claim about list, dict and set.

David Galles, University of San Francisco. *Data Structure Visualizations*. <https://www.cs.usfca.edu/~galles/visualization/Algorithms.html> [Free to use]

Step-through animations for most structures in Parts II to XV.

Steven Halim and others. *VisuAlgo - visualising data structures and algorithms*. <https://visualgo.net/en> [Free for self-study]

Useful when a diagram in this book is not enough and you want to press play.

MIT OpenCourseWare. *6.006 Introduction to Algorithms*. <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/> [CC BY-NC-SA]

Full lecture videos and notes. The formal treatment this book deliberately avoids.

LeetCode. *Problem set*. <https://leetcode.com/problemset/> [Free tier]

Where every problem named in the practice sheets is solved.

DATA STRUCTURES AND ALGORITHMS, BY PART

PART XV · GRAPHS (DAYS 125–142)

Wikipedia. *Breadth-first search*. https://en.wikipedia.org/wiki/Breadth-first_search [CC BY-SA]

The shortest-path guarantee on unweighted graphs, stated precisely.

Wikipedia. *Dijkstra's algorithm*. https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm [CC BY-SA]

Including why it fails on negative edges, which is a common follow-up.

Wikipedia. *Topological sorting*. https://en.wikipedia.org/wiki/Topological_sorting [CC BY-SA]

Kahn's algorithm and the DFS variant, both used in Part XV.

Wikipedia. *Disjoint-set data structure*. https://en.wikipedia.org/wiki/Disjoint-set_data_structure [CC BY-SA]

Union by rank and path compression, with the inverse-Ackermann bound.

SYSTEM DESIGN, BY PHASE

DISTRIBUTED SYSTEMS CORE (DAYS 113–128)

Diego Ongaro and John Ousterhout. *In Search of an Understandable Consensus Algorithm (Raft)*. <https://raft.github.io/raft.pdf> [Free to read]

The clearest consensus paper written. Read it once end to end.

Giuseppe DeCandia and others, Amazon. *Dynamo: Amazon's Highly Available Key-value Store*. <https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf> [Free to read]

Consistent hashing, vector clocks and quorums, in one paper.

Eric Brewer. *CAP Twelve Years Later: How the Rules Have Changed*. <https://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed/> [Free to read]

Brewer correcting the popular misreading of his own theorem.

MIT PDOS. *6.5840 Distributed Systems*. <https://pdos.csail.mit.edu/6.824/> [Free to read]

Lecture notes and the paper list. The best free syllabus on the subject.

Kyle Kingsbury. *Jepsen analyses*. <https://jepsen.io/analyses> [Free to read]

What real databases actually do under partition, as opposed to what they claim.

BUILDING BLOCKS OF BIG SYSTEMS (DAYS 129–144)

Apache Software Foundation. *Kafka design documentation*. <https://kafka.apache.org/documentation/#design> [Apache 2.0]

Partitions, offsets and delivery guarantees, from the source.

Jeffrey Dean and Sanjay Ghemawat, Google. *MapReduce: Simplified Data Processing on Large Clusters*. <https://research.google/pubs/mapreduce-simplified-data-processing-on-large-clusters/> [Free to read]

The paper that started the batch-processing chapter of every system design book.

Fay Chang and others, Google. *Bigtable: A Distributed Storage System for Structured Data*. <https://research.google/pubs/bigtable-a-distributed-storage-system-for-structured-data/> [Free to read]

LSM trees and column families, which underlie most wide-column stores.

Elastic. *Elasticsearch guide*. <https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html> [Free to read]

Inverted indexes in production, for the search-service designs.

Adam Wiggins. *The Twelve-Factor App*. <https://12factor.net/> [CC BY-SA 3.0]

Short. Explains configuration, statelessness and disposability better than most books.

Colophon: how this book was made

This book was typeset from the course sources with free software only. Every typeface is published under the SIL Open Font Licence, every library under a permissive licence, and the page engine is the open-source Chromium print pipeline. Licence labels below describe the linked project, not this book.

TYPE, TOOLS AND LIBRARIES

SIL International, after Matthew Carter. *Charis SIL*. <https://software.sil.org/charis/> [SIL Open Font Licence 1.1]

The text face. An extension of Bitstream Charter, which Carter drew in 1987 to stay readable on coarse output. It is the standard free choice for technical books.

Bold Monday for IBM. *IBM Plex Sans and Plex Sans Condensed*. <https://github.com/IBM/plex> [SIL Open Font Licence 1.1]

Headings, labels, tables, folios and the covers.

JetBrains. *JetBrains Mono*. <https://github.com/JetBrains/JetBrainsMono> [SIL Open Font Licence 1.1]
All code, output and ASCII diagrams.

Google. *Noto Serif Devanagari*. <https://github.com/notofonts/devanagari> [SIL Open Font Licence 1.1]
The Devanagari on the covers and title page.

Knut Sveidqvist and contributors. *Mermaid*. <https://github.com/mermaid-js/mermaid> [MIT Licence]
Renders every flow chart, sequence diagram and tree.

Georg Brandl and contributors. *Pygments*. <https://pygments.org/> [BSD 2-Clause]
Colours the code. The palette used here was written for this book.

Waylan Limberg and contributors. *Python-Markdown*. <https://python-markdown.github.io/> [BSD 3-Clause]
Turns the lesson sources into the pages you are reading.

Martin Thoma and contributors. *pypdf*. <https://github.com/py-pdf/pypdf> [BSD 3-Clause]
Joins the covers, front matter and body, and writes the bookmarks.

The Chromium Authors. *Chromium print-to-PDF, driven by Puppeteer*. <https://pptr.dev/> [Apache 2.0 / BSD 3-Clause]
The typesetting engine. No proprietary software was used to make this book.

HOW A PAGE WAS MADE

The lesson sources are Markdown. Fenced blocks are lifted out before the Markdown is parsed, so diagrams keep every space they have and code reaches the colouring pass untouched. Each monospace block is measured, and its type size chosen so the widest line fits the column rather than falling off the page.

The result is rendered in a browser engine at 6.75 by 9.5 inches and printed to PDF. The body is printed first so that the contents and the index can be built from the page numbers the printing actually produced, then the covers and front matter are printed and the three are joined.

Problem index

Every problem named in a practice sheet, in the order the book reaches it, with its source and the chapter that sets it. Problem statements are not reproduced anywhere in this book. Solve them where they live.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
125	Find if Path Exists in Graph	LeetCode 1971 (Easy)	Building an undirected adjacency list and not forgetting the second append.
125	Find the Town Judge	LeetCode 997 (Easy)	In-degree and out-degree, with no traversal at all.
125	Number of Provinces	LeetCode 547 (Medium)	A matrix that *is* the graph, plus the outer loop over all vertices.
125	Keys and Rooms	LeetCode 841 (Medium)	Reachability, and noticing that "all rooms" means checking the unreachable ones.
125	Accounts Merge	LeetCode 721 (Medium)	The modelling move: make the shared property a vertex.
126	Find Center of Star Graph	LeetCode 1791 (Easy)	Reading the edge list without building anything at all.
126	Number of Provinces	LeetCode 547 (Medium)	You are handed a matrix. Traverse it without converting.
126	Find if Path Exists in Graph	LeetCode 1971 (Medium)	Converting an edge list, at a size where a matrix would fail.
126	Course Schedule	LeetCode 207 (Medium)	Directed build, and the direction stated as a sentence first.
126	Network Delay Time	LeetCode 743 (Medium)	Weighted list of (neighbour, weight), stored the way the heap wants it.
127	Find if Path Exists in Graph	LeetCode 1971 (Medium)	Plain reachability. Ten lines, and deque not a list.
127	Shortest Path in Binary Matrix	LeetCode 1091 (Medium)	BFS on an implicit grid, with eight directions instead of four.
127	Word Ladder	LeetCode 127 (Hard)	The neighbour function, and why building edges is the trap.
127	Rotting Oranges	LeetCode 994 (Medium)	Level-by-level, and the check for what never gets reached.
127	Open the Lock	LeetCode 752 (Medium)	An implicit graph of states, plus a set of forbidden vertices.
128	Number of Islands	LeetCode 200 (Medium)	DFS on a grid, plus the outer loop over every cell.
128	Course Schedule	LeetCode 207 (Medium)	Three colours, and the outer loop over every vertex.
128	Clone Graph	LeetCode 133 (Medium)	The seen map doing double duty as the old-to-new mapping.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
128	All Paths From Source to Target	LeetCode 797 (Medium)	Marking on entry and unmarking on exit — the difference.
128	Pacific Atlantic Water Flow	LeetCode 417 (Medium)	Two DFS runs from the edges inward, not one from each cell.
129	Number of Provinces	LeetCode 547 (Medium)	The outer loop, on a matrix input.
129	Number of Islands	LeetCode 200 (Medium)	The grid version, iteratively, marking on push.
129	Max Area of Island	LeetCode 695 (Medium)	Component *sizes*, not just a count.
129	Number of Connected Components in an Undirected Graph	LeetCode 323 (Medium)	Build from range(n) — the isolated vertex.
129	Accounts Merge	LeetCode 721 (Medium)	Components after a modelling step, with labels not counts.
130	Flood Fill	LeetCode 733 (Easy)	The neighbour function, and the case where the new colour equals the old.
130	Number of Islands	LeetCode 200 (Medium)	The double loop as the outer loop, iteratively.
130	Max Area of Island	LeetCode 695 (Medium)	Component sizes, and returning 0 on an empty grid.
130	Rotting Oranges	LeetCode 994 (Medium)	Multi-source BFS, level counting, and the -1 case.
130	Surrounded Regions	LeetCode 130 (Medium)	Flooding from the border — the inverted question.
130	Making A Large Island	LeetCode 827 (Hard)	Labelling plus the *distinct* neighbouring ids.
131	Shortest Path in Binary Matrix	LeetCode 1091 (Medium)	BFS on a grid with eight directions, and the blocked-start case.
131	Word Ladder	LeetCode 127 (Hard)	The implicit neighbour function, then bidirectional.
131	Open the Lock	LeetCode 752 (Medium)	An implicit state graph, plus a set of forbidden vertices.
131	Minimum Genetic Mutation	LeetCode 433 (Medium)	The same shape as 2, small enough to write in ten minutes.
131	Number of Ways to Arrive at Destination	LeetCode 1976 (Medium)	Counting shortest paths — and it is weighted. Check first.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
132	Graph Valid Tree	LeetCode 261 (Medium)	Connected *and* acyclic, and the $E = V - 1$ shortcut.
132	Redundant Connection	LeetCode 684 (Medium)	Union-Find, and returning *which* edge closed the cycle.
132	Number of Connected Components	LeetCode 323 (Medium)	The same Union-Find, counting instead of detecting.
132	Find Critical and Pseudo-Critical Edges	LeetCode 1489 (Hard)	Cycles inside a minimum-spanning-tree build.
132	Detect Cycles in 2D Grid	LeetCode 1559 (Medium)	The parent check on an implicit grid graph.
133	Course Schedule	LeetCode 207 (Medium)	Three colours or Kahn's, and the outer loop.
133	Course Schedule II	LeetCode 210 (Medium)	The same pass, returning the order — why Kahn's wins.
133	Find Eventual Safe States	LeetCode 802 (Medium)	Colours again, but the answer is "not in or leading to a cycle".
133	Longest Cycle in a Graph	LeetCode 2360 (Hard)	Each vertex has one outgoing edge; the path index is the trick.
133	Minimum Height Trees	LeetCode 310 (Medium)	Kahn's on an undirected graph — peeling leaves, not sources.
134	Course Schedule II	LeetCode 210 (Medium)	Kahn's, returning the order, and building from $\text{range}(n)$.
134	Alien Dictionary	LeetCode 269 (Hard)	The modelling: adjacent pairs, first difference, prefix case.
134	Minimum Height Trees	LeetCode 310 (Medium)	Kahn's on an undirected graph, peeling degree-1 leaves.
134	Sequence Reconstruction	LeetCode 444 (Medium)	Is the order unique? One extra check inside the loop.
134	Parallel Courses III	LeetCode 2050 (Hard)	Longest path in a DAG — one pass in topological order.
134	Sort Items by Groups Respecting Dependencies	LeetCode 1203 (Hard)	Two topological sorts, nested.
135	Course Schedule	LeetCode 207 (Medium)	Shape 1, and the reversal that is invisible here.
135	Course Schedule II	LeetCode 210 (Medium)	Shape 2, where the reversal finally becomes visible.
135	Parallel Courses III	LeetCode 2050 (Hard)	Shape 4 with real durations — the max recurrence.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
135	Sequence Reconstruction	LeetCode 444 (Medium)	Shape 5, uniqueness, and building edges from subsequences.
135	Alien Dictionary	LeetCode 269 (Hard)	Shape 6 — adjacent pairs, first difference, prefix case.
135	Sort Items by Groups Respecting Dependencies	LeetCode 1203 (Hard)	Two nested sorts.
136	Network Delay Time	LeetCode 743 (Medium)	Plain Dijkstra, then max of the distances, then the unreachable check.
136	Path with Minimum Effort	LeetCode 1631 (Medium)	max instead of + — the generalised combining rule.
136	Cheapest Flights Within K Stops	LeetCode 787 (Medium)	The state variant, and why plain Dijkstra is wrong.
136	Swim in Rising Water	LeetCode 778 (Hard)	Same as 2, on an implicit grid.
136	Path with Maximum Probability	LeetCode 1514 (Medium)	Multiplication and a max-heap — the third combining rule.
136	Minimum Cost to Make at Least One Valid Path	LeetCode 1368 (Hard)	Weights are only 0 and 1 — use 0-1 BFS and compare.
137	Cheapest Flights Within K Stops	LeetCode 787 (Medium)	$k + 1$ rounds — the round count <i>*is*</i> the edge limit.
137	Network Delay Time	LeetCode 743 (Medium)	Solve with both, and compare the operation counts.
137	Currency arbitrage	Not on LeetCode — build it	The $-\log(\text{rate})$ transformation and cycle detection.
137	Find the City With the Smallest Number of Neighbors	LeetCode 1334 (Medium)	Floyd-Warshall, and why $V \leq 100$ permits it.
137	Minimum Cost to Make at Least One Valid Path	LeetCode 1368 (Hard)	0-1 BFS — the algorithm the decision table picks.
138	Number of Provinces	LeetCode 547 (Medium)	The groups counter, decremented only on a real merge.
138	Redundant Connection	LeetCode 684 (Medium)	The boolean return naming the offending edge.
138	Accounts Merge	LeetCode 721 (Medium)	Keying on the shared property to avoid n^2 comparisons.
138	Number of Islands II	LeetCode 305 (Hard)	The dynamic shape — count as land is added.
138	Satisfiability of Equality Equations	LeetCode 990 (Medium)	Equality as grouping; $\text{process} == \text{before} != .$

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
138	Most Stones Removed with Same Row or Column	LeetCode 947 (Medium)	Grouping by row and column indices, not by stone.
139	Min Cost to Connect All Points	LeetCode 1584 (Medium)	An implicit complete graph — dense Prim's, and never building the edge list.
139	Connecting Cities With Minimum Cost	LeetCode 1135 (Medium)	Plain Kruskal's, plus the disconnected check.
139	Optimize Water Distribution in a Village	LeetCode 1168 (Hard)	The virtual-vertex trick.
139	Find Critical and Pseudo-Critical Edges	LeetCode 1489 (Hard)	Kruskal's run repeatedly, with an edge forced in or excluded.
140	Is Graph Bipartite?	LeetCode 785 (Medium)	The colour check, and the loop over every component.
140	Possible Bipartition	LeetCode 886 (Medium)	Building the graph in both directions from a dislike list.
140	Satisfiability of Equality Equations	LeetCode 990 (Medium)	The same idea via Union-Find, with $=$ before $!=$.
140	Maximum Number of Edges to Keep Graph Fully Traversable	LeetCode 1579 (Hard)	Union-Find again — contrast with two-colouring.
140	Flower Planting With No Adjacent	LeetCode 1042 (Medium)	Four colours, greedy — and why two-colouring does not apply.
141	01 Matrix	LeetCode 542 (Medium)	The plainest multi-source: every 0 is a source.
141	Rotting Oranges	LeetCode 994 (Medium)	Level counting, and the -1 for what is never reached.
141	Walls and Gates	LeetCode 286 (Medium)	Many sources, and why one BFS per gate times out.
141	As Far from Land as Possible	LeetCode 1162 (Medium)	Inverting "for each X find the nearest Y", then max.
141	Minimum Obstacle Removal to Reach Corner	LeetCode 2290 (Hard)	0-1 BFS with a deque.
141	Minimum Cost to Make at Least One Valid Path	LeetCode 1368 (Hard)	0-1 BFS again, with the cost on direction changes.
142	Number of Islands	LeetCode 200 (Medium)	Grid as a graph, iterative, mark on push.
142	Course Schedule II	LeetCode 210 (Medium)	Kahn's, edge direction, build from range(n).
142	Network Delay Time	LeetCode 743 (Medium)	Dijkstra, (cost, vertex), max of the distances.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
142	Accounts Merge	LeetCode 721 (Medium)	The shared-property-as-vertex move.
142	Most Stones Removed	LeetCode 947 (Medium)	Mock problem 1, timed.
142	Min Cost to Connect All Points	LeetCode 1584 (Medium)	Mock problem 2's shape — dense Prim's.

H

Index

Page numbers in **bold** are where the term is defined. Terms are those the book defines; the glossary in Appendix D gives each one a sentence.

#

4 kb, 424, 428, 432, 435, 758, 761, 764, 769, 770, ...

A

a replicated log, 845
abstract class, 845
acid, 845, 846, 862
adaptee, 845
adapter, 845
add a tie-breaker, 845
algorithm, 1, 5, 7, 16, 17, 18, 45, 47, 48, ...
always comparable, 845, 855
amortised, 598, 600, 845
anaemic model, 845
and a direction, 845
anti-diagonals, 845
anti-sniping, 845
approximate, 845
array, 18, 54, 59, 62, 151, 195, 234, 240, 241, ...
artefact, 846, 848
assignment rule, 846
at-least-once, 124, 160, 163, **200**, 203, 205, 208, 210, 215, ...
atomic, 108, 110, 112, 118, 119, 122, 125, 155, 160, ...
atomicity, 845, 846
availability, 157, 386, 846, 848, 853, 860
aws ecs, 846, 856

B

backend, 846, 862
bearer token, 391, 846
best case, 846, 862
binary tree, 846
blast radius, 66, 76, 846
block, 1, 47, 53, 71, 72, 75, 85, 95, 98, ...
blocked, 71, 117, 179, 299, 304, 381, 417, 711, 750, ...
both, 3, 4, 5, 11, 13, 14, 20, 22, 26, ...
boundary, 306, 821, 846, 850, 855, 858, 860
bounded context, 846
bounded optimism, 846
breadth-first, 1, 88, 89, 90, 96, 102, 126, 130, 268, ...
break point, 846

broken links, 395, **425**, 433, 435, 436, 441, 828, 829, 846, ...
bucket by time, 846
bucket counts, 847
bulkhead, 44, 64, 68, 71, 84, 158, 162, 834, 844, ...
burst size, 847, 857

C

cache, 24, 28, 29, 31, 32, 33, 34, 37, 41, ...
cache it, 847
cache stampede, 847
candidates, 808, 845, 847, 861, 862
capacity, 72, 73, 162, 206, 252, 715, 804, 805, 806, ...
cdn, 378, 380, 382, 385, 386, 387, 388, 389, 391, ...
certificate, 452, 847
chaining, 847
channels, 847
checks permission, 847
chunks, 567, 568, 571, 576, 580, 582, 757, 759, 760, ...
circular buffer, 847, 858
class, 3, 42, 60, 69, 80, 196, 205, 213, 251, ...
client, 25, 26, 110, 111, 162, 426, 427, 429, 521, ...
collapse key, 847
collapsing writes, 847
collection, 8, 9, 10, 50, 69, 93, 96, 100, 141, ...
column, 45, 46, 118, 122, 208, 220, 225, 226, 236, ...
compact, 249, 848, 849
comparator, 848
comparison, 38, 52, 82, 218, 232, 322, 351, 397, 460, ...
complete, 9, 52, 96, 141, 165, 187, 232, 275, 302, ...
component, 1, 10, 11, 13, 14, 40, 94, 96, 97, ...
composite, 848
composite key, 848
composition, 848, 851
composition root, 848
connection, 386, 426, 432, 452, 620, 709, 710, 711, 713, ...
connection-oriented, 848
consequence, 848, 856, 859
consistency, 157, 159, 173, 384, 399, 431, 432, 845, 846, ...
constraints, 369, 695, 848, 850
constructor, 845, 848

container, 846, 848, 849, 864
container image, 846, 848
containerd, 848, 849
context, 523, 846, 848
coordinator, 155, 159, 848, 856
correlation id, 848
cost, 5, 11, 12, 16, 17, 27, 30, 31, 33, ...
count map, 849
counter, 33, 64, 67, 71, 80, 90, 109, 119, 122, ...
cri-o, 848, 849
critical path, 376, 387, 405, 407, 408, 410, 414, 417, 419, ...
cycle, 1, 3, 5, 7, 9, 14, 16, 20, 39, ...

D

dangerous, 849
dashed, 849
dau, 134, 135, 754, 849
deep-copied, 849
deleted, 8, 114, 119, 194, 203, 204, 256, 385, 390, ...
delta of delta, 567, 849
dependency, 1, 26, 68, 69, 158, 361, 367, 414, 448, ...
depth, 1, 14, 18, 47, 100, 103, 104, 132, 133, ...
depth-first, 1, 47, 104, 132, 133, 134, 141, 146, 171, ...
deque, 9, 10, 69, 93, 94, 96, 98, 100, 101, ...
diminishing returns, 477, 849
direct addressing, 849
domain name, 849
draw two diagrams, 849
duck typing, 849
dummy node, 849

E

earlier, 55, 60, 84, 249, 270, 281, 333, 339, 342, ...
edge cases, 105, 130, 241, 285, 459, 563, 749, 848, 850
edge compute, 850
effect, 23, 74, 78, 124, 160, 162, 189, 190, 194, ...
elastic scaling, 850
election timeout, 161, 850
empty, 8, 12, 16, 27, 46, 47, 107, 108, 133, ...
etl, 91, 168, 199, 274, 395, 530, 584, 612, 637, ...
event object, 850
eventual consistency, 157, 159, 850
exactly, 4, 13, 16, 21, 23, 28, 30, 32, 33, ...
exactly correct, 850
exactly one, 23, 28, 32, 37, 42, 43, 75, 105, 114, ...
exclusive lock, 850
expiry, 28, 30, 34, 109, 111, 112, 113, 114, 118, ...

F

facts, 3, 4, 5, 6, 9, 16, 303, 403, 497, ...
false negative, 850

false positive, 850
fanout, 293, 850
fee, 4, 66, 72, 149, 159, 160, 165, 198, 208, ...
fifo, 205, 207, 208, 212, 216, 220, 222, 296, 307, ...
file, 14, 36, 38, 56, 82, 100, 101, 144, 162, ...
filled diamond, 851
finite state machine, 851
float, 27, 51, 52, 69, 404, 502, 503, 504, 505, ...
four-direction move, 851
fragile, 202, 573, 851
frontend, 851

G

galloping mode, 851
gap, 4, 16, 48, 63, 155, 163, 169, 190, 256, ...
generator expression, 851
greedy simulation, 851
guest operating system, 851, 852

H

hash map, 851, 852
hash ring, 851
hateoas, 851
header interface, 851
heap, 5, 15, 35, 51, 61, 71, 72, 82, 91, ...
heapsort, 851
heartbeat, 712, 851
hidden spof, 851
hiding mechanism, 851
highest node, 851
historical snapshot, 852
hollow triangle, 852
hook, 45, 46, 47, 852
horizontal scaling, 852
hypervisor, 851, 852

I

image, 49, 201, 214, 215, 216, 219, 240, 378, 382, ...
implicit graph, 852, 868
in parallel, 383, 667, 852
in series, 150, 206, 219, 852
in-degree, 5, 38, 39, 357, 359, 360, 362, 365, 368, ...
index, 14, 40, 119, 122, 198, 201, 206, 210, 215, ...
indexed heap, 499, 507, 852
inheritance, 852
inorder, 852, 858
input, 5, 9, 13, 14, 16, 19, 38, 45, 47, ...
integrity, 666, 852
intent plus multiplicity, 852
interface, 849, 851, 852, 858
invariant, 735, 852
is possible, 852, 859

item, 118, 134, 257, 293, 301, 378, 387, 394, 429, ...
iterator, 364, 366, 852

J

json, 204, 250, 259, 523, 711, 712, 853
jump, 853, 854
jwt, 757, 853

K

k closest points, 853
k most frequent, 853
k-th largest specifically, 853
key, 13, 28, 29, 31, 33, 34, 36, 38, 41, ...
key property, 853

L

lag, 37, 62, 104, 108, 139, 149, 150, 167, 172, ...
largest, 180, 185, 192, 196, 218, 238, 240, 241, 304, ...
latency, 27, 30, 34, 36, 41, 64, 73, 76, 81, ...
lazy, 497, 499, 500, 502, 507, 509, 511, 517, 537, ...
lazy deletion, 507, 785, 853
lazy loading, 853
lazy refill, 853
leader, 107, 114, 116, 117, 155, 157, 159, 163, 246, ...
leaf, 643, 781, 793, 794, 804, 805, 806, 809, 813, ...
lease, 34, 36, 71, 89, 107, 109, 110, 111, 112, ...
ledger entries, 853
left, 9, 10, 28, 31, 32, 45, 57, 65, 68, ...
lfu, 853
linear scan, 853
linearizability, 854
list, 1, 3, 7, 8, 9, 10, 12, 13, 14, ...
list of lists, 46, 50, 54, 60, 234, 240, 413, 742, 854
load balancer, 710, 854
load factor, 854
load shedding, 854
log-structured merge tree, 854
logical cohesion, 854
logical time, 854
loop, 7, 9, 10, 11, 12, 13, 14, 15, 16, ...
loop body, 194, 854
low cohesion, 854
lru, 845, 854

M

manifest, 854
many times, 31, 61, 62, 215, 277, 347, 378, 593, 650, ...
many-to-many, 854, 861
matrix, 1, 44, 45, 48, 49, 50, 51, 52, 53, ...
mau, 854
max-heap, 536, 854, 855, 871
method resolution order, 854

milliseconds, 574, 620, 694, 848, 854
min-heap, 405, 409, 418, 854, 855
minimum spanning forest, 855
mixin, 855
monotone, 855
monotonic deque, 855

N

name mangling, 855
namespace, 379, 395, 828, 855
negative, 1, 507, 537, 542, 543, 548, 549, 550, 551, ...
negative infinity, 855
negatives, 507, 554, 850, 855
nested loop, 855
network partition, 855
never equal, 845, 855
never forgets, 855
never persisted, 855
no children, 853, 855
node, 4, 94, 96, 114, 116, 117, 121, 125, 129, ...
not a map, 855

O

o(1), 50, 52, 54, 55, 58, 63, 83, 93, 189, ...
object, 12, 18, 51, 54, 55, 56, 59, 60, 62, ...
observers, 855
occupied, 849, 850, 856
offset, 228, 232, 246, 248, 264, 293, 759, 761, 766, ...
on first touch, 856
one-shot, 189, 321, 327, 563, 856
opens, 35, 36, 64, 66, 67, 70, 74, 76, 81, ...
orchestrator, 663, 846, 856
origin, 387, 430, 479, 850, 856, 864
origin pull, 856
orphan rows, 856
orphans, 425, 432, 433, 435, 436, 441, 829, 846, 856
out-degree, 5, 38, 39, 368, 452, 852, 856, 868
overloading, 856
overriding, 856

P

page, 4, 11, 67, 72, 78, 79, 80, 168, 200, ...
parallel edges, 323, 856
parameters, 856
parent, 94, 96, 98, 106, 140, 145, 273, 274, 313, ...
partial failure, 856
participants, 848, 856, 858
partition key, 846, 856
partitions, 204, 207, 246, 247, 251, 252, 293, 620, 668, ...
path, 1, 3, 5, 6, 7, 10, 16, 38, 39, ...
port, 8, 9, 10, 13, 14, 23, 26, 27, 28, ...

prefix sums, 855, 857
preorder, 857
priority queue, 857
procedure, 857
process, 202, 203, 207, 215, 220, 248, 294, 310, 332, ...
program, 848, 857, 860
protecting state, 857

Q

qps, 84, 85, 857
queue, 9, 10, 12, 36, 60, 64, 66, 71, 90, ...
quorum, 116, 117, 121, 153, 155, 157, 159, 161, 169, ...

R

race condition, 857
ram, 5, 7, 9, 82, 103, 134, 135, 143, 147, ...
rank, 194, 321, 471, 473, 474, 476, 477, 478, 593, ...
rebalancing, 249, 857
rebuilding the tree, 857
recursion, 14, 100, 141, 142, 143, 144, 145, 147, 152, ...
references, 429, 846, 856, 857, 864
refill rate, 847, 857
register, 288, 719, 755, 756, 757, 759, 764, 765, 767, ...
relative epsilon, 858
removal, 149, 195, 328, 359, 370, 492, 594, 603, 607, ...
replica, 159, 202, 246, 248, 250, 251, 252, 258, 521, ...
replica divergence, 858
request-aware routing, 858
resolver, 858
resources, 45, 78, 431, 544, 858, 862
returned, 97, 137, 276, 280, 357, 359, 490, 505, 574, ...
ring buffer, 847, 858
role interface, 858
root, 137, 148, 195, 285, 316, 326, 351, 359, 373, ...
rotation, 858
routine, 155, 420, 466, 476, 526, 530, 565, 655, 668, ...
row, 5, 7, 8, 9, 11, 17, 18, 23, 24, ...
run, 4, 5, 9, 11, 13, 22, 27, 35, 45, ...

S

same remainder, 858
schema, 429, 523, 528, 612, 615, 616, 618, 619, 628, ...
scoping, 858
script, 110, 112, 119, 124, 125, 131, 160, 166, 204, ...
seam, 858
search space, 858
selectivity, 858
sentinel, 161, 584, 849, 858
separator, 858
sequence diagram, 858, 866
serialisable, 859
service credits, 859

session, 161, 335, 376, 853, 859, 860, 861
session guarantees, 859
set, 1, 3, 7, 10, 12, 14, 16, 19, 20, ...
shallow-copied, 859
shape, 7, 17, 21, 51, 70, 93, 95, 124, 127, ...
shard, 18, 480, 481, 483, 486, 487, 518, 520, 521, ...
shared lock, 859
shared store, 119, 859
singleton, 703, 859
singly linked list, 859
slice, 59, 62, 290, 328, 360, 364, 374, 398, 664, ...
smallest, 322, 409, 410, 411, 412, 413, 453, 457, 458, ...
snapshot, 852, 859
snipe, 859
sorted region, 859
sorting everything, 859
splits, 566, 576, 685, 693, 817, 859
spof, 851, 859
squares, 46, 105, 859
stable, 34, 124, 202, 619, 669, 674, 676, 831, 859
stack, 23, 42, 71, 75, 133, 134, 135, 138, 139, ...
stack frame, 134, 860
stack overflow, 860
state, 3, 6, 7, 8, 11, 14, 17, 20, 36, ...
statements, 857, 860, 867
status, 40, 122, 200, 208, 210, 215, 220, 566, 569, ...
still one machine, 853, 860
stonith, 118, 860
strategy, 338, 345, 845, 848, 860
strictly shorter, 860
structural, 646, 857, 860
subarray, 860
subject, 860, 865
subscription list, 860
substring, 860
subtree, 852, 857, 860
surge multiplier especially, 860
swap, 62, 95, 114, 115, 215, 229, 274, 408, 501, ...
system call, 860

T

table, 34, 39, 53, 64, 65, 122, 124, 165, 202, ...
tail call, 860
tail-recursive, 860
tcp, 713, 720, 722, 848, 860, 862
template, 782, 783, 784, 785, 819, 852, 860
test double, 861
the return address, 861
the running median, 851, 861
third table, 685, 854, 861
thread, 35, 36, 64, 65, 66, 70, 71, 72, 75, ...
three distinct orderings, 861

three-way partition, 861
tight coupling, 861
time-to-live, 861
tiny constraint, 861
toast, 426, 430, 432, 433, 438, 441, 445, 447, 829, ...
token, 27, 107, 109, 111, 112, 113, 114, 117, 118, ...
tombstone, 430, 861
top-k problems, 851, 861
total order, 245, 252, 254, 861
transitions itself, 861
transitive dependency, 861
tree, 1, 3, 5, 14, 20, 43, 47, 57, 61, ...
ttl, 27, 28, 29, 34, 37, 42, 46, 47, 62, ...
tuple, 8, 10, 50, 51, 52, 53, 62, 96, 139, ...
two, 1, 3, 4, 5, 6, 7, 8, 11, 12, ...
two implementations, 71, 861

U

udp, 860, 862
under-fetching, 862
union filesystem, 862
upper bound, 593, 857, 862
uri, 1, 74, 206, 288, 507, 560, 620, 683, 684, ...
url, 31, 353, 378, 380, 382, 385, 389, 390, 391, ...

V

value object, 862
version, 82, 111, 114, 126, 137, 171, 172, 187, 191, ...
vertex, 10, 12, 13, 14, 49, 50, 51, 55, 57, ...
vertical scaling, 862
victim, 550, 862
virtual address space, 862
virtual machine, 846, 856, 862
volatile, 862

W

weight, 1, 5, 6, 7, 16, 17, 19, 20, 38, ...
which class, 862
which is hours, 862
within, 21, 22, 29, 33, 74, 89, 101, 103, 269, ...
worst case, 37, 74, 143, 234, 277, 321, 352, 368, 398, ...
write skew, 862
write-around with invalidation, 862
write-through, 863

Y

yes, 6, 50, 57, 64, 68, 151, 157, 158, 183, ...

Two subjects, every day, for one hundred and eighty days.

Most preparation teaches algorithms for three months, then starts again from nothing on system design. Interviews do not work that way. A single loop will ask you to reverse a linked list in one room and design a rate limiter in the next.

So this book teaches both, every day, for one hundred and eighty days. Written for someone starting from zero - not a graduate, not somebody brushing up - and finishing able to answer either kind of question out loud, to a stranger, without notes.

- ◆ One hundred and eighty chapters. Two lessons and a practice sheet in every one.
- ◆ The same nine sections every time, so you learn one reading rhythm and keep it.
- ◆ Every lesson opens with a story about a person, told without a single technical word.
- ◆ Every solution shown in full, built in fragments and then given complete.
- ◆ The arithmetic done out loud, and error messages printed exactly as they appear.
- ◆ Real interviewer phrasings, an opening script, the follow-ups, and a model answer.